

Deep Learning

Neural Networks, CNN, Sequence Models, NLP, Transformer

Hanwha Life Vietnam

Phiên bản v1.0.0

Ngày 11 tháng 10 năm 2026

Mục lục

1 Tổng quan **2**
1.1 Cách đọc tài liệu . . . 2
1.2 Nội dung . . . 2

2 Mạng nơ-ron và Học sâu **4**
2.1 Giới thiệu về Deep Learning . . . 5
2.1.1 Mạng nơ-ron là gì . . . 6
2.1.2 Học có giám sát với mạng nơ-ron . . . 9
2.1.3 Vì sao học sâu bùng nổ gần đây . . . 10
2.1.4 Bối cảnh lịch sử và lời khuyên cho người mới . . . 13
2.1.5 Áp dụng vào dự án thực tế . . . 16
2.1.6 Tóm tắt cần nhớ . . . 17
2.2 Lập trình mạng nơ-ron: hồi quy logistic và vector hoá . . . 17
2.2.1 Phân loại nhị phân và cách biểu diễn dữ liệu . . . 17
2.2.2 Hồi quy logistic . . . 19
2.2.3 Hàm mất mát và hàm chi phí . . . 20
2.2.4 Đạo hàm là độ dốc . . . 21
2.2.5 Gradient descent . . . 23
2.2.6 Đồ thị tính toán và lan truyền ngược . . . 24
2.2.7 Gradient descent cho hồi quy logistic . . . 27
2.2.8 Vector hoá . . . 28
2.2.9 Broadcasting trong NumPy . . . 32
2.2.10 Tránh lỗi về chiều của mảng . . . 33
2.2.11 Đọc thêm: vì sao dùng cross-entropy . . . 34
2.2.12 Áp dụng vào dự án thực tế . . . 35
2.2.13 Tóm tắt cần nhớ . . . 37
2.3 Thực hành: các hàm NumPy cơ bản và nhận dạng ảnh có mèo . . . 37
2.3.1 Thực hành 1: các hàm NumPy cơ bản cho học sâu . . . 37
2.3.2 Thực hành 2: nhận dạng ảnh có mèo bằng hồi quy logistic . . . 40
2.4 Mạng nơ-ron một lớp ẩn . . . 46
2.4.1 Từ hồi quy logistic đến mạng nơ-ron . . . 46
2.4.2 Cấu trúc của mạng . . . 47
2.4.3 Mạng tính đầu ra như thế nào . . . 49
2.4.4 Hàm kích hoạt . . . 52
2.4.5 Huấn luyện: gradient descent và lan truyền ngược . . . 55
2.4.6 Khởi tạo tham số . . . 57
2.4.7 Áp dụng vào dự án thực tế . . . 58
2.4.8 Tóm tắt cần nhớ . . . 60
2.5 Thực hành: phân loại dữ liệu hình bông hoa . . . 61
2.5.1 Dữ liệu và đường cơ sở . . . 61
2.5.2 Cách nhanh: Keras và scikit-learn . . . 62

2.5.3 Mở hộp: tự viết bằng NumPy . . . 63
2.5.4 Các thí nghiệm . . . 65
2.5.5 Rút ra . . . 68
2.6 Mạng nơ-ron sâu . . . 69
2.6.1 Nông và sâu . . . 70
2.6.2 Tên gọi và hình dạng trong mạng nhiều lớp . . . 70
2.6.3 Lan truyền xuôi qua nhiều lớp . . . 71
2.6.4 Vì sao mạng sâu hiệu quả . . . 73
2.6.5 Lan truyền ngược bằng các khối xây dựng . . . 75
2.6.6 Tham số và siêu tham số . . . 77
2.6.7 Khởi tạo cho mạng sâu . . . 78
2.6.8 Về phép so sánh với bộ não . . . 78
2.6.9 Áp dụng vào dự án thực tế . . . 78
2.7 Thực hành: mạng sâu nhận dạng ảnh có mèo . . . 80
2.7.1 Nạp dữ liệu . . . 80
2.7.2 Mạng bốn lớp bằng Keras . . . 80
2.7.3 Mở hộp: các hàm trợ giúp bằng NumPy . . . 81
2.7.4 So sánh hồi quy logistic, mạng hai lớp và mạng bốn lớp . . . 84
2.7.5 Phân tích ảnh bị đoán sai . . . 86
2.7.6 Áp dụng vào dự án thực tế . . . 87
2.7.7 Tóm tắt chương . . . 88

3 Cải thiện mạng nơ-ron sâu 89
3.1 Thiết lập ứng dụng học máy . . . 91
3.1.1 Tập huấn luyện, tập kiểm định và tập kiểm tra . . . 92
3.1.2 Thiên lệch và phương sai . . . 94
3.1.3 Công thức cơ bản để cải thiện mô hình . . . 96
3.1.4 Áp dụng vào dự án thực tế . . . 97
3.2 Chính quy hoá . . . 98
3.2.1 Chính quy hoá L2 . . . 99
3.2.2 Dropout . . . 101
3.2.3 Các kỹ thuật khác: tăng cường dữ liệu và dừng sớm . . . 103
3.2.4 Chọn kỹ thuật nào . . . 106
3.2.5 Áp dụng vào dự án thực tế . . . 107
3.3 Huấn luyện ổn định: chuẩn hoá đầu vào, khởi tạo và kiểm tra độ dốc . . . 108
3.3.1 Chuẩn hoá đầu vào . . . 108
3.3.2 Bùng nổ và tiêu biến gradient . . . 109
3.3.3 Khởi tạo trọng số cho mạng sâu . . . 110
3.3.4 Kiểm tra độ dốc . . . 112
3.3.5 Áp dụng vào dự án thực tế . . . 115
3.4 Thực hành: khởi tạo, chính quy hoá và kiểm tra độ dốc . . . 116
3.4.1 Thực hành 1: ảnh hưởng của cách khởi tạo . . . 117
3.4.2 Thực hành 2: chính quy hoá L2 và dropout . . . 119
3.4.3 Thực hành 3: kiểm tra độ dốc . . . 121
3.4.4 Áp dụng vào dự án thực tế . . . 122
3.5 Thuật toán tối ưu . . . 123
3.5.1 Mini-batch gradient descent . . . 123
3.5.2 Trung bình mũ (exponentially weighted average) . . . 127
3.5.3 Momentum . . . 130
3.5.4 RMSprop . . . 131
3.5.5 Adam . . . 132
3.5.6 Giảm tốc độ học . . . 134

3.5.7 Cực trị cục bộ và đồng bằng . . . 135
3.5.8 Chọn bộ tối ưu . . . 136
3.5.9 Áp dụng vào dự án thực tế . . . 137
3.6 Thực hành: so sánh các bộ tối ưu . . . 138
3.6.1 Thiết lập . . . 138
3.6.2 Thí nghiệm 1: gradient descent, momentum và Adam . . . 140
3.6.3 Thí nghiệm 2: giảm tốc độ học . . . 140
3.6.4 Áp dụng vào dự án thực tế . . . 142
3.7 Tinh chỉnh siêu tham số . . . 143
3.7.1 Siêu tham số nào quan trọng . . . 143
3.7.2 Chọn giá trị để thử . . . 144
3.7.3 Chọn thang đo thích hợp . . . 145
3.7.4 Cách vận hành việc tìm kiếm . . . 146
3.7.5 Áp dụng vào dự án thực tế . . . 147
3.8 Batch Normalization . . . 148
3.8.1 Chuẩn hoá các kích hoạt bên trong mạng . . . 148
3.8.2 Gắn Batch Norm vào mạng . . . 149
3.8.3 Vì sao Batch Norm hoạt động . . . 150
3.8.4 Batch Norm lúc dự đoán . . . 151
3.8.5 Áp dụng vào dự án thực tế . . . 152
3.9 Phân loại nhiều lớp với softmax . . . 153
3.9.1 Lớp softmax . . . 154
3.9.2 Huấn luyện mạng có lớp softmax . . . 155
3.9.3 Softmax trong Keras . . . 157
3.9.4 Áp dụng vào dự án thực tế . . . 158
3.10 Framework học sâu và TensorFlow . . . 159
3.10.1 Chọn framework . . . 159
3.10.2 Cấu trúc một chương trình TensorFlow . . . 159
3.10.3 Đối chiếu NumPy và TensorFlow . . . 161
3.10.4 Áp dụng vào dự án thực tế . . . 162
3.11 Thực hành: mạng ba lớp bằng TensorFlow . . . 163
3.11.1 Bài toán và dữ liệu . . . 163
3.11.2 Làm quen với TensorFlow qua các hàm nhỏ . . . 165
3.11.3 Xây mạng ba lớp . . . 166
3.11.4 Huấn luyện mô hình . . . 167
3.11.5 Cùng bài toán bằng Keras . . . 169
3.11.6 Áp dụng vào dự án thực tế . . . 169
3.11.7 Tóm tắt chương . . . 171

4 Tổ chức dự án học máy 172
4.1 Trực giao hoá: mỗi vấn đề một núm chỉnh riêng . . . 174
4.1.1 Ví dụ cái tivi và chiếc xe . . . 174
4.1.2 Bốn điều kiện cần đạt và bốn nhóm núm chỉnh . . . 174
4.1.3 Vì sao tác giả ít dùng dừng sớm . . . 175
4.1.4 Áp dụng vào dự án thực tế . . . 176
4.2 Đặt mục tiêu bằng một chỉ số đánh giá . . . 176
4.2.1 Một con số thay vì nhiều con số . . . 176
4.2.2 Chỉ số tối ưu và chỉ số thoả mãn . . . 177
4.2.3 Áp dụng vào dự án thực tế . . . 178
4.3 Chia tập kiểm định và tập kiểm tra . . . 178
4.3.1 Dev và test phải cùng phân phối . . . 178
4.3.2 Kích thước tập kiểm định và kiểm tra . . . 179

4.3.3 Khi nào nên đổi bia (chỉ số hoặc tập dữ liệu) . . . 180
4.3.4 Áp dụng vào dự án thực tế . . . 180
4.4 So sánh với mức con người . . . 181
4.4.1 Sai số Bayes và vì sao tiến bộ chậm lại . . . 181
4.4.2 Mức con người là ước lượng tốt cho sai số Bayes . . . 181
4.4.3 Định nghĩa mức con người cho đúng mục đích . . . 182
4.4.4 Khi đã vượt mức con người . . . 183
4.4.5 Gom lại thành quy trình cải thiện . . . 183
4.5 Phân tích lỗi . . . 184
4.5.1 Đếm lỗi để ước lượng trần cải thiện . . . 184
4.5.2 Đánh giá nhiều ý tưởng song song . . . 184
4.5.3 Dữ liệu bị gán nhãn sai . . . 185
4.5.4 Dựng nhanh rồi lặp . . . 185
4.5.5 Áp dụng vào dự án thực tế . . . 186
4.6 Khi dữ liệu huấn luyện lệch phân phối với dev/test . . . 186
4.6.1 Chia dữ liệu khi hai nguồn khác nhau . . . 186
4.6.2 Tập train-dev để chẩn đoán đúng . . . 186
4.6.3 Xử lý lệch phân phối . . . 188
4.6.4 Áp dụng vào dự án thực tế . . . 188
4.7 Chuyển giao, đa nhiệm và học đầu-cuối . . . 189
4.7.1 Học chuyển giao . . . 189
4.7.2 Học đa nhiệm . . . 190
4.7.3 Học đầu-cuối . . . 191
4.7.4 Áp dụng vào dự án thực tế . . . 194

5 Convolutional Neural Networks 195
5.1 Nền tảng của mạng tích chập . . . 197
5.1.1 Vì sao cần phép tích chập . . . 197
5.1.2 Phép tích chập và phát hiện cạnh . . . 198
5.1.3 Đệm viền và bước nhảy . . . 199
5.1.4 Tích chập trên khối nhiều kênh . . . 199
5.1.5 Một lớp tích chập . . . 200
5.1.6 Lớp gộp . . . 200
5.1.7 Một mạng CNN đầy đủ . . . 201
5.1.8 Vì sao tích chập hiệu quả . . . 201
5.1.9 Thực hành: tự cài đặt khối CNN bằng NumPy . . . 202
5.1.10 Thực hành: xây CNN với Keras . . . 203
5.2 Các kiến trúc kinh điển và học chuyển giao . . . 204
5.2.1 Ba mạng kinh điển: LeNet, AlexNet, VGG . . . 204
5.2.2 Mạng phần dư (ResNet) . . . 205
5.2.3 Tích chập 1x1 và mạng Inception . . . 207
5.2.4 Mạng nhẹ: MobileNet và EfficientNet . . . 208
5.2.5 Dùng mã nguồn mở . . . 209
5.2.6 Học chuyển giao . . . 209
5.2.7 Tăng cường dữ liệu . . . 210
5.2.8 Trạng thái của thị giác máy tính . . . 211
5.3 Phát hiện và phân vùng vật thể . . . 212
5.3.1 Định vị và điểm mốc . . . 212
5.3.2 Cửa sổ trượt và cài đặt bằng tích chập . . . 213
5.3.3 YOLO: nhìn một lần . . . 213
5.3.4 Đánh giá và dọn kết quả: IoU, triệt phi cực đại . . . 214
5.3.5 Hộp neo . . . 216

5.3.6 Đề xuất vùng (R-CNN) . . . 216
5.3.7 Phân vùng ngữ nghĩa với U-Net . . . 216
5.4 Nhận diện khuôn mặt và chuyển phong cách . . . 219
5.4.1 Xác minh, nhận diện và học một lần . . . 219
5.4.2 Mạng Siamese . . . 220
5.4.3 Hàm mất mát bộ ba . . . 220
5.4.4 CNN sâu học được gì . . . 222
5.4.5 Chuyển phong cách . . . 222
5.4.6 Tích chập trên dữ liệu một chiều và ba chiều . . . 223

6 Sequence Models 225
6.1 Mạng nơ-ron hồi quy (RNN) . . . 227
6.1.1 Vì sao cần mô hình chuỗi . . . 227
6.1.2 RNN cơ bản . . . 227
6.1.3 Các kiểu kiến trúc RNN . . . 230
6.1.4 Huấn luyện RNN: lan truyền ngược theo thời gian . . . 232
6.1.5 Tiêu biến và bùng nổ gradient . . . 233
6.1.6 GRU – Gated Recurrent Unit . . . 234
6.1.7 LSTM – Long Short-Term Memory . . . 234
6.1.8 RNN hai chiều và RNN sâu . . . 236
6.1.9 Mô hình ngôn ngữ và sinh chuỗi . . . 237
6.1.10 Thực hành 1: Đặt tên khủng long . . . 240
6.1.11 Thực hành 2: Ứng tác jazz với LSTM . . . 243
6.1.12 Áp dụng vào dự án thực tế . . . 245
6.1.13 Tóm tắt cần nhớ . . . 248
6.2 Xử lý ngôn ngữ tự nhiên và Word Embedding . . . 248
6.2.1 Vì sao one-hot chưa đủ . . . 248
6.2.2 Embedding là gì và dùng thế nào . . . 250
6.2.3 Chuyển giao kiến thức bằng embedding . . . 252
6.2.4 Loại suy và độ tương đồng cosine . . . 253
6.2.5 Các thuật toán học embedding . . . 255
6.2.6 Phân loại cảm xúc bằng embedding . . . 262
6.2.7 Thiên kiến trong embedding . . . 264
6.2.8 Thực hành 1: Phép toán trên vector từ . . . 266
6.2.9 Thực hành 2: Emojify – chọn emoji cho câu . . . 267
6.2.10 Áp dụng vào dự án thực tế . . . 270
6.2.11 Tóm tắt cần nhớ . . . 272
6.3 Mô hình chuỗi sang chuỗi, cơ chế chú ý và xử lý giọng nói . . . 273
6.3.1 Mô hình encoder–decoder . . . 273
6.3.2 Dịch máy là mô hình ngôn ngữ có điều kiện . . . 275
6.3.3 Beam search . . . 277
6.3.4 Phân tích lỗi: beam search hay mô hình? . . . 281
6.3.5 Đánh giá bản dịch bằng điểm BLEU . . . 283
6.3.6 Cơ chế chú ý (attention) . . . 285
6.3.7 Nhận dạng giọng nói . . . 288
6.3.8 Phát hiện từ khoá đánh thức . . . 291
6.3.9 Thực hành 1: Chuẩn hoá ngày tháng bằng mô hình chú ý . . . 295
6.3.10 Thực hành 2: Phát hiện từ khoá “activate” . . . 298
6.3.11 Áp dụng vào dự án thực tế . . . 299
6.3.12 Tóm tắt cần nhớ . . . 300
6.4 Mạng Transformer . . . 301
6.4.1 Vì sao cần Transformer . . . 301

6.4.2 Self-attention . . . 302
6.4.3 Multi-head attention . . . 305
6.4.4 Ghép thành kiến trúc Transformer . . . 306
6.4.5 Mã hoá vị trí . . . 309
6.4.6 Mặt nạ (mask) . . . 310
6.4.7 Dựng Transformer bằng Keras . . . 311
6.4.8 Dùng mô hình Transformer huấn luyện sẵn . . . 316
6.4.9 Thực hành 1: Dựng Transformer từ các khối con . . . 318
6.4.10 Thực hành 2: Embedding cộng mã hoá vị trí . . . 319
6.4.11 Thực hành 3: Nhận dạng thực thể trên hồ sơ xin việc . . . 321
6.4.12 Áp dụng vào dự án thực tế . . . 325
6.4.13 Tóm tắt cần nhớ . . . 326

Chương 1

Tổng quan

Tài liệu này là bộ ghi chú học tập về học sâu (Deep Learning). Mỗi chương trình bày một chủ đề theo hướng **ý tưởng trước, ứng dụng sau**: hiểu vì sao một kỹ thuật ra đời, nó giải quyết vấn đề gì, rồi dùng nó qua thư viện như thế nào.

1.1 Cách đọc tài liệu

- **Lý thuyết** – giải thích bằng ví dụ và sơ đồ, hạn chế công thức.
- **Code** – ví dụ dùng TensorFlow/Keras, chạy được với Python 3.10+ và TensorFlow 2.x. Code đặt trong khung, có chú thích tiếng Việt.
- **Áp dụng** – checklist, bảng chọn kiến trúc, lỗi thường gặp khi đưa vào dự án.

1.2 Nội dung

Tài liệu gồm sáu chương, đi từ nền tảng của mạng nơ-ron đến các mô hình chuỗi. Chương 6 **Sequence Models** đã hoàn chỉnh; chương 2 **Mạng nơ-ron và Học sâu** đã có năm mục đầu (từ neuron đơn lẻ đến mạng một lớp ẩn); các chương còn lại đang được biên soạn dần. Chương Sequence Models đi từ mạng nơ-ron hồi quy đến Transformer qua bốn mục, mỗi mục xây tiếp trên mục trước (Hình 6.1).

Chương / mục	**Nội dung chính**
Chương 1: Tổng quan	mục đích, cách đọc và cấu trúc tài liệu (chương này).
Chương 2: Mạng nơ-ron và Học sâu	nền tảng của mạng nơ-ron và học sâu (gồm bảy mục, từ neuron đến mạng nhiều lớp).
Chương 3: Cải thiện mạng nơ-ron sâu	điều chỉnh, tối ưu và kiểm soát quá khớp: chia dữ liệu, chính quy hoá, thuật toán tối ưu, Batch Norm, softmax và framework.
Chương 4: Tổ chức dự án học máy	chiến lược định hướng một dự án học máy: trực giao hoá, chỉ số đánh giá, chia dev/test, so với mức con người, phân tích lỗi, dữ liệu lệch phân phối, chuyển giao và đa nhiệm.
Chương 5: Convolutional Neural Networks	mạng tích chập cho dữ liệu hình ảnh (đang biên soạn).
Chương 6: Sequence Models	Mô hình cho dữ liệu dạng chuỗi (văn bản, âm thanh, nhạc), từ RNN đến Transformer.

Bảng 1.1: Cấu trúc tài liệu

Bảng 1.2 liệt kê chi tiết các mục đã biên soạn.

Bảng 1.2: Các mục đã biên soạn

Mục	**Nội dung chính**
Mục 2.1: Giới thiệu về Deep Learning	neuron đơn lẻ và ReLU, hidden unit và mạng nhiều lớp, học có giám sát, dữ liệu có cấu trúc và không có cấu trúc, chọn kiến trúc theo dữ liệu, quy mô và ba động lực của học sâu (dữ liệu, tính toán, thuật toán), bối cảnh lịch sử, một ví dụ thực hành (mạng giá nhà bằng Keras).
Mục 2.2: Lập trình mạng nơ-ron: hồi quy logistic và vector hoá	biểu diễn dữ liệu và quy ước mỗi mẫu một cột, hồi quy logistic, hàm mất mát và chi phí cross-entropy, đạo hàm là độ dốc, gradient descent và tốc độ học, đồ thị tính toán và lan truyền ngược, vector hoá, broadcasting, tránh mảng hạng 1, đọc thêm về ước lượng hợp lý cực đại.
Mục 2.3: Thực hành: các hàm NumPy cơ bản và nhận dạng ảnh có mèo	các hàm NumPy thường dùng (sigmoid, softmax, chuẩn hoá hàng, L1, L2), bộ phân loại ảnh có mèo bằng hồi quy logistic viết bằng NumPy và làm lại bằng Keras, chọn tốc độ học, quá khớp.
Mục 2.4: Mạng nơ-ron một lớp ẩn	từ hồi quy logistic đến mạng nơ-ron, lớp vào, lớp ẩn, lớp ra, tính đầu ra cho một mẫu và nhiều mẫu, hàm kích hoạt (sigmoid, tanh, ReLU) và lý do cần phi tuyến, lan truyền ngược qua hai lớp, khởi tạo ngẫu nhiên và vấn đề đối xứng, chọn số nút ẩn.
Mục 2.5: Thực hành: phân loại dữ liệu hình bông hoa	dữ liệu không tách được bằng đường thẳng, Keras và scikit-learn, mạng viết bằng NumPy và kiểm tra độ dốc, thí nghiệm số nút ẩn, hàm kích hoạt, khởi tạo và bốn tập dữ liệu khác.
Mục 2.6: Mạng nơ-ron sâu	nông và sâu, tên gọi và hình dạng trong mạng nhiều lớp, lan truyền xuôi và ngược theo từng khối, vì sao sâu hiệu quả, tham số và siêu tham số, khởi tạo cho mạng sâu.
Mục 2.7: Thực hành: mạng sâu nhận dạng ảnh có mèo	Keras, mở hộp bằng NumPy, so sánh hồi quy logistic, mạng hai lớp và mạng bốn lớp, dừng sớm, khởi tạo, phân tích ảnh sai, tóm tắt chương.
Mục 6.1: Mạng nơ-ron hồi quy (RNN)	RNN cơ bản, các kiểu kiến trúc, BPTT, tiêu biến gradient, GRU, LSTM, RNN hai chiều và RNN sâu, mô hình ngôn ngữ, sinh chuỗi, hai ví dụ thực hành (đặt tên khủng long, ứng tác jazz).
Mục 6.2: Xử lý ngôn ngữ tự nhiên và Word Embedding	one-hot và embedding, chuyển giao kiến thức, loại suy và độ tương đồng cosine, Word2Vec (Skip-Gram, negative sampling) và GloVe, phân loại cảm xúc, thiên kiến trong embedding, hai ví dụ thực hành (phép toán trên vector từ, Emojify).
Mục 6.3: Mô hình chuỗi sang chuỗi, cơ chế chú ý và xử lý giọng nói	encoder–decoder, mô hình ngôn ngữ có điều kiện, beam search và phân tích lỗi, điểm BLEU, cơ chế chú ý, nhận dạng giọng nói (CTC), phát hiện từ khoá đánh thức, hai ví dụ thực hành (chuẩn hoá ngày tháng bằng mô hình chú ý, phát hiện từ khoá “activate”).
Mục 6.4: Mạng Transformer	vì sao cần, self-attention và multi-head attention, kiến trúc encoder–decoder, mã hoá vị trí và mặt nạ, dựng bằng Keras, dùng mô hình huấn luyện sẵn qua Hugging Face, ba ví dụ thực hành (dựng Transformer, embedding cộng mã hoá vị trí, nhận dạng thực thể trên hồ sơ xin việc).

Chương 2

Mạng nơ-ron và Học sâu

Deep Learning (học sâu) là việc huấn luyện các mạng nơ-ron (neural network), có khi rất lớn. Mọi chương sau của tài liệu đều đứng trên nền tảng này: ảnh, chuỗi, ngôn ngữ đều được xử lý bằng những biến thể khác nhau của cùng một ý tưởng cơ bản.

Chương này đặt nền móng theo từng mục. Mỗi mục trả lời một câu hỏi, và câu trả lời của mục trước dẫn đến câu hỏi của mục sau.

Mục	Câu hỏi	Nội dung chính
2.1 Giới thiệu về Deep Learning	Mạng nơ-ron là gì, dùng cho bài toán nào, và vì sao mới bùng nổ gần đây?	neuron và ReLU, hidden unit, học có giám sát, dữ liệu có cấu trúc và không có cấu trúc, ba động lực của học sâu, bối cảnh lịch sử
2.2 Lập trình mạng nơ-ron: hồi quy logistic và vector hoá	Máy học tham số bằng cách nào, và viết code sao cho chạy đủ nhanh?	hồi quy logistic, hàm mất mát và chi phí, gradient descent, đồ thị tính toán, lan truyền ngược, vector hoá, broadcasting
2.3 Thực hành: NumPy cơ bản và ảnh có mèo	Ghép các thành phần thành một bộ phân loại hoàn chỉnh như thế nào?	các hàm NumPy cơ bản, bộ phân loại ảnh có mèo bằng NumPy và bằng Keras
2.4 Mạng nơ-ron một lớp ẩn	Thêm lớp ẩn thì mạng học được gì thêm, và huấn luyện nó ra sao?	cấu trúc và cách tính đầu ra, hàm kích hoạt, lan truyền ngược, khởi tạo ngẫu nhiên, chọn cỡ mạng
2.5 Thực hành: dữ liệu hình bông hoa	Mạng một lớp ẩn vượt hồi quy logistic bao nhiêu, và nhạy với cấu hình nào?	Keras, scikit-learn, NumPy, kiểm tra độ dốc, thí nghiệm số nút ẩn, hàm kích hoạt, khởi tạo
2.6 Mạng nơ-ron sâu	Xếp nhiều lớp ẩn thì cần thêm gì, và vì sao sâu lại hiệu quả?	nông và sâu, tên gọi và hình dạng trong mạng nhiều lớp, lan truyền xuôi và ngược theo từng khối, tham số và siêu tham số, khởi tạo cho mạng sâu
2.7 Thực hành: mạng sâu nhận dạng ảnh có mèo	Mạng sâu hơn cải thiện bao nhiêu so với hồi quy logistic?	Keras, mở hộp bằng NumPy, so sánh theo độ sâu, dừng sớm, khởi tạo, phân tích ảnh sai, tóm tắt chương

Bảng 2.1: Các mục của chương Mạng nơ-ron và Học sâu

2.1 Giới thiệu về Deep Learning

Trước khi đi vào cách huấn luyện, cần nắm vững bốn điều: mạng nơ-ron thực chất là gì, nó giải quyết loại bài toán nào, vì sao ý tưởng đã có từ nhiều thập kỷ nhưng chỉ mới thành công gần đây, và nên làm gì khi đưa nó vào một dự án.

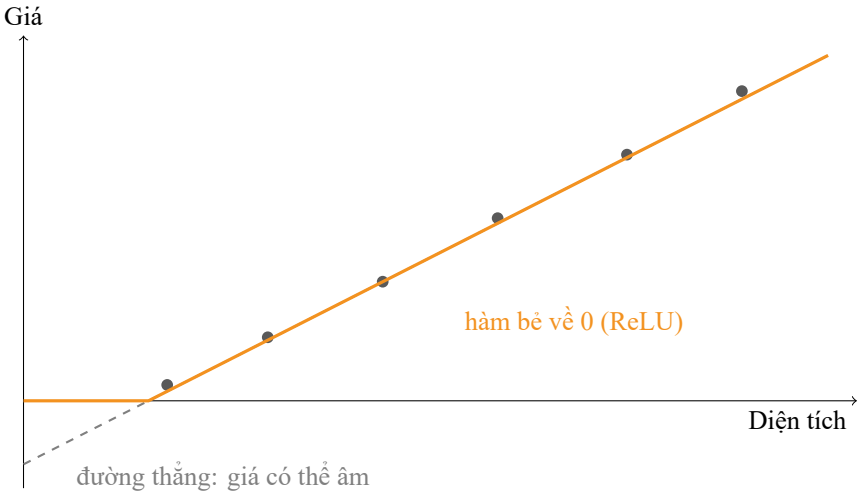
Mục này đi theo lộ trình: từ một neuron đơn lẻ → xếp nhiều neuron thành mạng → học có giám sát và các loại dữ liệu → ba động lực của làn sóng học sâu → bối cảnh lịch sử → áp dụng vào dự án. Code dùng TensorFlow/Keras; những đoạn NumPy ngắn chỉ nhằm cho thấy bản chất của một phép tính.

2.1.1 Mạng nơ-ron là gì

2.1.1.1 Bài toán: dự đoán giá nhà theo diện tích

Giả sử có dữ liệu sáu căn nhà, mỗi căn biết diện tích và giá bán. Cần một hàm cho ra giá khi biết diện tích. Cách quen thuộc nhất là hồi quy tuyến tính: kẻ một đường thẳng đi gần các điểm dữ liệu (Hình 2.1).

Đường thẳng có một khuyết điểm: với căn nhà rất nhỏ, nó cho ra giá âm, trong khi giá nhà không thể âm. Cách sửa tự nhiên là *bẻ* đường thẳng lại: giữ giá ở mức 0 cho đến một diện tích nào đó, rồi đi thẳng lên.

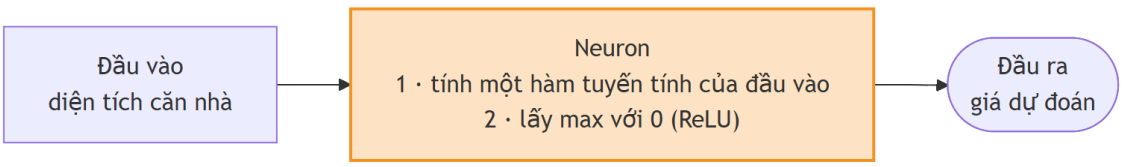


Hình 2.1: Sáu căn nhà, đường thẳng (nét đứt) có thể cho giá âm, còn đường bẻ về 0 (nét đậm) thì không

2.1.1.2 Neuron đơn lẻ và hàm ReLU

Đường bẻ vừa vẽ chính là một mạng nơ-ron, và là mạng đơn giản nhất có thể. Nó chỉ có một **neuron**: nhận diện tích làm đầu vào, tính một hàm tuyến tính của đầu vào, lấy giá trị lớn hơn giữa kết quả đó và 0, rồi trả về làm giá dự đoán (Hình 2.2).

Phép "lấy giá trị lớn hơn giữa kết quả và 0" có tên là **ReLU** (Rectified Linear Unit). "Rectify" nghĩa là cắt bỏ phần âm, nên hàm có dạng nằm ngang ở 0 rồi đi thẳng lên. ReLU xuất hiện ở gần như mọi mạng hiện đại.



Hình 2.2: Một neuron: tính hàm tuyến tính của đầu vào rồi lấy max với 0

```
import numpy as np

def relu_neuron(size, w, b):
    """Một neuron: tính hàm tuyến tính của đầu vào rồi lấy max với 0."""
    return np.maximum(0, w * size + b)

sizes = np.array([40, 60, 80, 100, 150, 200])      # diện tích (mét vuông)
print(relu_neuron(sizes, w=0.05, b=-2))             # [0.  1.  2.  3.  5.5 8. ]
```

Căn nhỏ nhất (40 mét vuông) cho kết quả 0 thay vì số âm. Hai con số (trọng số và hệ số điều chỉnh) chính là thứ mạng phải học từ dữ liệu; cách học được trình bày ở các mục sau của chương.

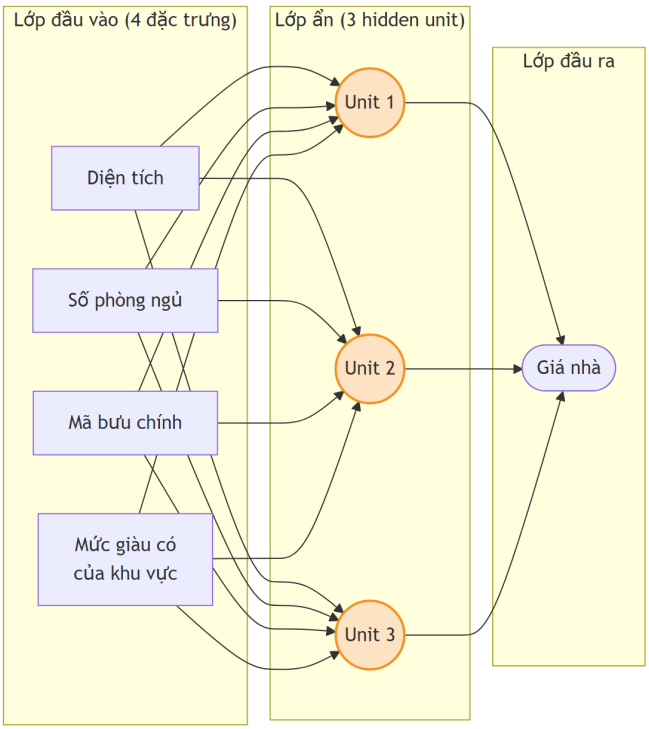
2.1.1.3 Xếp chồng neuron thành mạng

Mạng lớn hơn được tạo bằng cách xếp nhiều neuron lại với nhau, giống như xếp các viên Lego: mỗi neuron là một viên đơn giản, nhiều viên ghép lại thành công trình phức tạp.

Giả sử ngoài diện tích, ta còn biết thêm ba điều về mỗi căn nhà: số phòng ngủ, mã bưu chính và mức giàu có của khu vực. Giá nhà phụ thuộc vào những thứ người mua quan tâm, và ta có thể hình dung ba thứ đó:

- **Quy mô gia đình có thể ở**: do diện tích và số phòng ngủ quyết định.
- **Khả năng đi bộ** (đến chợ, trường học): do mã bưu chính quyết định.
- **Chất lượng trường học**: do mã bưu chính và mức giàu có của khu vực quyết định.

Ba yếu tố trung gian này cùng quyết định số tiền người mua sẵn sàng trả. Hình 2.3 vẽ mạng tương ứng: bốn đặc trưng đầu vào, một lớp ba neuron ở giữa và một neuron đầu ra là giá.



Hình 2.3: Mạng nơ-ron cho bài toán giá nhà: bốn đầu vào, ba hidden unit, một đầu ra

Điểm quan trọng nhất cần nhớ là **ta không tự chỉ định ba yếu tố trung gian đó**. Ta chỉ cung cấp đầu vào và đầu ra cho thật nhiều mẫu huấn luyện, còn mọi thứ ở giữa, mạng tự tìm ra. Mỗi neuron ở giữa (gọi là **hidden unit**, đơn vị ẩn) nhận *cả bốn* đặc trưng đầu vào và tự quyết định nó sẽ biểu diễn điều gì. Các cái tên như "quy mô gia đình" chỉ là cách con người diễn giải sau khi quan sát, không phải thứ ta gán cho mạng.

Vì mỗi đặc trưng đầu vào nối đến mọi neuron của lớp tiếp theo, lớp này được gọi là **kết nối dày đặc** (dense).

Thành phần	Trong ví dụ giá nhà	Ai quyết định nội dung
Lớp đầu vào	diện tích, số phòng ngủ, mã bưu chính, mức giàu có của khu vực	ta cung cấp
Lớp ẩn	ba hidden unit	mạng tự học khi huấn luyện
Lớp đầu ra	giá nhà	ta chọn loại đầu ra, mạng học giá trị

Bảng 2.2: Các lớp của mạng giá nhà

Với đủ dữ liệu gồm cả đầu vào lẫn đầu ra, mạng nơ-ron rất giỏi tìm ra hàm ánh xạ chính xác từ đầu vào sang đầu ra.

2.1.1.4 Dựng bằng Keras

Mạng ở Hình 2.3 viết trong Keras chỉ gồm hai lớp `Dense`. Dữ liệu dưới đây là dữ liệu tự tạo để chạy thử (mã bưu chính được thay bằng điểm khả năng đi bộ nằm trong khoảng 0 đến 1):

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

# Dữ liệu mẫu tự tạo: mỗi căn nhà có 4 đặc trưng
rng = np.random.default_rng(0)
n = 1000
size = rng.uniform(30, 250, n)          # diện tích
bedrooms = rng.integers(1, 6, n)        # số phòng ngủ
walk = rng.uniform(0, 1, n)             # khả năng đi bộ của khu vực
wealth = rng.uniform(0, 1, n)           # mức giàu có của khu vực
price = (np.maximum(0, 0.04 * size - 1) + 0.3 * bedrooms + 1.5 * walk + 2.5 *
↪  wealth
         + rng.normal(0, 0.2, n))
X = np.column_stack([size, bedrooms, walk, wealth]).astype("float32")
y = price.astype("float32")

norm = layers.Normalization()           # đưa 4 đặc trưng về cùng thang đo
norm.adapt(X)

model = keras.Sequential([
    keras.Input(shape=(4,)),
    norm,
    layers.Dense(3, activation="relu"),  # lớp ẩn: 3 hidden unit, mỗi unit nhận
    ↪  cả 4 đặc trưng
    layers.Dense(1),                     # đầu ra: giá (số thực, không cần hàm
    ↪  kích hoạt)
])
model.compile(optimizer="adam", loss="mse")
model.fit(X, y, validation_split=0.2, epochs=50, batch_size=32, verbose=0)
model.summary()                          # 19 tham số cần học: 15 ở lớp ẩn, 4 ở
↪  lớp đầu ra
```

Con số 19 được tính như sau: lớp ẩn có ba neuron, mỗi neuron có bốn trọng số cho bốn đặc trưng và một hệ số điều chỉnh (tổng cộng 15); lớp đầu ra có ba trọng số và một hệ số điều chỉnh (tổng cộng 4). Chỉ cần cho biết số neuron và hàm kích hoạt, Keras tự dựng các trọng số.

2.1.2 Học có giám sát với mạng nơ-ron

Gần như toàn bộ giá trị kinh tế mà mạng nơ-ron tạo ra đến nay đến từ một loại học máy: **học có giám sát** (supervised learning). Ở đó ta có một đầu vào và muốn học hàm ánh xạ sang một đầu ra, dựa trên các mẫu đã biết cả hai. Bài toán giá nhà ở mục trước là một ví dụ. Bảng 2.3 liệt kê thêm những ứng dụng mà mạng nơ-ron đã làm rất tốt.

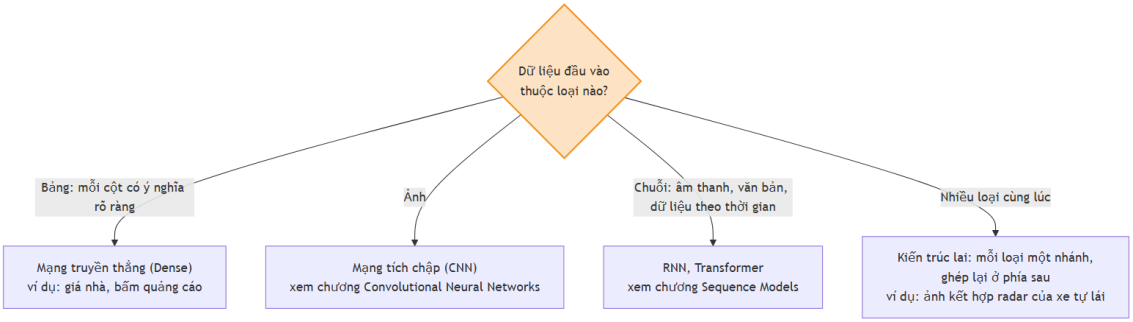
Ứng dụng	Đầu vào	Đầu ra
Dự đoán giá nhà	đặc trưng của căn nhà	giá
Quảng cáo trực tuyến	thông tin quảng cáo và người dùng	người dùng có bấm vào hay không
Thị giác máy tính (gắn thẻ ảnh)	ảnh	chỉ số lớp, ví dụ từ 1 đến 1000
Nhận dạng giọng nói	đoạn âm thanh	bản chép lời
Dịch máy	câu tiếng Anh	câu tiếng Trung
Lái xe tự động	ảnh phía trước xe và dữ liệu radar	vị trí các xe khác trên đường

Bảng 2.3: Một số ứng dụng học có giám sát của mạng nơ-ron

Quảng cáo trực tuyến có lẽ là ứng dụng sinh lời nhiều nhất hiện nay: đoán đúng quảng cáo nào người dùng dễ bấm vào tác động trực tiếp đến doanh thu của các công ty quảng cáo lớn. Điều này cho thấy một điểm chung của các ứng dụng trên: phần lớn giá trị đến từ việc **chọn khéo đầu vào và đầu ra cho đúng bài toán**, rồi gắn thành phần học có giám sát đó vào một hệ thống lớn hơn, như hệ thống xe tự lái.

2.1.2.1 Chọn kiến trúc theo loại dữ liệu

Các ứng dụng khác nhau dùng những kiểu mạng khác nhau (Hình 2.4). Bài toán giá nhà và quảng cáo dùng mạng truyền thẳng như mạng ở Hình 2.3. Ảnh thường dùng mạng tích chập (CNN). Âm thanh là chuỗi theo thời gian, còn câu chữ là chuỗi các từ đến lần lượt, nên cả hai thường dùng mạng hồi quy (RNN) và các phiên bản phức tạp hơn. Bài toán lai như xe tự lái (ảnh kết hợp radar) cần kiến trúc tuỳ biến, ghép nhiều nhánh.



Hình 2.4: Chọn họ kiến trúc theo loại dữ liệu đầu vào

2.1.2.2 Dữ liệu có cấu trúc và không có cấu trúc

Dữ liệu cho học máy chia thành hai loại lớn (Bảng 2.4).

	Có cấu trúc	Không có cấu trúc
Hình dạng	bảng, cơ sở dữ liệu	âm thanh thô, ảnh, văn bản
Đặc trưng	mỗi cột có ý nghĩa rõ ràng (diện tích, số phòng ngủ, tuổi người dùng)	giá trị điểm ảnh, từng từ trong câu
Ví dụ	giá nhà, dự đoán bấm quảng cáo	nhận ra vật trong ảnh, chép lời giọng nói
Kiến trúc thường dùng	mạng truyền thẳng	CNN, RNN, Transformer

Bảng 2.4: Dữ liệu có cấu trúc và không có cấu trúc

Trước đây máy khó hiểu dữ liệu không có cấu trúc hơn nhiều so với dữ liệu có cấu trúc, trong khi con người lại vốn giỏi nhận ra âm thanh và hình ảnh. Nhờ học sâu, máy nay xử lý dữ liệu không có cấu trúc tốt hơn hẳn so với vài năm trước, mở ra nhiều ứng dụng về giọng nói, ảnh và ngôn ngữ.

Cần tránh một hiểu lầm: truyền thông hay nhắc đến thành công trên dữ liệu không có cấu trúc (máy nhận ra con mèo) vì chúng dễ hình dung, nhưng phần lớn giá trị kinh tế ngắn hạn của mạng nơ-ron cũng đến từ dữ liệu có cấu trúc, như hệ thống quảng cáo và gợi ý sản phẩm tốt hơn, hay khả năng dự đoán chính xác từ các cơ sở dữ liệu lớn của doanh nghiệp. Các kỹ thuật trong chương này áp dụng cho cả hai loại.

2.1.2.3 Chọn lớp cuối và hàm mất mát theo loại đầu ra

Một khi đã chọn đầu ra, lớp cuối của mạng và hàm mất mát gần như được quyết định theo (Bảng 2.5). Bảng này chỉ để tra nhanh; ý nghĩa của từng hàm được giải thích ở các mục sau.

Loại đầu ra	Lớp cuối (Keras)	Hàm mất mát	Ví dụ
Một số thực	`Dense(1)`	`mse`	giá nhà
Xác suất của hai khả năng	`Dense(1, activation="sigmoid")`	`binary_crossentropy`	có bấm quảng cáo hay không
Một trong nhiều lớp	`Dense(num_classes, activation="softmax")`	`sparse_categorical_crossentropy`	gắn thẻ ảnh với 1000 lớp

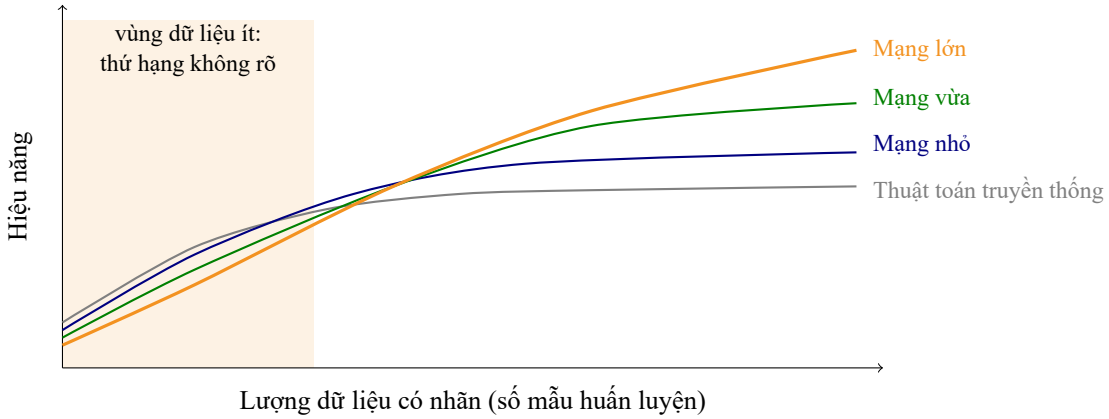
Bảng 2.5: Lớp cuối và hàm mất mát theo loại đầu ra (`num_classes` là số lớp cần phân biệt)

2.1.3 Vì sao học sâu bùng nổ gần đây

Các ý tưởng nền tảng của mạng nơ-ron đã có từ nhiều thập kỷ. Câu hỏi tự nhiên là vì sao chỉ mới gần đây chúng mới thành công đến vậy. Hiểu câu trả lời giúp nhận ra đâu là cơ hội đáng đầu tư trong một tổ chức.

2.1.3.1 Quy mô là động lực chính

Hãy xét một biểu đồ: trục ngang là lượng dữ liệu có nhãn cho một bài toán (số mẫu huấn luyện), trục dọc là hiệu năng của thuật toán, như độ chính xác của bộ lọc thư rác (Hình 2.5).



Hình 2.5: Hiệu năng theo lượng dữ liệu: thuật toán truyền thống đi ngang, mạng càng lớn càng được lợi từ dữ liệu

Từ biểu đồ có thể rút ra ba nhận xét:

- **Thuật toán truyền thống** (như SVM hay hồi quy logistic) cải thiện một lúc khi thêm dữ liệu, rồi đi ngang. Chúng không tận dụng được lượng dữ liệu rất lớn.
- **Mạng nơ-ron càng lớn càng tận dụng được dữ liệu**: mạng nhỏ đi ngang muộn hơn, mạng vừa muộn hơn nữa, còn mạng rất lớn gần như cứ tốt thêm khi có thêm dữ liệu.
- **Ở vùng dữ liệu ít**, thứ hạng giữa các thuật toán không rõ ràng. Lúc đó kết quả phụ thuộc nhiều vào tay nghề thiết kế đặc trưng thủ công, nên hoàn toàn có thể một thuật toán truyền thống được chăm chút kỹ vượt mạng nơ-ron. Chỉ ở vùng dữ liệu lớn, mạng lớn mới thắng ổn định.

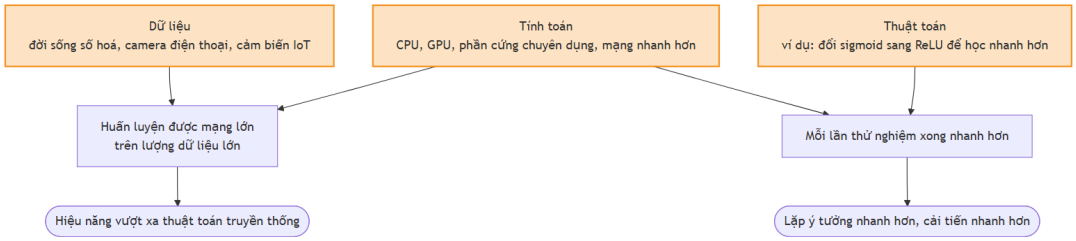
Vì vậy, để đạt hiệu năng cao cần cả hai điều: một mạng đủ lớn (nhiều hidden unit, tham số, kết nối) và lượng dữ liệu đủ lớn. Hai yếu tố này gộp lại gọi là **quy mô** (scale). Ngày nay, một trong những cách đáng tin cậy nhất để cải thiện mạng là huấn luyện mạng lớn hơn hoặc thêm dữ liệu. Cách này chỉ hiệu quả đến một giới hạn, vì cuối cùng ta hết dữ liệu hoặc mạng quá lớn nên huấn luyện quá lâu.

Lưu ý trục ngang là lượng dữ liệu *có nhãn*, tức các mẫu có cả đầu vào lẫn đầu ra. Có nhiều dữ liệu nhưng thiếu nhãn thì chưa giúp được học có giám sát.

2.1.3.2 Ba động lực

Quy mô không phải yếu tố duy nhất. Ba động lực cùng tác động (Hình 2.6):

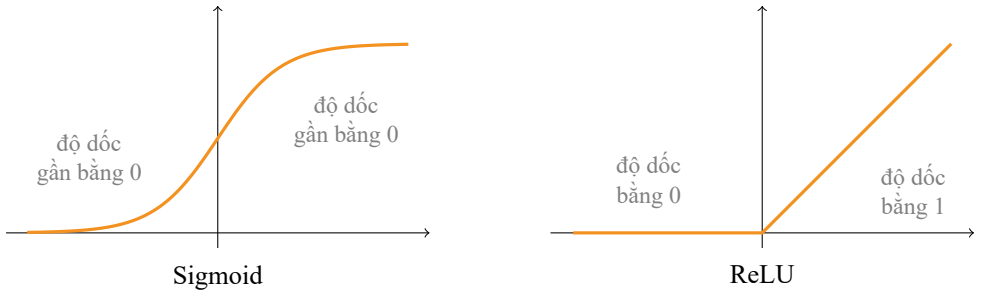
- **Dữ liệu**: xã hội số hoá khiến phần lớn hoạt động của con người để lại dữ liệu (web, ứng dụng di động); camera giá rẻ trên điện thoại, cảm biến gia tốc và các cảm biến của Internet vạn vật cũng liên tục thu thêm.
- **Tính toán**: CPU, GPU, phần cứng chuyên dụng và mạng nhanh hơn cho phép huấn luyện mạng rất lớn.
- **Thuật toán**: nhiều cải tiến thuật toán nhằm giúp mạng chạy nhanh hơn, kéo theo huấn luyện được mạng lớn hơn và thử nghiệm nhanh hơn.



Hình 2.6: Ba động lực thúc đẩy học sâu và tác động của chúng

2.1.3.3 Ví dụ về đổi mới thuật toán: từ sigmoid sang ReLU

Một bước đột phá nhỏ nhưng có tác động lớn là đổi hàm kích hoạt của lớp ẩn từ **sigmoid** sang **ReLU** (Hình 2.7). Sigmoid có hai vùng gần như nằm ngang ở hai đầu, nơi độ dốc (gradient) gần bằng 0. Khi độ dốc gần 0, gradient descent chỉ cập nhật tham số rất chậm nên mạng học chậm. ReLU có độ dốc bằng 1 với mọi đầu vào dương nên gradient ít bị co về 0, và phép tính cũng đơn giản hơn.



Hình 2.7: Sigmoid có hai vùng độ dốc gần bằng 0; ReLU có độ dốc bằng 1 ở mọi đầu vào dương

Có thể kiểm chứng bằng vài dòng NumPy: tính độ dốc của hai hàm tại các giá trị đầu vào khác nhau.

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

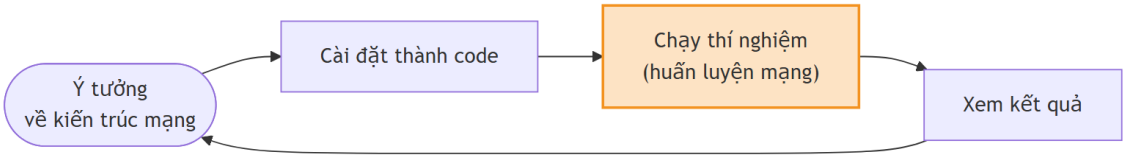
z = np.array([-10, -5, 0, 5, 10], dtype=float)
sigmoid_slope = sigmoid(z) * (1 - sigmoid(z))   # độ dốc của sigmoid
relu_slope = (z > 0).astype(float)              # độ dốc của ReLU: 1 nếu dương, 0
↪  nếu không

print("sigmoid:", sigmoid_slope.round(5))        # [0.00005 0.00665 0.25
↪  0.00665 0.00005]
print("ReLU   :", relu_slope)                    # [0. 0. 0. 1. 1.]
```

Với sigmoid, độ dốc tại đầu vào 10 chỉ khoảng 0,00005, tức mạng gần như không cập nhật được ở vùng đó. Với ReLU, độ dốc ở vùng dương luôn là 1. Ở phía âm, ReLU có độ dốc 0, nên trong thực tế người ta cân nhắc thêm các biến thể khi cần. Nhiều cải tiến thuật toán có cùng mục đích: giúp mã chạy nhanh hơn để huấn luyện được mạng lớn hơn.

2.1.3.4 Vì sao tốc độ huấn luyện quyết định năng suất

Xây một mạng hiệu quả hiếm khi đúng ngay lần đầu. Quy trình thật là một vòng lặp (Hình 2.8): nảy ra ý tưởng về kiến trúc, cài đặt thành code, chạy thí nghiệm để biết mạng tốt đến đâu, nhìn kết quả rồi quay lại chỉnh ý tưởng.



Hình 2.8: Vòng lặp thử nghiệm khi xây mạng nơ-ron

Bước tốn thời gian nhất là chạy thí nghiệm. Nếu một lần huấn luyện mất khoảng 10 phút, ta thử được vài chục ý tưởng trong một ngày làm việc. Nếu mất một tháng, ta thử được rất ít ý tưởng và dễ chọn sai hướng mà không biết. Tính toán nhanh vì vậy không chỉ giúp mạng tốt hơn, mà còn giúp cả người làm thực tế lẫn nhà nghiên cứu lặp nhanh hơn, và đó là một lý do cộng đồng nghiên cứu liên tục có thuật toán mới.

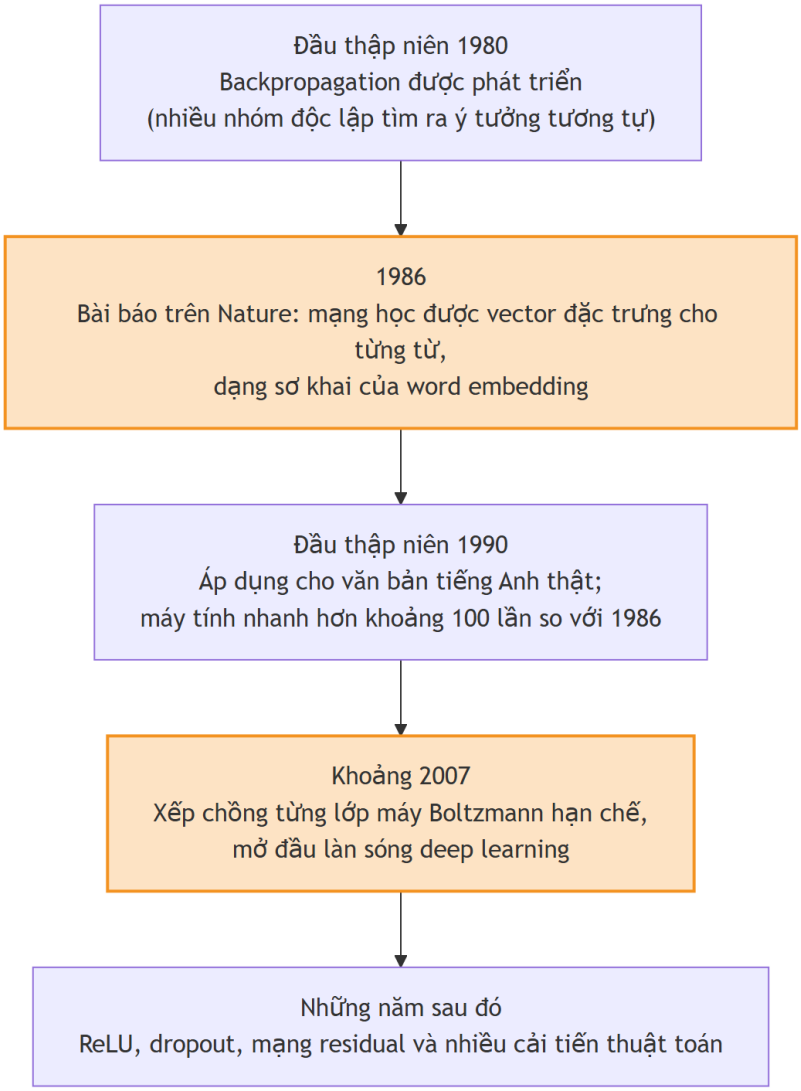
2.1.3.5 Xu hướng còn tiếp diễn

Cả ba động lực đều đang tiếp tục. Xã hội vẫn sinh thêm dữ liệu số, phần cứng chuyên dụng như GPU và mạng nhanh hơn vẫn tiếp tục được cải thiện, còn cộng đồng nghiên cứu vẫn liên tục đưa ra thuật toán mới. Vì vậy có cơ sở để kỳ vọng học sâu còn tốt hơn trong nhiều năm tới.

2.1.4 Bối cảnh lịch sử và lời khuyên cho người mới

Nhìn lại lịch sử giúp thấy rằng thành công hiện nay được xây từ nhiều thập kỷ, và lời khuyên của những người đặt nền móng cũng hữu ích cho người mới vào nghề. Các thông tin dưới đây theo lời kể của Geoffrey Hinton, một trong những người đã đóng góp nhiều ý tưởng nền tảng cho học sâu.

2.1.4.1 Vài cột mốc



Hình 2.9: Một số cột mốc trong lịch sử mạng nơ-ron

- **Backpropagation** – thuật toán lan truyền ngược, giúp tính cách chỉnh các trọng số của mạng nhiều lớp. Nó được phát triển trong nhóm của David Rumelhart, Hinton và Ronald Williams vào đầu thập niên 1980. Nhiều người khác cũng đã tìm ra những dạng tương tự trước đó, nên ý tưởng này không hoàn toàn mới. Bài báo năm 1986 trên tạp chí Nature mới khiến cộng đồng chú ý.
- **Biểu diễn từ học được** – điều gây ấn tượng trong bài báo năm 1986 là mạng, khi học trên các bộ ba từ về quan hệ trong gia đình, tự học ra vector đặc trưng cho từng từ, và các đặc trưng đó mang nghĩa như quốc tịch, thế hệ hay nhánh gia đình. Đó là dạng sơ khai của *word embedding* sẽ gặp lại ở mục 6.2. Đầu thập niên 1990, Yoshua Bengio áp dụng cùng ý tưởng cho văn bản tiếng Anh thật.
- **Sức mạnh tính toán** – năm 1986 máy tính dùng để thử nghiệm chạy chưa tới một phần mười megaflop; đến khoảng năm 1993 là cỡ mười megaflop, nhanh hơn khoảng 100 lần. Chính sự tăng tốc này làm mạng nơ-ron bắt đầu dùng được, khớp với vai trò của quy mô ở mục 2.1.3.

- **Máy Boltzmann hạn chế (RBM)** – học được một lớp đặc trưng từ dữ liệu, rồi xem các đặc trưng đó như dữ liệu để học lớp tiếp theo, cứ thế xếp chồng nhiều lớp. Cách làm này góp phần mở đầu làn sóng học sâu từ khoảng năm 2007, và RBM là một thành phần của đội thắng cuộc thi Netflix.
- **ReLU** – ReLU gần như tương đương một chồng đơn vị logistic. Nhờ vậy các kết quả toán học đã có cho đơn vị logistic vẫn áp dụng được, và đó là một lý do ReLU được chấp nhận.
- **Khởi tạo bằng ma trận đơn vị** – Hinton từng chỉ ra có thể huấn luyện hiệu quả mạng cỡ 300 lớp ẩn nếu khởi tạo trọng số theo cách để mỗi lớp ban đầu chỉ sao chép lớp bên dưới. Ông tiếc là không theo đuổi tiếp, vì ý tưởng tương tự về sau xuất hiện lại trong mạng *residual*.

2.1.4.2 Lời khuyên cho người mới vào nghề

- **Đọc vừa đủ**: đọc một ít tài liệu để có nhận định riêng, chú ý điều gì mọi người đang làm mà mình thấy chưa ổn, rồi tìm cách làm đúng và kiên trì với nó. Nhận định bị nhiều người phản đối không chứng minh ý tưởng đúng, nên cần kiểm chứng bằng thí nghiệm.
- **Tự tái hiện kết quả của các bài báo**: việc này tốn thời gian nhưng bộc lộ ra những mẹo nhỏ cần có để mô hình chạy được.
- **Đừng ngừng lập trình**: khi mô hình "không chạy", nguyên nhân thường là một quyết định nhỏ mà người làm chưa nhận ra là quan trọng. Người giỏi tự tìm ra và sửa nó.
- **Tìm người hướng dẫn có quan điểm gần với mình**: làm việc trong chủ đề người hướng dẫn quan tâm sâu sắc thì nhận được nhiều lời khuyên có giá trị hơn.

2.1.4.3 Một cách dùng máy tính mới

Một thay đổi sâu hơn công nghệ: trước đây ta *lập trình* cho máy tính từng bước, còn nay ta *cho máy xem ví dụ* và nó tự tìm cách làm. Hai cách dùng này khác nhau về bản chất, và cách thứ hai được kỳ vọng có tầm quan trọng không kém cách thứ nhất. Cũng theo cách nhìn này, một ý nghĩ trong mạng không phải là một chuỗi từ hay một biểu thức ký hiệu, mà là một vector rất lớn gồm các hoạt động thần kinh, và các vector như vậy tác động lên nhau.

2.1.5 Áp dụng vào dự án thực tế

2.1.5.1 Câu hỏi cần trả lời trước khi dùng mạng nơ-ron

Câu hỏi	Nếu câu trả lời là…	Thì
Có đầu vào và đầu ra rõ ràng, có nhãn không?	chưa có nhãn	chưa phải học có giám sát; cân nhắc tạo nhãn hoặc đổi cách đặt bài toán
Dữ liệu thuộc loại nào?	bảng, ảnh, chuỗi, hay nhiều loại	chọn họ kiến trúc theo Hình 2.4
Có bao nhiêu dữ liệu có nhãn?	ít	thử trước một baseline đơn giản (hồi quy, cây quyết định); mạng lớn chưa chắc hơn
Có bao nhiêu dữ liệu có nhãn?	nhiều	mạng lớn có cơ hội thắng; kích thước mạng và lượng dữ liệu cùng quyết định kết quả
Một lần huấn luyện mất bao lâu?	rất lâu	thử trước trên tập con nhỏ hoặc mạng nhỏ để vòng lặp thử nghiệm quay nhanh
Hiệu năng chưa đủ, nên làm gì trước?	mạng còn nhỏ hoặc dữ liệu còn ít	tăng kích thước mạng hoặc thêm dữ liệu có nhãn, đến khi gặp giới hạn

Bảng 2.6: Checklist trước khi dùng mạng nơ-ron cho một bài toán

2.1.5.2 Hiểu lầm và lỗi thường gặp

Hiểu lầm / triệu chứng	Thực tế và cách sửa
Mạng càng lớn càng tốt	chỉ đúng khi có đủ dữ liệu; ở vùng dữ liệu ít, mạng lớn không đảm bảo hơn các thuật toán đơn giản
Hidden unit sẽ học đúng thứ mình đặt tên (như "quy mô gia đình")	mạng tự quyết định hidden unit biểu diễn gì; tên chỉ là cách diễn giải sau khi quan sát
Có nhiều dữ liệu là đủ	học có giám sát cần dữ liệu *có nhãn*; số mẫu có nhãn mới là yếu tố quyết định
Lớp ẩn dùng sigmoid vì đầu ra cần xác suất	lớp ẩn nên bắt đầu với ReLU; sigmoid chỉ hợp ở lớp cuối của bài toán hai khả năng
Dự đoán một số thực (như giá) mà lớp cuối có `softmax` hoặc `sigmoid`	đầu ra bị ép vào khoảng cố định; dùng `Dense(1)` không kích hoạt và hàm mất mát `mse` (Bảng 2.5)
Đặc trưng thang đo rất khác nhau (diện tích hàng trăm, điểm 0 đến 1) khiến học chậm	chuẩn hoá đầu vào, ví dụ bằng `layers.Normalization()` như ở ví dụ giá nhà
Chờ nhiều giờ mới biết ý tưởng có ổn không	đổi cách làm: thử trên tập con và mạng nhỏ để mỗi vòng thử nghiệm chỉ vài phút

Bảng 2.7: Hiểu lầm và lỗi thường gặp khi mới làm việc với mạng nơ-ron

2.1.6 Tóm tắt cần nhớ

1. **Deep Learning là huấn luyện mạng nơ-ron**. Mạng đơn giản nhất chỉ có một neuron: tính hàm tuyến tính của đầu vào rồi lấy max với 0 (**ReLU**).
2. **Mạng lớn được ghép từ nhiều neuron**. Các neuron ở giữa là *hidden unit*; ta chỉ cung cấp đầu vào và đầu ra, mạng tự học phần ở giữa. Lớp mà mỗi neuron nhận mọi đầu vào của lớp trước gọi là *dense*.
3. **Học có giám sát** (đầu vào ánh xạ sang đầu ra, có nhãn) là loại học máy tạo ra phần lớn giá trị của mạng nơ-ron. Nhiều giá trị đến từ việc chọn đúng đầu vào và đầu ra cho bài toán.
4. **Chọn kiến trúc theo dữ liệu**: bảng dùng mạng truyền thẳng, ảnh dùng CNN, chuỗi dùng RNN hoặc Transformer, nhiều loại dữ liệu cùng lúc dùng kiến trúc lai.
5. **Dữ liệu có cấu trúc** (bảng, đặc trưng rõ nghĩa) và **không có cấu trúc** (ảnh, âm thanh, văn bản): học sâu giúp ích cho cả hai, và nhiều giá trị kinh tế đến từ dữ liệu có cấu trúc.
6. **Quy mô thúc đẩy học sâu**: mạng càng lớn càng tận dụng được dữ liệu, trong khi thuật toán truyền thống đi ngang. Ở vùng dữ liệu ít, thứ hạng không rõ ràng.
7. **Ba động lực**: dữ liệu, tính toán, thuật toán. Ví dụ đổi mới thuật toán là đổi sigmoid sang ReLU để gradient không bị co về 0.
8. **Tốc độ huấn luyện quyết định năng suất**, vì xây mạng là vòng lặp ý tưởng, code, thí nghiệm.
9. **Với người mới**: đọc vừa đủ, tự tái hiện kết quả, tiếp tục lập trình, và tin vào nhận định của mình nhưng phải kiểm chứng bằng thí nghiệm.

2.2 Lập trình mạng nơ-ron: hồi quy logistic và vector hoá

Mục trước cho thấy mạng nơ-ron là gì. Mục này trả lời câu hỏi tiếp theo: *máy học các tham số như thế nào, và viết code sao cho chạy đủ nhanh*. Để ý tưởng dễ theo dõi, ta dùng bài toán đơn giản nhất: **hồi quy logistic**, tức một neuron có hàm sigmoid, phân loại một ảnh là có mèo hay không có mèo. Hồi quy logistic chính là mạng nơ-ron nhỏ nhất, nên mọi thứ học được ở đây dùng lại nguyên vẹn cho mạng nhiều lớp ở các mục sau.

Mục này đi theo lộ trình: dữ liệu và quy ước → mô hình hồi quy logistic → hàm mất mát và hàm chi phí → đạo hàm → gradient descent → đồ thị tính toán và lan truyền ngược → áp dụng cho hồi quy logistic → vector hoá → áp dụng vào dự án. Hai ví dụ thực hành nằm ở mục 2.3. Trong dự án, những việc này do thư viện (scikit-learn, Keras) làm hộ; các đoạn NumPy ở đây dùng để thấy bản chất của từng phép tính. Văn bản dùng *tên mô tả* (trọng số, tổng có trọng số, xác suất dự đoán, …) thay cho ký hiệu; tên biến tương ứng trong code được ghi ở Bảng 2.8.

2.2.1 Phân loại nhị phân và cách biểu diễn dữ liệu

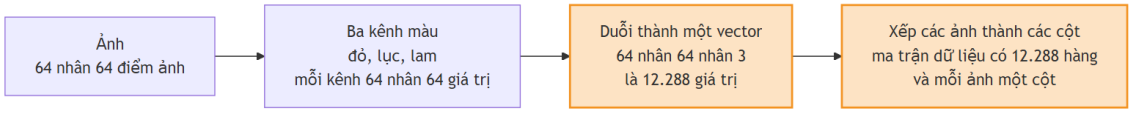
2.2.1.1 Bài toán phân loại nhị phân

Trong **phân loại nhị phân** (binary classification), nhãn đầu ra chỉ nhận một trong hai giá trị. Ví dụ: đầu vào là một ảnh, đầu ra là 1 nếu ảnh có mèo và 0 nếu không có mèo. Mục tiêu là học một bộ phân loại nhận ảnh và dự đoán nhãn của nó.

2.2.1.2 Ảnh trong máy tính

Máy tính lưu một ảnh màu thành ba ma trận, mỗi ma trận ứng với một kênh màu: đỏ, lục, lam. Giá trị trong ma trận là cường độ điểm ảnh, từ 0 đến 255. Ảnh 64 nhân 64 điểm ảnh cho ba ma trận cỡ 64 nhân 64.

Để đưa ảnh vào mô hình, ta **duỗi** (unroll) toàn bộ giá trị này thành một vector đặc trưng duy nhất (Hình 2.10). Với ảnh 64 nhân 64, vector có 64 nhân 64 nhân 3, tức **12.288** giá trị. Số này là số đặc trưng của một mẫu.



Hình 2.10: Từ ảnh màu đến cột của ma trận dữ liệu

2.2.1.3 Quy ước: mỗi mẫu là một cột

Tập huấn luyện có nhiều mẫu, mỗi mẫu là một cặp (vector đặc trưng, nhãn). Để xử lý cả tập cùng lúc, ta gom chúng thành ma trận theo một **quy ước dùng xuyên suốt**: mỗi mẫu là một *cột* (Bảng 2.8).

Tên gọi trong văn bản	Tên trong code	Hình dạng	Ý nghĩa
Ma trận dữ liệu	`X`	`(số đặc trưng, số mẫu)`	mỗi cột là một mẫu; ví dụ `(12288, 209)`
Ma trận nhãn	`Y`	`(1, số mẫu)`	nhãn 0 hoặc 1 của từng mẫu
Trọng số	`w`	`(số đặc trưng, 1)`	mỗi đặc trưng một trọng số
Hệ số điều chỉnh	`b`	một số thực	độ lệch (bias) cộng vào tổng
Tổng có trọng số	`Z`	`(1, số mẫu)`	trước khi qua sigmoid, mỗi mẫu một giá trị
Xác suất dự đoán	`A`	`(1, số mẫu)`	sau sigmoid, mỗi mẫu một giá trị
Sai số dự đoán	`dZ`	`(1, số mẫu)`	xác suất dự đoán trừ nhãn thật
Độ dốc theo trọng số, theo hệ số điều chỉnh	`dw`, `db`	như `w`, như `b`	đạo hàm của chi phí theo tham số
Tốc độ học	`alpha`	một số thực	độ dài mỗi bước cập nhật

Bảng 2.8: Tên gọi, tên biến trong code và hình dạng. Từ đây về sau, mọi dữ liệu gắn với từng mẫu đều được xếp theo cột

Một số tài liệu xếp mẫu theo *hàng*. Cách xếp theo cột làm việc cài đặt mạng nơ-ron gọn hơn, nên được dùng ở đây. Cần nhớ sự khác biệt này khi chuyển sang framework: Keras và scikit-learn nhận dữ liệu với mẫu nằm ở *hàng*, nên phải chuyển vị (`.T`) khi truyền dữ liệu đã xếp theo cột vào chúng.

```
import numpy as np

images = np.random.randint(0, 256, size=(5, 64, 64, 3), dtype=np.uint8)   # 5 ảnh
↪  64 nhân 64, 3 kênh màu

X = images.reshape(images.shape[0], -1).T        # mỗi cột là một ảnh đã duỗi
```

```
print(X.shape)                                          # (12288, 5)
assert (X[:3, 0] == images[0, 0, 0, :]).all()           # 3 giá trị đầu của cột 0 là 3
↪  kênh của điểm ảnh đầu tiên
X = X / 255.                                            # đưa giá trị về khoảng từ 0
↪  đến 1
```

`reshape(số mẫu, -1)` gộp mọi chiều còn lại thành một, rồi `.T` biến mỗi mẫu thành một cột. Thứ tự các giá trị trong cột (ở đây là từng điểm ảnh một, mỗi điểm ba kênh) không quan trọng, miễn là mọi ảnh dùng cùng một thứ tự. Bước chia cho 255 là chuẩn hoá đơn giản và hiệu quả cho dữ liệu ảnh: mọi đặc trưng cùng nằm trong khoảng từ 0 đến 1, giúp gradient descent chạy ổn định.

2.2.2 Hồi quy logistic

Hồi quy logistic là thuật toán dùng cho phân loại nhị phân. Với một mẫu, mô hình đưa ra **xác suất dự đoán**: khả năng nhãn bằng 1. Với ảnh, đó là xác suất ảnh có mèo, nên giá trị phải nằm trong khoảng từ 0 đến 1. Mô hình có hai nhóm tham số: **trọng số** (mỗi đặc trưng một trọng số) và **hệ số điều chỉnh**, hai nhóm này được giữ riêng.

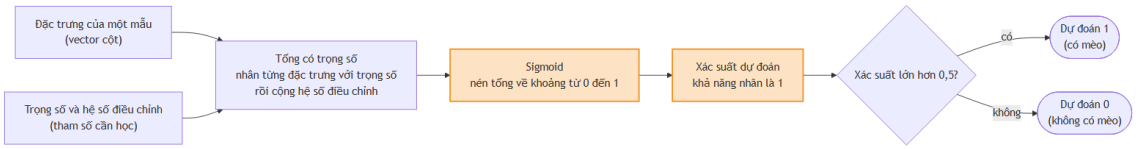
2.2.2.1 Vì sao cần hàm sigmoid

Cách đơn giản nhất là lấy tổng có trọng số của các đặc trưng rồi cộng hệ số điều chỉnh, như hồi quy tuyến tính. Nhưng kết quả có thể lớn hơn 1 hoặc âm, không thể là xác suất. Hồi quy logistic khắc phục bằng cách đưa tổng đó qua **hàm sigmoid** (Hình 2.11).

Sigmoid là một đường cong mượt đi từ 0 lên 1, cắt trục dọc ở 0,5. Đầu vào rất lớn thì đầu ra gần 1; đầu vào âm rất lớn thì đầu ra gần 0. Bảng 2.9 cho vài giá trị.

Đầu vào của sigmoid	−6	−2	0	2	6
Đầu ra	0,0025	0,1192	0,5000	0,8808	0,9975

Bảng 2.9: Giá trị của hàm sigmoid



Hình 2.11: Dự đoán của hồi quy logistic cho một mẫu

Sau khi có xác suất, nhãn dự đoán được chọn bằng **ngưỡng 0,5**: xác suất lớn hơn 0,5 thì dự đoán 1, ngược lại dự đoán 0. Việc học nhằm tìm trọng số và hệ số điều chỉnh sao cho xác suất này là ước lượng tốt của khả năng nhãn bằng 1.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))                 # áp dụng cho từng phần tử

w = np.array([[0.5], [-0.25]])                  # 2 đặc trưng
b = 0.1
X = np.array([[1.0, -2.0, 4.0],                 # mỗi cột một mẫu: 3 mẫu
              [2.0,  1.0, 0.0]])
A = sigmoid(np.dot(w.T, X) + b)                 # xác suất cho cả 3 mẫu cùng lúc
print(A.round(4), A.shape)                      # [[0.525  0.2405 0.8909]] (1,
↪  3)
print((A > 0.5).astype(int))                    # [[1 0 1]]
```

2.2.3 Hàm mất mát và hàm chi phí

Để tìm trọng số và hệ số điều chỉnh, cần một thước đo cho biết dự đoán sai đến mức nào. Có hai khái niệm cần phân biệt:

- **Hàm mất mát** (loss) đo mức sai của dự đoán trên *một mẫu*.
- **Hàm chi phí** (cost) đo mức sai trên *toàn bộ tập huấn luyện*: là trung bình của mất mát trên mọi mẫu. Huấn luyện là tìm tham số làm chi phí nhỏ nhất.

2.2.3.1 Vì sao không dùng sai số bình phương

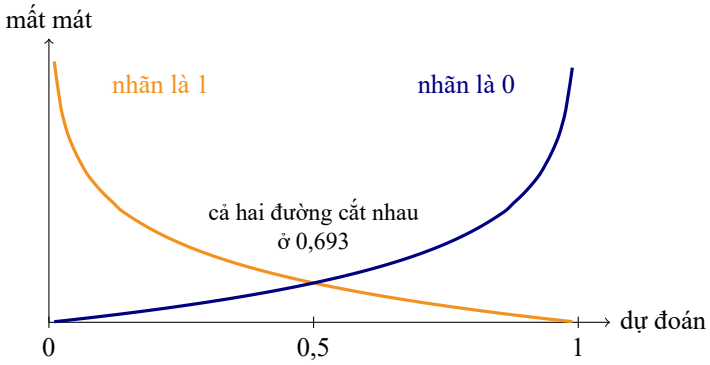
Sai số bình phương là lựa chọn quen thuộc, nhưng với hồi quy logistic nó biến bài toán tối ưu thành **không lồi**: có nhiều cực trị cục bộ, nên gradient descent có thể mắc kẹt mà không tìm được nghiệm tốt nhất. Hồi quy logistic dùng một hàm mất mát khác, có vai trò tương tự nhưng cho bài toán **lồi**, dễ tối ưu hơn rất nhiều. Đó là **cross-entropy nhị phân**.

2.2.3.2 Cross-entropy nhị phân

Hàm này tách thành hai trường hợp tuỳ nhãn thật:

- **Nhãn là 1**: mất mát là trừ logarit của xác suất dự đoán. Muốn mất mát nhỏ thì xác suất dự đoán phải gần 1.
- **Nhãn là 0**: mất mát là trừ logarit của (1 trừ xác suất dự đoán). Muốn mất mát nhỏ thì xác suất dự đoán phải gần 0.

Hình 2.12 vẽ hai đường cong. Điểm đáng chú ý là mất mát tăng rất nhanh khi mô hình *tự tin mà sai*: nhãn là 1 mà dự đoán 0,01 bị phạt nặng hơn nhiều so với dự đoán 0,4.



Hình 2.12: Mất mát theo dự đoán cho hai trường hợp nhãn. Dự đoán sai mà tự tin thì bị phạt rất nặng

Dự đoán	0,01	0,1	0,5	0,9	0,99
Mất mát khi nhãn là 1	4,605	2,303	0,693	0,105	0,010
Mất mát khi nhãn là 0	0,010	0,105	0,693	2,303	4,605

Bảng 2.10: Mất mát cross-entropy nhị phân ở vài mức dự đoán

Giá trị 0,693 (logarit tự nhiên của 2) là mất mát khi mô hình chỉ đoán 0,5, tức không biết gì. Đó cũng là chi phí ở lần lặp đầu tiên khi khởi tạo mọi tham số bằng 0, và là mốc để so sánh: chi phí phải giảm xuống dưới mức đó thì mô hình mới học được điều gì.

```
def loss(a, y):
    """Mất mát cross-entropy nhị phân cho từng mẫu."""
    return -(y * np.log(a) + (1 - y) * np.log(1 - a))

def cost(A, Y):
    """Chi phí: trung bình mất mát trên mọi mẫu."""
    return np.mean(loss(A, Y))

a = np.array([0.01, 0.1, 0.5, 0.9, 0.99])
print(loss(a, 1).round(3))                      # [4.605 2.303 0.693 0.105 0.01
↪  ]
print(loss(a, 0).round(3))                      # [0.01  0.105 0.693 2.303
↪  4.605]

A = np.array([[0.9, 0.2, 0.7, 0.4]])            # dự đoán cho 4 mẫu
Y = np.array([[1, 0, 1, 1]])                    # nhãn thật
print(cost(A, Y))                               # 0.4003674356962309

from tensorflow import keras
print(float(keras.losses.BinaryCrossentropy()(Y, A)))   # 0.40036746859... (cùng
↪  giá trị, khác nhau do độ chính xác số thực)
```

Trong dự án, nên dùng hàm mất mát có sẵn của framework (`binary_crossentropy` trong Keras) thay vì tự viết. Bản của framework còn xử lý các trường hợp biên mà bản viết tay ở trên chưa lo, như dự đoán đúng bằng 0 hoặc 1 làm logarit thành vô cùng.

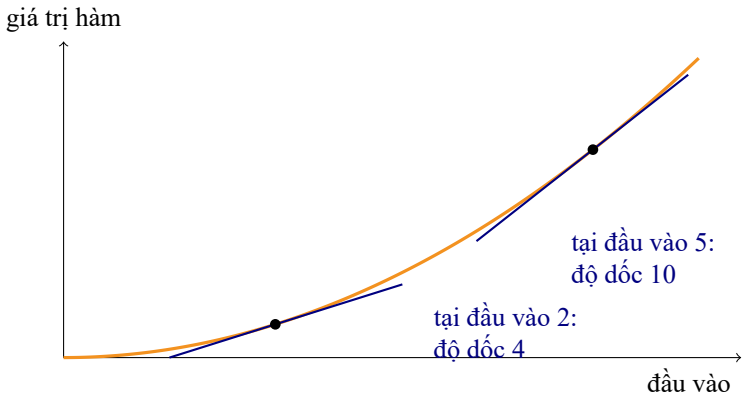
2.2.4 Đạo hàm là độ dốc

Huấn luyện cần biết *nên chỉnh tham số theo hướng nào để chi phí giảm*. Câu trả lời đến từ **đạo hàm**. Không cần giỏi giải tích để dùng mạng nơ-ron hiệu quả: chỉ cần hiểu **đạo hàm nghĩa là độ dốc** của hàm tại một điểm. (Về sau các hàm của framework sẽ lo hết phần tính toán này, nhưng nên mở hộp ra xem một lần.)

2.2.4.1 Đẩy nhẹ rồi quan sát

Cách hiểu đạo hàm dễ nhất là làm thí nghiệm: đẩy đầu vào lên một lượng rất nhỏ, ví dụ 0,001, rồi xem đầu ra thay đổi bao nhiêu lần so với lượng đã đẩy. Tỉ lệ đó chính là độ dốc.

- Với hàm *ba lần đầu vào* (một đường thẳng): tại đầu vào 2, hàm bằng 6. Đẩy đầu vào lên 2,001 thì hàm lên 6,003, tăng gấp **3** lần lượng đẩy. Tại đầu vào 5 cũng vậy. Đường thẳng có độ dốc không đổi.
- Với hàm *bình phương đầu vào* (một đường cong): tại đầu vào 2, hàm bằng 4; đẩy lên 2,001 thì hàm lên khoảng 4,004, gấp **4** lần. Nhưng tại đầu vào 5, hàm bằng 25; đẩy lên 5,001 thì hàm lên khoảng 25,010, gấp **10** lần. Độ dốc *khác nhau ở các điểm khác nhau* (Hình 2.13).



Hình 2.13: Hàm bình phương: tiếp tuyến dốc hơn khi đầu vào lớn hơn

Hàm	Đầu vào	Giá trị tại đó	Sau khi đẩy thêm 0,001	Độ dốc ước lượng	Độ dốc theo bảng tra
Ba lần đầu vào	2	6	6,003	3,0	luôn bằng 3
Ba lần đầu vào	5	15	15,003	3,0	luôn bằng 3
Bình phương	2	4	4,004001	4,001	gấp đôi đầu vào, tức 4
Bình phương	5	25	25,010001	10,001	gấp đôi đầu vào, tức 10
Lập phương	2	8	8,012006	12,006	ba lần bình phương đầu vào, tức 12
Logarit tự nhiên	2	0,6931	0,6936	0,4999	một chia đầu vào, tức 0,5

Bảng 2.11: Độ dốc ước lượng bằng cách đẩy nhẹ so với độ dốc theo bảng tra đạo hàm

Độ dốc ước lượng (4,001) lệch chút so với giá trị đúng (4) vì lượng đẩy 0,001 chưa nhỏ vô hạn. Định nghĩa chặt của đạo hàm dùng lượng đẩy nhỏ vô cùng; trong thực hành, lượng đẩy rất nhỏ như trên đã đủ cho mục đích hiểu và kiểm tra.

```
def numerical_slope(f, a, h=0.001):
    """Đẩy a lên một lượng nhỏ h rồi xem f thay đổi bao nhiêu lần so với h."""
```

```
    return (f(a + h) - f(a)) / h

print(numerical_slope(lambda a: 3 * a, 2))        # khoảng 3.0
print(numerical_slope(lambda a: a ** 2, 2))       # khoảng 4.001
print(numerical_slope(lambda a: a ** 2, 5))       # khoảng 10.001
print(numerical_slope(lambda a: a ** 3, 2))       # khoảng 12.006
print(numerical_slope(np.log, 2))                 # khoảng 0.4999
```

2.2.4.2 Tra bảng, hoặc để framework tự tính

Muốn biết đạo hàm của một hàm, có thể tra giáo trình giải tích hoặc Wikipedia thay vì tự suy ra. Trong thực tế còn dễ hơn: framework có cơ chế **tính đạo hàm tự động**. Ví dụ với TensorFlow:

```
import tensorflow as tf

x = tf.Variable(2.0)
with tf.GradientTape() as tape:                   # ghi lại các phép tính để tính
↪  đạo hàm
    f = x ** 2
print(tape.gradient(f, x).numpy())                # 4.0: đúng với độ dốc của hàm
↪  bình phương tại 2
```

2.2.5 Gradient descent

Có hàm chi phí và biết cách đo độ dốc, ta cần thuật toán tìm tham số làm chi phí nhỏ nhất. Đó là **gradient descent** (giảm theo độ dốc).

2.2.5.1 Ý tưởng

Hình dung chi phí là độ cao của một mặt phía trên mặt phẳng tham số (các trục ngang là trọng số và hệ số điều chỉnh). Cần tìm điểm thấp nhất. Hàm chi phí của hồi quy logistic là **lồi**: mặt có dạng một cái bát duy nhất, không có đáy phụ. Đây là một lý do lớn để chọn hàm chi phí cross-entropy.

Thuật toán làm như sau:

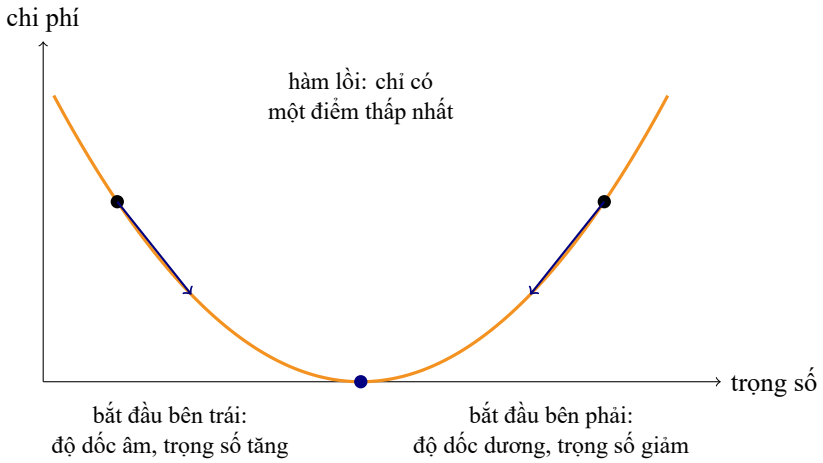
1. **Khởi tạo** trọng số và hệ số điều chỉnh. Với hồi quy logistic gần như cách nào cũng được, thường dùng 0. Vì hàm lồi, khởi tạo ở đâu cũng đi tới cùng một điểm đáy. (Với mạng nhiều lớp, khởi tạo toàn 0 không dùng được; xem mục 2.4.6.)
2. **Bước một bước theo hướng dốc xuống nhanh nhất**: cập nhật mỗi tham số bằng chính nó trừ đi tích của tốc độ học và độ dốc của chi phí theo tham số đó.
3. **Lặp lại** cho đến khi chi phí không giảm nữa.

Trong code, độ dốc theo trọng số đặt tên là `dw`, độ dốc theo hệ số điều chỉnh là `db`, và phép cập nhật có dạng:

```
w = w - alpha * dw         # alpha là tốc độ học (learning rate)
b = b - alpha * db
```

Tốc độ học (learning rate) quyết định mỗi bước dài hay ngắn. Quy tắc này hợp lý vì dấu của độ dốc chỉ đúng hướng cần đi (Hình 2.14):

- Đang ở bên phải đáy: độ dốc dương, trừ đi một số dương nên trọng số giảm, tức đi sang trái.
- Đang ở bên trái đáy: độ dốc âm, trừ đi một số âm nên trọng số tăng, tức đi sang phải.



Hình 2.14: Gradient descent trên hàm chi phí lồi: dấu của độ dốc luôn chỉ về phía đáy

2.2.5.2 Tốc độ học ảnh hưởng thế nào

Để thấy rõ, hãy chạy gradient descent trên một hàm chi phí đơn giản có đáy ở trọng số bằng 3: chi phí là bình phương khoảng cách từ trọng số đến 3, nên độ dốc là gấp đôi khoảng cách đó. Xuất phát từ trọng số bằng 0, chạy 10 bước:

```
def gradient_descent_1d(learning_rate, steps=10, start=0.0):
    w = start
    for _ in range(steps):
        dw = 2 * (w - 3)                    # độ dốc của chi phí (w - 3) bình
        ↪  phương tại w
        w = w - learning_rate * dw
    return w

for lr in [0.01, 0.1, 0.5, 0.95, 1.1]:
    print(lr, round(gradient_descent_1d(lr), 4))
# 0.01 0.5488 | 0.1 2.6779 | 0.5 3.0 | 0.95 1.954 | 1.1 -15.5752
```

Tốc độ học	Trọng số sau 10 bước	Nhận xét
0,01	0,549	quá nhỏ: tiến đúng hướng nhưng rất chậm, còn xa đáy (ở trọng số 3)
0,1	2,678	vừa phải: tiến đều về phía đáy
0,5	3,000	với hàm đặc biệt này, đến đáy ngay sau một bước
0,95	1,954	hơi lớn: nhảy qua lại hai bên đáy (0, 5,7, 0,57, 5,19, …) nên hội tụ chậm
1,1	−15,575	quá lớn: mỗi bước nhảy xa hơn bước trước, **phân kỳ**

Bảng 2.12: Cùng một hàm, cùng số bước, tốc độ học khác nhau cho kết quả rất khác nhau

Đây là lý do tốc độ học là siêu tham số quan trọng bậc nhất: quá nhỏ thì học chậm, quá lớn thì dao động hoặc phân kỳ. Cách chọn được bàn ở mục áp dụng vào dự án (mục 2.2.12).

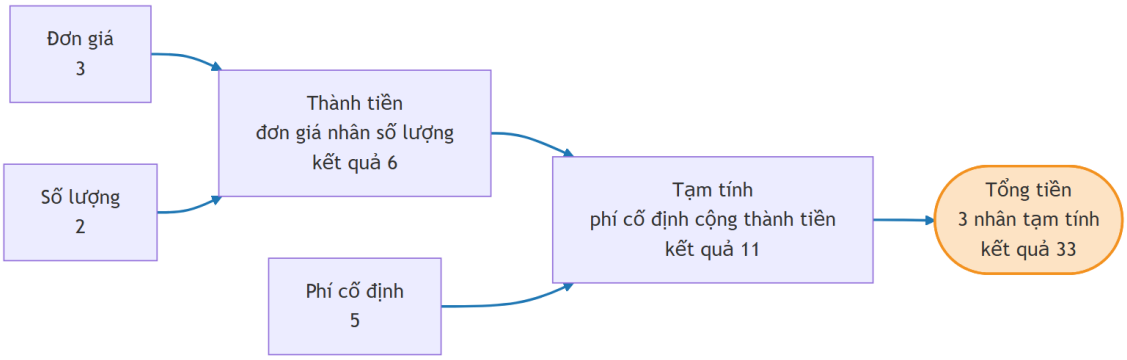
2.2.6 Đồ thị tính toán và lan truyền ngược

Phép tính của mạng nơ-ron luôn được tổ chức thành một lượt **xuôi** (lan truyền xuôi, forward propagation) để tính đầu ra và chi phí, rồi một lượt **ngược** (lan truyền ngược, backward propagation) để tính đạo hàm.

Đồ thị tính toán (computation graph) giải thích vì sao. Ta dùng một ví dụ đời thường, đơn giản hơn hồi quy logistic: tính *tổng tiền* của một đơn hàng. Tổng tiền bằng 3 lần *tạm tính*; tạm tính bằng *phí cố định* cộng *thành tiền*; thành tiền bằng *đơn giá* nhân *số lượng*.

2.2.6.1 Lượt xuôi: tính giá trị

Tính tổng tiền gồm ba bước riêng biệt, mỗi bước là một nút của đồ thị (Hình 2.15). Với phí cố định 5, đơn giá 3, số lượng 2 thì thành tiền là 6, tạm tính là 11 và tổng tiền là 33.

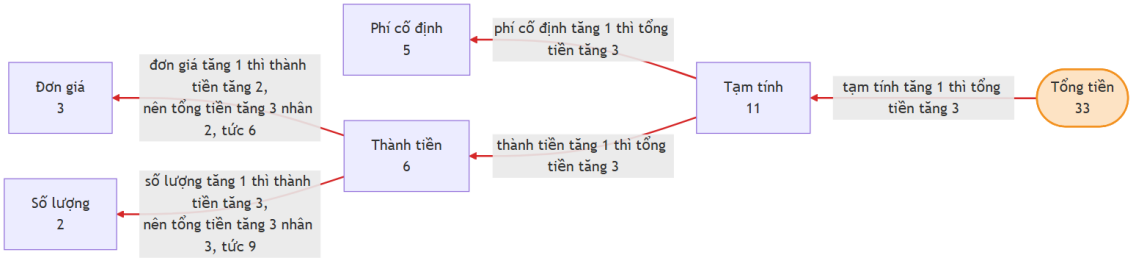


Hình 2.15: Đồ thị tính toán: lượt xuôi (trái sang phải) tính tổng tiền

Đồ thị hữu ích khi có một đầu ra đặc biệt cần tối ưu. Với hồi quy logistic, đó là hàm chi phí cần cực tiểu hoá.

2.2.6.2 Lượt ngược: tính đạo hàm

Giờ hỏi: *nếu tăng một đại lượng lên một chút thì tổng tiền tăng bao nhiêu?* Trả lời cho từng đại lượng, đi từ phải sang trái (Hình 2.16). Mỗi đạo hàm tìm được bằng cách đẩy một đại lượng lên 0,001 rồi xem tổng tiền đổi bao nhiêu (Bảng 2.13).



Hình 2.16: Lượt ngược (phải sang trái): độ dốc đã tính ở nút phía sau được dùng lại cho nút phía trước

Đại lượng	Đẩy lên 0,001 thì chuyện gì xảy ra	Độ dốc
Tạm tính	từ 11 lên 11,001; tổng tiền từ 33 lên 33,003 (gấp 3 lần)	3
Phí cố định	từ 5 lên 5,001; tạm tính lên 11,001; tổng tiền lên 33,003	3
Thành tiền	từ 6 lên 6,001; tạm tính lên 11,001; tổng tiền lên 33,003	3
Đơn giá	từ 3 lên 3,001; thành tiền lên 6,002 (vì số lượng là 2); tạm tính lên 11,002; tổng tiền lên 33,006	6
Số lượng	từ 2 lên 2,001; thành tiền lên 6,003 (vì đơn giá là 3); tạm tính lên 11,003; tổng tiền lên 33,009	9

Bảng 2.13: Độ dốc của tổng tiền theo từng đại lượng, từ phải sang trái

Có thể suy ra độ dốc theo đơn giá mà không cần đẩy thử, nhờ **quy tắc chuỗi** (chain rule): nếu đơn giá ảnh hưởng thành tiền và thành tiền ảnh hưởng tổng tiền, thì mức thay đổi của tổng tiền khi đẩy đơn giá bằng *mức thay đổi của thành tiền khi đẩy đơn giá, nhân với mức thay đổi của tổng tiền khi đẩy thành tiền*. Ở đây là 2 nhân 3, bằng 6. Nhờ vậy, kết quả đã tính ở nút phía sau được dùng lại: độ dốc theo tạm tính giúp tính cho phí cố định và thành tiền, độ dốc theo thành tiền giúp tính cho đơn giá và số lượng. Đó là lý do cách hiệu quả nhất là đi một lượt từ phải sang trái.

Quy ước đặt tên biến. Khi cài đặt, luôn có một đầu ra cuối cùng cần quan tâm (ở đây là tổng tiền, trong thực tế là hàm chi phí). Độ dốc của nó theo một đại lượng bất kỳ được đặt tên bằng tiền tố `d_` cộng tên đại lượng, ví dụ `d_don_gia`. Trong hồi quy logistic, tiền tố là `d` như `dw`, `db`.

```
phi_co_dinh, don_gia, so_luong = 5.0, 3.0, 2.0

# Lượt xuôi
thanh_tien = don_gia * so_luong
tam_tinh = phi_co_dinh + thanh_tien
tong_tien = 3 * tam_tinh
print(thanh_tien, tam_tinh, tong_tien)                # 6.0 11.0 33.0

# Lượt ngược (từ phải sang trái)
d_tam_tinh = 3.0                                      # tổng tiền là 3 lần tạm
↪  tính
d_phi_co_dinh = d_tam_tinh * 1.0                      # tăng phí cố định thì tạm
↪  tính tăng cùng lượng
d_thanh_tien = d_tam_tinh * 1.0
d_don_gia = d_thanh_tien * so_luong                   # tăng đơn giá 1 thì thành
↪  tiền tăng đúng số lượng
d_so_luong = d_thanh_tien * don_gia
print(d_tam_tinh, d_phi_co_dinh, d_thanh_tien, d_don_gia, d_so_luong)    # 3.0
↪  3.0 3.0 6.0 9.0
```

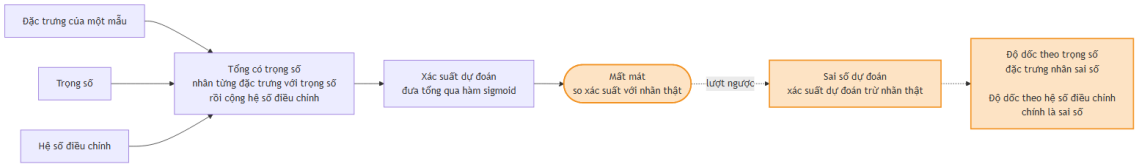
Framework làm đúng việc này tự động. Với cùng ví dụ:

```
phi_co_dinh, don_gia, so_luong = tf.Variable(5.0), tf.Variable(3.0),
↪  tf.Variable(2.0)
with tf.GradientTape() as tape:
    tong_tien = 3 * (phi_co_dinh + don_gia * so_luong)
do_doc = tape.gradient(tong_tien, [phi_co_dinh, don_gia, so_luong])
print([d.numpy() for d in do_doc])                    # [3.0, 6.0, 9.0]
```

2.2.7 Gradient descent cho hồi quy logistic

2.2.7.1 Trên một mẫu

Áp dụng đồ thị tính toán vào hồi quy logistic với một mẫu (Hình 2.17). Lượt xuôi tính tổng có trọng số, rồi xác suất dự đoán bằng sigmoid của tổng đó, rồi mất mát so xác suất với nhãn thật. Lượt ngược đi từ mất mát trở về, qua từng bước:



Hình 2.17: Đồ thị tính toán của hồi quy logistic trên một mẫu và kết quả của lượt ngược

1. Độ dốc theo xác suất dự đoán tính được từ công thức mất mát; người không quen giải tích có thể bỏ qua bước này, vì kết quả cuối rất gọn.
2. Độ dốc theo tổng có trọng số bằng **xác suất dự đoán trừ nhãn thật**. Có thể chứng minh bằng quy tắc chuỗi, nhưng chỉ cần nhớ kết quả: đó là *sai số của dự đoán* (trong code là `dz` cho một mẫu, `dZ` cho cả tập).
3. Độ dốc theo mỗi trọng số bằng *đặc trưng tương ứng nhân sai số*; độ dốc theo hệ số điều chỉnh *chính là* sai số.

Ví dụ số: mẫu có hai đặc trưng (1 và −2), nhãn 1; trọng số ban đầu (1 và 2), hệ số điều chỉnh 1,5; tốc độ học 0,1.

Bước	Kết quả	Ý nghĩa
Tổng có trọng số	−1,5	1 nhân 1, cộng 2 nhân −2, cộng 1,5
Xác suất dự đoán	0,1824	mô hình chỉ cho 18% khả năng có mèo, trong khi nhãn thật là 1
Sai số dự đoán	−0,8176	sai số lớn và âm: cần *tăng* tổng có trọng số
Độ dốc theo hai trọng số	−0,8176 và 1,6351	mỗi đặc trưng đóng góp theo độ lớn (và dấu) của nó
Độ dốc theo hệ số điều chỉnh	−0,8176	bằng sai số
Sau cập nhật	trọng số gần 1,0818 và 1,8365; hệ số gần 1,5818	tổng có trọng số mới lớn hơn −1,5

Bảng 2.14: Một bước gradient descent trên một mẫu

```
x = np.array([[1.0], [-2.0]])      # một mẫu, 2 đặc trưng (vector cột)
y = 1
w = np.array([[1.0], [2.0]])
b = 1.5

z = np.dot(w.T, x) + b             # lượt xuôi
a = sigmoid(z)
```

```
dz = a - y                          # lượt ngược
dw = x * dz
db = dz
print(z.item(), a.item(), dz.item())       # -1.5, khoảng 0.1824, khoảng -0.8176
print(dw.ravel(), db.item())               # khoảng [-0.8176  1.6351], -0.8176

alpha = 0.1
w = w - alpha * dw
b = b - alpha * db
print(w.ravel(), b.item())                 # khoảng [1.0818 1.8365], 1.5818
```

Kết quả khớp với cách đẩy nhẹ từng tham số 0,000001 rồi đo thay đổi của mất mát (cùng ba giá trị −0,8176; 1,6351; −0,8176).

2.2.7.2 Trên nhiều mẫu, bản dùng vòng lặp

Hàm chi phí là trung bình mất mát trên mọi mẫu, nên độ dốc của nó cũng là *trung bình* các độ dốc tính trên từng mẫu. Thuật toán một bước gradient descent dùng vòng lặp:

1. Khởi tạo các **bộ cộng dồn**: chi phí, độ dốc theo trọng số (vector toàn 0) và độ dốc theo hệ số điều chỉnh bằng 0.
2. Lặp qua từng mẫu: tính tổng có trọng số và xác suất dự đoán của mẫu đó; cộng mất mát vào chi phí; tính sai số dự đoán; cộng *đặc trưng nhân sai số* vào độ dốc theo trọng số và cộng sai số vào độ dốc theo hệ số điều chỉnh.
3. Chia ba bộ cộng dồn cho số mẫu để lấy trung bình.
4. Cập nhật trọng số và hệ số điều chỉnh bằng gradient descent.

Phân biệt hai loại đại lượng: tổng có trọng số, xác suất dự đoán và sai số thuộc về *một mẫu*; còn chi phí và hai độ dốc là bộ cộng dồn cho *cả tập*, nên không gắn với mẫu nào.

```
def step_with_loops(w, b, X, Y):
    """Tính chi phí và gradient bằng vòng lặp qua từng mẫu (chậm, chỉ để đối
    ↪  chiếu)."""
    n, m = X.shape
    J, dw, db = 0.0, np.zeros((n, 1)), 0.0
    for i in range(m):
        x_i = X[:, i:i + 1]                                  # cột thứ i, giữ hình
        ↪  dạng (n, 1)
        a_i = sigmoid(np.dot(w.T, x_i) + b)
        J += -(Y[0, i] * np.log(a_i) + (1 - Y[0, i]) * np.log(1 - a_i)).item()
        dz_i = a_i - Y[0, i]
        dw += x_i * dz_i
        db += dz_i.item()
    return J / m, dw / m, db / m
```

Cách này có hai điểm yếu: cần **hai vòng lặp** (qua các mẫu, và qua các đặc trưng nếu viết từng độ dốc riêng lẻ), và vòng lặp tường minh làm chương trình chạy chậm. Với tập dữ liệu lớn của thời học sâu, điều này không chấp nhận được. Giải pháp là vector hoá.

2.2.8 Vector hoá

Vector hoá (vectorization) là nghệ thuật loại bỏ các vòng lặp `for` tường minh trong code. Trong học sâu, dữ liệu thường rất lớn, nên code phải chạy nhanh, nếu không sẽ phải chờ rất lâu mới có kết quả. Vì vậy vector hoá đã trở thành một kỹ năng then chốt.

2.2.8.1 Một phép tính, hai cách

Lấy tích vô hướng của hai vector một triệu phần tử. Cách thứ nhất dùng vòng lặp, cách thứ hai gọi thẳng `np.dot`:

```
import time

a = np.random.rand(1_000_000)
b = np.random.rand(1_000_000)

tic = time.perf_counter()
c = np.dot(a, b)                                  # vector hoá
print("np.dot:", 1000 * (time.perf_counter() - tic), "ms")

tic = time.perf_counter()
c = 0.0
for i in range(1_000_000):                        # vòng lặp tường minh
    c += a[i] * b[i]
print("vòng lặp:", 1000 * (time.perf_counter() - tic), "ms")
```

Hai cách cho cùng một kết quả, nhưng thời gian khác nhau rất xa (Bảng 2.15).

Phép tính	Vòng lặp	Vector hoá	Nhanh hơn
Tích vô hướng, một triệu phần tử	khoảng 240 đến 350 ms	khoảng 1 ms	hơn 250 lần
Một bước gradient descent, 50 đặc trưng, 20.000 mẫu	khoảng 300 ms	khoảng 1,6 ms	gần 190 lần

Bảng 2.15: Thời gian đo trên máy dùng để viết tài liệu này (lấy lần chạy nhanh nhất trong vài lần). Con số tuyệt đối sẽ khác ở máy khác, nhưng chênh lệch cỡ hàng trăm lần thì thường gặp

Nếu mỗi lần huấn luyện chậm hơn 200 lần, chênh lệch là giữa chờ một phút và chờ hơn ba giờ. Nhớ lại mục 2.1.3: thời gian huấn luyện quyết định số ý tưởng ta thử được.

Lý do vector hoá nhanh: cả CPU lẫn GPU đều có các lệnh **song song hoá** (SIMD, một lệnh xử lý nhiều dữ liệu). Khi dùng hàm dựng sẵn như `np.dot`, NumPy tận dụng được chúng; vòng lặp `for` của Python thì không. GPU đặc biệt giỏi loại tính toán này, nhưng CPU cũng làm khá tốt, và ví dụ ở trên chạy hoàn toàn trên CPU.

Quy tắc ngón tay cái: bất cứ khi nào có thể, tránh vòng lặp `for` tường minh. Không phải lúc nào cũng làm được, nhưng nếu có hàm dựng sẵn hoặc cách khác thì thường nhanh hơn.

2.2.8.2 Nhân ma trận và các hàm theo từng phần tử

Nhân một ma trận với một vector cần hai vòng lặp nếu viết tay (qua hàng và qua cột), nhưng chỉ một lệnh `np.dot(A, v)`. Áp một phép toán lên từng phần tử của vector cũng vậy (Bảng 2.16).

Phép tính	Lệnh NumPy
Nhân ma trận với vector	`np.dot(A, v)`
Hàm mũ của từng phần tử	`np.exp(v)`
Logarit của từng phần tử	`np.log(v)`
Giá trị tuyệt đối của từng phần tử	`np.abs(v)`
Lấy max của từng phần tử với 0 (chính là ReLU)	`np.maximum(v, 0)`
Bình phương từng phần tử	`v ** 2`
Nghịch đảo từng phần tử	`1 / v`

Bảng 2.16: Một số hàm NumPy áp dụng theo từng phần tử

Cần phân biệt `np.dot` (nhân ma trận, hoặc tích vô hướng) với `np.multiply` và toán tử * (nhân từng phần tử với nhau).

```
v = np.array([1.0, 2.0, 3.0])
print(np.exp(v).round(3))                       # [ 2.718  7.389 20.086]
print(np.maximum(v - 2, 0))                     # [0. 0. 1.]
print(1 / v)                                    # 1.0, 0.5, 0.3333...

A = np.array([[1, 2], [3, 4], [5, 6]])
print(np.dot(A, np.array([[1], [1]])).ravel()) # [ 3  7 11]
```

Mỗi khi định viết vòng lặp `for`, hãy dừng lại và xem có hàm NumPy dựng sẵn nào làm được việc đó không.

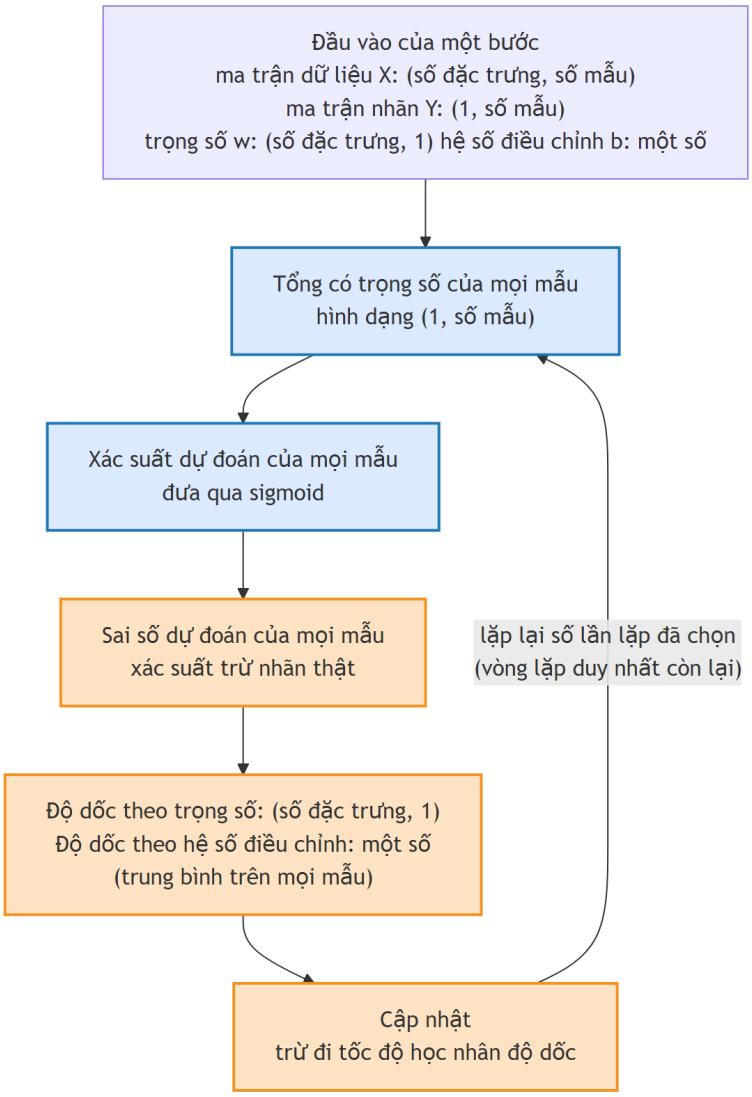
2.2.8.3 Bỏ vòng lặp theo đặc trưng

Vòng lặp thứ hai ở bản cũ chạy qua các đặc trưng (từng độ dốc riêng lẻ). Thay vì khởi tạo từng số, đặt độ dốc theo trọng số là một vector `np.zeros((n, 1))` và cộng cả vector một lần. Hàm `step_with_loops` ở mục trước đã làm đúng như vậy, nên chỉ còn một vòng lặp, qua các mẫu.

2.2.8.4 Bỏ nốt vòng lặp theo mẫu

Nhờ quy ước mỗi mẫu một cột, toàn bộ tập huấn luyện được xử lý cùng lúc (Hình 2.18).

- **Lan truyền xuôi**: nhân trọng số (chuyển vị) với ma trận dữ liệu rồi cộng hệ số điều chỉnh, ra một hàng gồm tổng có trọng số của mọi mẫu; đưa qua sigmoid được hàng xác suất dự đoán. Số thực hệ số điều chỉnh được cộng vào từng phần tử nhờ broadcasting (xem tiếp).
- **Lan truyền ngược**: sai số dự đoán của mọi mẫu bằng hàng xác suất trừ hàng nhãn. Độ dốc theo hệ số điều chỉnh là trung bình của hàng sai số. Độ dốc theo trọng số là ma trận dữ liệu nhân sai số (chuyển vị), chia cho số mẫu; phép nhân này cho đúng tổng các (đặc trưng nhân sai số) trên mọi mẫu, chính là thứ vòng lặp cũ cộng dồn.



Hình 2.18: Một bước gradient descent vector hoá, kèm hình dạng của từng đại lượng

```
def step_vectorized(w, b, X, Y):
    """Chi phí và gradient cho cả tập huấn luyện, không có vòng lặp nào."""
    m = X.shape[1]
    A = sigmoid(np.dot(w.T, X) + b)                                    # (1, m)
    J = np.mean(-(Y * np.log(A) + (1 - Y) * np.log(1 - A)))
    dZ = A - Y                                                         # (1, m)
    dw = np.dot(X, dZ.T) / m                                           # (n, 1)
    db = np.sum(dZ) / m
    return J, dw, db

rng = np.random.default_rng(0)
n, m = 50, 20000
X = rng.standard_normal((n, m))
Y = (rng.random((1, m)) > 0.5).astype(float)
w, b = rng.standard_normal((n, 1)) * 0.1, 0.0

r1 = step_with_loops(w, b, X, Y)                                       # bản vòng lặp
r2 = step_vectorized(w, b, X, Y)                                       # bản vector hoá
print(np.isclose(r1[0], r2[0]), np.allclose(r1[1], r2[1]), np.isclose(r1[2],
↪  r2[2]))   # True True True
```

Hai bản cho cùng kết quả. Để huấn luyện, chỉ còn lại **một vòng lặp ngoài cùng** qua số lần lặp (vòng này không thể bỏ):

```
alpha, num_iterations = 0.01, 1000
for i in range(num_iterations):
    J, dw, db = step_vectorized(w, b, X, Y)
    w = w - alpha * dw
    b = b - alpha * db
```

Đối chiếu với Keras. Lớp `Dense(1, activation="sigmoid")` làm đúng hai bước tính tổng có trọng số và xác suất ở trên, còn `model.fit` lo lan truyền ngược và cập nhật. Chỉ cần nhớ khác biệt về quy ước: Keras nhận ma trận với mẫu ở hàng, nên truyền `X.T` và `Y.T`.

2.2.9 Broadcasting trong NumPy

Broadcasting là cách NumPy xử lý phép toán giữa các mảng khác hình dạng: mảng nhỏ hơn được tự động *sao chép* cho khớp mảng lớn hơn, rồi phép toán áp dụng theo từng phần tử. Đây là điều giúp `np.dot(w.T, X) + b` chạy được dù hệ số điều chỉnh chỉ là một số.

2.2.9.1 Ví dụ: tỉ lệ calo

Bảng dưới là số calo từ chất bột đường, đạm và béo trong 100 gam của bốn thực phẩm (số liệu minh hoạ, không phải số liệu dinh dưỡng chính xác). Cần tính phần trăm calo của từng loại cho mỗi thực phẩm, không dùng vòng lặp: tính tổng từng cột, rồi chia cả ma trận cho tổng đó.

```
A = np.array([[56.0,   0.0,  4.4, 68.0],      # calo từ chất bột đường
              [ 1.2, 104.0, 52.0,  8.0],      # calo từ chất đạm
              [ 1.8, 135.0, 99.0,  0.9]])     # calo từ chất béo; cột: táo, thịt
              ↪  bò, trứng, khoai tây

cal = A.sum(axis=0)                            # axis=0: cộng theo chiều dọc, mỗi
↪  cột một tổng
print(cal)                                     # [ 59.  239.  155.4  76.9]
percentage = 100 * A / cal.reshape(1, 4)       # (3, 4) chia (1, 4): broadcasting
print(percentage.round(1))
# [[94.9  0.   2.8 88.4]
#  [ 2.  43.5 33.5 10.4]
#  [ 3.1 56.5 63.7  1.2]]
```

Táo có 94,9% calo từ chất bột đường, còn thịt bò chủ yếu từ đạm và béo. `axis=0` nghĩa là cộng theo chiều dọc, `axis=1` là theo chiều ngang. Lệnh `reshape` tốn thời gian không đổi và rất rẻ, nên cứ dùng để chắc chắn mảng có đúng hình dạng mong muốn, dù đôi khi dư thừa.

2.2.9.2 Các dạng broadcasting thường gặp

Hình dạng	NumPy làm gì	Ví dụ kết quả
mảng với một số thực	sao chép số đó cho khớp	`[1,2,3,4] + 100` cho `[101,102,103,104]`
(hàng, cột) với (1, cột)	sao chép dòng đơn đó cho mọi hàng	cộng `[100,200,300]` vào mỗi hàng
(hàng, cột) với (hàng, 1)	sao chép cột đơn đó cho mọi cột	cộng 100 vào hàng đầu, 200 vào hàng hai
(hàng, 1) với (1, cột)	cả hai cùng được mở rộng	ra ma trận (hàng, cột)

Bảng 2.17: Quy tắc broadcasting thường dùng cho phép cộng, trừ, nhân, chia

```
M = np.array([[1, 2, 3], [4, 5, 6]])
print(M + np.array([[100, 200, 300]]))        # [[101 202 303], [104 205 306]]
print(M + np.array([[100], [200]]))           # [[101 102 103], [204 205 206]]
```

Tài liệu NumPy có định nghĩa tổng quát hơn nữa, nhưng các dạng trên là những dạng cần dùng khi cài đặt mạng nơ-ron. Broadcasting tiện lợi nhưng cũng là nguồn lỗi tinh vi, như mục sau cho thấy.

2.2.10 Tránh lỗi về chiều của mảng

Sự linh hoạt của NumPy là điểm mạnh (một dòng code làm được nhiều việc) nhưng cũng là điểm yếu: có thể sinh ra lỗi lạ, khó tìm nếu không nắm rõ cách broadcasting hoạt động. Ví dụ, cộng một vector cột với một vector hàng, ta chờ lỗi không khớp chiều, nhưng NumPy lại trả về một *ma trận*:

```
col = np.array([[1], [2], [3]])      # (3, 1)
row = np.array([[10, 20, 30]])       # (1, 3)
print((col + row).shape)             # (3, 3): không báo lỗi, mà ra ma trận
```

2.2.10.1 Mảng hạng 1

Một nguồn lỗi hay gặp là **mảng hạng 1** (rank 1 array). `np.random.randn(5)` tạo mảng có hình dạng `(5,)`: không phải vector hàng cũng không phải vector cột. Hành vi của nó khó đoán: chuyển vị không đổi gì, và tích của nó với chính nó chuyển vị ra *một số* chứ không phải ma trận.

Cách tạo	Hình dạng	Chuyển vị	Tích với chính nó chuyển vị
`np.random.randn(5)`	`(5,)`	`(5,)`	một số (không nên dùng)
`np.random.randn(5, 1)`	`(5, 1)`	`(1, 5)`	ma trận `(5, 5)` (tích ngoài)
`np.random.randn(1, 5)`	`(1, 5)`	`(5, 1)`	ma trận `(1, 1)`

Bảng 2.18: Mảng hạng 1 so với vector cột và vector hàng

```
a = np.random.randn(5)               # mảng hạng 1: hình dạng (5,)
print(a.shape, a.T.shape)            # (5,) (5,): chuyển vị không đổi gì
print(np.dot(a, a.T).shape)          # (): ra một số

a = np.random.randn(5, 1)            # vector cột rõ ràng
print(a.shape, a.T.shape)            # (5, 1) (1, 5)
print(np.dot(a, a.T).shape)          # (5, 5): tích ngoài

assert a.shape == (5, 1)             # kiểm tra chiều: rẻ, và đóng vai trò tài
↪  liệu cho code
```

```
a = np.random.randn(5).reshape(5, 1)  # lỡ có mảng hạng 1 thì đổi lại bằng
↪  reshape
```

Bốn thói quen giúp giảm lỗi chiều:

1. **Không dùng mảng hạng 1**: luôn tạo vector cột `(n, 1)` hoặc vector hàng `(1, n)`.
2. **Dùng `assert`** để kiểm tra hình dạng ở những chỗ không chắc; chi phí gần như bằng 0.
3. **Dùng `reshape`** thoải mái để đưa mảng về đúng hình dạng.
4. **In `.shape`** khi gặp kết quả lạ, và đối chiếu với Bảng 2.8.

2.2.11 Đọc thêm: vì sao dùng cross-entropy

Mục này tuỳ chọn, giải thích hàm mất mát ở mục 2.2.3 không phải ngẫu nhiên mà xuất phát từ nguyên lý **ước lượng hợp lý cực đại** (maximum likelihood).

1. **Xác suất mô hình gán cho nhãn thật của một mẫu**: nếu nhãn là 1, đó là xác suất dự đoán; nếu nhãn là 0, đó là 1 trừ xác suất dự đoán. (Hai trường hợp có thể gộp lại thành một biểu thức duy nhất, nhưng bảng dưới đã đủ ý.)
2. **Xác suất của cả tập nhãn**: nếu các mẫu độc lập và cùng phân phối, đó là *tích* xác suất của từng mẫu.
3. **Ước lượng hợp lý cực đại**: chọn tham số làm xác suất của dữ liệu quan sát lớn nhất. Tích khó làm việc, nên lấy logarit (hàm tăng, nên vị trí cực đại không đổi): tích biến thành tổng.
4. **Kết quả**: logarit xác suất của nhãn thật đúng bằng *trừ mất mát cross-entropy*. Cực đại hoá logarit xác suất tương đương cực tiểu hoá tổng mất mát. Chia thêm cho số mẫu cho dễ co giãn, ta được hàm chi phí.

Mẫu	Nhãn	Xác suất dự đoán	Xác suất của nhãn thật	Mất mát
1	1	0,9	0,9	0,105
2	0	0,2	0,8	0,223
3	1	0,7	0,7	0,357
Cả tập			tích bằng 0,504	tổng 0,685

Bảng 2.19: Ba mẫu: tích xác suất bằng 0,504, còn tổng mất mát là trừ logarit của tích đó (0,685)

```
p = np.array([0.9, 0.8, 0.7])                  # xác suất mô hình gán cho nhãn
↪  thật của 3 mẫu
print(np.prod(p), np.exp(np.sum(np.log(p))))  # 0.504 0.504: tích và exp(tổng
↪  log) giống nhau
print(-np.sum(np.log(p)))                      # 0.685...: tổng mất mát

p = np.full(3000, 0.7)                         # 3000 mẫu, mỗi mẫu xác suất 0,7
print(np.prod(p))                              # 5e-324: tích tràn dưới, mất hết
↪  thông tin
print(np.sum(np.log(p)))                       # -1070.02...: tổng logarit vẫn
↪  dùng được
```

Ví dụ cuối cho thấy một lý do thực tế để dùng logarit: tích của nhiều xác suất nhỏ nhanh chóng vượt khỏi khả năng biểu diễn của số thực, còn tổng logarit thì ổn định.

2.2.12 Áp dụng vào dự án thực tế

2.2.12.1 Quy trình chung

1. **Tìm hiểu dữ liệu**: số mẫu, số đặc trưng, tỉ lệ từng nhãn; in `.shape` của mọi mảng.
2. **Tiền xử lý**: đưa mỗi mẫu thành một vector (một cột), chuẩn hoá về cùng thang đo (ảnh thì chia cho 255).
3. **Khởi tạo** trọng số và hệ số điều chỉnh bằng 0.
4. **Lặp**: tính chi phí và độ dốc bằng bản vector hoá, rồi cập nhật tham số.
5. **Đánh giá**: so độ chính xác trên tập huấn luyện và tập kiểm tra, và so với một baseline đơn giản (ví dụ luôn dự đoán nhãn phổ biến nhất).

2.2.12.2 Chọn tốc độ học và số lần lặp

Quan sát đường chi phí	Việc nên làm
Giảm đều qua các lần lặp	tốc độ học hợp lý; nếu vẫn đang giảm, tăng số lần lặp
Dao động lên xuống, hoặc tăng	tốc độ học quá lớn: giảm đi 3 đến 10 lần
Giảm rất chậm, gần như phẳng	tốc độ học quá nhỏ: tăng lên (ví dụ nhân 3 đến 10 lần)
Chi phí thấp nhưng độ chính xác kiểm tra thấp hơn nhiều so với huấn luyện	quá khớp: chi phí thấp hơn chưa chắc là mô hình tốt hơn; cần thêm dữ liệu hoặc kỹ thuật chống quá khớp (sẽ học ở chương sau)

Bảng 2.20: Đọc đường chi phí để chỉnh siêu tham số. Gợi ý bắt đầu thử tốc độ học trong 0,1; 0,01; 0,001

2.2.12.3 Hiểu lầm và lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Chi phí tăng hoặc nhảy loạn	tốc độ học quá lớn (Bảng 2.12): giảm tốc độ học
Chi phí thành `nan` hoặc `inf`	dự đoán đúng bằng 0 hoặc 1 làm logarit thành vô cùng (sigmoid của 40 đã bằng 1,0 trong số thực); dùng hàm mất mát của framework, hoặc chuẩn hoá đầu vào để tổng có trọng số không quá lớn
Chi phí thành `nan` ngay sau vài lần lặp, kể cả với tốc độ học rất nhỏ	quên chia ảnh cho 255: đặc trưng lên tới 255 làm tổng có trọng số rất lớn, sigmoid bão hoà và logarit thành vô cùng (đã thử với ảnh mèo, tốc độ học 0,0001)
Lỗi nhân ma trận về chiều	xếp mẫu theo hàng thay vì theo cột, hoặc quên chuyển vị trọng số (`np.dot(w.T, X)`); kiểm tra bằng Bảng 2.8
Kết quả lạ, không báo lỗi	mảng hạng 1 hoặc broadcasting ngoài ý muốn (mục 2.2.10)
Độ chính xác kiểm tra cao nhưng mô hình không học được gì	tập kiểm tra mất cân bằng nhãn: so với baseline luôn đoán nhãn phổ biến
Huấn luyện quá chậm	còn vòng lặp `for` qua mẫu; chuyển sang bản vector hoá
Chi phí giảm nhưng không tìm được giá trị nhỏ nhất	đã dừng quá sớm: tăng số lần lặp, kiểm tra đường chi phí

Bảng 2.21: Lỗi thường gặp khi cài đặt hồi quy logistic

2.2.12.4 Khi nào dùng hồi quy logistic

Hồi quy logistic là bộ phân loại *tuyến tính*: nó vẽ một ranh giới thẳng giữa hai lớp trong không gian đặc trưng. Nó hợp làm baseline cho phân loại nhị phân, hoạt động tốt khi dữ liệu ít và mối quan hệ gần tuyến tính. Với ảnh thô, nó khá hạn chế (xem kết quả ở mục 2.3.2); các kiến trúc nhiều lớp và CNN sẽ làm tốt hơn nhiều.

Khái niệm	NumPy tự viết	Framework
Một neuron có sigmoid	`sigmoid(np.dot(w.T, X) + b)`	Keras: `Dense(1, activation="sigmoid")`
Hàm mất mát	tự viết `loss`, `cost`	Keras: `binary_crossentropy`
Độ dốc	tự viết `dZ = A - Y`, `dw`, `db`	`tf.GradientTape` hoặc `model.fit` tự tính
Cập nhật tham số	`w = w - alpha * dw`	Keras: `SGD(learning_rate=alpha)`
Xếp dữ liệu	mẫu ở cột	Keras, scikit-learn: mẫu ở hàng
Thư viện sẵn có cho hồi quy logistic		scikit-learn: `LogisticRegression`

Bảng 2.22: Đối chiếu bản tự viết bằng NumPy và các thư viện

2.2.13 Tóm tắt cần nhớ

1. **Phân loại nhị phân**: nhãn là 0 hoặc 1. Ảnh được *duỗi* thành vector; **mỗi mẫu là một cột** của ma trận dữ liệu, nhãn xếp thành một hàng.
2. **Hồi quy logistic** là một neuron có sigmoid: tổng có trọng số cộng hệ số điều chỉnh, đưa qua sigmoid để ra xác suất nhãn bằng 1, rồi so với ngưỡng 0,5.
3. **Mất mát** (một mẫu) và **chi phí** (trung bình trên cả tập). Dùng cross-entropy nhị phân, vì cho bài toán lồi; nó phạt rất nặng khi mô hình tự tin mà sai.
4. **Đạo hàm là độ dốc**. Có thể ước lượng bằng cách đẩy nhẹ, tra bảng, hoặc để framework tự tính.
5. **Gradient descent**: lặp lại cập nhật tham số bằng chính nó trừ tích của tốc độ học và độ dốc. Tốc độ học quá nhỏ thì chậm, quá lớn thì dao động hoặc phân kỳ.
6. **Đồ thị tính toán**: lượt xuôi tính giá trị, lượt ngược (phải sang trái) tính độ dốc nhờ quy tắc chuỗi, và dùng lại kết quả đã tính ở nút sau.
7. **Với hồi quy logistic**: sai số dự đoán là xác suất trừ nhãn thật; độ dốc theo trọng số là đặc trưng nhân sai số; độ dốc theo hệ số điều chỉnh là chính sai số; cho cả tập thì lấy trung bình.
8. **Vector hoá**: bỏ vòng lặp `for` bằng các phép toán trên cả ma trận. Một bước gradient descent chỉ gồm năm phép tính trên ma trận; nhanh hơn cỡ hàng trăm lần so với vòng lặp.
9. **Broadcasting** tự mở rộng mảng nhỏ cho khớp mảng lớn. **Tránh mảng hạng 1**, dùng `assert` và `reshape` để giữ chiều đúng.

2.3 Thực hành: các hàm NumPy cơ bản và nhận dạng ảnh có mèo

Mục này gồm hai ví dụ thực hành dùng lại những gì đã học ở mục 2.2. Ví dụ thứ nhất viết các hàm NumPy nhỏ thường gặp trong học sâu. Ví dụ thứ hai ghép chúng lại thành một bộ phân loại ảnh có mèo hoàn chỉnh bằng hồi quy logistic, rồi làm lại bằng Keras để đối chiếu.

2.3.1 Thực hành 1: các hàm NumPy cơ bản cho học sâu

Bài toán: viết các hàm nhỏ mà mọi mô hình học sâu đều cần: sigmoid và độ dốc của nó, duỗi ảnh thành vector, chuẩn hoá theo hàng, softmax, và hai hàm mất mát đơn giản. Mỗi hàm chỉ vài dòng NumPy, không dùng vòng lặp.

Hàm	Làm gì	Ví dụ kết quả
`basic_sigmoid(x)`	sigmoid của một số thực, dùng `math.exp`	`basic_sigmoid(1)` khoảng 0,7311
`sigmoid(x)`	sigmoid của số hoặc mảng bất kỳ, dùng `np.exp`	với `[1, 2, 3]`: khoảng `[0,731 ; 0,881 ; 0,953]`
`sigmoid_derivative(x)`	độ dốc của sigmoid: kích hoạt nhân (1 trừ kích hoạt)	với `[1, 2, 3]`: khoảng `[0,197 ; 0,105 ; 0,045]`
`image2vector(image)`	duỗi ảnh (dài, cao, 3) thành cột	ảnh `(3, 3, 2)` thành `(18, 1)`
`normalize_rows(x)`	chia mỗi hàng cho độ dài của hàng đó	hàng `[0, 3, 4]` thành `[0 ; 0,6 ; 0,8]`
`softmax(x)`	biến mỗi hàng thành phân phối xác suất	mỗi hàng có tổng bằng 1
`L1`, `L2`	tổng sai lệch tuyệt đối, tổng sai lệch bình phương	1,1 và 0,43 trên ví dụ bên dưới

Bảng 2.23: Các hàm NumPy cơ bản trong ví dụ thực hành

```
import math
import numpy as np

def basic_sigmoid(x):
    """Sigmoid của một số thực (dùng thư viện math)."""
    return 1 / (1 + math.exp(-x))

def sigmoid(x):
    """Sigmoid của số thực hoặc mảng NumPy bất kỳ."""
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

def image2vector(image):
    return image.reshape(-1, 1)                        # duỗi thành một cột; -1:
    ↪  NumPy tự tính số hàng

def normalize_rows(x):
    x_norm = np.linalg.norm(x, ord=2, axis=1, keepdims=True)   # độ dài từng
    ↪  hàng, hình dạng (số hàng, 1)
    return x / x_norm                                          # broadcasting:
    ↪  mỗi hàng chia cho độ dài của nó

def softmax(x):
    x_exp = np.exp(x)
    return x_exp / np.sum(x_exp, axis=1, keepdims=True)        # chia mỗi hàng
    ↪  cho tổng của hàng đó

def L1(yhat, y):
    return np.sum(np.abs(y - yhat))

def L2(yhat, y):
    return np.sum((y - yhat) ** 2)

print(basic_sigmoid(1))                                  # 0.7310585786300049
print(sigmoid(np.array([1, 2, 3])))                      # [0.73105858 0.88079708
↪  0.95257413]
```

```
print(sigmoid_derivative(np.array([1, 2, 3])))        # [0.19661193 0.10499359
↪  0.04517666]
print(image2vector(np.zeros((3, 3, 2))).shape)        # (18, 1)
print(normalize_rows(np.array([[0., 3., 4.], [1., 6., 4.]])))
# [[0.         0.6        0.8       ]
#  [0.13736056 0.82416338 0.54944226]]
s = softmax(np.array([[9, 2, 5, 0, 0], [7, 5, 0, 0, 0]]))
print(s.sum(axis=1))                                  # [1. 1.]: mỗi hàng là một
↪  phân phối xác suất
yhat = np.array([.9, 0.2, 0.1, .4, .9])
y = np.array([1, 0, 0, 1, 1])
print(L1(yhat, y), L2(yhat, y))                       # 1.1 0.43 (làm tròn)
```

2.3.1.1 Những điểm cần lưu ý

- **`math` chỉ nhận số thực, NumPy nhận cả mảng.** Gọi `basic_sigmoid([1, 2, 3])` báo lỗi vì đầu vào là danh sách; `sigmoid(np.array([1, 2, 3]))` chạy bình thường. Đó là lý do học sâu dùng NumPy.
- **`keepdims=True`** giữ lại chiều đã gộp, nên `x_norm` có hình dạng `(số hàng, 1)` và chia được cho ma trận ban đầu nhờ broadcasting.
- **Softmax có thể tràn số.** Với đầu vào rất lớn, `np.exp` cho vô cùng và kết quả thành `nan`. Cách khắc phục là trừ đi giá trị lớn nhất của hàng trước khi lấy mũ; kết quả không đổi về mặt toán học.

```
print(softmax(np.array([[1000., 1000.]])))            # [[nan nan]]: np.exp(1000)
↪  tràn số

def stable_softmax(x):
    x_exp = np.exp(x - np.max(x, axis=1, keepdims=True))      # trừ max của hàng
    ↪  để tránh tràn số
    return x_exp / np.sum(x_exp, axis=1, keepdims=True)

print(stable_softmax(np.array([[1000., 1000.]])))     # [[0.5 0.5]]
```

2.3.1.2 Vector hoá: so sánh vòng lặp và hàm dựng sẵn

Với bốn phép toán thường gặp (tích vô hướng, tích ngoài, nhân từng phần tử, nhân ma trận với vector), mỗi phép đều có một hàm NumPy tương ứng:

```
x1 = [9, 2, 5, 0, 0, 7, 5, 0, 0, 0, 9, 2, 5, 0, 0]
x2 = [9, 2, 2, 9, 0, 9, 2, 5, 0, 0, 9, 2, 5, 0, 0]
W = np.random.rand(3, len(x1))

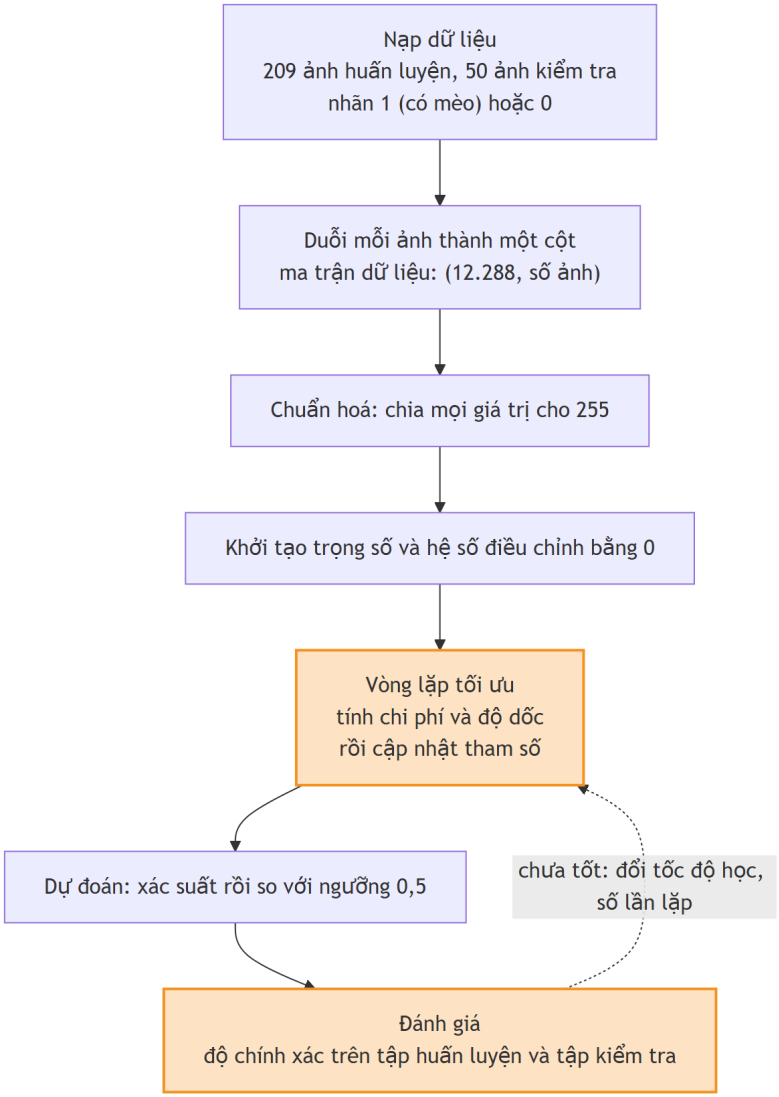
print(np.dot(x1, x2))                    # 278: tích vô hướng
print(np.outer(x1, x2).shape)            # (15, 15): tích ngoài
print(np.multiply(x1, x2)[:5])           # [81  4 10  0  0]: nhân từng phần tử
print(np.dot(W, x1).shape)               # (3,): nhân ma trận với vector
```

Với mảng nhỏ như trên (15 phần tử), thời gian chạy của vòng lặp và của hàm dựng sẵn khác nhau không đáng kể, thậm chí hàm dựng sẵn có thể chậm hơn chút vì chi phí gọi hàm. Lợi ích chỉ lộ rõ khi mảng lớn, như ở Bảng 2.15. Dù vậy, bản vector hoá luôn gọn và rõ ràng hơn.

2.3.2 Thực hành 2: nhận dạng ảnh có mèo bằng hồi quy logistic

2.3.2.1 Bài toán

Cho một bộ ảnh màu cỡ 64 nhân 64 điểm ảnh, mỗi ảnh được gán nhãn 1 (có mèo) hoặc 0 (không có mèo). Xây một thuật toán nhận dạng ảnh đơn giản phân biệt hai loại này, theo đúng quy trình của một mạng nơ-ron: định nghĩa cấu trúc, khởi tạo tham số, lặp (lan truyền xuôi, lan truyền ngược, cập nhật), rồi dự đoán (Hình 2.19). Dữ liệu gồm hai tệp `.h5`, đặt trong thư mục `datasets`.



Hình 2.19: Quy trình huấn luyện bộ phân loại ảnh có mèo

2.3.2.2 Nạp và chuẩn bị dữ liệu

Tập	Số ảnh	Hình dạng ảnh gốc	Tỉ lệ ảnh có mèo
Huấn luyện	209	`(209, 64, 64, 3)`	khoảng 34%
Kiểm tra	50	`(50, 64, 64, 3)`	66%

Bảng 2.24: Bộ dữ liệu ảnh có mèo và không có mèo

Tỉ lệ nhãn của hai tập khác nhau nhiều: tập huấn luyện chủ yếu là ảnh không có mèo, còn tập kiểm tra chủ yếu là ảnh có mèo. Điều này ảnh hưởng đến cách đọc kết quả ở phần sau.

```
import numpy as np
import h5py

def load_dataset():
    with h5py.File("datasets/train_catvnoncat.h5", "r") as f:
        train_x = np.array(f["train_set_x"][:])
        train_y = np.array(f["train_set_y"][:])
    with h5py.File("datasets/test_catvnoncat.h5", "r") as f:
        test_x = np.array(f["test_set_x"][:])
        test_y = np.array(f["test_set_y"][:])
        classes = np.array(f["list_classes"][:])
    return train_x, train_y.reshape(1, -1), test_x, test_y.reshape(1, -1),
    ↪  classes

train_x_orig, train_y, test_x_orig, test_y, classes = load_dataset()
m_train, m_test, num_px = train_x_orig.shape[0], test_x_orig.shape[0],
↪  train_x_orig.shape[1]
print(train_x_orig.shape, train_y.shape)         # (209, 64, 64, 3) (1, 209)
print(test_x_orig.shape, test_y.shape)           # (50, 64, 64, 3) (1, 50)
print(train_y.mean(), test_y.mean())             # khoảng 0.344 và 0.66: tỉ lệ
↪  ảnh có mèo

train_x = train_x_orig.reshape(m_train, -1).T / 255.     # mỗi ảnh một cột, giá
↪  trị trong [0, 1]
test_x = test_x_orig.reshape(m_test, -1).T / 255.
print(train_x.shape, test_x.shape)               # (12288, 209) (12288, 50)
```

2.3.2.3 Xây từng phần rồi ghép lại

Cấu trúc code giống với quy trình ở Hình 2.19: mỗi bước một hàm, cuối cùng ghép lại thành `model`. Hàm `propagate` chính là bản vector hoá của mục 2.2.8.

```
import copy

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def initialize_with_zeros(dim):
    return np.zeros((dim, 1)), 0.0                 # w là vector cột toàn 0, b
    ↪  bằng 0

def propagate(w, b, X, Y):
    """Chi phí và gradient cho cả tập (lan truyền xuôi rồi ngược)."""
    m = X.shape[1]
    A = sigmoid(np.dot(w.T, X) + b)
    cost = -np.mean(Y * np.log(A) + (1 - Y) * np.log(1 - A))
    dZ = A - Y
    dw = np.dot(X, dZ.T) / m
    db = np.sum(dZ) / m
    return {"dw": dw, "db": db}, float(cost)

def optimize(w, b, X, Y, num_iterations=2000, learning_rate=0.005):
    """Gradient descent; ghi lại chi phí mỗi 100 lần lặp để vẽ đường học."""
    w, b = copy.deepcopy(w), copy.deepcopy(b)
    costs = []
```

```
    for i in range(num_iterations):
        grads, cost = propagate(w, b, X, Y)
        w = w - learning_rate * grads["dw"]
        b = b - learning_rate * grads["db"]
        if i % 100 == 0:
            costs.append(cost)
    return {"w": w, "b": b}, costs

def predict(w, b, X):
    A = sigmoid(np.dot(w.T, X) + b)
    return (A > 0.5).astype(float)                  # ngưỡng 0,5

def model(X_train, Y_train, X_test, Y_test, num_iterations=2000,
↪   learning_rate=0.005):
    w, b = initialize_with_zeros(X_train.shape[0])
    params, costs = optimize(w, b, X_train, Y_train, num_iterations,
    ↪   learning_rate)
    pred_train = predict(params["w"], params["b"], X_train)
    pred_test = predict(params["w"], params["b"], X_test)
    return {"w": params["w"], "b": params["b"], "costs": costs,
            "train_acc": 100 - np.mean(np.abs(pred_train - Y_train)) * 100,
            "test_acc": 100 - np.mean(np.abs(pred_test - Y_test)) * 100}
```

Trước khi huấn luyện, nên kiểm tra `propagate` trên một ví dụ nhỏ có đáp án biết trước (hai đặc trưng, ba mẫu):

```
w = np.array([[1.], [2.]])
b = 1.5
X = np.array([[1., -2., -1.], [3., 0.5, -3.2]])
Y = np.array([[1, 1, 0]])
grads, cost = propagate(w, b, X, Y)
print(grads["dw"].ravel(), grads["db"], cost)    # [ 0.2507 -0.0660] -0.1250
↪   0.1590 (làm tròn)
```

2.3.2.4 Huấn luyện và đọc kết quả

```
d = model(train_x, train_y, test_x, test_y, num_iterations=2000,
↪   learning_rate=0.005)
print(d["costs"][0], d["costs"][-1])             # khoảng 0.6931 và 0.1409
print(round(d["train_acc"], 2), round(d["test_acc"], 2))     # 99.04 70.0
```

Trên máy dùng để viết tài liệu, 2000 lần lặp chạy trong khoảng 7 giây trên CPU nhờ bản vector hoá. Kết quả:

Lần lặp	0	100	200	500	1000	1500	1900
Chi phí	0,6931	0,5845	0,4669	0,3033	0,2148	0,1665	0,1409

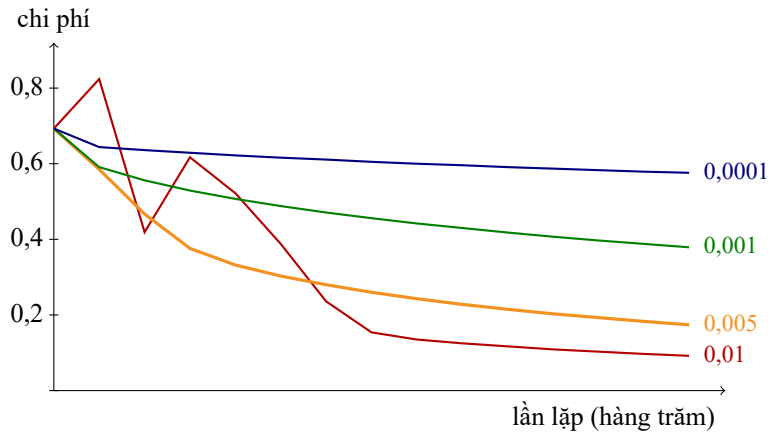
Bảng 2.25: Chi phí giảm dần khi huấn luyện (tốc độ học 0,005)

- **Chi phí đầu tiên đúng bằng 0,6931**: với trọng số và hệ số điều chỉnh bằng 0, mô hình đoán 0,5 cho mọi ảnh, và đó là mất mát của việc không biết gì. Chi phí giảm dần cho thấy tham số đang được học.
- **Độ chính xác huấn luyện khoảng 99%**: mô hình đủ khả năng khớp dữ liệu huấn luyện. Đây là một phép kiểm tra tính đúng đắn hữu ích.

- **Độ chính xác kiểm tra 70%**: thấp hơn nhiều, và có 15 trong 50 ảnh bị đoán sai. Đây là dấu hiệu của **quá khớp** (overfitting): mỗi ảnh có 12.288 đặc trưng, mô hình có 12.289 tham số nhưng chỉ có 209 ảnh để học, nên nó có thể "học thuộc" tập huấn luyện thay vì học điều tổng quát.
- **Cần so với baseline**: tập kiểm tra có 66% ảnh có mèo, nên một mô hình luôn đoán "có mèo" đã đạt 66%. Kết quả 70% chỉ hơn mức đó vài điểm. Với hồi quy logistic trên ảnh thô và tập dữ liệu nhỏ như vậy thì chấp nhận được, nhưng đừng đọc con số 70% một cách riêng lẻ.

2.3.2.5 Chọn tốc độ học

So sánh bốn tốc độ học với cùng số lần lặp (Hình 2.20, Bảng 2.26).



Hình 2.20: Đường chi phí theo tốc độ học (nhãn bên phải): 0,01 dao động ở những lần lặp đầu, 0,0001 giảm rất chậm

```
for lr in [0.01, 0.001, 0.0001]:
    r = model(train_x, train_y, test_x, test_y, num_iterations=1500,
    ↪  learning_rate=lr)
    print(lr, round(r["train_acc"], 2), round(r["test_acc"], 2))
# 0.01 99.52 68.0 | 0.001 89.0 64.0 | 0.0001 68.42 36.0
```

Tốc độ học	Chi phí ghi lần cuối	Chính xác huấn luyện	Chính xác kiểm tra	Nhận xét
0,01	0,092	99,5%	68%	dao động ở những lần lặp đầu (0,69 lên 0,82 rồi xuống) trước khi ổn định
0,005	0,174	97,6%	70%	giảm đều, mượt
0,001	0,379	89,0%	64%	giảm đều nhưng chậm: chưa học xong sau 1500 lần
0,0001	0,576	68,4%	36%	quá chậm: gần như chưa học được gì

Bảng 2.26: Ảnh hưởng của tốc độ học sau 1500 lần lặp (chi phí được ghi mỗi 100 lần, nên giá trị cuối là tại lần lặp thứ 1400)

Hai điểm cần rút ra. Thứ nhất, tốc độ học quá lớn làm chi phí dao động, quá nhỏ thì học chậm. Thứ hai, **chi phí thấp hơn chưa chắc là mô hình tốt hơn**: tốc độ 0,01 cho chi phí thấp nhất nhưng độ chính xác kiểm tra (68%) không hơn tốc độ 0,005 (70%), vì mô hình đã bắt đầu khớp quá mức. Nên chọn tốc độ học làm chi phí giảm tốt, và dùng kỹ thuật chống quá khớp (sẽ học ở chương sau) nếu độ chính xác huấn luyện cao hơn nhiều so với kiểm tra.

2.3.2.6 Huấn luyện lâu hơn

```
for it in [500, 2000, 5000, 10000]:                # mất khoảng nửa phút cho cả
↪  vòng lặp
    r = model(train_x, train_y, test_x, test_y, num_iterations=it,
    ↪  learning_rate=0.005)
    print(it, round(r["train_acc"], 2), round(r["test_acc"], 2))
# 500 92.34 74.0 | 2000 99.04 70.0 | 5000 100.0 70.0 | 10000 100.0 70.0
```

Số lần lặp	500	2000	5000	10000
Chính xác huấn luyện	92,3%	99,0%	100%	100%
Chính xác kiểm tra	74%	70%	70%	70%

Bảng 2.27: Huấn luyện lâu hơn đưa độ chính xác huấn luyện lên 100% nhưng không cải thiện kiểm tra

Huấn luyện lâu hơn chỉ làm tập huấn luyện khớp hơn, độ chính xác kiểm tra không tăng. Cần thận trọng khi so 74% với 70%: tập kiểm tra chỉ có 50 ảnh, mỗi ảnh đóng góp 2 điểm phần trăm, nên chênh lệch vài ảnh chưa đủ để kết luận chắc.

2.3.2.7 Thử với ảnh riêng

Để dự đoán một ảnh mới, cần *đúng cùng cách tiền xử lý* như lúc huấn luyện: đổi kích thước về 64 nhân 64, chia cho 255, rồi duỗi thành một cột.

```
from PIL import Image
```

```
image = np.array(Image.open("images/my_image.jpg").convert("RGB").resize((num_px,
↪  num_px)))
x = (image / 255.).reshape(-1, 1)                          # cùng tiền xử lý như
↪  dữ liệu huấn luyện
prob = sigmoid(np.dot(d["w"].T, x) + d["b"]).item()
print(round(prob, 3), classes[int(prob > 0.5)].decode("utf-8"))   # xác suất có
↪  mèo và lớp được đoán
```

2.3.2.8 Làm bằng Keras

Toàn bộ mô hình chỉ là một lớp `Dense` với một neuron và hàm sigmoid. Để kết quả giống bản NumPy, khởi tạo trọng số bằng 0 và dùng gradient descent thuần (`SGD` không động lượng) với cả tập trong một lô. Lưu ý Keras cần mẫu ở *hàng*, nên truyền `.T`:

```
from tensorflow import keras
from tensorflow.keras import layers

keras.utils.set_random_seed(0)
kmodel = keras.Sequential([
    keras.Input(shape=(12288,)),                                          # mỗi
    ↪  mẫu là một hàng 12.288 giá trị
    layers.Dense(1, activation="sigmoid", kernel_initializer="zeros"),    # đúng
    ↪  một neuron
])
kmodel.compile(optimizer=keras.optimizers.SGD(learning_rate=0.005),
               loss="binary_crossentropy", metrics=["accuracy"])
kmodel.fit(train_x.T, train_y.T, epochs=2000, batch_size=train_x.shape[1],
↪  verbose=0)   # mỗi epoch là một bước trên cả tập
print(kmodel.evaluate(train_x.T, train_y.T, verbose=0))    # khoảng [0.1356,
↪  0.9904]
print(kmodel.evaluate(test_x.T, test_y.T, verbose=0))      # khoảng [0.7504, 0.7]
```

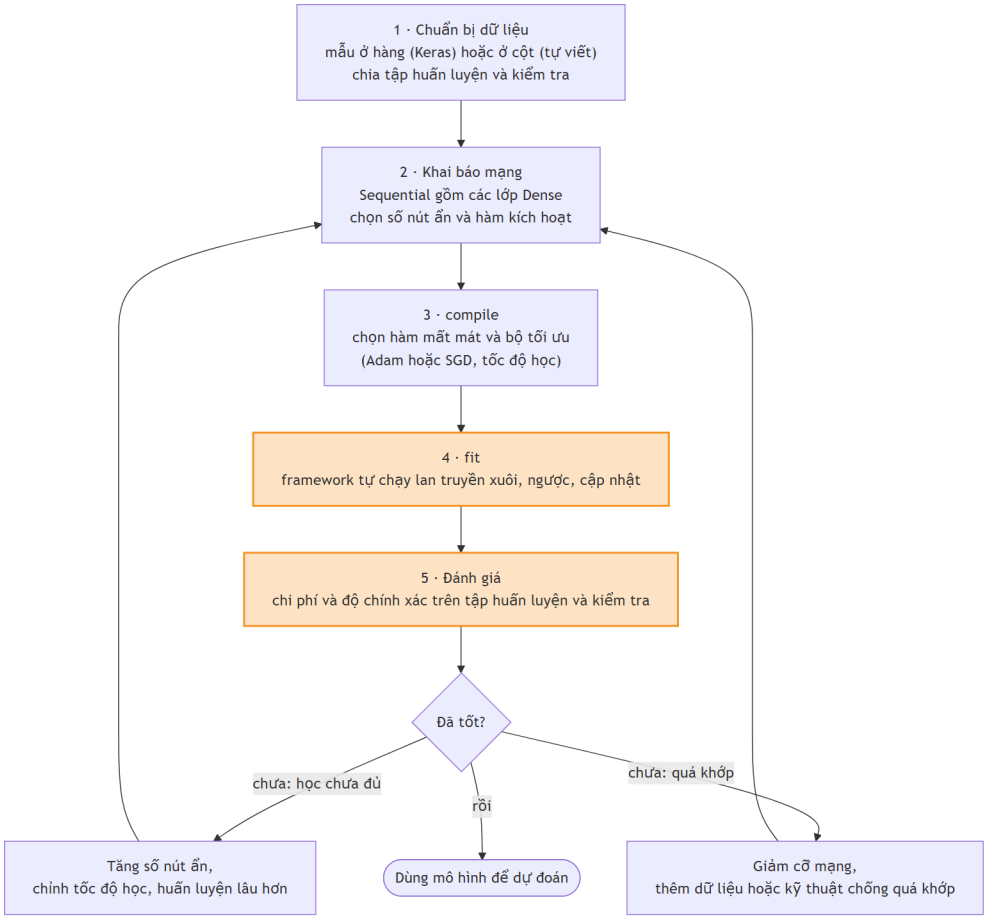
Kết quả trùng với bản NumPy: độ chính xác huấn luyện 99,04% và kiểm tra 70%, mô hình có 12.289 tham số (12.288 trọng số và một hệ số điều chỉnh). Khi bỏ `kernel_initializer="zeros"` và dùng khởi tạo mặc định của Keras, độ chính xác cũng cho ra y hệt trong lần chạy thử; riêng giá trị mất mát trên tập kiểm tra hơi khác (khoảng 0,78 so với 0,75). Với hồi quy logistic, cách khởi tạo ít ảnh hưởng vì bài toán lồi.

2.3.2.9 Rút ra

- Tiền xử lý dữ liệu quan trọng: tìm hiểu hình dạng, duỗi mỗi ảnh thành một cột, chuẩn hoá bằng cách chia cho 255.
- Xây từng hàm riêng (khởi tạo, `propagate`, `optimize`, `predict`) rồi ghép thành `model` giúp dễ kiểm tra từng phần.
- Tốc độ học là một **siêu tham số**; chỉnh nó tạo ra khác biệt lớn. Sẽ còn nhiều ví dụ về siêu tham số ở các chương sau.
- Độ chính xác huấn luyện cao với độ chính xác kiểm tra thấp là quá khớp; luôn so với baseline.
- Framework cho kết quả tương đương chỉ với vài dòng, nhưng hiểu bản NumPy giúp biết framework đang làm gì.

2.4 Mạng nơ-ron một lớp ẩn

Hồi quy logistic ở mục 2.2 chỉ vẽ được một ranh giới thẳng giữa hai lớp. Mục này nâng cấp nó thành **mạng nơ-ron một lớp ẩn**: ý tưởng là xếp chồng nhiều neuron sigmoid, để mô hình học được ranh giới cong và các mẫu phức tạp. Cách trình bày đi từ ý tưởng đến cách dùng thư viện trước, rồi mới mở hộp xem bên trong:

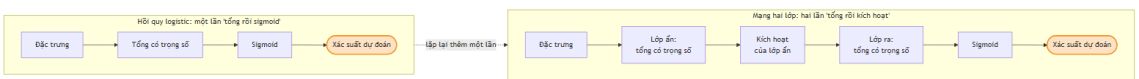


Hình 2.21: Quy trình xây một mạng nơ-ron với Keras. Phần còn lại của mục đi qua từng bước

Lộ trình: từ hồi quy logistic đến mạng nơ-ron → cấu trúc của mạng → mạng tính đầu ra như thế nào → hàm kích hoạt → huấn luyện bằng lan truyền ngược → khởi tạo tham số → áp dụng vào dự án. Ví dụ code ưu tiên Keras; các đoạn NumPy chỉ để thấy rõ phép tính bên trong.

2.4.1 Từ hồi quy logistic đến mạng nơ-ron

Một neuron trong hồi quy logistic làm hai việc: lấy tổng có trọng số của các đặc trưng, rồi đưa qua hàm sigmoid để ra xác suất. **Mạng nơ-ron chỉ là việc làm đúng như vậy nhiều lần**: nhiều neuron chạy song song tạo thành một *lớp*; đầu ra của lớp này trở thành đầu vào của lớp kế tiếp (Hình 2.22).



Hình 2.22: Mạng hai lớp là hồi quy logistic được lặp lại hai lần

Trong Keras, điều đó chính xác là hai lớp `Dense` xếp nối tiếp. Mạng dưới đây nhận 3 đặc trưng, có

một lớp ẩn gồm 4 nút dùng hàm kích hoạt tanh, và một nút đầu ra dùng sigmoid:

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    keras.Input(shape=(3,)),                    # 3 đặc trưng đầu vào
    layers.Dense(4, activation="tanh"),         # lớp ẩn: 4 nút
    layers.Dense(1, activation="sigmoid"),      # lớp ra: 1 nút cho xác suất
])
model.summary()                                 # tổng cộng 21 tham số: 16 ở
↪  lớp ẩn, 5 ở lớp ra
```

Con số 21 được tính như sau: mỗi nút ẩn có 3 trọng số (một cho mỗi đặc trưng) và 1 hệ số điều chỉnh, nên 4 nút có 16 tham số; nút ra có 4 trọng số (một cho mỗi nút ẩn) và 1 hệ số điều chỉnh, tổng cộng 5.

	Hồi quy logistic	**Mạng một lớp ẩn**
Số lần "tổng có trọng số rồi kích hoạt"	một	hai (lớp ẩn, rồi lớp ra)
Ranh giới giữa hai lớp	thẳng	cong, tuỳ độ lớn của lớp ẩn
Kích hoạt của lớp ẩn	không có	tanh, ReLU, …
Lan truyền ngược	một bước lùi	hai bước lùi
Khởi tạo tham số	có thể bằng 0	phải ngẫu nhiên (mục 2.4.6)

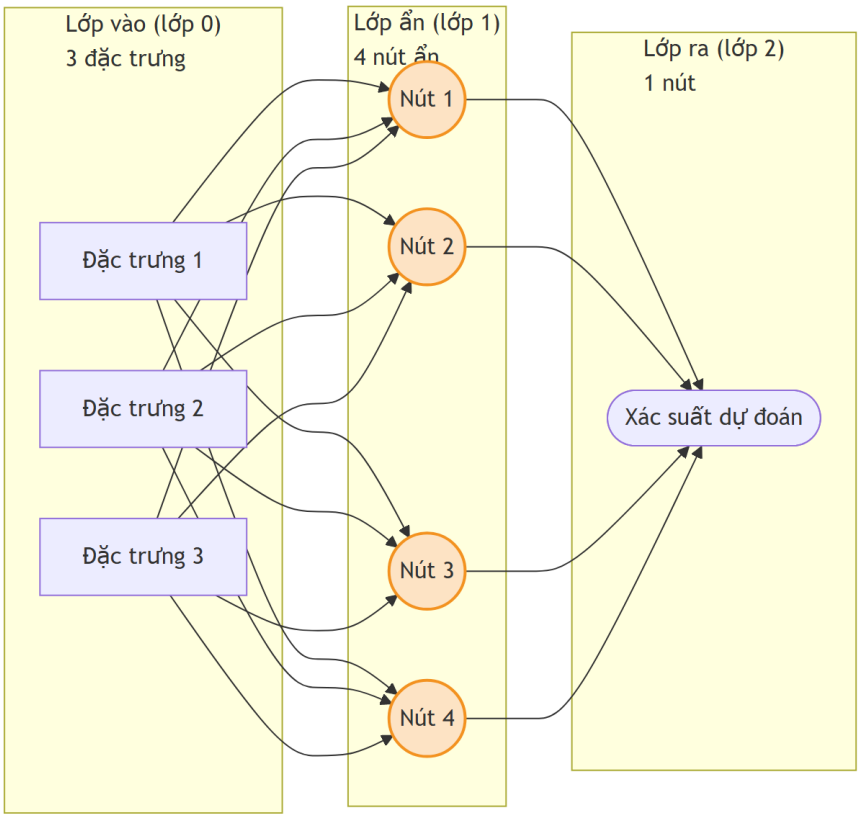
Bảng 2.28: Hồi quy logistic và mạng một lớp ẩn

2.4.2 Cấu trúc của mạng

2.4.2.1 Lớp vào, lớp ẩn, lớp ra

Mạng trong Hình 2.23 gồm ba loại lớp:

- **Lớp vào** (input layer): các đặc trưng của một mẫu, xếp dọc. Nó không tính toán gì, chỉ chuyển dữ liệu đi tiếp.
- **Lớp ẩn** (hidden layer): các nút ở giữa, mỗi nút là một neuron nhận *mọi* đặc trưng của lớp trước.
- **Lớp ra** (output layer): ở đây chỉ một nút, cho ra dự đoán của mạng.



Hình 2.23: Mạng hai lớp: 3 đặc trưng, 4 nút ẩn, 1 nút ra. Mọi nút của lớp trước nối với mọi nút của lớp sau

Lớp ẩn có tên như vậy vì trong tập huấn luyện ta chỉ thấy *đầu vào* và *đầu ra mong muốn*; giá trị đúng của các nút ở giữa không bao giờ xuất hiện trong dữ liệu. Mạng tự quyết định chúng biểu diễn điều gì.

Cách đếm lớp. Mạng trên được gọi là **mạng hai lớp** (hay mạng một lớp ẩn), vì quy ước *không đếm lớp vào*: lớp ẩn là lớp một, lớp ra là lớp hai, còn lớp vào là lớp không. Hầu hết bài báo và tài liệu dùng cách đếm này. Mạng giá nhà ở mục 2.1 cũng là một mạng hai lớp theo cách đếm đó.

2.4.2.2 Kích hoạt và tham số của từng lớp

Kích hoạt (activation) là tên chung cho những giá trị mà một lớp truyền cho lớp kế tiếp. Lớp vào chuyển dữ liệu nguyên bản (kích hoạt của lớp không); lớp ẩn sinh ra một vector 4 chiều (mỗi nút một giá trị); lớp ra sinh ra một số duy nhất, chính là xác suất dự đoán. Lớp ẩn và lớp ra mỗi lớp có tham số riêng: một ma trận *trọng số* và một vector *hệ số điều chỉnh*. Để văn bản không phải dùng ký hiệu, Bảng 2.29 gán mỗi đại lượng một tên mô tả, kèm tên biến trong code.

Tên gọi trong văn bản	Tên trong code	Hình dạng (3 đặc trưng, 4 nút ẩn)
Ma trận dữ liệu (cũng là kích hoạt của lớp vào)	`X`	`(3, số mẫu)`
Trọng số của lớp ẩn	`W1`	`(4, 3)`
Hệ số điều chỉnh của lớp ẩn	`b1`	`(4, 1)`
Tổng có trọng số của lớp ẩn	`Z1`	`(4, số mẫu)`
Kích hoạt của lớp ẩn	`A1`	`(4, số mẫu)`
Trọng số của lớp ra	`W2`	`(1, 4)`
Hệ số điều chỉnh của lớp ra	`b2`	`(1, 1)`
Tổng có trọng số của lớp ra	`Z2`	`(1, số mẫu)`
Đầu ra của mạng (xác suất dự đoán)	`A2`	`(1, số mẫu)`

Bảng 2.29: Tên gọi trong văn bản và tên biến trong code

Quy tắc dễ nhớ cho hình dạng của ma trận trọng số: **số hàng bằng số nút của chính lớp đó, số cột bằng số nút của lớp liền trước**. Lớp ẩn có 4 nút và nhận 3 đặc trưng nên có hình dạng (4, 3); lớp ra có 1 nút và nhận 4 nút ẩn nên có hình dạng (1, 4). Cách đọc này khớp với quy ước "mỗi mẫu một cột" đã dùng từ mục 2.2.1.

Tên mô tả dùng trong văn bản này còn phân biệt hai loại chỉ số mà tài liệu khác hay ghi bằng ngoặc: *lớp* (lớp ẩn hay lớp ra, nằm trong tên đại lượng) và *mẫu* (mẫu nào trong tập huấn luyện, nằm ở số cột của ma trận).

Lưu ý khi dùng Keras. Keras lưu trọng số của mỗi lớp *chuyển vị* so với quy ước ở trên: ma trận có số hàng bằng số đầu vào và số cột bằng số nút của lớp, vì Keras xếp mẫu theo hàng. Hệ số điều chỉnh được lưu dưới dạng vector phẳng.

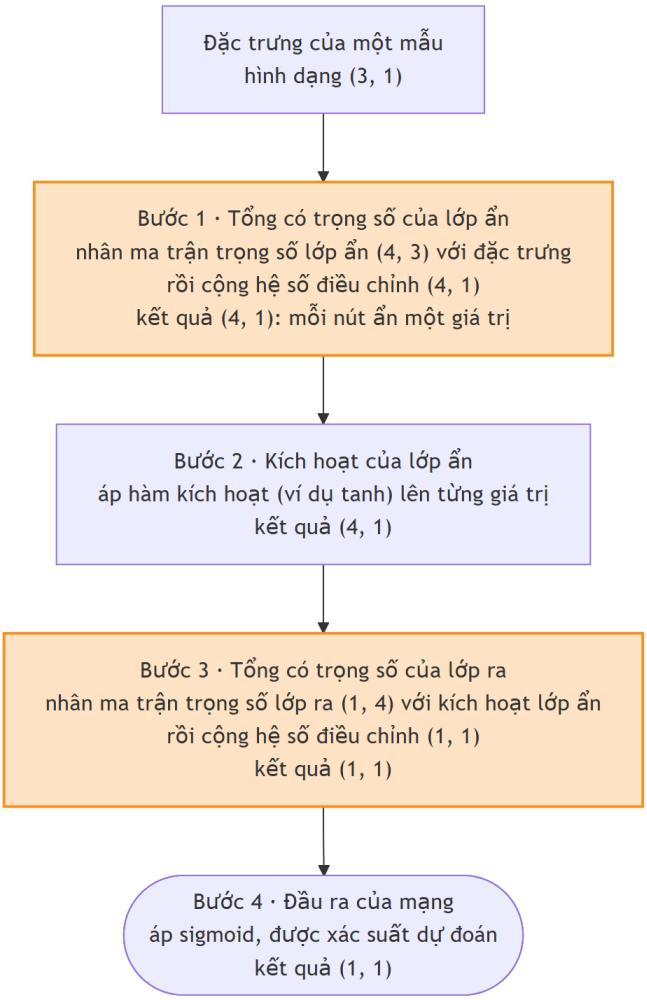
```
for layer in model.layers:
    w, b = layer.get_weights()
    print(w.shape, b.shape)          # (3, 4) (4,)  rồi  (4, 1) (1,)
```

2.4.3 Mạng tính đầu ra như thế nào

2.4.3.1 Một mẫu: bốn bước

Mỗi nút ẩn là một neuron hồi quy logistic có vector trọng số riêng. Thay vì tính từng nút một, xếp các vector trọng số của bốn nút thành *các hàng* của một ma trận 4 nhân 3. Nhân ma trận đó với đặc trưng của mẫu, một phép nhân cho ra tổng có trọng số của cả bốn nút cùng lúc. Quy tắc ngắn gọn: **các nút khác nhau trong cùng một lớp được xếp dọc thành một vector**.

Toàn bộ phép tính của mạng gồm bốn bước, cùng hình dạng ở từng bước (Hình 2.24):



Hình 2.24: Bốn bước tính đầu ra của mạng cho một mẫu, kèm hình dạng của từng kết quả

Hai bước đầu giống hệt hồi quy logistic cho từng nút ẩn, hai bước sau là một hồi quy logistic nữa, nhưng nhận kích hoạt của lớp ẩn làm đầu vào.

```
import numpy as np

rng = np.random.default_rng(0)
W1, b1 = rng.standard_normal((4, 3)) * 0.5, np.zeros((4, 1))   # lớp ẩn: 4 nút, 3
↪  đặc trưng
W2, b2 = rng.standard_normal((1, 4)) * 0.5, np.zeros((1, 1))   # lớp ra: 1 nút
x = np.array([[1.0], [-2.0], [0.5]])                           # một mẫu, 3 đặc
↪  trưng (vector cột)

Z1 = np.dot(W1, x) + b1                  # bước 1: tổng có trọng số của lớp ẩn
A1 = np.tanh(Z1)                         # bước 2: kích hoạt của lớp ẩn
Z2 = np.dot(W2, A1) + b2                 # bước 3: tổng có trọng số của lớp ra
A2 = 1 / (1 + np.exp(-Z2))               # bước 4: xác suất dự đoán
print(Z1.shape, A1.shape, Z2.shape, A2.shape)      # (4, 1) (4, 1) (1, 1) (1, 1)

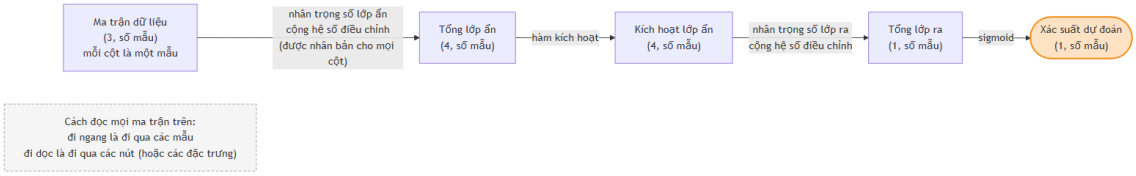
# Tính từng nút ẩn bằng vòng lặp để thấy phép nhân ma trận làm đúng việc đó
Z1_tung_nut = np.array([[np.dot(W1[i], x[:, 0]) + b1[i, 0]] for i in range(4)])
print(np.allclose(Z1_tung_nut, Z1))               # True
```

2.4.3.2 Nhiều mẫu cùng lúc

Với nhiều mẫu, không cần vòng lặp: xếp các mẫu thành các cột của ma trận dữ liệu rồi dùng *đúng bốn dòng đó* (Hình 2.25). Mọi ma trận kết quả đều có một cột cho mỗi mẫu. Cách đọc chúng:

- **đi ngang** (từ trái sang phải) là đi qua các *mẫu huấn luyện*;
- **đi dọc** (từ trên xuống dưới) là đi qua các *nút* của lớp (hoặc các đặc trưng, với ma trận dữ liệu).

Ví dụ, phần tử ở góc trên bên trái của kích hoạt lớp ẩn là kích hoạt của nút ẩn thứ nhất với mẫu đầu tiên; đi xuống một ô là sang nút ẩn thứ hai cho cùng mẫu đó; đi sang phải một ô là cùng nút ẩn thứ nhất nhưng với mẫu kế tiếp.



Hình 2.25: Vector hoá qua nhiều mẫu: cùng bốn bước, mỗi ma trận có một cột cho mỗi mẫu

Vì sao cách làm này đúng. Chọn tạm hệ số điều chỉnh bằng 0 cho dễ thấy. Nhân trọng số của lớp ẩn với từng mẫu riêng cho ra một vector cột. Khi các mẫu xếp cạnh nhau thành ma trận dữ liệu, theo cách nhân ma trận, *cột thứ i của kết quả đúng bằng trọng số nhân mẫu thứ i*, nên kết quả là các vector cột đó xếp cạnh nhau. Khi có hệ số điều chỉnh khác 0, broadcasting cộng nó vào từng cột. Lập luận tương tự áp dụng cho ba bước còn lại: xếp đầu vào theo cột thì đầu ra cũng xếp theo cột.

```
X = rng.standard_normal((3, 5))                    # 5 mẫu, mỗi mẫu một cột

Z1_all = np.dot(W1, X) + b1                        # (4, 5): cả 5 mẫu cùng lúc
A1_all = np.tanh(Z1_all)
Z2_all = np.dot(W2, A1_all) + b2                   # (1, 5)
A2_all = 1 / (1 + np.exp(-Z2_all))
print(Z1_all.shape, A2_all.shape)                  # (4, 5) (1, 5)

# Cột thứ 3 của kết quả chung đúng bằng kết quả khi tính riêng mẫu thứ 3
mau_3 = X[:, [2]]
print(np.allclose(Z1_all[:, [2]], np.dot(W1, mau_3) + b1))     # True
```

Một hệ quả đẹp: vì ma trận dữ liệu chính là kích hoạt của lớp vào, hai cặp phương trình của hai lớp có cùng dạng, chỉ khác lớp đầu vào của chúng. Các lớp của mạng *làm cùng một việc lặp đi lặp lại*. Mạng sâu hơn (chương sau) chỉ là lặp hai bước "tổng có trọng số rồi kích hoạt" nhiều lần hơn.

Đối chiếu với Keras. Đặt đúng các trọng số vừa dùng vào mô hình Keras (nhớ chuyển vị như đã nói) rồi cho dự đoán. Kết quả trùng khít với phép tính bằng tay:

```
km = keras.Sequential([keras.Input(shape=(3,)),
                       layers.Dense(4, activation="tanh"),
                       layers.Dense(1, activation="sigmoid")])
km.set_weights([W1.T, b1.ravel(), W2.T, b2.ravel()])            # Keras lưu
↪  trọng số chuyển vị
print(np.allclose(km.predict(X.T, verbose=0).T, A2_all, atol=1e-6))   # True
↪  (Keras nhận mẫu ở hàng)
```

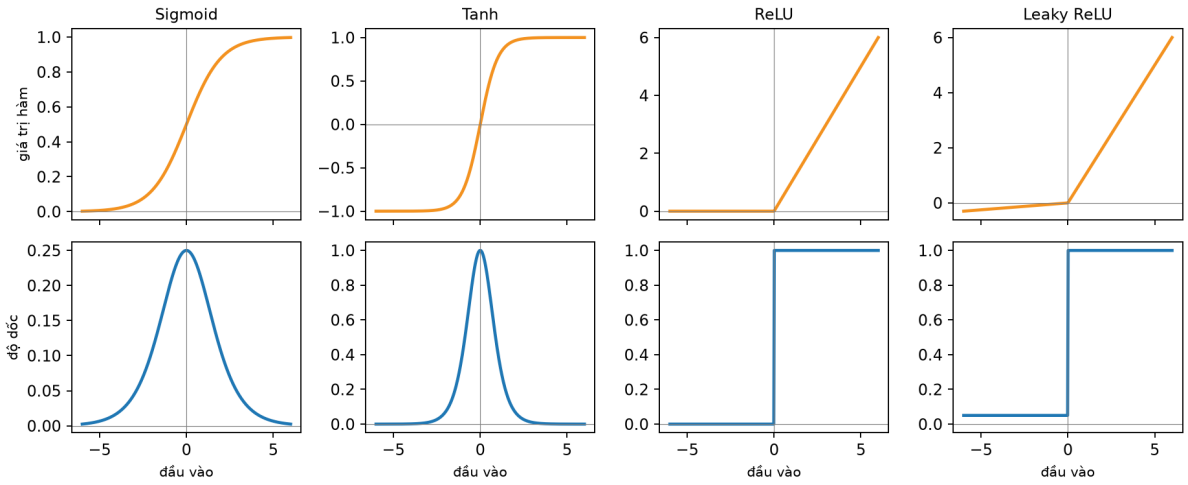
2.4.4 Hàm kích hoạt

Ở mỗi nút, sau bước "tổng có trọng số" là một **hàm kích hoạt** (activation function) biến tổng đó thành giá trị đưa đi tiếp. Cho đến giờ ta chỉ dùng sigmoid, nhưng đó thường không phải lựa chọn tốt nhất. Mỗi lớp có thể dùng một hàm kích hoạt riêng; ví dụ lớp ẩn dùng tanh còn lớp ra dùng sigmoid.

2.4.4.1 Bốn hàm thường gặp

Hàm	Làm gì	Khoảng giá trị	Độ dốc
Sigmoid	nén mọi số về đường cong chữ S	từ 0 đến 1	tối đa 0,25 ở giữa, gần 0 ở hai đầu; bằng kích hoạt nhân (1 trừ kích hoạt)
Tanh	giống sigmoid nhưng đi qua điểm gốc, cân đối quanh 0	từ −1 đến 1	tối đa 1 ở giữa, gần 0 ở hai đầu; bằng 1 trừ bình phương kích hoạt
ReLU	giữ nguyên số dương, đặt số âm về 0	từ 0 trở lên	1 với số dương, 0 với số âm
Leaky ReLU	giữ nguyên số dương, nhân số âm với một hệ số nhỏ như 0,01	mọi số	1 với số dương, hệ số nhỏ với số âm

Bảng 2.30: Bốn hàm kích hoạt thường dùng và độ dốc của chúng

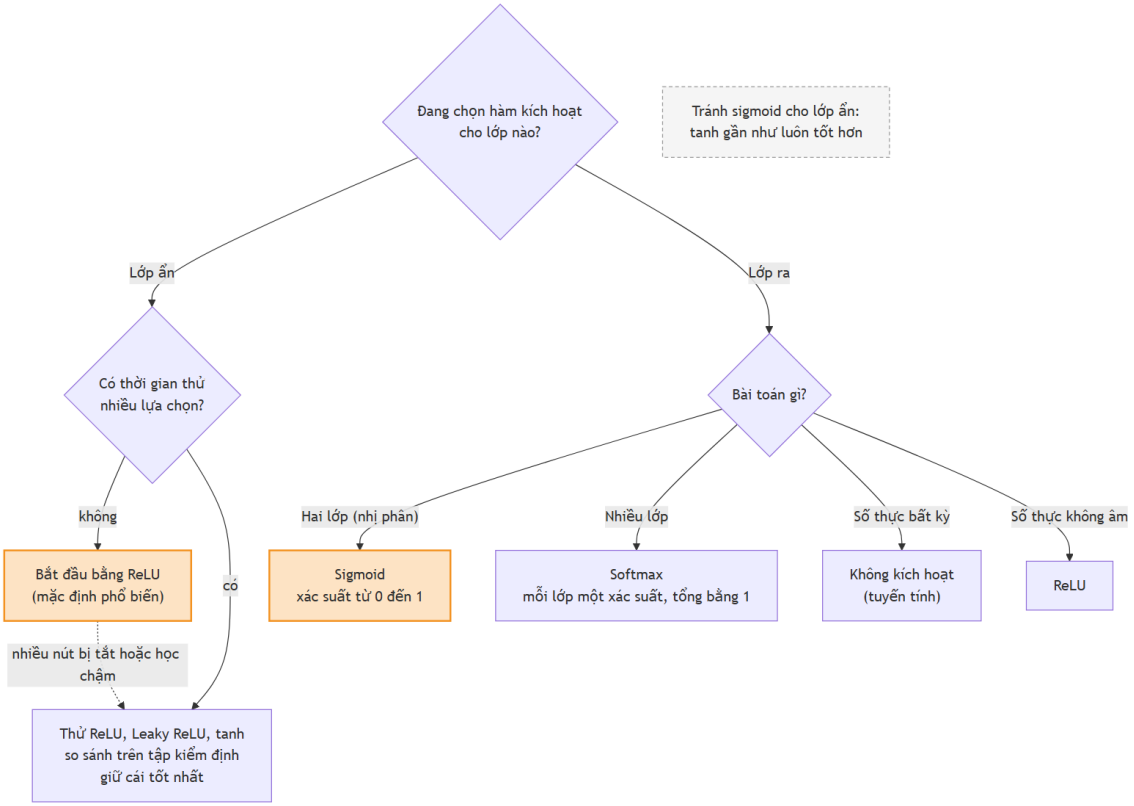


Hình 2.26: Bốn hàm kích hoạt (hàng trên) và độ dốc của chúng (hàng dưới). Sigmoid và tanh gần như phẳng ở hai đầu, nên độ dốc về 0

2.4.4.2 Chọn hàm nào

- **Tanh gần như luôn tốt hơn sigmoid cho lớp ẩn.** Giá trị của tanh nằm giữa −1 và 1, nên trung bình kích hoạt của lớp ẩn gần 0. Điều này giống việc chuẩn hoá dữ liệu cho trung bình về 0, và giúp lớp kế tiếp học dễ hơn.
- **Ngoại lệ duy nhất của sigmoid** là lớp ra của bài toán hai lớp: nhãn là 0 hoặc 1 nên dự đoán cần nằm giữa 0 và 1.
- **Nhược điểm chung của sigmoid và tanh**: khi tổng có trọng số rất lớn hoặc rất nhỏ, độ dốc gần 0 và gradient descent cập nhật rất chậm.

- **ReLU và Leaky ReLU** giữ độ dốc khác 0 trên phần lớn miền giá trị, nên thường học nhanh hơn. ReLU là lựa chọn mặc định phổ biến cho mạng nhiều lớp. Dù nửa miền âm của ReLU có độ dốc 0, thực tế vẫn đủ nút có tổng dương cho phần lớn mẫu.
- **Khó biết trước hàm nào tốt nhất cho bài toán cụ thể**, nên cách đáng tin là thử nhiều hàm và so trên tập kiểm định.



Hình 2.27: Chọn hàm kích hoạt theo vị trí của lớp và loại bài toán

Bảng 2.31 là một thí nghiệm nhỏ trên dữ liệu hình bông hoa ở mục 2.5: cùng mạng 8 nút ẩn, cùng tốc độ học 0,5, chỉ đổi hàm kích hoạt của lớp ẩn, lấy trung bình ba lần khởi tạo khác nhau.

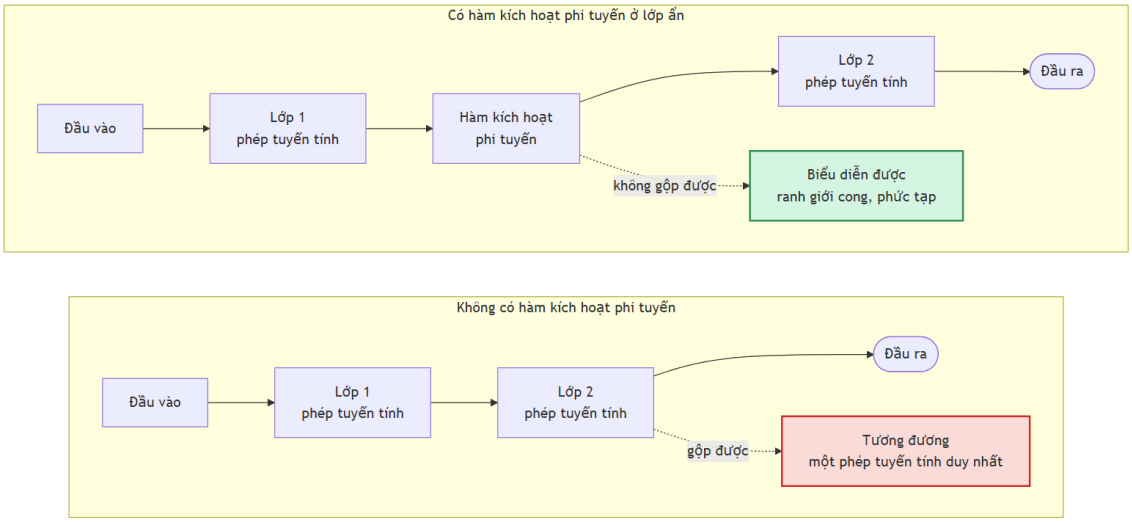
Hàm của lớp ẩn	Chi phí sau 100 lần lặp	Sau 1000 lần	Sau 5000 lần	Chính xác
Tanh	0,457	0,289	0,190	90,2%
ReLU	0,602	0,461	0,450	78,6%
Leaky ReLU	0,604	0,471	0,440	78,7%
Sigmoid	0,622	0,350	0,281	88,3%
Tuyến tính (không kích hoạt)	0,673	0,673	0,673	47,0%

Bảng 2.31: Hàm kích hoạt của lớp ẩn ảnh hưởng đến tốc độ học và độ chính xác (dữ liệu hình bông hoa, 8 nút ẩn)

Đọc kết quả một cách thận trọng: trên bài toán nhỏ, hai chiều và mạng chỉ 8 nút này, **tanh thắng** chứ không phải ReLU, và sigmoid tuy chậm nhưng không tệ. Một cách giải thích hợp lý (chưa kiểm chứng ở đây) là ReLU tạo ranh giới gồm các đoạn thẳng nên cần nhiều nút hơn để uốn cong mượt. Thông điệp thực hành không phải "ReLU luôn tốt hơn" mà là: lời khuyên chung chỉ là điểm bắt đầu, còn quyết định cuối cùng nên dựa trên thí nghiệm với dữ liệu của bạn. Dòng cuối của bảng cho thấy lý do ta cần hàm kích hoạt phi tuyến, được giải thích ngay dưới đây.

2.4.4.3 Vì sao cần hàm kích hoạt phi tuyến

Nếu bỏ hàm kích hoạt (hoặc dùng hàm tuyến tính, còn gọi là hàm đồng nhất vì nó trả về nguyên đầu vào) thì mạng chỉ tính một hàm tuyến tính của đầu vào. Lý do: nhân hai phép biến đổi tuyến tính liên tiếp vẫn là một phép biến đổi tuyến tính. Dù mạng có bao nhiêu lớp ẩn, nó cũng gộp lại thành đúng một phép tuyến tính, nghĩa là *không cần lớp ẩn nào cả* (Hình 2.28). Kết quả 47% ở dòng cuối của Bảng 2.31 bằng đúng độ chính xác của hồi quy logistic.



Hình 2.28: Không có hàm phi tuyến, nhiều lớp thu gọn thành một lớp

Có thể kiểm chứng bằng số: gộp hai lớp thành một lớp với trọng số là tích hai ma trận trọng số.

```
W1, b1 = rng.standard_normal((4, 3)), rng.standard_normal((4, 1))
W2, b2 = rng.standard_normal((1, 4)), rng.standard_normal((1, 1))
X = rng.standard_normal((3, 5))

hai_lop = np.dot(W2, np.dot(W1, X) + b1) + b2              # hai lớp, không có
↪  hàm kích hoạt
W_gop, b_gop = np.dot(W2, W1), np.dot(W2, b1) + b2         # gộp thành một lớp
mot_lop = np.dot(W_gop, X) + b_gop
print(np.allclose(hai_lop, mot_lop))                       # True: hai lớp tuyến
↪  tính = một lớp tuyến tính
```

Chỗ duy nhất dùng hàm tuyến tính. Đó là *lớp ra của bài toán hồi quy*, khi đầu ra là một số thực bất kỳ (như giá nhà): lớp ra không cần kích hoạt, đúng như mạng giá nhà ở mục 2.1 dùng `Dense(1)`. Nếu giá không được âm, có thể dùng ReLU ở lớp ra. Các lớp ẩn vẫn phải dùng hàm phi tuyến.

2.4.4.4 Độ dốc của các hàm kích hoạt

Lan truyền ngược cần độ dốc của hàm kích hoạt tại từng giá trị. Có ba điều đáng nhớ:

- Với sigmoid và tanh, độ dốc tính được trực tiếp từ kích hoạt đã có, không cần tính lại hàm: sigmoid là kích hoạt nhân (1 trừ kích hoạt), tanh là 1 trừ bình phương kích hoạt.
- Hai hàm này có độ dốc gần 0 ở hai đầu: tổng có trọng số bằng -10 hoặc 10 cho độ dốc bằng 0 với tanh và 0,00005 với sigmoid. Độ dốc lớn nhất của sigmoid chỉ là 0,25.
- ReLU và Leaky ReLU có độ dốc không xác định đúng tại 0, nhưng khả năng một tổng đúng bằng 0 rất nhỏ, nên trong thực tế gán độ dốc ở đó là 0 hoặc 1 đều chạy tốt.

Có thể xác nhận bằng cách để TensorFlow tự tính độ dốc rồi so với các công thức ngắn trên:

```
import tensorflow as tf

z = tf.constant([-2.0, -0.5, 0.5, 2.0])
cac_ham = [("sigmoid", tf.sigmoid), ("tanh", tf.tanh), ("relu", tf.nn.relu),
           ("leaky_relu", lambda t: tf.nn.leaky_relu(t, alpha=0.01))]
for ten, f in cac_ham:
    with tf.GradientTape() as tape:
        tape.watch(z)
        y = f(z)
    print(ten, tape.gradient(y, z).numpy().round(4))
# sigmoid    [0.105  0.235  0.235  0.105]
# tanh       [0.0707 0.7864 0.7864 0.0707]
# relu       [0. 0. 1. 1.]
# leaky_relu [0.01 0.01 1. 1.]

a = 1 / (1 + np.exp(-z.numpy()))
print((a * (1 - a)).round(4))                       # khớp với dòng sigmoid ở
↪  trên
print((1 - np.tanh(z.numpy()) ** 2).round(4))       # khớp với dòng tanh ở trên
```

2.4.5 Huấn luyện: gradient descent và lan truyền ngược

2.4.5.1 Vòng lặp học

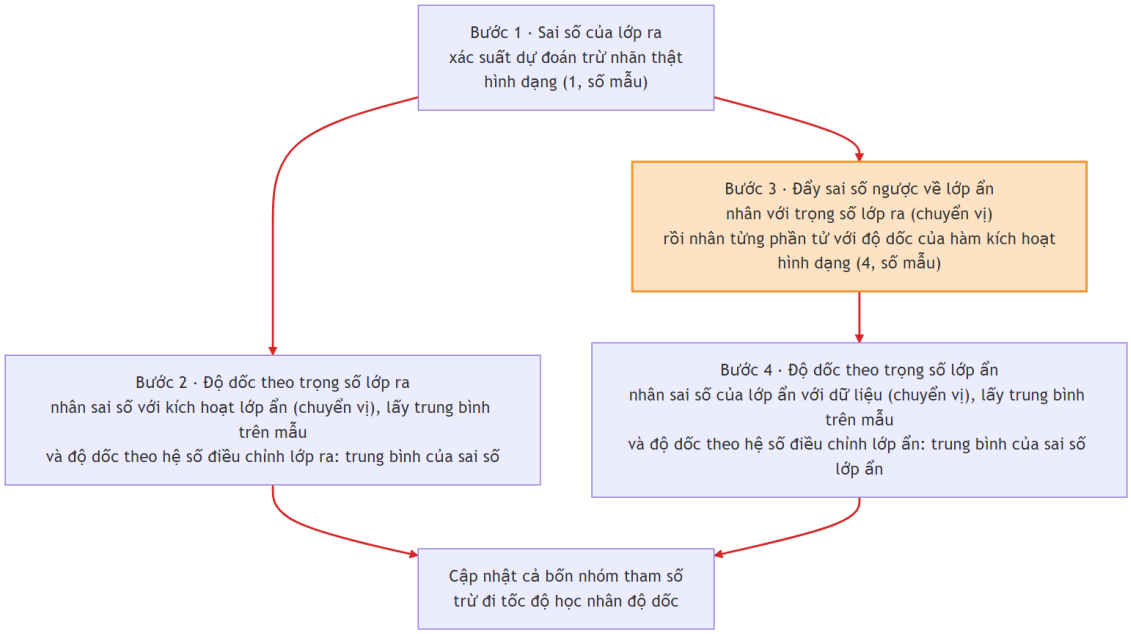
Huấn luyện mạng nơ-ron là lặp lại một vòng bốn bước đã quen từ hồi quy logistic:

1. **Lan truyền xuôi**: tính xác suất dự đoán cho mọi mẫu (bốn bước ở mục trước).
2. **Tính chi phí**: cross-entropy nhị phân, trung bình trên các mẫu (mục 2.2.3).
3. **Lan truyền ngược**: tính độ dốc của chi phí theo mỗi nhóm tham số.
4. **Cập nhật**: mỗi tham số bằng chính nó trừ đi tích của tốc độ học và độ dốc, rồi lặp lại.

Với Keras, ba bước cuối nằm gọn trong `compile` và `fit`: bạn chỉ chọn hàm mất mát, bộ tối ưu và số lần lặp; framework tự tính độ dốc cho cả mạng. Phần còn lại của mục này giải thích bên trong để hiểu những gì xảy ra, nhưng *trong dự án không cần tự viết lan truyền ngược*.

2.4.5.2 Lan truyền ngược qua hai lớp

Lan truyền ngược cho hai lớp làm đúng việc của hồi quy logistic, nhưng hai lần, đi từ phải sang trái (Hình 2.29). Ý tưởng: tính sai số ở đầu ra, rồi *đẩy sai số ngược về* từng nút ẩn theo mức nút đó đóng góp (qua trọng số của lớp ra), rồi điều chỉnh theo độ dốc của hàm kích hoạt.



Hình 2.29: Lan truyền ngược qua hai lớp: bốn bước, ba bước đầu giống hồi quy logistic

Bước	Phép tính (bằng lời)	Tên trong code	Hình dạng
1	sai số của lớp ra: xác suất dự đoán trừ nhãn thật	`dZ2`	`(1, số mẫu)`
2	độ dốc theo trọng số lớp ra: nhân sai số với kích hoạt lớp ẩn (chuyển vị), lấy trung bình trên các mẫu	`dW2`	`(1, 4)`
3	độ dốc theo hệ số điều chỉnh lớp ra: trung bình của sai số	`db2`	`(1, 1)`
4	sai số của lớp ẩn: nhân trọng số lớp ra (chuyển vị) với sai số lớp ra, rồi nhân *từng phần tử* với độ dốc hàm kích hoạt	`dZ1`	`(4, số mẫu)`
5	độ dốc theo trọng số lớp ẩn: nhân sai số lớp ẩn với dữ liệu (chuyển vị), lấy trung bình	`dW1`	`(4, 3)`
6	độ dốc theo hệ số điều chỉnh lớp ẩn: trung bình của sai số lớp ẩn	`db1`	`(4, 1)`

Bảng 2.32: Sáu bước của lan truyền ngược cho mạng một lớp ẩn

Ba bước đầu (với lớp ra) giống hệt hồi quy logistic, chỉ khác là kích hoạt của lớp ẩn đóng vai trò dữ liệu. Phần mới là bước 4: sai số được kéo về lớp ẩn, rồi nhân từng phần tử với độ dốc của hàm kích hoạt của lớp ẩn.

Mẹo kiểm tra bằng hình dạng. Một đại lượng và độ dốc của nó *luôn có cùng hình dạng*: trọng số và độ dốc theo trọng số, hệ số điều chỉnh và độ dốc của nó, tổng có trọng số và sai số tương ứng. Khi tự viết lan truyền ngược, hãy kiểm tra các hình dạng khớp nhau; việc này loại bỏ được rất nhiều lỗi.

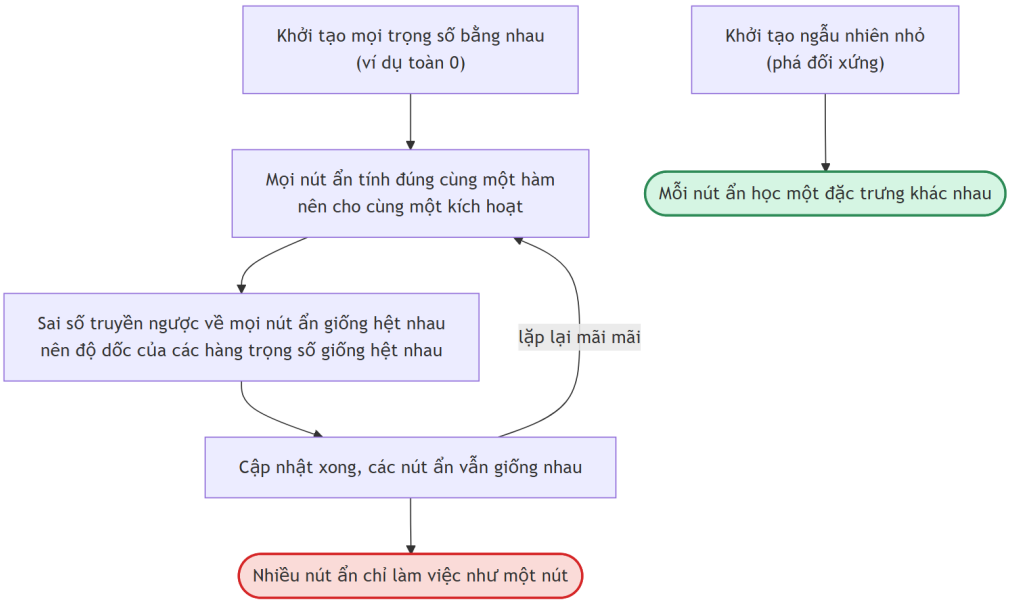
Đọc thêm. Suy ra các phép tính này từ đầu cần cả đại số tuyến tính lẫn đạo hàm của ma trận, và được xem là một trong những phép chứng minh khó nhất của học máy. Nhiều người dùng học sâu thành thạo chỉ cần hiểu đến mức ý tưởng ở trên. Ai muốn chứng minh có thể thử; ai không thì chỉ cần biết rằng độ dốc tính theo bảng trên là đúng, và **cách kiểm tra đơn giản nhất là đẩy nhẹ từng tham số một lượng rất nhỏ rồi đo chi phí thay đổi**. Kiểm tra bằng cách này ở mục 2.5 cho độ lệch chỉ cỡ một phần mười tỉ.

2.4.6 Khởi tạo tham số

Với hồi quy logistic, khởi tạo mọi tham số bằng 0 vẫn được. **Với mạng nơ-ron thì không.**

2.4.6.1 Vấn đề đối xứng

Giả sử mọi trọng số bằng nhau (ví dụ toàn 0). Khi đó mọi nút ẩn tính *đúng cùng một hàm*: với mọi mẫu, chúng cho cùng một kích hoạt. Khi lan truyền ngược, sai số truyền về mọi nút ẩn giống hệt nhau, nên các hàng của ma trận độ dốc giống nhau. Cập nhật xong, các hàng trọng số vẫn giống nhau. Lập luận này lặp lại ở mọi vòng, nên **dù huấn luyện bao lâu, các nút ẩn vẫn tính cùng một hàm**, và việc có nhiều nút ẩn trở nên vô nghĩa (Hình 2.30).



Hình 2.30: Khởi tạo đồng nhất làm mọi nút ẩn đối xứng; khởi tạo ngẫu nhiên phá đối xứng

Thí nghiệm ở mục 2.5 xác nhận: khởi tạo mọi trọng số bằng 0,5 thì sau 2000 lần lặp, bốn hàng của ma trận trọng số lớp ẩn vẫn *giống hệt nhau* và độ chính xác chỉ 65,5%, ngang một mạng một nút ẩn (67%). Khởi tạo toàn 0 còn tệ hơn: mọi trọng số mãi bằng 0 và độ chính xác là 50%, tức đoán mò.

2.4.6.2 Cách khởi tạo

Giải pháp là **khởi tạo ngẫu nhiên**: lấy số ngẫu nhiên theo phân phối Gauss rồi nhân với một số nhỏ như 0,01. Hệ số điều chỉnh có thể khởi tạo bằng 0, vì nó không gây vấn đề đối xứng: chỉ cần trọng số ngẫu nhiên là các nút ẩn đã tính những thứ khác nhau.

```
def initialize_parameters(n_x, n_h, n_y, scale=0.01, seed=3):
    rng = np.random.RandomState(seed)
    return {"W1": rng.randn(n_h, n_x) * scale, "b1": np.zeros((n_h, 1)),
            "W2": rng.randn(n_y, n_h) * scale, "b2": np.zeros((n_y, 1))}
```

2.4.6.3 Vì sao nhỏ

Nếu trọng số quá lớn, tổng có trọng số ngay từ đầu đã rất lớn hoặc rất nhỏ, đẩy tanh hoặc sigmoid vào vùng phẳng (bão hoà) nơi độ dốc gần 0, nên học rất chậm. Bảng 2.33 đo điều đó trên dữ liệu hình bông hoa (4 nút ẩn, 1000 lần lặp).

Độ lớn khởi tạo	Chi phí ban đầu	Sau 100 lần	Sau 900 lần	Chính xác
0,01	0,693	0,610	0,262	89,3%
1	0,771	0,337	0,265	89,0%
10	6,113	0,520	0,479	70,8%
100	16,355	0,865	0,609	58,5%

Bảng 2.33: Khởi tạo quá lớn làm chi phí ban đầu rất cao và mạng kẹt ở mức chính xác thấp

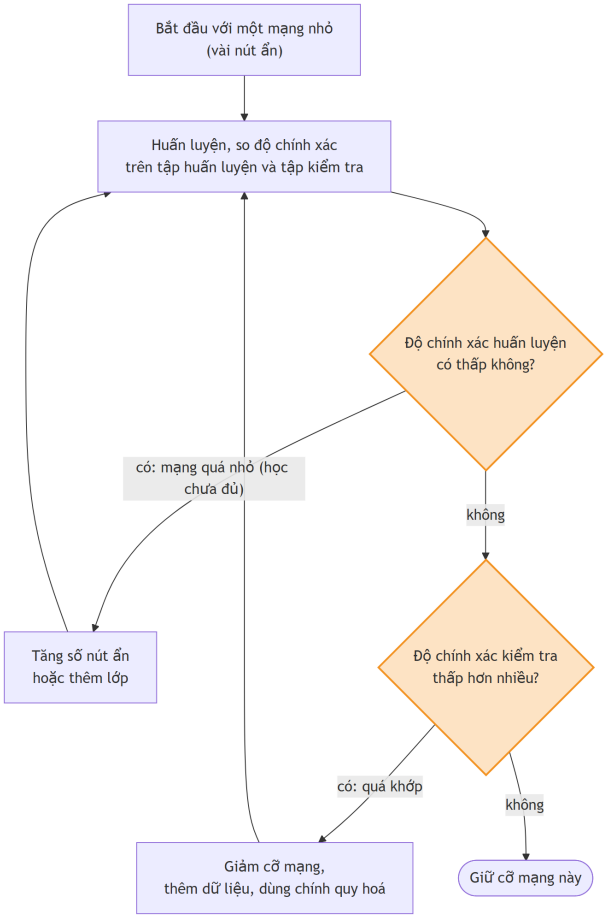
Độ lớn 0,01 và 1 đều ổn trên mạng nhỏ này; 10 và 100 làm bão hoà và kẹt. Với mạng một lớp ẩn, 0,01 là điểm khởi đầu hợp lý. Với mạng rất sâu có thể cần hằng số khác (sẽ nói ở chương sau); luôn là một số tương đối nhỏ. Keras mặc định dùng cách khởi tạo khác (`glorot_uniform`, tự co theo kích thước lớp) và hệ số điều chỉnh bằng 0, nên không cần tự chọn trong đa số trường hợp:

```
layers.Dense(4, activation="tanh",
             kernel_initializer="glorot_uniform",      # mặc định của Keras, có
             ↪  thể đổi
             bias_initializer="zeros")                 # hệ số điều chỉnh khởi
             ↪  tạo bằng 0
```

2.4.7 Áp dụng vào dự án thực tế

2.4.7.1 Chọn số nút ẩn

Số nút ẩn là một siêu tham số. Quy trình thực dụng: bắt đầu với mạng nhỏ, huấn luyện, rồi nhìn hai độ chính xác (Hình 2.31).



Hình 2.31: Chọn số nút ẩn dựa trên độ chính xác huấn luyện và độ chính xác kiểm tra

2.4.7.2 Siêu tham số khởi đầu

Siêu tham số	Gợi ý	Ghi chú
Số nút ẩn	4 đến 16 cho bài toán nhỏ	tăng nếu độ chính xác huấn luyện thấp; giảm nếu quá khớp
Hàm kích hoạt lớp ẩn	ReLU hoặc tanh	thử cả hai trên tập kiểm định
Hàm kích hoạt lớp ra	theo loại bài toán	sigmoid cho hai lớp (Hình 2.27)
Bộ tối ưu	Adam, tốc độ học 0,001 đến 0,01	hoặc SGD (gradient descent thuần) với tốc độ học lớn hơn
Số epoch	vài trăm đến vài nghìn với dữ liệu nhỏ	dừng khi chi phí kiểm định thôi giảm
Khởi tạo	mặc định của Keras	không bao giờ để toàn 0

Bảng 2.34: Điểm khởi đầu cho mạng một lớp ẩn

2.4.7.3 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Mạng không học được, độ chính xác ngang hồi quy logistic	thiếu hàm kích hoạt phi tuyến ở lớp ẩn (mọi lớp đều tuyến tính, Bảng 2.31)
Chi phí đứng yên ở 0,693, độ chính xác 50%	khởi tạo mọi trọng số bằng 0 hoặc bằng nhau (mục 2.4.6)
Chi phí ban đầu rất lớn, rồi kẹt ở mức cao	khởi tạo trọng số quá lớn làm tanh hoặc sigmoid bão hoà; nhân với số nhỏ
Học rất chậm với lớp ẩn dùng sigmoid	độ dốc sigmoid tối đa chỉ 0,25; đổi sang tanh hoặc ReLU
Lỗi hình dạng khi nối code tự viết với Keras	Keras xếp mẫu theo hàng và lưu trọng số chuyển vị; dùng `.T`
Độ chính xác huấn luyện cao, kiểm tra thấp	quá khớp: giảm cỡ mạng hoặc thêm dữ liệu; luôn đo trên tập kiểm tra
Hệ số điều chỉnh khởi tạo ngẫu nhiên mà vẫn chạy	không sai, nhưng không cần; chỉ trọng số cần ngẫu nhiên
Kết quả mỗi lần chạy mỗi khác	chưa cố định hạt giống (`keras.utils.set_random_seed`) khi so sánh các cấu hình

Bảng 2.35: Lỗi thường gặp khi xây mạng nơ-ron một lớp ẩn

2.4.7.4 Đối chiếu giữa các thư viện

Việc cần làm	Keras	PyTorch / scikit-learn
Lớp ẩn 4 nút, tanh	`Dense(4, activation="tanh")`	PyTorch: `nn.Linear(3, 4)` rồi `nn.Tanh()`
Lớp ra, sigmoid	`Dense(1, activation="sigmoid")`	PyTorch: `nn.Linear(4, 1)` rồi `nn.Sigmoid()`
Toàn bộ mạng một lớp ẩn	`Sequential([...])`	scikit-learn: `MLPClassifier(hidden_layer_sizes=(4,))`
Hàm mất mát cross-entropy nhị phân	`binary_crossentropy`	PyTorch: `nn.BCELoss`
Huấn luyện	`compile` rồi `fit`	PyTorch: tự viết vòng lặp; scikit-learn: `fit`
Độ dốc tự động	`tf.GradientTape`	PyTorch: `loss.backward()`
Xếp mẫu	theo hàng	theo hàng

Bảng 2.36: Đối chiếu API cho mạng một lớp ẩn

2.4.8 Tóm tắt cần nhớ

1. **Mạng nơ-ron là hồi quy logistic được lặp lại**: nhiều neuron chạy song song tạo một lớp, đầu ra của lớp này là đầu vào của lớp sau. Trong Keras, đó là các lớp `Dense` xếp nối tiếp.

2. **Lớp vào, lớp ẩn, lớp ra.** Lớp ẩn có tên vì giá trị đúng của nó không xuất hiện trong dữ liệu. Mạng này là *mạng hai lớp* vì không đếm lớp vào.

3. **Quy tắc hình dạng**: ma trận trọng số có số hàng bằng số nút của lớp đó và số cột bằng số nút của lớp liền trước. Mẫu xếp theo cột (khi tự viết) hoặc theo hàng (Keras).

4. **Bốn bước tính đầu ra**: tổng có trọng số của lớp ẩn, kích hoạt của lớp ẩn, tổng có trọng số của lớp ra, xác suất dự đoán. Với nhiều mẫu, dùng đúng bốn bước đó cho cả ma trận dữ liệu; đi ngang là đi qua các mẫu, đi dọc là đi qua các nút.

5. **Hàm kích hoạt phi tuyến là bắt buộc** ở lớp ẩn; nếu không, mạng chỉ bằng một phép tuyến tính (47% trên dữ liệu hình bông hoa). Sigmoid cho lớp ra nhị phân; tanh hoặc ReLU cho lớp ẩn; nên thử và so trên tập kiểm định.

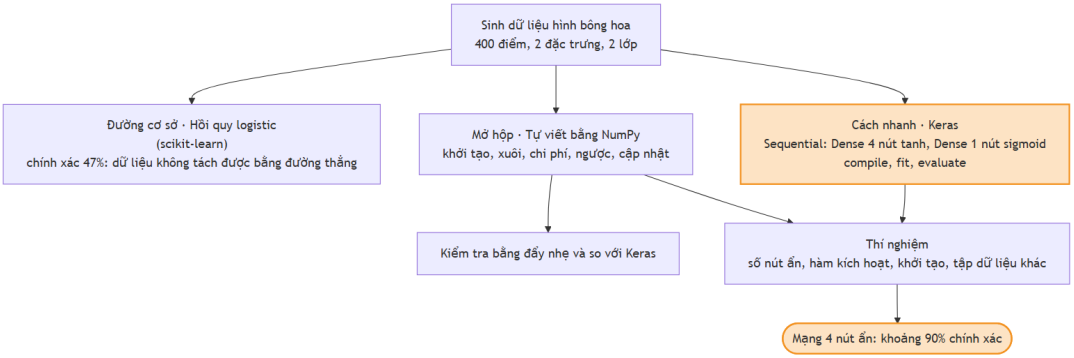
6. **Lan truyền ngược có sáu bước**; ba bước đầu giống hồi quy logistic, phần mới là đẩy sai số về lớp ẩn rồi nhân từng phần tử với độ dốc của hàm kích hoạt. Framework tự làm; kiểm tra bằng cách đẩy nhẹ tham số hoặc dùng `tf.GradientTape`.

7. **Khởi tạo ngẫu nhiên nhỏ cho trọng số**, không để toàn 0 hoặc bằng nhau (vấn đề đối xứng). Hệ số điều chỉnh khởi tạo bằng 0 được.

8. **Chọn cỡ mạng bằng hai độ chính xác**: huấn luyện thấp thì tăng cỡ mạng; kiểm tra thấp hơn nhiều thì quá khớp, giảm cỡ mạng hoặc thêm dữ liệu.

2.5 Thực hành: phân loại dữ liệu hình bông hoa

Bài toán: có 400 điểm trên mặt phẳng, mỗi điểm thuộc một trong hai lớp, xếp thành hình một bông hoa với các cánh xen kẽ nhau. Xây bộ phân loại cho biết một điểm thuộc lớp nào. Dữ liệu này *không tách được bằng một đường thẳng*, nên hồi quy logistic sẽ thất bại và ta cần mạng nơ-ron có lớp ẩn (Hình 2.32).



Hình 2.32: Các bước của ví dụ thực hành: đường cơ sở, cách nhanh bằng Keras, rồi mở hộp bằng NumPy và làm thí nghiệm

2.5.1 Dữ liệu và đường cơ sở

Hàm dưới đây sinh dữ liệu: hai lớp, mỗi lớp 200 điểm, bán kính của mỗi điểm thay đổi theo góc để tạo cánh hoa. Ma trận dữ liệu có hình dạng (2, 400) và nhãn có hình dạng (1, 400), theo quy ước mỗi mẫu một cột. Cố định hạt giống để kết quả tái hiện được.

```
import numpy as np
from sklearn.linear_model import LogisticRegressionCV

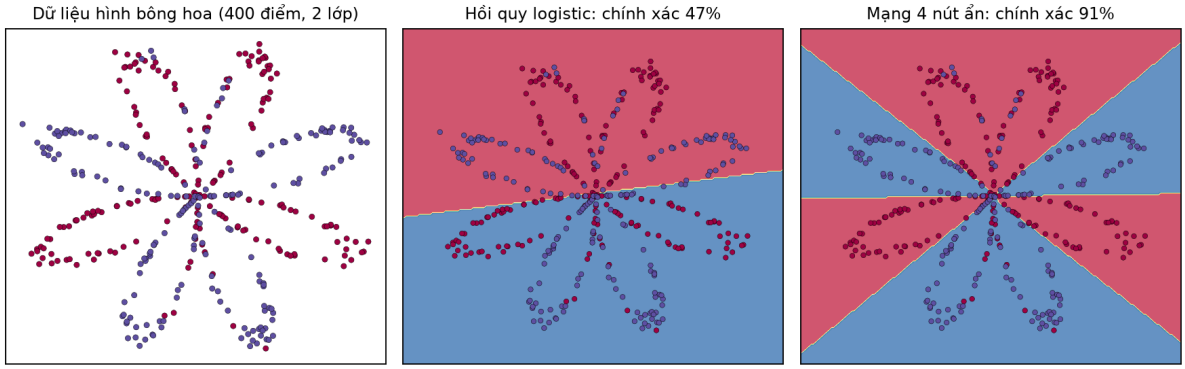
def make_flower_dataset(m=400, seed=1):
    """Hai lớp điểm xếp thành các cánh hoa. Trả về X cỡ (2, m) và Y cỡ (1, m)."""
    rng = np.random.RandomState(seed)
    n = m // 2                                  # số điểm mỗi lớp
```

```
    X = np.zeros((m, 2))
    Y = np.zeros((m, 1), dtype="uint8")
    for j in range(2):
        idx = range(n * j, n * (j + 1))
        t = np.linspace(j * 3.12, (j + 1) * 3.12, n) + rng.randn(n) * 0.2   # góc
        r = 4 * np.sin(4 * t) + rng.randn(n) * 0.2                          # bán
        ↪ kính
        X[idx] = np.c_[r * np.sin(t), r * np.cos(t)]
        Y[idx] = j
    return X.T, Y.T

X, Y = make_flower_dataset()                        # tập huấn luyện
Xt, Yt = make_flower_dataset(seed=2)                # tập kiểm tra: cùng quy luật,
↪ điểm khác
print(X.shape, Y.shape)                             # (2, 400) (1, 400)

clf = LogisticRegressionCV(max_iter=1000).fit(X.T, Y.ravel())    # scikit-learn
↪ nhận mẫu ở hàng
print(clf.score(X.T, Y.ravel()))                    # 0.47
```

Hồi quy logistic chỉ đạt **47%**, thấp hơn cả đoán mò (hai lớp cân bằng nên đoán mò được khoảng 50%). Đường thẳng không thể chia bông hoa (Hình 2.33, ô giữa). Đây là lý do ta cần mạng nơ-ron.



Hình 2.33: Trái: dữ liệu hình bông hoa. Giữa: ranh giới của hồi quy logistic (một đường thẳng). Phải: ranh giới của mạng 4 nút ẩn (uốn thành các cánh hoa)

2.5.2 Cách nhanh: Keras và scikit-learn

Với Keras, mạng gồm đúng hai lớp `Dense`: 4 nút ẩn dùng tanh và 1 nút ra dùng sigmoid. Dùng bộ tối ưu Adam và đưa cả tập vào mỗi bước (gradient descent trên toàn lô, như các mục trước). Keras nhận mẫu ở *hàng*, nên truyền `.T`.

```
from tensorflow import keras
from tensorflow.keras import layers

keras.utils.set_random_seed(0)
model = keras.Sequential([
    keras.Input(shape=(2,)),
    layers.Dense(4, activation="tanh"),
    layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.01),
              loss="binary_crossentropy", metrics=["accuracy"])
```

```
model.fit(X.T, Y.T, epochs=1000, batch_size=X.shape[1], verbose=0)    # mỗi epoch
↪  là một bước trên cả tập
print(model.evaluate(X.T, Y.T, verbose=0))       # khoảng [0.30, 0.90]: chi phí,
↪  độ chính xác
print(model.count_params())                      # 17 tham số
```

Kết quả: chi phí khoảng 0,30 và **độ chính xác khoảng 90%** với chỉ 17 tham số (con số chính xác phụ thuộc phiên bản thư viện và hạt giống). Với scikit-learn, cùng ý tưởng chỉ cần một đối tượng. Tuy nhiên bộ tối ưu mặc định của nó dừng sớm trên tập nhỏ này, nên chọn `lbfgs`:

```
from sklearn.neural_network import MLPClassifier

for solver in ("adam", "lbfgs"):
    mlp = MLPClassifier(hidden_layer_sizes=(4,), activation="tanh",
    ↪  solver=solver,
                        max_iter=5000, random_state=0).fit(X.T, Y.ravel())
    print(solver, mlp.score(X.T, Y.ravel()), mlp.score(Xt.T, Yt.ravel()))
# adam  khoảng 0.62 0.62   (dừng sớm vì chi phí giảm quá chậm)
# lbfgs khoảng 0.92 0.92
```

Bài học thực dụng: *bộ tối ưu và cấu hình mặc định ảnh hưởng đến kết quả*; cùng một kiến trúc có thể cho 62% hay 92% tuỳ cách huấn luyện. Luôn kiểm tra chi phí có giảm hay không thay vì tin vào cấu hình mặc định.

Về tốc độ. Với dữ liệu bé như thế này, Keras tốn khoảng 40 giây cho 1000 epoch trên CPU của máy viết tài liệu, phần lớn là chi phí cố định của framework cho mỗi bước. Bản NumPy ở dưới chạy 10.000 lần lặp chỉ trong khoảng 1 giây. Framework chỉ thật sự vượt trội khi dữ liệu và mạng lớn, hoặc khi cần GPU.

2.5.3 Mở hộp: tự viết bằng NumPy

Để thấy từng bước hoạt động, dưới đây là toàn bộ mạng bằng NumPy: khởi tạo, lan truyền xuôi, chi phí, lan truyền ngược (sáu bước ở Bảng 2.32), cập nhật và dự đoán.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def initialize_parameters(n_x, n_h, n_y, scale=0.01, seed=3):
    rng = np.random.RandomState(seed)
    return {"W1": rng.randn(n_h, n_x) * scale, "b1": np.zeros((n_h, 1)),
            "W2": rng.randn(n_y, n_h) * scale, "b2": np.zeros((n_y, 1))}

def forward(X, p):
    Z1 = np.dot(p["W1"], X) + p["b1"]
    A1 = np.tanh(Z1)
    Z2 = np.dot(p["W2"], A1) + p["b2"]
    A2 = sigmoid(Z2)
    return A2, {"Z1": Z1, "A1": A1, "Z2": Z2, "A2": A2}

def compute_cost(A2, Y):
    m = Y.shape[1]
    return float(-np.sum(Y * np.log(A2) + (1 - Y) * np.log(1 - A2)) / m)

def backward(p, cache, X, Y):
    m = X.shape[1]
    A1, A2 = cache["A1"], cache["A2"]
```

```
    dZ2 = A2 - Y                                    # bước 1: sai số của
    ↪ lớp ra
    dW2 = np.dot(dZ2, A1.T) / m                     # bước 2
    db2 = np.sum(dZ2, axis=1, keepdims=True) / m    # bước 3
    dZ1 = np.dot(p["W2"].T, dZ2) * (1 - A1 ** 2)    # bước 4: đẩy sai số về
    ↪ lớp ẩn, nhân độ dốc của tanh
    dW1 = np.dot(dZ1, X.T) / m                      # bước 5
    db1 = np.sum(dZ1, axis=1, keepdims=True) / m    # bước 6
    return {"dW1": dW1, "db1": db1, "dW2": dW2, "db2": db2}

def train(X, Y, n_h, num_iterations=10000, learning_rate=1.2, scale=0.01, seed=3,
↪   zero_init=False, log_every=0):
    p = initialize_parameters(X.shape[0], n_h, Y.shape[0], scale, seed)
    if zero_init:
        p = {k: np.zeros_like(v) for k, v in p.items()}
    costs = []
    for i in range(num_iterations):
        A2, cache = forward(X, p)
        g = backward(p, cache, X, Y)
        for k in ("W1", "b1", "W2", "b2"):
            p[k] = p[k] - learning_rate * g["d" + k]
        if log_every and i % log_every == 0:
            costs.append(round(compute_cost(np.clip(A2, 1e-12, 1 - 1e-12), Y),
            ↪  6))
    return p, costs

def predict(p, X):
    return (forward(X, p)[0] > 0.5).astype(int)

def accuracy(p, X, Y):
    return float(np.mean(predict(p, X) == Y) * 100)
```

Huấn luyện mạng 4 nút ẩn với tốc độ học 1,2 trong 10.000 lần lặp:

```
p4, costs = train(X, Y, 4, 10000, 1.2, log_every=1000)
print(costs)                          # [0.6932, 0.2586, 0.2393, 0.2308, 0.2255,
↪  0.2218, 0.2191, 0.2206, 0.2194, 0.2185]
print(accuracy(p4, X, Y))             # 90.75
```

Chi phí đầu tiên là 0,6932 (đoán 0,5 cho mọi điểm) rồi giảm xuống khoảng 0,22; **độ chính xác 90,75%** so với 47% của hồi quy logistic. Hai bản (Keras và NumPy) đạt mức tương đương, khác nhau chút vì bộ tối ưu và khởi tạo khác nhau.

2.5.3.1 Kiểm tra bản tự viết

Có hai cách đáng tin để biết lan truyền ngược viết đúng. Cách một là đẩy nhẹ từng tham số một lượng rất nhỏ rồi đo chi phí đổi bao nhiêu, so với độ dốc tính bằng `backward`. Cách hai là cho TensorFlow tự tính độ dốc trên cùng trọng số rồi so sánh.

```
Xs, Ys = X[:, :20], Y[:, :20]                       # 20 mẫu đủ để kiểm tra
ps = initialize_parameters(2, 4, 1, scale=0.5, seed=0)
_, cache = forward(Xs, ps)
g = backward(ps, cache, Xs, Ys)

# Cách 1: đẩy nhẹ từng tham số (độ dốc bằng số)
worst = 0.0
for k in ("W1", "b1", "W2", "b2"):
```

```
    so = np.zeros_like(ps[k])
    for idx in np.ndindex(*ps[k].shape):
        old = ps[k][idx]
        ps[k][idx] = old + 1e-6; c_tren = compute_cost(forward(Xs, ps)[0], Ys)
        ps[k][idx] = old - 1e-6; c_duoi = compute_cost(forward(Xs, ps)[0], Ys)
        ps[k][idx] = old
        so[idx] = (c_tren - c_duoi) / 2e-6
    worst = max(worst, np.max(np.abs(so - g["d" + k])))
print(worst)                                          # khoảng 1.8e-10: khớp

# Cách 2: để TensorFlow tự tính, trên cùng trọng số (đặt vào Keras, nhớ chuyển
↪  vị)
import tensorflow as tf
km = keras.Sequential([keras.Input(shape=(2,)), layers.Dense(4,
↪  activation="tanh"), layers.Dense(1, activation="sigmoid")])
km.set_weights([ps["W1"].T, ps["b1"].ravel(), ps["W2"].T, ps["b2"].ravel()])
with tf.GradientTape() as tape:
    du_doan = km(tf.constant(Xs.T, dtype=tf.float32))
    chi_phi = tf.reduce_mean(keras.losses.binary_crossentropy(tf.constant(Ys.T,
    ↪  dtype=tf.float32), du_doan))
gk = tape.gradient(chi_phi, km.trainable_variables)
print(np.allclose(gk[0].numpy().T, g["dW1"], atol=1e-5),
↪  np.allclose(gk[2].numpy().T, g["dW2"], atol=1e-5))   # True True
```

Độ lệch lớn nhất giữa độ dốc tính bằng số và độ dốc tính bằng `backward` chỉ cỡ một phần mười tỉ, và TensorFlow cho cùng kết quả: sáu bước lan truyền ngược viết đúng.

2.5.4 Các thí nghiệm

2.5.4.1 Thay đổi số nút ẩn

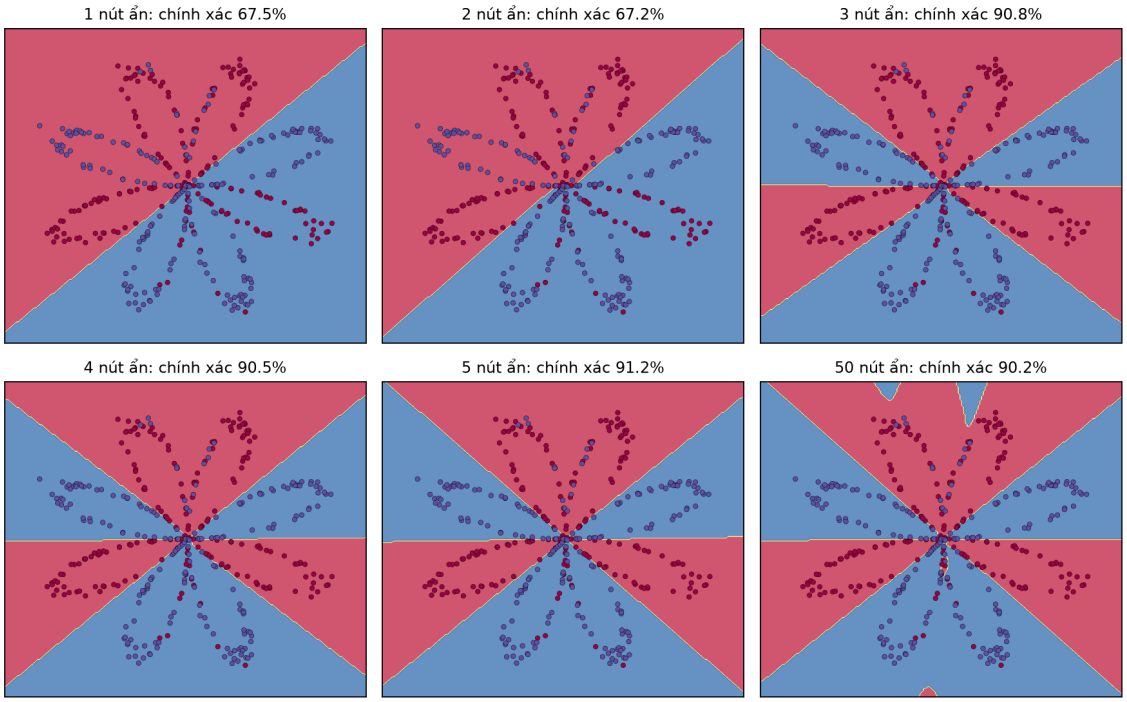
Huấn luyện các mạng có số nút ẩn khác nhau (5000 lần lặp) và đo độ chính xác trên cả tập huấn luyện lẫn tập kiểm tra chưa từng thấy:

```
for n_h in [1, 2, 3, 4, 5, 20, 50, 100]:
    p, _ = train(X, Y, n_h, 5000, 1.2)
    print(n_h, round(accuracy(p, X, Y), 2), round(accuracy(p, Xt, Yt), 2))

p, _ = train(X, Y, 50, 20000, 1.2)                    # mạng lớn, huấn luyện lâu
↪  hơn
print(50, round(accuracy(p, X, Y), 2), round(accuracy(p, Xt, Yt), 2))     # 93.75
↪  88.25
```

Số nút ẩn	1	2	3	4	5	20	50	100
Chính xác huấn luyện (%)	67,5	67,3	90,8	90,5	91,3	90,8	90,3	91,0
Chính xác kiểm tra (%)	66,5	64,3	90,5	90,3	90,3	89,0	90,0	89,0

Bảng 2.37: Độ chính xác theo số nút ẩn (5000 lần lặp, tốc độ học 1,2)



Hình 2.34: Ranh giới quyết định theo số nút ẩn: 1 và 2 nút chỉ vẽ được đường gần thẳng; từ 3 nút trở lên uốn được thành cánh hoa

Có ba điều đáng chú ý:

- **Từ 1 lên 3 nút ẩn là bước nhảy lớn** (67% lên 90%): một hoặc hai nút không đủ để biểu diễn ranh giới cong của bông hoa. Từ 3 nút trở lên, độ chính xác gần như không đổi (khoảng 90%).
- **Mạng lớn hơn không tốt hơn trên tập kiểm tra.** Mạng 20, 50 hay 100 nút không vượt mạng 3 đến 5 nút, vì dữ liệu có nhiễu và mức tối đa đạt được vào khoảng 90%.
- **Dấu hiệu quá khớp xuất hiện khi mạng lớn và huấn luyện lâu.** Mạng 50 nút huấn luyện thêm đến 20.000 lần lặp đạt 93,75% trên tập huấn luyện nhưng chỉ 88,25% trên tập kiểm tra: khoảng cách giữa hai con số mới là điều đáng lo, không phải riêng độ chính xác huấn luyện. Kỹ thuật chính quy hoá (chương sau) cho phép dùng mạng lớn mà ít quá khớp hơn.

2.5.4.2 Thay đổi hàm kích hoạt

Thí nghiệm ở Bảng 2.31 đã cho thấy tanh thắng ReLU và sigmoid trên bài toán nhỏ này, còn mạng không có hàm kích hoạt phi tuyến chỉ đạt 47%. Đọc lại kết quả đó như lời nhắc thử nhiều lựa chọn thay vì mặc định.

2.5.4.3 Thay đổi cách khởi tạo

Hai thí nghiệm xác nhận mục 2.4.6. Thứ nhất, khởi tạo toàn bộ trọng số bằng một hằng số (0,5) thay vì ngẫu nhiên:

```
p = initialize_parameters(2, 4, 1)
p["W1"][:] = 0.5                                    # mọi trọng số lớp ẩn bằng
↪  nhau
p["W2"][:] = 0.5
for i in range(2000):
    A2, cache = forward(X, p)
```

```
    g = backward(p, cache, X, Y)
    for k in ("W1", "b1", "W2", "b2"):
        p[k] = p[k] - 1.2 * g["d" + k]
print(p["W1"])                                          # 4 hàng giống hệt nhau,
↪  khoảng [2.51 2.37]
print(np.allclose(p["W1"], p["W1"][0]), accuracy(p, X, Y))    # True 65.5

pz, _ = train(X, Y, 4, 2000, 1.2, zero_init=True)    # toàn 0
print(pz["W1"].ravel(), accuracy(pz, X, Y))          # toàn 0, độ chính xác 50.0
```

Sau huấn luyện, bốn hàng của ma trận trọng số lớp ẩn *vẫn giống hệt nhau* và độ chính xác chỉ 65,5%, ngang mạng một nút ẩn: bốn nút ẩn chỉ làm việc như một nút. Khởi tạo toàn 0 còn tệ hơn: mọi trọng số mãi bằng 0 và độ chính xác 50%.

Thứ hai, thay đổi độ lớn của khởi tạo ngẫu nhiên (xem Bảng 2.33):

```
for scale in [0.01, 1.0, 10.0, 100.0]:
    p, costs = train(X, Y, 4, 1000, 1.2, scale=scale, log_every=100)
    print(scale, costs[0], costs[-1], round(accuracy(p, X, Y), 2))
```

Khởi tạo 0,01 và 1 đều đạt khoảng 89%; khởi tạo 10 và 100 làm bão hoà tanh nên chi phí ban đầu rất cao và mạng kẹt ở 70,8% và 58,5%.

2.5.4.4 Các tập dữ liệu khác

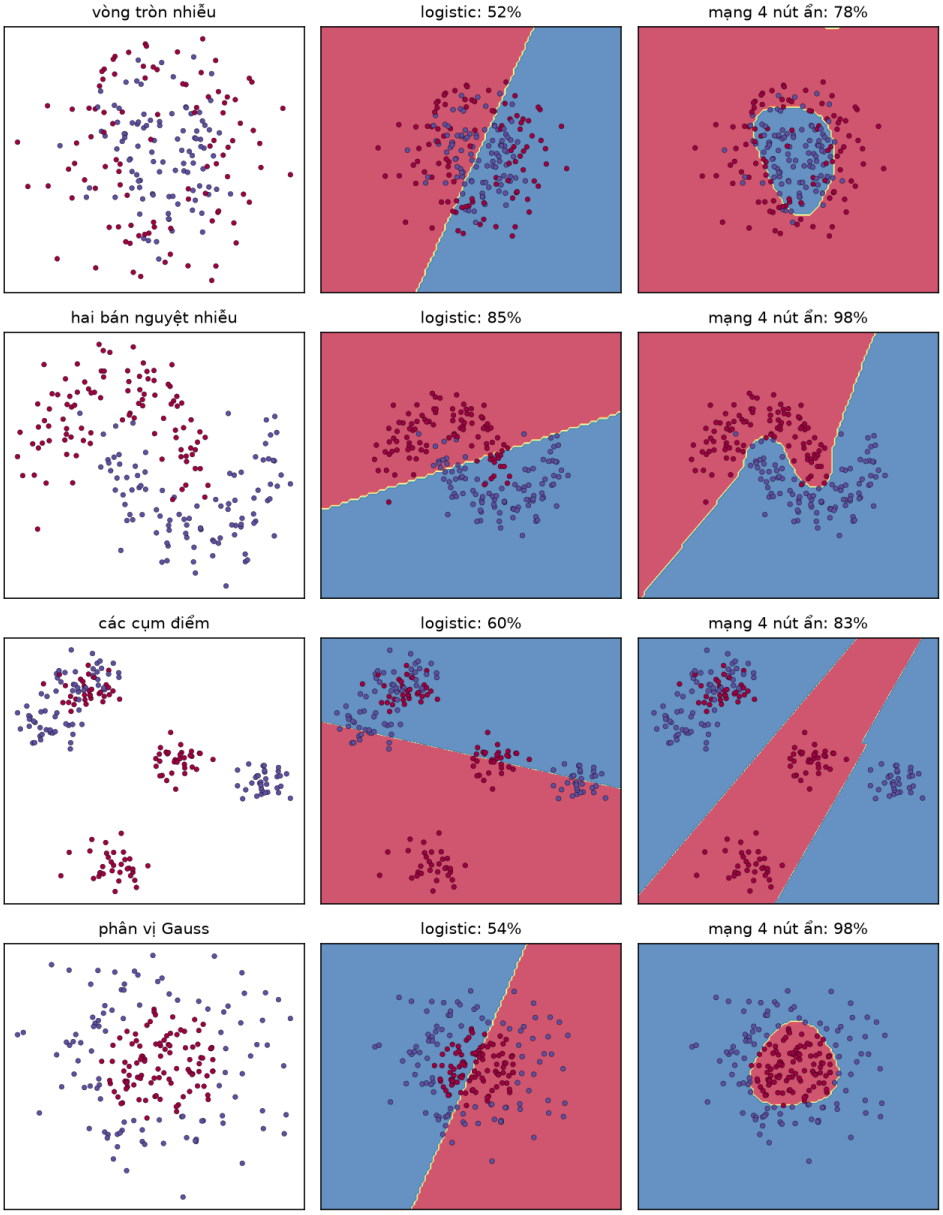
Cùng mạng 4 nút ẩn trên bốn tập dữ liệu hai lớp khác (200 điểm mỗi tập, độ chính xác trên chính tập huấn luyện):

```
import sklearn.datasets

tap = {
    "vòng tròn nhiễu": sklearn.datasets.make_circles(n_samples=200, factor=0.5,
    ↪  noise=0.3, random_state=0),
    "hai bán nguyệt nhiễu": sklearn.datasets.make_moons(n_samples=200, noise=0.2,
    ↪  random_state=0),
    "các cụm điểm": sklearn.datasets.make_blobs(n_samples=200, random_state=5,
    ↪  n_features=2, centers=6),
    "phân vị Gauss": sklearn.datasets.make_gaussian_quantiles(n_samples=200,
    ↪  cov=0.5, n_features=2, n_classes=2, random_state=0),
}
for ten, (Xo, Yo) in tap.items():
    Xo, Yo = Xo.T, Yo.reshape(1, -1)
    if ten == "các cụm điểm":
        Yo = Yo % 2                                     # 6 cụm thành 2 lớp: cụm
        ↪  chẵn và cụm lẻ
    lr_ = LogisticRegressionCV(max_iter=1000).fit(Xo.T, Yo.ravel())
    po, _ = train(Xo, Yo, 4, 5000, 1.2)
    print(ten, round(lr_.score(Xo.T, Yo.ravel()) * 100, 1), round(accuracy(po,
    ↪  Xo, Yo), 1))
```

Tập dữ liệu	Hồi quy logistic	Mạng 4 nút ẩn
Vòng tròn nhiễu	52,0%	78,0%
Hai bán nguyệt nhiễu	85,0%	98,5%
Các cụm điểm (chia thành hai lớp chẵn lẻ)	60,5%	83,0%
Phân vị Gauss	54,0%	98,0%

Bảng 2.38: Mạng có lớp ẩn vượt hồi quy logistic trên mọi tập dữ liệu không tách được bằng đường thẳng



Hình 2.35: Bốn tập dữ liệu khác: dữ liệu (trái), ranh giới hồi quy logistic (giữa), ranh giới mạng 4 nút ẩn (phải)

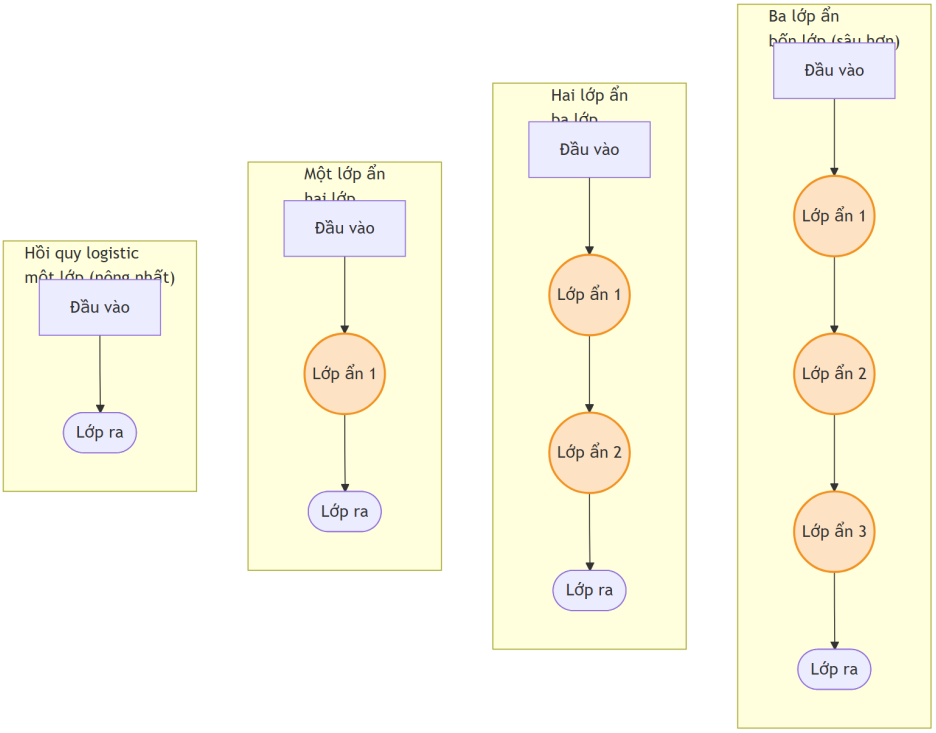
2.5.5 Rút ra

- **Lớp ẩn cộng hàm kích hoạt phi tuyến cho phép học ranh giới cong**: 47% của hồi quy logistic lên khoảng 90% với 4 nút ẩn.

- **Trong dự án, dùng Keras (hoặc scikit-learn)**: vài dòng là xong. Bản tự viết bằng NumPy chỉ để hiểu cơ chế và kiểm tra; đối chiếu độ dốc với `tf.GradientTape` là cách tốt để chắc chắn mọi thứ đúng.
- **Số nút ẩn**: vài nút là đủ cho bài toán này; mạng lớn hơn không giúp ích và có thể quá khớp. Luôn so độ chính xác huấn luyện với kiểm tra.
- **Hàm kích hoạt, tốc độ học, khởi tạo, bộ tối ưu** đều ảnh hưởng đến kết quả; thử nghiệm có hệ thống thay vì mặc định.
- **Cố định hạt giống** khi so sánh các cấu hình, để chênh lệch đến từ cấu hình chứ không phải may rủi.

2.6 Mạng nơ-ron sâu

Mục 2.4 dừng ở mạng một lớp ẩn. Mục này đi bước cuối của chương: **xếp chồng nhiều lớp ẩn** để có mạng nơ-ron sâu. Tin vui là gần như không có khái niệm mới. Mạng sâu chỉ là việc lặp lại cùng hai bước "tổng có trọng số rồi kích hoạt" nhiều lần hơn, nên mọi thứ đã học (lan truyền xuôi, lan truyền ngược, gradient descent) áp dụng nguyên xi, chỉ cần thêm một vòng lặp qua các lớp. Trong dự án thật, bạn chỉ cần xếp thêm vài dòng `Dense` trong Keras; phần *bên trong* được trình bày ở đây để biết cái gì đang chạy và lỗi nằm ở đâu.



Hình 2.36: "Nông" và "sâu" là chuyện nhiều hay ít lớp ẩn, không có ranh giới cứng

Lộ trình: nông và sâu → tên gọi và hình dạng trong mạng nhiều lớp → lan truyền xuôi qua nhiều lớp → vì sao mạng sâu hiệu quả → lan truyền ngược bằng các "khối xây dựng" → tham số và siêu tham số → khởi tạo cho mạng sâu → áp dụng vào dự án.

2.6.1 Nông và sâu

Hồi quy logistic là mạng *nông nhất*: chỉ một lớp, không có lớp ẩn. Mạng một lớp ẩn vẫn còn khá nông. Mạng có vài lớp ẩn bắt đầu được gọi là sâu, và mạng có hàng chục hay hàng trăm lớp là rất sâu. Đây là chuyện nhiều hay ít chứ không phải hai loại khác nhau (Hình 2.36).

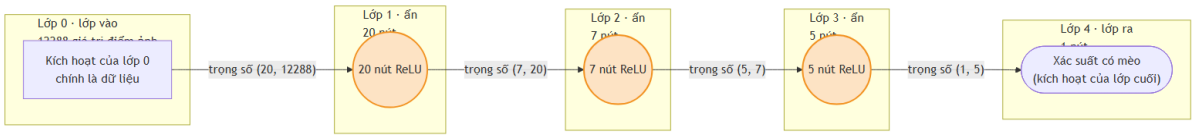
Mô hình	Số lớp ẩn	Số lớp (không đếm lớp vào)	Nhận xét
Hồi quy logistic	0	1	ranh giới thẳng
Mạng một lớp ẩn	1	2	ranh giới cong, học được mẫu đơn giản
Mạng hai lớp ẩn	2	3	bắt đầu "sâu"
Mạng ba lớp ẩn	3	4	mạng dùng trong phần thực hành

Bảng 2.39: Số lớp ẩn và số lớp của mạng. Quy ước đếm lớp giống mục 2.4.2: lớp vào không được đếm

Một lời khuyên thực dụng: **coi số lớp ẩn là một siêu tham số** (mục 2.6.6). Bắt đầu bằng hồi quy logistic làm mốc so sánh, rồi thử một lớp ẩn, hai lớp ẩn, và cứ thế, đánh giá mỗi lựa chọn bằng tập kiểm định. Không có cách biết trước độ sâu phù hợp cho từng bài toán.

2.6.2 Tên gọi và hình dạng trong mạng nhiều lớp

Mạng trong Hình 2.37 có bốn lớp (ba lớp ẩn và một lớp ra), nhận ảnh 64 nhân 64 điểm ảnh, ba kênh màu, đã duỗi thành 12288 giá trị. Đây chính là mạng sẽ dùng ở mục 2.7.



Hình 2.37: Mạng bốn lớp: ba lớp ẩn (20, 7 và 5 nút) và một nút ra. Nhãn trên mũi tên là hình dạng ma trận trọng số

Cách đặt tên giống mạng một lớp ẩn, chỉ có nhiều lớp hơn: mỗi lớp có ma trận trọng số, vector hệ số điều chỉnh, tổng có trọng số và kích hoạt riêng, đánh số từ lớp một đến lớp cuối. **Kích hoạt của lớp không chính là dữ liệu đầu vào, và kích hoạt của lớp cuối chính là dự đoán của mạng.** Nhờ quy ước đó mà các lớp ẩn và lớp ra có thể viết theo cùng một khuôn.

Tên gọi trong văn bản	Tên trong code	Hình dạng (mạng 12288, 20, 7, 5, 1)
Số lớp (không đếm lớp vào)	`L`	4
Số nút của từng lớp	`layer_dims`	`[12288, 20, 7, 5, 1]`
Trọng số của từng lớp	`W1` đến `W4`	`(20, 12288)`, `(7, 20)`, `(5, 7)`, `(1, 5)`
Hệ số điều chỉnh của lớp	`b1` đến `b4`	`(20, 1)`, `(7, 1)`, `(5, 1)`, `(1, 1)`
Tổng có trọng số của lớp	`Z1` đến `Z4`	`(số nút của lớp, số mẫu)`
Kích hoạt của lớp	`A1` đến `A4`	`(số nút của lớp, số mẫu)`
Kích hoạt của lớp không (dữ liệu)	`X` (hay `A0`)	`(12288, số mẫu)`
Kích hoạt của lớp cuối (dự đoán)	`AL`	`(1, số mẫu)`

Bảng 2.40: Tên gọi trong văn bản và tên biến trong code của mạng bốn lớp

Quy tắc hình dạng từ mục 2.4.2 vẫn đúng cho mọi lớp: **ma trận trọng số có số hàng bằng số nút của chính lớp đó và số cột bằng số nút của lớp liền trước**. Lớp đầu tiên chiếm gần như toàn bộ tham số,

vì nó nối 12288 giá trị đầu vào với 20 nút: 20 nhân 12288 trọng số cộng 20 hệ số điều chỉnh là 245 780 tham số, trong khi ba lớp còn lại cộng lại chỉ có 193 (147 + 40 + 6). Tổng cộng 245 973 tham số.

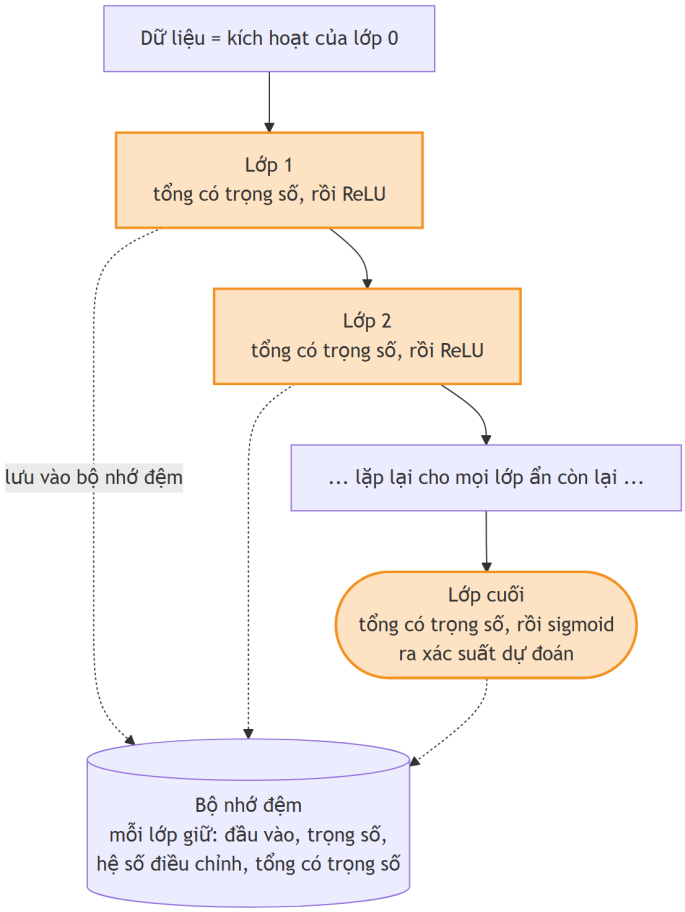
Keras dựng mạng đó chỉ bằng năm dòng. Mỗi lớp `Dense` tự suy ra số cột của trọng số từ lớp trước, nên bạn chỉ khai báo số nút:

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    keras.Input(shape=(12288,)),                    # 64 x 64 x 3 điểm ảnh, đã
    ↪  duỗi thành một vector
    layers.Dense(20, activation="relu"),            # lớp ẩn 1: 20 nút
    layers.Dense(7, activation="relu"),             # lớp ẩn 2: 7 nút
    layers.Dense(5, activation="relu"),             # lớp ẩn 3: 5 nút
    layers.Dense(1, activation="sigmoid"),          # lớp ra: xác suất
])
print(model.count_params())                         # 245973
print([layer.count_params() for layer in model.layers])   # [245780, 147, 40, 6]
```

2.6.3 Lan truyền xuôi qua nhiều lớp

Lan truyền xuôi trong mạng sâu là **một vòng lặp qua các lớp**: lấy kích hoạt của lớp trước làm đầu vào, tính tổng có trọng số, áp hàm kích hoạt, và cứ thế cho đến lớp cuối. Các lớp ẩn dùng ReLU; lớp cuối dùng sigmoid để ra xác suất (Hình 2.38).



Hình 2.38: Lan truyền xuôi qua nhiều lớp. Mỗi lớp lưu lại một "bộ nhớ đệm" để lan truyền ngược dùng sau

Trong các mục trước, vector hoá giúp bỏ vòng lặp *qua các mẫu*. Ở đây, **vòng lặp qua các lớp thì không thể bỏ**, vì lớp sau cần kết quả của lớp trước. Đó là một vòng lặp tường minh duy nhất trong cả mạng, và nó chỉ lặp vài lần:

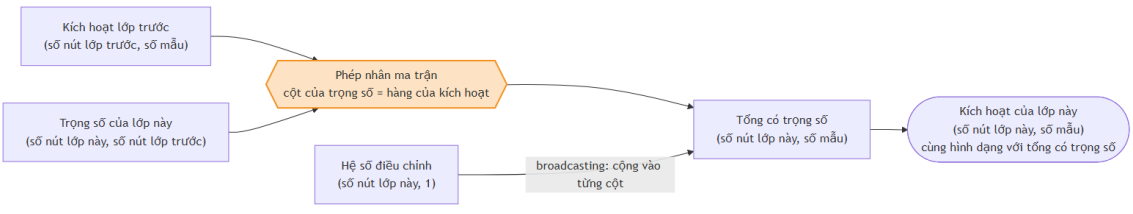
```
import numpy as np

layer_dims = [12288, 20, 7, 5, 1]                 # số nút của lớp 0 (vào) đến
↪  lớp 4 (ra)
rng = np.random.RandomState(1)
parameters = {}
for l in range(1, len(layer_dims)):               # khởi tạo từng lớp
    parameters["W" + str(l)] = rng.randn(layer_dims[l], layer_dims[l - 1]) /
    ↪  np.sqrt(layer_dims[l - 1])
    parameters["b" + str(l)] = np.zeros((layer_dims[l], 1))
print(sum(v.size for v in parameters.values()))    # 245973, khớp với Keras

X = rng.rand(12288, 6)                            # 6 mẫu giả, mỗi mẫu một cột
A = X                                             # kích hoạt của lớp 0 chính là
↪  dữ liệu
L = len(layer_dims) - 1                           # 4 lớp
for l in range(1, L + 1):
    Z = np.dot(parameters["W" + str(l)], A) + parameters["b" + str(l)]      #
    ↪  tổng có trọng số
    A = np.maximum(0, Z) if l < L else 1 / (1 + np.exp(-Z))                 #
    ↪  ReLU, riêng lớp cuối là sigmoid
    print("lớp", l, "| W", parameters["W" + str(l)].shape, "| Z", Z.shape, "| A",
    ↪  A.shape)
# lớp 1 | W (20, 12288) | Z (20, 6) | A (20, 6)
# lớp 2 | W (7, 20) | Z (7, 6) | A (7, 6)
# lớp 3 | W (5, 7) | Z (5, 6) | A (5, 6)
# lớp 4 | W (1, 5) | Z (1, 6) | A (1, 6)
```

2.6.3.1 Kiểm tra hình dạng: công cụ gỡ lỗi quan trọng nhất

Lỗi phổ biến nhất khi tự viết mạng sâu là ma trận sai hình dạng. Chỉ cần nhớ quy tắc ở Hình 2.39: ở mỗi lớp, trọng số có số cột bằng số hàng của kích hoạt lớp trước, và kết quả có số hàng bằng số nút của lớp đó, số cột bằng số mẫu.



Hình 2.39: Kiểm tra hình dạng ở một lớp: hai ma trận chỉ nhân được khi số cột của trọng số bằng số hàng của kích hoạt trước

Đại lượng	Hình dạng	Ghi chú
Trọng số của lớp	(số nút của lớp, số nút lớp trước)	độ dốc theo trọng số cũng có đúng hình dạng này
Hệ số điều chỉnh của lớp	(số nút của lớp, 1)	độ dốc theo hệ số điều chỉnh cũng vậy; broadcasting cộng vào từng cột
Tổng có trọng số và kích hoạt	(số nút của lớp, số mẫu)	luôn cùng hình dạng với nhau
Độ dốc theo kích hoạt	cùng hình dạng với kích hoạt	kích hoạt lớp trước cũng nhận độ dốc cùng hình dạng

Bảng 2.41: Các hình dạng cần kiểm tra mỗi khi nghi ngờ code

Nhờ quy tắc đó, lỗi hình dạng báo ngay chỗ sai: nếu nhân nhầm trọng số của lớp hai với dữ liệu thay vì kích hoạt của lớp một, NumPy dừng ngay với thông báo nói rõ hai chiều nào không khớp.

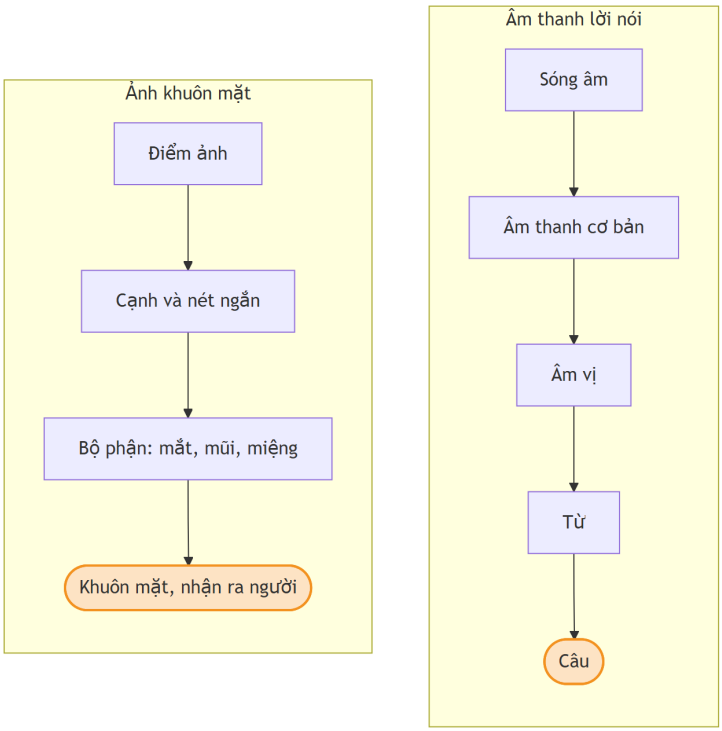
```
try:
    np.dot(parameters["W2"], X)                    # nhầm: lớp 2 phải nhân với
    ↪  kích hoạt của lớp 1, không phải dữ liệu
except ValueError as e:
    print(e)    # shapes (7,20) and (12288,6) not aligned: 20 (dim 1) != 12288
    ↪  (dim 0)
```

2.6.4 Vì sao mạng sâu hiệu quả

Nhiều bài toán chỉ cần mạng khá nông, nhưng mạng sâu thắng ở những bài toán khó. Có hai cách giải thích bổ sung cho nhau.

2.6.4.1 Các lớp học đặc trưng từ đơn giản đến phức tạp

Cách hình dung phổ biến: **các lớp đầu học thứ đơn giản, các lớp sau ghép chúng thành thứ phức tạp hơn** (Hình 2.40).



Hình 2.40: Mỗi lớp ghép kết quả của lớp trước thành đặc trưng phức tạp hơn

- **Ảnh**: nút ở lớp đầu nhìn một vùng rất nhỏ và phản ứng với cạnh theo một hướng; lớp sau gộp các cạnh thành bộ phận như mắt, mũi; lớp sau nữa gộp bộ phận thành khuôn mặt, và nhận ra khuôn mặt của ai.
- **Âm thanh**: lớp đầu bắt các đặc điểm thấp như âm cao hay thấp, âm trắng; lớp sau ghép thành âm vị (các đơn vị nhỏ nhất tạo nên từ); lớp sau nữa ghép thành từ, rồi cụm từ, câu.

Đây là cách hiểu giúp hình dung, không phải điều mọi mạng đều làm đúng như vậy. Nhưng nó cho thấy vì sao nhận dạng ảnh và giọng nói là nơi học sâu mạnh nhất: cả hai đều có cấu trúc "bộ phận nhỏ ghép thành tổng thể lớn". Ở chương về xử lý ảnh, ta sẽ thấy các lớp tích chập làm đúng việc này.

2.6.4.2 Mạng sâu gọn hơn mạng nông

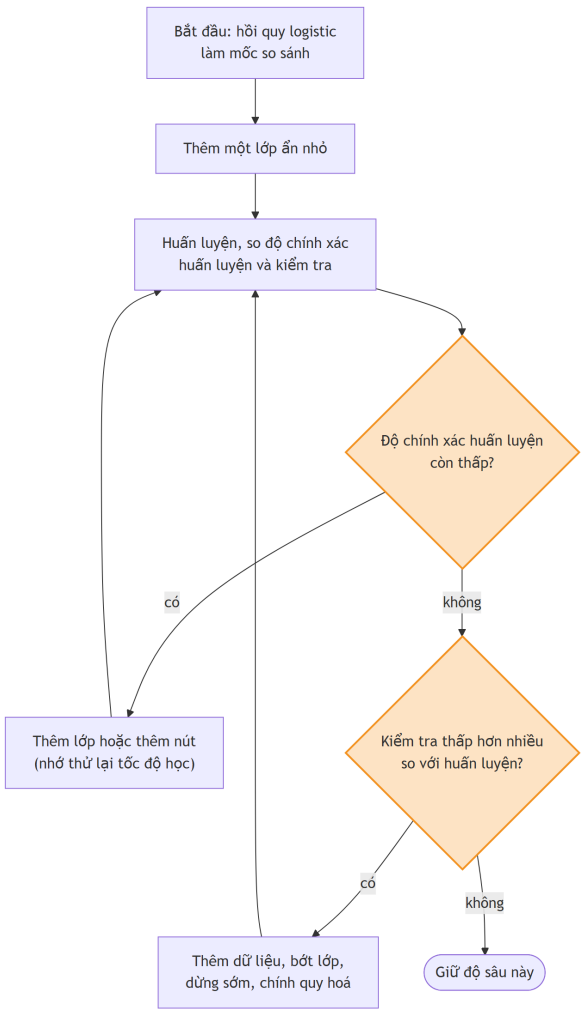
Cách giải thích thứ hai đến từ lý thuyết mạch: **có những hàm mà mạng sâu và nhỏ tính được, còn mạng nông phải rất lớn mới tính nổi**. Ví dụ kinh điển là tính xem một dãy bit có số lượng bit 1 lẻ hay chẵn (phép XOR của mọi bit):

- Với mạng sâu, ghép các bit thành cặp, tính XOR từng cặp, rồi ghép các kết quả thành cặp tiếp, theo dạng cây. Số cổng cần tính chỉ tăng cùng tỷ lệ với số bit, còn độ sâu tăng rất chậm (mỗi khi số bit gấp đôi chỉ thêm một lớp).
- Với đúng một lớp ẩn, mạng phải liệt kê hầu hết các tổ hợp của các bit, nên số nút ẩn tăng gấp đôi mỗi khi thêm một bit đầu vào.

Cách tính chung không đổi: chia bài toán lớn thành các bài nhỏ, giải từng phần rồi ghép lại. Trong thực tế, bạn không chứng minh điều này cho từng bài toán; chỉ cần nhớ lý do vì sao "thêm lớp" thường hiệu quả hơn "thêm nút vào một lớp".

Về tên gọi. Cụm "học sâu" (deep learning) một phần là thương hiệu cho ý tưởng cũ: mạng nơ-ron có nhiều lớp ẩn. Tên gọi này gây chú ý hơn, nhưng bản chất vẫn là mạng nơ-ron.

Đừng bắt đầu quá sâu. Lý thuyết nói mạng sâu *có thể* gọn hơn, không nói bài toán nào cũng cần sâu. Quy trình ở Hình 2.41 bắt đầu từ mô hình đơn giản rồi mới tăng dần độ sâu.

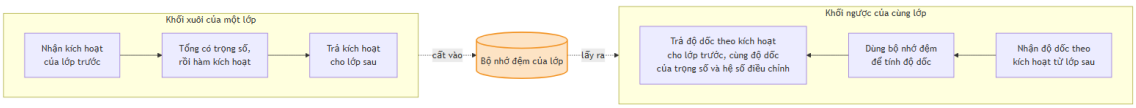


Hình 2.41: Chọn độ sâu: bắt đầu từ mô hình đơn giản và chỉ tăng khi độ chính xác huấn luyện còn thấp

2.6.5 Lan truyền ngược bằng các khối xây dựng

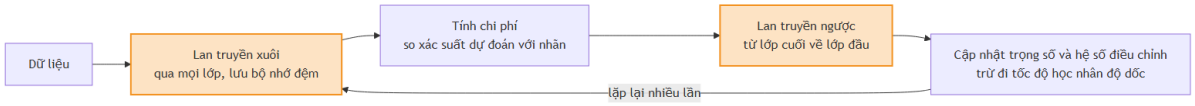
2.6.5.1 Mỗi lớp là một khối xuôi và một khối ngược

Cách tổ chức code cho mạng sâu: **mỗi lớp có một khối xuôi và một khối ngược**, nối với nhau bằng một "bộ nhớ đệm" (cache). Khối xuôi nhận kích hoạt của lớp trước, tính kích hoạt của lớp này, và *cất vào bộ nhớ đệm* các giá trị cần sau này: đầu vào của lớp, trọng số, hệ số điều chỉnh và tổng có trọng số. Khối ngược nhận độ dốc theo kích hoạt từ lớp sau, *lấy lại bộ nhớ đệm đó* để tính độ dốc cho lớp này, rồi chuyển độ dốc tiếp cho lớp trước (Hình 2.42).



Hình 2.42: Một lớp gồm khối xuôi, khối ngược và bộ nhớ đệm nối giữa chúng

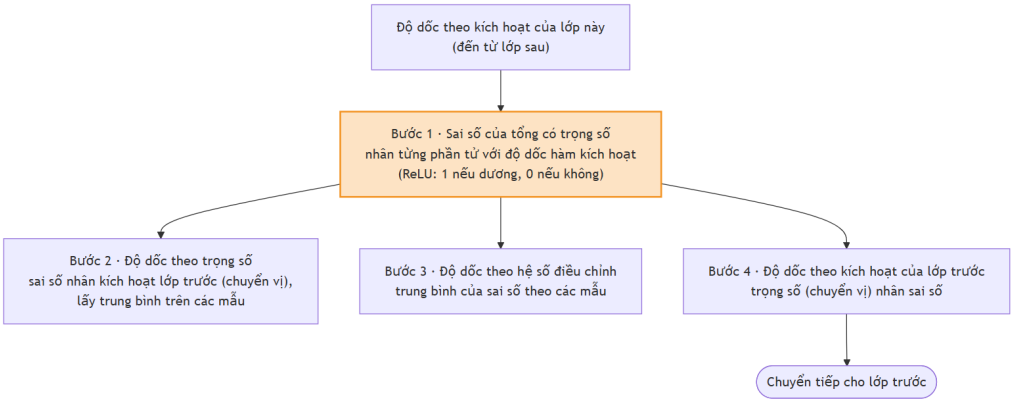
Ghép các khối lại, một bước huấn luyện đi theo vòng như Hình 2.43: lan truyền xuôi từ lớp một đến lớp cuối, tính chi phí, lan truyền ngược từ lớp cuối về lớp một, rồi cập nhật tham số.



Hình 2.43: Một bước huấn luyện của mạng sâu, lặp lại nhiều lần

2.6.5.2 Bốn bước của khối ngược

Mỗi khối ngược làm đúng bốn việc (Hình 2.44), và nhận ra ngay vì chúng giống lan truyền ngược của mạng một lớp ẩn ở mục 2.4.5. Khác biệt duy nhất là chúng được lặp lại cho mọi lớp.



Hình 2.44: Bốn bước của khối ngược ở một lớp

Bước	Phép tính (bằng lời)	Tên trong code
1	sai số của tổng có trọng số: độ dốc theo kích hoạt nhân *từng phần tử* với độ dốc của hàm kích hoạt (ReLU: giữ nguyên nơi tổng dương, đặt 0 nơi tổng không dương)	`dZ`
2	độ dốc theo trọng số: nhân sai số với kích hoạt lớp trước (chuyển vị), rồi lấy trung bình theo số mẫu	`dW`
3	độ dốc theo hệ số điều chỉnh: trung bình của sai số theo số mẫu	`db`
4	độ dốc theo kích hoạt lớp trước: nhân trọng số (chuyển vị) với sai số	`dA_prev`

Bảng 2.42: Bốn bước của khối ngược và tên biến tương ứng

2.6.5.3 Điểm xuất phát của lan truyền ngược

Vòng lùi cần một điểm bắt đầu: độ dốc của chi phí theo kích hoạt của lớp cuối, tức theo xác suất dự đoán. Với cross-entropy nhị phân, độ dốc đó bằng *trừ* nhãn chia xác suất dự đoán, *cộng* một trừ nhãn chia một trừ xác suất dự đoán, tính riêng cho từng mẫu. Từ đó lớp cuối dùng khối ngược với sigmoid, mọi lớp ẩn còn lại dùng khối ngược với ReLU, đi từ lớp cuối về lớp một:

```
def L_model_backward(AL, Y, caches):
    grads = {}
    L = len(caches)                                    # số lớp
    Y = Y.reshape(AL.shape)
    dAL = -(np.divide(Y, AL) - np.divide(1 - Y, 1 - AL))        # điểm xuất phát:
    ↪  độ dốc theo xác suất dự đoán
    # lớp cuối: khối ngược với sigmoid
```

```
    dA_prev, grads["dW" + str(L)], grads["db" + str(L)] =
    ↪  linear_activation_backward(dAL, caches[L - 1], "sigmoid")
    # các lớp ẩn: khối ngược với ReLU, đi ngược từ lớp L-1 về lớp 1
    for l in reversed(range(L - 1)):
        dA_prev, grads["dW" + str(l + 1)], grads["db" + str(l + 1)] =
        ↪  linear_activation_backward(dA_prev, caches[l], "relu")
    return grads
```

Đoạn này chỉ minh hoạ cấu trúc; mục 2.7 chạy cả bộ hàm trợ giúp. Trong dự án, Keras tự làm tất cả các bước này khi bạn gọi `fit`.

2.6.6 Tham số và siêu tham số

Tham số (parameters) là thứ mô hình tự học: trọng số và hệ số điều chỉnh của mọi lớp. **Siêu tham số** (hyperparameters) là thứ *bạn chọn* trước khi huấn luyện, và chúng quyết định tham số cuối cùng học được ra sao (Hình 2.45).

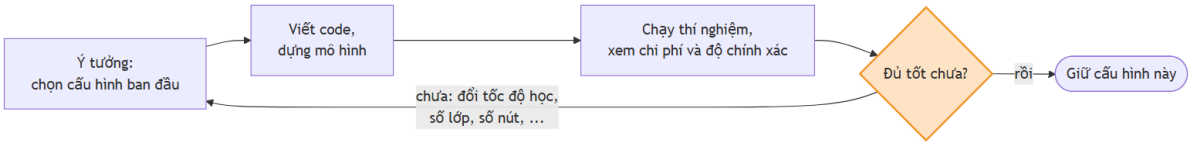


Hình 2.45: Siêu tham số do bạn chọn; tham số do mô hình học

Siêu tham số	Trong Keras	Ảnh hưởng
Tốc độ học	`learning_rate` của bộ tối ưu	quá nhỏ: học chậm; quá lớn: chi phí dao động hoặc tăng
Số lần lặp	`epochs` (kèm `batch_size`)	quá ít: chưa học đủ; quá nhiều: có thể quá khớp
Số lớp ẩn	số dòng `Dense`	mạng càng sâu càng mạnh nhưng khó huấn luyện hơn
Số nút mỗi lớp ẩn	đối số đầu của `Dense`	càng nhiều càng linh hoạt, tốn tính toán hơn
Hàm kích hoạt	`activation`	ReLU là mặc định phổ biến cho lớp ẩn

Bảng 2.43: Các siêu tham số chính của mạng sâu

Học sâu là một quá trình thực nghiệm. Không có công thức chọn đúng siêu tham số ngay từ đầu. Quy trình thực tế (Hình 2.46) là vòng lặp: đưa ra một ý tưởng về cấu hình, viết code, chạy thí nghiệm, xem kết quả, rồi quay lại sửa ý tưởng. Lần đầu thử tốc độ học 0,01 mà không ổn thì thử 0,001; xem chi phí giảm nhanh hay chậm rồi điều chỉnh.



Hình 2.46: Chọn siêu tham số là vòng lặp ý tưởng, code, thí nghiệm

Hai kinh nghiệm quen thuộc:

- Cấu hình tốt hôm nay có thể không còn tốt sau vài tháng, khi dữ liệu thay đổi hoặc phần cứng khác đi. Hãy thử lại siêu tham số định kỳ trên tập kiểm định.

- Các lĩnh vực khác nhau (ảnh, giọng nói, quảng cáo, tìm kiếm) đều dùng cùng cách làm này, và kinh nghiệm ở lĩnh vực này đôi khi chuyển sang lĩnh vực khác, nhưng không đảm bảo. Cứ thử và đo.

2.6.7 Khởi tạo cho mạng sâu

Mục 2.4.6 đã nói: hằng số nhỏ như 0,01 hợp với mạng nông, còn mạng rất sâu có thể cần hằng số khác. Lý do: qua mỗi lớp, tín hiệu bị nhân với trọng số. Nếu trọng số quá nhỏ, tín hiệu và độ dốc co dần về 0 sau nhiều lớp; nếu quá lớn, chúng phình to. Vì vậy cách khởi tạo càng quan trọng khi mạng càng sâu.

Cách thông dụng: **chia số ngẫu nhiên Gauss cho căn bậc hai của số nút lớp trước** thay vì nhân với 0,01. Lớp nào có nhiều đầu vào thì trọng số của nó nhỏ hơn, nhờ vậy tổng có trọng số giữ được độ lớn ổn định qua các lớp. Mã khởi tạo ở trên đã dùng cách này. Mục 2.7 dùng cách này cho mạng bốn lớp, còn mạng hai lớp dùng hằng số 0,01. Trong Keras, các tên gọi phổ biến là `glorot_uniform` (mặc định) và `he_normal` (dành cho ReLU), nên thường không cần tự khởi tạo.

2.6.8 Về phép so sánh với bộ não

Bạn sẽ thấy nhiều bài viết ví mạng nơ-ron với bộ não: neuron sinh học nhận tín hiệu, cộng lại, rồi "bắn" đi nếu đủ mạnh, giống nút của mạng nhân trọng số, cộng và kích hoạt. Phép so sánh này **rất lỏng lẻo**. Neuron sinh học phức tạp hơn nhiều so với một phép nhân ma trận kèm ReLU, và các nhà khoa học thần kinh vẫn chưa hiểu hết một neuron làm gì. Cũng chưa rõ não có học bằng thứ gì giống lan truyền ngược hay không. Dùng phép so sánh để ghi nhớ thì được, đừng dùng nó để suy luận xem mạng sẽ hành xử thế nào.

2.6.9 Áp dụng vào dự án thực tế

2.6.9.1 Quy trình dựng mạng sâu

1. Chuẩn hoá dữ liệu (ảnh: chia 255) và xếp mỗi mẫu một hàng cho Keras.
2. Dựng `Sequential` với các lớp `Dense`: ReLU cho lớp ẩn, sigmoid cho lớp ra của bài toán hai lớp.
3. Chạy mô hình *đơn giản nhất* (hồi quy logistic) làm mốc, rồi thêm lớp ẩn.
4. Theo dõi chi phí và độ chính xác của cả tập huấn luyện lẫn tập kiểm định sau mỗi epoch.
5. Chỉnh siêu tham số theo vòng lặp trong Hình 2.46; chỉ đổi một thứ mỗi lần.

2.6.9.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Lỗi "shapes not aligned" khi nhân ma trận	hình dạng trọng số hoặc kích hoạt sai; in hình dạng ở từng lớp (Bảng 2.41)
Chi phí giảm rất chậm hoặc đứng yên ở mạng sâu khi dùng khởi tạo 0,01	thử khởi tạo co theo số nút lớp trước (mục 2.6.7)
Chi phí tăng hoặc dao động	tốc độ học quá lớn; giảm bớt
Độ chính xác huấn luyện cao, kiểm tra thấp hơn nhiều	quá khớp; thêm dữ liệu, giảm cỡ mạng, dừng sớm
Độ chính xác huấn luyện thấp	mạng quá nhỏ hoặc huấn luyện chưa đủ; tăng nút, lớp, số lần lặp
Tự viết lan truyền ngược nhưng không chắc đúng	so với phép đo độ dốc bằng cách đẩy nhẹ từng tham số (mục 2.7)

Bảng 2.44: Lỗi thường gặp khi huấn luyện mạng sâu

2.6.9.3 Bảng tra cứu API

Việc cần làm	Keras
Dựng mạng nhiều lớp	`keras.Sequential([keras.Input(...),layers.Dense(n, activation="relu"),...])`
Lớp ra cho bài toán hai lớp	`layers.Dense(1,activation="sigmoid")`
Đếm tham số	`model.count_params()`, `model.summary()`
Chọn hàm mất mát, bộ tối ưu	`model.compile(optimizer=..., loss="binary_crossentropy")`
Huấn luyện kèm tập kiểm định	`model.fit(X,y,epochs=...,validation_data=...)`
Đổi cách khởi tạo	`Dense(...,kernel_initializer="he_normal")`
Lấy trọng số của từng lớp	`layer.get_weights()`

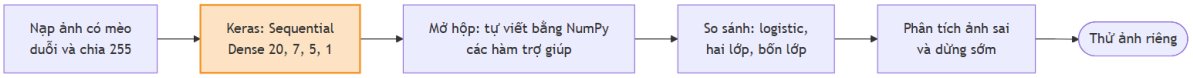
Bảng 2.45: Các lệnh Keras hay dùng cho mạng sâu

2.6.9.4 Tóm tắt cần nhớ

- Nông và sâu là chuyện nhiều hay ít lớp ẩn; hồi quy logistic là mạng một lớp, và lớp vào không được đếm.
- Mạng sâu lặp lại cùng hai bước "tổng có trọng số rồi kích hoạt"; vòng lặp qua các lớp là vòng lặp tường minh duy nhất.
- Trọng số mỗi lớp có số hàng bằng số nút của lớp đó, số cột bằng số nút lớp trước; mọi độ dốc có cùng hình dạng với đại lượng của nó.
- Mạng sâu giúp học đặc trưng từ đơn giản đến phức tạp và có thể gọn hơn mạng nông cho một số hàm.
- Mỗi lớp gồm khối xuôi, khối ngược, và bộ nhớ đệm cất những gì khối ngược cần.
- Siêu tham số do bạn chọn qua vòng lặp ý tưởng, code, thí nghiệm; tham số do mô hình tự học.
- Khởi tạo cho mạng sâu: chia cho căn bậc hai của số nút lớp trước, đừng chỉ nhân 0,01.
- Phép so sánh với bộ não chỉ để ghi nhớ, không dùng để suy luận.

2.7 Thực hành: mạng sâu nhận dạng ảnh có mèo

Mục này quay lại bài toán ảnh có mèo ở mục 2.3.2, nơi hồi quy logistic chỉ đạt 70% trên tập kiểm tra. Câu hỏi: **thêm các lớp ẩn có cải thiện được không?** Cách làm theo thứ tự quen thuộc: dựng mạng bằng Keras trước, rồi mở hộp bằng NumPy để thấy chính xác các hàm bên trong làm gì, rồi mới so sánh các mô hình, bàn về dừng sớm và khởi tạo, và phân tích ảnh sai (Hình 2.47). Các con số trong mục này là kết quả của chính bài thực hành gốc (cùng dữ liệu, cùng kiến trúc, cùng siêu tham số); mục không tự đo thêm số liệu nào khác.



Hình 2.47: Quy trình thực hành: Keras trước, NumPy để mở hộp, rồi so sánh và phân tích lỗi

2.7.1 Nạp dữ liệu

Dữ liệu và cách xử lý giống hệt mục 2.3.2: 209 ảnh huấn luyện, 50 ảnh kiểm tra, mỗi ảnh 64 nhân 64 điểm ảnh ba kênh màu được duỗi thành cột 12288 giá trị và chia cho 255. Chạy trong thư mục có chứa `datasets`.

```
import numpy as np
import h5py

def load_dataset():
    with h5py.File("datasets/train_catvnoncat.h5", "r") as f:
        train_x = np.array(f["train_set_x"][:])
        train_y = np.array(f["train_set_y"][:])
    with h5py.File("datasets/test_catvnoncat.h5", "r") as f:
        test_x = np.array(f["test_set_x"][:])
        test_y = np.array(f["test_set_y"][:])
        classes = np.array(f["list_classes"][:])
    return train_x, train_y.reshape(1, -1), test_x, test_y.reshape(1, -1),
    ↪  classes

train_x_orig, train_y, test_x_orig, test_y, classes = load_dataset()
train_x = train_x_orig.reshape(train_x_orig.shape[0], -1).T / 255.     # (12288,
↪  209): mỗi ảnh một cột
test_x = test_x_orig.reshape(test_x_orig.shape[0], -1).T / 255.        # (12288,
↪  50)
print(train_x.shape, train_y.shape, test_x.shape, test_y.shape)
# (12288, 209) (1, 209) (12288, 50) (1, 50)
```

2.7.2 Mạng bốn lớp bằng Keras

Mạng cần dựng là mạng ở Hình 2.37: ba lớp ẩn ReLU (20, 7, 5 nút) và một nút ra sigmoid. Thiết lập huấn luyện tương ứng với phần NumPy bên dưới: gradient descent thuần (`SGD`), tốc độ học 0,0075, cả tập trong một lô, 2500 lần lặp. Đoạn dưới đây chỉ dựng và huấn luyện; bài thực hành gốc báo kết quả cho phiên bản NumPy ở mục sau, nên con số được nêu ở đó.

```
from tensorflow import keras
from tensorflow.keras import layers

def keras_model(hidden=(20, 7, 5)):
    return keras.Sequential(
        [keras.Input(shape=(12288,))] +
```

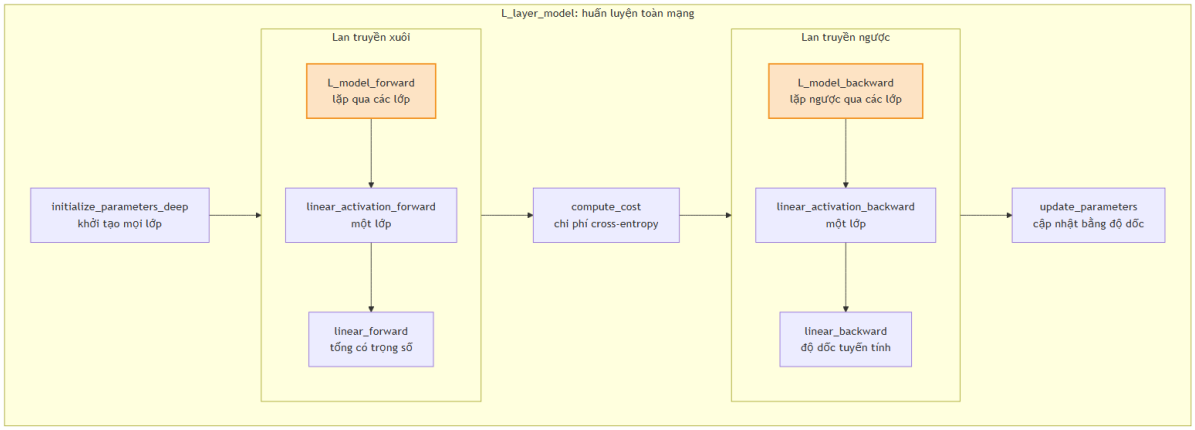
```
        [layers.Dense(n, activation="relu") for n in hidden] +
        [layers.Dense(1, activation="sigmoid")])

km = keras_model()
print(km.count_params())                                    # 245973
km.compile(optimizer=keras.optimizers.SGD(learning_rate=0.0075),
↪  loss="binary_crossentropy", metrics=["accuracy"])
km.fit(train_x.T, train_y.T, epochs=2500, batch_size=train_x.shape[1], verbose=0)
↪  # mỗi epoch là một bước trên cả tập
print(km.evaluate(test_x.T, test_y.T, verbose=0))      # [chi phí, độ chính xác]
↪  trên tập kiểm tra
```

Lưu ý: Keras cần mẫu ở *hàng* nên truyền `.T`, và cách khởi tạo mặc định của Keras không giống cách chia cho căn bậc hai dùng ở phần NumPy, nên kết quả của hai bên không nhất thiết giống nhau từng số.

2.7.3 Mở hộp: các hàm trợ giúp bằng NumPy

Để thấy Keras làm gì, ta tự viết các hàm của mục 2.6.5. Hình 2.48 cho thấy chúng gọi nhau như thế nào: hàm huấn luyện toàn mạng gọi hàm lan truyền xuôi, hàm chi phí, hàm lan truyền ngược và hàm cập nhật; mỗi hàm lặp qua các lớp bằng cách gọi hàm cho một lớp, và hàm cho một lớp lại gọi hàm tuyến tính bên dưới.



Hình 2.48: Bản đồ các hàm trợ giúp. Mỗi cặp xuôi và ngược chia nhau cùng một bộ nhớ đệm

2.7.3.1 Khởi tạo và lan truyền xuôi

```
def relu(Z):
    return np.maximum(0, Z)

def sigmoid(Z):
    return 1 / (1 + np.exp(-Z))

def initialize_parameters_deep(layer_dims, scale="sqrt", seed=1):
    """scale="sqrt": chia cho căn bậc hai số nút lớp trước; hoặc truyền một số,
    ↪  ví dụ 0.01."""
    rng = np.random.RandomState(seed)
    parameters = {}
    for l in range(1, len(layer_dims)):
        s = 1 / np.sqrt(layer_dims[l - 1]) if scale == "sqrt" else scale
        parameters["W" + str(l)] = rng.randn(layer_dims[l], layer_dims[l - 1]) *
        ↪  s
        parameters["b" + str(l)] = np.zeros((layer_dims[l], 1))
```

```
    return parameters

def linear_forward(A, W, b):
    Z = np.dot(W, A) + b                            # tổng có trọng số
    return Z, (A, W, b)                             # kèm thứ cần cất cho lan
    ↪ truyền ngược

def linear_activation_forward(A_prev, W, b, activation):
    Z, linear_cache = linear_forward(A_prev, W, b)
    A = relu(Z) if activation == "relu" else sigmoid(Z)
    return A, (linear_cache, Z)                     # bộ nhớ đệm của lớp: đầu vào,
    ↪ trọng số, hệ số điều chỉnh, tổng

def L_model_forward(X, parameters):
    caches, A = [], X
    L = len(parameters) // 2                        # số lớp
    for l in range(1, L):                           # các lớp ẩn: ReLU
        A, cache = linear_activation_forward(A, parameters["W" + str(l)],
        ↪ parameters["b" + str(l)], "relu")
        caches.append(cache)
    AL, cache = linear_activation_forward(A, parameters["W" + str(L)],
    ↪ parameters["b" + str(L)], "sigmoid")
    caches.append(cache)                            # lớp cuối: sigmoid
    return AL, caches

def compute_cost(AL, Y):
    m = Y.shape[1]
    return float(-np.sum(Y * np.log(AL) + (1 - Y) * np.log(1 - AL)) / m)    #
    ↪ cross-entropy nhị phân
```

2.7.3.2 Lan truyền ngược và cập nhật

```
def linear_backward(dZ, linear_cache):
    A_prev, W, b = linear_cache
    m = A_prev.shape[1]
    dW = np.dot(dZ, A_prev.T) / m                   # bước 2: độ dốc theo trọng số
    db = np.sum(dZ, axis=1, keepdims=True) / m      # bước 3: độ dốc theo hệ số
    ↪ điều chỉnh
    dA_prev = np.dot(W.T, dZ)                       # bước 4: độ dốc cho lớp trước
    return dA_prev, dW, db

def linear_activation_backward(dA, cache, activation):
    linear_cache, Z = cache
    if activation == "relu":
        dZ = dA * (Z > 0)                           # bước 1: ReLU giữ nơi tổng
        ↪ dương, đặt 0 nơi còn lại
    else:
        s = sigmoid(Z)
        dZ = dA * s * (1 - s)                       # bước 1 với sigmoid: độ dốc
        ↪ là kích hoạt nhân (1 trừ kích hoạt)
    return linear_backward(dZ, linear_cache)

def L_model_backward(AL, Y, caches):
    grads = {}
    L = len(caches)
    Y = Y.reshape(AL.shape)
    dAL = -(np.divide(Y, AL) - np.divide(1 - Y, 1 - AL))        # điểm xuất phát
    ↪ của vòng lùi
```

```
    dA_prev, grads["dW" + str(L)], grads["db" + str(L)] =
    ↪  linear_activation_backward(dAL, caches[L - 1], "sigmoid")
    for l in reversed(range(L - 1)):
        dA_prev, grads["dW" + str(l + 1)], grads["db" + str(l + 1)] =
        ↪  linear_activation_backward(dA_prev, caches[l], "relu")
    return grads

def update_parameters(parameters, grads, learning_rate):
    new = dict(parameters)
    for l in range(1, len(parameters) // 2 + 1):
        new["W" + str(l)] = parameters["W" + str(l)] - learning_rate * grads["dW"
        ↪  + str(l)]
        new["b" + str(l)] = parameters["b" + str(l)] - learning_rate * grads["db"
        ↪  + str(l)]
    return new
```

2.7.3.3 Ghép thành mô hình

Hàm huấn luyện chỉ ghép các bước ở Hình 2.43. Với `print_cost=True`, nó in chi phí mỗi 100 lần lặp.

```
def predict(X, Y, parameters):
    AL, _ = L_model_forward(X, parameters)
    p = (AL > 0.5).astype(float)                    # ngưỡng 0,5
    return p, float(np.mean(p == Y))

def L_layer_model(X, Y, layers_dims, learning_rate=0.0075, num_iterations=2500,
↪  scale="sqrt", seed=1, print_cost=False):
    parameters = initialize_parameters_deep(layers_dims, scale, seed)
    costs = []
    for i in range(num_iterations):
        AL, caches = L_model_forward(X, parameters)
        cost = compute_cost(AL, Y)
        grads = L_model_backward(AL, Y, caches)
        parameters = update_parameters(parameters, grads, learning_rate)
        if i % 100 == 0 or i == num_iterations - 1:
            costs.append((i, cost))
            if print_cost:
                print("Cost after iteration {}: {}".format(i, cost))
    return parameters, costs
```

2.7.3.4 Kiểm tra độ dốc

Trước khi tin các hàm ngược, kiểm tra bằng cách đẩy nhẹ từng tham số (mục 2.4.5). Mạng nhỏ với 4 đầu vào, hai lớp ẩn (5 và 3 nút) và một nút ra là đủ; mọi độ dốc theo trọng số và hệ số điều chỉnh đều được so. Nếu các hàm ngược đúng, độ lệch lớn nhất rất nhỏ (gần 0).

```
rs = np.random.RandomState(0)
Xs, Ys = rs.randn(4, 12), (rs.rand(1, 12) > 0.5).astype(float)
ps = initialize_parameters_deep([4, 5, 3, 1], scale=0.5, seed=2)
AL, caches = L_model_forward(Xs, ps)
g = L_model_backward(AL, Ys, caches)
worst = 0.0
for k in ps:
    num = np.zeros_like(ps[k])
    for idx in np.ndindex(*ps[k].shape):
        old = ps[k][idx]
        ps[k][idx] = old + 1e-6
```

```
        c1 = compute_cost(L_model_forward(Xs, ps)[0], Ys)
        ps[k][idx] = old - 1e-6
        c2 = compute_cost(L_model_forward(Xs, ps)[0], Ys)
        ps[k][idx] = old
        num[idx] = (c1 - c2) / 2e-6                # độ dốc đo trực tiếp
    worst = max(worst, float(np.max(np.abs(num - g["d" + k]))))
print(worst)                                       # rất nhỏ: độ dốc tính tay
↪  khớp với độ dốc đo
```

2.7.4 So sánh hồi quy logistic, mạng hai lớp và mạng bốn lớp

Chạy hai mô hình bằng cùng hàm huấn luyện: mạng hai lớp (một lớp ẩn 7 nút, trọng số khởi tạo nhân 0,01) và mạng bốn lớp (ba lớp ẩn 20, 7, 5 nút, khởi tạo chia căn bậc hai). Cả hai dùng tốc độ học 0,0075 và 2500 lần lặp.

```
p2, c2 = L_layer_model(train_x, train_y, [12288, 7, 1], scale=0.01,
↪  print_cost=True)
p4, c4 = L_layer_model(train_x, train_y, [12288, 20, 7, 5, 1], scale="sqrt",
↪  print_cost=True)

for ten, p in (("hai lớp", p2), ("bốn lớp", p4)):
    print(ten, "| huấn luyện", predict(train_x, train_y, p)[1], "| kiểm tra",
    ↪  predict(test_x, test_y, p)[1])
```

Bảng 2.46 tóm tắt kết quả theo bài thực hành gốc.

Mô hình	Số tham số	Chi phí cuối	Chính xác huấn luyện	Chính xác kiểm tra
Hồi quy logistic (mục 2.3.2)	12 289	—	—	70%
Mạng hai lớp (một lớp ẩn, 7 nút)	86 031	0,0442	gần 100%	72%
Mạng bốn lớp (ba lớp ẩn: 20, 7, 5 nút)	245 973	0,0884	98,6%	80%

Bảng 2.46: Ba mô hình trên cùng bộ ảnh có mèo (tốc độ học 0,0075, 2500 lần lặp cho hai mạng). Dấu — là chỗ bài thực hành gốc không nêu số

Cách đọc kết quả, theo thứ tự đáng tin cậy:

- **Mạng bốn lớp tốt nhất trên tập kiểm tra** (80%, so với 72% và 70%), dù độ chính xác huấn luyện không cao hơn mạng hai lớp. Đó là điều ta muốn: khớp tốt dữ liệu chưa thấy, không phải dữ liệu đã thấy.
- **Chi phí thấp hơn không có nghĩa là mô hình tốt hơn.** Mạng hai lớp có chi phí cuối thấp hơn (0,044 so với 0,088) nhưng kém hơn ở tập kiểm tra. Chi phí đo mức khớp với tập huấn luyện; độ chính xác kiểm tra mới cho biết mức tổng quát hoá.
- **Cẩn thận với tập kiểm tra nhỏ.** Chỉ có 50 ảnh, mỗi ảnh là 2 điểm phần trăm, nên chênh lệch 80% với 72% là 4 ảnh. Mốc so sánh cần nhớ là 66%: luôn đoán "có mèo" đã đạt tỷ lệ đó trên tập kiểm tra này.
- **Còn nhiều chỗ để cải thiện.** Theo bài thực hành gốc, tìm kiếm có hệ thống các siêu tham số (tốc độ học, số lớp và số nút, số lần lặp) có thể cho độ chính xác cao hơn nữa; đó là nội dung của chương sau.

2.7.4.1 Chi phí theo số lần lặp

Bảng 2.47 là chi phí do bài thực hành gốc in ra khi huấn luyện mạng hai lớp. Chi phí giảm đều: chậm ở đầu rồi nhanh hơn, và sau 2500 lần lặp còn rất thấp. Với mạng bốn lớp, bài gốc chỉ nêu ba mốc.

Số lần lặp	Mạng hai lớp	Mạng bốn lớp
0	0,6930	0,7717
100	0,6464	0,6721
200	0,6325	—
500	0,5158	—
1000	0,3394	—
1300	0,2468	—
2499 (cuối)	0,0442	0,0884

Bảng 2.47: Chi phí theo số lần lặp (theo bài thực hành gốc). Dấu — là mốc không được nêu

2.7.4.2 Dừng sớm

Bài thực hành gốc lưu ý rằng chạy mạng hai lớp với *ít lần lặp hơn* (khoảng 1500) có thể cho độ chính xác kiểm tra tốt hơn chạy đủ 2500 lần. Đây là **dừng sớm** (early stopping): dừng huấn luyện khi độ chính xác trên tập kiểm định không còn tăng, vì huấn luyện lâu hơn chỉ làm mô hình khớp tập huấn luyện sát hơn. Cách làm đúng trong dự án là dùng một tập kiểm định riêng để chọn điểm dừng, rồi mới đo bằng tập kiểm tra. Keras hỗ trợ sẵn:

```
early = keras.callbacks.EarlyStopping(monitor="val_loss", patience=50,
↪  restore_best_weights=True)
km.fit(train_x.T, train_y.T, epochs=2500, batch_size=train_x.shape[1],
↪  validation_split=0.2,
      callbacks=[early], verbose=0)              # dừng khi chi phí kiểm định
       ↪  không giảm trong 50 epoch liên tiếp
```

2.7.4.3 Khởi tạo: bài học của mạng sâu

Hai mạng ở trên dùng hai cách khởi tạo khác nhau, và sự khác biệt có thể đọc ngay từ chi phí ở lần lặp đầu (Bảng 2.47). Với trọng số nhân 0,01, đầu ra của mạng hai lớp gần như luôn là 0,5 nên chi phí ban đầu đúng bằng 0,6930, tức mức của việc không biết gì. Mạng bốn lớp, khởi tạo bằng cách chia cho căn bậc hai số nút lớp trước, bắt đầu ở 0,7717.

Như đã nói ở mục 2.4.6, hằng số 0,01 phù hợp mạng nông, còn mạng nhiều lớp có thể cần hằng số khác vì tín hiệu bị nhân với trọng số qua từng lớp. Cách khởi tạo chia cho căn bậc hai số nút lớp trước giữ độ lớn của tín hiệu ổn định hơn qua các lớp. Tài liệu này không đo thêm thí nghiệm so sánh trực tiếp hai cách trên mạng bốn lớp; hãy xem đây là quy tắc thực hành khuyến nghị, và nếu nghi ngờ thì thử cả hai trên tập kiểm định của bạn.

2.7.4.4 Đối chiếu với Keras

Cách kiểm tra Keras và bản mở hộp tính cùng một thứ: đặt đúng các trọng số NumPy vừa học vào mô hình Keras cùng kiến trúc rồi so dự đoán. Nếu hai bên khớp, đoạn dưới in `True`.

```
km3 = keras_model()
w = []
for l in range(1, 5):
    w += [p4["W" + str(l)].T, p4["b" + str(l)].ravel()]          # Keras lưu
     ↪  trọng số chuyển vị, hệ số điều chỉnh dạng phẳng
```

```
km3.set_weights(w)
print(np.allclose(km3.predict(test_x.T, verbose=0).T, L_model_forward(test_x,
↪  p4)[0], atol=1e-5))    # True nếu khớp
```

2.7.5 Phân tích ảnh bị đoán sai

Việc đầu tiên khi độ chính xác chưa đủ là **nhìn tận mắt các ảnh bị đoán sai**. Đoạn dưới lấy chỉ số các ảnh kiểm tra bị sai và đếm theo hai loại lỗi: ảnh không mèo bị đoán có mèo, và ảnh có mèo bị bỏ sót.

```
pred_te, acc_te = predict(test_x, test_y, p4)
sai = np.where(pred_te != test_y)[1]                        # chỉ số các ảnh bị
↪  đoán sai
print(len(sai), sai.tolist())
print("thật không mèo bị đoán có mèo:", int(np.sum((pred_te == 1) & (test_y ==
↪  0))))
print("thật có mèo bị đoán không mèo:", int(np.sum((pred_te == 0) & (test_y ==
↪  1))))
```

Bài thực hành gốc liệt kê những kiểu ảnh mà mô hình hay đoán sai (Bảng 2.48); đối chiếu với từng ảnh sai để xem lỗi thuộc nhóm nào, rồi bổ sung dữ liệu cho đúng nhóm đó.

Kiểu ảnh hay bị đoán sai	Cách xử lý gợi ý
Thân mèo ở tư thế bất thường	thu thêm ảnh mèo ở nhiều tư thế
Mèo nằm trên nền có màu gần giống	thu thêm ảnh có nền phức tạp, tương phản thấp
Màu lông hoặc giống mèo khác thường	thu thêm ảnh nhiều màu, nhiều giống
Góc chụp lạ	thu thêm ảnh ở nhiều góc
Độ sáng của ảnh	thêm ảnh sáng và tối, hoặc biến đổi độ sáng khi huấn luyện
Tỷ lệ: mèo quá lớn hoặc quá nhỏ trong ảnh	thu thêm ảnh ở nhiều khoảng cách

Bảng 2.48: Các kiểu ảnh mô hình hay đoán sai (theo bài thực hành gốc) và hướng xử lý

Ngoài ra nên hỏi thêm: nhãn của ảnh có bị gán sai không (sửa nhãn trước khi huấn luyện lại), lỗi có nghiêng hẳn về một lớp không (xem lại cân bằng nhãn giữa tập huấn luyện và tập kiểm tra, vốn khác nhau nhiều ở bộ dữ liệu này), và độ chính xác huấn luyện có cao hơn nhiều so với kiểm tra không (quá khớp: thêm dữ liệu, giảm cỡ mạng, dừng sớm).

2.7.5.1 Thử với ảnh riêng

Giống mục 2.3.2, ảnh mới cần đúng cùng cách tiền xử lý: đổi về 64 nhân 64, chia 255, duỗi thành một cột. Đoạn dưới thử ảnh `images/my_image.jpg`, ảnh có mèo (nhãn thật là 1).

```
from PIL import Image

image = np.array(Image.open("images/my_image.jpg").convert("RGB").resize((64,
↪  64)))
x = (image / 255.).reshape(-1, 1)                           # cùng tiền xử lý như
↪  dữ liệu huấn luyện
prob = float(L_model_forward(x, p4)[0].item())
print(round(prob, 3), classes[int(prob > 0.5)].decode("utf-8"))      # xác suất
↪  có mèo và lớp được đoán
```

2.7.6 Áp dụng vào dự án thực tế

2.7.6.1 Quy trình

1. Nạp dữ liệu, duỗi, chia 255; kiểm tra tỷ lệ nhãn của tập huấn luyện và tập kiểm tra.
2. Chạy mốc đơn giản nhất: đoán lớp phổ biến, rồi hồi quy logistic.
3. Dựng `Sequential` nhiều lớp `Dense`; ghi lại chi phí và độ chính xác theo thời gian (hoặc dùng `validation_data`).
4. Nếu chi phí đứng yên ngay từ đầu, nghi ngờ khởi tạo hoặc tốc độ học trước tiên.
5. Chọn điểm dừng bằng tập kiểm định; chỉ dùng tập kiểm tra để báo cáo cuối cùng.
6. Đọc các ảnh sai và phân tích lỗi trước khi nghĩ đến mạng lớn hơn.

2.7.6.2 Lỗi thường gặp trong phần thực hành này

Triệu chứng	Nguyên nhân và cách sửa
Mạng nhiều lớp, chi phí giảm rất chậm hoặc đứng yên gần mức 0,693	khởi tạo trọng số quá nhỏ cho mạng sâu: thử chia cho căn bậc hai số nút lớp trước (mục 2.6.7); kiểm tra thêm tốc độ học
Độ chính xác huấn luyện rất cao, kiểm tra thấp hơn nhiều	quá khớp; dừng sớm, thêm dữ liệu, giảm cỡ mạng
`log(0)` cho vô cực hoặc chi phí `nan`	xác suất dự đoán chạm đúng 0 hoặc 1; kẹp xác suất cách xa hai đầu bằng `np.clip`
Dự đoán ảnh riêng vô lý	thiếu tiền xử lý đúng như lúc huấn luyện (kích thước, chia 255, duỗi thành cột)
Độ chính xác kiểm tra khác nhiều giữa các lần chạy	tập kiểm tra quá nhỏ (50 ảnh, mỗi ảnh 2 điểm phần trăm); đừng đọc chênh lệch vài ảnh

Bảng 2.49: Lỗi thường gặp khi huấn luyện mạng sâu trên bộ ảnh có mèo

2.7.6.3 Tóm tắt cần nhớ

- Trên bộ ảnh có mèo, độ chính xác kiểm tra tăng từ 70% (hồi quy logistic) lên 72% (mạng hai lớp) và 80% (mạng bốn lớp).
- Chi phí huấn luyện thấp hơn không có nghĩa là mô hình tốt hơn: mạng hai lớp có chi phí cuối thấp hơn nhưng kém hơn ở tập kiểm tra.
- Độ chính xác kiểm tra có thể tốt hơn nếu dừng sớm, trước khi huấn luyện kết thúc; chọn điểm dừng bằng tập kiểm định.
- Mạng nông dùng khởi tạo nhân 0,01; mạng nhiều lớp dùng khởi tạo chia căn bậc hai số nút lớp trước.
- Kiểm tra độ dốc bằng cách đẩy nhẹ từng tham số: độ lệch rất nhỏ nghĩa là lan truyền ngược tự viết đúng.
- Phân tích lỗi bằng cách xem ảnh sai và nhóm theo kiểu (tư thế, nền, màu, góc chụp, độ sáng, tỷ lệ), rồi bổ sung dữ liệu tương ứng.

2.7.7 Tóm tắt chương

Chương này đi từ một neuron đến mạng nhiều lớp. Năm ý đáng mang theo:

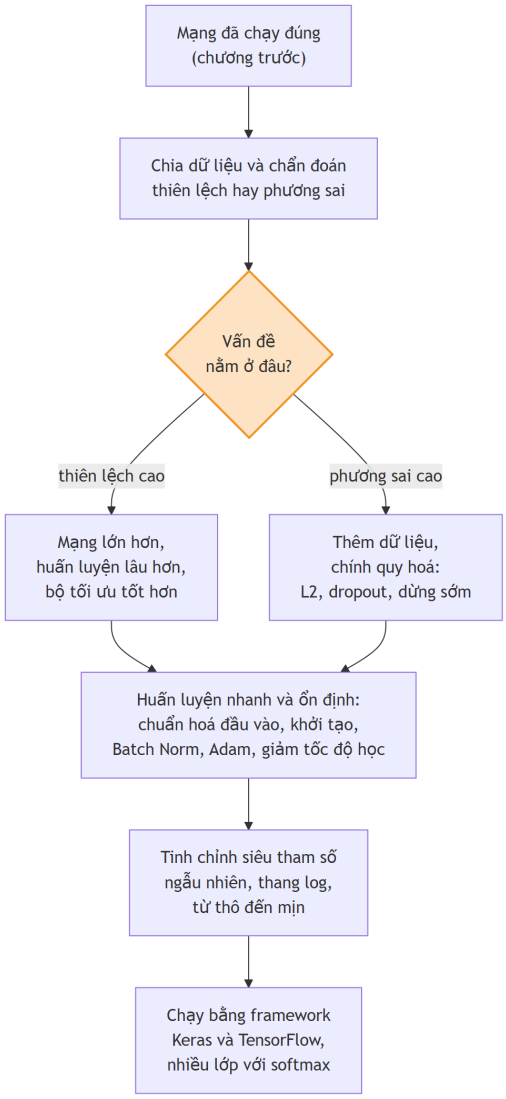
1. **Mạng nơ-ron là các neuron xếp thành lớp**, mỗi neuron tính tổng có trọng số rồi áp một hàm kích hoạt phi tuyến. Không có hàm phi tuyến, nhiều lớp thu gọn thành một lớp (mục 2.4.4).
2. **Học là lặp bốn bước**: lan truyền xuôi, tính chi phí, lan truyền ngược, cập nhật tham số. Vector hoá và các framework như Keras làm việc đó nhanh và gọn (mục 2.2, 2.4.5).
3. **Hình dạng ma trận là công cụ gỡ lỗi mạnh nhất**: trọng số của lớp có số hàng bằng số nút của lớp, số cột bằng số nút lớp trước, và mọi độ dốc cùng hình dạng với đại lượng của nó (mục 2.6.2).
4. **Khởi tạo, tốc độ học và độ sâu là siêu tham số** cần chọn bằng thực nghiệm; khởi tạo toàn 0 làm các nút đối xứng, và mạng sâu cần cách khởi tạo co theo số nút lớp trước (mục 2.4.6, 2.6.7).
5. **Đo đúng thứ cần đo**: so với mốc đơn giản, dùng tập kiểm định, đọc ảnh sai, và thận trọng với chênh lệch nhỏ trên tập kiểm tra nhỏ (mục 2.7.5).

Các chương sau dùng đúng bộ khung này; chỉ có loại lớp thay đổi: lớp tích chập cho ảnh, lớp hồi quy và attention cho chuỗi.

Chương 3

Cải thiện mạng nơ-ron sâu

Chương trước dừng ở việc dựng được một mạng nơ-ron chạy đúng. Chương này trả lời câu hỏi kế tiếp: **làm thế nào để mạng chạy tốt, chạy nhanh và dễ tinh chỉnh**. Nội dung chia làm ba phần theo thứ tự công việc thực tế: chẩn đoán và sửa lỗi của mô hình (chia dữ liệu, thiên lệch và phương sai, chính quy hoá, khởi tạo, kiểm tra độ dốc), làm cho huấn luyện nhanh và ổn định (mini-batch, Momentum, RMSprop, Adam, giảm tốc độ học), và tinh chỉnh siêu tham số cùng Batch Norm, rồi dùng framework để chạy mô hình nhiều lớp (Hình 3.1).



Hình 3.1: Bản đồ chương: chẩn đoán, sửa, làm huấn luyện nhanh, tinh chỉnh, rồi chạy bằng framework

Mỗi mục giải thích ý tưởng bằng lời và sơ đồ, đưa ví dụ bằng Keras hoặc TensorFlow trước, rồi mở hộp bằng NumPy khi cần thấy rõ bên trong. Các bài thực hành dùng số liệu của chính bài thực hành gốc (cùng dữ liệu, kiến trúc và siêu tham số); tài liệu không tự đo thêm số liệu.

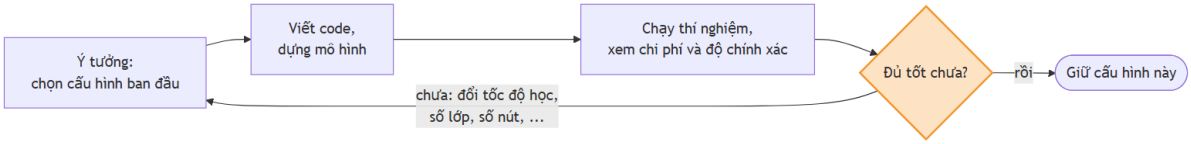
Mục	Câu hỏi	Nội dung chính
3.1 Thiết lập ứng dụng học máy	Chia dữ liệu và đọc kết quả thế nào để thử nghiệm nhanh và đáng tin?	tập huấn luyện, kiểm định, kiểm tra; thiên lệch và phương sai; công thức cải thiện
3.2 Chính quy hoá	Làm sao giảm quá khớp?	L2, dropout, tăng cường dữ liệu, dừng sớm, chọn kỹ thuật
3.3 Huấn luyện ổn định	Vì sao mạng sâu khó huấn luyện và sửa ra sao?	chuẩn hoá đầu vào, bùng nổ và tiêu biến gradient, khởi tạo, kiểm tra độ dốc
3.4 Thực hành: khởi tạo, chính quy hoá, kiểm tra độ dốc	Các kỹ thuật trên cho kết quả ra sao?	so sánh ba cách khởi tạo, L2 và dropout, kiểm tra độ dốc
3.5 Thuật toán tối ưu	Làm sao huấn luyện nhanh hơn gradient descent thuần?	mini-batch, trung bình mũ, Momentum, RMSprop, Adam, giảm tốc độ học
3.6 Thực hành: so sánh các bộ tối ưu	Bộ tối ưu và lịch tốc độ học khác nhau thế nào?	gradient descent, Momentum, Adam; giảm tốc độ học
3.7 Tinh chỉnh siêu tham số	Thử giá trị nào và theo thứ tự nào?	mức ưu tiên, lấy mẫu ngẫu nhiên, thang log, panda và caviar
3.8 Batch Normalization	Làm sao chuẩn hoá bên trong mạng?	chuẩn hoá theo lô, vị trí trong mạng, vì sao hiệu quả, lúc dự đoán
3.9 Phân loại nhiều lớp với softmax	Làm sao dự đoán nhiều hơn hai lớp?	softmax, mất mát nhiều lớp, softmax trong Keras
3.10 Framework học sâu và TensorFlow	Vì sao và làm sao dùng framework?	tiêu chí chọn, GradientTape, đồ thị tính toán
3.11 Thực hành: mạng ba lớp bằng TensorFlow	Dựng mạng nhiều lớp đầy đủ bằng TensorFlow thế nào?	tf.data, khởi tạo, lan truyền xuôi, mất mát, huấn luyện, Keras

Bảng 3.1: Các mục của chương Cải thiện mạng nơ-ron sâu

3.1 Thiết lập ứng dụng học máy

Chương trước dừng ở việc xây được một mạng nơ-ron chạy đúng. Câu hỏi của chương này khác: **làm thế nào để mạng chạy tốt**, và tốt nhanh? Một mạng có rất nhiều lựa chọn phải quyết định: bao nhiêu lớp, mỗi lớp bao nhiêu nút, tốc độ học là bao nhiêu, dùng hàm kích hoạt nào. Với một ứng dụng mới, gần như không ai đoán đúng các giá trị này ngay lần đầu, kể cả người có nhiều kinh nghiệm.

Vì vậy học máy ứng dụng là một **quá trình lặp**: bắt đầu từ một ý tưởng về cấu hình, viết code, chạy thí nghiệm, đọc kết quả, rồi sửa ý tưởng (Hình 3.2). Tốc độ tiến bộ phụ thuộc vào việc bạn đi hết một vòng nhanh đến đâu, và cách chia dữ liệu cùng cách đọc kết quả quyết định bạn đi nhanh hay chậm.



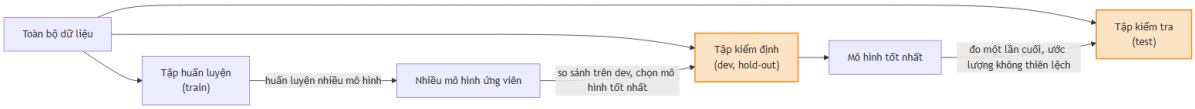
Hình 3.2: Vòng lặp ý tưởng, code, thí nghiệm. Các mục dưới đây giúp mỗi vòng nhanh và đáng tin hơn

Có một điều đáng nhớ: **kinh nghiệm từ lĩnh vực này thường không chuyển sang lĩnh vực khác**. Học sâu đã thành công ở xử lý ngôn ngữ, thị giác máy tính, nhận dạng giọng nói và cả dữ liệu có cấu trúc như quảng cáo, tìm kiếm, an ninh, logistics. Nhưng lựa chọn tốt nhất còn phụ thuộc vào lượng dữ liệu, số đặc trưng đầu vào, cấu hình máy (GPU hay CPU), nên cách duy nhất đáng tin là thử và đo.

3.1.1 Tập huấn luyện, tập kiểm định và tập kiểm tra

Dữ liệu thường được chia làm ba phần (Hình 3.3):

- **Tập huấn luyện** (training set): dùng để học tham số.
- **Tập kiểm định** (dev set, còn gọi là hold-out hoặc cross-validation set): dùng để so sánh các mô hình hoặc cấu hình khác nhau và chọn cái tốt nhất.
- **Tập kiểm tra** (test set): dùng *một lần cuối* để đo chất lượng của mô hình đã chọn, cho ước lượng không thiên lệch.

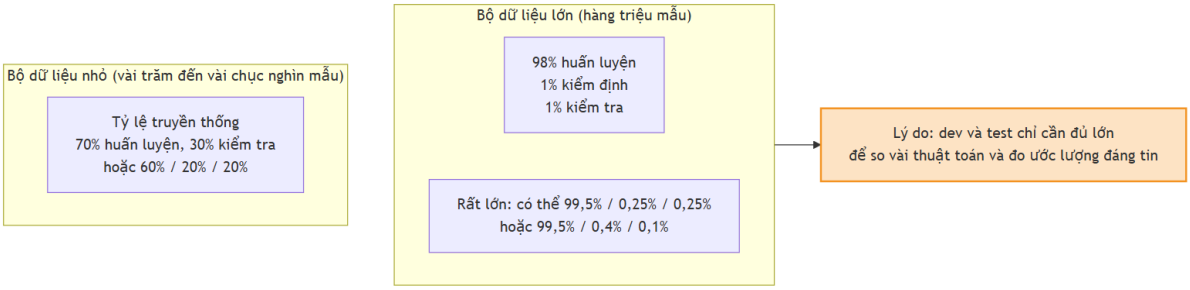


Hình 3.3: Quy trình dùng ba tập dữ liệu: huấn luyện nhiều mô hình, chọn bằng tập kiểm định, đo cuối bằng tập kiểm tra

Lý do cần hai tập khác nhau: khi bạn thử mười cấu hình và chọn cái có sai số kiểm định thấp nhất, bạn đã *khớp lựa chọn của mình với tập kiểm định*, nên con số đó không còn là ước lượng khách quan nữa. Tập kiểm tra chưa bị đụng đến mới cho con số đó.

3.1.1.1 Tỷ lệ chia

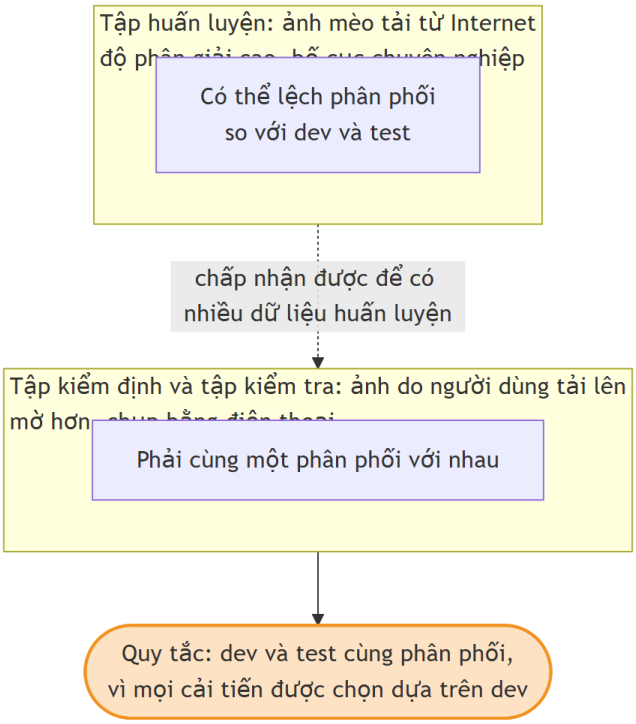
Thời kỳ dữ liệu còn ít (vài trăm đến vài chục nghìn mẫu), tỷ lệ phổ biến là 70% huấn luyện và 30% kiểm tra (khi không có tập kiểm định riêng), hoặc 60% huấn luyện, 20% kiểm định, 20% kiểm tra. Với dữ liệu hàng triệu mẫu thì khác. Tập kiểm định chỉ cần đủ lớn để so vài thuật toán và nhanh chóng biết cái nào tốt hơn; tập kiểm tra chỉ cần đủ lớn để đo một mô hình cuối cùng một cách tin cậy. Ví dụ với một triệu mẫu, 10 000 mẫu cho mỗi tập đã dư, tức tỷ lệ 98% / 1% / 1%. Với dữ liệu còn lớn hơn, có nơi dùng 99,5% / 0,25% / 0,25% hoặc 99,5% / 0,4% / 0,1% (Hình 3.4).



Hình 3.4: Dữ liệu càng lớn, phần dành cho kiểm định và kiểm tra càng nhỏ

3.1.1.2 Dữ liệu huấn luyện lệch phân phối

Một xu hướng khác: ngày càng nhiều người huấn luyện trên dữ liệu có phân phối *khác* với dữ liệu kiểm định và kiểm tra. Ví dụ ứng dụng cho người dùng tải ảnh lên để tìm ảnh mèo. Tập huấn luyện có thể là ảnh mèo cào từ Internet, thường độ phân giải cao và bố cục đẹp. Ảnh người dùng tải lên thì mờ hơn, chụp bằng điện thoại trong điều kiện tuỳ ý (Hình 3.5).



Hình 3.5: Tập huấn luyện được phép khác, nhưng tập kiểm định và tập kiểm tra phải cùng phân phối

Quy tắc nên theo: **tập kiểm định và tập kiểm tra phải cùng một phân phối**, vì bạn dùng tập kiểm định để chọn mô hình và cố cải thiện nó; nếu hai tập này khác nhau, bạn tối ưu cho một đích rồi bị đo ở đích khác. Mạng nơ-ron cần rất nhiều dữ liệu, nên chấp nhận cho tập huấn luyện khác phân phối để có nhiều dữ liệu hơn vẫn là lựa chọn hợp lý, miễn quy tắc trên được giữ.

3.1.1.3 Khi nào được bỏ tập kiểm tra

Mục đích của tập kiểm tra là cho ước lượng không thiên lệch về chất lượng cuối cùng. Nếu bạn không cần con số đó, có thể chỉ giữ tập huấn luyện và tập kiểm định. Điều cần cẩn thận là thuật ngữ: nhiều đội

gọi tập kiểm định là "tập kiểm tra". Khi nghe ai nói chỉ có tập huấn luyện và tập kiểm tra, hãy hỏi lại xem thực chất là họ đang khớp mô hình vào tập đó hay không.

3.1.1.4 Chia dữ liệu trong code

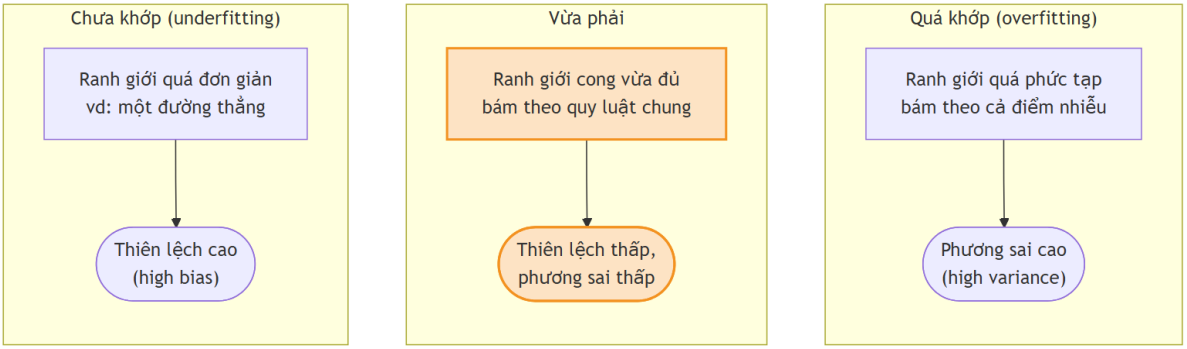
Với scikit-learn, chia hai lần là cách gọn nhất. Ví dụ dưới đây chia 98% / 1% / 1%:

```
from sklearn.model_selection import train_test_split

# lần 1: tách 2% làm "phần còn lại"
X_train, X_rest, y_train, y_rest = train_test_split(X, y, test_size=0.02,
↪  random_state=0)
# lần 2: chia đôi phần còn lại thành tập kiểm định và tập kiểm tra
X_dev, X_test, y_dev, y_test = train_test_split(X_rest, y_rest, test_size=0.5,
↪  random_state=0)
```

3.1.2 Thiên lệch và phương sai

Những người làm học máy giỏi thường có hiểu biết rất sâu về thiên lệch (bias) và phương sai (variance). Khái niệm này dễ học nhưng khó thành thạo. Hình 3.6 cho thấy ba trường hợp trên dữ liệu hai chiều:



Hình 3.6: Chưa khớp (thiên lệch cao), vừa phải, và quá khớp (phương sai cao)

- Một đường thẳng khớp với dữ liệu kém: **thiên lệch cao**, còn gọi là chưa khớp (underfitting).
- Một bộ phân loại quá phức tạp, ôm lấy mọi điểm kể cả nhiễu: **phương sai cao**, còn gọi là quá khớp (overfitting).
- Ở giữa là mức phức tạp vừa phải, bám theo quy luật chung.

Với hai đặc trưng thì vẽ được ranh giới. Với dữ liệu nhiều chiều thì không vẽ được, nên ta dùng hai con số: **sai số trên tập huấn luyện** và **sai số trên tập kiểm định**.

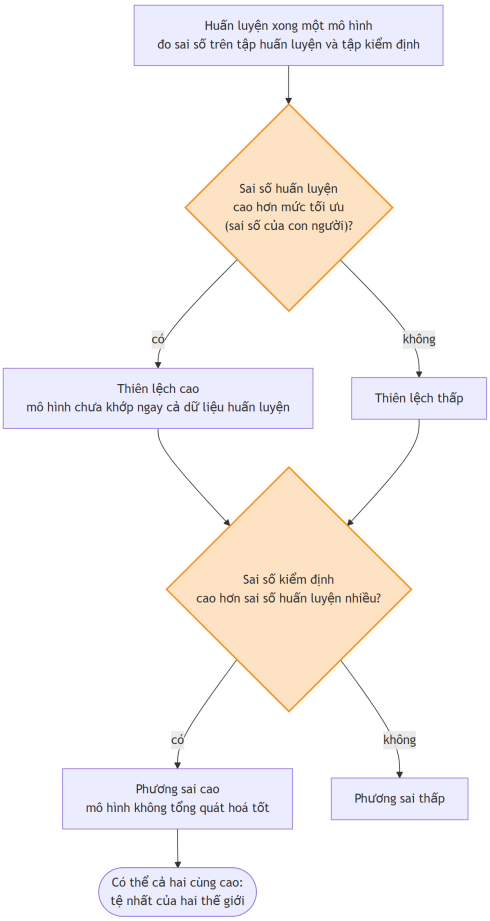
3.1.2.1 Đọc hai con số

Giả sử bài toán nhận dạng ảnh mèo mà con người làm gần như không sai (sai số khoảng 0%). Bảng 3.2 liệt kê bốn tình huống và chẩn đoán tương ứng.

Sai số huấn luyện	Sai số kiểm định	Chẩn đoán
1%	11%	phương sai cao: khớp tốt dữ liệu huấn luyện nhưng tổng quát hoá kém
15%	16%	thiên lệch cao: không khớp nổi dữ liệu huấn luyện; chênh lệch với kiểm định nhỏ
15%	30%	cả thiên lệch cao lẫn phương sai cao: tệ nhất của hai thế giới
0,5%	1%	thiên lệch thấp, phương sai thấp: tốt

Bảng 3.2: Chẩn đoán từ sai số huấn luyện và sai số kiểm định, khi mức sai số tối ưu gần 0%

Cách đọc ngắn gọn: **sai số huấn luyện cho biết thiên lệch; phần sai số tăng thêm khi chuyển sang tập kiểm định cho biết phương sai** (Hình 3.7). Có thể cả hai cùng cao: ví dụ một mô hình gần như tuyến tính (nên không khớp được dạng cong của dữ liệu) nhưng lại có chỗ quá linh hoạt, ôm cả vài điểm gán nhãn nhầm ở giữa. Trong không gian nhiều chiều, tình huống này không hiếm.



Hình 3.7: Hai câu hỏi liên tiếp để chẩn đoán thiên lệch và phương sai

Một giả định cần nhớ. Bảng 3.2 giả định sai số tối ưu (còn gọi là sai số Bayes, hay mức tốt nhất mà con người hoặc bất kỳ hệ thống nào đạt được) gần 0%, và tập huấn luyện cùng tập kiểm định cùng phân phối. Nếu ảnh quá mờ khiến ngay cả con người cũng sai nhiều, giả sử mức tối ưu là 15%, thì sai số huấn luyện 15% lại hoàn toàn hợp lý, không phải thiên lệch cao. Luôn so sai số của bạn với mức tối ưu của bài toán, không phải với 0.

3.1.2.2 Theo dõi hai con số trong Keras

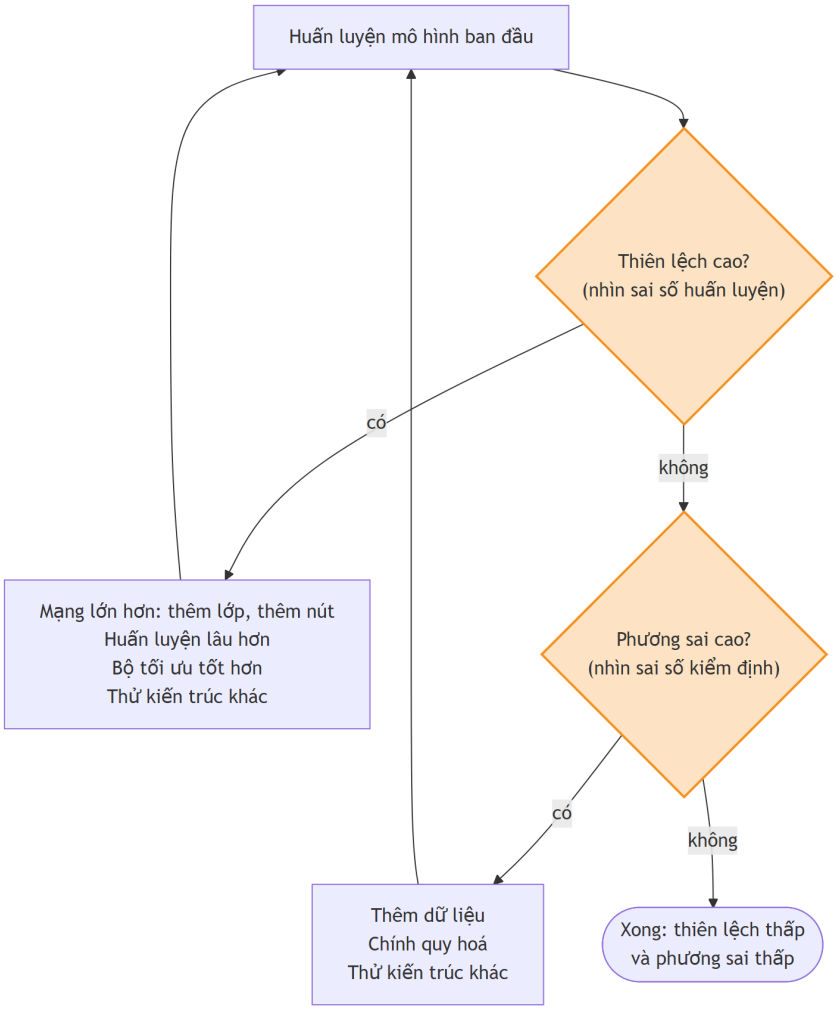
Keras tính sẵn cả hai khi bạn truyền tập kiểm định vào `fit`. Sau đó đọc từ `history`:

```
history = model.fit(X_train, y_train, epochs=50, batch_size=64,
                    validation_data=(X_dev, y_dev), verbose=0)

sai_so_huan_luyen = 1 - history.history["accuracy"][-1]        # nhìn thiên lệch
sai_so_kiem_dinh = 1 - history.history["val_accuracy"][-1]     # nhìn phương sai
print(sai_so_huan_luyen, sai_so_kiem_dinh)
```

3.1.3 Công thức cơ bản để cải thiện mô hình

Sau khi chẩn đoán, bước tiếp theo là một "công thức" có hệ thống (Hình 3.8). Thứ tự quan trọng: **xử lý thiên lệch trước, rồi mới đến phương sai**.



Hình 3.8: Công thức cơ bản: hỏi về thiên lệch trước, rồi phương sai

	Thiên lệch cao (khớp kém dữ liệu huấn luyện)	**Phương sai cao (tổng quát hoá kém)**
Cách thử	mạng lớn hơn (thêm lớp, thêm nút); huấn luyện lâu hơn; bộ tối ưu tốt hơn (mục 3.5); thử kiến trúc khác	thêm dữ liệu; chính quy hoá (mục 3.2); thử kiến trúc khác
Cách chắc chắn nhất	mạng lớn hơn gần như luôn có ích	thêm dữ liệu gần như luôn có ích
Ghi chú	huấn luyện lâu hơn không phải lúc nào cũng giúp nhưng không bao giờ hại	kiến trúc khác phải thử mới biết

Bảng 3.3: Việc nên thử tuỳ theo chẩn đoán

Hai điểm quan trọng:

- **Chẩn đoán trước rồi mới chọn việc.** Nếu thiên lệch cao thì thêm dữ liệu huấn luyện không phải việc hiệu quả nhất. Biết mình đang thiếu thiên lệch hay phương sai giúp tập trung vào đúng việc.
- **Trong thời đại học sâu, thiên lệch và phương sai ít đánh đổi nhau hơn.** Trước đây hầu hết công cụ làm giảm cái này lại làm tăng cái kia, nên người ta nói nhiều về "đánh đổi" (trade-off). Giờ có hai công cụ gần như chỉ tác động một phía: mạng lớn hơn (kèm chính quy hoá phù hợp) gần như chỉ giảm thiên lệch, còn thêm dữ liệu gần như chỉ giảm phương sai. Cái giá chính của mạng lớn là thời gian tính toán.

3.1.4 Áp dụng vào dự án thực tế

3.1.4.1 Danh sách việc cần làm

1. Chia dữ liệu thành huấn luyện, kiểm định, kiểm tra; giữ tập kiểm định và kiểm tra cùng phân phối, và cùng phân phối với dữ liệu thực tế bạn sẽ gặp.
2. Chọn tỷ lệ chia theo quy mô dữ liệu: nhỏ thì chia truyền thống, rất lớn thì chỉ cần một phần nhỏ cho kiểm định và kiểm tra.
3. Đo sai số huấn luyện và sai số kiểm định; so với mức tối ưu của bài toán (ví dụ mức của con người).
4. Chẩn đoán thiên lệch hay phương sai hay cả hai, rồi dùng Bảng 3.3 để chọn việc.
5. Chỉ chạm vào tập kiểm tra một lần, khi đã chọn xong mô hình cuối.

3.1.4.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Mô hình tốt trên tập kiểm định nhưng kém khi triển khai	tập kiểm định không cùng phân phối với dữ liệu thực tế; lấy dữ liệu kiểm định từ nguồn giống thực tế
Kết quả trên tập kiểm tra "quá đẹp", chạy thật lại kém	đã chọn mô hình dựa vào tập kiểm tra nên nó không còn khách quan; tách tập kiểm định riêng
Tập kiểm định quá nhỏ, kết quả dao động	với dữ liệu lớn vẫn cần đủ ảnh để so sánh tin cậy; đừng rút xuống quá thấp
Cố thêm dữ liệu mà sai số huấn luyện vẫn cao	vấn đề là thiên lệch, không phải phương sai; tăng cỡ mạng, huấn luyện lâu hơn
Cho rằng sai số huấn luyện 15% là thiên lệch cao dù ảnh rất mờ	chưa so với mức sai số tối ưu của bài toán

Bảng 3.4: Lỗi thường gặp khi thiết lập thí nghiệm

3.1.4.3 Bảng tra cứu API

Việc cần làm	Công cụ
Chia dữ liệu	`sklearn.model_selection.train_test_split`
Theo dõi sai số kiểm định lúc huấn luyện	`model.fit(..., validation_data=(X_dev, y_dev))`
Lấy một phần dữ liệu huấn luyện làm kiểm định	`model.fit(..., validation_split=0.1)`
Đọc lịch sử huấn luyện	`history.history["accuracy"]`, `history.history["val_accuracy"]`

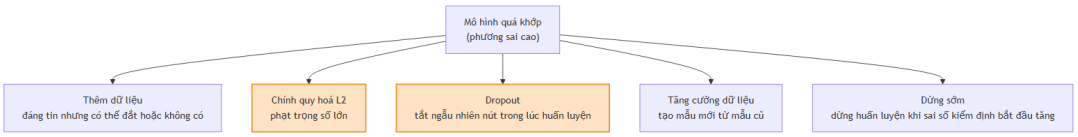
Bảng 3.5: Công cụ để thiết lập và theo dõi thí nghiệm

3.1.4.4 Tóm tắt cần nhớ

- Học máy ứng dụng là vòng lặp ý tưởng, code, thí nghiệm; chia dữ liệu tốt giúp vòng lặp nhanh.
- Ba tập: huấn luyện để học, kiểm định để chọn, kiểm tra để đo cuối. Dữ liệu càng lớn, kiểm định và kiểm tra càng chiếm tỷ lệ nhỏ.
- Kiểm định và kiểm tra phải cùng phân phối; huấn luyện có thể khác nếu cần thêm dữ liệu.
- Sai số huấn luyện cho biết thiên lệch; chênh lệch với sai số kiểm định cho biết phương sai; luôn so với mức tối ưu của bài toán.
- Xử lý thiên lệch trước (mạng lớn hơn, huấn luyện lâu hơn), rồi phương sai (thêm dữ liệu, chính quy hoá).

3.2 Chính quy hoá

Khi chẩn đoán cho thấy **phương sai cao** (quá khớp), công cụ đáng thử đầu tiên là chính quy hoá (regularization). Cách chắc chắn nhất là thêm dữ liệu huấn luyện, nhưng không phải lúc nào cũng làm được hoặc chi phí hợp lý. Chính quy hoá giảm quá khớp mà không cần thêm dữ liệu thật. Hình 3.9 liệt kê năm hướng mà mục này đi qua; hai kỹ thuật tô màu (L2 và dropout) là hai kỹ thuật dùng nhiều nhất.

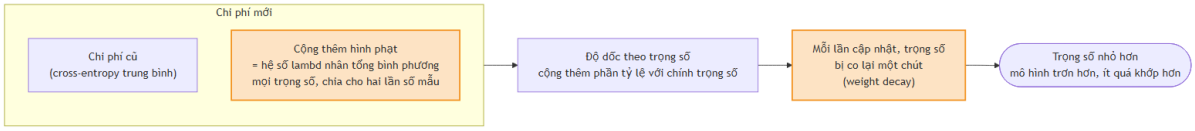


Hình 3.9: Các cách giảm quá khớp. L2 và dropout được dùng nhiều nhất

3.2.1 Chính quy hoá L2

3.2.1.1 Ý tưởng

Chính quy hoá L2 thêm vào chi phí một **hình phạt cho trọng số lớn**. Hồi quy logistic là ví dụ đơn giản nhất. Chi phí cũ là trung bình mất mát cross-entropy trên các mẫu. Chi phí mới bằng chi phí cũ, *cộng* một hệ số nhân với tổng bình phương của mọi trọng số, rồi chia cho hai lần số mẫu. Tài liệu này gọi hệ số đó là `lambd` (tên biến trong code, vì `lambda` là từ khoá của Python), và mô tả công thức bằng lời cùng Hình 3.10.



Hình 3.10: Chính quy hoá L2: phạt trọng số lớn, nên mỗi lần cập nhật trọng số bị co lại một chút

Vài chi tiết:

- **Chỉ phạt trọng số, bỏ qua hệ số điều chỉnh.** Trọng số có hàng nghìn đến hàng triệu giá trị, còn hệ số điều chỉnh chỉ vài chục; thêm hệ số điều chỉnh vào hình phạt gần như không đổi gì.
- **L2 và L1.** L2 dùng tổng bình phương; L1 dùng tổng giá trị tuyệt đối và làm nhiều trọng số bằng đúng 0 (mô hình "thưa", có thể nén nhỏ hơn). Thực tế L1 ít khi giúp nén đáng kể, nên L2 được dùng nhiều hơn hẳn.
- **`lambd` là siêu tham số**, chọn bằng tập kiểm định. Quá nhỏ thì không đủ chống quá khớp; quá lớn thì trọng số bị ép quá nhỏ và mô hình bị thiên lệch cao (làm phẳng quá mức).

Với mạng nhiều lớp, hình phạt là tổng bình phương mọi phần tử của mọi ma trận trọng số. Tên chính thức của đại lượng "tổng bình phương các phần tử của ma trận" là chuẩn Frobenius, nhưng bạn chỉ cần nhớ nó là tổng bình phương.

3.2.1.2 Vì sao gọi là "weight decay"

Vì hình phạt thay đổi chi phí, độ dốc theo mỗi ma trận trọng số cũng đổi: nó bằng độ dốc từ lan truyền ngược *cộng thêm* `lambd` chia số mẫu, nhân chính ma trận trọng số đó. Thay vào bước cập nhật, ta thấy mỗi lần cập nhật, trọng số bị *nhân với một số hơi nhỏ hơn 1* (bằng 1 trừ tốc độ học nhân `lambd` chia số mẫu) rồi mới trừ phần độ dốc thông thường. Trọng số "mục dần" đi một chút ở mọi bước, nên L2 còn được gọi là suy giảm trọng số (weight decay).

3.2.1.3 Vì sao L2 giảm quá khớp

Có hai cách hình dung, đều chỉ là gợi ý chứ không phải chứng minh chặt:

1. **Mạng trở nên đơn giản hơn.** Nếu `lambd` rất lớn, trọng số bị đẩy gần về 0, tác động của nhiều nút ẩn giảm đi, mạng giống như bị thu nhỏ, gần với một hồi quy logistic xếp chồng nhiều lớp.

Trên thực tế các nút không bị tắt hẳn, chỉ mỗi nút có ảnh hưởng nhỏ hơn. Dù vậy, kết quả là mạng đơn giản hơn, ít khớp quá mức. Từ vùng quá khớp, tăng `lambd` dịch mô hình về phía thiên lệch cao; sẽ có giá trị ở giữa cho kết quả "vừa phải".

2. **Mỗi lớp gần tuyến tính hơn.** Giả sử dùng hàm tanh. Khi trọng số nhỏ, tổng có trọng số nằm trong khoảng hẹp quanh 0, nơi tanh gần như thẳng. Mỗi lớp lúc đó gần như một phép tuyến tính, và như đã biết ở chương trước, mạng sâu chỉ gồm các lớp tuyến tính thì chỉ tính được một hàm tuyến tính. Nó không còn đủ khả năng vẽ ranh giới cực kỳ phức tạp để ôm lấy từng điểm nhiễu.

Mẹo gỡ lỗi. Khi dùng chính quy hoá, chi phí đã đổi định nghĩa: nó gồm cả hình phạt. Nếu bạn vẽ đường chi phí để kiểm tra gradient descent chạy đúng, hãy vẽ *chi phí mới* (có hình phạt). Nếu vẽ chi phí cũ, đường cong có thể không giảm đều ở mỗi lần lặp, dù thuật toán chạy đúng.

3.2.1.4 L2 trong code

Với Keras, thêm `kernel_regularizer` vào lớp. Có một chi tiết dễ nhầm: Keras cộng *hệ số nhân tổng bình phương trọng số* vào chi phí, không chia cho số mẫu và không chia 2. Nên để tương ứng với `lambd` ở trên, giá trị Keras bằng `lambd` chia hai lần số mẫu.

```
from tensorflow import keras
from tensorflow.keras import layers, regularizers

he_so_l2 = 0.01                                          # giá trị cần chọn
↪  bằng tập kiểm định
model = keras.Sequential([
    keras.Input(shape=(2,)),
    layers.Dense(20, activation="relu",
    ↪  kernel_regularizer=regularizers.l2(he_so_l2)),
    layers.Dense(3, activation="relu",
    ↪  kernel_regularizer=regularizers.l2(he_so_l2)),
    layers.Dense(1, activation="sigmoid",
    ↪  kernel_regularizer=regularizers.l2(he_so_l2)),
])
```

Để thấy bên trong, đây là cách tính chi phí và độ dốc có chính quy hoá bằng NumPy cho mạng ba lớp. Chi phí gồm phần cross-entropy cộng hình phạt; độ dốc theo mỗi ma trận trọng số chỉ thêm đúng một số hạng (`lambd` chia số mẫu, nhân trọng số):

```
def compute_cost_with_regularization(A3, Y, parameters, lambd):
    m = Y.shape[1]
    W1, W2, W3 = parameters["W1"], parameters["W2"], parameters["W3"]
    cross_entropy_cost = compute_cost(A3, Y)                          # phần
    ↪  cross-entropy
    L2_regularization_cost = (lambd / (2 * m)) * (np.sum(np.square(W1)) +
    ↪  np.sum(np.square(W2)) + np.sum(np.square(W3)))
    return cross_entropy_cost + L2_regularization_cost

# trong lan truyền ngược, chỉ ba dòng độ dốc theo trọng số thay đổi:
dW3 = 1./m * np.dot(dZ3, A2.T) + (lambd / m) * W3
dW2 = 1./m * np.dot(dZ2, A1.T) + (lambd / m) * W2
dW1 = 1./m * np.dot(dZ1, X.T) + (lambd / m) * W1
```

3.2.2 Dropout

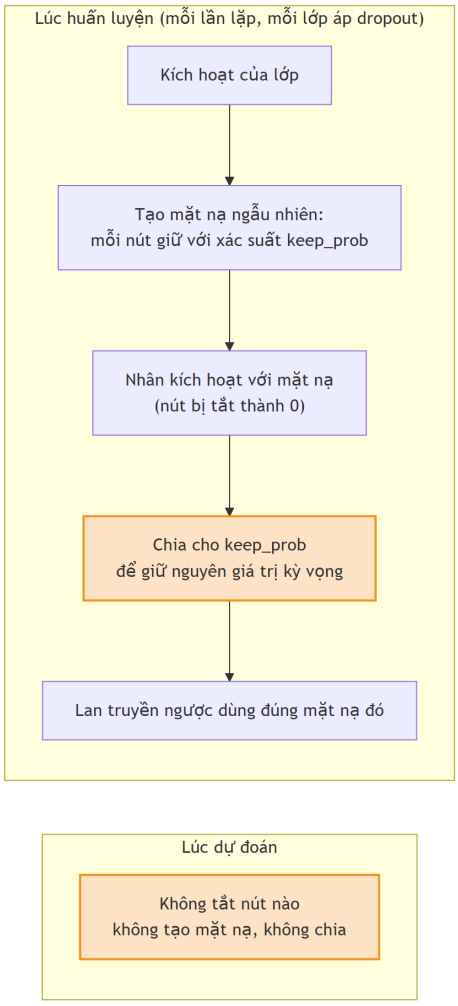
3.2.2.1 Ý tưởng

Dropout là kỹ thuật chính quy hoá đặc trưng của học sâu: **mỗi lần lặp, tắt ngẫu nhiên một số nút**. Với mỗi lớp, mỗi nút được giữ với xác suất `keep_prob` và bị tắt với xác suất còn lại. Nút bị tắt thì mọi liên kết ra khỏi nó cũng biến mất, nên mạng thu nhỏ đáng kể. Sau đó huấn luyện một bước (lan truyền xuôi và ngược) trên mạng nhỏ này. Ở mẫu hoặc lần lặp kế tiếp, tung đồng xu lại, giữ một tập nút khác. Nghe có vẻ kỳ quặc nhưng hiệu quả. Một cách hình dung: mỗi lần lặp, ta huấn luyện một mạng nhỏ khác, và những mạng nhỏ thì khó quá khớp.

3.2.2.2 Cách cài đặt: inverted dropout

Cách phổ biến nhất gọi là **inverted dropout**. Với một lớp, các bước là (Hình 3.11):

1. Tạo ma trận mặt nạ ngẫu nhiên cùng kích thước với kích hoạt; mỗi phần tử bằng 1 với xác suất `keep_prob`, bằng 0 với xác suất còn lại.
2. Nhân từng phần tử kích hoạt với mặt nạ: nút nào có mặt nạ 0 thì kích hoạt thành 0.
3. **Chia kết quả cho `keep_prob`.** Đây là bước đảo ngược của tên gọi.



Hình 3.11: Inverted dropout: tắt và chia lúc huấn luyện, không làm gì lúc dự đoán

Vì sao phải chia? Nếu một lớp có 50 nút và `keep_prob` là 0,8 thì trung bình 10 nút bị tắt, giá trị kỳ vọng của kích hoạt giảm khoảng 20%. Chia cho `keep_prob` bù lại phần giảm đó, nên giá trị kỳ vọng của kích hoạt giữ nguyên. Nhờ vậy, **lúc dự đoán không cần thêm hệ số điều chỉnh nào**. Các phiên bản dropout cũ thiếu bước chia khiến lúc dự đoán phức tạp hơn; ngày nay hầu như ai cũng dùng inverted dropout.

Mặt nạ phải thay đổi theo từng mẫu và từng lần lặp: không có chuyện luôn tắt cùng một nút. Và **lan truyền ngược dùng đúng mặt nạ của lan truyền xuôi**, kèm chia cho `keep_prob`.

```
# lan truyền xuôi, áp dụng cho kích hoạt A1 của lớp ẩn thứ nhất
D1 = np.random.rand(A1.shape[0], A1.shape[1])      # bước 1: ma trận ngẫu nhiên
↪  cùng hình dạng với A1
D1 = (D1 < keep_prob).astype(int)                  # bước 2: 1 với xác suất
↪  keep_prob, 0 với xác suất còn lại
A1 = A1 * D1                                       # bước 3: tắt một số nút
A1 = A1 / keep_prob                                # bước 4: bù lại để giữ giá
↪  trị kỳ vọng

# lan truyền ngược: dùng đúng mặt nạ D1 đã lưu trong bộ nhớ đệm
dA1 = dA1 * D1                                     # tắt đúng những nút đã tắt
↪  khi xuôi
dA1 = dA1 / keep_prob                              # chia cho keep_prob như bước
↪  xuôi
```

3.2.2.3 Lúc dự đoán thì không dropout

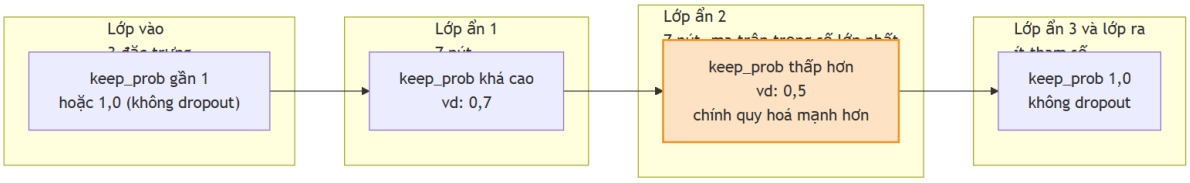
Khi dự đoán, **không tắt nút nào và không tung đồng xu**, vì bạn không muốn đầu ra ngẫu nhiên; dropout lúc dự đoán chỉ thêm nhiễu. Lỗi thường gặp nhất là dùng dropout cả ở lúc huấn luyện lẫn lúc kiểm tra. Về lý thuyết có thể chạy nhiều lần với các nút bị tắt ngẫu nhiên rồi lấy trung bình, nhưng tốn công và cho kết quả gần như như không dùng dropout.

3.2.2.4 Vì sao dropout hoạt động

- Mỗi lần lặp làm việc với một mạng nhỏ hơn, nên có tác dụng chính quy hoá.
- **Một nút không dám dựa hẳn vào một đầu vào nào**, vì đầu vào đó có thể biến mất ngẫu nhiên. Nó buộc phải rải trọng số ra nhiều đầu vào. Rải đều trọng số làm tổng bình phương trọng số nhỏ đi, giống tác dụng của L2. Có thể chứng minh dropout là một dạng L2 thích nghi, trong đó mức phạt ở mỗi trọng số phụ thuộc độ lớn của kích hoạt nhân với trọng số đó.

3.2.2.5 Chọn keep_prob theo từng lớp

`keep_prob` có thể khác nhau giữa các lớp (Hình 3.12). Lớp nào có ma trận trọng số lớn, dễ quá khớp, thì chọn `keep_prob` thấp hơn (ví dụ 0,5) để chính quy hoá mạnh hơn, giống như tăng `lambd` cho lớp đó. Lớp ít lo quá khớp thì chọn cao hơn (0,7). Lớp không cần thì chọn 1,0, tức không dropout. Với lớp vào, thường dùng 1,0 hoặc một giá trị rất gần 1, vì ít khi muốn loại bỏ cả nửa số đặc trưng đầu vào. Cái giá của việc tuỳ lớp là thêm siêu tham số phải tìm; cách đơn giản là chọn vài lớp áp dụng dropout và dùng chung một `keep_prob`.



Hình 3.12: Lớp có nhiều tham số dễ quá khớp hơn thì dùng keep_prob thấp hơn

3.2.2.6 Khi nào dùng, và hạn chế

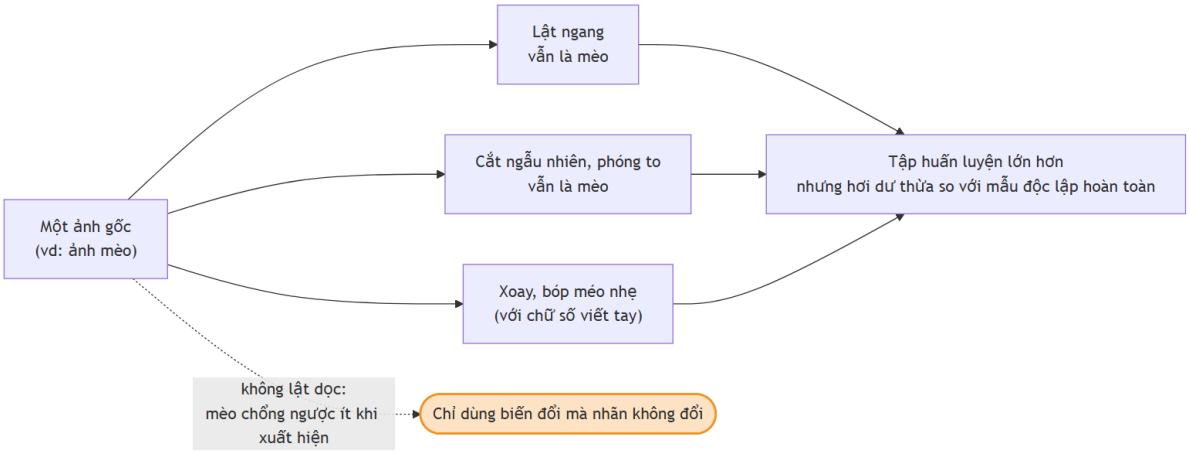
- Dropout được dùng nhiều ở thị giác máy tính, vì ảnh có số điểm ảnh khổng lồ và gần như không bao giờ đủ dữ liệu. Nhiều nhà nghiên cứu thị giác dùng nó gần như mặc định. Nhưng cách nhìn này không nên suy rộng sang lĩnh vực khác. Dropout là kỹ thuật chống quá khớp; nếu mô hình không quá khớp thì không cần dùng.
- **Hạn chế lớn nhất:** chi phí không còn được định nghĩa rõ ràng ở mỗi lần lặp vì nút bị tắt ngẫu nhiên, nên đường chi phí mất khả năng làm công cụ gỡ lỗi. Cách làm thường thấy: đặt `keep_prob` bằng 1, chạy để chắc chắn chi phí giảm đều, rồi mới bật dropout.

```
keep_prob = 0.86
model = keras.Sequential([
    keras.Input(shape=(2,)),
    layers.Dense(20, activation="relu"),
    layers.Dropout(1 - keep_prob),          # Keras dùng "rate" = tỷ lệ bị TẮT,
    ↪  nên rate = 1 - keep_prob
    layers.Dense(3, activation="relu"),
    layers.Dropout(1 - keep_prob),
    layers.Dense(1, activation="sigmoid"),
])
# Keras cài đặt inverted dropout và chỉ tắt nút trong lúc fit; predict và
↪  evaluate không dropout
```

3.2.3 Các kỹ thuật khác: tăng cường dữ liệu và dừng sớm

3.2.3.1 Tăng cường dữ liệu

Nếu quá khớp và không thể thu thêm dữ liệu, bạn có thể **tạo thêm mẫu giả từ mẫu sẵn có** (Hình 3.13). Với ảnh mèo: lật ngang cho ra một ảnh mèo khác, cắt ngẫu nhiên hoặc phóng to vẫn là ảnh mèo. Với chữ số viết tay thì xoay và bóp méo nhẹ. Ta đang nói với thuật toán: nếu cái này là mèo thì lật ngang nó vẫn là mèo. Không nên lật dọc, vì mèo chổng ngược ít khi xuất hiện. Các mẫu giả đem lại ít thông tin hơn một mẫu độc lập hoàn toàn mới (vì còn dư thừa với mẫu gốc), nhưng chúng gần như miễn phí, chỉ tốn tính toán.

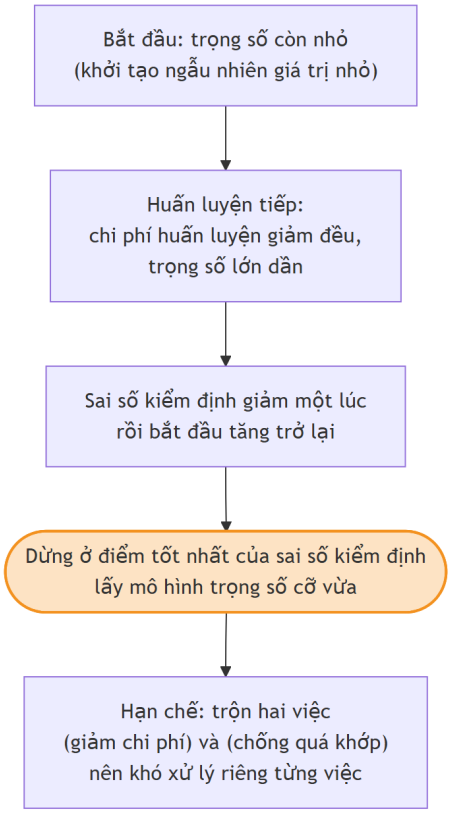


Hình 3.13: Tăng cường dữ liệu: chỉ dùng biến đổi giữ nguyên nhãn

```
augment = keras.Sequential([
    layers.RandomFlip("horizontal"),        # chỉ lật ngang
    layers.RandomZoom(0.1),                 # phóng to ngẫu nhiên
    layers.RandomRotation(0.05),            # xoay nhẹ
])
model = keras.Sequential([keras.Input(shape=(64, 64, 3)), augment,
                          layers.Flatten(), layers.Dense(1,
                          ↪  activation="sigmoid")])
# các lớp tăng cường chỉ hoạt động lúc huấn luyện
```

3.2.3.2 Dừng sớm

Dừng sớm (early stopping) là dừng huấn luyện giữa chừng (Hình 3.14). Khi chạy gradient descent, vẽ cả sai số huấn luyện (hoặc chi phí huấn luyện) lẫn sai số kiểm định. Chi phí huấn luyện giảm đều, còn sai số kiểm định thường giảm một lúc rồi tăng trở lại. Hãy dừng ở điểm sai số kiểm định thấp nhất.



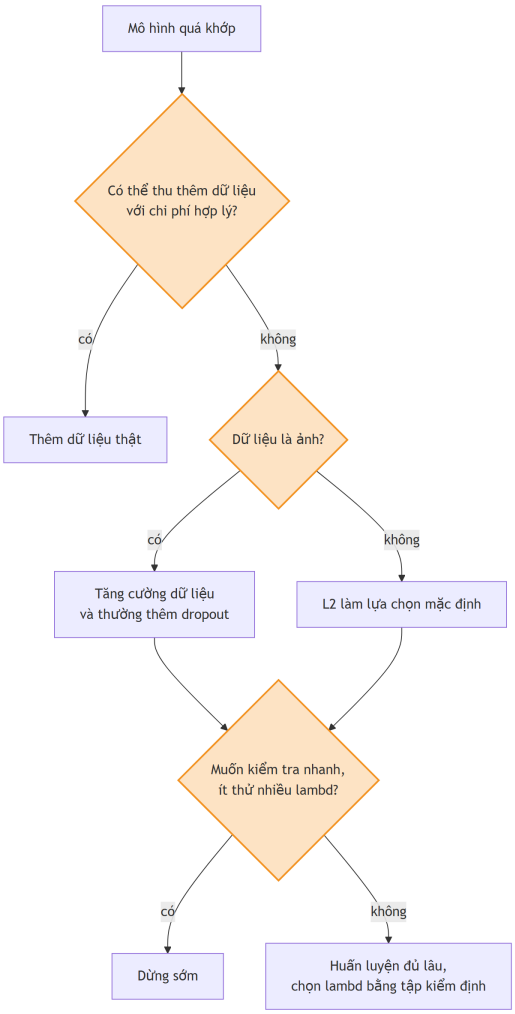
Hình 3.14: Dừng sớm: lấy mô hình ở điểm sai số kiểm định thấp nhất

Lý do hiệu quả: lúc đầu trọng số còn nhỏ vì khởi tạo ngẫu nhiên giá trị nhỏ; càng huấn luyện, trọng số càng lớn. Dừng giữa chừng giữ lại mô hình có chuẩn trọng số cỡ vừa, tương tự tác dụng của L2.

Hạn chế của dừng sớm là nó trộn hai việc khác nhau vào một: tối ưu chi phí (tìm tham số làm chi phí nhỏ nhất) và chống quá khớp. Khi tách riêng, ta có một bộ công cụ cho việc đầu (gradient descent và các bộ tối ưu ở mục 3.5) và một bộ công cụ cho việc sau (chính quy hoá, thêm dữ liệu); tư duy này gọi là *trực giao hoá*. Dừng sớm làm hai việc dính vào nhau nên khó tách để phân tích. Lựa chọn thay thế là dùng L2 và huấn luyện đủ lâu, đổi lại phải thử nhiều giá trị `lambd`. Ưu điểm của dừng sớm là chạy gradient descent một lần mà đã thử qua trọng số nhỏ, vừa và lớn, không cần thử nhiều `lambd`.

```
early = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10,
↪  restore_best_weights=True)
model.fit(X_train, y_train, epochs=500, batch_size=64,
          validation_data=(X_dev, y_dev), callbacks=[early], verbose=0)
# dừng khi val_loss không cải thiện trong 10 epoch liên tiếp, rồi khôi phục trọng
↪  số tốt nhất
```

3.2.4 Chọn kỹ thuật nào



Hình 3.15: Gợi ý chọn kỹ thuật chống quá khớp

Kỹ thuật	Ưu điểm	Lưu ý
Thêm dữ liệu	chắc chắn nhất, hầu như chỉ giảm phương sai	đắt hoặc không làm được
L2	phổ biến nhất, đơn giản	phải chọn `lambd`, thử nhiều giá trị
Dropout	mạnh với mạng lớn, đặc biệt ảnh	chi phí không còn xác định rõ để gỡ lỗi; phải chọn `keep_prob`
Tăng cường dữ liệu	gần như miễn phí	chỉ dùng biến đổi giữ nguyên nhãn; mẫu giả kém thông tin hơn mẫu thật
Dừng sớm	chạy một lần, thử nhiều cỡ trọng số	trộn việc tối ưu và việc chống quá khớp

Bảng 3.6: So sánh các kỹ thuật chống quá khớp

3.2.5 Áp dụng vào dự án thực tế

3.2.5.1 Danh sách việc cần làm

1. Chắc chắn là phương sai cao rồi mới chính quy hoá: sai số huấn luyện thấp, sai số kiểm định cao hơn nhiều (mục 3.1.2).
2. Bắt đầu với L2 hoặc dropout; thử vài giá trị `lambd` hoặc `keep_prob` trên tập kiểm định, không chỉnh theo tập kiểm tra.
3. Nếu dữ liệu là ảnh, thêm tăng cường dữ liệu.
4. Theo dõi sai số kiểm định để chọn điểm dừng.
5. Khi gỡ lỗi, tắt dropout (`keep_prob` bằng 1) và vẽ đúng chi phí đã có hình phạt.

3.2.5.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Mô hình chính quy hoá mà cả sai số huấn luyện lẫn kiểm định đều cao	`lambd` quá lớn (hoặc `keep_prob` quá thấp) làm thiên lệch cao; giảm bớt
Dự đoán lúc kiểm tra dao động, mỗi lần chạy mỗi khác	vẫn bật dropout khi dự đoán; chỉ dùng lúc huấn luyện
Đường chi phí không giảm đều dù code đúng	đang vẽ chi phí cũ trong khi huấn luyện chi phí đã thêm hình phạt; vẽ chi phí mới
Dùng L2 trong Keras với `lambd` theo công thức NumPy mà kết quả khác	Keras không chia cho số mẫu và hai; đổi bằng `lambd` chia hai lần số mẫu
Dropout ở lớp vào gây mất thông tin	lớp vào nên dùng `keep_prob` gần hoặc bằng 1

Bảng 3.7: Lỗi thường gặp khi chính quy hoá

3.2.5.3 Bảng tra cứu API

Việc cần làm	Keras
Chính quy hoá L2 cho một lớp	`Dense(..., kernel_regularizer=regularizers.l2(giá_trị))`
Dropout	`layers.Dropout(rate)`, với `rate` bằng 1 trừ `keep_prob`
Lật, phóng, xoay ảnh	`layers.RandomFlip`, `layers.RandomZoom`, `layers.RandomRotation`
Dừng sớm	`keras.callbacks.EarlyStopping(monitor="val_loss", patience=..., restore_best_weights=True)`

Bảng 3.8: Các lệnh Keras cho chính quy hoá

3.2.5.4 Tóm tắt cần nhớ

- Phương sai cao thì thêm dữ liệu hoặc chính quy hoá; chính quy hoá không cần thêm dữ liệu thật.
- L2 phạt tổng bình phương trọng số; mỗi lần cập nhật trọng số bị co lại một chút (weight decay); `lambd` là siêu tham số.

- Dropout tắt ngẫu nhiên nút lúc huấn luyện, chia cho `keep_prob` để giữ giá trị kỳ vọng; lúc dự đoán thì không dropout.
- Tăng cường dữ liệu và dừng sớm là hai công cụ nữa; dừng sớm đơn giản nhưng trộn hai việc.
- Dropout làm chi phí không xác định rõ nên khó gỡ lỗi: tắt dropout khi gỡ lỗi.

3.3 Huấn luyện ổn định: chuẩn hoá đầu vào, khởi tạo và kiểm tra độ dốc

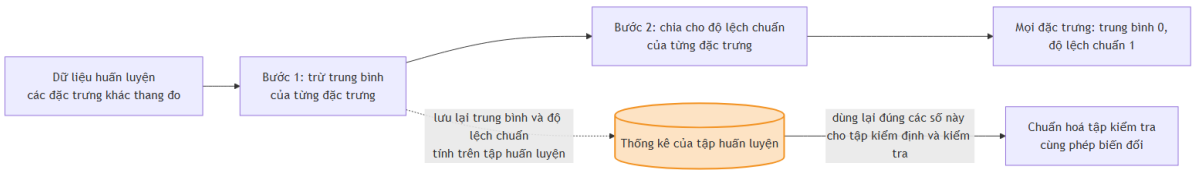
Mục trước dạy cách giảm quá khớp. Mục này giải quyết ba vấn đề thực tế khác, đều xoay quanh câu hỏi **làm sao để huấn luyện chạy nhanh, ổn định và đúng**:

- **Chuẩn hoá đầu vào**: đưa các đặc trưng về cùng thang đo để gradient descent đi nhanh hơn.
- **Khởi tạo trọng số**: chọn độ lớn ban đầu để tín hiệu và độ dốc không bùng nổ hay tiêu biến trong mạng sâu.
- **Kiểm tra độ dốc**: công cụ gỡ lỗi để chắc chắn lan truyền ngược viết tay là đúng.

3.3.1 Chuẩn hoá đầu vào

Một mẹo đơn giản và hiệu quả để tăng tốc huấn luyện là **chuẩn hoá đặc trưng đầu vào**. Cách làm gồm hai bước, thực hiện riêng cho từng đặc trưng (Hình 3.16):

1. **Trừ trung bình**: tính trung bình của đặc trưng trên tập huấn luyện rồi trừ nó khỏi mọi mẫu. Dữ liệu được dời sao cho có trung bình bằng 0.
2. **Chia cho độ lệch chuẩn**: tính độ lệch chuẩn của đặc trưng (sau khi đã trừ trung bình) rồi chia mọi mẫu cho nó. Phương sai mỗi đặc trưng bằng 1.

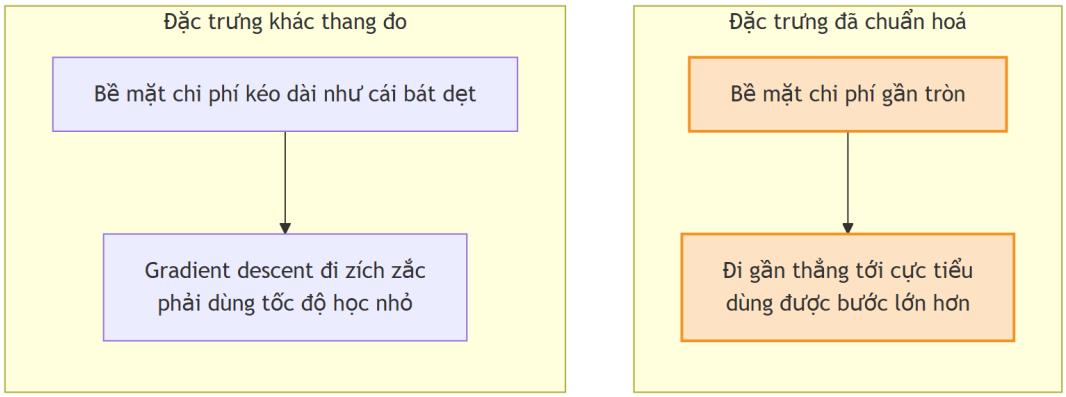


Hình 3.16: Chuẩn hoá đầu vào. Tập kiểm định và tập kiểm tra dùng lại đúng trung bình và độ lệch chuẩn của tập huấn luyện

Lưu ý quan trọng. Trung bình và độ lệch chuẩn **chỉ tính trên tập huấn luyện**, rồi dùng *đúng các số đó* cho tập kiểm định và tập kiểm tra. Không tính riêng cho từng tập: mọi mẫu, lúc huấn luyện lẫn lúc kiểm tra, phải đi qua cùng một phép biến đổi.

3.3.1.1 Vì sao chuẩn hoá làm huấn luyện nhanh hơn

Nếu hai đặc trưng có thang đo rất khác nhau (một cái từ 1 đến 1000, cái kia từ 0 đến 1), thì các trọng số tương ứng cũng nhận giá trị ở những thang đo rất khác. Bề mặt chi phí khi đó kéo dài như một cái bát dẹt (Hình 3.17). Gradient descent phải đi zích zắc qua lại nhiều lần mới tới đáy, và buộc dùng tốc độ học nhỏ. Sau khi chuẩn hoá, bề mặt chi phí gần tròn hơn; từ đâu bắt đầu, gradient descent cũng đi gần thẳng tới cực tiểu và dùng được bước lớn hơn.



Hình 3.17: Đặc trưng khác thang đo làm bề mặt chi phí dẹt; chuẩn hoá làm nó tròn hơn

Thực tế các đặc trưng thường nhiều chiều, hình hai chiều chỉ để hình dung. Nếu các đặc trưng đã cùng thang đo tương tự (ví dụ một cái từ 0 đến 1, một cái từ -1 đến 1, một cái từ 1 đến 2) thì bước này ít quan trọng; còn khi chúng khác thang đo rất nhiều thì chuẩn hoá giúp đáng kể. Chuẩn hoá hầu như không bao giờ gây hại, nên cứ làm nếu không chắc.

3.3.1.2 Trong code

```
from sklearn.preprocessing import StandardScaler

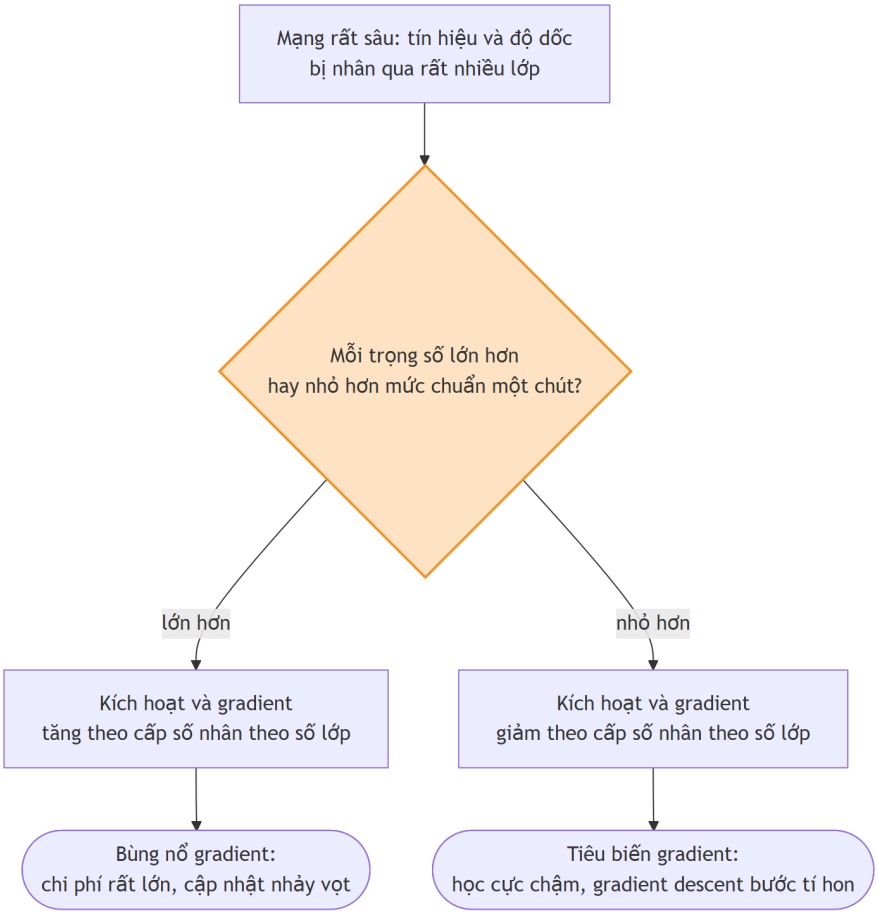
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)     # tính trung bình và độ lệch chuẩn
↪  TRÊN TẬP HUẤN LUYỆN, rồi áp dụng
X_dev = scaler.transform(X_dev)             # dùng lại đúng các số đó: chỉ
↪  transform, không fit
X_test = scaler.transform(X_test)

# Hoặc đặt ngay trong mô hình Keras:
norm = keras.layers.Normalization()
norm.adapt(X_train)                          # học trung bình và phương sai từ
↪  tập huấn luyện
```

3.3.2 Bùng nổ và tiêu biến gradient

Mạng càng sâu, một vấn đề càng rõ: **độ dốc (và cả kích hoạt) có thể trở nên cực lớn hoặc cực nhỏ**. Giả sử mạng có rất nhiều lớp, dùng hàm kích hoạt đồng nhất (cho dễ phân tích) và bỏ qua hệ số điều chỉnh. Khi đó đầu ra bằng tích của mọi ma trận trọng số nhân với đầu vào.

Giả sử mỗi ma trận trọng số *lớn hơn ma trận đơn vị một chút*, ví dụ đường chéo bằng 1,5. Đầu ra khi đó tăng theo kiểu nhân 1,5 sau mỗi lớp, tức tăng theo cấp số nhân của số lớp. Ngược lại, nếu đường chéo bằng 0,5 thì kích hoạt giảm: với đầu vào 1 và 1, các kích hoạt qua từng lớp lần lượt là một nửa, một phần tư, một phần tám, và cứ thế, giảm theo cấp số nhân khi số lớp tăng. Lập luận tương tự áp dụng cho độ dốc trong lan truyền ngược (Hình 3.18).



Hình 3.18: Bùng nổ và tiêu biến gradient trong mạng rất sâu

Với những mạng hiện đại có tới hàng trăm lớp (ví dụ một mạng 152 lớp đã đạt kết quả rất tốt), điều này nghiêm trọng: nếu độ dốc nhỏ theo cấp số nhân, gradient descent chỉ bước những bước tí hon và rất lâu mới học được gì. Trong nhiều năm, đây là một rào cản lớn của việc huấn luyện mạng sâu. Có một giải pháp một phần, không giải quyết hết nhưng giúp nhiều: **chọn cẩn thận độ lớn khi khởi tạo trọng số**.

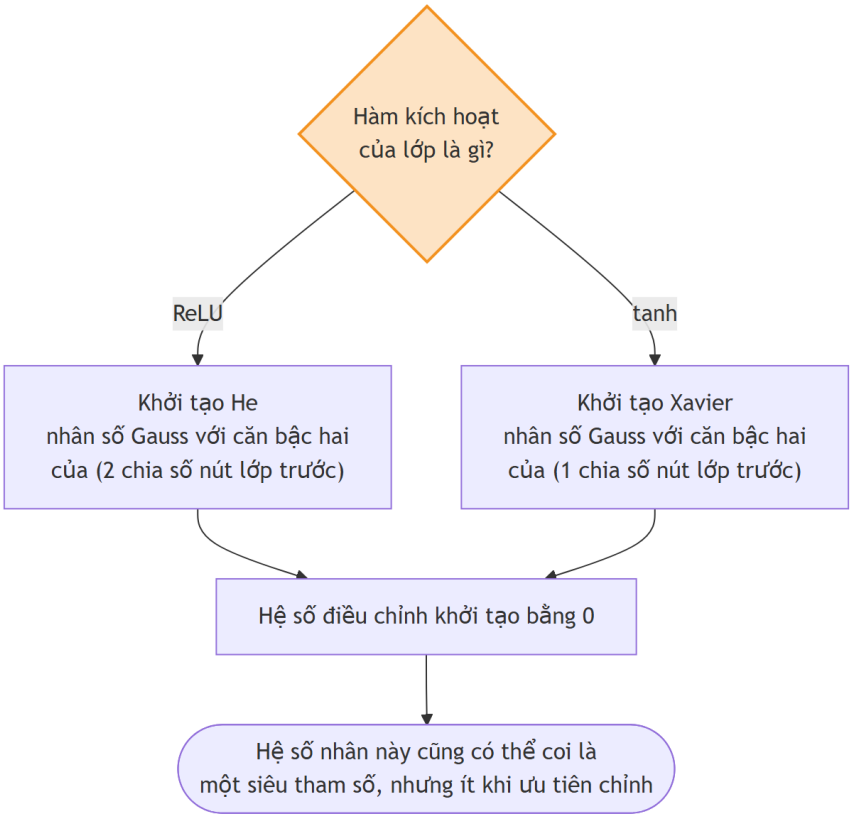
3.3.3 Khởi tạo trọng số cho mạng sâu

3.3.3.1 Ý tưởng cho một nút

Xét một nút có nhiều đầu vào. Tổng có trọng số của nó là tổng của từng số hạng (mỗi số hạng là trọng số nhân với đầu vào tương ứng). Số đầu vào càng lớn thì có càng nhiều số hạng cộng vào nhau, nên để tổng không quá lớn, **mỗi trọng số phải càng nhỏ**. Một lựa chọn hợp lý: đặt phương sai của trọng số bằng một chia số đầu vào của nút. Trong code, đó là nhân số ngẫu nhiên Gauss với căn bậc hai của (một chia số nút của lớp trước). Khi đầu vào có trung bình 0 và phương sai 1, tổng có trọng số cũng giữ thang đo tương tự. Cách này không giải quyết hoàn toàn bùng nổ hay tiêu biến, nhưng đẩy mỗi ma trận trọng số về mức "không lớn hơn nhiều và không nhỏ hơn nhiều so với 1", nên tín hiệu không phình ra hoặc co lại quá nhanh.

3.3.3.2 Chọn hệ số theo hàm kích hoạt

Hệ số cụ thể tuỳ hàm kích hoạt (Hình 3.19):



Hình 3.19: Chọn hệ số khởi tạo theo hàm kích hoạt

Tên	Dùng với	Hệ số nhân với số Gauss
Khởi tạo He	ReLU (phổ biến nhất)	căn bậc hai của (2 chia số nút lớp trước)
Khởi tạo Xavier	tanh	căn bậc hai của (1 chia số nút lớp trước)
Biến thể khác	tuỳ nghiên cứu	dựa trên cả số nút lớp trước và số nút lớp hiện tại; cũng cho ra giá trị khởi đầu hợp lý

Bảng 3.9: Các cách khởi tạo thông dụng. Hệ số nhân này cho ra giá trị mặc định để bắt đầu

Các công thức trên chỉ là điểm xuất phát. Bản thân hệ số này cũng có thể được coi là một siêu tham số: nhân thêm một số rồi thử vài giá trị. Có bài toán việc này giúp ở mức khiêm tốn; thường nó nằm khá thấp trong danh sách siêu tham số cần chỉnh trước.

3.3.3.3 Trong code

```
def initialize_parameters_he(layers_dims):
    np.random.seed(3)
    parameters = {}
    L = len(layers_dims) - 1                    # số lớp
    for l in range(1, L + 1):
        # số Gauss nhân với căn bậc hai của (2 / số nút lớp trước): khởi tạo He
        parameters['W' + str(l)] = np.random.randn(layers_dims[l],
        ↪  layers_dims[l-1]) * np.sqrt(2. / layers_dims[l-1])
        parameters['b' + str(l)] = np.zeros((layers_dims[l], 1))      # hệ số
        ↪  điều chỉnh khởi tạo bằng 0
    return parameters
```

Keras đã cung cấp sẵn mọi cách này qua tham số `kernel_initializer`:

```
layers.Dense(64, activation="relu", kernel_initializer="he_normal")          #
↪  He cho ReLU
layers.Dense(64, activation="tanh", kernel_initializer="glorot_uniform")     #
↪  mặc định của Keras; cùng họ với Xavier
```

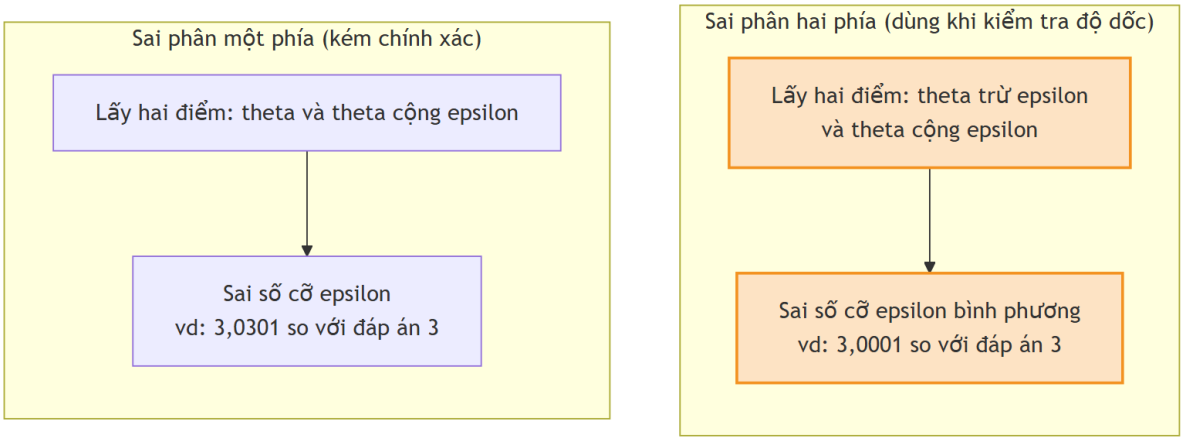
Kết quả thực nghiệm của ba cách khởi tạo (toàn 0, ngẫu nhiên giá trị lớn, He) được trình bày ở mục 3.4.1.

3.3.4 Kiểm tra độ dốc

Khi tự viết lan truyền ngược, rất dễ có lỗi nhỏ mà kết quả vẫn trông "gần đúng". **Kiểm tra độ dốc** (gradient checking) giúp phát hiện những lỗi đó bằng cách so độ dốc tự tính với độ dốc đo số học.

3.3.4.1 Đo độ dốc bằng số

Để đo độ dốc của hàm tại một điểm mà không cần đạo hàm, ta đẩy nhẹ điểm đó và xem hàm thay đổi bao nhiêu. Có hai cách (Hình 3.20):

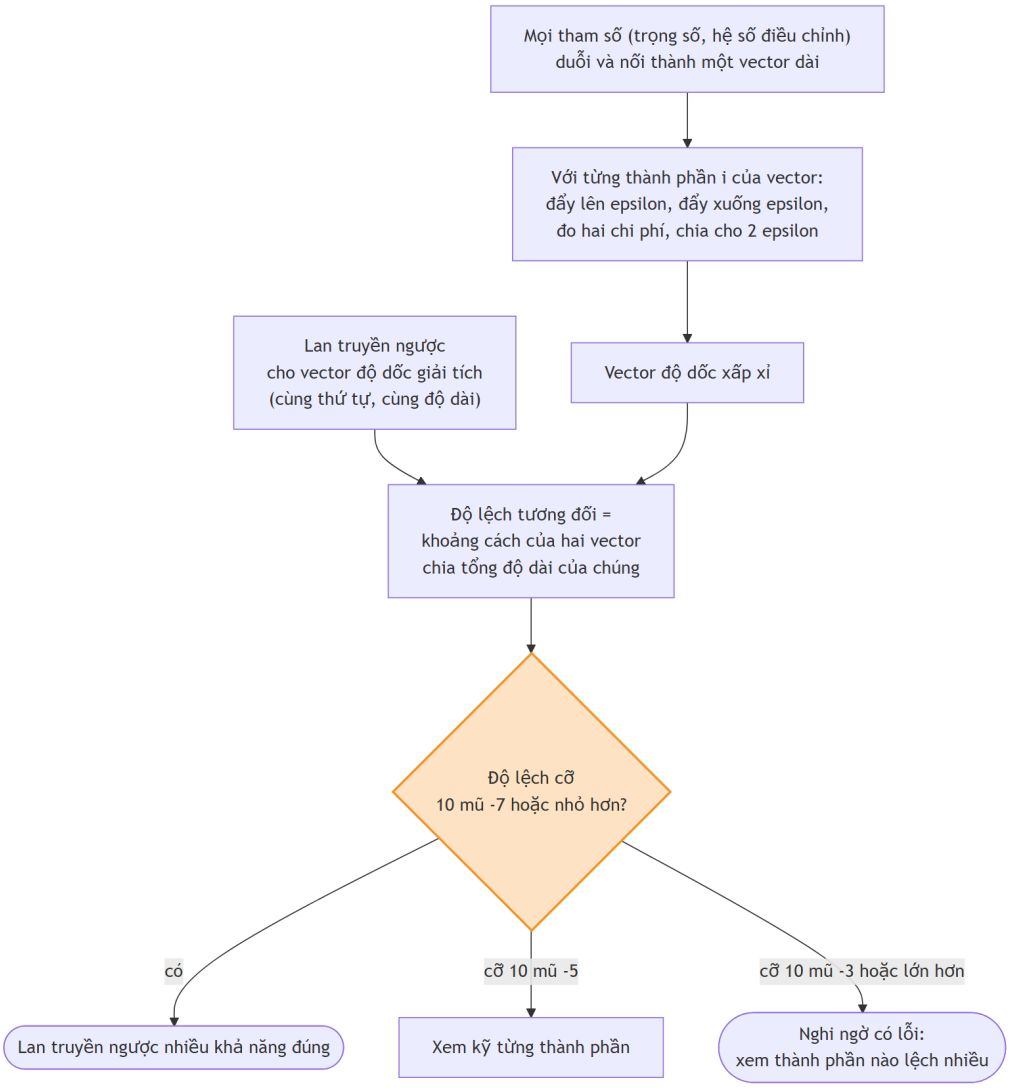


Hình 3.20: Sai phân một phía và hai phía, ví dụ với hàm lập phương tại điểm 1 và độ dịch 0,01

- **Một phía**: so giá trị tại điểm và tại điểm đẩy sang phải.
- **Hai phía**: so giá trị tại điểm đẩy sang trái và tại điểm đẩy sang phải, rồi chia cho hai lần độ dịch.

Ví dụ hàm lập phương tại 1, có độ dốc thật bằng 3 (ba lần bình phương của điểm). Với độ dịch 0,01, cách một phía cho 3,0301 (sai 0,03) còn cách hai phía cho 3,0001 (sai 0,0001). Cách hai phía chính xác hơn nhiều: sai số của nó cỡ bình phương độ dịch, còn cách một phía chỉ cỡ chính độ dịch. Cái giá là chậm gấp đôi, nhưng đáng.

3.3.4.2 Quy trình kiểm tra cho cả mạng



Hình 3.21: Quy trình kiểm tra độ dốc cho cả mạng

1. Duỗi và nối mọi trọng số và hệ số điều chỉnh thành một vector tham số dài. Với độ dốc từ lan truyền ngược, làm tương tự, được vector độ dốc cùng thứ tự và cùng độ dài.

2. Với từng thành phần của vector tham số: đẩy lên một lượng rất nhỏ, đo chi phí; đẩy xuống một lượng đó, đo chi phí; lấy hiệu chia cho hai lần độ dịch. Được vector độ dốc xấp xỉ.

3. So hai vector bằng **độ lệch tương đối**: khoảng cách giữa hai vector chia cho tổng độ dài của hai vector. Phép chia chuẩn hoá để kết quả có ý nghĩa dù các vector rất nhỏ hay rất lớn.

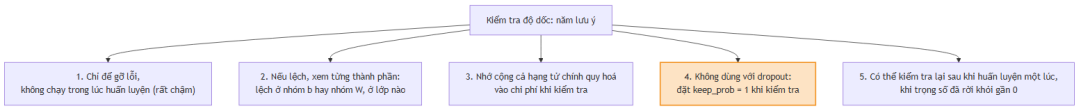
Với độ dịch cỡ một phần mười triệu (1e-7), cách đọc kết quả:

Độ lệch tương đối	Kết luận
Cỡ một phần mười triệu (1e-7) hoặc nhỏ hơn	rất tốt: lan truyền ngược nhiều khả năng đúng
Cỡ một phần trăm nghìn (1e-5)	xem kỹ từng thành phần, có thể vẫn ổn
Cỡ một phần nghìn (1e-3) hoặc lớn hơn	đáng lo: nhiều khả năng có lỗi

Bảng 3.10: Đọc độ lệch tương đối khi kiểm tra độ dốc với độ dịch cỡ 1e-7

Khi độ lệch lớn, nhìn từng thành phần: nếu các thành phần lệch nhiều đều thuộc nhóm hệ số điều chỉnh của một lớp mà nhóm trọng số vẫn khớp, lỗi nhiều khả năng nằm ở cách tính độ dốc theo hệ số điều chỉnh ở lớp đó. Nó không chỉ ra ngay chỗ lỗi nhưng thu hẹp phạm vi tìm.

3.3.4.3 Năm lưu ý thực tế



Hình 3.22: Năm lưu ý khi kiểm tra độ dốc

1. **Chỉ dùng để gỡ lỗi**, không chạy ở mỗi lần lặp của huấn luyện: nó phải tính chi phí hai lần cho mỗi tham số, rất chậm.
2. **Nếu lệch, xem từng thành phần** để lần ra nhóm tham số có lỗi.
3. **Nhớ hạng tử chính quy hoá** nếu chi phí của bạn có hạng tử đó; độ dốc phải gồm cả phần của hạng tử.
4. **Không dùng với dropout**, vì mỗi lần lặp dropout tắt một tập nút khác nhau nên không có hàm chi phí cố định để đo. Cách làm: đặt `keep_prob` bằng 1, kiểm tra độ dốc, rồi mới bật dropout.
5. Hiếm gặp nhưng có thể: lan truyền ngược đúng khi trọng số gần 0 (lúc mới khởi tạo) nhưng sai khi trọng số đã lớn lên. Nếu muốn chắc chắn, kiểm tra lại sau khi huấn luyện một lúc.

3.3.4.4 Trong code

Với một tham số duy nhất, hàm chi phí là tích của tham số và đầu vào, độ dốc thật bằng đầu vào. Hàm kiểm tra tính độ lệch tương đối theo đúng các bước trên:

```
def gradient_check(x, theta, epsilon=1e-7):
    theta_plus = theta + epsilon                                # đẩy lên
    theta_minus = theta - epsilon                               # đẩy xuống
    J_plus = forward_propagation(x, theta_plus)                 # chi phí khi đẩy lên
    J_minus = forward_propagation(x, theta_minus)               # chi phí khi đẩy
    ↪  xuống
    gradapprox = (J_plus - J_minus) / (2 * epsilon)             # độ dốc đo bằng số
    ↪  (hai phía)
    grad = backward_propagation(x, theta)                       # độ dốc từ lan
    ↪  truyền ngược

    numerator = np.linalg.norm(grad - gradapprox)                           # khoảng
    ↪  cách hai vector
    denominator = np.linalg.norm(grad) + np.linalg.norm(gradapprox)     # tổng độ
    ↪  dài
    return numerator / denominator
```

Với cả mạng, vòng lặp chạy qua từng thành phần của vector tham số:

```
def gradient_check_n(parameters, gradients, X, Y, epsilon=1e-7):
    parameters_values, _ = dictionary_to_vector(parameters)       # nối mọi tham
    ↪  số thành một vector
    grad = gradients_to_vector(gradients)                         # nối mọi độ
    ↪  dốc theo cùng thứ tự
    num_parameters = parameters_values.shape[0]
    J_plus = np.zeros((num_parameters, 1))
    J_minus = np.zeros((num_parameters, 1))
    gradapprox = np.zeros((num_parameters, 1))

    for i in range(num_parameters):
        theta_plus = np.copy(parameters_values)                   # copy để không
        ↪  đụng vector gốc
        theta_plus[i] = theta_plus[i] + epsilon                   # chỉ đẩy thành
        ↪  phần thứ i
        J_plus[i], _ = forward_propagation_n(X, Y,
        ↪  vector_to_dictionary(theta_plus))

        theta_minus = np.copy(parameters_values)
        theta_minus[i] = theta_minus[i] - epsilon
        J_minus[i], _ = forward_propagation_n(X, Y,
        ↪  vector_to_dictionary(theta_minus))

        gradapprox[i] = (J_plus[i] - J_minus[i]) / (2 * epsilon)

    numerator = np.linalg.norm(grad - gradapprox)
    denominator = np.linalg.norm(grad) + np.linalg.norm(gradapprox)
    return numerator / denominator
```

Kết quả chạy cho cả hai hàm trên nằm ở mục 3.4.3.

3.3.5 Áp dụng vào dự án thực tế

3.3.5.1 Danh sách việc cần làm

1. Chuẩn hoá mọi đặc trưng đầu vào bằng trung bình và độ lệch chuẩn của tập huấn luyện, dùng lại cho tập kiểm định và kiểm tra.
2. Chọn khởi tạo theo hàm kích hoạt: He cho ReLU, Xavier (hoặc mặc định của Keras) cho tanh; hệ số điều chỉnh khởi tạo bằng 0.
3. Nếu tự viết lan truyền ngược, kiểm tra độ dốc trước khi huấn luyện, với dropout tắt.
4. Sau khi kiểm tra xong, tắt kiểm tra độ dốc; dùng lan truyền ngược để học.

3.3.5.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Mô hình hoạt động tốt khi huấn luyện, kém hẳn khi kiểm tra dù không quá khớp	chuẩn hoá tập kiểm tra bằng thống kê của chính nó thay vì của tập huấn luyện; dùng `transform`, không `fit`
Chi phí giảm rất chậm ở mạng sâu	khởi tạo quá nhỏ hoặc quá lớn; dùng He hoặc Xavier
Chi phí lúc đầu vô cực hoặc rất lớn	khởi tạo trọng số quá lớn làm đầu ra sigmoid gần 0 hoặc 1 và đoán sai; giảm độ lớn khởi tạo
Mọi nút ẩn học giống nhau, mô hình thua cả hồi quy logistic	khởi tạo toàn 0 hoặc toàn giá trị giống nhau (mục 2.4.6)
Kiểm tra độ dốc cho độ lệch lớn	lỗi trong lan truyền ngược; xem thành phần nào lệch, kiểm tra cả hạng tử chính quy hoá
Kiểm tra độ dốc cho kết quả bất thường khi dùng dropout	dropout làm chi phí ngẫu nhiên; đặt `keep_prob` bằng 1

Bảng 3.11: Lỗi thường gặp liên quan đến chuẩn hoá, khởi tạo và kiểm tra độ dốc

3.3.5.3 Bảng tra cứu API

Việc cần làm	Công cụ
Chuẩn hoá bằng thống kê của tập huấn luyện	`StandardScaler.fit_transform` cho tập huấn luyện, `transform` cho phần còn lại
Chuẩn hoá ngay trong mô hình	`keras.layers.Normalization()` rồi `adapt(X_train)`
Khởi tạo He	`kernel_initializer="he_normal"`
Khởi tạo mặc định	`glorot_uniform` (mặc định của lớp `Dense`)
Khởi tạo hệ số điều chỉnh	`bias_initializer="zeros"`
Kiểm tra độ dốc tự viết	`np.linalg.norm` để tính khoảng cách và độ dài vector

Bảng 3.12: Công cụ cho chuẩn hoá, khởi tạo và kiểm tra độ dốc

3.3.5.4 Tóm tắt cần nhớ

- Chuẩn hoá đầu vào: trừ trung bình, chia độ lệch chuẩn, tính trên tập huấn luyện và dùng lại cho các tập khác. Bề mặt chi phí tròn hơn, gradient descent nhanh hơn.
- Mạng sâu dễ bị bùng nổ hoặc tiêu biến gradient vì tín hiệu bị nhân qua nhiều lớp.
- Khởi tạo He (ReLU) và Xavier (tanh) co hệ số theo số nút lớp trước để giảm vấn đề đó.
- Kiểm tra độ dốc bằng sai phân hai phía và độ lệch tương đối: cỡ một phần mười triệu là tốt, cỡ một phần nghìn là đáng lo.
- Kiểm tra độ dốc chậm, chỉ để gỡ lỗi, và phải tắt dropout.

3.4 Thực hành: khởi tạo, chính quy hoá và kiểm tra độ dốc

Mục này đưa ba kỹ thuật ở các mục trước vào thực hành bằng ba bài toán nhỏ, mỗi bài trả lời một câu hỏi:

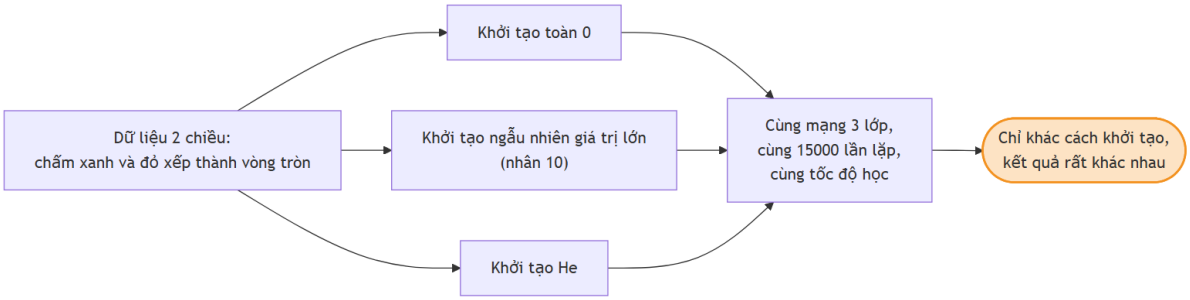
- **Khởi tạo**: cách khởi tạo trọng số ảnh hưởng thế nào đến kết quả? (mục 3.4.1)
- **Chính quy hoá**: L2 và dropout giúp mô hình quá khớp được bao nhiêu? (mục 3.4.2)
- **Kiểm tra độ dốc**: làm sao biết lan truyền ngược viết tay là đúng? (mục 3.4.3)

Các con số trong mục này là kết quả của chính bài thực hành gốc (cùng dữ liệu, cùng kiến trúc, cùng siêu tham số); mục không tự đo thêm số liệu nào khác. Phần code tập trung vào các đoạn quan trọng; các hàm trợ giúp như `forward_propagation`, `predict` có sẵn trong các tệp tiện ích của bài.

3.4.1 Thực hành 1: ảnh hưởng của cách khởi tạo

3.4.1.1 Bài toán

Dữ liệu hai chiều gồm các chấm xanh và đỏ xếp thành hai vòng tròn lồng nhau; mô hình cần tách hai màu. Mạng dùng chung cho cả ba thí nghiệm: ba lớp (hai lớp ẩn 10 và 5 nút dùng ReLU, lớp ra sigmoid), tốc độ học 0,01, 15 000 lần lặp. Duy nhất cách khởi tạo thay đổi (Hình 3.23).



Hình 3.23: Ba cách khởi tạo trên cùng một mạng và cùng dữ liệu

```
def model(X, Y, learning_rate=0.01, num_iterations=15000, print_cost=True,
↪   initialization="he"):
    layers_dims = [X.shape[0], 10, 5, 1]                    # 2 đặc trưng, hai
    ↪  lớp ẩn, 1 nút ra
    if initialization == "zeros":
        parameters = initialize_parameters_zeros(layers_dims)
    elif initialization == "random":
        parameters = initialize_parameters_random(layers_dims)
    elif initialization == "he":
        parameters = initialize_parameters_he(layers_dims)

    for i in range(num_iterations):
        a3, cache = forward_propagation(X, parameters)      # lan truyền xuôi
        cost = compute_loss(a3, Y)                          # chi phí
        grads = backward_propagation(X, Y, cache)           # lan truyền ngược
        parameters = update_parameters(parameters, grads, learning_rate)
    return parameters
```

3.4.1.2 Ba hàm khởi tạo

```
def initialize_parameters_zeros(layers_dims):
    parameters = {}
    for l in range(1, len(layers_dims)):
        parameters['W' + str(l)] = np.zeros((layers_dims[l], layers_dims[l-1]))
        parameters['b' + str(l)] = np.zeros((layers_dims[l], 1))
    return parameters
```

```
def initialize_parameters_random(layers_dims):                # trọng số ngẫu
↪  nhiên nhưng nhân với 10: RẤT LỚN
    np.random.seed(3)
    parameters = {}
    for l in range(1, len(layers_dims)):
        parameters['W' + str(l)] = np.random.randn(layers_dims[l],
        ↪  layers_dims[l-1]) * 10
        parameters['b' + str(l)] = np.zeros((layers_dims[l], 1))
    return parameters

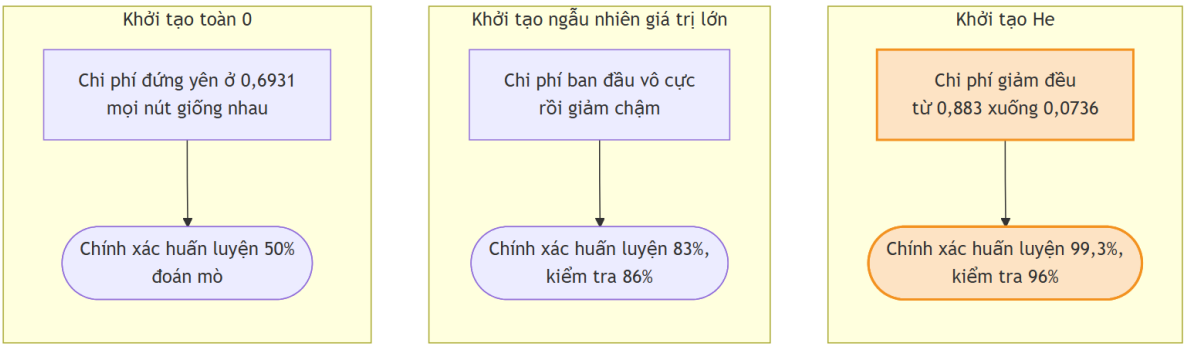
# initialize_parameters_he: đã trình bày ở mục 3.3, nhân số Gauss với căn bậc hai
↪  của (2 / số nút lớp trước)

parameters = initialize_parameters_he([2, 4, 1])
print(parameters["W1"])
# [[ 1.78862847  0.43650985]
#  [ 0.09649747 -1.8634927 ]
#  [-0.2773882  -0.35475898]
#  [-0.08274148 -0.62700068]]
```

3.4.1.3 Kết quả

Khởi tạo	Lần 0	Lần 1000	Lần 7000	Lần 14000	Chính xác huấn luyện / kiểm tra	Nhận xét
Toàn 0	0,6931	0,6931	0,6931	0,6931	50% / 50%	chi phí đứng yên; không phá được đối xứng
Ngẫu nhiên, giá trị lớn	vô cực	0,6248	0,4716	0,3828	83% / 86%	trọng số quá lớn, học chậm
He	0,8831	0,6880	0,3118	0,0736	99,3% / 96%	khuyến nghị

Bảng 3.13: Chi phí ở các lần lặp và độ chính xác của ba cách khởi tạo, cùng mạng, cùng 15 000 lần lặp, cùng siêu tham số



Hình 3.24: Tóm tắt ba cách khởi tạo

Cách đọc từng dòng:

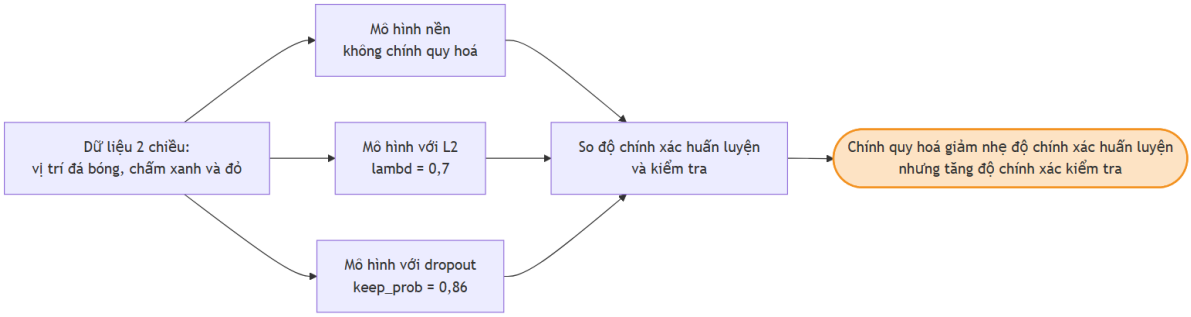
- **Toàn 0.** Mọi tổng có trọng số bằng 0 nên mọi kích hoạt ReLU bằng 0. Lớp ra sigmoid nhận 0 và cho đúng 0,5 với mọi mẫu, nên chi phí luôn bằng mất mát của việc không biết gì (logarit của hai, tức 0,6931) và độ dốc không đủ thông tin để điều chỉnh gì. Mô hình đoán 0 cho mọi ảnh. Kết luận: khởi tạo toàn 0 làm các nút giữ nguyên đối xứng; mạng không mạnh hơn một bộ phân loại tuyến tính như hồi quy logistic.
- **Ngẫu nhiên giá trị lớn.** Đã phá được đối xứng nên độ chính xác tăng rõ (83% huấn luyện, 86% kiểm tra). Nhưng chi phí lúc đầu rất cao, thậm chí báo vô cực: với trọng số lớn, sigmoid ở lớp ra cho giá trị rất gần 0 hoặc 1; khi đoán sai một mẫu, mất mát của mẫu đó cực lớn, và khi logarit chạm đúng 0 thì ra vô cực (do làm tròn số). Khởi tạo kém cũng gây bùng nổ hoặc tiêu biến gradient, làm tối ưu chậm. Huấn luyện lâu hơn sẽ khá hơn, nhưng khởi tạo quá lớn làm chậm đáng kể.
- **He.** Chi phí giảm đều từ 0,8831 xuống 0,0736, độ chính xác huấn luyện 99,3% và kiểm tra 96%. Với mạng ReLU, đây là cách khuyến nghị.

Gaussian hay đều? `np.random.randn` lấy số theo phân phối chuẩn (Gauss), tập trung quanh giữa; `np.random.rand` lấy số đều trong khoảng 0 đến 1. Dùng Gauss cho trọng số giúp phần lớn giá trị không rơi vào hai đầu cực, nơi sigmoid gần phẳng và học chậm.

3.4.2 Thực hành 2: chính quy hoá L2 và dropout

3.4.2.1 Bài toán

Một đội bóng cần khuyến nghị vị trí thủ môn nên phát bóng sao cho cầu thủ nhà đánh đầu được. Dữ liệu là các vị trí trên sân từ 10 trận trước: chấm xanh là cầu thủ nhà đánh đầu được, chấm đỏ là cầu thủ đối phương. Dữ liệu hơi nhiễu nhưng nhìn chung một đường chéo đã tách khá tốt. Mạng dùng ba lớp (hai lớp ẩn 20 và 3 nút), tốc độ học 0,3, 30 000 lần lặp (Hình 3.25).



Hình 3.25: Ba mô hình trên cùng dữ liệu: nền, L2 và dropout

Hàm `model` nhận hai tham số để chọn chế độ: `lambd` khác 0 thì dùng L2, `keep_prob` nhỏ hơn 1 thì dùng dropout. Hai chế độ không dùng chung trong bài này.

```
def model(X, Y, learning_rate=0.3, num_iterations=30000, print_cost=True,
↪   lambd=0, keep_prob=1):
    layers_dims = [X.shape[0], 20, 3, 1]
    parameters = initialize_parameters(layers_dims)
    for i in range(0, num_iterations):
        # lan truyền xuôi: có hoặc không có dropout
        if keep_prob == 1:
            a3, cache = forward_propagation(X, parameters)
        elif keep_prob < 1:
            a3, cache = forward_propagation_with_dropout(X, parameters,
            ↪   keep_prob)
        # chi phí: có hoặc không có hình phạt L2
        if lambd == 0:
            cost = compute_cost(a3, Y)
        else:
            cost = compute_cost_with_regularization(a3, Y, parameters, lambd)
        # lan truyền ngược: chọn đúng hàm
        assert (lambd == 0 or keep_prob == 1)          # chỉ dùng một trong hai
        if lambd == 0 and keep_prob == 1:
            grads = backward_propagation(X, Y, cache)
        elif lambd != 0:
            grads = backward_propagation_with_regularization(X, Y, cache, lambd)
        elif keep_prob < 1:
            grads = backward_propagation_with_dropout(X, Y, cache, keep_prob)
        parameters = update_parameters(parameters, grads, learning_rate)
    return parameters

parameters = model(train_X, train_Y)                                    # nền:
↪   không chính quy hoá
parameters = model(train_X, train_Y, lambd=0.7)                         # L2
parameters = model(train_X, train_Y, keep_prob=0.86, learning_rate=0.3)   #
↪   dropout: tắt 14% nút mỗi lớp ẩn
```

Hàm chi phí có L2 và hàm lan truyền xuôi, ngược có dropout đã trình bày ở mục 3.2.1 và mục 3.2.2. Dropout ở đây áp dụng cho hai lớp ẩn, không áp dụng cho lớp vào và lớp ra. Chạy kiểm tra nhỏ cho chi phí có hình phạt (`lambd` bằng 0,1) trên dữ liệu thử của bài cho giá trị 1,7865.

3.4.2.2 Kết quả

Mô hình	Chi phí lần 0	Lần 20000	Chính xác huấn luyện / kiểm tra	Nhận xét
Không chính quy hoá	0,6557	0,1385	94,8% / 91,5%	khớp cả điểm nhiễu
L2 (`lambd` = 0,7)	0,6974	0,2681	93,8% / 93%	ranh giới trơn hơn
Dropout (`keep_prob` = 0,86)	0,6544	0,0606	92,9% / 95%	tốt nhất trên kiểm tra

Bảng 3.14: Kết quả ba mô hình trên dữ liệu bóng đá (mạng 20 và 3 nút ẩn, tốc độ học 0,3, 30 000 lần lặp)

Ba điều đáng rút ra:

- **Chính quy hoá làm giảm độ chính xác huấn luyện nhưng tăng độ chính xác kiểm tra.** Giảm từ 94,8% xuống 93,8% rồi 92,9% ở tập huấn luyện, nhưng tăng từ 91,5% lên 93% rồi 95% ở tập kiểm tra. Lý do: chính quy hoá hạn chế khả năng khớp quá mức vào tập huấn luyện, nên điểm huấn luyện thấp đi, nhưng mô hình tổng quát hoá tốt hơn.
- **Chi phí của các mô hình không so được trực tiếp.** Chi phí của L2 (0,268) cao hơn mô hình nền (0,139) vì nó đã cộng hình phạt; không phải mô hình L2 tệ hơn.
- **`lambd` là siêu tham số.** L2 làm ranh giới trơn hơn; nếu `lambd` quá lớn thì làm phẳng quá mức và quay lại thiên lệch cao. Chọn bằng tập kiểm định.

Lỗi hay mắc. Dùng dropout cả khi huấn luyện lẫn khi kiểm tra. Chỉ dùng lúc huấn luyện; `predict` không được tắt nút. Các framework như Keras, TensorFlow, PaddlePaddle và Caffe đều có sẵn lớp dropout.

3.4.3 Thực hành 3: kiểm tra độ dốc

3.4.3.1 Bài toán

Bạn xây mô hình phát hiện gian lận cho thanh toán di động. Đây là ứng dụng quan trọng, nên cần chứng minh lan truyền ngược đúng. Công cụ là kiểm tra độ dốc: so độ dốc từ lan truyền ngược với độ dốc đo bằng cách đẩy nhẹ tham số (mục 3.3.4). Làm hai lần: một chiều trước, rồi nhiều chiều.

3.4.3.2 Một chiều

Mô hình đơn giản nhất: hàm chi phí là tích của một tham số và một đầu vào, nên độ dốc thật chính là đầu vào. Với đầu vào bằng 3 và tham số bằng 4, hàm chi phí bằng 12, độ dốc thật bằng 3. Hàm `gradient_check` ở mục 3.3.4 cho độ lệch tương đối:

```
x, theta = 3, 4
difference = gradient_check(x, theta, print_msg=True)
# Your backward propagation works perfectly fine! difference =
↪  7.814075313343006e-11
```

Độ lệch cỡ 7,8e-11, nhỏ hơn ngưỡng 2e-7 mà bài đặt ra, nên có thể tin lan truyền ngược trong trường hợp này đúng.

3.4.3.3 Nhiều chiều

Mạng thật có nhiều ma trận trọng số và vector hệ số điều chỉnh. Hàm `dictionary_to_vector` nối mọi tham số thành một vector; `gradients_to_vector` làm tương tự với độ dốc. Mạng có kiến trúc: ba lớp (trọng số `(5, 4)`, `(3, 5)`, `(1, 3)`), hai lớp ẩn ReLU và lớp ra sigmoid. Hàm lan truyền ngược mà bài cung cấp có hai lỗi cố ý:

```
dW2 = 1. / m * np.dot(dZ2, A1.T) * 2            # LỖI: nhân thừa 2
db1 = 4. / m * np.sum(dZ1, axis=1, keepdims=True)   # LỖI: nhân thừa 4
```

Chạy kiểm tra độ dốc bằng `gradient_check_n` (mục 3.3.4):

```
X, Y, parameters = gradient_check_n_test_case()
cost, cache = forward_propagation_n(X, Y, parameters)
gradients = backward_propagation_n(X, Y, cache)
difference = gradient_check_n(parameters, gradients, X, Y, 1e-7, True)
# There is a mistake in the backward propagation! difference = 0.2850931567761623
```

Độ lệch 0,285, lớn hơn ngưỡng rất nhiều: kiểm tra độ dốc đã cảnh báo đúng. Gợi ý của bài là nhìn dW2 và db1, đúng hai dòng có lỗi ở trên. Sửa hai dòng về dạng chuẩn (chia cho số mẫu, không nhân thừa) thì độ lệch rơi về cỡ một phần mười triệu: bài ghi giá trị mong đợi sau khi sửa là khoảng 1,19e-07.

Hai điều cần nhớ từ bài:

- Kiểm tra độ dốc xác nhận độ dốc từ lan truyền ngược khớp với độ dốc xấp xỉ bằng cách chỉ dùng lan truyền xuôi (vì ta tin hàm chi phí đúng hơn).
- Nó chậm nên chỉ chạy vài lần để kiểm tra, không chạy trong lúc huấn luyện. Và nó không dùng được nguyên trạng với dropout.

3.4.4 Áp dụng vào dự án thực tế

3.4.4.1 Danh sách việc cần làm

1. Dùng khởi tạo He cho mạng ReLU; không bao giờ khởi tạo toàn 0, và tránh trọng số ngẫu nhiên quá lớn.
2. Nếu quá khớp, thử dropout và L2 và so sánh trên tập kiểm định; nhớ chi phí có hình phạt không so được với chi phí không có.
3. Khi tự viết lan truyền ngược, chạy kiểm tra độ dốc một lần trên dữ liệu nhỏ, với dropout tắt.
4. Khi độ lệch lớn, tìm ở các dòng tính độ dốc theo từng nhóm tham số, như bài làm với `dW2` và `db1`.

3.4.4.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Chi phí đứng yên ở 0,6931, độ chính xác 50%	khởi tạo toàn 0 hoặc toàn giá trị giống nhau; khởi tạo ngẫu nhiên có độ lớn phù hợp
Chi phí ban đầu báo vô cực	trọng số quá lớn, logarit của 0; giảm độ lớn khởi tạo
Mô hình L2 có chi phí cao hơn mô hình nền nên tưởng tệ hơn	chi phí L2 đã cộng hình phạt; so bằng độ chính xác kiểm tra
Kết quả kiểm tra thay đổi mỗi lần gọi khi dùng dropout	vẫn bật dropout lúc dự đoán
Độ lệch kiểm tra độ dốc cỡ một phần mười	có lỗi ở một nhóm độ dốc; xem từng thành phần

Bảng 3.15: Lỗi thường gặp trong ba bài thực hành

3.4.4.3 Tóm tắt cần nhớ

- Cùng mạng, cùng dữ liệu, chỉ đổi cách khởi tạo: toàn 0 cho 50%, ngẫu nhiên giá trị lớn cho 83%, He cho 99,3% (độ chính xác huấn luyện).
- Chính quy hoá đổi một chút độ chính xác huấn luyện lấy độ chính xác kiểm tra: từ 91,5% lên 93% (L2) và 95% (dropout).
- Chi phí có hình phạt không so được với chi phí không có hình phạt.
- Kiểm tra độ dốc phát hiện đúng lỗi cố ý (độ lệch 0,285 so với ngưỡng 2e-7) và xác nhận trường hợp đúng (7,8e-11).

3.5 Thuật toán tối ưu

Học máy ứng dụng là quá trình thực nghiệm lặp đi lặp lại: huấn luyện rất nhiều mô hình để tìm ra cái tốt. Học sâu lại chạy tốt nhất trên dữ liệu lớn, mà huấn luyện trên dữ liệu lớn thì chậm. Vì vậy một **thuật toán tối ưu nhanh** có thể đổi chuyện chờ nhiều ngày thành chờ vài giờ, và tăng hiệu quả của cả đội.

Mục này đi từ cách dùng dữ liệu hiệu quả hơn (mini-batch), qua một công cụ toán nhỏ nhưng nền tảng (trung bình mũ), đến ba bộ tối ưu thay gradient descent thuần: momentum, RMSprop và Adam. Cuối cùng là giảm tốc độ học và vài hiểu biết về bề mặt chi phí của mạng nơ-ron.

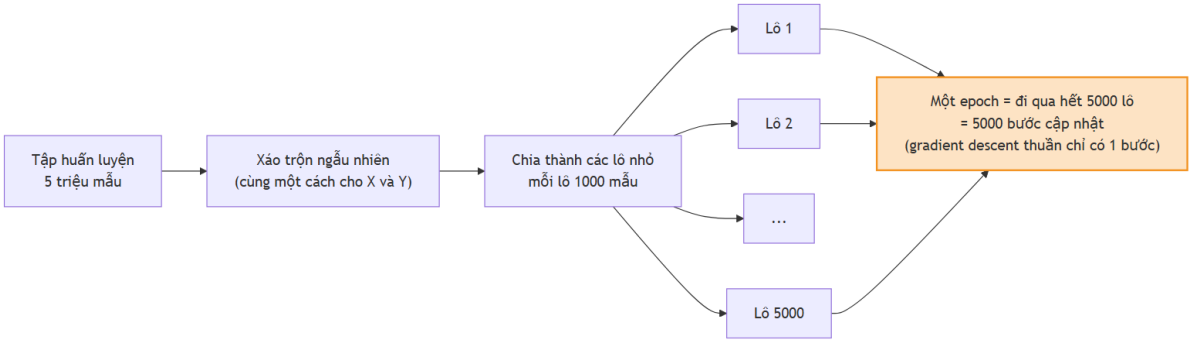
3.5.1 Mini-batch gradient descent

3.5.1.1 Vấn đề của gradient descent theo cả tập

Vector hoá cho phép tính trên mọi mẫu cùng lúc mà không cần vòng lặp. Nhưng nếu số mẫu cực lớn (năm triệu, năm mươi triệu), một bước gradient descent vẫn phải duyệt *toàn bộ* tập huấn luyện rồi mới đi được một bước nhỏ, rồi lại duyệt toàn bộ để đi bước kế tiếp. Có cách nhanh hơn: cho gradient descent bắt đầu tiến bộ **ngay cả trước khi duyệt hết tập**.

3.5.1.2 Ý tưởng: chia thành các lô nhỏ

Chia tập huấn luyện thành nhiều "tập huấn luyện nhỏ" gọi là **mini-batch** (lô nhỏ). Ví dụ năm triệu mẫu, mỗi lô 1000 mẫu, ta có 5000 lô (Hình 3.26). Ký hiệu trong tài liệu: lô thứ t gồm một ma trận đặc trưng (số đặc trưng, 1000) và một vector nhãn (1, 1000). Ở tài liệu này, lô được đánh số bằng chỉ số trong ngoặc nhọn để không lẫn với chỉ số lớp (ngoặc vuông) và chỉ số mẫu (ngoặc tròn) đã dùng trước đây.



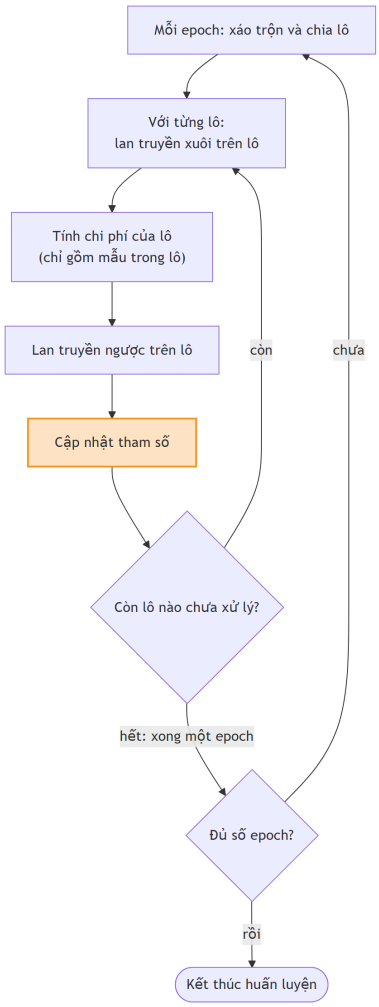
Hình 3.26: Xáo trộn rồi chia thành các lô nhỏ; mỗi epoch đi qua hết các lô

Hai tên cần phân biệt:

- **Batch gradient descent**: gradient descent xử lý toàn bộ tập huấn luyện mỗi bước (cách dùng ở các mục trước).
- **Mini-batch gradient descent**: xử lý *một lô* mỗi bước.

3.5.1.3 Một vòng huấn luyện

Với mỗi lô, ta làm đúng như gradient descent quen thuộc nhưng chỉ trên lô đó (Hình 3.27): lan truyền xuôi trên lô (vẫn vector hoá, nhưng chỉ 1000 mẫu thay vì năm triệu), tính chi phí chỉ trên lô, lan truyền ngược, cập nhật tham số. Một lần đi qua toàn bộ tập huấn luyện gọi là một **epoch**. Với batch gradient descent, một epoch chỉ có một bước cập nhật; với mini-batch, một epoch có 5000 bước. Thường ta cần nhiều epoch, nên có thêm một vòng lặp bên ngoài.



Hình 3.27: Vòng huấn luyện mini-batch: mỗi epoch xáo trộn lại rồi duyệt các lô

Khi tập huấn luyện lớn, mini-batch gradient descent chạy nhanh hơn hẳn và là lựa chọn mặc định trong học sâu.

3.5.1.4 Cách làm trong code

Phần việc của bạn gồm hai bước: **xáo trộn** cặp (X, Y) theo cùng một hoán vị, và **chia** thành các lô. Lô cuối có thể nhỏ hơn nếu số mẫu không chia hết cho cỡ lô:

```
def random_mini_batches(X, Y, mini_batch_size=64, seed=0):
    np.random.seed(seed)
    m = X.shape[1]                                  # số mẫu
    mini_batches = []

    # bước 1: xáo trộn X và Y theo cùng một hoán vị
    permutation = list(np.random.permutation(m))
    shuffled_X = X[:, permutation]
    shuffled_Y = Y[:, permutation].reshape((1, m))

    # bước 2: chia thành các lô đủ cỡ
    inc = mini_batch_size
    num_complete_minibatches = math.floor(m / mini_batch_size)
    for k in range(0, num_complete_minibatches):
```

```
        mini_batch_X = shuffled_X[:, k * inc : (k + 1) * inc]
        mini_batch_Y = shuffled_Y[:, k * inc : (k + 1) * inc]
        mini_batches.append((mini_batch_X, mini_batch_Y))

    # lô cuối: phần còn dư, có thể nhỏ hơn mini_batch_size
    if m % mini_batch_size != 0:
        mini_batch_X = shuffled_X[:, num_complete_minibatches * inc :]
        mini_batch_Y = shuffled_Y[:, num_complete_minibatches * inc :]
        mini_batches.append((mini_batch_X, mini_batch_Y))
    return mini_batches
```

Bài thực hành gốc kiểm tra hàm này với 148 mẫu 12288 đặc trưng và cỡ lô 64: kết quả là ba lô, hình dạng ma trận đặc trưng lần lượt `(12288, 64)`, `(12288, 64)`, `(12288, 20)`, vector nhãn `(1, 64)`, `(1, 64)`, `(1, 20)`. Lô cuối nhỏ hơn vì 148 không chia hết cho 64.

Với Keras, bạn chỉ cần đặt `batch_size`; việc xáo trộn và chia lô do framework làm:

```
model.fit(X_train, y_train, epochs=100, batch_size=64, shuffle=True)    # mỗi
↪  epoch xáo trộn lại rồi duyệt từng lô
```

3.5.1.5 Đường chi phí và ba chế độ

Với batch gradient descent, chi phí phải giảm ở mỗi lần lặp; nếu có lần tăng thì có điều gì đó sai (ví dụ tốc độ học quá lớn). Với mini-batch, mỗi lần lặp bạn đang huấn luyện trên một lô khác nhau, và có lô dễ, có lô khó (ví dụ có vài mẫu gán nhãn nhầm). Nên đường chi phí vẽ theo lần lặp có xu hướng giảm nhưng **nhiễu hơn**. Dao động như vậy là bình thường.

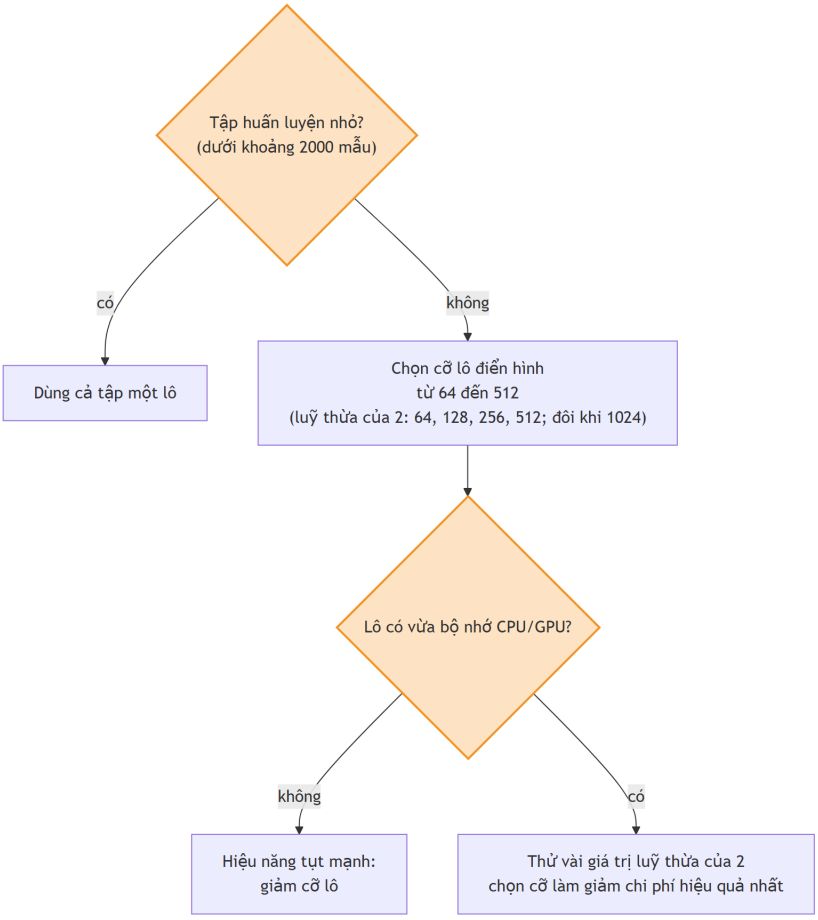
Cỡ lô là một tham số phải chọn; hai thái cực (Hình 3.28):



Hình 3.28: Cỡ lô bằng cả tập, vừa phải, và bằng một mẫu

- **Cỡ lô bằng cả tập**: chính là batch gradient descent. Mỗi bước ổn định nhưng quá lâu với tập lớn.
- **Cỡ lô bằng 1** (mỗi mẫu một lô): gọi là **stochastic gradient descent** (SGD). Tiến bộ ngay sau mỗi mẫu, nhưng rất nhiễu (có thể giảm bằng tốc độ học nhỏ hơn), và bạn mất gần hết lợi ích tăng tốc của vector hoá vì chỉ xử lý một mẫu mỗi lần. SGD không bao giờ hội tụ hẳn mà chỉ quanh quẩn gần cực tiểu.
- **Ở giữa**: vừa tận dụng vector hoá (xử lý cả lô cùng lúc), vừa tiến bộ không phải đợi hết cả tập. Thực tế cho tốc độ học nhanh nhất.

3.5.1.6 Chọn cỡ lô



Hình 3.29: Gợi ý chọn cỡ lô

- Tập nhỏ (dưới khoảng 2000 mẫu): dùng cả tập một lô.
- Tập lớn hơn: cỡ lô điển hình từ 64 đến 512; thấy cả 1024 nhưng ít hơn.
- Chọn luỹ thừa của 2 (64, 128, 256, 512, …) vì cách bố trí bộ nhớ máy tính có thể làm code chạy nhanh hơn.
- Đảm bảo một lô vừa bộ nhớ CPU hoặc GPU. Nếu không, hiệu năng tụt mạnh.
- Cỡ lô là một siêu tham số: thử vài giá trị luỹ thừa của 2 rồi chọn cái làm giảm chi phí hiệu quả nhất.

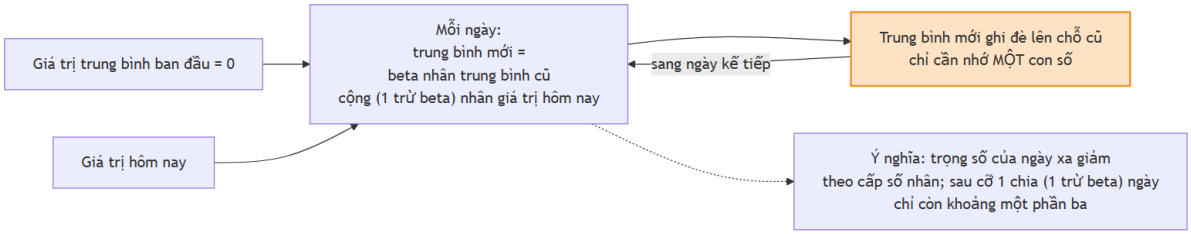
3.5.2 Trung bình mũ (exponentially weighted average)

Các bộ tối ưu nhanh hơn gradient descent đều dựa vào một công cụ nhỏ: **trung bình mũ**, còn gọi là trung bình trượt có trọng số mũ.

3.5.2.1 Ví dụ nhiệt độ hàng ngày

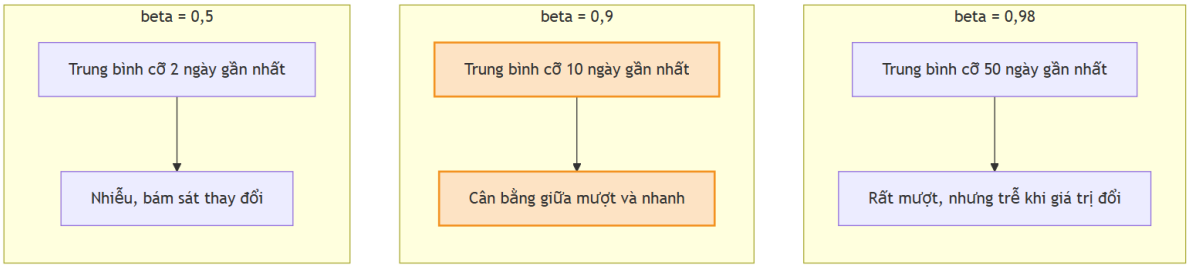
Giả sử bạn có nhiệt độ mỗi ngày trong năm tại London; đường dữ liệu khá nhiễu. Để thấy xu hướng, ta tính một "trung bình trượt" như sau. Bắt đầu với giá trị trung bình bằng 0. Mỗi ngày, lấy **0,9 lần trung**

bình ngày hôm trước cộng 0,1 lần nhiệt độ hôm nay (Hình 3.30). Tổng quát với hệ số `beta`: trung bình mới bằng `beta` nhân trung bình cũ, cộng (1 trừ `beta`) nhân giá trị hôm nay.



Hình 3.30: Cập nhật trung bình mũ: chỉ cần nhớ một con số

Hệ số `beta` quyết định cửa sổ trung bình (Hình 3.31): giá trị trung bình xấp xỉ trung bình của khoảng **một chia (1 trừ beta)** ngày gần nhất.



Hình 3.31: `beta` lớn thì mượt hơn nhưng phản ứng chậm hơn khi giá trị đổi

beta	Trung bình xấp xỉ qua	Hành vi
0,5	khoảng 2 ngày	nhiễu, nhạy với giá trị ngoại lai nhưng đổi theo rất nhanh
0,9	khoảng 10 ngày	cân bằng
0,98	khoảng 50 ngày	mượt hơn nhiều, nhưng đường cong dịch sang phải (trễ) khi nhiệt độ đổi

Bảng 3.16: `beta` và cửa sổ trung bình

Lý do đúng là "khoảng một chia (1 trừ beta) ngày": khi khai triển, trung bình hôm nay là tổng của nhiệt độ các ngày trước nhân với trọng số giảm theo cấp số nhân (hôm nay 0,1; hôm qua 0,1 nhân 0,9; hôm kia 0,1 nhân 0,9 bình phương; …). Trọng số của một ngày cách đây 10 ngày bằng khoảng một phần ba trọng số hôm nay (0,9 mũ 10 xấp xỉ 0,35). Với `beta` bằng 0,98 thì mốc tương ứng là 50 ngày. Đây là quy tắc ngón tay cái, không phải phát biểu toán học chặt.

3.5.2.2 Vì sao dùng

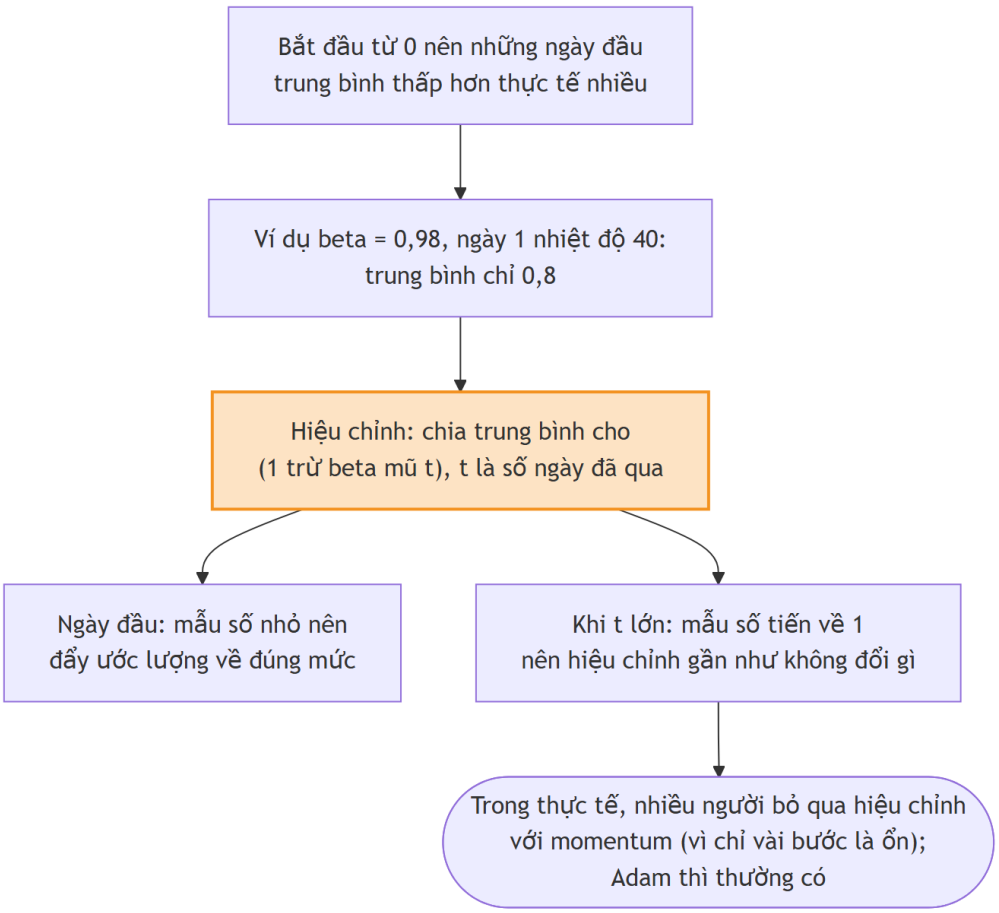
Cách này **rất tiết kiệm**: chỉ cần giữ một con số trong bộ nhớ và cập nhật bằng một dòng code. Tính trung bình bằng cửa sổ trượt thật (cộng 10 hoặc 50 giá trị gần nhất) thường chính xác hơn một chút, nhưng phải lưu cả cửa sổ, phức tạp và tốn bộ nhớ hơn. Khi cần tính trung bình cho hàng triệu tham số (như ở các bộ tối ưu bên dưới), trung bình mũ vượt trội.

```
beta = 0.9
v = 0.0                                          # khởi tạo bằng 0
for theta_t in nhiet_do_moi_ngay:                # lặp qua từng ngày
```

```
    v = beta * v + (1 - beta) * theta_t              # trung bình mũ: chỉ một
    ↪  dòng, chỉ một biến
```

3.5.2.3 Hiệu chỉnh độ lệch (bias correction)

Vì bắt đầu từ 0, những ngày đầu trung bình thấp hơn thực tế rất nhiều (Hình 3.32). Với `beta` bằng 0,98 và nhiệt độ ngày đầu là 40, trung bình ngày 1 chỉ là 0,8. Ngày 2 cũng còn thấp. Cách sửa: **chia trung bình cho (1 trừ `beta` mũ số ngày đã qua)**. Khi số ngày còn nhỏ, mẫu số nhỏ nên bù lại phần thiếu. Khi số ngày lớn, `beta` mũ số ngày đó tiến về 0, mẫu số về 1, và hiệu chỉnh hầu như không đổi gì.



Hình 3.32: Hiệu chỉnh độ lệch giúp ước lượng đúng trong giai đoạn khởi động

```
v, t = 0.0, 0
for theta_t in nhiet_do_moi_ngay:
    t += 1
    v = beta * v + (1 - beta) * theta_t
    v_da_hieu_chinh = v / (1 - beta ** t)            # chia cho (1 - beta^t): bù
    ↪  độ lệch giai đoạn đầu
```

Trong thực tế, đa số cài đặt không bận tâm hiệu chỉnh này (chấp nhận vài bước đầu hơi lệch), nhưng nếu quan tâm đến giai đoạn khởi động thì nên dùng. Adam (mục 3.5.5) thì có dùng hiệu chỉnh.

3.5.3 Momentum

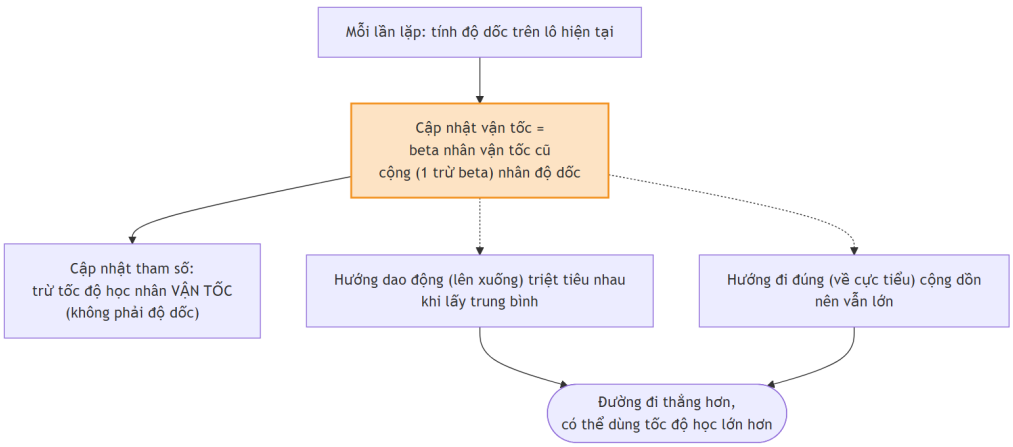
3.5.3.1 Vấn đề

Giả sử các đường đồng mức của chi phí dẹt như một ellipse. Gradient descent thuần dao động lên xuống và đi chậm về phía cực tiểu. Dao động đó cũng cản bạn dùng tốc độ học lớn: nếu lớn, nó bắn quá đà và phân kỳ. Điều ta muốn: ở hướng dọc (lên xuống) học chậm lại để bớt dao động, ở hướng ngang (về phía cực tiểu) học nhanh.

3.5.3.2 Ý tưởng

Gradient descent với momentum tính *trung bình mũ của các độ dốc* rồi dùng nó thay cho độ dốc để cập nhật tham số (Hình 3.33). Mỗi lần lặp:

1. Tính độ dốc trên lô hiện tại.
2. Cập nhật "vận tốc": vận tốc mới bằng `beta` nhân vận tốc cũ cộng (1 trừ `beta`) nhân độ dốc.
3. Cập nhật tham số: trừ tốc độ học nhân *vận tốc* (không phải độ dốc).



Hình 3.33: Momentum: dùng trung bình của các độ dốc gần đây

Vì sao có ích: lấy trung bình vài độ dốc liên tiếp, thành phần dao động lên xuống (lúc dương, lúc âm) triệt tiêu nhau gần về 0, còn thành phần hướng về cực tiểu (luôn cùng chiều) vẫn lớn. Kết quả: đường đi thẳng hơn, ít dao động. Có người thích hình dung quả bóng lăn xuống bát: độ dốc đóng vai gia tốc, `beta` nhỏ hơn 1 đóng vai ma sát; nếu cách hình dung này không hợp bạn thì bỏ qua.

3.5.3.3 Siêu tham số và chi tiết

- Có hai siêu tham số: tốc độ học và `beta`. Giá trị `beta` phổ biến nhất là 0,9 (trung bình qua khoảng 10 lần lặp gần nhất); đây là một giá trị rất vững. Có thể thử trong khoảng từ 0,8 đến 0,999.
- Vận tốc được khởi tạo bằng 0, cùng hình dạng với từng nhóm tham số.
- Thường không cần hiệu chỉnh độ lệch vì sau khoảng mười lần lặp thì trung bình đã "ấm lên".
- Một số tài liệu bỏ hệ số (1 trừ `beta`) trong cập nhật vận tốc. Hai cách đều chạy tốt; chúng chỉ khác thang đo của vận tốc, nên tốc độ học tốt nhất đổi theo. Cách có hệ số (1 trừ `beta`) dễ chỉnh hơn vì đổi `beta` không làm đổi thang đo.

```
def initialize_velocity(parameters):
    L = len(parameters) // 2
    v = {}
    for l in range(1, L + 1):
        v["dW" + str(l)] = np.zeros(parameters["W" + str(l)].shape)       # vận
        ↪  tốc ban đầu bằng 0
        v["db" + str(l)] = np.zeros(parameters["b" + str(l)].shape)
    return v

def update_parameters_with_momentum(parameters, grads, v, beta, learning_rate):
    L = len(parameters) // 2
    for l in range(1, L + 1):
        v["dW" + str(l)] = beta * v["dW" + str(l)] + (1 - beta) * grads["dW" +
        ↪  str(l)]       # cập nhật vận tốc
        v["db" + str(l)] = beta * v["db" + str(l)] + (1 - beta) * grads["db" +
        ↪  str(l)]
        parameters["W" + str(l)] = parameters["W" + str(l)] - learning_rate *
        ↪  v["dW" + str(l)]   # đi theo vận tốc
        parameters["b" + str(l)] = parameters["b" + str(l)] - learning_rate *
        ↪  v["db" + str(l)]
    return parameters, v
```

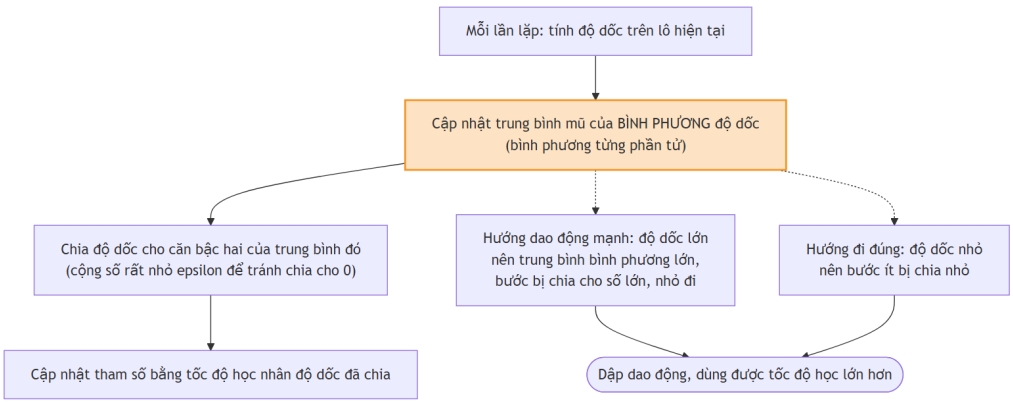
Với beta bằng 0, đây chính là gradient descent thuần. Trong Keras chỉ cần một đối số:

```
optimizer = keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
# Keras cập nhật vận tốc theo kiểu không có hệ số (1 - beta), nên tốc độ học tốt
↪  nhất có thể khác công thức ở trên
```

3.5.4 RMSprop

RMSprop (root mean square propagation) cũng nhằm dập dao động, bằng cách khác: **chia độ dốc cho độ lớn trung bình gần đây của chính nó**. Mỗi lần lặp (Hình 3.34):

1. Tính độ dốc trên lô hiện tại.
2. Cập nhật một trung bình mũ của *bình phương* độ dốc (bình phương từng phần tử): giá trị mới bằng `beta` nhân giá trị cũ cộng (1 trừ `beta`) nhân bình phương độ dốc.
3. Cập nhật tham số: trừ tốc độ học nhân (độ dốc chia căn bậc hai của trung bình bình phương), có cộng một số rất nhỏ `epsilon` vào mẫu số để tránh chia cho 0; một giá trị mặc định hợp lý là 1e-8.



Hình 3.34: RMSprop: chia độ dốc cho độ lớn trung bình gần đây

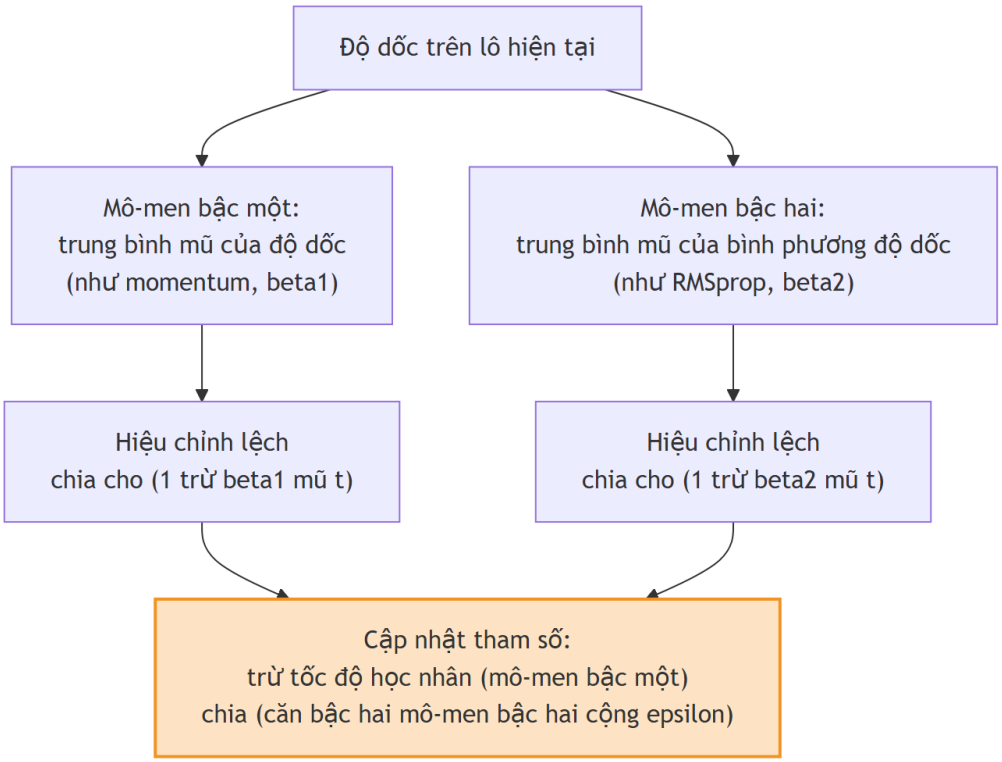
Vì sao có tác dụng: ở hướng đang dao động mạnh, độ dốc lớn nên trung bình bình phương lớn, và bước bị chia cho một số lớn, nhỏ đi nhiều. Ở hướng đi đúng, độ dốc nhỏ nên bước ít bị chia nhỏ. Kết quả: dao động được dập, và bạn có thể dùng tốc độ học lớn hơn mà không phân kỳ, học nhanh hơn. Trong thực tế không gian tham số có rất nhiều chiều, nên "hướng dọc" và "hướng ngang" thực ra là các nhóm tham số khác nhau, nhưng cách hình dung vẫn đúng.

```
optimizer = keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)      # rho
↪  chính là beta của trung bình bình phương
```

3.5.5 Adam

Trong lịch sử học sâu, nhiều thuật toán tối ưu mới được đề xuất và chạy tốt trên một vài bài toán, nhưng rồi hoá ra không tổng quát hoá được sang nhiều loại mạng. Cộng đồng vì thế khá hoài nghi các thuật toán mới. RMSprop và **Adam** là hai trong số rất ít thuật toán đã trụ vững qua thời gian và chạy tốt trên rất nhiều kiến trúc.

Adam (adaptive moment estimation) về cơ bản **ghép momentum với RMSprop** (Hình 3.35):

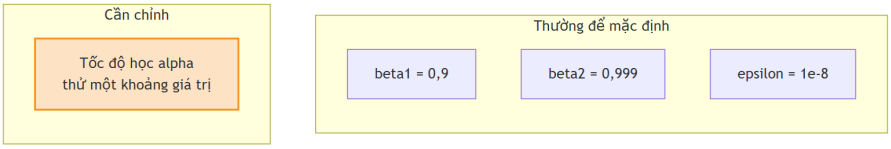


Hình 3.35: Adam kết hợp trung bình mũ của độ dốc (momentum) với trung bình mũ của bình phương độ dốc (RMSprop)

1. Khởi tạo mọi biến trung bình (của độ dốc và của bình phương độ dốc) bằng 0, cùng hình dạng với tham số.
2. Mỗi lần lặp tính độ dốc trên lô, cập nhật trung bình mũ của độ dốc (hệ số `beta1`) và trung bình mũ của bình phương độ dốc (hệ số `beta2`).
3. Hiệu chỉnh độ lệch cả hai bằng cách chia cho (1 trừ hệ số mũ số bước đã qua).
4. Cập nhật tham số: trừ tốc độ học nhân (trung bình độ dốc đã hiệu chỉnh) chia (căn bậc hai của trung bình bình phương đã hiệu chỉnh, cộng `epsilon`).

Tên Adam là viết tắt của ước lượng mô-men thích nghi: trung bình của độ dốc gọi là mô-men bậc một, trung bình của bình phương độ dốc gọi là mô-men bậc hai.

3.5.5.1 Siêu tham số



Hình 3.36: Chỉ cần chỉnh tốc độ học; các hệ số còn lại dùng mặc định

Siêu tham số	Giá trị phổ biến	Ghi chú
Tốc độ học	cần thử một khoảng giá trị	vẫn là siêu tham số quan trọng nhất
`beta1` (momentum)	0,9	ít khi chỉnh
`beta2` (bình phương)	0,999	ít khi chỉnh
`epsilon`	1e-8	hầu như không ảnh hưởng

Bảng 3.17: Siêu tham số của Adam; giá trị mặc định do tác giả đề nghị

```
def initialize_adam(parameters):
    L = len(parameters) // 2
    v, s = {}, {}
    for l in range(1, L + 1):
        v["dW" + str(l)] = np.zeros(parameters["W" + str(l)].shape)       # trung
        ↪  bình của độ dốc
        v["db" + str(l)] = np.zeros(parameters["b" + str(l)].shape)
        s["dW" + str(l)] = np.zeros(parameters["W" + str(l)].shape)       # trung
        ↪  bình của bình phương độ dốc
        s["db" + str(l)] = np.zeros(parameters["b" + str(l)].shape)
    return v, s

def update_parameters_with_adam(parameters, grads, v, s, t, learning_rate=0.01,
↪  beta1=0.9, beta2=0.999, epsilon=1e-8):
    L = len(parameters) // 2
    v_corrected, s_corrected = {}, {}
    for l in range(1, L + 1):
        v["dW" + str(l)] = beta1 * v["dW" + str(l)] + (1 - beta1) * grads["dW" +
        ↪  str(l)]            # mô-men bậc một
        v["db" + str(l)] = beta1 * v["db" + str(l)] + (1 - beta1) * grads["db" +
        ↪  str(l)]
        v_corrected["dW" + str(l)] = v["dW" + str(l)] / (1 - beta1 ** t)
        ↪  # hiệu chỉnh lệch
        v_corrected["db" + str(l)] = v["db" + str(l)] / (1 - beta1 ** t)

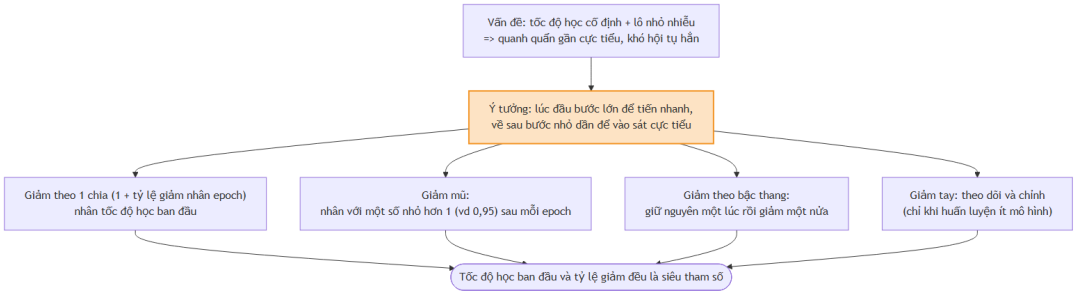
        s["dW" + str(l)] = beta2 * s["dW" + str(l)] + (1 - beta2) *
        ↪  np.square(grads["dW" + str(l)])  # mô-men bậc hai
        s["db" + str(l)] = beta2 * s["db" + str(l)] + (1 - beta2) *
        ↪  np.square(grads["db" + str(l)])
        s_corrected["dW" + str(l)] = s["dW" + str(l)] / (1 - beta2 ** t)
        s_corrected["db" + str(l)] = s["db" + str(l)] / (1 - beta2 ** t)
```

```
        parameters["W" + str(l)] = parameters["W" + str(l)] - learning_rate *
        ↪  v_corrected["dW" + str(l)] / (np.sqrt(s_corrected["dW" + str(l)]) +
        ↪  epsilon)
        parameters["b" + str(l)] = parameters["b" + str(l)] - learning_rate *
        ↪  v_corrected["db" + str(l)] / (np.sqrt(s_corrected["db" + str(l)]) +
        ↪  epsilon)
    return parameters, v, s, v_corrected, s_corrected

optimizer = keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999,
↪  epsilon=1e-8)    # trong Keras
```

3.5.6 Giảm tốc độ học

Giả sử bạn dùng mini-batch nhỏ (64 hoặc 128 mẫu). Các bước hơi nhiễu; nếu tốc độ học cố định, thuật toán tiến về vùng cực tiểu nhưng quanh quẩn quanh đó mà không hội tụ hẳn. Nếu **giảm dần tốc độ học theo thời gian**, lúc đầu tốc độ học lớn cho phép học nhanh, về sau tốc độ nhỏ dần nên các bước nhỏ lại và dao động trong vùng chặt hơn quanh cực tiểu (Hình 3.37).



Hình 3.37: Các cách giảm tốc độ học theo epoch

3.5.6.1 Các công thức thường dùng

- **Giảm theo 1 chia (1 cộng tỷ lệ giảm nhân số epoch)**, nhân với tốc độ học ban đầu. Tỷ lệ giảm là một siêu tham số mới. Ví dụ tốc độ ban đầu 0,2 và tỷ lệ giảm 1: epoch 1 còn 0,1; epoch 2 còn khoảng 0,067; epoch 3 còn 0,05; epoch 4 còn 0,04.
- **Giảm mũ**: nhân với một số nhỏ hơn 1 (ví dụ 0,95) mũ số epoch; giảm rất nhanh.
- **Giảm theo căn bậc hai của số epoch hoặc số lô**, nhân một hằng số.
- **Giảm theo bậc thang**: giữ nguyên một lúc rồi giảm một nửa, rồi lặp lại.
- **Giảm tay**: theo dõi mô hình đang huấn luyện (vài ngày hoặc vài tuần) rồi chỉnh tốc độ bằng tay. Chỉ thực tế khi huấn luyện rất ít mô hình.

3.5.6.2 Lập lịch theo khoảng cố định

Giảm sau *mỗi* epoch có thể làm tốc độ học về gần 0 quá nhanh. Cách khắc phục là **giảm theo khoảng cố định**: tốc độ học chỉ đổi khi số epoch là bội của một khoảng (ví dụ cứ 1000 epoch giảm một lần), bằng cách chia số epoch cho khoảng và làm tròn xuống.

```
def update_lr(learning_rate0, epoch_num, decay_rate):                  # giảm sau
↪  mỗi epoch
    return learning_rate0 / (1 + decay_rate * epoch_num)
```

```
def schedule_lr_decay(learning_rate0, epoch_num, decay_rate, time_interval=1000):
↪   # giảm theo khoảng cố định
    return learning_rate0 / (1 + decay_rate * np.floor(epoch_num /
    ↪  time_interval))

print(update_lr(0.5, 2, 1))                         # 0.16666666666666666
print(schedule_lr_decay(0.5, 10, 0.3, 100))         # 0.5: chưa đến mốc 100 nên chưa
↪   giảm
print(schedule_lr_decay(0.5, 100, 0.3, 100))        # 0.3846153846153846
```

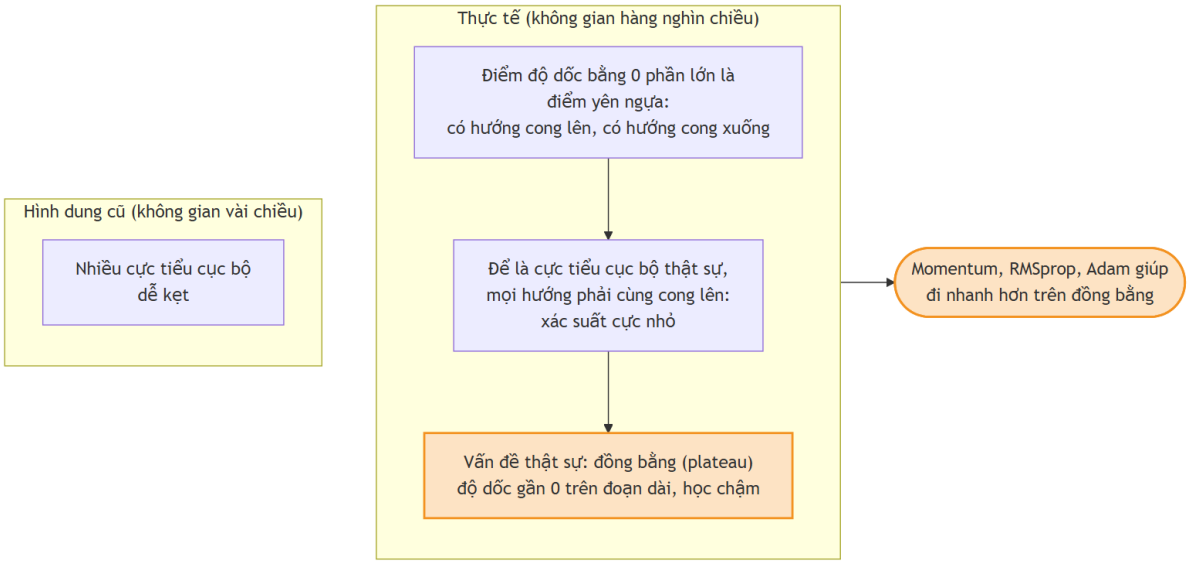
Trong Keras, các lịch này có sẵn dưới dạng đối tượng gắn vào bộ tối ưu. Lưu ý Keras đếm theo *bước* (mỗi lô là một bước), nên đặt khoảng giảm bằng số bước của một epoch nếu muốn giảm theo epoch:

```
steps_per_epoch = len(X_train) // 64
lr_schedule = keras.optimizers.schedules.InverseTimeDecay(
    initial_learning_rate=0.1, decay_steps=steps_per_epoch, decay_rate=1.0,
    ↪  staircase=False)
optimizer = keras.optimizers.Adam(learning_rate=lr_schedule)
# tốc độ học = ban đầu / (1 + decay_rate * bước / decay_steps); staircase=True
↪   thì làm tròn xuống như lịch theo khoảng
```

Giảm tốc độ học thường nằm khá thấp trong danh sách việc nên thử: chỉnh tốt một tốc độ học cố định có tác động lớn hơn nhiều. Nhưng khi cần, giảm tốc độ học đôi lúc tăng tốc huấn luyện rõ rệt.

3.5.7 Cực trị cục bộ và đồng bằng

Thời kỳ đầu của học sâu, người ta lo thuật toán kẹt ở các cực tiểu cục bộ xấu. Hình dung cũ đến từ các hình vẽ hai chiều, trong đó bề mặt chi phí có rất nhiều "thung lũng". Nhưng cách hình dung từ không gian vài chiều *không đúng* cho không gian hàng nghìn hay hàng chục nghìn chiều (Hình 3.38).



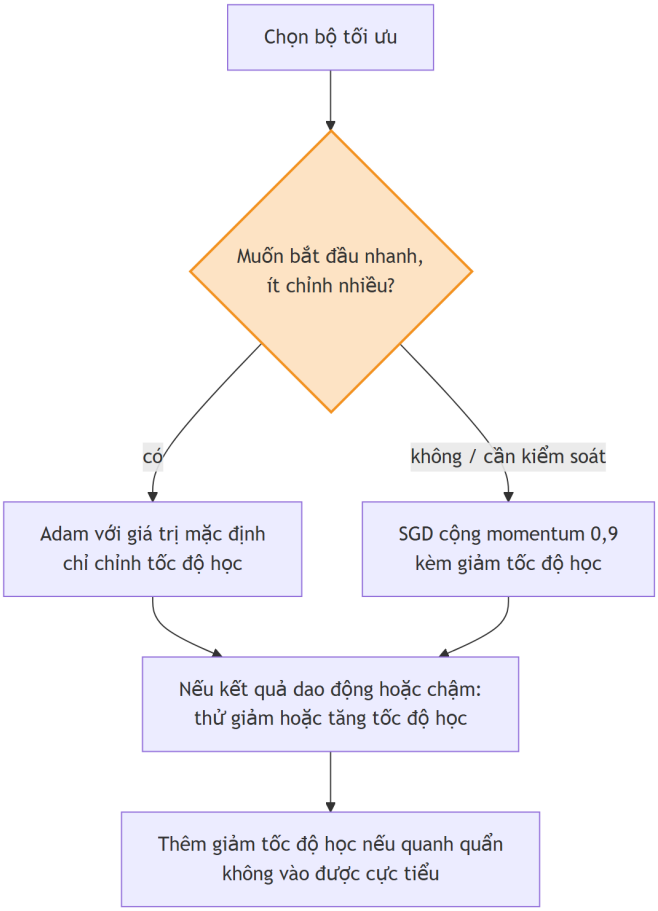
Hình 3.38: Trong không gian nhiều chiều, điểm độ dốc bằng 0 phần lớn là điểm yên ngựa chứ không phải cực tiểu cục bộ

- Trong mạng nơ-ron thật, hầu hết các điểm có độ dốc bằng 0 là **điểm yên ngựa**: theo một số hướng, bề mặt cong lên; theo hướng khác, cong xuống. Để một điểm là cực tiểu cục bộ thật sự, mọi hướng (ví dụ cả 20 000 hướng) phải cùng cong lên, và xác suất việc đó xảy ra là cực nhỏ.

- Vì vậy, nếu mạng đủ lớn (nhiều tham số) và hàm chi phí nằm trong không gian nhiều chiều, **bạn khó kẹt ở cực tiểu cục bộ xấu**.
- Vấn đề thật sự là **đồng bằng** (plateau): vùng độ dốc gần 0 trên một đoạn dài, nơi gradient descent đi rất chậm trước khi nhờ một nhiễu ngẫu nhiên mà thoát ra. Chính ở đây, momentum, RMSprop và Adam giúp được nhiều nhất.

Đây là một bài học lịch sử: nhiều cách hình dung từ không gian ít chiều không chuyển được sang không gian rất nhiều chiều mà các thuật toán tối ưu đang làm việc, và hiểu biết của chúng ta về các không gian này vẫn đang tiến triển.

3.5.8 Chọn bộ tối ưu



Hình 3.39: Gợi ý chọn bộ tối ưu

Bộ tối ưu	Ý tưởng	Siêu tham số	Ghi chú
Mini-batch gradient descent	đi theo độ dốc của lô	tốc độ học, cỡ lô	đơn giản nhất; dễ dao động
Momentum	trung bình mũ của độ dốc	thêm `beta` (0,9)	thường chạy nhanh hơn gradient descent thuần
RMSprop	chia độ dốc cho độ lớn trung bình gần đây	thêm `beta`, `epsilon`	dập dao động, dùng được tốc độ học lớn hơn
Adam	ghép momentum và RMSprop	thêm `beta1`, `beta2`, `epsilon` (đều dùng mặc định)	dùng bộ nhớ nhiều hơn một chút; thường chạy tốt ít cần chỉnh ngoài tốc độ học

Bảng 3.18: So sánh các bộ tối ưu

3.5.9 Áp dụng vào dự án thực tế

3.5.9.1 Danh sách việc cần làm

1. Dùng mini-batch (cỡ 64 đến 512 làm điểm bắt đầu) khi dữ liệu lớn.
2. Bắt đầu với Adam và các hệ số mặc định; chỉnh tốc độ học đầu tiên.
3. Nếu cần, thử SGD với momentum 0,9 kèm giảm tốc độ học.
4. Nếu huấn luyện quanh quẩn mà không hội tụ, thêm giảm tốc độ học (theo khoảng cố định).
5. Đừng vội chỉnh `beta1`, `beta2`, `epsilon` của Adam; chúng hầu như ít ảnh hưởng.

3.5.9.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Chi phí theo lô dao động nhiều	bình thường ở mini-batch; nhìn xu hướng, hoặc vẽ trung bình qua nhiều lô
Chi phí tăng hoặc phân kỳ	tốc độ học quá lớn; giảm bớt
Dùng cỡ lô bằng 1 mà chạy rất chậm	mất vector hoá; tăng cỡ lô lên 64 hoặc hơn
Hiệu năng sụt mạnh khi tăng cỡ lô	lô không còn vừa bộ nhớ; giảm cỡ lô
Tốc độ học giảm về gần 0 sớm, mô hình ngừng học	giảm sau mỗi epoch quá nhanh; giảm theo khoảng cố định
Chi phí không giảm khi dùng tốc độ học giảm quá mạnh	tốc độ học ban đầu hoặc tỷ lệ giảm đặt quá cực đoan; thử lại giá trị khác
Tự cài trung bình mũ cho ra giá trị đầu quá thấp	thiếu hiệu chỉnh độ lệch; chia cho (1 trừ beta mũ số bước đã qua)

Bảng 3.19: Lỗi thường gặp với bộ tối ưu

3.5.9.3 Bảng tra cứu API

Việc cần làm	Keras
Đặt cỡ lô	`model.fit(..., batch_size=64)`
Momentum	`keras.optimizers.SGD(learning_rate=..., momentum=0.9)`
RMSprop	`keras.optimizers.RMSprop(learning_rate=..., rho=0.9)`
Adam	`keras.optimizers.Adam(learning_rate=..., beta_1=0.9, beta_2=0.999)`
Giảm tốc độ học	`keras.optimizers.schedules.InverseTimeDecay`, `ExponentialDecay`, `PiecewiseConstantDecay`

Bảng 3.20: Các lệnh Keras cho tối ưu hoá

3.5.9.4 Tóm tắt cần nhớ

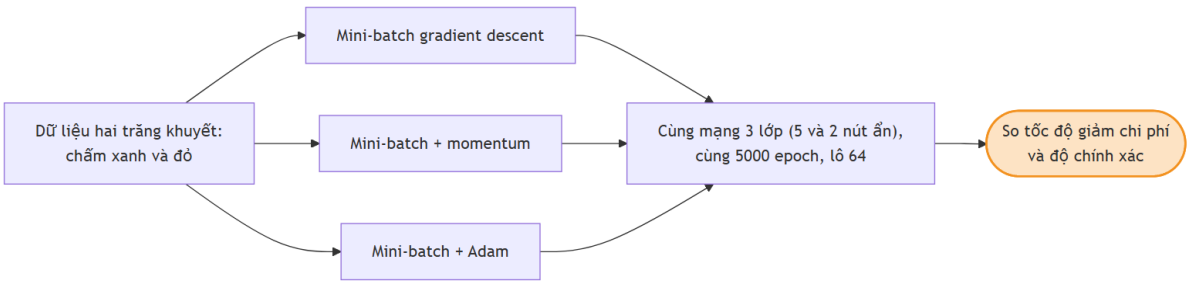
- Mini-batch cho phép tiến bộ trước khi hết cả tập; cỡ lô điển hình 64 đến 512, luỹ thừa của 2.
- Trung bình mũ: trung bình qua khoảng một chia (1 trừ beta) giá trị gần nhất, chỉ cần một biến; cần hiệu chỉnh độ lệch ở giai đoạn đầu.
- Momentum làm mượt bước đi; RMSprop dập dao động bằng cách chia cho độ lớn gần đây; Adam ghép cả hai.
- Adam: `beta1` bằng 0,9, `beta2` bằng 0,999, `epsilon` bằng 1e-8; chỉ cần chỉnh tốc độ học.
- Giảm tốc độ học giúp hội tụ hẳn; giảm theo khoảng cố định tránh về 0 quá nhanh.
- Trong không gian nhiều chiều, cực tiểu cục bộ xấu hiếm; đồng bằng mới là vấn đề.

3.6 Thực hành: so sánh các bộ tối ưu

Mục này đặt các thuật toán ở mục 3.5 cạnh nhau trên cùng một bài toán và cùng một mạng, để thấy chúng khác nhau thế nào. Có hai thí nghiệm: so ba bộ tối ưu (mục 3.6.2), rồi xem giảm tốc độ học ảnh hưởng ra sao (mục 3.6.3). Các con số là kết quả của chính bài thực hành gốc; mục không tự đo thêm số liệu nào.

3.6.1 Thiết lập

Dữ liệu là bộ hai trăng khuyết: chấm xanh và đỏ mỗi lớp có hình lưỡi liềm. Mạng có ba lớp (hai lớp ẩn 5 và 2 nút), tốc độ học ban đầu 0,0007, cỡ lô 64, 5000 epoch, và các hệ số của momentum và Adam dùng giá trị mặc định (`beta` 0,9; `beta1` 0,9; `beta2` 0,999; `epsilon` 1e-8). Chỉ bộ tối ưu thay đổi (Hình 3.40).



Hình 3.40: Ba bộ tối ưu trên cùng mạng và cùng dữ liệu

Hàm `model` là vòng huấn luyện mini-batch đã trình bày ở mục 3.5.1, với một điểm chung: chọn bộ tối ưu bằng đối số. Mỗi epoch tăng `seed` lên một để xáo trộn khác đi:

```
def model(X, Y, layers_dims, optimizer, learning_rate=0.0007, mini_batch_size=64,
↪  beta=0.9,
          beta1=0.9, beta2=0.999, epsilon=1e-8, num_epochs=5000,
           ↪  print_cost=True):
    t = 0                                                    # bộ đếm bước cho
    ↪  hiệu chỉnh độ lệch của Adam
    seed = 10
    m = X.shape[1]
    parameters = initialize_parameters(layers_dims)

    # khởi tạo biến riêng của từng bộ tối ưu
    if optimizer == "gd":
        pass                                                 # gradient descent
         ↪  thuần: không cần gì thêm
    elif optimizer == "momentum":
        v = initialize_velocity(parameters)
    elif optimizer == "adam":
        v, s = initialize_adam(parameters)

    for i in range(num_epochs):
        seed = seed + 1                                      # mỗi epoch xáo trộn
         ↪  khác
        minibatches = random_mini_batches(X, Y, mini_batch_size, seed)
        cost_total = 0
        for minibatch in minibatches:
            (minibatch_X, minibatch_Y) = minibatch
            a3, caches = forward_propagation(minibatch_X, parameters)       #
             ↪  xuôi
            cost_total += compute_cost(a3, minibatch_Y)                     # chi
             ↪  phí của lô
            grads = backward_propagation(minibatch_X, minibatch_Y, caches)  #
             ↪  ngược

            if optimizer == "gd":
                parameters = update_parameters_with_gd(parameters, grads,
                 ↪  learning_rate)
            elif optimizer == "momentum":
                parameters, v = update_parameters_with_momentum(parameters,
                 ↪  grads, v, beta, learning_rate)
            elif optimizer == "adam":
                t = t + 1
                parameters, v, s, _, _ = update_parameters_with_adam(parameters,
                 ↪  grads, v, s, t, learning_rate, beta1, beta2, epsilon)
        cost_avg = cost_total / m
    return parameters
```

Hàm cập nhật của gradient descent thuần chỉ là một phép trừ cho từng nhóm tham số:

```
def update_parameters_with_gd(parameters, grads, learning_rate):
    L = len(parameters) // 2
    for l in range(1, L + 1):
        parameters["W" + str(l)] = parameters["W" + str(l)] - learning_rate *
         ↪  grads["dW" + str(l)]
        parameters["b" + str(l)] = parameters["b" + str(l)] - learning_rate *
         ↪  grads["db" + str(l)]
    return parameters
```

3.6.2 Thí nghiệm 1: gradient descent, momentum và Adam

```
layers_dims = [train_X.shape[0], 5, 2, 1]
parameters = model(train_X, train_Y, layers_dims, optimizer="gd")
↪  # mini-batch gradient descent
parameters = model(train_X, train_Y, layers_dims, beta=0.9, optimizer="momentum")
↪  # thêm momentum
parameters = model(train_X, train_Y, layers_dims, optimizer="adam")
↪  # Adam
predictions = predict(train_X, train_Y, parameters)
↪  # in độ chính xác
```

Bộ tối ưu	Chi phí epoch 0	Epoch 1000	Epoch 4000	Chính xác	Hình dạng đường chi phí
Gradient descent (mini-batch)	0,7024	0,6681	0,5734	71,7%	mượt, giảm chậm
Momentum	0,7024	0,6682	0,5734	71,7%	mượt, gần như trùng gradient descent
Adam	0,7022	0,1678	0,1361	94,3%	mượt hơn, giảm rất nhanh

Bảng 3.21: Ba bộ tối ưu trên bộ hai trăng khuyết (tốc độ học 0,0007, cỡ lô 64, 5000 epoch)

Cách đọc:

- **Momentum gần như không khác gradient descent** ở đây. Bài toán đơn giản và tốc độ học nhỏ nên tác động của momentum rất ít; trên bài toán phức tạp hơn mới thấy mức tăng lớn hơn.
- **Adam vượt xa**: chi phí giảm từ 0,70 xuống 0,17 chỉ trong 1000 epoch, trong khi hai bộ kia mới giảm xuống khoảng 0,67. Độ chính xác 94,3% so với 71,7%.
- Nếu chạy thêm nhiều epoch, cả ba cuối cùng đều cho kết quả rất tốt trên bài toán đơn giản này; điểm khác biệt là Adam hội tụ *nhanh hơn nhiều*.

Ưu điểm của Adam: yêu cầu bộ nhớ tương đối thấp (dù cao hơn gradient descent và momentum), và thường chạy tốt với rất ít chỉnh siêu tham số (ngoài tốc độ học).

3.6.3 Thí nghiệm 2: giảm tốc độ học

Hàm `model` lần này nhận thêm hai đối số: `decay` (hàm tính tốc độ học mới) và `decay_rate`. Sau mỗi epoch, nó gọi `decay` để cập nhật tốc độ học từ giá trị ban đầu. Hai hàm `update_lr` và `schedule_lr_decay` đã có ở mục 3.5.6.

```
# bên trong vòng epoch của hàm model:
if decay:
    learning_rate = decay(learning_rate0, i, decay_rate)       # cập nhật tốc độ
    ↪  học theo số epoch i

# gọi:
parameters = model(train_X, train_Y, layers_dims, optimizer="gd",
↪  learning_rate=0.1, num_epochs=5000, decay=update_lr)
```

3.6.3.1 Giảm sau mỗi epoch: về 0 quá nhanh

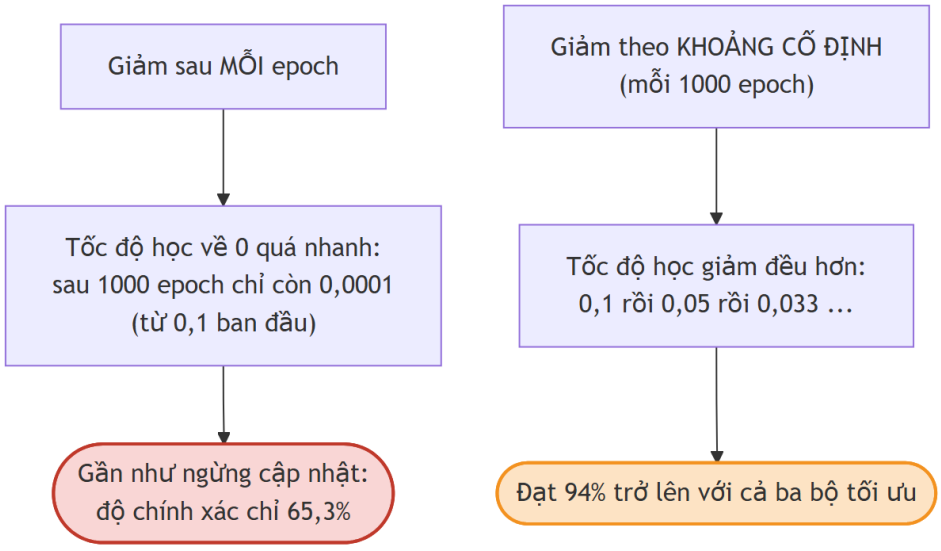
Với gradient descent, tốc độ học ban đầu 0,1 và giảm sau *mỗi* epoch, tốc độ học rơi rất nhanh:

Epoch	Tốc độ học	Chi phí
0	0,100000	0,701091
1000	0,000100	0,661884
2000	0,000050	0,658620
3000	0,000033	0,656765
4000	0,000025	0,655486

Bảng 3.22: Giảm tốc độ học sau mỗi epoch: tốc độ học nhanh chóng tiến về 0 và chi phí gần như đứng yên

Độ chính xác chỉ đạt 65,3%. Khi số epoch lớn, thuật toán gần như ngừng cập nhật. Cách khắc phục phổ biến: **giảm theo khoảng cố định** (ở đây cứ 1000 epoch giảm một lần).

3.6.3.2 Giảm theo khoảng cố định



Hình 3.41: Giảm sau mỗi epoch so với giảm theo khoảng cố định

Bộ tối ưu	Tốc độ học ban đầu	Chi phí epoch 4000	Chính xác	Nhận xét
Gradient descent	0,1	0,1175	94,3%	giờ ngang Adam
Momentum	0,1	0,1232	95,3%	tốt hơn một chút
Adam	0,01	0,1284	94%	ngang, nhưng đạt nhanh hơn

Bảng 3.23: Giảm tốc độ học theo khoảng cố định (1000 epoch) cho từng bộ tối ưu, 5000 epoch

Khi thêm giảm tốc độ học theo khoảng cố định, gradient descent và momentum, vốn kém Adam rất nhiều ở thí nghiệm 1, đạt độ chính xác và tốc độ tương đương Adam. Với Adam, đường học cho độ chính xác tương đương nhưng **nhanh hơn**. Bài học: không có một bộ tối ưu "thắng mọi nơi"; giảm tốc độ học và chọn tốc độ học đúng quan trọng không kém chọn thuật toán.

Ghi chú. Các thí nghiệm dùng tốc độ học ban đầu khác nhau (0,0007 cho thí nghiệm 1, 0,1 và 0,01 cho thí nghiệm 2), nên đừng so trực tiếp mọi con số giữa hai bảng; so trong từng bảng.

3.6.4 Áp dụng vào dự án thực tế

3.6.4.1 Danh sách việc cần làm

1. Bắt đầu với Adam; nếu cần thì mới thử SGD với momentum.
2. Nếu dùng SGD hoặc momentum, hãy bật giảm tốc độ học theo khoảng cố định.
3. Vẽ cả đường chi phí lẫn độ chính xác; kết quả khác nhau rõ khi đổi bộ tối ưu hoặc lịch tốc độ học.
4. Luôn so sánh các bộ tối ưu với cùng mạng, cùng dữ liệu, cùng số epoch.

3.6.4.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Mô hình chạy mãi mà chi phí gần như không đổi	tốc độ học giảm về 0 quá sớm (giảm sau mỗi epoch); giảm theo khoảng cố định
Momentum không khác gì gradient descent	bài toán quá đơn giản hoặc tốc độ học quá nhỏ; thử tốc độ học lớn hơn
Adam không học	tốc độ học quá lớn hoặc quá nhỏ; thử một khoảng giá trị theo thang log
Quên khởi tạo biến của bộ tối ưu	momentum cần `initialize_velocity`, Adam cần `initialize_adam`
Quên tăng bộ đếm bước của Adam	hiệu chỉnh độ lệch dùng số bước; tăng bộ đếm mỗi lần cập nhật

Bảng 3.24: Lỗi thường gặp khi so sánh bộ tối ưu

3.6.4.3 Tóm tắt cần nhớ

- Trên bộ hai trăng khuyết, Adam đạt 94,3% trong khi gradient descent và momentum chỉ 71,7% với cùng số epoch.
- Momentum chỉ cải thiện rõ khi bài toán đủ phức tạp hoặc tốc độ học đủ lớn.

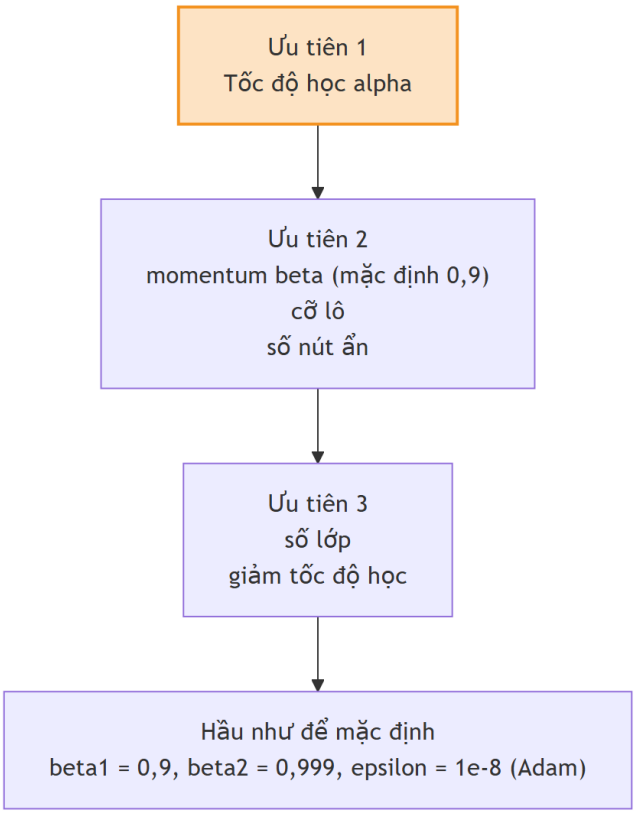
- Giảm tốc độ học sau mỗi epoch làm tốc độ về 0 quá nhanh và độ chính xác còn 65,3%.
- Giảm tốc độ học theo khoảng cố định đưa gradient descent (94,3%) và momentum (95,3%) lên ngang Adam (94%).

3.7 Tinh chỉnh siêu tham số

Một điều khó chịu của việc huấn luyện mạng nơ-ron là số lượng siêu tham số phải xử lý: tốc độ học, hệ số momentum, các hệ số của Adam, số lớp, số nút ở mỗi lớp, giảm tốc độ học, cỡ lô, và hơn thế nữa. Mục này đưa ra **cách tổ chức việc tìm kiếm** để hội tụ về cấu hình tốt hiệu quả hơn, gồm: siêu tham số nào quan trọng hơn, cách chọn các giá trị để thử, trên thang đo nào, và cách vận hành quá trình tìm kiếm.

3.7.1 Siêu tham số nào quan trọng

Không phải siêu tham số nào cũng quan trọng như nhau. Một thứ tự ưu tiên thường gặp (không phải quy tắc cứng, và các chuyên gia khác nhau có thể có thứ tự khác) được tóm tắt ở Hình 3.42:



Hình 3.42: Thứ tự ưu tiên khi chỉnh siêu tham số

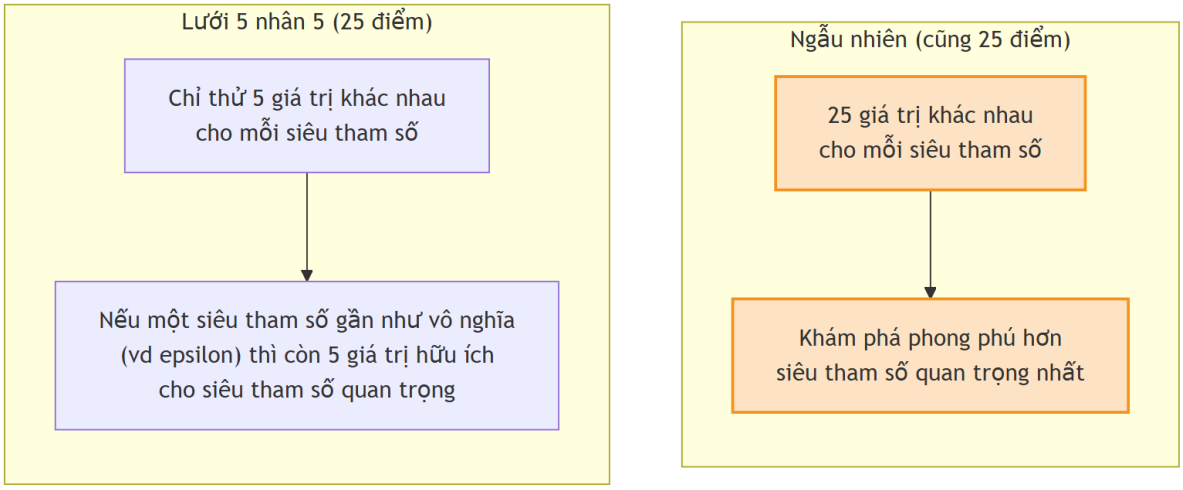
Mức	Siêu tham số	Ghi chú
1 (quan trọng nhất)	tốc độ học	gần như luôn là siêu tham số đầu tiên cần chỉnh
2	hệ số momentum, cỡ lô, số nút ẩn	momentum 0,9 là mặc định tốt; cỡ lô để bộ tối ưu chạy hiệu quả
3	số lớp, giảm tốc độ học	đôi khi tạo khác biệt lớn
Ít khi chỉnh	`beta1`, `beta2`, `epsilon` của Adam	thường dùng 0,9; 0,999; 1e-8

Bảng 3.25: Mức ưu tiên của các siêu tham số

3.7.2 Chọn giá trị để thử

3.7.2.1 Ngẫu nhiên thay vì lưới

Ở các thế hệ thuật toán trước, nếu có hai siêu tham số, người ta hay thử theo một lưới đều, ví dụ 5 nhân 5 bằng 25 điểm. Với học sâu, khuyến nghị là **chọn các điểm ngẫu nhiên** (cũng 25 điểm, nhưng rải ngẫu nhiên). Lý do: ta không biết trước siêu tham số nào quan trọng với bài toán, và như đã thấy, có tham số quan trọng hơn hẳn tham số khác (Hình 3.43).

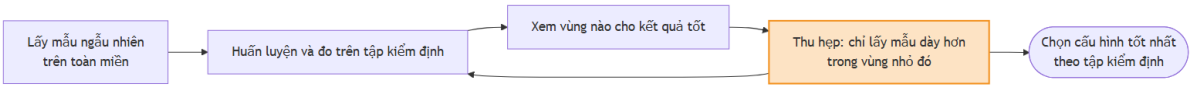


Hình 3.43: Với cùng 25 lần huấn luyện, chọn ngẫu nhiên thử được nhiều giá trị khác nhau hơn cho tham số quan trọng

Ví dụ siêu tham số thứ nhất là tốc độ học, rất quan trọng, còn siêu tham số thứ hai là `epsilon` của Adam, hầu như không quan trọng. Lưới 5 nhân 5 chỉ thử được năm giá trị của tốc độ học (mỗi giá trị lặp lại năm lần với `epsilon` khác nhau, cho kết quả gần như nhau), dù đã huấn luyện 25 mô hình. Chọn ngẫu nhiên thì 25 mô hình có 25 tốc độ học khác nhau, nên dễ gặp giá trị tốt hơn nhiều. Nếu có ba siêu tham số, thay vì lưới là một khối lập phương; càng nhiều siêu tham số, ngẫu nhiên càng có lợi vì khám phá phong phú hơn ở những chiều quan trọng nhất, dù ta không biết trước đó là chiều nào.

3.7.2.2 Từ thô đến mịn

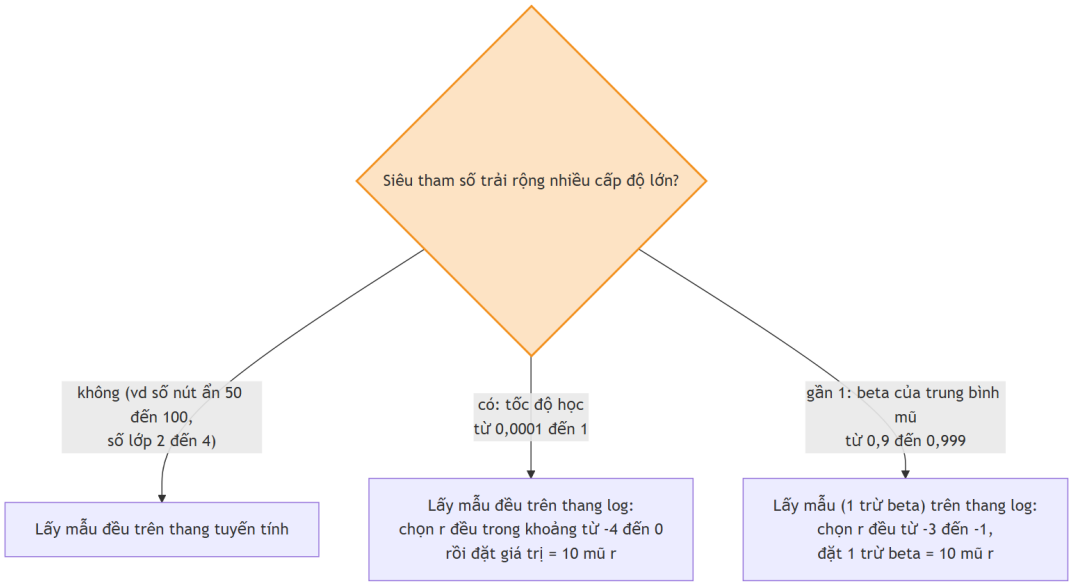
Cách thứ hai là **tìm kiếm từ thô đến mịn** (coarse to fine, Hình 3.44). Lấy mẫu ngẫu nhiên thô trên toàn miền; nếu một vài điểm trong một vùng cho kết quả tốt, thu hẹp về vùng đó và lấy mẫu dày hơn. Có thể lặp lại nhiều vòng. Sau đó chọn cấu hình cho kết quả tốt nhất theo tiêu chí bạn quan tâm (thường là tập kiểm định).



Hình 3.44: Tìm kiếm từ thô đến mịn

3.7.3 Chọn thang đo thích hợp

"Ngẫu nhiên" không có nghĩa là ngẫu nhiên *đều* trên khoảng giá trị. Cần chọn thang đo phù hợp (Hình 3.45).



Hình 3.45: Siêu tham số nào dùng thang tuyến tính, siêu tham số nào dùng thang log

3.7.3.1 Thang tuyến tính khi phạm vi hẹp

Số nút ẩn của một lớp có thể thử trong khoảng 50 đến 100; chọn đều ngẫu nhiên là hợp lý. Số lớp có thể thử 2, 3, 4: chọn đều hoặc thử cả ba đều ổn.

3.7.3.2 Thang log cho tốc độ học

Giả sử tốc độ học có thể từ 0,0001 đến 1. Nếu chọn đều trên đoạn thẳng từ 0,0001 đến 1, khoảng 90% giá trị rơi vào khoảng 0,1 đến 1: tức 90% công sức dành cho một phần mười khoảng giá trị (theo cấp độ lớn), 10% còn lại cho phần còn lại. Điều đó không hợp lý. Thay vào đó, chọn đều trên **thang logarit**: mỗi khoảng 0,0001 đến 0,001, rồi 0,001 đến 0,01, rồi 0,01 đến 0,1, rồi 0,1 đến 1 nhận một lượng thử như nhau.

```
import numpy as np

r = -4 * np.random.rand()           # r nằm đều trong khoảng từ -4 đến 0
learning_rate = 10 ** r             # 10 mũ r nằm trong khoảng 0,0001 đến 1, đều
↪  theo thang log

# tổng quát: lấy mẫu từ 10 mũ a đến 10 mũ b
# a = log10(giá trị nhỏ), b = log10(giá trị lớn); r đều trong [a, b]; giá trị =
↪  10 ** r
```

3.7.3.3 Hệ số của trung bình mũ

Với hệ số `beta` của momentum hoặc Adam, giả sử muốn tìm trong khoảng 0,9 đến 0,999. Nhớ rằng 0,9 tương đương trung bình qua khoảng 10 giá trị, còn 0,999 tương đương khoảng 1000 giá trị. Chọn đều trong khoảng đó là không hợp lý. Nên thay bằng **tìm (1 trừ `beta`)** trong khoảng từ 0,1 đến 0,001, trên thang log:

```
r = np.random.uniform(-3, -1)      # r đều trong khoảng từ -3 đến -1
beta = 1 - 10 ** r                 # beta nằm trong khoảng 0,9 đến 0,999
```

Cách này dành lượng thử bằng nhau cho khoảng 0,9 đến 0,99 và khoảng 0,99 đến 0,999. Có lý do toán học: khi `beta` gần 1, kết quả rất nhạy với thay đổi nhỏ của nó. Đổi `beta` từ 0,9 lên 0,9005 gần như không thay đổi gì; nhưng đổi từ 0,999 lên 0,9995 thì cửa sổ trung bình nhảy từ khoảng 1000 lên khoảng 2000 giá trị, ảnh hưởng lớn.

Nếu bạn lỡ chọn thang chưa đúng, đừng lo quá: tìm kiếm từ thô đến mịn ở các vòng sau sẽ tập trung vào vùng giá trị tốt.

3.7.3.4 Công cụ có sẵn

Thay vì tự viết vòng lặp, có thể dùng thư viện như Keras Tuner. Nó hỗ trợ lấy mẫu theo thang log:

```
import keras_tuner as kt          # pip install keras-tuner

def build(hp):
    model = keras.Sequential([
        keras.Input(shape=(12288,)),
        layers.Dense(hp.Int("so_nut", 50, 100), activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ])
    lr = hp.Float("lr", 1e-4, 1.0, sampling="log")         # tốc độ học lấy mẫu
    ↪  theo thang log
    model.compile(optimizer=keras.optimizers.Adam(lr),
    ↪  loss="binary_crossentropy", metrics=["accuracy"])
    return model

tuner = kt.RandomSearch(build, objective="val_accuracy", max_trials=25)    # tìm
↪  ngẫu nhiên, 25 lần thử
tuner.search(X_train, y_train, epochs=20, validation_data=(X_dev, y_dev))
```

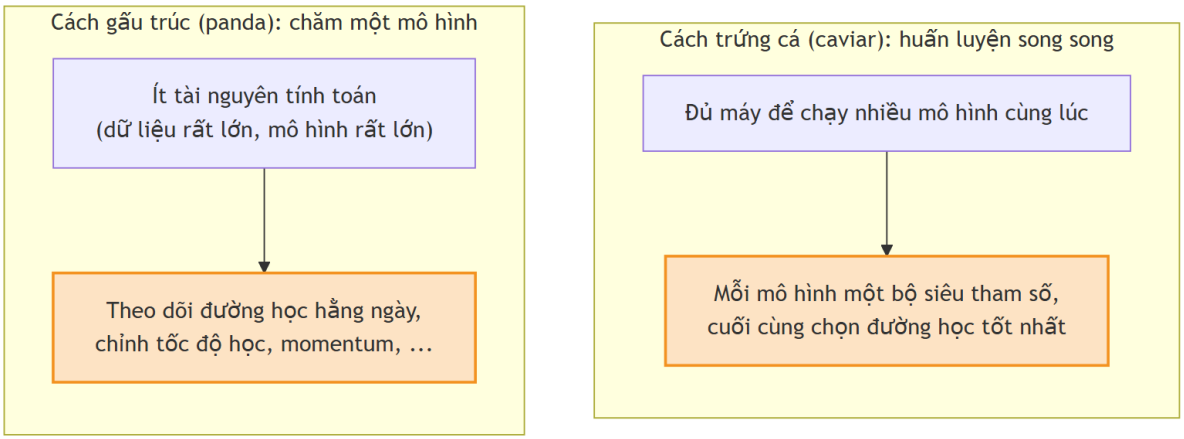
3.7.4 Cách vận hành việc tìm kiếm

3.7.4.1 Siêu tham số bị lỗi thời

Học sâu được dùng ở nhiều lĩnh vực, và ý tưởng được vay mượn qua lại (ví dụ các kiến trúc thị giác được dùng cho giọng nói, ý tưởng giọng nói được dùng cho ngôn ngữ). Nhưng **giá trị siêu tham số thì dễ bị lỗi thời**. Ngay cả khi bạn làm việc trên đúng một bài toán, dữ liệu thay đổi dần trong vài tháng, hoặc máy chủ được nâng cấp, thì cấu hình tốt nhất có thể đổi. Nên kiểm tra lại siêu tham số ít nhất vài tháng một lần.

3.7.4.2 Hai cách làm: gấu trúc và trứng cá

Có hai hướng tiếp cận thường thấy khi tìm siêu tham số (Hình 3.46):



Hình 3.46: Cách gấu trúc (chăm một mô hình) so với cách trứng cá (nhiều mô hình song song)

- **Cách gấu trúc** (panda): chăm một mô hình. Dùng khi tài nguyên tính toán ít, nên chỉ huấn luyện được một mô hình hoặc rất ít mô hình mỗi lần (thường gặp khi dữ liệu và mô hình rất lớn, như một số bài toán quảng cáo hoặc thị giác). Ngày thứ nhất khởi tạo ngẫu nhiên và xem đường học; sang ngày thứ hai tăng nhẹ tốc độ học; ngày thứ ba chỉnh momentum; nếu tốc độ học quá lớn thì quay lại mô hình hôm trước, và cứ thế, mỗi ngày nhích tham số lên xuống. Gọi như vậy vì gấu trúc đẻ rất ít con, mỗi lần một con, và dồn công sức chăm cho con đó sống sót.
- **Cách trứng cá** (caviar): huấn luyện nhiều mô hình song song. Mỗi mô hình một bộ siêu tham số, chạy riêng một đường học; cuối cùng chọn đường học tốt nhất. Giống loài cá đẻ hàng trăm triệu trứng mỗi mùa và không chăm từng quả, chỉ mong một số sống tốt.

Chọn cách nào tuỳ vào tài nguyên tính toán: đủ máy thì nên làm theo cách trứng cá và thử nhiều cấu hình. Ngay cả cách gấu trúc, sau một thời gian bạn có thể thấy mô hình không tốt và khởi động một mô hình khác để chăm tiếp.

3.7.5 Áp dụng vào dự án thực tế

3.7.5.1 Danh sách việc cần làm

1. Bắt đầu bằng tốc độ học; chỉnh nó trước.
2. Lấy mẫu ngẫu nhiên (không lưới); dùng thang log cho tốc độ học và cho (1 trừ `beta`).
3. Tìm từ thô đến mịn: sau vòng đầu, thu hẹp về vùng tốt.
4. Chọn theo kết quả trên tập kiểm định, không phải tập kiểm tra.
5. Ghi lại mỗi lần thử (cấu hình, đường học, kết quả) để so sánh.
6. Kiểm tra lại siêu tham số định kỳ khi dữ liệu hoặc hạ tầng đổi.

3.7.5.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Tìm tốc độ học mà gần như toàn thử giá trị lớn	lấy mẫu đều trên thang tuyến tính; chuyển sang thang log
Thử `beta` mà gần như không thấy khác biệt	lấy mẫu đều trong khoảng 0,9 đến 0,999; lấy mẫu (1 trừ `beta`) trên thang log
Dành nhiều công chỉnh `epsilon` của Adam mà không thấy gì	siêu tham số ít quan trọng; dành công cho tốc độ học
Cấu hình từng tốt nay kém đi	dữ liệu hoặc hạ tầng đã đổi; tìm kiếm lại
Kết quả chọn bằng tập kiểm tra rồi đo lại cũng bằng tập kiểm tra	dễ quá khớp lên tập kiểm tra; chọn bằng tập kiểm định

Bảng 3.26: Lỗi thường gặp khi tinh chỉnh siêu tham số

3.7.5.3 Bảng tra cứu API

Việc cần làm	Công cụ
Lấy mẫu theo thang log	`10 ** np.random.uniform(a, b)`
Lấy mẫu số nguyên	`np.random.randint`, hoặc `hp.Int` trong Keras Tuner
Tìm kiếm ngẫu nhiên có sẵn	`keras_tuner.RandomSearch`
Siêu tham số thang log trong Keras Tuner	`hp.Float(..., sampling="log")`

Bảng 3.27: Công cụ cho tìm kiếm siêu tham số

3.7.5.4 Tóm tắt cần nhớ

- Tốc độ học là siêu tham số quan trọng nhất; momentum, cỡ lô, số nút ẩn tiếp theo; số lớp và giảm tốc độ học sau nữa; hệ số Adam hầu như để mặc định.
- Lấy mẫu ngẫu nhiên, không dùng lưới; kết hợp tìm từ thô đến mịn.
- Lấy mẫu đúng thang: log cho tốc độ học; (1 trừ `beta`) trên thang log cho hệ số của trung bình mũ.
- Có đủ tài nguyên thì huấn luyện nhiều mô hình song song; ít tài nguyên thì chăm một mô hình.
- Siêu tham số bị lỗi thời: kiểm tra lại định kỳ.

3.8 Batch Normalization

Một trong những ý tưởng quan trọng nhất của thời kỳ học sâu bùng nổ là **Batch Normalization** (chuẩn hoá theo lô, gọi tắt là Batch Norm), do Sergey Ioffe và Christian Szegedy đề xuất. Nó mang lại ba lợi ích thực tế: việc tìm siêu tham số dễ hơn, mạng bền vững hơn với lựa chọn siêu tham số (dải siêu tham số chạy tốt rộng hơn), và việc huấn luyện cả những mạng rất sâu trở nên dễ hơn.

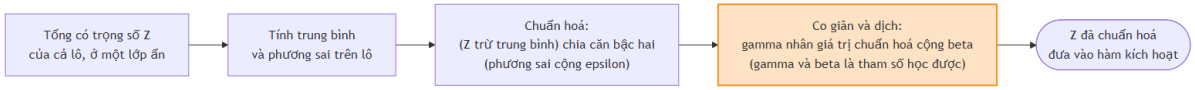
3.8.1 Chuẩn hoá các kích hoạt bên trong mạng

Ở mục 3.3.1 ta đã biết chuẩn hoá đặc trưng đầu vào làm huấn luyện nhanh hơn. Ý tưởng của Batch Norm là đưa đúng kỹ thuật đó vào *bên trong* mạng: với một lớp ẩn bất kỳ, chuẩn hoá các giá trị của nó để việc

huấn luyện các tham số của lớp kế tiếp hiệu quả hơn. Về mặt kỹ thuật, ta chuẩn hoá **tổng có trọng số** của lớp (trước hàm kích hoạt) chứ không phải kích hoạt (sau hàm kích hoạt). Hai lựa chọn đều có người dùng, nhưng chuẩn hoá tổng có trọng số phổ biến hơn nhiều, và là mặc định nên theo.

3.8.1.1 Các bước

Với một lô, tại một lớp ẩn có ma trận tổng có trọng số (mỗi cột một mẫu), Batch Norm làm bốn việc (Hình 3.47):



Hình 3.47: Bốn bước của Batch Norm tại một lớp

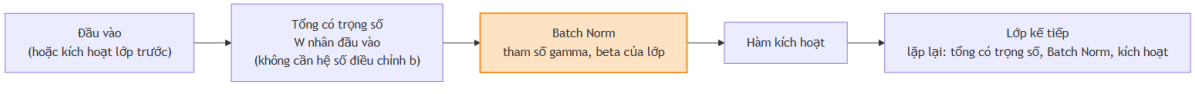
1. Tính **trung bình** của tổng có trọng số trên lô, cho từng nút.
2. Tính **phương sai** trên lô, cho từng nút.
3. **Chuẩn hoá**: trừ trung bình rồi chia cho căn bậc hai của (phương sai cộng `epsilon`). `epsilon` nhỏ để ổn định số khi phương sai bằng 0. Sau bước này, mỗi nút có trung bình 0 và phương sai 1.
4. **Co giãn và dịch**: nhân với `gamma` rồi cộng `beta`. `gamma` và `beta` là *tham số học được*, mỗi nút một cặp.

Vì sao có bước thứ tư? Không phải lúc nào ta cũng muốn các nút có đúng trung bình 0 và phương sai 1. Ví dụ với hàm sigmoid, ta có thể muốn các giá trị trải rộng hơn hoặc lệch khỏi 0 để tận dụng phần cong của hàm thay vì chỉ ở đoạn gần thẳng. `gamma` và `beta` cho phép thuật toán tự chọn trung bình và phương sai nó muốn. Thậm chí nếu `gamma` bằng căn bậc hai của (phương sai cộng `epsilon`) và `beta` bằng trung bình, thì bốn bước trên chính xác trở thành phép đồng nhất, trả lại giá trị gốc. Nên Batch Norm không làm mạng yếu đi; nó chỉ chuẩn hoá trung bình và phương sai về những giá trị do hai tham số điều khiển.

```
def batch_norm_forward(Z, gamma, beta, epsilon=1e-8):
    # Z có hình dạng (số nút, số mẫu trong lô); gamma và beta có hình dạng (số
    ↪  nút, 1)
    mu = np.mean(Z, axis=1, keepdims=True)                # trung bình của từng
    ↪  nút trên lô
    var = np.var(Z, axis=1, keepdims=True)                # phương sai của từng
    ↪  nút trên lô
    Z_norm = (Z - mu) / np.sqrt(var + epsilon)            # chuẩn hoá: trung
    ↪  bình 0, phương sai 1
    return gamma * Z_norm + beta                          # co giãn và dịch bằng
    ↪  tham số học được
```

3.8.2 Gắn Batch Norm vào mạng

Mỗi nút tính hai thứ: tổng có trọng số, rồi hàm kích hoạt. Batch Norm chen vào *giữa* hai bước đó (Hình 3.48): tổng có trọng số của lớp đi qua Batch Norm (với `gamma` và `beta` riêng của lớp) rồi mới vào hàm kích hoạt. Mạng giờ có thêm các tham số `gamma` và `beta` cho mỗi lớp áp dụng Batch Norm.



Hình 3.48: Vị trí của Batch Norm trong mỗi lớp: giữa tổng có trọng số và hàm kích hoạt

Lưu ý: `beta` và `gamma` ở đây *không liên quan* đến hệ số `beta` của momentum, RMSprop và Adam; chỉ là trùng tên. Chúng được cập nhật như mọi tham số khác, bằng gradient descent hoặc momentum, RMSprop, Adam.

3.8.2.1 Batch Norm làm việc theo từng lô

Trong thực tế Batch Norm được áp dụng cùng mini-batch. Với mỗi lô, tại mỗi lớp, trung bình và phương sai được tính *chỉ trên các mẫu trong lô đó*. Lô sau tính lại với các mẫu của lô sau.

3.8.2.2 Hệ số điều chỉnh không còn cần thiết

Khi dùng Batch Norm, tổng có trọng số của lớp chỉ cần phép nhân trọng số với kích hoạt lớp trước. Hệ số điều chỉnh `b` bị bỏ đi, vì bước trừ trung bình của Batch Norm sẽ triệt tiêu mọi hằng số được cộng vào tất cả các mẫu trong lô. Vai trò dịch được thay bởi tham số `beta`. Hình dạng của `gamma` và `beta` ở mỗi lớp là (số nút của lớp, 1).

3.8.2.3 Thuật toán huấn luyện

Với mini-batch, mỗi lần lặp: lan truyền xuôi trên lô và ở mỗi lớp ẩn dùng Batch Norm thay cho tổng có trọng số gốc; lan truyền ngược để tính độ dốc cho trọng số, `beta` và `gamma`; cập nhật. Các bộ tối ưu momentum, RMSprop, Adam đều dùng được.

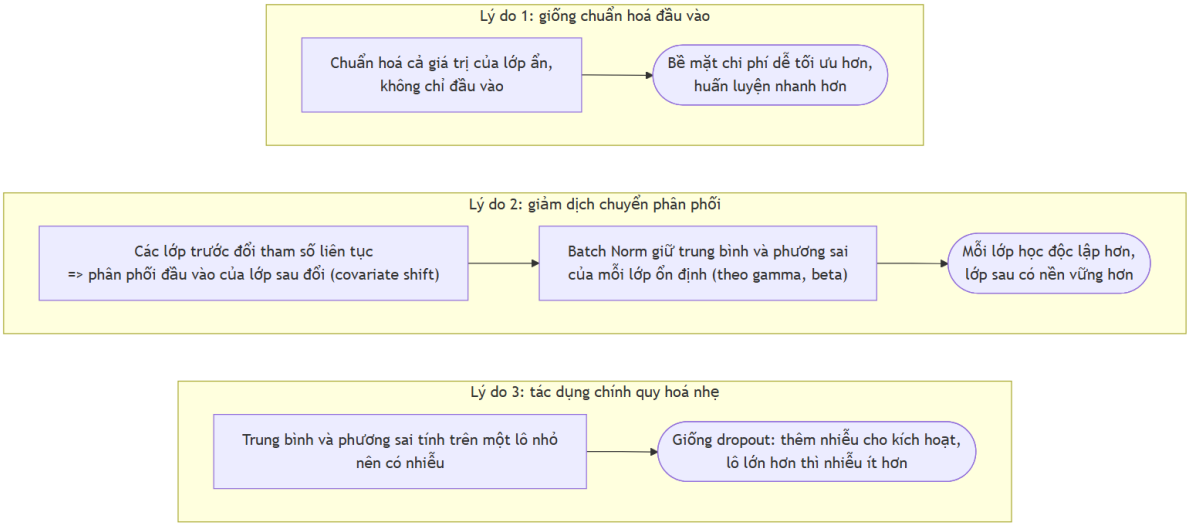
3.8.2.4 Trong Keras

Bạn hiếm khi tự cài đặt Batch Norm; framework có sẵn lớp. Chú ý đặt giữa lớp tuyến tính và hàm kích hoạt, và bỏ hệ số điều chỉnh của lớp `Dense`:

```
model = keras.Sequential([
    keras.Input(shape=(12288,)),
    layers.Dense(64, use_bias=False),           # bỏ hệ số điều chỉnh vì Batch
    ↪  Norm đã có beta
    layers.BatchNormalization(),                # chuẩn hoá theo lô, tham số
    ↪  gamma và beta học được
    layers.Activation("relu"),                  # hàm kích hoạt đặt SAU Batch
    ↪  Norm
    layers.Dense(32, use_bias=False),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.Dense(1, activation="sigmoid"),
])
```

3.8.3 Vì sao Batch Norm hoạt động

Có ba lý do bổ sung cho nhau (Hình 3.49):



Hình 3.49: Ba lý do Batch Norm giúp huấn luyện

3.8.3.1 Lý do 1: giống chuẩn hoá đầu vào

Chuẩn hoá đầu vào đưa các đặc trưng về cùng thang đo giúp học nhanh hơn. Batch Norm làm tương tự cho các giá trị của lớp ẩn, không chỉ lớp vào. Đây mới chỉ là một phần bức tranh.

3.8.3.2 Lý do 2: giảm dịch chuyển phân phối

Giả sử bạn huấn luyện bộ phát hiện mèo chỉ trên ảnh mèo đen. Nếu đem mô hình đó thử trên ảnh mèo đủ màu, nó có thể làm không tốt, dù hàm đúng "ảnh này có phải mèo hay không" vẫn không đổi. Phân phối của đầu vào đã đổi, và mô hình có thể cần huấn luyện lại. Hiện tượng phân phối đầu vào thay đổi này có tên **dịch chuyển hiệp biến** (covariate shift).

Trong mạng sâu, hiện tượng này xảy ra *bên trong*. Nhìn từ lớp ẩn thứ ba, nó nhận kích hoạt từ lớp thứ hai làm đầu vào và cố ánh xạ chúng thành dự đoán. Nhưng trong khi đó, các lớp đầu tiên cũng đang cập nhật tham số, nên các kích hoạt đó cứ thay đổi liên tục. Lớp thứ ba luôn phải thích nghi với đầu vào đang trôi.

Batch Norm giảm mức trôi này. Giá trị cụ thể của tổng có trọng số ở lớp thứ hai vẫn đổi khi các lớp trước cập nhật, nhưng trung bình và phương sai của chúng được giữ cố định (theo `gamma`, `beta` của lớp đó). Lớp sau có nền ổn định hơn để học. Nói cách khác, Batch Norm làm yếu sự ràng buộc giữa việc học của các lớp đầu và các lớp sau, để mỗi lớp học độc lập hơn một chút, và điều đó tăng tốc cả mạng.

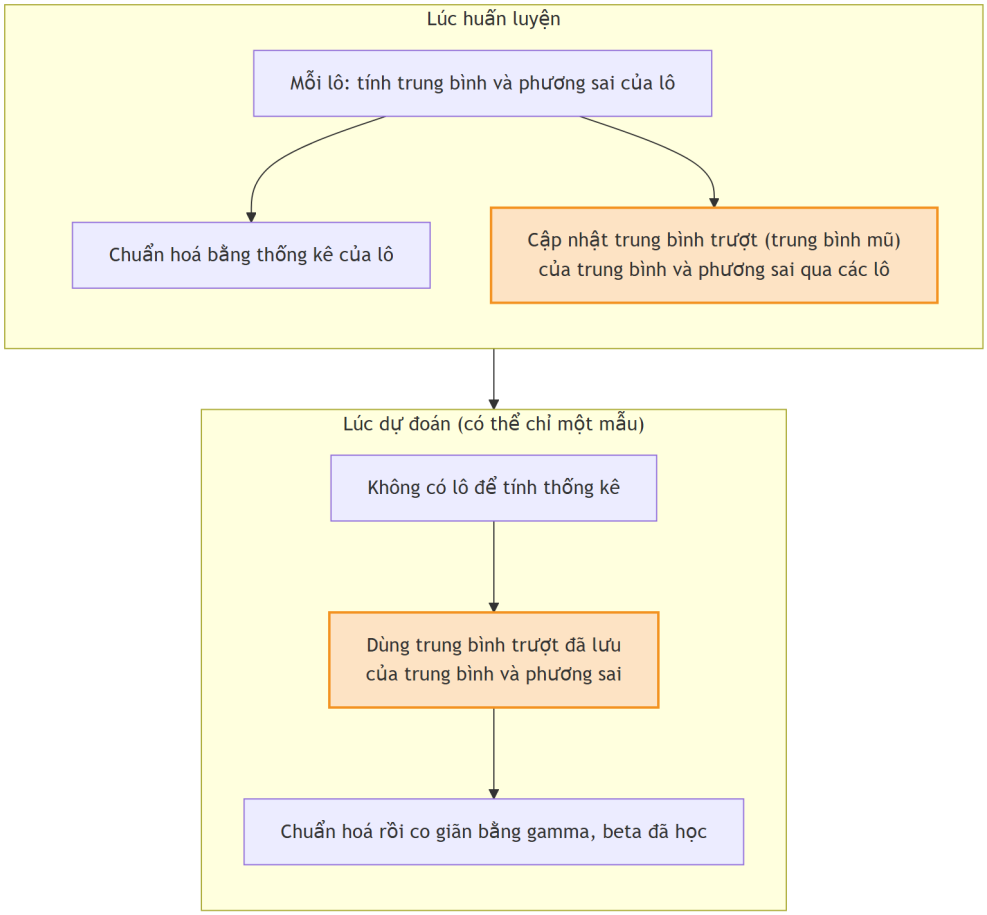
3.8.3.3 Lý do 3: tác dụng chính quy hoá nhẹ

Trung bình và phương sai được tính trên một lô nhỏ (ví dụ 64, 128, 256 mẫu), không phải cả tập, nên chúng có nhiễu. Việc chuẩn hoá bằng giá trị nhiễu thêm nhiễu cho kích hoạt, tương tự dropout (dropout nhân với 0 hoặc 1 ngẫu nhiên; Batch Norm vừa nhân vừa cộng nhiễu qua trung bình và độ lệch chuẩn ước lượng). Như dropout, nó buộc các nút phía sau không dựa quá nhiều vào một nút nào, nên có chính quy hoá nhẹ. Dùng lô lớn hơn (ví dụ 512 thay vì 64) làm giảm nhiễu và vì thế giảm tác dụng này. Có thể dùng Batch Norm cùng dropout nếu muốn chính quy hoá mạnh hơn.

Tuy nhiên, **đừng dùng Batch Norm như một công cụ chính quy hoá**; mục đích của nó là chuẩn hoá để tăng tốc học. Chính quy hoá chỉ là tác dụng phụ ngoài ý muốn.

3.8.4 Batch Norm lúc dự đoán

Lúc huấn luyện, Batch Norm tính trung bình và phương sai trên lô. Nhưng lúc dự đoán bạn có thể chỉ có *một mẫu*, và trung bình hay phương sai của một mẫu không có ý nghĩa. Giải pháp (Hình 3.50):



Hình 3.50: Lúc huấn luyện dùng thống kê của lô; lúc dự đoán dùng trung bình trượt đã lưu

- Trong lúc huấn luyện, với mỗi lớp, giữ một **trung bình trượt** (trung bình mũ, đã học ở mục 3.5.2) của trung bình và của phương sai qua các lô.
- Lúc dự đoán, dùng đúng hai giá trị trượt đó thay cho thống kê của lô để chuẩn hoá, rồi co giãn bằng `gamma` và `beta` đã học.

Cách ước lượng này khá bền vững; bất kỳ cách ước lượng hợp lý nào cũng chạy tốt. Framework thường tự xử lý.

```
# Keras tự chuyển chế độ:
model.fit(X_train, y_train, epochs=20)       # huấn luyện: dùng thống kê của lô
↪  và cập nhật trung bình trượt
model.predict(X_test)                        # dự đoán: dùng trung bình trượt đã
↪  lưu, một mẫu cũng được
```

3.8.5 Áp dụng vào dự án thực tế

3.8.5.1 Danh sách việc cần làm

1. Thêm `BatchNormalization` giữa lớp tuyến tính và hàm kích hoạt, bỏ hệ số điều chỉnh (`use_bias=False`).
2. Dùng cỡ lô vừa phải; lô quá nhỏ làm thống kê quá nhiễu.

3. Nếu huấn luyện chậm hoặc không ổn định ở mạng sâu, thử thêm Batch Norm trước khi chỉnh nhiều thứ khác.
4. Không tự tính thống kê lúc dự đoán: để framework dùng trung bình trượt.
5. Không dựa vào Batch Norm để chống quá khớp.

3.8.5.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Kết quả dự đoán một mẫu khác hẳn kết quả khi dự đoán cả lô	dự đoán đang dùng thống kê của chính lô thay vì trung bình trượt; để framework ở chế độ dự đoán
Lô quá nhỏ (ví dụ 2 hoặc 4) thì huấn luyện bất ổn	trung bình và phương sai của lô quá nhiễu; tăng cỡ lô
Đặt hàm kích hoạt trước Batch Norm rồi kết quả khác dự kiến	thứ tự phổ biến là tuyến tính, Batch Norm, rồi kích hoạt
Nhầm `beta` của Batch Norm với `beta` của momentum	hai thứ hoàn toàn khác nhau, chỉ trùng tên
Vẫn giữ hệ số điều chỉnh `b` cạnh Batch Norm	vô ích, vì bị triệt tiêu; bỏ đi

Bảng 3.28: Lỗi thường gặp với Batch Norm

3.8.5.3 Bảng tra cứu API

Việc cần làm	Keras
Thêm Batch Norm	`layers.BatchNormalization()`
Bỏ hệ số điều chỉnh của lớp trước	`layers.Dense(..., use_bias=False)`
Hàm kích hoạt tách riêng	`layers.Activation("relu")`
Huấn luyện / dự đoán	`fit` dùng thống kê của lô; `predict` dùng trung bình trượt

Bảng 3.29: Lệnh Keras cho Batch Norm

3.8.5.4 Tóm tắt cần nhớ

- Batch Norm chuẩn hoá tổng có trọng số của lớp ẩn theo từng lô, rồi co giãn và dịch bằng `gamma`, `beta` học được.
- Nó đặt giữa tổng có trọng số và hàm kích hoạt; hệ số điều chỉnh `b` không còn cần thiết.
- Hiệu quả vì: chuẩn hoá giá trị lớp ẩn, giảm dịch chuyển phân phối giữa các lớp, và có chính quy hoá nhẹ.
- Lúc dự đoán dùng trung bình trượt của trung bình và phương sai tích luỹ lúc huấn luyện.
- Trong dự án, chỉ cần một lớp trong framework.

3.9 Phân loại nhiều lớp với softmax

Mọi bài toán phân loại từ đầu đến giờ chỉ có hai nhãn: mèo hay không phải mèo, 0 hay 1. Thực tế thường nhiều hơn: nhận dạng mèo, chó, gà con, hoặc "không thuộc loại nào". **Hồi quy softmax** (softmax regression) là phiên bản tổng quát của hồi quy logistic cho trường hợp có nhiều lớp. Mục này trả lời ba

câu hỏi: lớp ra tính xác suất cho từng lớp thế nào, mất mát nào dùng để huấn luyện, và trong code thì cần gì.

3.9.1 Lớp softmax

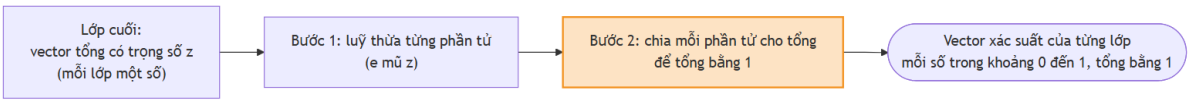
3.9.1.1 Bài toán có nhiều lớp

Giả sử cần nhận dạng bốn loại ảnh: "không thuộc loại nào" (lớp 0), mèo (lớp 1), chó (lớp 2) và gà con (lớp 3). Các lớp được đánh số từ 0 đến (số lớp trừ 1); ở đây có bốn lớp, đánh số 0, 1, 2, 3. Lớp ra của mạng có **một nút cho mỗi lớp**, mỗi nút cho biết xác suất ảnh thuộc lớp tương ứng. Vì bốn xác suất phải cộng lại bằng 1, đầu ra của mạng là một vector bốn số có tổng bằng 1, thay vì một số như trước.

Cách thực hiện chuẩn là dùng một **lớp softmax** ở cuối mạng.

3.9.1.2 Hai bước của softmax

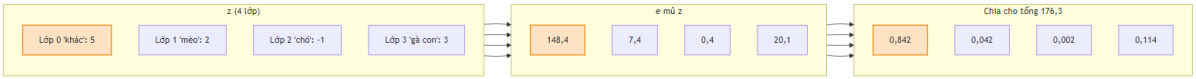
Ở lớp cuối, bước đầu giống mọi lớp khác: tính vector tổng có trọng số bằng cách nhân trọng số với kích hoạt lớp trước rồi cộng hệ số điều chỉnh. Vector này có bốn phần tử, mỗi lớp một số. Sau đó hàm kích hoạt softmax làm hai việc (Hình 3.51):



Hình 3.51: Softmax biến vector tổng có trọng số thành vector xác suất

1. **Luỹ thừa từng phần tử**: thay mỗi số bằng luỹ thừa của hằng số tự nhiên (khoảng 2,718) với số mũ là chính số đó. Mọi kết quả đều dương.
2. **Chuẩn hoá**: chia mỗi kết quả cho tổng của tất cả, để tổng bằng 1.

Hình 3.52 làm đủ hai bước trên một ví dụ: vector tổng có trọng số là 5, 2, trừ 1 và 3.



Hình 3.52: Ví dụ softmax với bốn lớp. Tổng sau bước luỹ thừa là 176,3. Lớp có số lớn nhất nhận xác suất lớn nhất

```
import numpy as np

def softmax(z):
    t = np.exp(z)                  # bước 1: luỹ thừa từng phần tử
    return t / np.sum(t)           # bước 2: chia cho tổng để tổng bằng 1

z = np.array([5., 2., -1., 3.])    # tổng có trọng số của lớp cuối, mỗi lớp một
↪   số
a = softmax(z)
print(a.round(3))                  # [0.842 0.042 0.002 0.114]
print(a.sum())                     # tổng bằng 1
```

Đọc kết quả: với ảnh này, mạng cho 84,2% khả năng thuộc lớp 0, 4,2% lớp mèo, 0,2% lớp chó và 11,4% lớp gà con. Phép luỹ thừa làm số lớn nhất nổi bật hẳn, phép chia cho tổng đưa tất cả về thang xác suất.

3.9.1.3 Điểm khác của softmax so với các hàm kích hoạt trước

Sigmoid và ReLU nhận một số và trả một số. Softmax thì **nhận cả vector và trả cả vector**, vì mỗi xác suất phụ thuộc vào mọi phần tử của vector đầu vào (qua phép chia cho tổng). Vì thế softmax thường chỉ dùng ở lớp ra.

3.9.1.4 Vì sao tên là "softmax"

Đối lập với softmax là **hardmax**: đặt 1 vào vị trí của phần tử lớn nhất và 0 vào mọi vị trí khác. Softmax là phiên bản "mềm": thay vì chọn một lớp cứng, nó gán một xác suất cho từng lớp, lớp lớn nhất có xác suất lớn nhất.

```
hard = (z == z.max()).astype(float)
print(hard)                          # [1. 0. 0. 0.]: hardmax chọn cứng một lớp
print(softmax(z).round(3))           # [0.842 0.042 0.002 0.114]: softmax cho xác
↪  suất từng lớp
```

3.9.1.5 Softmax tổng quát hoá hồi quy logistic

Nếu chỉ có hai lớp, softmax cho ra hai số có tổng bằng 1 (ví dụ 0,842 và 0,158). Hai số này thừa nhau: biết một số thì biết số kia. Tính chỉ một trong hai là đủ, và cách tính đó chính là sigmoid của hồi quy logistic. Nói cách khác, **softmax với hai lớp thu về hồi quy logistic**, và softmax là phiên bản cho nhiều lớp.

3.9.1.6 Softmax biểu diễn được gì

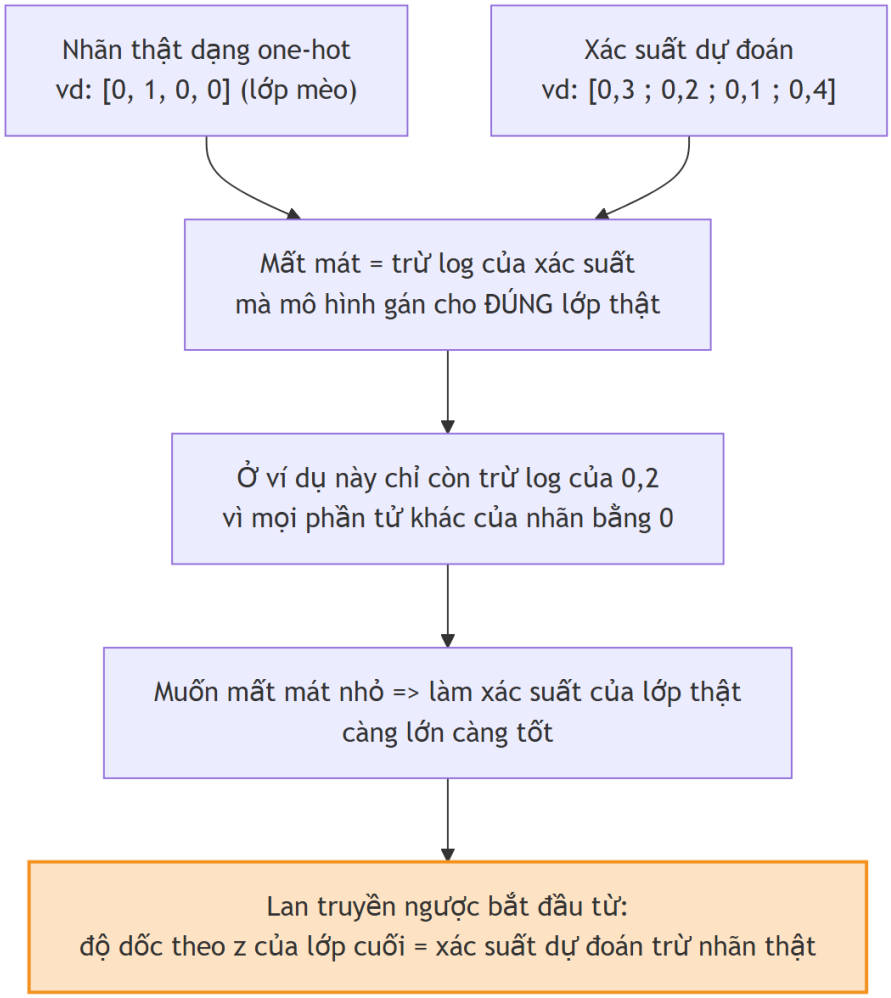
Xét mạng không có lớp ẩn: chỉ tính tổng có trọng số của đặc trưng đầu vào rồi áp softmax. Ranh giới giữa các lớp lúc đó là những đường thẳng, nhưng nhiều đường thẳng ghép lại chia không gian thành ba, bốn, năm hay sáu vùng, mỗi vùng thuộc một lớp (vùng của lớp nào là nơi xác suất của lớp đó cao nhất). Khi thêm các lớp ẩn phía trước, mạng học được những ranh giới cong, phức tạp hơn giữa nhiều lớp.

3.9.2 Huấn luyện mạng có lớp softmax

3.9.2.1 Hàm mất mát

Với một mẫu, nhãn thật được viết dưới dạng **one-hot**: vector có đúng một phần tử bằng 1 ở vị trí lớp thật, còn lại bằng 0. Ví dụ ảnh mèo (lớp 1) có nhãn là bốn số 0, 1, 0, 0. Giả sử mạng dự đoán xác suất 0,3; 0,2; 0,1; 0,4. Mạng làm chưa tốt: ảnh là mèo mà chỉ gán 20% cho lớp mèo.

Mất mát của softmax (cross-entropy nhiều lớp) là **trừ tổng, qua mọi lớp, của (nhãn nhân logarit của xác suất dự đoán)**. Vì nhãn one-hot chỉ khác 0 ở lớp thật, mọi số hạng khác biến mất, chỉ còn **trừ logarit của xác suất mà mô hình gán cho đúng lớp thật** (Hình 3.53). Ở ví dụ này, đó là trừ logarit của 0,2.



Hình 3.53: Mất mát softmax chỉ nhìn vào xác suất của lớp thật; lan truyền ngược bắt đầu từ "xác suất dự đoán trừ nhãn thật"

Để giảm mất mát khi huấn luyện bằng gradient descent, cách duy nhất là làm xác suất của lớp thật *càng lớn càng tốt* (tối đa là 1). Nghĩa là thuật toán cố đẩy xác suất của lớp đúng lên, và nhờ tổng xác suất bằng 1, xác suất của các lớp sai tự giảm theo. Chi phí cho cả tập huấn luyện là trung bình mất mát của từng mẫu. (Người học thống kê sẽ nhận ra đây là ước lượng hợp lý cực đại.)

```
y = np.array([0., 1., 0., 0.])                  # nhãn one-hot: ảnh mèo (lớp 1)
y_hat = np.array([0.3, 0.2, 0.1, 0.4])          # xác suất mạng dự đoán
loss = -np.sum(y * np.log(y_hat))               # các số hạng có nhãn bằng 0 biến
↪  mất, chỉ còn -log(0.2)
print(round(loss, 3))                           # 1.609
```

3.9.2.2 Dạng ma trận khi xử lý nhiều mẫu

Khi vector hoá qua nhiều mẫu, ma trận nhãn (tên trong code là Y) xếp các nhãn one-hot thành các cột nên có hình dạng (số lớp, số mẫu); ma trận dự đoán cũng có hình dạng (số lớp, số mẫu). Ví dụ với bốn lớp: cột đầu của ma trận nhãn là 0, 1, 0, 0 nếu mẫu đầu là mèo, cột hai có thể là chó, cột ba có thể là "không thuộc loại nào", và cứ thế.

3.9.2.3 Lan truyền ngược bắt đầu từ đâu

Phương trình then chốt để khởi động lan truyền ngược là độ dốc của chi phí theo vector tổng có trọng số của lớp cuối, và nó có dạng gọn: **vector xác suất dự đoán trừ vector nhãn thật**, cả hai cùng hình dạng (số lớp, 1) cho một mẫu. Từ đó phần còn lại của lan truyền ngược chạy như các mục trước. Nếu tự viết từ đầu, hãy nhớ biểu thức này; còn khi dùng framework thì không cần, vì framework tự tính độ dốc cho bạn.

```
dz = y_hat - y                                  # điểm xuất phát của lan truyền
↪  ngược
print(dz.round(2))                              # [ 0.3 -0.8  0.1  0.4]: kéo xác
↪  suất lớp 1 lên, đẩy các lớp khác xuống
```

3.9.3 Softmax trong Keras

Trong Keras, lớp ra có một nút cho mỗi lớp và hàm kích hoạt softmax; mất mát là cross-entropy nhiều lớp. Có hai cách ghi nhãn, và mỗi cách đi với một tên mất mát:

Dạng nhãn	Mất mát trong Keras	Ghi chú
One-hot (ví dụ `[0, 1, 0, 0]`)	`categorical_crossentropy`	dạng nhãn đã học ở trên
Số nguyên chỉ lớp (ví dụ `1`)	`sparse_categorical_crossentropy`	không cần đổi nhãn thành one-hot

Bảng 3.30: Hai cách ghi nhãn và mất mát tương ứng

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    keras.Input(shape=(12288,)),
    layers.Dense(25, activation="relu"),
    layers.Dense(12, activation="relu"),
    layers.Dense(4, activation="softmax"),            # lớp ra: 4 nút, 4 xác
    ↪  suất có tổng bằng 1
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",   # nhãn là số nguyên 0, 1,
              ↪  2, 3
              metrics=["accuracy"])
```

Một cách làm khác: để mất mát tự áp softmax. Bài thực hành ở mục 3.11 không đặt softmax vào lớp cuối. Lớp cuối chỉ cho vector tổng có trọng số (còn gọi là *logits*), và hàm mất mát được khai báo `from_logits=True` để tự áp softmax bên trong. Cách này gọn và ổn định số hơn; cả hai cách đều dùng được.

```
model = keras.Sequential([
    keras.Input(shape=(12288,)),
    layers.Dense(25, activation="relu"),
    layers.Dense(12, activation="relu"),
    layers.Dense(4),                                  # không kích hoạt: trả về
    ↪  logits
])
model.compile(optimizer="adam",
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
```

```
              metrics=["accuracy"])
# Khi dự đoán xác suất: áp softmax lên logits
# probs = keras.activations.softmax(model(X_test))
```

3.9.4 Áp dụng vào dự án thực tế

3.9.4.1 Danh sách việc cần làm

1. Số nút lớp ra bằng số lớp; hàm kích hoạt lớp ra là softmax (hai lớp thì dùng một nút sigmoid là đủ).
2. Chọn dạng nhãn trước rồi chọn mất mát khớp với nó (Bảng 3.30): nhãn one-hot đi với một loại, nhãn số nguyên đi với loại còn lại.
3. Nếu lớp cuối không có softmax, khai báo `from_logits=True`; nếu đã có softmax thì không.
4. Đọc kết quả bằng `argmax` (lớp có xác suất cao nhất), không phải bằng ngưỡng 0,5 như hai lớp.

3.9.4.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Lỗi hình dạng giữa nhãn và dự đoán	nhãn số nguyên đi với mất mát `categorical`, hoặc nhãn one-hot đi với mất mát `sparse`; đổi mất mát cho khớp
Chi phí lạ hoặc học kém dù mạng không đổi	áp softmax hai lần: lớp cuối đã có softmax mà mất mát vẫn khai báo `from_logits=True`
Xác suất các lớp không cộng lại bằng 1	lớp ra dùng sigmoid thay vì softmax
Đọc lớp dự đoán bằng ngưỡng 0,5	với nhiều lớp phải lấy `argmax`

Bảng 3.31: Lỗi thường gặp với phân loại nhiều lớp

3.9.4.3 Bảng tra cứu API

Việc cần làm	Công cụ
Lớp ra nhiều lớp	`layers.Dense(C, activation="softmax")`
Mất mát với nhãn one-hot	`"categorical_crossentropy"`
Mất mát với nhãn số nguyên	`"sparse_categorical_crossentropy"`
Mất mát nhận logits	`SparseCategoricalCrossentropy(from_logits=True)`
Đổi nhãn sang one-hot	`tf.one_hot(nhan, C)` hoặc `keras.utils.to_categorical`
Lấy lớp dự đoán	`np.argmax(xac_suat, axis=1)`

Bảng 3.32: Lệnh dùng cho phân loại nhiều lớp

3.9.4.4 Tóm tắt cần nhớ

- Softmax tổng quát hoá hồi quy logistic cho nhiều lớp; với hai lớp nó thu về hồi quy logistic.
- Hai bước: luỹ thừa từng phần tử rồi chia cho tổng, được vector xác suất có tổng bằng 1.
- Softmax nhận vector và trả vector; hardmax chọn cứng một lớp, softmax cho xác suất mềm.

- Mất mát softmax chỉ nhìn xác suất của lớp thật: giảm mất mát nghĩa là đẩy xác suất lớp thật lên.
- Lan truyền ngược bắt đầu từ "xác suất dự đoán trừ nhãn thật"; framework tự lo phần này.

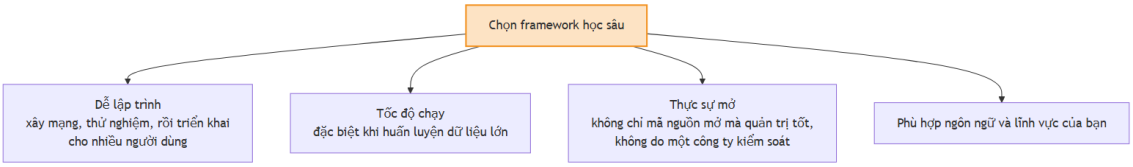
3.10 Framework học sâu và TensorFlow

Đến đây bạn đã tự viết nhiều thuật toán bằng Python và NumPy, và việc đó có giá trị: bạn hiểu mạng nơ-ron thực sự làm gì. Nhưng khi chuyển sang những mô hình phức tạp hơn như mạng tích chập, mạng hồi quy, hay mô hình rất lớn, tự viết mọi thứ từ đầu không còn thực tế với hầu hết mọi người. Đó là lúc cần **framework học sâu** (deep learning framework).

Có thể so sánh với phép nhân ma trận: bạn nên hiểu cách nhân hai ma trận và đủ khả năng tự viết, nhưng khi xây ứng dụng lớn, bạn gọi một thư viện đại số tuyến tính đã được tối ưu thay vì tự viết lại. Học sâu đã đủ trưởng thành để ở vào thế đó: dùng framework hiệu quả hơn, và hiểu cách nó hoạt động giúp bạn biết code của mình đang làm gì.

3.10.1 Chọn framework

Hiện có nhiều framework giúp xây mạng nơ-ron dễ dàng; ví dụ TensorFlow, Keras, PaddlePaddle, Torch, Caffe. Mỗi framework có cộng đồng người dùng và người phát triển riêng, và mỗi cái là lựa chọn đáng tin cho một số loại ứng dụng. Chúng đang tiến bộ rất nhanh từng tháng, nên thay vì khuyến nghị một cái cụ thể, hãy dùng ba tiêu chí sau (Hình 3.54):



Hình 3.54: Các tiêu chí để chọn framework

- **Dễ lập trình**: gồm cả việc xây mạng và thử nghiệm nhanh, lẫn việc đưa vào chạy thực tế cho hàng nghìn hay hàng triệu người dùng.
- **Tốc độ chạy**: đặc biệt khi huấn luyện trên tập dữ liệu lớn; có framework huấn luyện hiệu quả hơn framework khác.
- **Thực sự mở**: không chỉ là mã nguồn mở mà còn cần quản trị tốt. Có những công ty mở mã nguồn nhưng giữ quyền kiểm soát duy nhất, rồi theo thời gian đóng dần các phần mở hoặc chuyển chức năng sang dịch vụ đám mây riêng. Vì vậy cần cân nhắc xem framework có khả năng còn mở lâu dài hay chỉ phụ thuộc một công ty.

Ngoài ba tiêu chí, ngôn ngữ lập trình bạn thích (Python, Java, C++, …) và lĩnh vực ứng dụng (thị giác máy tính, xử lý ngôn ngữ, quảng cáo trực tuyến, …) cũng ảnh hưởng. Trong ngắn hạn, nhiều framework đều là lựa chọn tốt. Dù chọn cái nào, framework cung cấp mức trừu tượng cao hơn một thư viện đại số tuyến tính thuần, nên giúp bạn phát triển nhanh hơn.

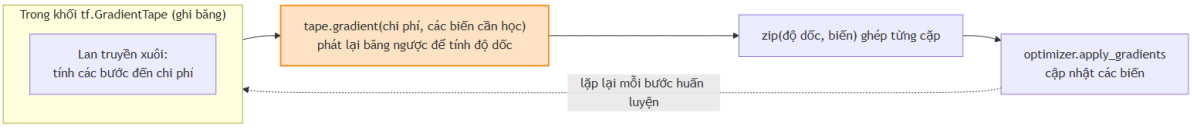
3.10.2 Cấu trúc một chương trình TensorFlow

Phần còn lại của mục này đi qua cấu trúc cơ bản của một chương trình TensorFlow. Ý chính: **bạn chỉ cần viết phần lan truyền xuôi (tính chi phí); TensorFlow tự tính độ dốc và cập nhật tham số**.

3.10.2.1 Một bài toán minh hoạ

Giả sử cần cực tiểu hoá hàm chi phí: w bình phương, trừ 10 lần w, cộng 25. Hàm này thực chất là (w trừ 5) bình phương, nên giá trị w làm chi phí nhỏ nhất là 5. Ta coi như chưa biết và để TensorFlow tự tìm. Cấu trúc này giống hệt khi huấn luyện mạng nơ-ron, chỉ khác là chi phí phụ thuộc vào rất nhiều trọng số.

Mỗi bước huấn luyện gồm bốn việc (Hình 3.55):



Hình 3.55: Một bước huấn luyện trong TensorFlow: ghi băng khi tính chi phí, phát lại băng ngược để lấy độ dốc, rồi cập nhật

1. Khai báo biến cần học bằng `tf.Variable` (ở đây là w, khởi tạo bằng 0, kiểu số thực 32 bit).
2. Chọn bộ tối ưu, ở đây là Adam với tốc độ học 0,1.
3. Trong khối `tf.GradientTape`, tính chi phí. "Băng" ghi lại thứ tự các phép tính, giống băng cát-sét ghi âm.
4. Gọi `tape.gradient` để "phát băng ngược" và lấy độ dốc, rồi `optimizer.apply_gradients` để cập nhật biến. Hàm `zip` ghép từng độ dốc với biến tương ứng.

```
import numpy as np
import tensorflow as tf

w = tf.Variable(0, dtype=tf.float32)                    # biến cần học, khởi tạo
↪  bằng 0
optimizer = tf.keras.optimizers.Adam(0.1)               # bộ tối ưu Adam, tốc độ
↪  học 0,1

def train_step():
    with tf.GradientTape() as tape:                     # bắt đầu ghi băng
        cost = w ** 2 - 10 * w + 25                     # chỉ cần viết phần lan
        ↪  truyền xuôi
    trainable_variables = [w]
    grads = tape.gradient(cost, trainable_variables)    # phát băng ngược để lấy
    ↪  độ dốc
    optimizer.apply_gradients(zip(grads, trainable_variables))   # cập nhật w

print(w)                                                # w ban đầu bằng 0
train_step()
print(w)                                                # sau một bước, w tăng
↪  nhẹ, khoảng 0,1
for i in range(1000):
    train_step()
print(w)                                                # gần 5, đúng giá trị cực
↪  tiểu
```

Theo tài liệu gốc, sau một bước w tăng từ 0 lên khoảng 0,1, và sau một nghìn bước w gần bằng 5, đúng giá trị cực tiểu đã biết. Điều đáng chú ý là *không viết một đạo hàm nào*.

3.10.2.2 Đưa dữ liệu vào hàm chi phí

Ở ví dụ trên, hàm chi phí là một hàm cố định của w. Với mạng thật, chi phí còn phụ thuộc dữ liệu huấn luyện. Để minh hoạ, đặt một mảng `x` gồm ba số 1, trừ 10 và 25: chúng đóng vai các hệ số của hàm chi

phí, như "dữ liệu" điều khiển hàm. Chi phí viết lại thành: x[0] nhân w bình phương, cộng x[1] nhân w, cộng x[2]; vẫn cùng hàm như trước.

```
w = tf.Variable(0, dtype=tf.float32)
x = np.array([1.0, -10.0, 25.0], dtype=np.float32)     # "dữ liệu" điều khiển các
↪  hệ số của hàm chi phí
optimizer = tf.keras.optimizers.Adam(0.1)

def cost_fn():
    return x[0] * w ** 2 + x[1] * w + x[2]             # cùng hàm chi phí như
    ↪  trước

for i in range(1000):
    optimizer.minimize(cost_fn, [w])                   # gom ba việc: ghi băng,
    ↪  lấy độ dốc, cập nhật

print(w)                                               # lại gần 5
```

Dòng `optimizer.minimize(hàm chi phí, danh sách biến)` là cách viết gọn thay cho khối ghi băng, `tape.gradient` và `apply_gradients` ở trên. Hai cách đều dùng được: viết đầy đủ khi cần can thiệp vào từng bước (như bài thực hành ở mục 3.11), viết gọn khi chỉ cần giảm chi phí.

3.10.2.3 Đồ thị tính toán

Khi bạn viết phép tính chi phí, TensorFlow xây dựng một **đồ thị tính toán** (computation graph): lấy w, bình phương, nhân với hệ số đầu, cộng các số hạng còn lại, cho đến khi ra chi phí. Vì đã có đồ thị của lan truyền xuôi, TensorFlow tự suy ra mọi bước lan truyền ngược cần thiết, nên bạn không phải viết. Đó là một trong những lý do framework giúp làm việc hiệu quả.

Một lợi ích khác: muốn đổi thuật toán tối ưu thì chỉ đổi *một dòng*. Ví dụ thay Adam bằng gradient descent thuần chỉ cần đổi dòng khai báo bộ tối ưu:

```
optimizer = tf.keras.optimizers.SGD(0.01)              # đổi một dòng là đổi
↪  thuật toán tối ưu
# optimizer = tf.keras.optimizers.Adam(0.1)
```

3.10.3 Đối chiếu NumPy và TensorFlow

Hầu hết khái niệm đã quen thuộc từ NumPy, chỉ khác tên gọi. Bảng 3.33 là bảng tra nhanh. Các đối tượng chính của TensorFlow là `tf.Tensor`, tương đương mảng NumPy nhưng có thêm thông tin về đồ thị tính toán.

Việc cần làm	NumPy	TensorFlow
Mảng không đổi	`np.array`	`tf.constant`
Giá trị có thể thay đổi (tham số cần học)	mảng thường	`tf.Variable`
Nhân ma trận	`np.dot`	`tf.matmul` (hoặc `tf.linalg.matmul`)
Cộng	`+`	`tf.add` (hoặc `tf.math.add`)
Đổi hình dạng	`reshape`	`tf.reshape`
Đổi kiểu số	`astype`	`tf.cast`
Hàm kích hoạt	tự viết	`tf.keras.activations.relu`, `.sigmoid`
Nhãn one-hot	tự viết	`tf.one_hot`
Lấy độ dốc	tự viết lan truyền ngược	`tf.GradientTape`

Bảng 3.33: Đối chiếu một số thao tác giữa NumPy và TensorFlow

Khác biệt cần nhớ: `tf.constant` không đổi được trạng thái, còn `tf.Variable` đổi được. Biến chỉ được tạo một lần, giá trị khởi tạo quyết định hình dạng và kiểu; nên nêu kiểu số (`dtype`) rõ ràng để tránh lỗi bất ngờ. Với mô hình lớn hơn, bạn thường dùng lớp `Dense`, `Sequential` của Keras (tích hợp trong TensorFlow) thay vì tự quản lý từng biến; nhưng khi cần kiểm soát từng bước, vòng lặp với `GradientTape` vẫn là công cụ chính.

3.10.4 Áp dụng vào dự án thực tế

3.10.4.1 Danh sách việc cần làm

1. Dùng framework cho mô hình thật; tự viết bằng NumPy chỉ để học hoặc kiểm tra.
2. Chọn framework theo tiêu chí dễ lập trình, tốc độ, độ mở, rồi theo ngôn ngữ và lĩnh vực.
3. Với mô hình chuẩn, dùng Keras (`Sequential`, `fit`); với thí nghiệm cần can thiệp từng bước, dùng `GradientTape` và `apply_gradients`.
4. Khai báo tham số bằng `tf.Variable` và nêu rõ `dtype`.
5. Chỉ cần đảm bảo lan truyền xuôi đúng; đồ thị tính toán lo phần độ dốc.

3.10.4.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
`tape.gradient` trả về `None`	phép tính chi phí nằm ngoài khối `GradientTape`, hoặc biến không phải `tf.Variable`; tính chi phí trong khối
Biến không thay đổi sau khi huấn luyện	quên gọi `apply_gradients`, hoặc quên đưa biến vào danh sách cần học
Lỗi kiểu số khi nhân ma trận	trộn `float32` và `float64`; dùng `tf.cast` hoặc nêu `dtype`
Muốn đổi thuật toán tối ưu mà viết lại cả vòng lặp	chỉ cần đổi dòng khai báo `optimizer`

Bảng 3.34: Lỗi thường gặp khi bắt đầu với TensorFlow

3.10.4.3 Bảng tra cứu API

Việc cần làm	Công cụ
Khai báo tham số cần học	`tf.Variable(giá_trị, dtype=tf.float32)`
Chọn bộ tối ưu	`tf.keras.optimizers.Adam(tốc_độ_học)`, `SGD`
Ghi băng	`with tf.GradientTape() as tape:`
Lấy độ dốc	`tape.gradient(chi_phí, danh_sách_biến)`
Cập nhật biến	`optimizer.apply_gradients(zip(grads, biến))`
Cách viết gọn	`optimizer.minimize(hàm_chi_phí, danh_sách_biến)`

Bảng 3.35: Các lệnh cốt lõi của vòng huấn luyện TensorFlow

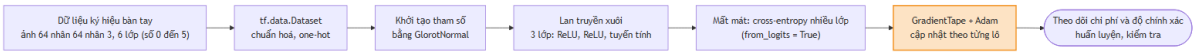
3.10.4.4 Tóm tắt cần nhớ

- Với mô hình lớn hoặc phức tạp, dùng framework thay vì tự viết từ đầu; nhưng vẫn cần hiểu bên trong.

- Chọn framework theo dễ lập trình, tốc độ chạy, độ mở thực sự, rồi theo ngôn ngữ và lĩnh vực.
- TensorFlow: bạn viết lan truyền xuôi; `GradientTape` ghi lại phép tính để tự lấy độ dốc.
- Đổi thuật toán tối ưu chỉ cần đổi một dòng.
- `tf.constant` không đổi, `tf.Variable` đổi được; nêu rõ `dtype`.

3.11 Thực hành: mạng ba lớp bằng TensorFlow

Mục này dựng một mạng nơ-ron hoàn chỉnh bằng TensorFlow, đi qua mọi bước của Hình 3.56: nạp dữ liệu vào `tf.data`, khởi tạo tham số, lan truyền xuôi, tính mất mát nhiều lớp, rồi huấn luyện bằng `GradientTape` và Adam. Các con số trong mục này là kết quả của chính bài thực hành gốc (TensorFlow 2.3); mục không tự đo thêm số liệu nào khác.



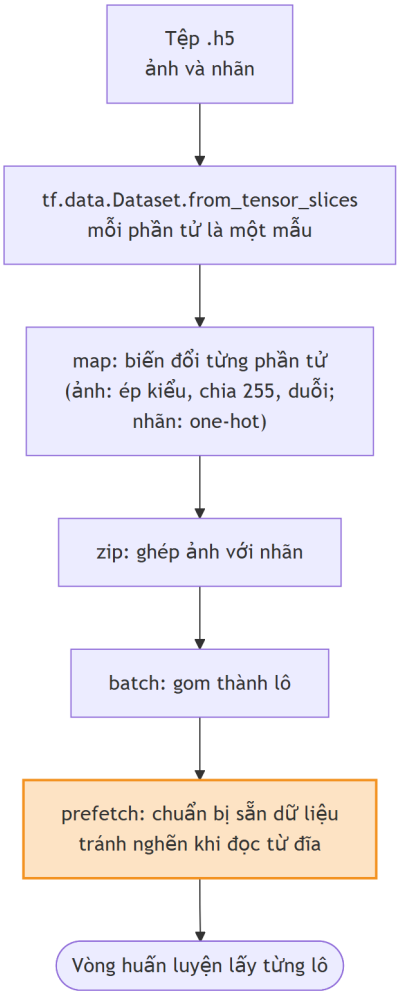
Hình 3.56: Quy trình bài thực hành: dữ liệu, khởi tạo, lan truyền xuôi, mất mát nhiều lớp, huấn luyện

3.11.1 Bài toán và dữ liệu

Dữ liệu là tập ảnh ký hiệu bàn tay biểu diễn các chữ số từ 0 đến 5, nên có **sáu lớp**. Mỗi ảnh cỡ 64 nhân 64 điểm ảnh, ba kênh màu. Tập huấn luyện có 1080 ảnh, tập kiểm tra có 120 ảnh. Dữ liệu nằm trong hai tệp `.h5`.

3.11.1.1 Đưa dữ liệu vào `tf.data`

`tf.data.Dataset` là một "bộ sinh dữ liệu" của TensorFlow: dùng thay mảng NumPy. Không đọc trực tiếp nội dung được, mà phải lặp qua hoặc lấy từng phần tử bằng `iter` và `next`. Có thể xem hình dạng và kiểu của mỗi phần tử bằng `element_spec`. Muốn biến đổi toàn bộ phần tử thì dùng `map`. Hình 3.57 tóm tắt toàn bộ đường đi của dữ liệu.



Hình 3.57: Đường đi của dữ liệu: nạp, biến đổi từng phần tử, ghép ảnh với nhãn, gom lô, chuẩn bị sẵn

```
import h5py
import numpy as np
import tensorflow as tf

train_dataset = h5py.File('datasets/train_signs.h5', "r")
test_dataset = h5py.File('datasets/test_signs.h5', "r")

x_train = tf.data.Dataset.from_tensor_slices(train_dataset['train_set_x'])   #
↪  mỗi phần tử là một ảnh
y_train = tf.data.Dataset.from_tensor_slices(train_dataset['train_set_y'])   #
↪  mỗi phần tử là một nhãn
x_test = tf.data.Dataset.from_tensor_slices(test_dataset['test_set_x'])
y_test = tf.data.Dataset.from_tensor_slices(test_dataset['test_set_y'])

print(x_train.element_spec)        # TensorSpec(shape=(64, 64, 3),
↪  dtype=tf.uint8, name=None)

unique_labels = set()
for element in y_train:
    unique_labels.add(element.numpy())
print(unique_labels)               # {0, 1, 2, 3, 4, 5}: sáu lớp
```

3.11.1.2 Chuẩn hoá ảnh và mã hoá nhãn one-hot

Mỗi ảnh được ép sang số thực, chia cho 255 và duỗi thành vector 12288 giá trị, giống mọi bài ảnh trước. Mỗi nhãn được đổi sang one-hot (mục 3.9.2) bằng `tf.one_hot` với `axis=0`, tức trục mới đặt ở chiều đầu tiên.

```
def normalize(image):
    image = tf.cast(image, tf.float32) / 255.0      # ép kiểu và đưa về khoảng 0
    ↪  đến 1
    image = tf.reshape(image, [-1,])                # duỗi thành vector 12288 giá
    ↪  trị
    return image

new_train = x_train.map(normalize)
new_test = x_test.map(normalize)
print(new_train.element_spec)      # TensorSpec(shape=(12288,), dtype=tf.float32,
↪  name=None)

def one_hot_matrix(label, C=6):
    one_hot = tf.reshape(tf.one_hot(label, C, axis=0), shape=[C, ])
    return one_hot

new_y_train = y_train.map(one_hot_matrix)
new_y_test = y_test.map(one_hot_matrix)
print(next(iter(new_y_test)))      # tf.Tensor([1. 0. 0. 0. 0. 0.], shape=(6,),
↪  dtype=float32)
```

3.11.2 Làm quen với TensorFlow qua các hàm nhỏ

Trước khi xây mạng, bài gốc cho thử vài phép tính cơ bản. Chúng cho thấy cách TensorFlow thay thế các thao tác NumPy quen thuộc.

3.11.2.1 Hàm tuyến tính: nhân ma trận cộng hệ số

Tính tổng có trọng số của một lớp với ma trận trọng số cỡ (4, 3), đầu vào cỡ (3, 1) và hệ số điều chỉnh cỡ (4, 1), cả ba lấy ngẫu nhiên từ phân phối chuẩn và để là `tf.constant`:

```
def linear_function():
    np.random.seed(1)
    X = tf.constant(np.random.randn(3, 1), name="X")
    W = tf.constant(np.random.randn(4, 3), name="W")
    b = tf.constant(np.random.randn(4, 1), name="b")
    Y = tf.add(tf.matmul(W, X), b)
    return Y

print(linear_function())
# tf.Tensor(
# [[-2.15657382]
#  [ 2.95891446]
#  [-1.08926781]
#  [-0.84538042]], shape=(4, 1), dtype=float64)
```

3.11.2.2 Sigmoid có sẵn

TensorFlow có sẵn các hàm kích hoạt. Hàm dưới ép đầu vào sang `float32` rồi gọi sigmoid của Keras:

```
def sigmoid(z):
    z = tf.cast(z, tf.float32)
    a = tf.keras.activations.sigmoid(z)
    return a

print(sigmoid(-1))                # tf.Tensor(0.26894143, shape=(),
↪  dtype=float32)
print(sigmoid(0.0))               # tf.Tensor(0.5, shape=(), dtype=float32)
print(sigmoid(12))                # tf.Tensor(0.9999939, shape=(),
↪  dtype=float32)
```

3.11.2.3 One-hot

Hàm `one_hot_matrix` ở trên nhận một nhãn và số lớp, trả về vector one-hot:

```
print(one_hot_matrix(tf.constant(1), 4))       # tf.Tensor([0. 1. 0. 0.],
↪  shape=(4,), dtype=float32)
print(one_hot_matrix([2], 5))                  # tf.Tensor([0. 0. 1. 0. 0.],
↪  shape=(5,), dtype=float32)
```

3.11.3 Xây mạng ba lớp

Mạng gồm ba lớp: tuyến tính rồi ReLU, tuyến tính rồi ReLU, tuyến tính (lớp cuối chỉ cho logits, chưa áp softmax). Kích thước các lớp: 12288 giá trị vào, 25 nút, 12 nút, 6 nút ra.

3.11.3.1 Khởi tạo tham số

Tham số khởi tạo bằng bộ khởi tạo Glorot dạng chuẩn (`GlorotNormal`): lấy số từ phân phối chuẩn cắt hai đầu, độ lệch chuẩn co theo số nút vào và ra của lớp. Mỗi tham số là một `tf.Variable` để TensorFlow theo dõi.

```
def initialize_parameters():
    initializer = tf.keras.initializers.GlorotNormal(seed=1)
    W1 = tf.Variable(initializer(shape=(25, 12288)))
    b1 = tf.Variable(initializer(shape=(25, 1)))
    W2 = tf.Variable(initializer(shape=(12, 25)))
    b2 = tf.Variable(initializer(shape=(12, 1)))
    W3 = tf.Variable(initializer(shape=(6, 12)))
    b3 = tf.Variable(initializer(shape=(6, 1)))
    return {"W1": W1, "b1": b1, "W2": W2, "b2": b2, "W3": W3, "b3": b3}

parameters = initialize_parameters()
for k in parameters:
    print(k, tuple(parameters[k].shape))
# W1 (25, 12288)   b1 (25, 1)   W2 (12, 25)   b2 (12, 1)   W3 (6, 12)   b3 (6, 1)
```

Hình dạng khớp quy tắc ở mục 2.6.2: số hàng bằng số nút của lớp, số cột bằng số nút lớp trước.

3.11.3.2 Lan truyền xuôi

Chỉ cần viết lan truyền xuôi bằng các hàm của TensorFlow; không áp softmax ở đây.

```
def forward_propagation(X, parameters):
    W1, b1 = parameters['W1'], parameters['b1']
```

```
    W2, b2 = parameters['W2'], parameters['b2']
    W3, b3 = parameters['W3'], parameters['b3']
    Z1 = tf.math.add(tf.linalg.matmul(W1, X), b1)
    A1 = tf.keras.activations.relu(Z1)
    Z2 = tf.math.add(tf.linalg.matmul(W2, A1), b2)
    A2 = tf.keras.activations.relu(Z2)
    Z3 = tf.math.add(tf.linalg.matmul(W3, A2), b3)
    return Z3                                       # logits, hình dạng (6, số
    ↪  mẫu)
```

Chạy thử với một lô hai ảnh cho kết quả cỡ (6, 2): mỗi cột là sáu logits của một ảnh. Khi gọi hàm này bên trong khối `GradientTape`, các độ dốc theo mọi tham số đều lấy được, nghĩa là TensorFlow đã theo dõi đúng các phép tính.

3.11.3.3 Tổng mất mát

Bài toán có sáu lớp nên dùng cross-entropy nhiều lớp. Vì lớp cuối chưa có softmax, đặt `from_logits=True` để hàm mất mát tự áp softmax bên trong (mục 3.9.3). Hàm mất mát của Keras mong nhận hình dạng (số mẫu, số lớp), nên cần chuyển vị logits và nhãn.

Hàm tính **tổng** mất mát của một lô thay vì trung bình, rồi cuối epoch mới chia cho tổng số mẫu. Lý do là để kết quả nhất quán khi lô cuối nhỏ hơn: ví dụ có 5 mẫu với mất mát 0, 1, 2, 3, 4 và cỡ lô 4. Trung bình toàn bộ là 2. Nếu lấy trung bình từng lô (1,5 và 4) rồi lại lấy trung bình hai số đó thì được 2,75, sai. Còn lấy tổng của từng lô (6 và 4) rồi chia cho 5 vẫn ra đúng 2.

```
def compute_total_loss(logits, labels):
    total_loss = tf.reduce_sum(
        tf.keras.losses.categorical_crossentropy(tf.transpose(labels),
        ↪  tf.transpose(logits), from_logits=True))
    return total_loss

# Kiểm tra nhỏ của bài gốc:
labels = tf.constant([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]])
logits = tf.constant([[1., 0., 0.], [1., 0., 0.], [1., 0., 0.]])
print(compute_total_loss(logits, labels))           # tf.Tensor(3.295837,
↪  shape=(), dtype=float32)
```

Một lưu ý của bài gốc: khi dùng tổng mất mát để tính độ dốc, cần giảm tốc độ học khi cỡ lô tăng, để bước đi không quá lớn.

3.11.4 Huấn luyện mô hình

Vòng huấn luyện ghép các phần trên: với mỗi epoch, lặp qua các lô; ở mỗi lô ghi băng, lấy độ dốc và cập nhật bằng Adam. Hai điểm đáng chú ý:

- `dataset.batch(cỡ lô).prefetch(8)`: `prefetch` chuẩn bị sẵn dữ liệu cho bước tiếp theo, tránh nghẽn khi đọc từ đĩa. Dữ liệu được đọc theo dòng nên không cần chứa vừa bộ nhớ.
- `tf.keras.metrics.CategoricalAccuracy` theo dõi độ chính xác. Độ chính xác huấn luyện của một epoch được tích luỹ qua các lô trong chính epoch đó, nên phản ánh mô hình ở nhiều thời điểm khác nhau trong epoch.

```
def model(X_train, Y_train, X_test, Y_test, learning_rate=0.0001,
↪  num_epochs=1500, minibatch_size=32, print_cost=True):
    parameters = initialize_parameters()
    W1, b1 = parameters['W1'], parameters['b1']
```

```
    W2, b2 = parameters['W2'], parameters['b2']
    W3, b3 = parameters['W3'], parameters['b3']

    optimizer = tf.keras.optimizers.Adam(learning_rate)
    test_accuracy = tf.keras.metrics.CategoricalAccuracy()
    train_accuracy = tf.keras.metrics.CategoricalAccuracy()

    dataset = tf.data.Dataset.zip((X_train, Y_train))              # ghép ảnh với
    ↪ nhãn
    test_dataset = tf.data.Dataset.zip((X_test, Y_test))
    m = dataset.cardinality().numpy()                              # số mẫu huấn
    ↪ luyện
    minibatches = dataset.batch(minibatch_size).prefetch(8)
    test_minibatches = test_dataset.batch(minibatch_size).prefetch(8)

    for epoch in range(num_epochs):
        epoch_total_loss = 0.
        train_accuracy.reset_states()                              # đo lại độ
        ↪ chính xác từ 0 mỗi epoch
        for (minibatch_X, minibatch_Y) in minibatches:
            with tf.GradientTape() as tape:                        # ghi băng
                Z3 = forward_propagation(tf.transpose(minibatch_X), parameters)
                minibatch_total_loss = compute_total_loss(Z3,
                ↪ tf.transpose(minibatch_Y))
            train_accuracy.update_state(minibatch_Y, tf.transpose(Z3))
            trainable_variables = [W1, b1, W2, b2, W3, b3]
            grads = tape.gradient(minibatch_total_loss, trainable_variables)
            optimizer.apply_gradients(zip(grads, trainable_variables))
            epoch_total_loss += minibatch_total_loss
        epoch_total_loss /= m                                      # chia tổng
        ↪ cho số mẫu để ra chi phí

        if print_cost == True and epoch % 10 == 0:                 # in và đo tập
        ↪ kiểm tra mỗi 10 epoch
            print("Cost after epoch %i: %f" % (epoch, epoch_total_loss))
            print("Train accuracy:", train_accuracy.result())
            for (minibatch_X, minibatch_Y) in test_minibatches:
                Z3 = forward_propagation(tf.transpose(minibatch_X), parameters)
                test_accuracy.update_state(minibatch_Y, tf.transpose(Z3))
            print("Test_accuracy:", test_accuracy.result())
            test_accuracy.reset_states()
    return parameters

parameters = model(new_train, new_y_train, new_test, new_y_test, num_epochs=100)
```

3.11.4.1 Kết quả

Bài gốc chạy 100 epoch với tốc độ học 0,0001 và cỡ lô 32. Bảng 3.36 liệt kê các mốc được in ra.

Epoch	Chi phí	Chính xác huấn luyện	Chính xác kiểm tra
0	1,8302	17,0%	20,0%
10	1,5524	35,9%	30,8%
20	1,3476	50,8%	45,0%
30	1,1627	61,1%	51,7%
40	1,0353	65,7%	55,0%
50	0,9462	67,9%	61,7%
60	0,8804	70,1%	63,3%
70	0,8260	73,1%	66,7%
80	0,7811	74,1%	67,5%
90	0,7447	75,5%	69,2%

Bảng 3.36: Chi phí và độ chính xác theo epoch (tốc độ học 0,0001, cỡ lô 32, ba lớp 25, 12, 6 nút)

Chi phí giảm đều và độ chính xác tăng đều trên cả hai tập, đúng điều ta mong sau mỗi epoch. Đây chưa phải điểm cuối: sau 90 epoch chi phí vẫn còn đang giảm, và tham số mặc định của hàm là 1500 epoch. Sáu lớp ngang xác suất đoán mò khoảng 16,7%, nên điểm khởi đầu 17,0% huấn luyện và 20,0% kiểm tra đúng như mạng chưa biết gì. Khoảng cách giữa độ chính xác huấn luyện (75,5%) và kiểm tra (69,2%) ở cuối là dấu hiệu cần theo dõi: nếu tiếp tục huấn luyện mà khoảng cách này mở rộng, đó là quá khớp và cần đến các công cụ chính quy hoá ở mục 3.2.

3.11.5 Cùng bài toán bằng Keras

Mọi thứ ở trên đều có thể làm gọn bằng Keras: lớp `Dense`, `compile` với mất mát và bộ tối ưu, `fit` lo toàn bộ vòng lặp, lô và cập nhật. Phiên bản này tương ứng kiến trúc và siêu tham số ở trên; vì ở đây đóng vai chỉ ra cách viết, bài gốc không báo kết quả cho nó nên mục này cũng không nêu số.

```
from tensorflow import keras
from tensorflow.keras import layers

keras_model = keras.Sequential([
    keras.Input(shape=(12288,)),
    layers.Dense(25, activation="relu"),
    layers.Dense(12, activation="relu"),
    layers.Dense(6),                                          # logits, chưa áp
    ↪  softmax
])
keras_model.compile(optimizer=keras.optimizers.Adam(0.0001),
                    loss=keras.losses.CategoricalCrossentropy(from_logits=True),
                    ↪  # nhãn one-hot
                    metrics=["accuracy"])

# Dữ liệu cho Keras: mẫu ở hàng, nhãn one-hot; từ tf.data có thể gom lô và truyền
↪  thẳng vào fit
train_ds = tf.data.Dataset.zip((new_train, new_y_train)).batch(32).prefetch(8)
test_ds = tf.data.Dataset.zip((new_test, new_y_test)).batch(32).prefetch(8)
keras_model.fit(train_ds, validation_data=test_ds, epochs=100, verbose=0)
```

3.11.6 Áp dụng vào dự án thực tế

3.11.6.1 Danh sách việc cần làm

1. Dùng `tf.data` cho dữ liệu: `map` để chuẩn hoá, `batch` để gom lô, `prefetch` để tránh nghẽn đọc.

2. Với phân loại nhiều lớp: lớp cuối không softmax, mất mát khai báo `from_logits=True`.
3. Khi cần kiểm soát từng bước, dùng `GradientTape` và `apply_gradients`; khi không, dùng Keras `fit`.
4. Theo dõi chi phí và độ chính xác của cả hai tập sau mỗi vài epoch.
5. Tính tổng mất mát của lô rồi chia cho số mẫu ở cuối epoch để chi phí nhất quán.

3.11.6.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Lỗi hình dạng khi tính mất mát	Keras mong hình dạng (số mẫu, số lớp) nhưng bài dùng (số lớp, số mẫu); chuyển vị cả logits và nhãn
Độ dốc là `None` cho một số biến	phép tính nằm ngoài `GradientTape` hoặc biến không phải `tf.Variable`
Chi phí rất lạ vì áp softmax hai lần	lớp cuối đã có softmax mà mất mát vẫn khai báo `from_logits=True`; bỏ một trong hai
Chi phí cuối epoch không nhất quán khi lô cuối nhỏ	tính trung bình từng lô; dùng tổng rồi chia cho số mẫu
Bộ nhớ đầy khi nạp dữ liệu	dùng `tf.data` (đọc theo dòng) thay vì nạp cả tập

Bảng 3.37: Lỗi thường gặp trong bài thực hành TensorFlow

3.11.6.3 Bảng tra cứu API

Việc cần làm	Công cụ
Tạo dữ liệu từ mảng hoặc tệp	`tf.data.Dataset.from_tensor_slices`
Biến đổi từng phần tử	`dataset.map(ham)`
Ghép hai dataset	`tf.data.Dataset.zip((a, b))`
Gom lô và chuẩn bị sẵn	`dataset.batch(32).prefetch(8)`
Khởi tạo Glorot dạng chuẩn	`tf.keras.initializers.GlorotNormal(seed=1)`
Tổng mất mát nhiều lớp	`tf.reduce_sum(tf.keras.losses.categorical_crossentropy(..., from_logits=True))`
Theo dõi độ chính xác	`tf.keras.metrics.CategoricalAccuracy()`

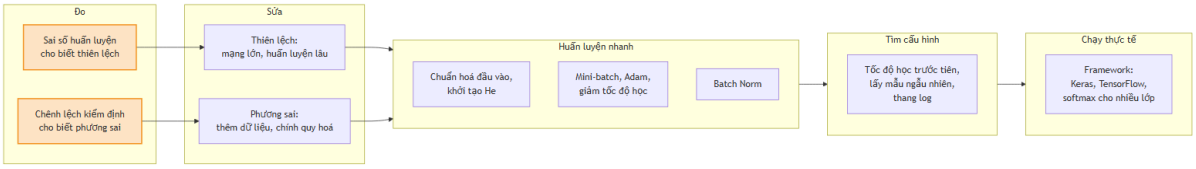
Bảng 3.38: Lệnh dùng trong bài thực hành TensorFlow

3.11.6.4 Tóm tắt cần nhớ

- `tf.data` thay mảng NumPy: `map` biến đổi, `zip` ghép, `batch` gom lô, `prefetch` chuẩn bị sẵn.
- Tham số là `tf.Variable`; chỉ cần viết lan truyền xuôi, `GradientTape` lo độ dốc.
- Lớp cuối trả logits; mất mát nhiều lớp dùng `from_logits=True`.
- Trên ảnh ký hiệu bàn tay sáu lớp, sau 90 epoch chi phí giảm từ 1,83 xuống 0,74 và độ chính xác kiểm tra tăng từ 20% lên 69%.
- Có thể làm gọn toàn bộ bằng Keras `Sequential` và `fit`.

3.11.7 Tóm tắt chương

Chương này đi từ "mạng chạy đúng" đến "mạng chạy tốt". Hình 3.58 gom toàn bộ thành một quy trình: đo trước, sửa theo kết quả đo, làm huấn luyện nhanh và ổn định, tìm cấu hình, rồi chạy bằng framework.



Hình 3.58: Quy trình tổng kết chương: đo, sửa, huấn luyện nhanh, tìm cấu hình, chạy thực tế

Sáu ý đáng mang theo:

1. **Chẩn đoán trước khi sửa.** Chia dữ liệu thành huấn luyện, kiểm định, kiểm tra (kiểm định và kiểm tra cùng phân phối). Sai số huấn luyện cho biết thiên lệch; phần chênh lệch với kiểm định cho biết phương sai. Xử lý thiên lệch trước, rồi phương sai (mục 3.1).

2. **Chính quy hoá chống quá khớp.** L2 và dropout thường đổi một ít độ chính xác huấn luyện lấy độ chính xác kiểm tra; trên bài toán bóng đá của bài thực hành, kiểm tra tăng từ 91,5% lên 93% (L2) và 95% (dropout) (mục 3.2 và 3.4).

3. **Chuẩn hoá đầu vào và khởi tạo đúng.** Khởi tạo toàn 0 không phá được đối xứng (chính xác 50%), khởi tạo quá lớn học chậm (83%), khởi tạo He cho mạng ReLU học tốt (99,3%). Kiểm tra độ dốc xác nhận lan truyền ngược tự viết (mục 3.3).

4. **Tối ưu thông minh hơn gradient descent thuần.** Mini-batch, Momentum, RMSprop và Adam; Adam thường là điểm bắt đầu tốt. Giảm tốc độ học theo khoảng cố định đưa các bộ tối ưu đơn giản lên ngang Adam trong bài thực hành (mục 3.5 và 3.6).

5. **Tinh chỉnh có hệ thống.** Tốc độ học quan trọng nhất; lấy mẫu ngẫu nhiên thay vì lưới; dùng thang log cho tốc độ học và cho một trừ beta; đi từ thô đến mịn. Batch Norm làm mạng bền hơn với lựa chọn siêu tham số (mục 3.7 và 3.8).

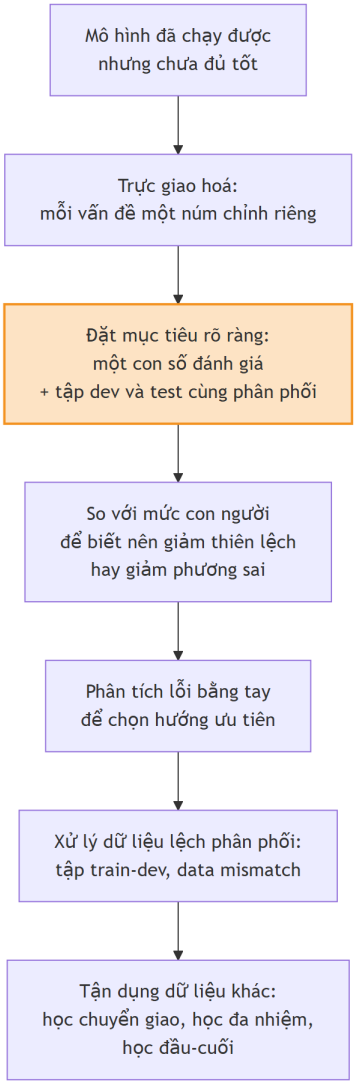
6. **Dùng framework.** Softmax tổng quát hoá hồi quy logistic cho nhiều lớp. TensorFlow chỉ cần viết lan truyền xuôi, `GradientTape` tự lấy độ dốc; trên ảnh ký hiệu bàn tay, chi phí giảm từ 1,83 xuống 0,74 sau 90 epoch (mục 3.9, 3.10 và 3.11).

Các chương sau áp dụng lại đúng những công cụ này; điểm khác là loại lớp (tích chập cho ảnh, hồi quy và chú ý cho chuỗi) và cách tổ chức dự án học máy ở chương kế tiếp.

Chương 4

Tổ chức dự án học máy

Hai chương trước trả lời câu hỏi *làm sao dựng và cải thiện một mạng nơ-ron*. Chương này lùi lại một bước và hỏi câu khác, mang tính chiến lược: **khi mô hình đã chạy nhưng chưa đủ tốt, nên làm gì tiếp theo?** Trước một mô hình đạt 90% độ chính xác, bạn có hàng chục ý tưởng: thu thập thêm dữ liệu, huấn luyện lâu hơn, đổi bộ tối ưu, thêm chính quy hoá, đổi kiến trúc. Chọn sai hướng có thể tốn hàng tháng trời mà chẳng cải thiện được gì. Chương này là bộ công cụ **chiến lược học máy** (machine learning strategy) giúp bạn chẩn đoán đâu là nút thắt và chọn đúng việc để làm (Hình 4.1).



Hình 4.1: Bản đồ chương: trực giao hoá, đặt mục tiêu, so với mức con người, phân tích lỗi, xử lý lệch phân phối, rồi tận dụng dữ liệu khác

Khác với các chương kỹ thuật, phần lớn nội dung ở đây là **cách tư duy và quy trình**, không phải thuật toán mới. Đây cũng chính là loại kinh nghiệm hiếm khi được dạy bài bản nhưng lại quyết định một đội học máy đi nhanh hay chậm. Mỗi mục giải thích ý tưởng bằng ví dụ và sơ đồ, kèm checklist và bảng tra cứu để áp dụng ngay vào dự án.

Mục	Câu hỏi	Nội dung chính
4.1 Trực giao hoá	Chỉnh một thứ mà không làm hỏng thứ khác thế nào?	bốn điều kiện cần đạt, mỗi vấn đề một núm chỉnh riêng
4.2 Đặt mục tiêu bằng chỉ số đánh giá	Làm sao biết nhanh mô hình nào tốt hơn?	một con số duy nhất, chỉ số tối ưu và thoả mãn
4.3 Chia tập kiểm định và kiểm tra	Đặt bia nhắm ở đâu?	cùng phân phối, kích thước, khi nào đổi bia
4.4 So với mức con người	Nên giảm thiên lệch hay phương sai?	sai số Bayes, thiên lệch tránh được, mức con người
4.5 Phân tích lỗi	Hướng nào đáng theo đuổi nhất?	đếm lỗi bằng tay, nhãn sai, dựng nhanh rồi lặp
4.6 Dữ liệu lệch phân phối	Train và test khác phân phối thì sao?	tập train-dev, data mismatch, tổng hợp dữ liệu
4.7 Chuyển giao, đa nhiệm, đầu-cuối	Tận dụng dữ liệu từ bài toán khác thế nào?	transfer learning, multi-task, end-to-end

Bảng 4.1: Các mục của chương Tổ chức dự án học máy

4.1 Trực giao hoá: mỗi vấn đề một núm chỉnh riêng

Một trong những điểm chung của những người làm học máy giỏi là họ rất rõ ràng về việc **chỉnh cái gì để đạt được tác dụng nào**. Ý tưởng này gọi là **trực giao hoá** (orthogonalization).

4.1.1 Ví dụ cái tivi và chiếc xe

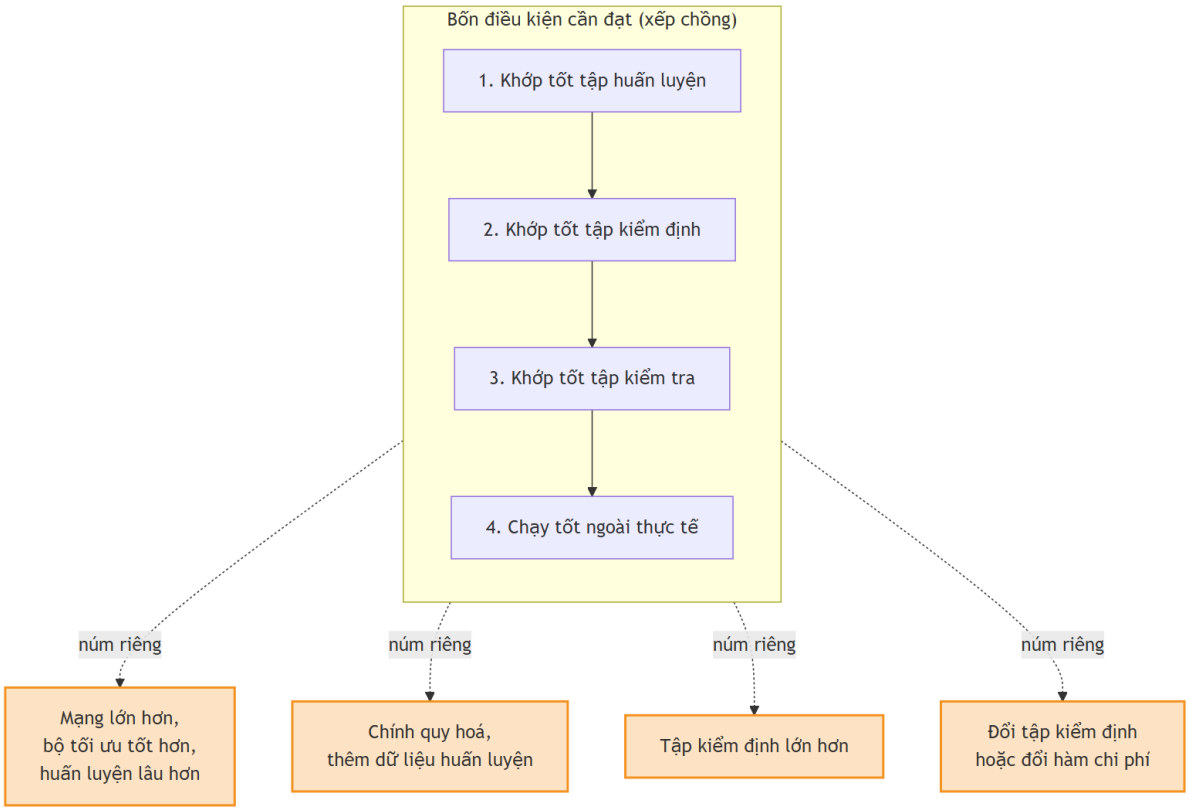
Hãy tưởng tượng một chiếc tivi cũ với nhiều núm vặn: một núm chỉnh chiều cao hình ảnh, một núm chỉnh chiều rộng, một núm chỉnh độ méo hình thang, một núm dịch trái phải. Mỗi núm làm *đúng một việc* nên dễ căn chỉnh. Giờ hình dung một chiếc tivi tai hại hơn: chỉ có một núm, mà vặn nó thì chiều cao, chiều rộng, độ méo và vị trí hình *thay đổi cùng lúc*. Với một núm trộn lẫn mọi thứ như vậy, gần như không thể căn hình vào giữa màn hình.

Lái xe cũng vậy. Một chiếc xe bình thường tách bạch: vô-lăng điều khiển hướng, chân ga và chân phanh điều khiển tốc độ. Nếu ai đó chế ra một cần điều khiển mà một trục trộn lẫn "góc lái cộng tốc độ", về lý thuyết vẫn lái được, nhưng chỉnh để xe đi đúng hướng và đúng tốc độ sẽ khó hơn nhiều.

Trực giao nghĩa là vuông góc, mỗi điều khiển tác động một chiều độc lập. Khi các núm chỉnh được thiết kế trực giao và khớp với đúng thứ bạn muốn điều khiển, việc tinh chỉnh trở nên dễ hơn hẳn.

4.1.2 Bốn điều kiện cần đạt và bốn nhóm núm chỉnh

Để một hệ thống học có giám sát chạy tốt, thường cần bốn điều kiện đạt được *theo thứ tự*, và quan trọng là mỗi điều kiện có một nhóm "núm chỉnh" riêng (Hình 4.2):



Hình 4.2: Bốn điều kiện xếp chồng, mỗi điều kiện có một nhóm núm chỉnh trực giao riêng

1. **Khớp tốt tập huấn luyện** (có khi cần đạt gần mức con người). Nếu chưa đạt: mạng lớn hơn, bộ tối ưu tốt hơn như Adam, huấn luyện lâu hơn.
2. **Khớp tốt tập kiểm định**. Nếu khớp tốt tập huấn luyện nhưng kém trên tập kiểm định: dùng nhóm núm chính quy hoá, hoặc thêm dữ liệu huấn luyện.
3. **Khớp tốt tập kiểm tra**. Nếu tốt trên kiểm định nhưng kém trên kiểm tra: thường là đã khớp quá tay vào tập kiểm định, cần tập kiểm định lớn hơn.
4. **Chạy tốt ngoài thực tế**. Nếu tốt trên tập kiểm tra nhưng người dùng thật không hài lòng: tập kiểm định hoặc hàm chi phí đang đo sai thứ cần đo, hãy đổi tập kiểm định hoặc đổi hàm chi phí.

Điểm cốt lõi là: **chẩn đoán xem đang hỏng ở tầng nào, rồi chỉ dùng đúng nhóm núm của tầng đó**. Đừng dùng một việc để chữa nhiều vấn đề cùng lúc.

4.1.3 Vì sao tác giả ít dùng dừng sớm

Dừng sớm (early stopping) không phải kỹ thuật tồi và nhiều người vẫn dùng. Nhưng nó là một núm *kém trực giao*: dừng sớm vừa làm mô hình khớp tập huấn luyện kém đi, vừa nhằm cải thiện tập kiểm định, nghĩa là tác động hai thứ cùng lúc, giống cái núm chỉnh cả chiều cao lẫn chiều rộng tivi. Vì khó suy nghĩ rạch ròi theo kiểu đó, nhiều người thích các núm trực giao hơn: chỉnh thiên lệch bằng mạng lớn hơn, chỉnh phương sai bằng chính quy hoá, mỗi thứ một đường.

4.1.4 Áp dụng vào dự án thực tế

4.1.4.1 Danh sách việc cần làm

1. Khi mô hình chưa đủ tốt, hỏi lần lượt: hỏng ở tập huấn luyện, tập kiểm định, tập kiểm tra, hay ngoài thực tế?
2. Xác định đúng tầng đang hỏng, rồi chỉ chạm vào nhóm núm của tầng đó.
3. Ưu tiên các núm trực giao (mỗi núm một tác dụng) hơn các núm trộn lẫn như dừng sớm.

4.1.4.2 Bảng tra cứu: triệu chứng và núm chỉnh

Hỏng ở đâu	Nhóm núm nên dùng
Tập huấn luyện	mạng lớn hơn, bộ tối ưu tốt hơn (Adam), huấn luyện lâu hơn
Tập kiểm định	chính quy hoá (L2, dropout), thêm dữ liệu huấn luyện
Tập kiểm tra	tập kiểm định lớn hơn
Ngoài thực tế	đổi tập kiểm định cho sát thực tế, hoặc đổi hàm chi phí

Bảng 4.2: Mỗi tầng hỏng có một nhóm núm chỉnh trực giao riêng

4.1.4.3 Tóm tắt cần nhớ

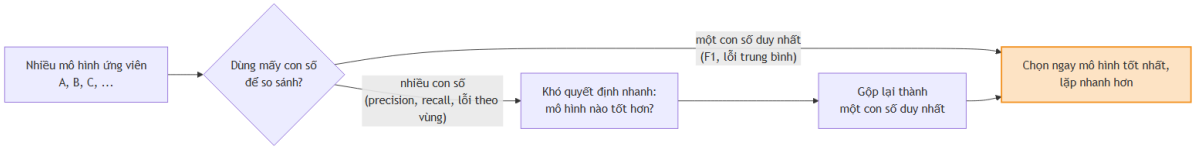
- Trực giao hoá: mỗi vấn đề có một núm (hoặc một nhóm núm) chỉnh riêng, không trộn lẫn.
- Bốn điều kiện xếp chồng: khớp tập huấn luyện, khớp tập kiểm định, khớp tập kiểm tra, chạy tốt thực tế.
- Chẩn đoán đúng tầng hỏng rồi mới chọn núm; đừng chữa nhầm chỗ.

4.2 Đặt mục tiêu bằng một chỉ số đánh giá

Dù bạn đang tinh chỉnh siêu tham số hay thử các ý tưởng khác nhau, tiến độ sẽ nhanh hơn hẳn nếu có **một con số đánh giá duy nhất** để biết ngay ý tưởng mới tốt hơn hay tệ hơn ý tưởng cũ.

4.2.1 Một con số thay vì nhiều con số

Học máy ứng dụng là một vòng lặp thực nghiệm: nảy ý tưởng, viết code, chạy thí nghiệm, đọc kết quả, rồi tinh chỉnh ý tưởng. Nếu mỗi vòng bạn phải so nhiều con số cùng lúc, việc ra quyết định chậm hẳn (Hình 4.3).



Hình 4.3: Gộp nhiều con số thành một con số duy nhất giúp chọn mô hình và lặp nhanh hơn

Ví dụ với bộ phân loại mèo, hai chỉ số thường dùng là **độ chuẩn xác** (precision) và **độ phủ** (recall):

- **Độ chuẩn xác**: trong số các ảnh mô hình gọi là mèo, bao nhiêu phần trăm thật sự là mèo.
- **Độ phủ**: trong số các ảnh thật sự là mèo, mô hình bắt đúng được bao nhiêu phần trăm.

Hai chỉ số này thường đánh đổi nhau: mô hình A có thể cao độ phủ còn mô hình B cao độ chuẩn xác, và bạn không biết chọn ai. Cách giải quyết chuẩn trong học máy là gộp hai chỉ số thành **điểm F1**. Về trực giác, điểm F1 là một kiểu "trung bình" của độ chuẩn xác và độ phủ, nhưng là *trung bình điều hoà* (ưu ái sự cân bằng: nếu một trong hai thấp thì điểm F1 cũng thấp) thay vì trung bình cộng. Có điểm F1, bạn so hai mô hình chỉ bằng một con số và chọn ngay được.

Tương tự, nếu bạn theo dõi lỗi của bộ phân loại ở bốn vùng địa lý (Mỹ, Trung Quốc, Ấn Độ, phần còn lại), đừng nhìn bốn con số rời rạc, hãy tính thêm **lỗi trung bình** trên các vùng và dùng nó làm con số duy nhất để chọn mô hình.

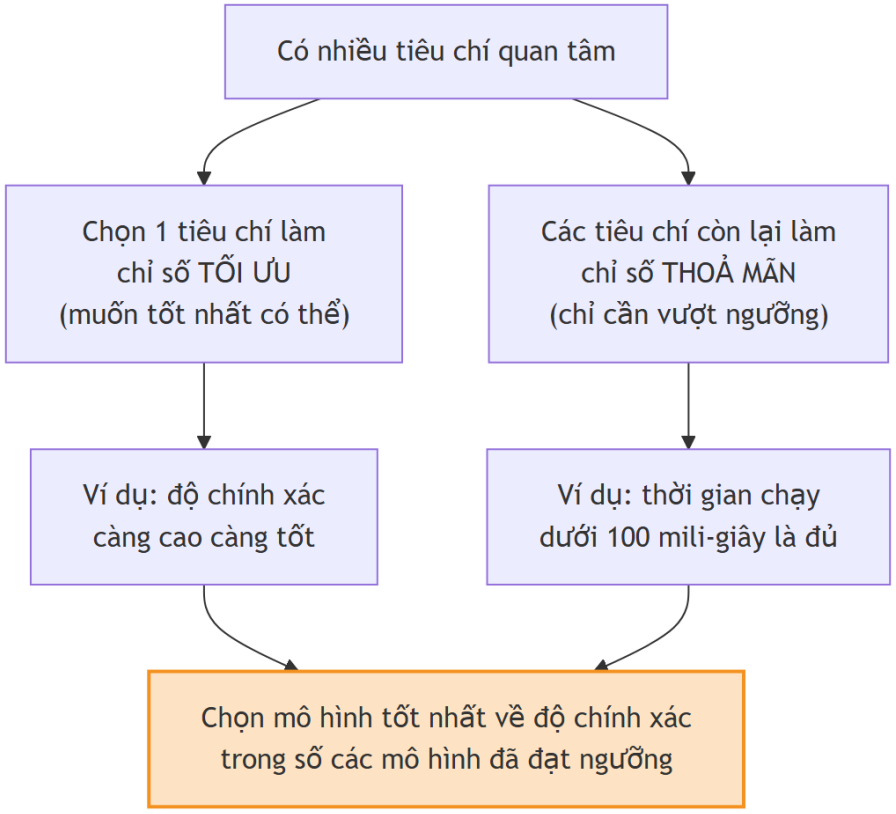
Tính nhanh bằng scikit-learn. Thư viện có sẵn các hàm cho precision, recall và F1:

```
from sklearn.metrics import precision_score, recall_score, f1_score

p = precision_score(y_true, y_pred)   # độ chuẩn xác
r = recall_score(y_true, y_pred)      # độ phủ
f1 = f1_score(y_true, y_pred)         # một con số duy nhất để so sánh
print(p, r, f1)
```

4.2.2 Chỉ số tối ưu và chỉ số thoả mãn

Không phải lúc nào cũng gộp được mọi thứ bạn quan tâm vào một con số. Ví dụ bạn quan tâm cả độ chính xác lẫn thời gian chạy. Gộp chúng bằng một công thức cộng có trọng số nghe hơi gượng ép. Cách tự nhiên hơn: chọn một chỉ số làm **chỉ số tối ưu** (optimizing metric) để làm tốt nhất có thể, và các chỉ số còn lại làm **chỉ số thoả mãn** (satisficing metric) chỉ cần vượt một ngưỡng là đủ (Hình 4.4).



Hình 4.4: Một chỉ số tối ưu để tối đa hoá, các chỉ số còn lại chỉ cần đạt ngưỡng

Ví dụ: *tối đa hoá độ chính xác, miễn là thời gian phân loại một ảnh dưới 100 mili-giây*. Khi đó độ chính xác là chỉ số tối ưu, thời gian chạy là chỉ số thoả mãn. Trong ba mô hình chạy lần lượt 80, 95 và 1 500 mili-giây, ta loại mô hình 1 500 mili-giây và chọn mô hình có độ chính xác cao nhất trong hai mô hình còn lại.

Ví dụ khác với hệ phát hiện từ khoá đánh thức (như "Alexa", "OK Google"): tối đa hoá độ chính xác bắt đúng từ khoá, với ràng buộc nhiều nhất một lần báo nhầm trong mỗi 24 giờ. Tổng quát: nếu có nhiều tiêu chí, chọn một làm tối ưu và các tiêu chí còn lại làm thoả mãn, bạn sẽ có cách gần như tự động để chọn nhanh mô hình tốt nhất.

4.2.3 Áp dụng vào dự án thực tế

4.2.3.1 Danh sách việc cần làm

1. Chọn một con số duy nhất để so sánh mô hình; nếu đang có nhiều con số, gộp lại (ví dụ F1, lỗi trung bình).
2. Nếu không gộp được, tách thành một chỉ số tối ưu và các chỉ số thoả mãn kèm ngưỡng rõ ràng.
3. Đặt chỉ số này lên tập kiểm định để mỗi vòng lặp đều biết ngay mô hình nào tốt hơn.

4.2.3.2 Bảng tra cứu

Tình huống	Cách đặt chỉ số
Quan tâm độ chuẩn xác và độ phủ	gộp thành điểm F1
Quan tâm lỗi ở nhiều vùng/thị trường	lấy lỗi trung bình (có thể có trọng số)
Quan tâm độ chính xác và thời gian chạy	chính xác là tối ưu, thời gian là thoả mãn (ngưỡng)
Quan tâm độ chính xác và tỷ lệ báo nhầm	chính xác là tối ưu, số lần báo nhầm là thoả mãn

Bảng 4.3: Chọn chỉ số đánh giá theo thứ bạn quan tâm

4.2.3.3 Tóm tắt cần nhớ

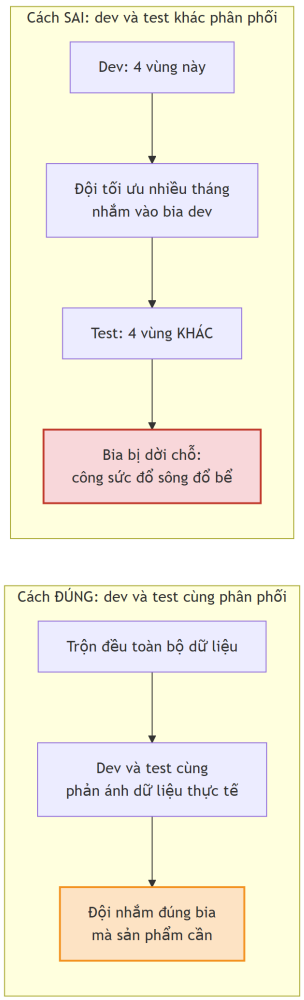
- Một con số đánh giá duy nhất giúp đội quyết định nhanh và lặp nhanh.
- Khi nhiều tiêu chí, tách một chỉ số tối ưu và các chỉ số thoả mãn (chỉ cần đạt ngưỡng).
- Chỉ số phải tính được trên tập kiểm định để dùng ngay trong vòng lặp thực nghiệm.

4.3 Chia tập kiểm định và tập kiểm tra

Cách bạn chia tập kiểm định (dev set) và tập kiểm tra (test set) ảnh hưởng rất lớn đến tốc độ tiến bộ của cả đội. Có những đội, kể cả ở công ty lớn, chia dữ liệu theo cách tự làm chậm chính mình.

4.3.1 Dev và test phải cùng phân phối

Hãy hình dung tập kiểm định cùng chỉ số đánh giá như một tấm bia: nó cho cả đội biết đích cần nhắm. Đội học máy rất giỏi bắn mũi tên ngày càng gần tâm bia. Nhưng nếu tập kiểm định và tập kiểm tra đến từ *hai phân phối khác nhau*, bạn giống như để đội nhắm vào một bia suốt nhiều tháng rồi đến lúc kiểm tra lại dời bia sang chỗ khác (Hình 4.5).



Hình 4.5: Dev và test khác phân phối giống dời bia sau nhiều tháng; cùng phân phối mới là nhắm đúng đích

Ví dụ có thật (đã đổi vài chi tiết): một đội tối ưu nhiều tháng trên tập kiểm định gồm hồ sơ vay của khu vực *thu nhập trung bình*, rồi mới đem kiểm tra trên dữ liệu khu vực *thu nhập thấp*. Hai phân phối khác hẳn nhau, mô hình họ dày công tối ưu lại chạy kém, và đội mất khoảng ba tháng phải làm lại.

Quy tắc: **chọn tập kiểm định và tập kiểm tra phản ánh dữ liệu bạn kỳ vọng gặp trong tương lai và muốn làm tốt trên đó, và hai tập này phải cùng một phân phối**. Dù dữ liệu tương lai là gì, hãy đưa loại dữ liệu đó vào cả hai tập.

4.3.2 Kích thước tập kiểm định và kiểm tra

Quy tắc cũ 70%/30% hay 60%/20%/20% hợp lý khi dữ liệu còn nhỏ (vài trăm đến vài chục nghìn mẫu). Trong kỷ nguyên học sâu với dữ liệu rất lớn thì khác: với một triệu mẫu, chỉ cần 1% (tức 10 000 mẫu) cho mỗi tập kiểm định và kiểm tra đã quá đủ, tức tỷ lệ 98%/1%/1%.

Nguyên tắc kích thước:

- **Tập kiểm định** chỉ cần đủ lớn để so sánh các mô hình và nhanh chóng biết cái nào tốt hơn.
- **Tập kiểm tra** chỉ cần đủ lớn để cho ước lượng tin cậy về chất lượng của hệ thống cuối cùng.

Khi nào được bỏ tập kiểm tra. Nếu bạn không cần ước lượng không thiên lệch về chất lượng cuối, có thể chỉ giữ tập huấn luyện và tập kiểm định. Nhưng hãy cẩn thận thuật ngữ: nhiều đội gọi tập kiểm

định là "tập kiểm tra" trong khi họ vẫn đang tinh chỉnh trên nó. Nếu thật sự đang tinh chỉnh trên một tập, nó là tập kiểm định, không phải tập kiểm tra. Tác giả vẫn khuyên nên có tập kiểm tra riêng để yên tâm trước khi triển khai.

4.3.3 Khi nào nên đổi bia (chỉ số hoặc tập dữ liệu)

Đôi khi giữa chừng dự án bạn phát hiện mình đặt bia sai chỗ. Dấu hiệu: **chỉ số đánh giá không còn xếp hạng đúng thứ tự ưu tiên giữa các mô hình**.

Ví dụ bộ phân loại mèo dùng chỉ số lỗi phân loại: mô hình A có 3% lỗi, mô hình B có 5% lỗi, nên theo chỉ số thì A tốt hơn. Nhưng A lại để lọt nhiều ảnh phản cảm, điều không thể chấp nhận với người dùng và công ty. Rõ ràng B mới là mô hình tốt hơn trong thực tế. Khi chỉ số nói ngược với cái bạn thật sự muốn, đó là lúc đổi chỉ số.

Cách sửa trong ví dụ này: thêm **trọng số** vào lỗi, cho ảnh phản cảm nặng hơn nhiều (ví dụ gấp 10 hay 100 lần ảnh thường), để mỗi lần phân loại nhầm ảnh phản cảm bị phạt rất nặng. Chi tiết trọng số không quan trọng bằng ý tưởng: chỉ số phải nói đúng cho bạn biết mô hình nào tốt hơn cho ứng dụng của bạn.

Đây lại là một ví dụ của trực giao hoá, tách bài toán thành hai bước riêng:

1. **Đặt bia**: định nghĩa chỉ số đánh giá nắm bắt đúng thứ bạn muốn.
2. **Bắn vào bia**: tìm cách làm tốt theo chỉ số đó (ví dụ sửa hàm chi phí mà mạng tối ưu).

Trường hợp khác cần đổi bia: tập kiểm định/kiểm tra toàn ảnh chất lượng cao tải từ Internet, nhưng người dùng thật tải lên ảnh mờ, chụp vội. Nếu làm tốt trên tập hiện tại mà không tương ứng với làm tốt trên dữ liệu thật, hãy đổi tập dữ liệu cho sát thực tế.

Lời khuyên thực dụng: dù chưa có chỉ số và tập dev hoàn hảo, cứ **dựng một cái gì đó nhanh** rồi dùng để chạy, phát hiện chưa ổn thì đổi sau. Đừng để đội chạy quá lâu mà không có chỉ số và tập dev nào.

4.3.4 Áp dụng vào dự án thực tế

4.3.4.1 Danh sách việc cần làm

1. Lấy tập kiểm định và kiểm tra từ *cùng* phân phối, và sát với dữ liệu bạn sẽ gặp thật.
2. Chọn kích thước theo quy mô dữ liệu: dữ liệu rất lớn thì dev/test chỉ cần một phần nhỏ.
3. Trộn và chia ngẫu nhiên để dev/test cùng phản ánh dữ liệu thực tế.
4. Nếu chỉ số xếp hạng sai mô hình, hoặc tốt trên dev mà kém thực tế, hãy đổi chỉ số hoặc đổi tập dev/test.

Trộn và chia bằng scikit-learn.

```
from sklearn.model_selection import train_test_split

# Trộn và chia 98% / 1% / 1% để dev và test cùng phân phối với nhau
X_train, X_rest, y_train, y_rest = train_test_split(
    X, y, test_size=0.02, random_state=0, shuffle=True)
X_dev, X_test, y_dev, y_test = train_test_split(
    X_rest, y_rest, test_size=0.5, random_state=0, shuffle=True)
```

4.3.4.2 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Tốt nhiều tháng trên dev, đến test lại tệ	dev và test khác phân phối; lấy cả hai từ cùng nguồn sát thực tế
Chỉ số nói A tốt hơn B nhưng người dùng thích B	chỉ số không nắm bắt đúng cái cần; đổi chỉ số (ví dụ thêm trọng số)
Tốt trên dev/test nhưng kém khi triển khai	dev/test không giống dữ liệu người dùng thật; đổi tập dev/test
Tốt trên dev mà kém trên test	khớp quá tay vào dev; lấy tập dev lớn hơn

Bảng 4.4: Lỗi thường gặp khi chia dev/test và đặt chỉ số

4.3.4.3 Tóm tắt cần nhớ

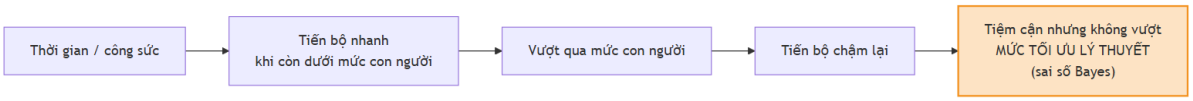
- Tập kiểm định cùng chỉ số là tấm bia định hướng cho cả đội; dev và test phải cùng phân phối.
- Dữ liệu càng lớn, phần dành cho dev/test càng nhỏ; dev đủ để so sánh, test đủ để đo tin cậy.
- Khi chỉ số xếp hạng sai hoặc không khớp thực tế, hãy đổi bia; tách việc "đặt bia" và "bắn vào bia".

4.4 So sánh với mức con người

Ngày càng nhiều đội học máy so hệ thống của mình với **mức con người** (human-level performance). Lý do: nhờ học sâu, thuật toán đã đủ tốt để cạnh tranh với con người ở nhiều lĩnh vực; và quy trình xây dựng hệ thống hiệu quả hơn hẳn khi bài toán là thứ con người cũng làm được.

4.4.1 Sai số Bayes và vì sao tiến bộ chậm lại

Khi một đội làm một bài toán theo thời gian, độ chính xác thường tăng nhanh cho tới khi *vượt mức con người*, sau đó chậm lại và tiệm cận một giới hạn lý thuyết gọi là **sai số Bayes** (Bayes optimal error), mức tốt nhất mà bất kỳ ánh xạ nào từ đầu vào tới đầu ra có thể đạt được (Hình 4.6).

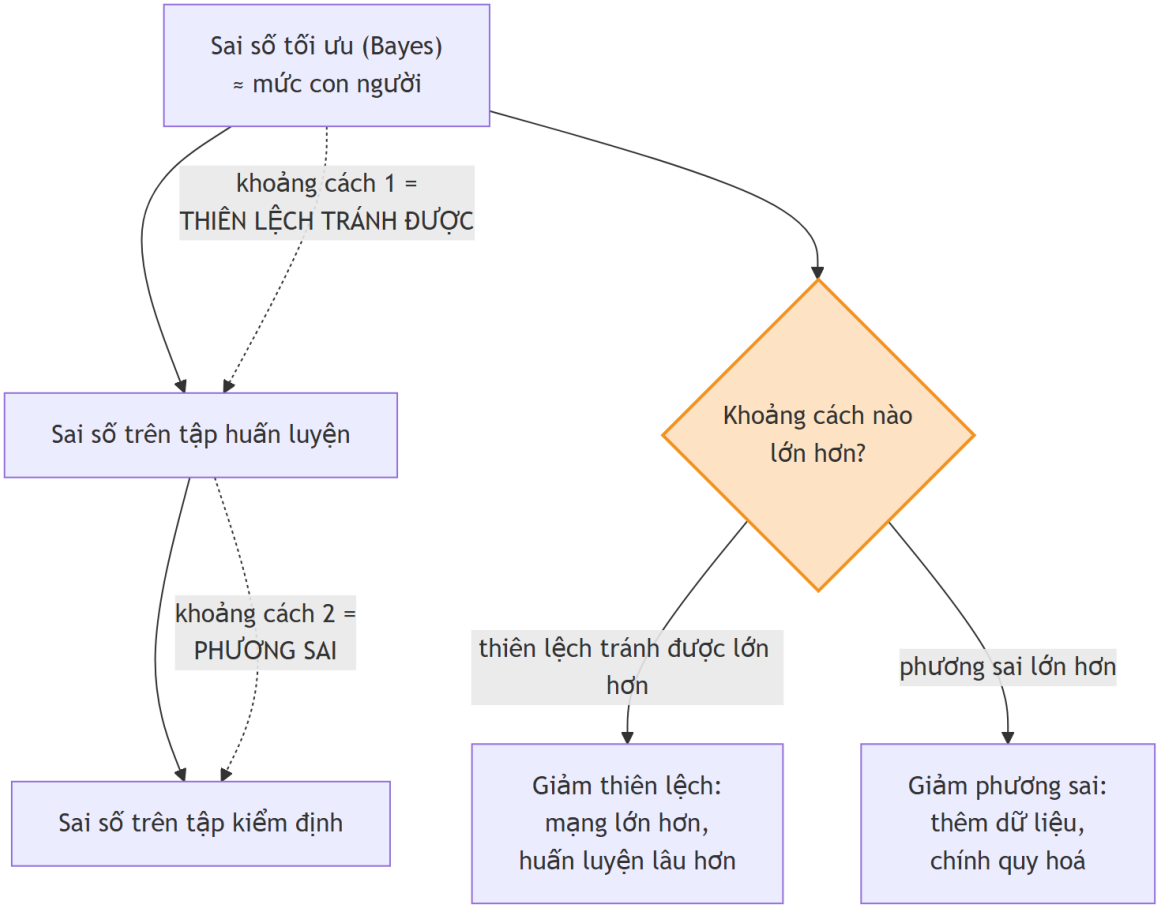


Hình 4.6: Tiến bộ nhanh dưới mức con người, chậm lại sau khi vượt, và không bao giờ vượt sai số Bayes

Sai số Bayes thường khác 0: có đoạn âm thanh quá nhiễu không ai nghe ra, có ảnh quá mờ không ai nói được có mèo hay không. Có hai lý do tiến bộ chậm lại khi vượt mức con người. Thứ nhất, mức con người ở nhiều bài toán vốn đã rất gần sai số Bayes, nên còn ít dư địa. Thứ hai, khi còn kém con người, bạn có nhiều công cụ dễ dùng mà vượt rồi thì khó: lấy nhãn từ con người, phân tích lỗi dựa vào trực giác con người, và phân tích thiên lệch/phương sai dựa vào mức con người làm mốc.

4.4.2 Mức con người là ước lượng tốt cho sai số Bayes

Với các bài toán *tri giác tự nhiên* (nhìn ảnh, nghe tiếng, đọc chữ) con người rất giỏi, nên mức con người là một **ước lượng tốt cho sai số Bayes**. Điều này giúp ta định nghĩa hai khái niệm quan trọng (Hình 4.7):



Hình 4.7: Khoảng cách tới mức con người là thiên lệch tránh được; khoảng cách train tới dev là phương sai

- **Thiên lệch tránh được** (avoidable bias): khoảng cách giữa sai số tập huấn luyện và mức con người (xem như sai số Bayes). Bạn muốn thu hẹp nó, nhưng không nên cố vượt sai số Bayes vì chỉ đạt được bằng cách khớp quá tay.
- **Phương sai** (variance): khoảng cách giữa sai số tập huấn luyện và sai số tập kiểm định.

Cùng một cặp số (ví dụ 8% lỗi huấn luyện, 10% lỗi kiểm định) lại dẫn tới quyết định khác nhau tuỳ mức con người:

Mức con người	Lỗi huấn luyện	Lỗi kiểm định	Chẩn đoán và hướng đi
1%	8%	10%	thiên lệch tránh được lớn (7%); tập trung giảm thiên lệch (mạng lớn hơn, huấn luyện lâu hơn)
7,5%	8%	10%	thiên lệch tránh được nhỏ (0,5%), phương sai lớn (2%); tập trung giảm phương sai (chính quy hoá, thêm dữ liệu)

Bảng 4.5: Cùng số liệu nhưng mức con người khác nhau dẫn tới hướng đi khác nhau

4.4.3 Định nghĩa mức con người cho đúng mục đích

Trong một ví dụ chẩn đoán ảnh y khoa: người thường sai 3%, bác sĩ thường sai 1%, bác sĩ giàu kinh nghiệm sai 0,7%, một nhóm bác sĩ giàu kinh nghiệm hội chẩn sai 0,5%. Vậy "mức con người" là bao nhiêu? Câu trả lời phụ thuộc mục đích:

- Nếu dùng mức con người làm **ước lượng cho sai số Bayes** (để phân tích thiên lệch/phương sai), hãy lấy mức tốt nhất khả thi: 0,5% (nhóm bác sĩ hội chẩn). Vì có hệ thống đạt được 0,5%, sai số Bayes chắc chắn không cao hơn 0,5%.
- Nếu mục đích là **chứng minh vượt một con người** để triển khai, có thể chỉ cần vượt một bác sĩ đơn lẻ.

Việc chọn đúng mốc đặc biệt quan trọng khi bạn đã làm rất tốt. Nếu lỗi huấn luyện 0,7% và lỗi kiểm định 0,8%, mà bạn lấy sai số Bayes là 0,5%, thì thiên lệch tránh được là 0,2% còn phương sai chỉ 0,1%, gợi ý nên tập trung giảm thiên lệch. Nếu lỡ lấy mốc 0,7%, bạn sẽ tưởng thiên lệch gần 0 và bỏ lỡ hướng đi đúng.

4.4.4 Khi đã vượt mức con người

Khi hệ thống vượt cả một nhóm con người, việc chẩn đoán khó hơn: bạn không còn mốc tốt cho sai số Bayes, và cũng khó dựa vào trực giác con người để biết cải thiện tiếp ra sao. Nhiều bài toán máy đã vượt xa con người, đáng chú ý là chúng thường học từ **dữ liệu có cấu trúc** với lượng dữ liệu khổng lồ, không phải tri giác tự nhiên: quảng cáo trực tuyến, gợi ý sản phẩm, dự đoán thời gian giao hàng, duyệt khoản vay. Với tri giác tự nhiên (thị giác, giọng nói) thì khó vượt hơn vì con người vốn rất giỏi, nhưng học sâu cũng đã vượt ở một số tác vụ hẹp.

4.4.5 Gom lại thành quy trình cải thiện

Làm một hệ thống học có giám sát chạy tốt về cơ bản là đạt hai điều: khớp tốt tập huấn luyện (thiên lệch tránh được thấp) và tổng quát hoá tốt sang tập kiểm định (phương sai thấp). Theo tinh thần trực giao hoá, mỗi điều có một nhóm công cụ riêng.

Vấn đề	Công cụ nên thử
Thiên lệch tránh được lớn	mạng lớn hơn, huấn luyện lâu hơn, bộ tối ưu tốt hơn (Momentum, RMSprop, Adam), đổi kiến trúc/siêu tham số
Phương sai lớn	thêm dữ liệu, chính quy hoá (L2, dropout, tăng cường dữ liệu), đổi kiến trúc/siêu tham số

Bảng 4.6: Chọn công cụ theo chẩn đoán thiên lệch tránh được và phương sai

4.4.5.1 Danh sách việc cần làm

1. Với bài toán con người làm tốt, lấy mức con người làm ước lượng cho sai số Bayes.
2. Tính thiên lệch tránh được (train trừ mức con người) và phương sai (dev trừ train).
3. So hai khoảng cách: cái nào lớn hơn thì tập trung vào đó.
4. Khi đã vượt mức con người, cẩn thận hơn vì không còn mốc Bayes tốt.

4.4.5.2 Tóm tắt cần nhớ

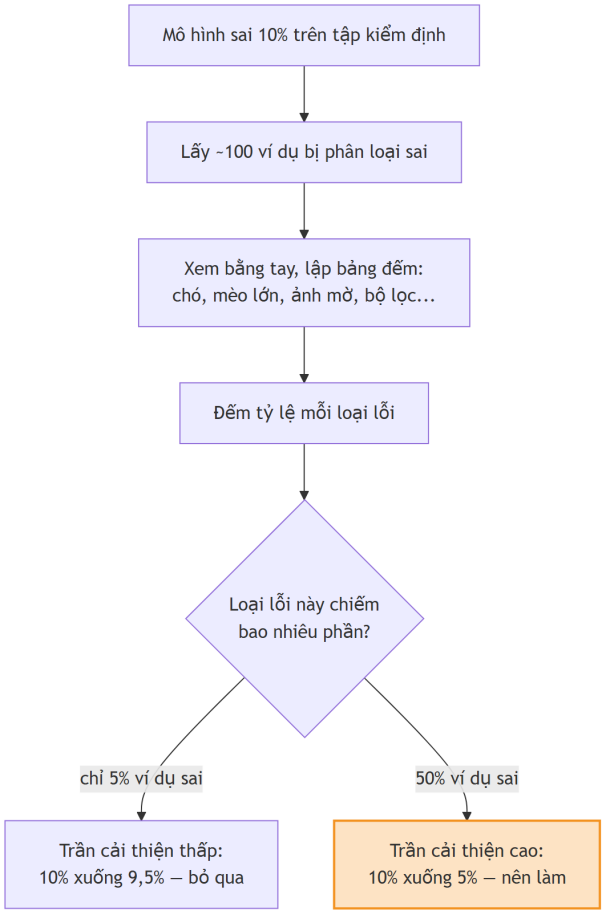
- Mức con người là ước lượng tốt cho sai số Bayes ở các bài toán tri giác tự nhiên.
- Thiên lệch tránh được = train trừ mức con người; phương sai = dev trừ train.
- Cùng một cặp số nhưng mức con người khác nhau sẽ dẫn tới hướng cải thiện khác nhau.
- Vượt mức con người thì tiến bộ chậm lại và khó chẩn đoán hơn.

4.5 Phân tích lỗi

Khi mô hình chưa đạt mức con người, việc **xem bằng tay** những lỗi mà mô hình mắc phải thường cho bạn gợi ý rõ ràng về việc nên làm tiếp. Quy trình này gọi là **phân tích lỗi** (error analysis).

4.5.1 Đếm lỗi để ước lượng trần cải thiện

Giả sử bộ phân loại mèo đạt 90% độ chính xác (tức 10% lỗi) và bạn thấy chưa đủ. Một đồng nghiệp nhận ra mô hình đôi khi nhầm chó thành mèo và đề xuất bỏ vài tháng xử lý riêng chuyện chó. Có đáng không? Thay vì lao vào, hãy làm một bước nhanh (Hình 4.8):



Hình 4.8: Lấy khoảng 100 ví dụ sai, đếm tỷ lệ từng loại để ước lượng trần cải thiện của mỗi hướng

Lấy khoảng 100 ví dụ bị phân loại sai trong tập kiểm định, đếm xem bao nhiêu là chó. Nếu chỉ 5% là chó, thì dù giải quyết hoàn hảo chuyện chó, lỗi cũng chỉ giảm từ 10% xuống 9,5% — trần cải thiện quá thấp, nên bỏ qua. Nếu 50% là chó, giải quyết tốt sẽ đưa lỗi từ 10% xuống còn 5% — rất đáng làm. Việc đếm này chỉ tốn 5–10 phút nhưng giúp bạn khỏi lãng phí hàng tháng.

4.5.2 Đánh giá nhiều ý tưởng song song

Thường bạn có nhiều ý tưởng cùng lúc: cải thiện chuyện chó, chuyện mèo lớn (sư tử, báo bị nhầm là mèo nhà), chuyện ảnh mờ. Hãy lập một bảng tính, mỗi hàng là một ảnh, mỗi cột là một loại lỗi, kèm cột ghi chú:

Ảnh	Chó	Mèo lớn	Ảnh mờ	Ghi chú
1	x			chó pit bull
2			x	ảnh chụp thiếu sáng
3		x	x	sư tử trong mưa ở sở thú
…				
Tỷ lệ	8%	43%	61%	(một ảnh có thể dính nhiều loại)

Bảng 4.7: Bảng phân tích lỗi: đếm tỷ lệ mỗi loại để biết hướng nào có trần cải thiện cao nhất

Giữa chừng nếu phát hiện loại lỗi mới (ví dụ ảnh bị bộ lọc Instagram làm hỏng), cứ thêm cột rồi đếm tiếp. Bảng này không cho bạn công thức cứng, nhưng cho biết hướng nào đáng ưu tiên: ở ví dụ trên, ảnh mờ và mèo lớn có trần cao hơn hẳn chuyện chó.

4.5.3 Dữ liệu bị gán nhãn sai

Nếu phát hiện một số ví dụ bị *gán nhãn sai* (nhãn do con người gán sai, khác với mô hình đoán sai), xử lý thế nào?

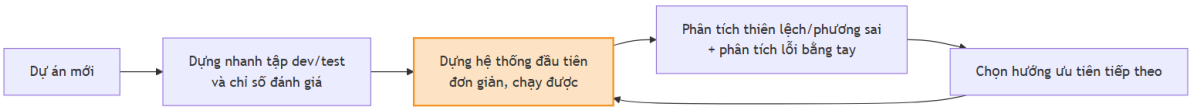
- **Trong tập huấn luyện**: học sâu khá chịu được *lỗi ngẫu nhiên*. Nếu nhãn sai mang tính ngẫu nhiên và tỷ lệ không quá cao, thường cứ để nguyên cũng ổn. Nhưng học sâu *kém chịu lỗi hệ thống* — ví dụ người gán nhãn luôn gọi chó trắng là mèo, mô hình sẽ học sai theo.
- **Trong tập kiểm định/kiểm tra**: thêm một cột "nhãn sai" vào bảng phân tích lỗi để đếm. Có đáng sửa hay không tuỳ ba con số: tổng lỗi trên dev, phần lỗi do nhãn sai, và phần lỗi do nguyên nhân khác.

So sánh hai tình huống: nếu tổng lỗi 10% và lỗi do nhãn sai chỉ chiếm 0,6% (phần còn lại 9,4% do nguyên nhân khác), thì sửa nhãn chưa phải việc gấp. Nhưng nếu bạn đã đưa tổng lỗi xuống 2% mà lỗi do nhãn sai vẫn 0,6% (tức 30% tổng lỗi), thì nhãn sai đang che mờ việc so sánh mô hình — lúc này nên sửa.

Vài nguyên tắc khi sửa nhãn: áp dụng *cùng quy trình* cho cả tập dev và test để chúng vẫn cùng phân phối; nên xem cả những ví dụ mô hình *đoán đúng* chứ không chỉ đoán sai (dù việc này khó hơn khi mô hình đã rất chính xác); và không nhất thiết phải sửa tập huấn luyện vì nó lớn và học sâu chịu được lỗi ngẫu nhiên.

4.5.4 Dựng nhanh rồi lặp

Với một bài toán hoàn toàn mới, lời khuyên là **dựng hệ thống đầu tiên thật nhanh rồi lặp** (Hình 4.9):



Hình 4.9: Dựng nhanh tập dev/test và chỉ số, dựng hệ thống đơn giản, rồi phân tích để chọn hướng lặp tiếp

Một hệ nhận dạng giọng nói có thể cải thiện theo hàng chục hướng (chống nhiễu nền, giọng vùng miền, người nói xa micro, giọng trẻ em...). Khó biết trước nên ưu tiên gì. Vì vậy: dựng nhanh tập dev/test và chỉ số, dựng một hệ thống ban đầu đơn giản dù còn thô, rồi dùng phân tích thiên lệch/phương sai và phân tích lỗi để quyết hướng đi tiếp. Lời khuyên này nhẹ hơn nếu bạn đã có nhiều kinh nghiệm trong lĩnh vực đó, hoặc có sẵn kho tài liệu học thuật cho đúng bài toán đó. Trên thực tế, có nhiều đội mắc lỗi nghĩ quá phức tạp hơn là làm quá đơn giản.

4.5.5 Áp dụng vào dự án thực tế

4.5.5.1 Danh sách việc cần làm

1. Lấy khoảng 100 ví dụ sai trong tập kiểm định, xem và phân loại bằng tay.
2. Lập bảng đếm tỷ lệ từng loại lỗi; thêm cột khi phát hiện loại mới và cột "nhãn sai".
3. Ưu tiên loại lỗi có trần cải thiện cao nhất.
4. Với bài toán mới: dựng hệ thống đầu tiên nhanh, rồi dùng phân tích lỗi để lặp.

4.5.5.2 Tóm tắt cần nhớ

- Đếm lỗi bằng tay (5–10 phút) cho bạn trần cải thiện của mỗi hướng, tránh lãng phí nhiều tháng.
- Dùng bảng để đánh giá nhiều ý tưởng song song và phát hiện loại lỗi mới.
- Lỗi ngẫu nhiên trong nhãn huấn luyện thường chấp nhận được; lỗi hệ thống thì không.
- Sửa nhãn sai ở dev/test khi nó che mờ việc so sánh mô hình; dựng nhanh rồi lặp cho dự án mới.

4.6 Khi dữ liệu huấn luyện lệch phân phối với dev/test

Học sâu rất "khát" dữ liệu, nên nhiều đội gom mọi dữ liệu kiếm được nhét vào tập huấn luyện, kể cả khi phần lớn không cùng phân phối với dev/test. Mục này bàn cách chia dữ liệu và chẩn đoán trong tình huống đó.

4.6.1 Chia dữ liệu khi hai nguồn khác nhau

Ví dụ ứng dụng nhận mèo: bạn có 10 000 ảnh người dùng tải lên (mờ, chụp vội — thứ bạn thật sự quan tâm) và 200 000 ảnh chất lượng cao cào từ web (khác phân phối). Có hai cách chia:

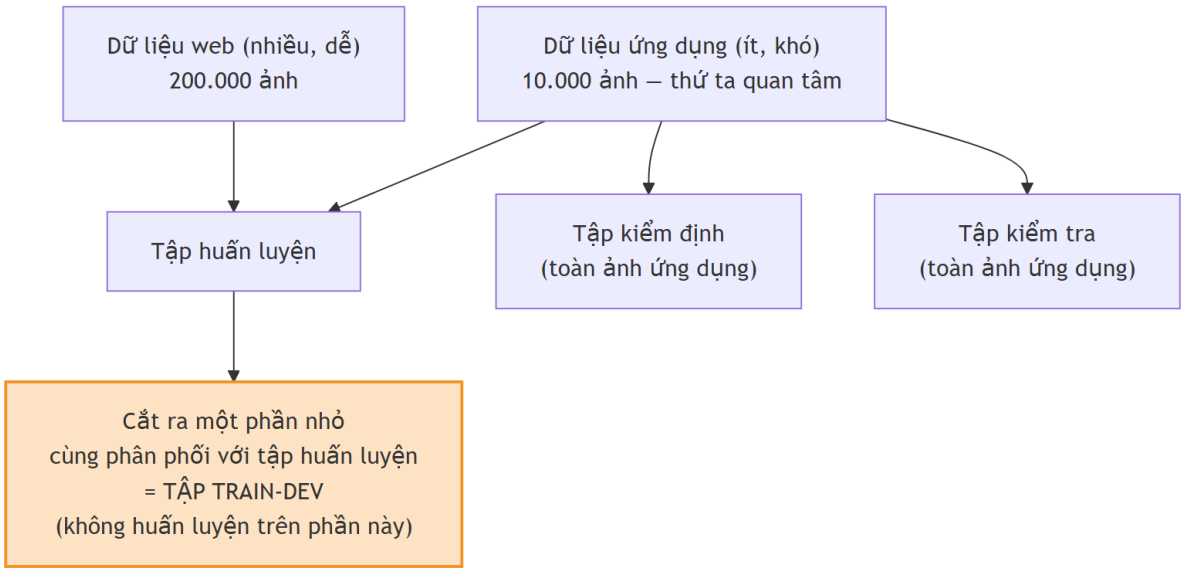
- **Cách sai**: trộn chung 210 000 ảnh rồi chia ngẫu nhiên. Ưu điểm là train/dev/test cùng phân phối, nhưng tập dev khi đó gần như toàn ảnh web — tức bạn đang bảo đội nhắm vào phân phối web, không phải phân phối ứng dụng mà bạn quan tâm.
- **Cách nên dùng**: tập huấn luyện gồm 200 000 ảnh web cộng một phần ảnh ứng dụng; còn *toàn bộ* tập dev và test là ảnh ứng dụng. Nhược điểm là train khác phân phối với dev/test, nhưng bạn đã nhắm đúng bia mà sản phẩm cần, và về lâu dài cho kết quả tốt hơn.

Nguyên tắc giữ nguyên: đặt dev/test ở đúng phân phối bạn muốn làm tốt, còn tập huấn luyện được phép khác để có thêm dữ liệu.

4.6.2 Tập train-dev để chẩn đoán đúng

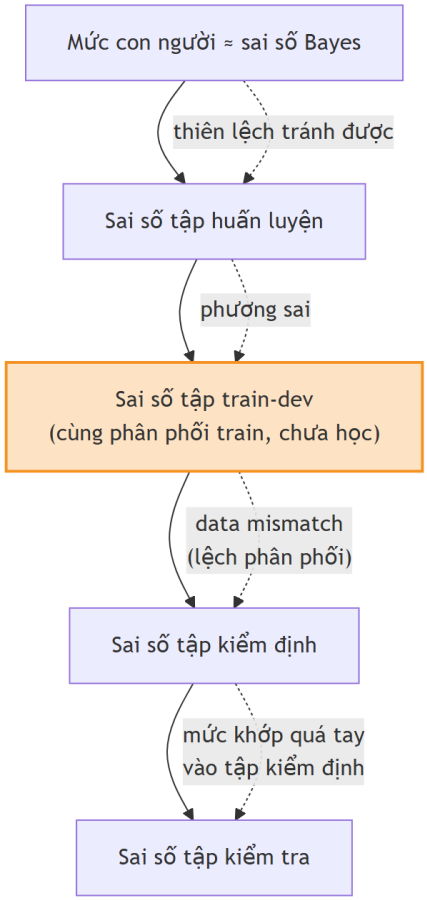
Khi train và dev khác phân phối, phân tích thiên lệch/phương sai cũ không còn an toàn. Nếu lỗi train 1% và lỗi dev 10%, bạn không biết 9% chênh lệch là do mô hình không tổng quát hoá (phương sai) hay do dev vốn là dữ liệu khó hơn (lệch phân phối). Hai thứ thay đổi cùng lúc.

Giải pháp: cắt ra một **tập train-dev** (training-dev set) từ tập huấn luyện, cùng phân phối với train nhưng *không* dùng để huấn luyện (Hình 4.10).



Hình 4.10: Cắt một phần tập huấn luyện làm tập train-dev: cùng phân phối với train nhưng mô hình chưa học

Giờ nhìn bốn con số theo thứ tự để tách bạch các loại vấn đề (Hình 4.11):



Hình 4.11: Bốn khoảng cách giữa mức con người, train, train-dev, dev và test chỉ ra bốn loại vấn đề

- Mức con người tới lỗi train: **thiên lệch tránh được**.

- Lỗi train tới lỗi train-dev: **phương sai** (cùng phân phối, nhưng dữ liệu chưa học).
- Lỗi train-dev tới lỗi dev: **lệch phân phối** (data mismatch).
- Lỗi dev tới lỗi test: mức **khớp quá tay vào tập kiểm định**; nếu chênh lớn, cần tập dev lớn hơn.

Lỗi train	Lỗi train-dev	Lỗi dev	Chẩn đoán (mức con người ≈ 0%)
1%	9%	10%	phương sai lớn (train tới train-dev nhảy mạnh)
1%	1,5%	10%	lệch phân phối lớn (chỉ nhảy khi sang dev)
10%	11%	12%	thiên lệch tránh được lớn
10%	11%	20%	vừa thiên lệch lớn vừa lệch phân phối lớn

Bảng 4.8: Đọc bốn con số để tách thiên lệch, phương sai và lệch phân phối

Đôi khi các con số không tăng dần: nếu dev/test lại *dễ hơn* train (hay gặp trong nhận dạng giọng nói), lỗi dev có thể thấp hơn lỗi train-dev. Khi gặp hiện tượng lạ, có thể lập bảng tổng quát theo hai trục: trục ngang là loại dữ liệu (dữ liệu chung, dữ liệu sát ứng dụng), trục dọc là "mức con người", "lỗi trên dữ liệu đã học", "lỗi trên dữ liệu chưa học".

4.6.3 Xử lý lệch phân phối

Thành thật mà nói, **không có cách hệ thống hoàn toàn** để xử lý lệch phân phối, nhưng có vài hướng hiệu quả:

1. **Phân tích lỗi bằng tay** để hiểu train khác dev ở điểm nào (ví dụ dev nhiều tiếng ồn xe, nhiều số nhà trong câu lệnh định vị). Chỉ xem tập dev, tránh đụng tập test.
2. **Làm cho train giống dev hơn**, hoặc thu thập thêm dữ liệu giống dev.
3. **Tổng hợp dữ liệu nhân tạo** (artificial data synthesis): ví dụ cộng âm thanh sạch với tiếng ồn trong xe để tạo dữ liệu giống trong xe, thay vì phải đi thu hàng nghìn giờ.

Một cảnh báo về tổng hợp dữ liệu. Nếu bạn có 10 000 giờ âm thanh sạch nhưng chỉ 1 giờ tiếng ồn xe rồi lặp 1 giờ đó 10 000 lần, tai người nghe vẫn thấy ổn, nhưng mô hình có thể *khớp quá tay* vào đúng 1 giờ tiếng ồn đó — vốn chỉ là một tập con rất nhỏ của mọi tiếng ồn xe có thể có. Tương tự, nếu dựng xe bằng đồ hoạ game chỉ có 20 mẫu xe, mô hình có thể khớp quá tay vào 20 mẫu đó. Hãy cẩn thận kẻo vô tình chỉ tổng hợp từ một tập con nhỏ của không gian dữ liệu.

4.6.4 Áp dụng vào dự án thực tế

4.6.4.1 Danh sách việc cần làm

1. Đặt dev/test ở đúng phân phối sản phẩm cần; cho phép train khác phân phối để có thêm dữ liệu.
2. Cắt một tập train-dev cùng phân phối với train nhưng không huấn luyện trên nó.
3. Đọc bốn con số để tách thiên lệch tránh được, phương sai, lệch phân phối và mức khớp quá tay.
4. Nếu lệch phân phối lớn: phân tích lỗi, làm train giống dev hơn, hoặc tổng hợp dữ liệu (cẩn thận khớp quá tay).

4.6.4.2 Tóm tắt cần nhớ

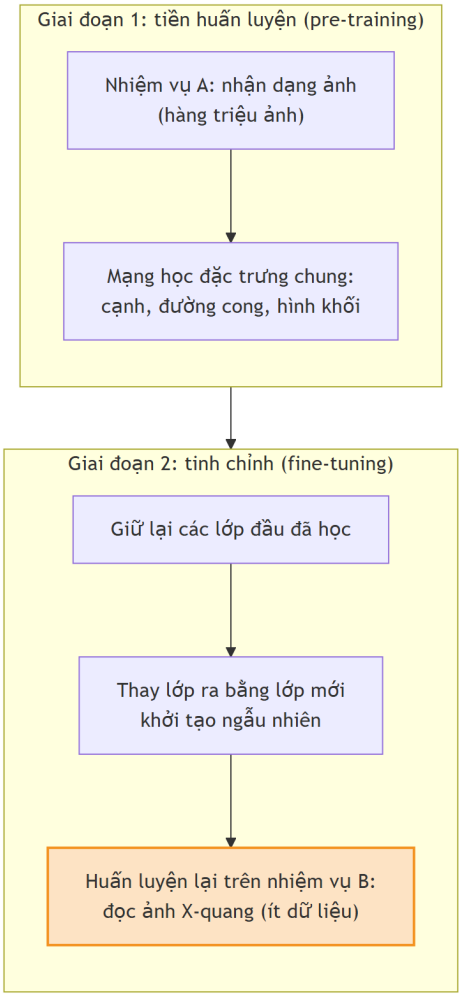
- Train được phép khác phân phối để có nhiều dữ liệu, nhưng dev/test phải đúng phân phối sản phẩm cần.
- Tập train-dev giúp tách phương sai khỏi lệch phân phối.
- Lệch phân phối chưa có cách sửa hệ thống; dựa vào phân tích lỗi và tổng hợp dữ liệu, nhưng coi chừng khớp quá tay vào tập con nhỏ.

4.7 Chuyển giao, đa nhiệm và học đầu-cuối

Ba ý tưởng mạnh mẽ để tận dụng dữ liệu từ bài toán khác hoặc đơn giản hoá hệ thống: học chuyển giao, học đa nhiệm, và học đầu-cuối.

4.7.1 Học chuyển giao

Học chuyển giao (transfer learning) là lấy tri thức mạng học được từ một nhiệm vụ rồi áp dụng sang nhiệm vụ khác. Ví dụ: huấn luyện mạng nhận dạng ảnh (mèo, chó, chim...) rồi chuyển sang đọc ảnh X-quang (Hình 4.12).



Hình 4.12: Tiền huấn luyện trên nhiệm vụ nhiều dữ liệu, rồi thay lớp ra và tinh chỉnh cho nhiệm vụ ít dữ liệu

Cách làm: giữ lại các lớp đầu của mạng (đã học đặc trưng chung như cạnh, đường cong, hình khối), xoá lớp ra cuối và thay bằng lớp mới khởi tạo ngẫu nhiên cho nhiệm vụ mới, rồi huấn luyện lại trên dữ liệu mới. Hai thuật ngữ liên quan:

- **Tiền huấn luyện** (pre-training): giai đoạn huấn luyện ban đầu trên nhiệm vụ nhiều dữ liệu.
- **Tinh chỉnh** (fine-tuning): huấn luyện lại trên dữ liệu mới. Nếu dữ liệu mới ít, chỉ huấn luyện lại một hai lớp cuối; nếu nhiều, có thể huấn luyện lại toàn bộ.

Khi nào học chuyển giao có nghĩa. Khi chuyển từ nhiệm vụ A sang nhiệm vụ B (và B mới là thứ bạn muốn làm tốt):

- A và B có cùng kiểu đầu vào (đều là ảnh, hoặc đều là âm thanh).
- A có *nhiều* dữ liệu hơn B rất nhiều (vì mỗi mẫu của B quý hơn mỗi mẫu của A cho mục tiêu B).
- Các đặc trưng tầng thấp của A có khả năng hữu ích cho B.

Nếu ngược lại (A ít dữ liệu hơn B), chuyển giao gần như không mang lại lợi ích đáng kể.

Dùng trong Keras. Hệ sinh thái Keras có sẵn nhiều mô hình đã tiền huấn luyện; chỉ cần đóng băng phần nền và gắn lớp ra mới:

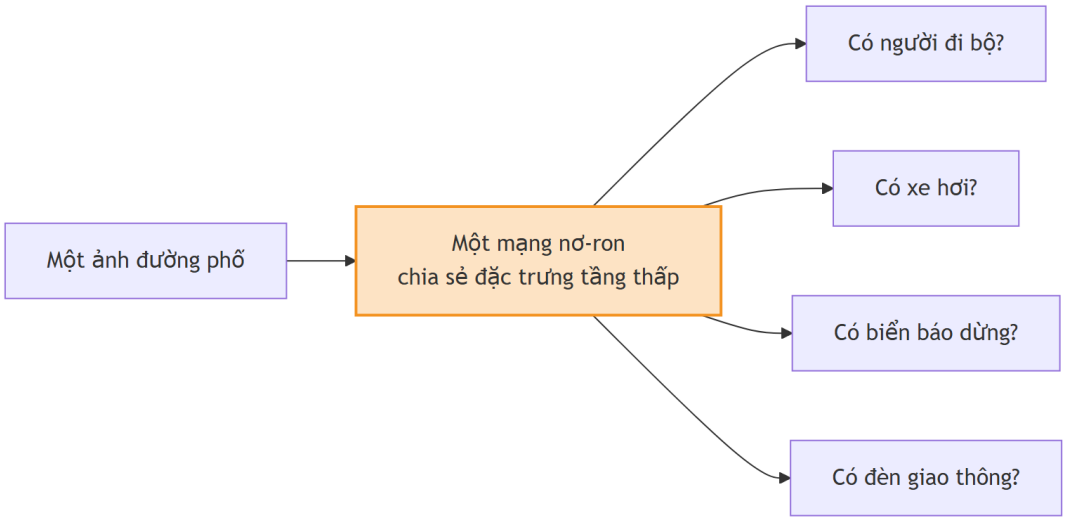
```
import tensorflow as tf

# Mô hình nền đã tiền huấn luyện trên ImageNet, bỏ lớp ra cũ
base = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3), include_top=False, weights="imagenet")
base.trainable = False  # đóng băng phần nền khi dữ liệu mới ít

# Gắn lớp ra mới cho nhiệm vụ mới (vd: chẩn đoán nhị phân từ ảnh X-quang)
model = tf.keras.Sequential([
    base,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
# Khi có nhiều dữ liệu, có thể mở băng để tinh chỉnh:
# base.trainable = True  rồi biên dịch lại với tốc độ học nhỏ
```

4.7.2 Học đa nhiệm

Khác với chuyển giao (tuần tự: học A rồi mới sang B), **học đa nhiệm** (multi-task learning) cho một mạng làm *nhiều việc cùng lúc*, và mỗi việc hỗ trợ các việc còn lại. Ví dụ xe tự lái cần đồng thời phát hiện người đi bộ, xe, biển báo dừng, đèn giao thông từ cùng một ảnh (Hình 4.13).



Hình 4.13: Một mạng chia sẻ đặc trưng tầng thấp, cho nhiều đầu ra, mỗi đầu ra một nhãn nhị phân

Mỗi ảnh có nhiều nhãn cùng lúc (có người đi bộ *và* có xe...), khác với softmax vốn gán *một* nhãn cho mỗi ảnh. Mạng cho ra nhiều đầu ra, và sai số được cộng dồn qua các nhiệm vụ. Một chi tiết hữu ích: học đa nhiệm vẫn chạy được khi *một số ảnh chỉ gán nhãn một phần* — chỉ cần cộng sai số trên những nhãn đã có, bỏ qua ô còn trống.

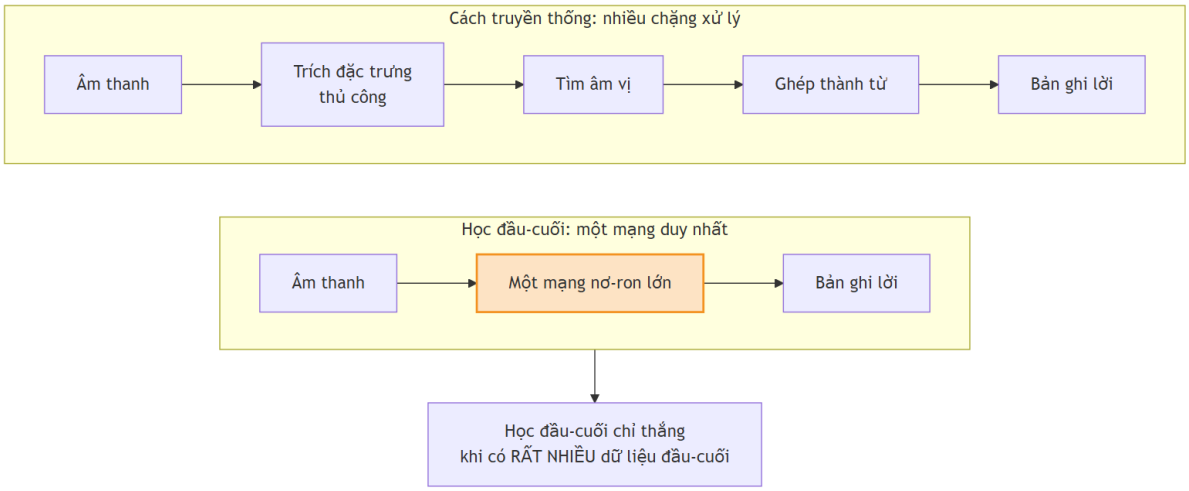
Khi nào học đa nhiệm có nghĩa. Thường khi ba điều sau đúng:

1. Các nhiệm vụ chia sẻ được đặc trưng tầng thấp (người đi bộ, xe, biển báo đều là đặc trưng đường phố).
2. Lượng dữ liệu mỗi nhiệm vụ khá tương đương; gộp lại, các nhiệm vụ khác có nhiều dữ liệu hơn hẳn một nhiệm vụ đơn lẻ.
3. Bạn huấn luyện được một mạng đủ lớn cho tất cả nhiệm vụ (chỉ khi mạng không đủ lớn thì đa nhiệm mới thua việc tách riêng).

Thực tế học chuyển giao được dùng nhiều hơn hẳn học đa nhiệm; ngoại lệ đáng chú ý là phát hiện vật thể trong thị giác máy tính, nơi một mạng nhận nhiều vật cùng lúc chạy tốt hơn nhiều mạng tách riêng.

4.7.3 Học đầu-cuối

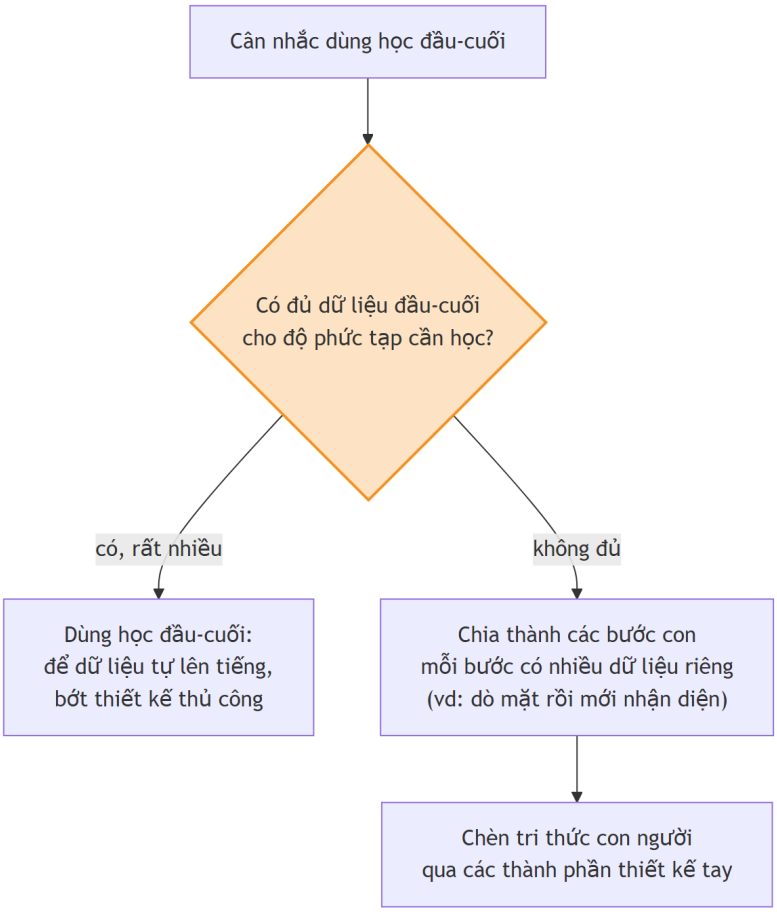
Học đầu-cuối (end-to-end deep learning) thay cả một chuỗi nhiều chặng xử lý bằng *một* mạng nơ-ron duy nhất (Hình 4.14).



Hình 4.14: Nhận dạng giọng nói truyền thống qua nhiều chặng, so với một mạng duy nhất đi thẳng từ âm thanh tới bản ghi

Ví dụ nhận dạng giọng nói truyền thống đi qua nhiều chặng: trích đặc trưng thủ công, tìm âm vị, ghép từ, ra bản ghi. Học đầu-cuối huấn luyện một mạng lớn đi thẳng từ âm thanh tới bản ghi. Điểm mấu chốt: học đầu-cuối chỉ thắng khi có **rất nhiều** dữ liệu đầu-cuối (ví dụ hàng chục nghìn giờ); với dữ liệu ít hơn, chuỗi nhiều chặng truyền thống thường chạy tốt hơn.

Khi nào nên tách bước thay vì đi thẳng. Có những bài toán tách bước lại tốt hơn vì mỗi bước con đơn giản hơn và có nhiều dữ liệu riêng (Hình 4.15):



Hình 4.15: Câu hỏi then chốt: có đủ dữ liệu đầu-cuối cho độ phức tạp cần học không?

Ví dụ cổng nhận diện khuôn mặt: thay vì đi thẳng từ ảnh camera tới danh tính, cách tốt nhất hiện nay là hai bước — dò vị trí khuôn mặt, rồi cắt sát và nhận diện. Mỗi bước có nhiều dữ liệu riêng, trong khi dữ liệu đầu-cuối (ảnh camera kèm danh tính) lại ít. Tương tự, ước lượng tuổi trẻ em từ ảnh X-quang bàn tay nên tách thành dò xương rồi tra bảng, vì chưa đủ dữ liệu để học thẳng.

Ưu và nhược. Ưu điểm của học đầu-cuối: để dữ liệu tự lên tiếng (không ép mô hình theo định kiến con người như khái niệm "âm vị"), và bớt công thiết kế thủ công. Nhược điểm: cần rất nhiều dữ liệu, và loại bỏ các thành phần thiết kế tay vốn hữu ích khi dữ liệu ít (thiết kế tay là cách chèn tri thức con người vào thuật toán). Câu hỏi then chốt khi quyết định: *bạn có đủ dữ liệu để học ánh xạ phức tạp từ đầu vào tới đầu ra không?*

Ví dụ xe tự lái minh hoạ cách phối hợp: dùng học sâu để *phát hiện* xe và người đi bộ, nhưng *lập lộ trình* dùng phần mềm hoạch định chuyển động, rồi *điều khiển* vô-lăng và chân ga bằng thuật toán điều khiển riêng. Đi thẳng từ ảnh tới lệnh lái không phải hướng tốt nhất với dữ liệu và công nghệ hiện nay.

4.7.4 Áp dụng vào dự án thực tế

4.7.4.1 Bảng chọn kỹ thuật

Tình huống	Kỹ thuật nên dùng
Bài toán đích ít dữ liệu, có bài toán liên quan nhiều dữ liệu, cùng kiểu đầu vào	học chuyển giao (tiền huấn luyện rồi tinh chỉnh)
Nhiều nhiệm vụ chia sẻ đặc trưng, dữ liệu tương đương, mạng đủ lớn	học đa nhiệm
Có rất nhiều dữ liệu đầu-cuối, muốn bớt thiết kế tay	học đầu-cuối
Ít dữ liệu đầu-cuối nhưng nhiều dữ liệu cho từng bước con	tách bước, dùng học sâu cho từng thành phần

Bảng 4.9: Chọn kỹ thuật theo lượng dữ liệu và cấu trúc bài toán

4.7.4.2 Tóm tắt cần nhớ

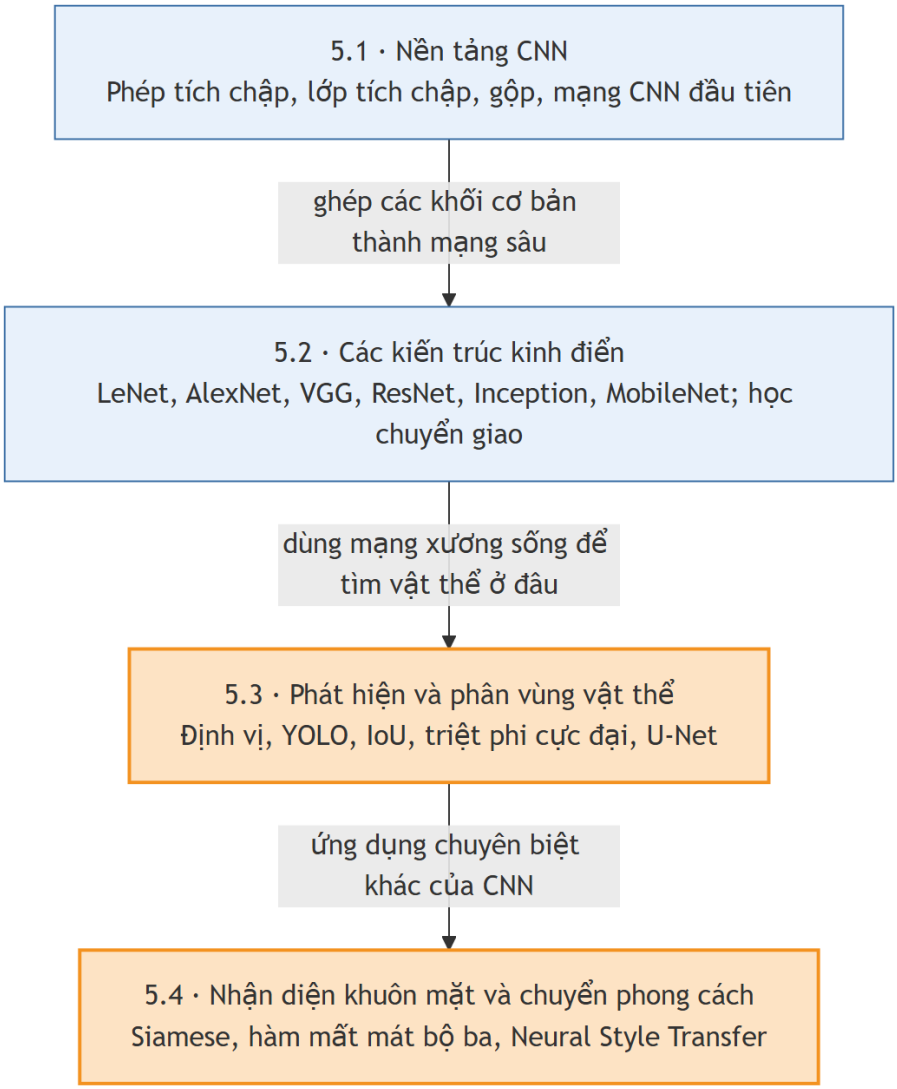
- Học chuyển giao: tiền huấn luyện trên nhiệm vụ nhiều dữ liệu, tinh chỉnh cho nhiệm vụ ít dữ liệu cùng kiểu đầu vào.
- Học đa nhiệm: một mạng làm nhiều việc cùng lúc, hữu ích khi chia sẻ đặc trưng và mạng đủ lớn; thực tế dùng ít hơn chuyển giao.
- Học đầu-cuối: một mạng thay cả chuỗi, chỉ thắng khi có rất nhiều dữ liệu đầu-cuối; nếu không, tách bước thường tốt hơn.
- Câu hỏi then chốt cho đầu-cuối: có đủ dữ liệu cho độ phức tạp cần học không?

Chương 5

Convolutional Neural Networks

Thị giác máy tính (computer vision) là một trong những lĩnh vực tiến nhanh nhất nhờ học sâu. Xe tự lái nhận biết xe và người đi bộ xung quanh, điện thoại mở khoá bằng khuôn mặt, các ứng dụng gợi ý ảnh đẹp, và cả những dạng nghệ thuật mới – tất cả đều dựa trên **mạng nơ-ron tích chập** (Convolutional Neural Network – CNN, hay ConvNet). Đây là họ mạng được thiết kế riêng cho dữ liệu dạng lưới như ảnh, nơi một đặc trưng học được ở một vùng có thể dùng lại cho mọi vùng khác.

Chương này đi một mạch qua bốn mục. Mỗi mục trả lời một câu hỏi, và câu trả lời của mục trước dẫn tới câu hỏi của mục sau (Hình 5.1).



Hình 5.1: Lộ trình của chương Convolutional Neural Networks

Mục	Câu hỏi	Nội dung chính
5.1 Nền tảng CNN	Làm sao để mạng xử lý được ảnh lớn mà không bùng nổ tham số?	tích chập, phát hiện cạnh, đệm viền, bước nhảy, lớp tích chập, gộp, mạng CNN đầy đủ
5.2 Các kiến trúc kinh điển	Người ta đã ghép các khối cơ bản thành mạng mạnh ra sao?	LeNet, AlexNet, VGG, ResNet, tích chập 1x1, Inception, MobileNet, EfficientNet, học chuyển giao, tăng cường dữ liệu
5.3 Phát hiện và phân vùng	Làm sao biết vật thể *ở đâu* trong ảnh, tới từng điểm ảnh?	định vị, điểm mốc, cửa sổ trượt, YOLO, IoU, triệt phi cực đại, hộp neo, U-Net
5.4 Khuôn mặt và phong cách	Làm sao nhận ra một người từ một tấm ảnh, và vẽ lại ảnh theo phong cách khác?	nhận diện khuôn mặt, học một lần, Siamese, hàm mất mát bộ ba, chuyển phong cách

Bảng 5.1: Bốn mục của chương Convolutional Neural Networks

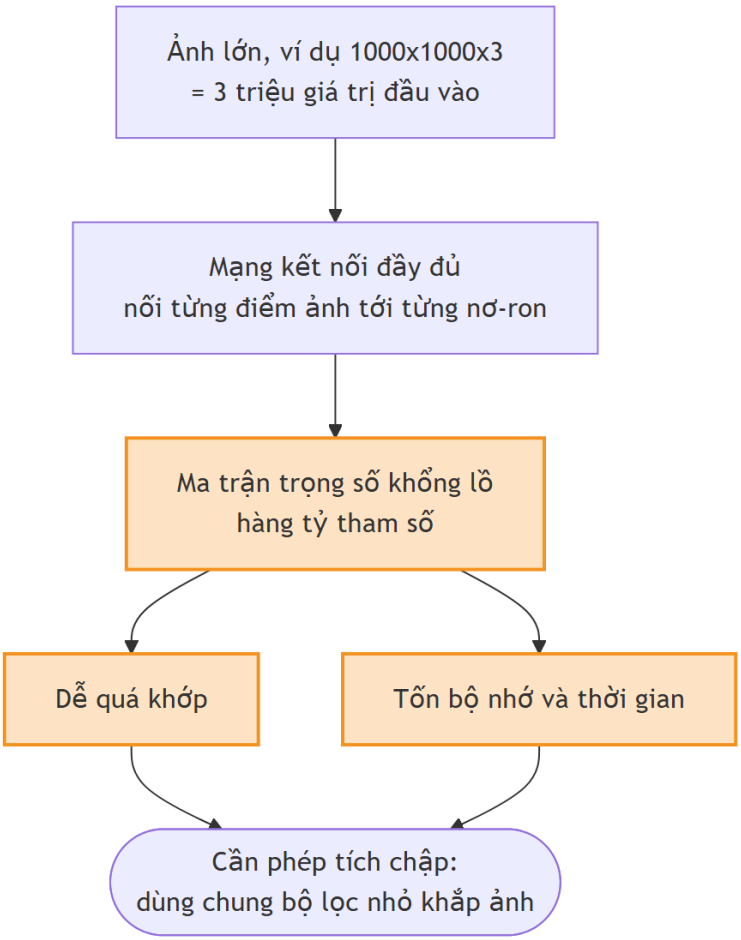
Nên đọc theo thứ tự, vì các mục sau dùng lại khái niệm của các mục trước. Code trong chương dùng TensorFlow/Keras; những đoạn tự cài đặt bằng NumPy chỉ nhằm hiểu bản chất, không phải để dùng trong dự án. Tinh thần xuyên suốt là *thực chiến*: ưu tiên dùng kiến trúc và trọng số có sẵn, hiểu đủ sâu để chỉnh cho đúng bài toán của mình.

5.1 Nền tảng của mạng tích chập

Ảnh là dữ liệu dạng lưới: một ảnh màu là một khối ba chiều gồm chiều cao, chiều rộng và ba kênh màu đỏ, lục, lam (RGB). Mục này xây dựng CNN từ đúng một ý tưởng – **phép tích chập** – rồi lắp dần thành một mạng nhận dạng ảnh hoàn chỉnh. Lộ trình: vì sao mạng thường không hợp với ảnh lớn → phép tích chập và phát hiện cạnh → đệm viền và bước nhảy → tích chập trên khối nhiều kênh → một lớp tích chập → lớp gộp → một mạng CNN đầy đủ → vì sao tích chập hiệu quả → hai bài thực hành.

5.1.1 Vì sao cần phép tích chập

Trong các chương trước, ta làm việc với ảnh nhỏ cỡ 64x64. Nhân ba chiều lại được khoảng 12.288 giá trị đầu vào – không quá lớn. Nhưng ảnh thực tế thường to hơn nhiều. Một ảnh 1000x1000 điểm ảnh có ba kênh màu cho tới ba triệu giá trị đầu vào. Nếu nối thẳng đầu vào này vào một lớp ẩn chỉ 1000 nơ-ron theo kiểu **kết nối đầy đủ** (fully connected), ma trận trọng số đã có tới ba tỷ số. Với ngần ấy tham số, mạng rất dễ **quá khớp** và đòi hỏi bộ nhớ, thời gian huấn luyện không khả thi (Hình 5.2).

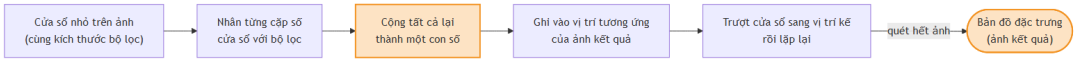


Hình 5.2: Mạng kết nối đầy đủ không hợp với ảnh lớn; phép tích chập giải quyết bằng cách dùng chung bộ lọc nhỏ

Phép tích chập giải quyết vấn đề này bằng cách *dùng chung một bộ lọc nhỏ* trượt khắp ảnh, thay vì nối riêng từng điểm ảnh. Nhờ đó số tham số nhỏ đi rất nhiều và không phụ thuộc kích thước ảnh.

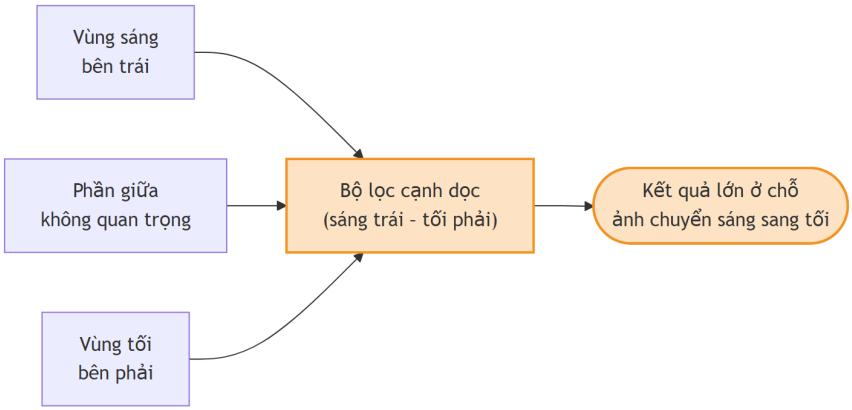
5.1.2 Phép tích chập và phát hiện cạnh

Hãy bắt đầu từ bài toán đơn giản: phát hiện **cạnh dọc** trong ảnh. Ý tưởng là lấy một **bộ lọc** (filter, đôi khi gọi là kernel) kích thước nhỏ, ví dụ 3x3, rồi trượt nó khắp ảnh. Ở mỗi vị trí, ta đặt bộ lọc đè lên vùng ảnh cùng kích thước, nhân từng cặp số tương ứng, cộng tất cả lại thành một con số, rồi ghi vào ảnh kết quả. Trượt hết ảnh thì được một ảnh mới gọi là **bản đồ đặc trưng** (feature map). Quy trình này minh hoạ ở Hình 5.3.



Hình 5.3: Phép tích chập: trượt một cửa sổ nhỏ, nhân rồi cộng, ghi ra bản đồ đặc trưng

Một bộ lọc cạnh dọc kinh điển có ba cột: cột trái toàn số dương, cột giữa toàn số 0, cột phải toàn số âm. Trực giác (Hình 5.4): bộ lọc này cho kết quả lớn ở nơi ảnh *sáng bên trái, tối bên phải* – đúng là một cạnh dọc. Nếu ảnh ngược lại (tối trái, sáng phải), kết quả đổi dấu, cho biết đây là cạnh chuyển từ tối sang sáng. Lật bộ lọc 90 độ thì thành bộ phát hiện cạnh ngang.



Hình 5.4: Trực giác bộ lọc cạnh dọc: sáng bên trái, tối bên phải thì cho tín hiệu mạnh

Trong lịch sử thị giác máy tính, người ta tranh luận nhiều về bộ chín số nào là tốt nhất, sinh ra các bộ lọc có tên riêng như Sobel hay Scharr. Điểm mấu chốt của học sâu là: **đừng chọn tay chín số đó**. Hãy coi chúng là *tham số* và để mạng tự học qua lan truyền ngược. Khi đó mạng có thể học bộ lọc cạnh dọc, cạnh ngang, hay cạnh nghiêng bất kỳ góc nào – thậm chí những đặc trưng ta không có tên gọi – miễn là chúng giúp nhận dạng tốt. Đây là một trong những ý tưởng mạnh nhất của thị giác máy tính.

Thuật ngữ	Ý nghĩa trực quan
Bộ lọc (filter / kernel)	cửa sổ trọng số nhỏ, trượt khắp ảnh để dò một đặc trưng
Bản đồ đặc trưng	ảnh kết quả sau khi tích chập, sáng ở nơi tìm thấy đặc trưng
Phép tích chập (deep learning)	nhân từng cặp số rồi cộng; thực ra là tương quan chéo (bỏ bước lật bộ lọc), nhưng giới học sâu quen gọi là tích chập

Bảng 5.2: Vài thuật ngữ cơ bản của phép tích chập

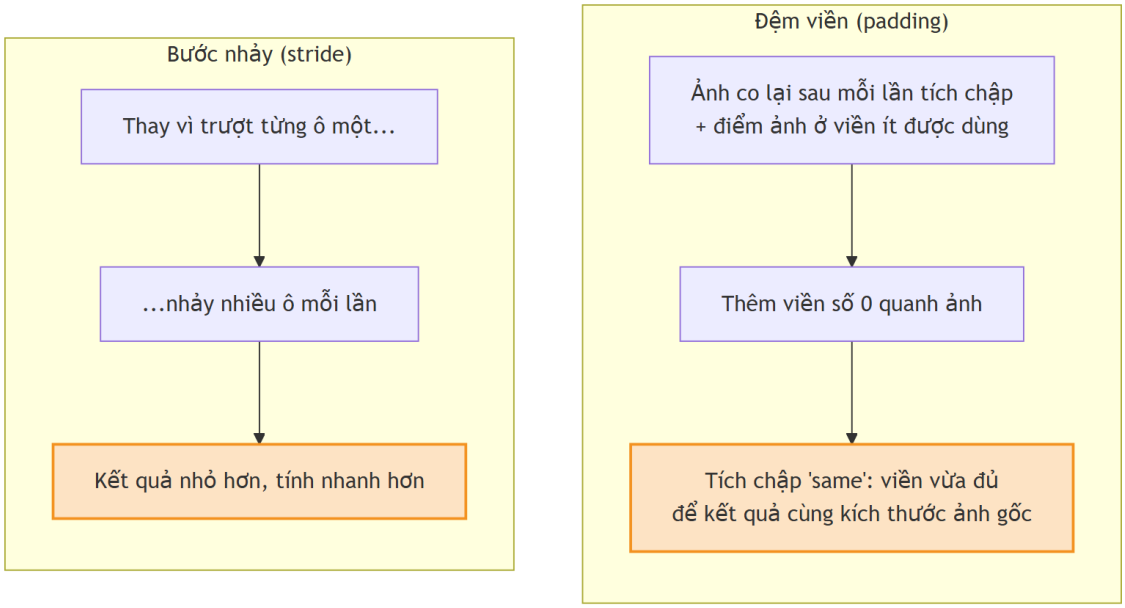
Về cài đặt: mọi framework đều có sẵn phép này, ví dụ `tf.nn.conv2d` hoặc lớp `Conv2D` trong Keras, nên trong dự án không cần tự viết vòng lặp tích chập.

5.1.3 Đệm viền và bước nhảy

Phép tích chập cơ bản có hai nhược điểm (Hình 5.5). Thứ nhất, ảnh **co lại** sau mỗi lần: một ảnh 6x6 tích chập với bộ lọc 3x3 cho ra 4x4. Qua nhiều lớp, ảnh teo dần, không thể làm mạng thật sâu. Thứ hai, các điểm ảnh ở *góc và viền* chỉ được một ô kết quả chạm tới, trong khi điểm ở giữa được rất nhiều ô dùng; như vậy thông tin ở rìa ảnh bị bỏ phí.

Đệm viền (padding) khắc phục cả hai: thêm một hay vài viền số 0 quanh ảnh trước khi tích chập. Có hai lựa chọn quen thuộc. Tích chập *valid* nghĩa là không đệm, chấp nhận ảnh co lại. Tích chập *same* nghĩa là đệm vừa đủ để kết quả có *cùng kích thước* ảnh gốc. Theo quy ước, kích thước bộ lọc thường là số lẻ (3x3, 5x5, 7x7) để viền đệm cân xứng và có một điểm trung tâm.

Bước nhảy (stride) là khoảng cách mỗi lần trượt cửa sổ. Thay vì nhích từng ô, ta có thể nhảy hai ô một lần. Bước nhảy lớn làm kết quả nhỏ hơn và tính nhanh hơn. Quy ước: nếu cửa sổ trượt ra ngoài ảnh thì bỏ qua vị trí đó, và khi kích thước kết quả không chia hết thì làm tròn xuống.



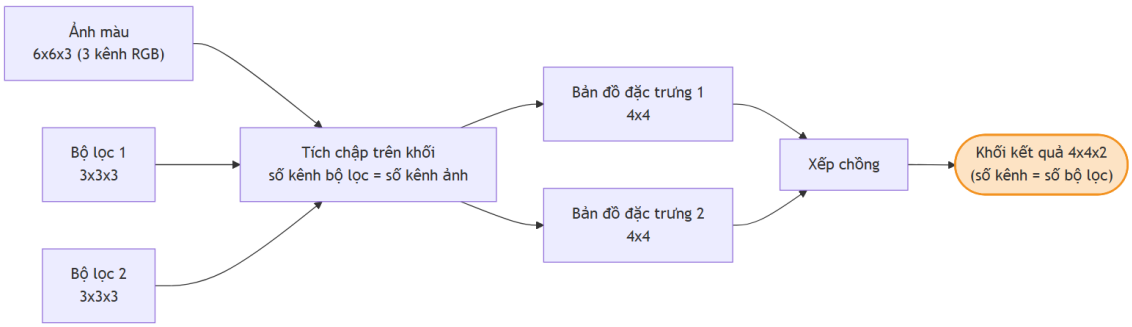
Hình 5.5: Đệm viền giữ nguyên kích thước và dùng đều thông tin ở rìa; bước nhảy làm kết quả nhỏ và nhanh hơn

Cách tính kích thước đầu ra: lấy kích thước ảnh, cộng hai lần số viền đệm, trừ kích thước bộ lọc, chia cho bước nhảy, cộng một, rồi làm tròn xuống. Ví dụ ảnh 7x7, bộ lọc 3x3, không đệm, bước nhảy 2 cho kết quả 3x3.

5.1.4 Tích chập trên khối nhiều kênh

Ảnh màu có ba kênh, nên bộ lọc cũng phải có ba kênh: một bộ lọc 3x3 trên ảnh RGB thực chất là khối 3x3x3. Quy tắc bất di bất dịch: **số kênh của bộ lọc phải bằng số kênh của đầu vào**. Khi tích chập, ta nhân toàn bộ các số trong khối bộ lọc (27 số ở ví dụ này) với vùng ảnh tương ứng rồi cộng lại thành *một* con số. Kết quả của một bộ lọc trên ảnh màu vì thế lại là một bản đồ hai chiều.

Sức mạnh thật sự đến khi dùng **nhiều bộ lọc cùng lúc**: một bộ dò cạnh dọc, một bộ dò cạnh ngang, và nhiều bộ khác nữa. Mỗi bộ lọc cho một bản đồ đặc trưng; xếp chồng chúng lại thành một khối kết quả mới (Hình 5.6). Số kênh của khối kết quả bằng đúng *số bộ lọc* dùng. Trong tài liệu, số kênh này đôi khi gọi là *độ sâu* (depth) của khối.



Hình 5.6: Tích chập trên khối ba kênh với nhiều bộ lọc; số kênh kết quả bằng số bộ lọc

5.1.5 Một lớp tích chập

Một lớp tích chập hoàn chỉnh thêm hai bước nữa sau phép tích chập (Hình 5.7): cộng một **độ lệch** (bias) cho mỗi bộ lọc, rồi đưa qua một **hàm kích hoạt phi tuyến** như ReLU. Đây chính là bản sao của một lớp mạng nơ-ron thường: phép tích chập đóng vai trò nhân trọng số, cộng bias, rồi kích hoạt.



Hình 5.7: Một lớp tích chập = tích chập (tuyến tính) + cộng độ lệch + hàm kích hoạt ReLU

Một tính chất rất đẹp: số tham số của lớp tích chập **không phụ thuộc kích thước ảnh**. Nếu dùng 10 bộ lọc 3x3x3, mỗi bộ có 27 trọng số cộng một độ lệch là 28, tổng cộng 280 tham số – dù ảnh là 1000x1000 hay lớn hơn nữa. Chính đặc điểm này giúp CNN ít quá khớp.

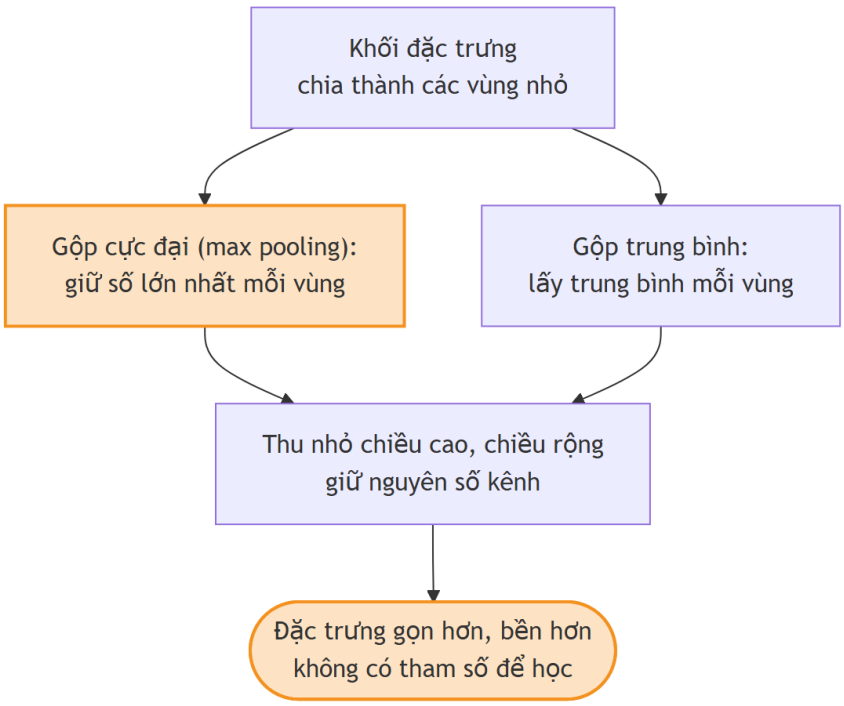
Siêu tham số của lớp	Vai trò
Kích thước bộ lọc	cửa sổ nhìn rộng bao nhiêu (thường 3x3, 5x5)
Số bộ lọc	số đặc trưng dò cùng lúc; bằng số kênh của khối kết quả
Đệm viền	valid (không đệm) hay same (giữ nguyên kích thước)
Bước nhảy	trượt bao nhiêu ô mỗi lần; lớn thì kết quả nhỏ và nhanh hơn

Bảng 5.3: Các siêu tham số cần chọn cho một lớp tích chập

Quy luật thường thấy khi đi sâu vào mạng: chiều cao và chiều rộng *giảm dần*, còn số kênh *tăng dần*. Ví dụ một mạng có thể đi từ 39x39x3 qua 37x37x10, rồi 17x17x20, rồi 7x7x40.

5.1.6 Lớp gộp

Ngoài lớp tích chập, CNN hay dùng **lớp gộp** (pooling) để thu nhỏ biểu diễn, tăng tốc tính toán và làm đặc trưng bền hơn (Hình 5.8). Phổ biến nhất là **gộp cực đại** (max pooling): chia khối thành các vùng nhỏ và chỉ giữ số lớn nhất mỗi vùng. Trực giác: nếu một đặc trưng xuất hiện mạnh ở đâu đó trong vùng thì con số lớn được giữ lại; nếu không có thì số nhỏ. Còn **gộp trung bình** (average pooling) lấy trung bình, ít dùng hơn, trừ khi gom cả bản đồ về một vector ở gần cuối mạng.

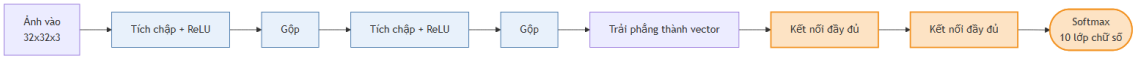


Hình 5.8: Lớp gộp thu nhỏ chiều cao và chiều rộng, giữ nguyên số kênh; không có tham số để học

Lớp gộp chỉ có vài siêu tham số (kích thước cửa sổ và bước nhảy, lựa chọn phổ biến là cửa sổ 2x2 bước nhảy 2 để giảm một nửa chiều cao và chiều rộng). Điều quan trọng: lớp gộp **không có tham số để học** – lan truyền ngược không chỉnh gì ở đây, nó chỉ là một hàm cố định. Gộp áp dụng độc lập trên từng kênh, nên số kênh vào bằng số kênh ra.

5.1.7 Một mạng CNN đầy đủ

Ghép các khối lại, ta có một CNN hoàn chỉnh. Hình 5.9 lấy cảm hứng từ mạng kinh điển LeNet-5: ảnh vào đi qua vài cặp *tích chập – gộp*, rồi trải phẳng thành vector, qua vài lớp **kết nối đầy đủ**, cuối cùng là softmax cho ra lớp dự đoán.



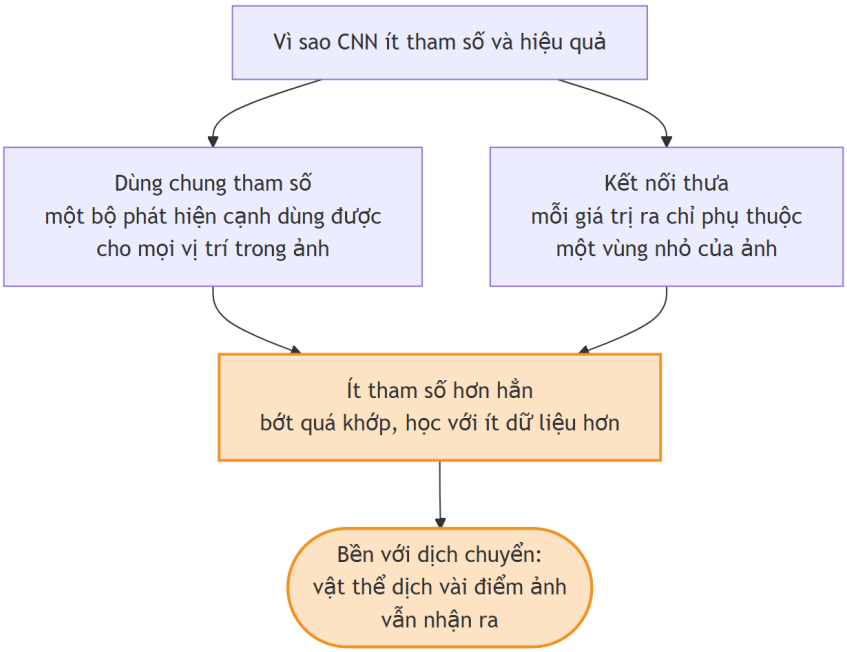
Hình 5.9: Một CNN điển hình: vài cặp tích chập–gộp, trải phẳng, vài lớp kết nối đầy đủ, rồi softmax

Có một lưu ý về cách đếm lớp: vì lớp gộp không có trọng số, nhiều người gộp một cặp tích chập + gộp thành *một lớp*. Ba điều đáng nhớ khi quan sát bảng kích thước của một CNN: lớp gộp không có tham số; lớp tích chập có tương đối ít tham số; và phần lớn tham số nằm ở các lớp kết nối đầy đủ cuối mạng. Kích thước biểu diễn cũng nên giảm *từ từ*, giảm quá nhanh thường hại cho hiệu năng.

5.1.8 Vì sao tích chập hiệu quả

Có hai lý do chính khiến lớp tích chập vượt trội lớp kết nối đầy đủ (Hình 5.10).

- **Dùng chung tham số** – một bộ dò cạnh dọc hữu ích ở góc trên trái thì cũng hữu ích ở góc dưới phải, nên dùng chung một bộ lọc cho mọi vị trí. Nhờ vậy số tham số giảm mạnh.
- **Kết nối thưa** – mỗi giá trị đầu ra chỉ phụ thuộc một vùng nhỏ của ảnh (bằng kích thước bộ lọc), không phụ thuộc mọi điểm ảnh. Phần còn lại của ảnh không ảnh hưởng tới giá trị đó.



Hình 5.10: Hai lý do CNN ít tham số và bền: dùng chung tham số và kết nối thưa

Hệ quả là CNN ít tham số, bớt quá khớp, học được với ít dữ liệu hơn, và **bền với dịch chuyển**: ảnh con mèo dịch sang phải vài điểm ảnh vẫn là con mèo. Huấn luyện CNN cũng giống mọi mạng khác: định nghĩa hàm chi phí trên tập huấn luyện rồi dùng gradient descent (hoặc Adam, RMSProp...) để tối ưu các trọng số và độ lệch.

5.1.9 Thực hành: tự cài đặt khối CNN bằng NumPy

Bài thực hành đầu tiên tự cài đặt các khối CNN bằng NumPy để hiểu bản chất. Đây là phần *học hiểu*, không dùng trong dự án thực tế (dự án nên dùng `Conv2D` của Keras). Phép đệm viền dùng `np.pad`, còn một bước tích chập đơn là nhân theo từng phần tử rồi cộng.

```
import numpy as np

def zero_pad(X, pad):
    # X có dạng (số ảnh, cao, rộng, kênh); chỉ đệm chiều cao và rộng
    return np.pad(X, ((0, 0), (pad, pad), (pad, pad), (0, 0)),
                  mode='constant', constant_values=(0, 0))

def conv_single_step(a_slice_prev, W, b):
    s = a_slice_prev * W          # nhân từng cặp số
    Z = np.sum(s)                 # cộng tất cả lại
    return Z + float(b)           # cộng độ lệch
```

Lớp tích chập xuôi chỉ là vòng lặp đặt bộ lọc lên từng vị trí; kích thước đầu ra tính theo công thức ở mục 5.1.3:

```
def conv_forward(A_prev, W, b, hparameters):
    stride, pad = hparameters["stride"], hparameters["pad"]
    (m, n_H_prev, n_W_prev, n_C_prev) = A_prev.shape
    (f, f, n_C_prev, n_C) = W.shape
    n_H = int((n_H_prev - f + 2 * pad) / stride) + 1
    n_W = int((n_W_prev - f + 2 * pad) / stride) + 1
```

```
    Z = np.zeros((m, n_H, n_W, n_C))
    A_prev_pad = zero_pad(A_prev, pad)
    # ... trượt cửa sổ, với mỗi vị trí gọi conv_single_step để điền Z
    return Z, (A_prev, W, b, hparameters)
```

Khi chạy các ô kiểm tra, notebook đã lưu sẵn các con số xác nhận cài đặt đúng (trích nguyên từ ô kết quả, với `np.random.seed(1)`). Chúng chỉ để kiểm tra cú pháp, không phải số liệu thí nghiệm:

- `zero_pad`: hình dạng đi từ `(4, 3, 3, 2)` sang `(4, 9, 9, 2)` khi đệm viền 3.
- `conv_single_step`: `Z = -6.999089450680221`.
- `conv_forward`: trung bình của `Z` là `0.5511276474566768`.
- Gộp cực đại và gộp trung bình cho các khối kết quả hình dạng `(2, 3, 3, 3)` (cửa sổ 3, bước nhảy 1) và `(2, 2, 2, 3)` (cửa sổ 3, bước nhảy 2), khớp với ô "Expected Output".

Lớp gộp xuôi cũng chỉ là lấy `np.max` hoặc `np.mean` trên mỗi cửa sổ. Phần lan truyền ngược cho tích chập và gộp có trong notebook nhưng không bắt buộc; trong thực tế framework lo hết việc này.

5.1.10 Thực hành: xây CNN với Keras

Bài thực hành thứ hai dùng TensorFlow/Keras. Có hai cách dựng mô hình, nên nắm cả hai.

Sequential API hợp với mô hình xếp lớp tuyến tính một mạch. Ví dụ bộ phân loại "nhà vui" (nhận biết khuôn mặt có đang cười hay không) trên ảnh 64x64x3:

```
import tensorflow as tf
import tensorflow.keras.layers as tfl

happy_model = tf.keras.Sequential([
    tfl.ZeroPadding2D(padding=(3, 3), input_shape=(64, 64, 3)),
    tfl.Conv2D(32, (7, 7), strides=(1, 1)),
    tfl.BatchNormalization(axis=3),
    tfl.ReLU(),
    tfl.MaxPooling2D(),
    tfl.Flatten(),
    tfl.Dense(1, activation='sigmoid'),
])
happy_model.compile(optimizer='adam', loss='binary_crossentropy',
↪  metrics=['accuracy'])
happy_model.fit(X_train, Y_train, epochs=10, batch_size=16)
```

Kết quả đã lưu trong notebook: sau 10 vòng lặp, độ chính xác trên tập huấn luyện quanh `0.96`; đánh giá trên tập kiểm tra cho độ chính xác `0.8867` (mất mát `0.1918`). Những con số này lấy trực tiếp từ ô kết quả đã lưu, không chạy lại.

Functional API linh hoạt hơn, cho phép nhánh rẽ và nhiều đầu vào/ra. Ví dụ bộ phân loại ngôn ngữ ký hiệu sáu lớp (chữ số 0–5) trên bộ SIGNS:

```
def convolutional_model(input_shape):
    input_img = tf.keras.Input(shape=input_shape)
    Z1 = tfl.Conv2D(8, (4, 4), strides=1, padding='same')(input_img)
    A1 = tfl.ReLU()(Z1)
    P1 = tfl.MaxPool2D(pool_size=(8, 8), strides=8, padding='same')(A1)
    Z2 = tfl.Conv2D(16, (2, 2), strides=1, padding='same')(P1)
    A2 = tfl.ReLU()(Z2)
    P2 = tfl.MaxPool2D(pool_size=(4, 4), strides=4, padding='same')(A2)
```

```
    F = tfl.Flatten()(P2)
    outputs = tfl.Dense(6, activation='softmax')(F)
    return tf.keras.Model(inputs=input_img, outputs=outputs)

conv_model = convolutional_model((64, 64, 3))
conv_model.compile(optimizer='adam', loss='categorical_crossentropy',
↪  metrics=['accuracy'])
train_dataset = tf.data.Dataset.from_tensor_slices((X_train, Y_train)).batch(64)
history = conv_model.fit(train_dataset, epochs=100, validation_data=test_dataset)
```

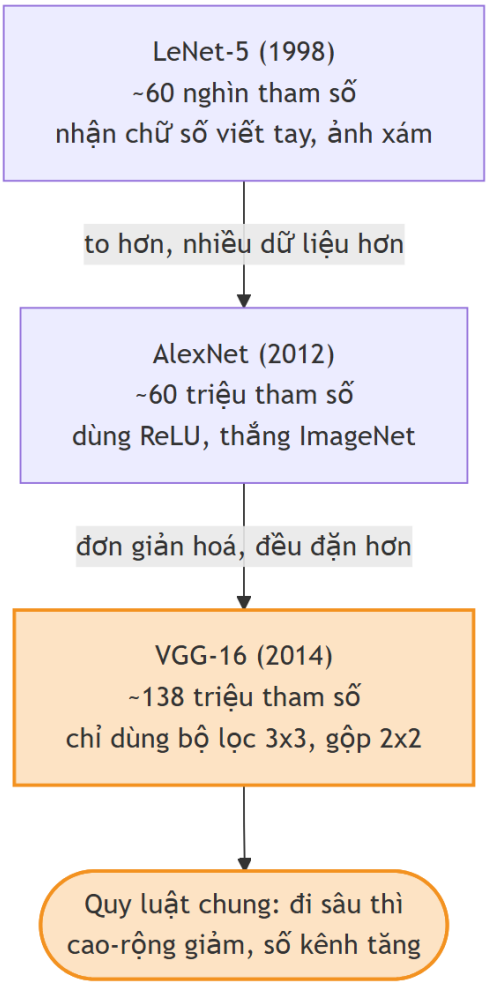
Kết quả đã lưu: sau 100 vòng lặp, độ chính xác huấn luyện khoảng `0.93`, độ chính xác kiểm chứng (validation) khoảng `0.87`. Mô hình này nhỏ gọn (chỉ hơn 1.300 tham số) nhưng đã minh hoạ đầy đủ nếp làm việc với Keras: dựng mô hình, `compile`, `fit`. Mục 5.2 sẽ dùng chính nếp này với các kiến trúc mạnh hơn nhiều.

5.2 Các kiến trúc kinh điển và học chuyển giao

Những năm qua, phần lớn tiến bộ của thị giác máy tính nằm ở việc *ghép các khối cơ bản* (tích chập, gộp, kết nối đầy đủ) thành những kiến trúc hiệu quả. Một kiến trúc chạy tốt cho bài toán này thường cũng tốt cho bài toán khác, nên cách học nhanh nhất là xem lại vài ví dụ thành công. Mục này điểm qua các mạng kinh điển, hai ý tưởng lớn (phần dư và Inception), nhánh mạng nhẹ (MobileNet, EfficientNet), rồi hai kỹ thuật thực chiến quan trọng nhất: **học chuyển giao** và **tăng cường dữ liệu**.

5.2.1 Ba mạng kinh điển: LeNet, AlexNet, VGG

Hình 5.11 tóm tắt ba cột mốc. **LeNet-5** (1998) nhận chữ số viết tay trên ảnh xám, chỉ khoảng 60 nghìn tham số, dùng xen kẽ tích chập và gộp trung bình. **AlexNet** (2012) có cùng tinh thần nhưng lớn hơn nghìn lần (khoảng 60 triệu tham số), dùng hàm kích hoạt ReLU và huấn luyện trên bộ ImageNet; chính nó thuyết phục cộng đồng rằng học sâu thật sự hiệu quả cho thị giác. **VGG-16** (2014) đơn giản hoá triệt để: mọi lớp tích chập đều là bộ lọc 3x3 bước nhảy 1 đệm same, mọi lớp gộp đều 2x2 bước nhảy 2; số kênh nhân đôi sau mỗi cụm. Mạng rất đều đặn nhưng khá nặng (khoảng 138 triệu tham số).

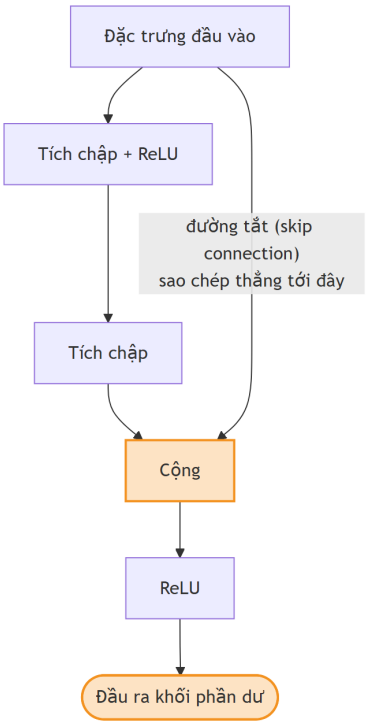


Hình 5.11: Ba mạng kinh điển và quy luật chung: đi sâu thì chiều cao–rộng giảm, số kênh tăng

Điểm chung đáng nhớ: một mẫu lặp phổ biến là vài lớp tích chập rồi một lớp gộp, lặp lại nhiều lần, cuối cùng là vài lớp kết nối đầy đủ và softmax. Nếu đọc bài báo gốc, nên bắt đầu từ AlexNet (dễ nhất), rồi VGG, rồi LeNet.

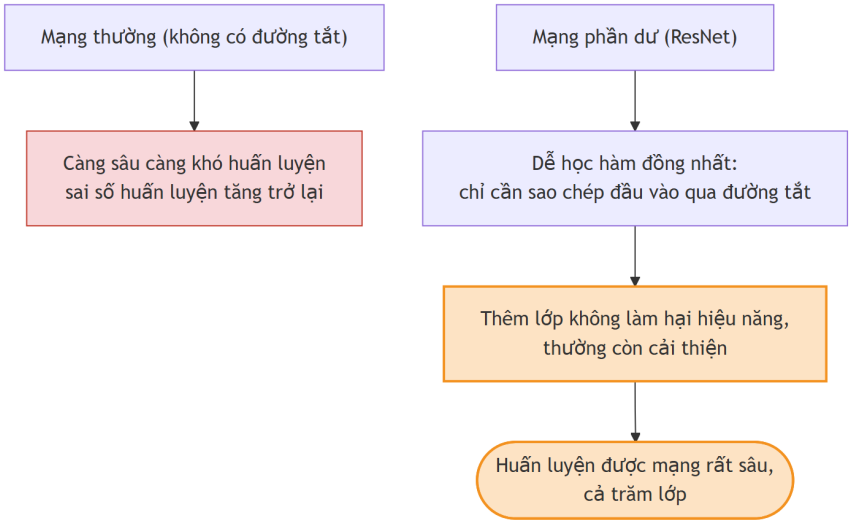
5.2.2 Mạng phần dư (ResNet)

Mạng rất sâu khó huấn luyện vì gradient dễ tiêu biến hoặc bùng nổ. **ResNet** giải quyết bằng **đường tắt** (skip connection): lấy đặc trưng ở một lớp và cộng thẳng vào một lớp sâu hơn, bỏ qua vài lớp ở giữa (Hình 5.12). Hai lớp kèm đường tắt tạo thành một **khối phần dư** (residual block); xếp nhiều khối lại thành ResNet, có thể sâu tới cả trăm lớp.



Hình 5.12: Khối phần dư: đầu vào được cộng thẳng vào đầu ra qua đường tắt, trước hàm ReLU cuối

Vì sao hiệu quả (Hình 5.13)? Với mạng thường, càng sâu càng khó, sai số huấn luyện có thể tăng trở lại. Với khối phần dư, mạng *rất dễ học hàm đồng nhất*: nếu các trọng số trong khối bằng 0 thì đầu ra đơn giản bằng đầu vào được sao chép qua đường tắt. Vì vậy thêm lớp gần như không bao giờ làm hại, và thường còn cải thiện. Một chi tiết: phép cộng đòi hai vế cùng kích thước, nên ResNet dùng nhiều tích chập same; khi số kênh đổi, thêm một phép chiếu nhỏ ở đường tắt để khớp chiều.



Hình 5.13: Khối phần dư dễ học hàm đồng nhất nên cho phép huấn luyện mạng rất sâu

Trong bài thực hành, ResNet50 được dựng từ hai loại khối. **Khối đồng nhất** (identity block) dùng khi đầu vào và đầu ra cùng kích thước; đường tắt sao chép thẳng. **Khối tích chập** (convolutional block) dùng khi cần đổi kích thước; đường tắt có thêm một lớp tích chập 1x1.

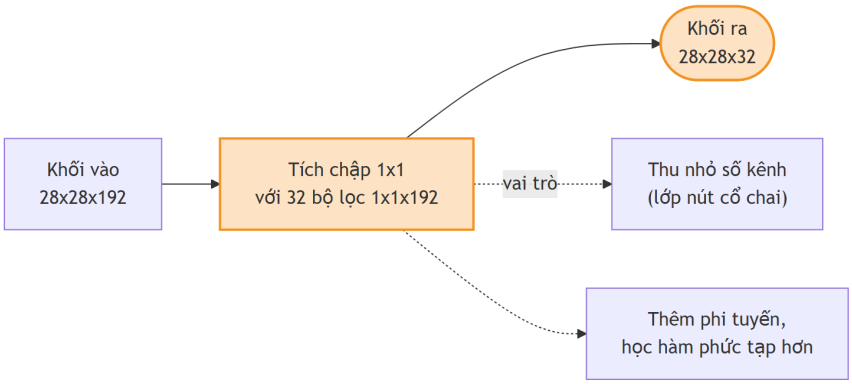
```
from tensorflow.keras.layers import Conv2D, BatchNormalization, Activation, Add
```

```
def identity_block(X, f, filters):
    F1, F2, F3 = filters
    X_shortcut = X                                  # giữ lại để cộng vào cuối
    X = Conv2D(F1, 1, strides=1, padding='valid')(X); X =
    ↪  BatchNormalization(axis=3)(X); X = Activation('relu')(X)
    X = Conv2D(F2, f, strides=1, padding='same')(X);  X =
    ↪  BatchNormalization(axis=3)(X); X = Activation('relu')(X)
    X = Conv2D(F3, 1, strides=1, padding='valid')(X); X =
    ↪  BatchNormalization(axis=3)(X)
    X = Add()([X, X_shortcut])                      # cộng đường tắt
    return Activation('relu')(X)
```

ResNet50 xếp các khối này thành năm giai đoạn, số bộ lọc tăng dần: giai đoạn 2 dùng `[64, 64, 256]`, giai đoạn 3 `[128, 128, 512]`, giai đoạn 4 `[256, 256, 1024]`, giai đoạn 5 `[512, 512, 2048]`, rồi gộp trung bình, trải phẳng và softmax. Kết quả đã lưu trong notebook (phân loại ngôn ngữ ký hiệu sáu lớp): mô hình sinh viên tự huấn luyện 10 vòng lặp đạt độ chính xác kiểm tra `0.7583`; còn mô hình đã huấn luyện sẵn (`resnet50.h5`) đạt `0.95`. Các con số lấy trực tiếp từ ô kết quả đã lưu.

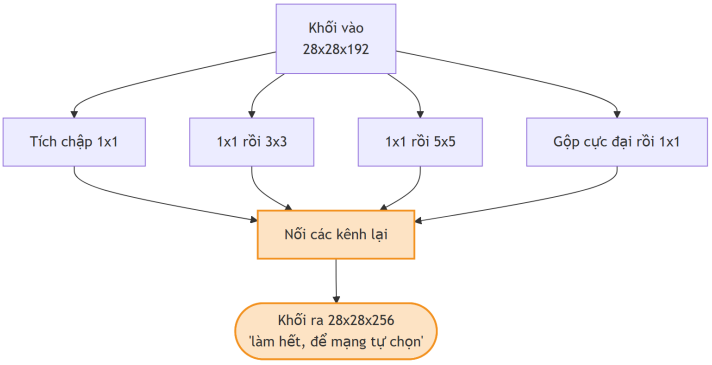
5.2.3 Tích chập 1x1 và mạng Inception

Một bộ lọc **1x1** nghe có vẻ vô dụng trên ảnh một kênh (chỉ là nhân với một số), nhưng trên khối nhiều kênh nó rất hữu ích (Hình 5.14). Ở mỗi vị trí, nó lấy toàn bộ các số dọc theo chiều kênh, trộn lại thành một giá trị – tựa như một mạng kết nối đầy đủ nhỏ áp cho từng vị trí. Hai công dụng chính: **thu nhỏ số kênh** (tạo lớp "nút cổ chai" để tiết kiệm tính toán) và **thêm phi tuyến** để học hàm phức tạp hơn.



Hình 5.14: Tích chập 1x1 trộn thông tin giữa các kênh, dùng để thu nhỏ số kênh hoặc thêm phi tuyến

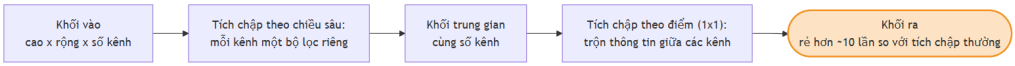
Mạng Inception xuất phát từ câu hỏi: nên chọn bộ lọc 1x1, 3x3, 5x5 hay lớp gộp? Câu trả lời của nó là *làm hết rồi nối lại*, để mạng tự chọn tổ hợp tốt nhất (Hình 5.15). Vấn đề là chi phí tính toán. Giải pháp là chèn tích chập 1x1 làm nút cổ chai *trước* các bộ lọc lớn, giảm chi phí xuống khoảng một phần mười mà hầu như không hại hiệu năng. Một mô-đun Inception vì thế gồm bốn nhánh (1x1; 1x1 rồi 3x3; 1x1 rồi 5x5; gộp rồi 1x1) được nối theo chiều kênh. Mạng GoogLeNet chỉ là xếp nhiều mô-đun như vậy, kèm vài nhánh phụ dự đoán ở giữa để chống quá khớp. Về sau có các bản Inception v2, v3, v4, và cả bản kết hợp với ý tưởng phần dư.



Hình 5.15: Mô-đun Inception: làm hết các lựa chọn bộ lọc rồi nối lại, để mạng tự chọn tổ hợp tốt nhất

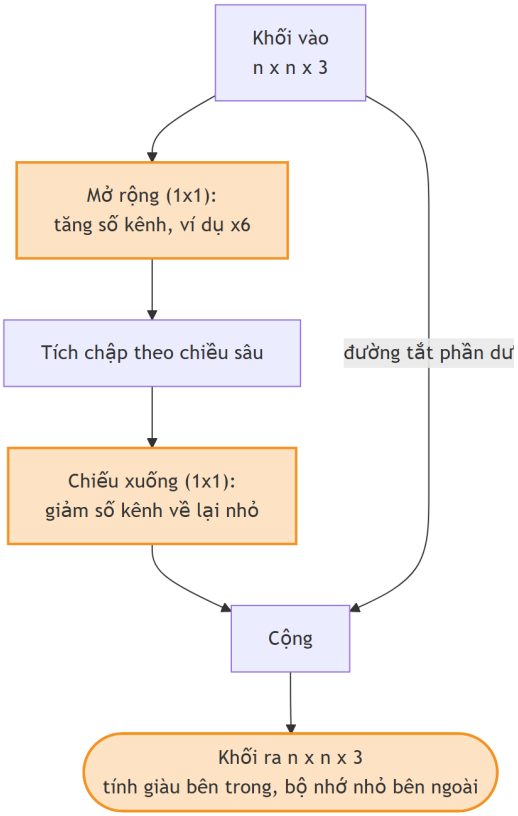
5.2.4 Mạng nhẹ: MobileNet và EfficientNet

Các mạng trên khá nặng. **MobileNet** được thiết kế để chạy trên thiết bị ít tài nguyên như điện thoại, dựa trên **tích chập tách theo chiều sâu** (depthwise separable convolution). Thay cho một phép tích chập thường, nó làm hai bước (Hình 5.16): **tích chập theo chiều sâu** (mỗi kênh một bộ lọc riêng, chỉ lọc không gian) rồi **tích chập theo điểm** 1x1 (trộn thông tin giữa các kênh). Cho cùng đầu vào và đầu ra, cách này rẻ hơn nhiều – trong ví dụ của bài giảng chỉ tốn khoảng 31% chi phí; ở mạng thực tế thường rẻ khoảng mười lần.



Hình 5.16: Tích chập tách theo chiều sâu: lọc không gian từng kênh, rồi trộn kênh bằng tích chập 1x1

MobileNet v2 cải tiến bằng khối "nút cổ chai" có đường tắt phần dư (Hình 5.17): ba bước gồm *mở rộng* số kênh bằng 1x1, rồi tích chập theo chiều sâu, rồi *chiếu* xuống lại bằng 1x1. Khối này cho mạng học hàm giàu hơn bên trong mà vẫn giữ bộ nhớ truyền giữa các lớp nhỏ.



Hình 5.17: Khối nút cổ chai của MobileNet v2: mở rộng, tích chập theo chiều sâu, chiếu xuống, cộng đường tắt

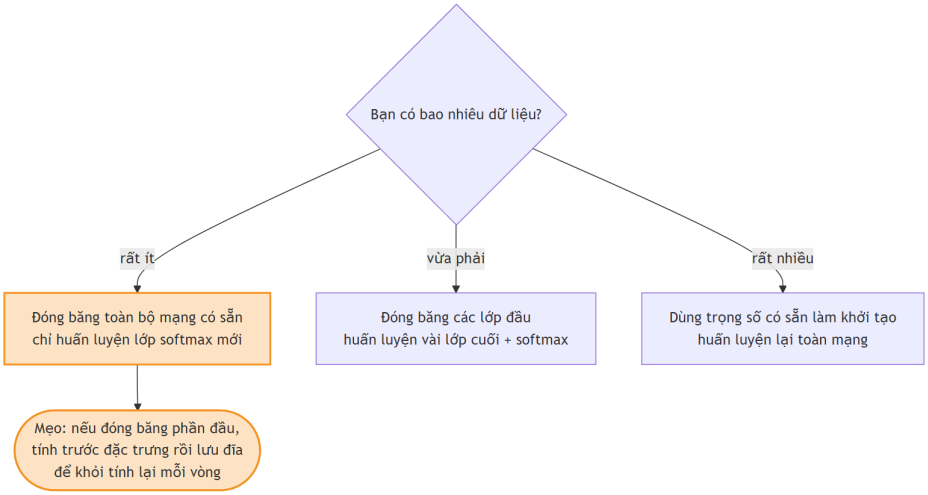
EfficientNet trả lời câu hỏi: làm sao co giãn mạng cho vừa thiết bị? Có ba cách – tăng độ phân giải ảnh, tăng độ sâu, tăng độ rộng (số kênh). EfficientNet cho một cách *co giãn hợp nhất* để chọn tỉ lệ tốt giữa ba yếu tố này trong một ngân sách tính toán cho trước. Khi cần điều chỉnh mạng cho một thiết bị cụ thể, nên dựa vào một bản cài đặt mã nguồn mở của EfficientNet.

5.2.5 Dùng mã nguồn mở

Nhiều mạng rất khó tái lập chỉ từ bài báo, vì phụ thuộc nhiều chi tiết chỉnh siêu tham số. May là giới nghiên cứu thường công bố mã và trọng số trên GitHub. Nếp làm việc thực chiến: chọn một kiến trúc đã biết, tìm bản cài đặt mã nguồn mở, `git clone` về rồi bắt đầu từ đó. Lợi ích lớn là có thể tải luôn trọng số đã huấn luyện sẵn trên dữ liệu khổng lồ – nền tảng cho học chuyển giao ở mục sau.

5.2.6 Học chuyển giao

Học chuyển giao (transfer learning) gần như luôn đáng dùng trong thị giác máy tính, trừ khi bạn có tập dữ liệu cực lớn và ngân sách tính toán dư dả. Ý tưởng: tải một mạng đã huấn luyện trên bộ dữ liệu lớn (ImageNet, MS COCO...), bỏ lớp softmax cũ, gắn lớp softmax mới cho các lớp của bạn. Lượng dữ liệu bạn có quyết định đóng băng bao nhiêu lớp (Hình 5.18): rất ít dữ liệu thì đóng băng toàn bộ mạng và chỉ huấn luyện lớp softmax; vừa phải thì đóng băng các lớp đầu và huấn luyện vài lớp cuối; rất nhiều thì dùng trọng số có sẵn làm khởi tạo và huấn luyện lại toàn mạng. Một mẹo tăng tốc: nếu phần đầu bị đóng băng, hãy tính trước đặc trưng của phần đó cho mọi ảnh rồi lưu đĩa, khỏi tính lại mỗi vòng lặp.



Hình 5.18: Chọn mức đóng băng theo lượng dữ liệu: càng nhiều dữ liệu, càng huấn luyện lại nhiều lớp

Bài thực hành dùng MobileNetV2 (huấn luyện sẵn trên ImageNet) làm bộ phân loại nhị phân "lạc đà Alpaca / không phải". Lấy mô hình nền với `include_top=False`, đóng băng nó, rồi gắn phần đầu phân loại mới:

```
import tensorflow as tf
import tensorflow.keras.layers as tfl

def alpaca_model(image_shape=(160, 160), data_augmentation=data_augmenter()):
    input_shape = image_shape + (3,)
    base_model = tf.keras.applications.MobileNetV2(input_shape=input_shape,
                                                    include_top=False,
                                                    ↪  weights='imagenet')
    base_model.trainable = False                # đóng băng mạng nền
    inputs = tf.keras.Input(shape=input_shape)
    x = data_augmentation(inputs)               # tăng cường dữ liệu
    x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
    x = base_model(x, training=False)           # giữ nguyên thống kê batch-norm
    x = tfl.GlobalAveragePooling2D()(x)
    x = tfl.Dropout(0.2)(x)
    outputs = tfl.Dense(1)(x)                   # một nơ-ron, trả logit
    return tf.keras.Model(inputs, outputs)

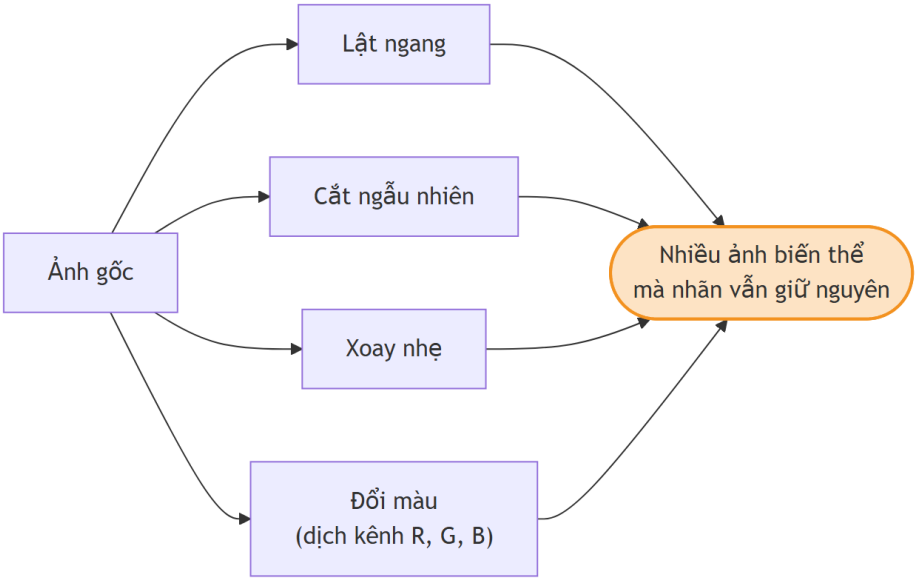
model = alpaca_model()
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
              loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
              ↪  metrics=['accuracy'])
history = model.fit(train_dataset, validation_data=validation_dataset, epochs=5)
```

Kết quả đã lưu: chỉ sau 5 vòng lặp ở giai đoạn trích đặc trưng, độ chính xác kiểm chứng đã lên tới `0.8462` (mất mát kiểm chứng `0.2834`), dù dữ liệu rất nhỏ (chỉ 327 ảnh). Đây là sức mạnh của học chuyển giao: tận dụng công sức huấn luyện của người khác trên hàng triệu ảnh.

5.2.7 Tăng cường dữ liệu

Hầu hết bài toán thị giác đều thiếu dữ liệu, nên **tăng cường dữ liệu** (data augmentation) gần như luôn giúp ích, dù bạn huấn luyện từ đầu hay dùng học chuyển giao. Ý tưởng: tạo thêm biến thể của ảnh mà *nhãn vẫn giữ nguyên* (Hình 5.19). Các phép thường dùng: **lật ngang**, **cắt ngẫu nhiên** (miễn là vùng cắt

đủ lớn), và **đổi màu** (dịch các kênh đỏ, lục, lam đi chút ít để mạng bền hơn với thay đổi ánh sáng). Xoay và làm méo cũng dùng được nhưng ít phổ biến hơn.



Hình 5.19: Tăng cường dữ liệu: tạo nhiều biến thể của ảnh mà nhãn không đổi

Trong Keras, có thể gói các phép tăng cường thành một khối `Sequential` rồi đặt ngay sau đầu vào (như đã thấy trong `alpaca_model`):

```
from tensorflow.keras.layers.experimental.preprocessing import RandomFlip,
↪  RandomRotation

def data_augmenter():
    return tf.keras.Sequential([
        RandomFlip('horizontal'),    # lật ngang ngẫu nhiên
        RandomRotation(0.2),         # xoay nhẹ ngẫu nhiên
    ])
```

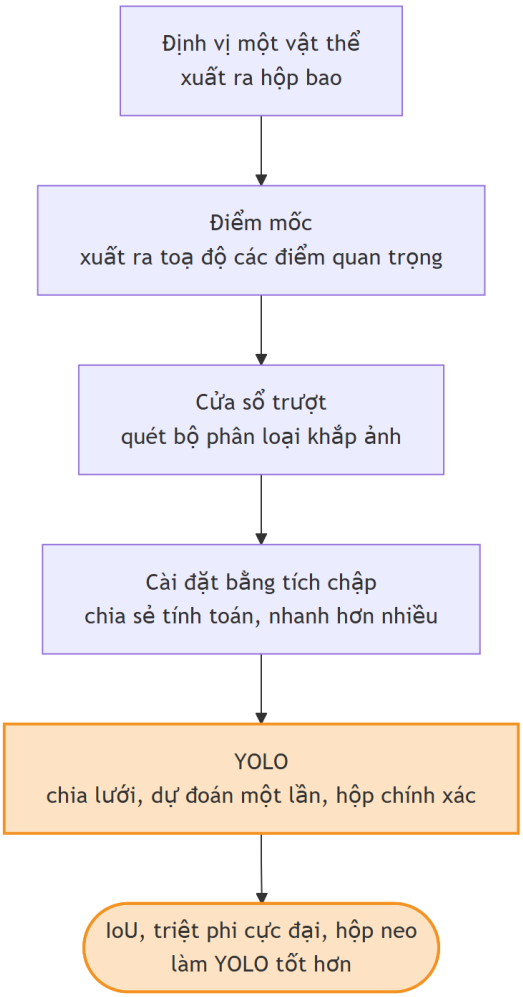
Thực tế, các phép này thường chạy trên CPU theo luồng song song với việc huấn luyện trên GPU. Giống nhiều phần khác của thị giác máy tính, nếu phân vân về siêu tham số (đổi màu bao nhiêu, cắt thế nào), nên bắt đầu từ một bản cài đặt mã nguồn mở rồi điều chỉnh.

5.2.8 Trạng thái của thị giác máy tính

Vài quan sát để định hướng. Các bài toán học máy nằm đâu đó trên trục từ *ít dữ liệu* đến *nhiều dữ liệu*. Khi nhiều dữ liệu, người ta dùng thuật toán đơn giản hơn và ít chỉnh tay hơn, để mạng tự học. Khi ít dữ liệu – như phần lớn bài toán thị giác, nhất là phát hiện vật thể – người ta chỉnh tay nhiều hơn (thiết kế đặc trưng, thiết kế kiến trúc). Đây là lý do kiến trúc thị giác thường phức tạp, và cũng là lý do học chuyển giao rất quý. Vài mẹo giúp thắng các cuộc thi điểm chuẩn như *ensembling* (huấn luyện nhiều mạng rồi lấy trung bình dự đoán) hay *multi-crop* (cắt nhiều vùng khi kiểm tra rồi lấy trung bình) thường **không** dùng trong hệ thống sản phẩm thật vì quá tốn tài nguyên. Lời khuyên thực chiến: khởi đầu từ kiến trúc và trọng số có sẵn của người khác, rồi tinh chỉnh trên dữ liệu của mình.

5.3 Phát hiện và phân vùng vật thể

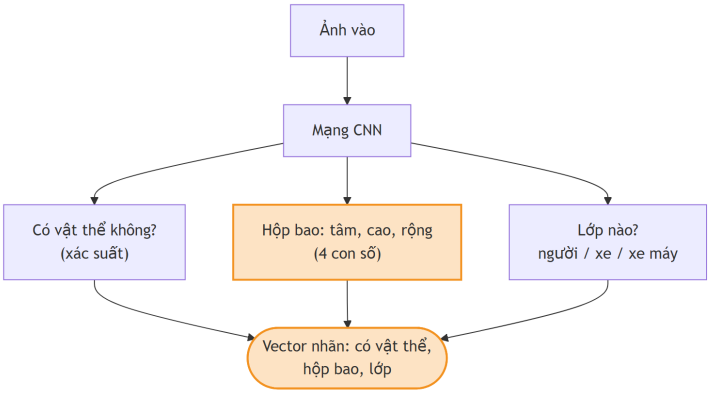
Nhận dạng ảnh chỉ trả lời *có cái gì*. Mục này tiến thêm hai bậc: phát hiện vật thể trả lời *vật ở đâu* (vẽ hộp bao), còn phân vùng ngữ nghĩa trả lời tới *từng điểm ảnh*. Lộ trình (Hình 5.20): định vị một vật → điểm mốc → cửa sổ trượt → cài đặt bằng tích chập → YOLO → IoU, triệt phi cực đại, hộp neo → đề xuất vùng → phân vùng với U-Net.



Hình 5.20: Lộ trình từ định vị một vật tới YOLO và phân vùng ngữ nghĩa

5.3.1 Định vị và điểm mốc

Phân loại kèm định vị (classification with localization) không chỉ nói "đây là xe" mà còn vẽ hộp bao quanh xe. Cách làm: cho mạng xuất thêm vài con số (Hình 5.21). Vector nhãn gồm: một số cho biết *có vật thể hay không*; bốn số mô tả **hộp bao** (toạ độ tâm, chiều cao, chiều rộng, quy ước góc trên trái là 0 và góc dưới phải là 1); và vài số cho biết *lớp nào* (người, xe, xe máy). Khi không có vật thể, các số còn lại là "không quan tâm". Hàm mất mát có thể dùng sai số bình phương; khi không có vật thể thì chỉ phạt sai ở ô "có vật thể hay không".



Hình 5.21: Định vị: mạng xuất thêm hộp bao và lớp bên cạnh "có vật thể hay không"

Ý tưởng "cho mạng xuất một chùm số thực" còn dùng cho **điểm mốc** (landmark): xuất toạ độ các điểm quan trọng trên khuôn mặt (khoé mắt, miệng, mũi) hay các khớp cơ thể (vai, khuỷu, cổ tay). Đây là nền tảng cho nhận diện cảm xúc, các bộ lọc thực tế ảo kiểu Snapchat, và nhận dạng tư thế. Yêu cầu quan trọng: nhãn của điểm mốc phải *nhất quán* giữa các ảnh (điểm số một luôn là cùng một khoé mắt).

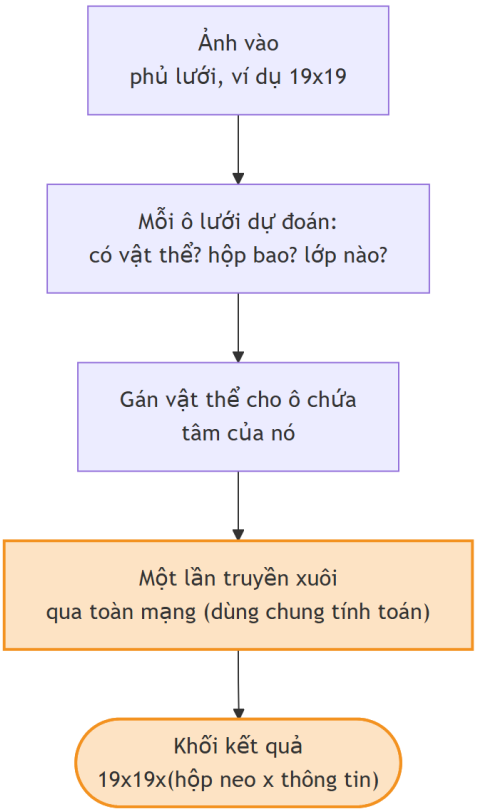
5.3.2 Cửa sổ trượt và cài đặt bằng tích chập

Cách phát hiện vật thể đơn giản là **cửa sổ trượt** (sliding windows): huấn luyện một bộ phân loại trên ảnh cắt sát vật, rồi trượt một cửa sổ khắp ảnh kiểm tra, mỗi vị trí hỏi "có xe không". Lặp lại với vài kích thước cửa sổ. Nhược điểm lớn là *chi phí tính toán*: chạy mạng độc lập trên vô số vùng.

May là có thể cài **cửa sổ trượt bằng tích chập** để chia sẻ phần lớn tính toán. Mấu chốt là biến các lớp kết nối đầy đủ thành lớp tích chập: một lớp 400 nơ-ron nối từ khối 5x5x16 chính là 400 bộ lọc 5x5x16, cho ra khối 1x1x400; các lớp sau dùng bộ lọc 1x1. Khi đó, thay vì chạy mạng riêng cho từng cửa sổ, ta chạy *một lần* trên cả ảnh lớn và thu được kết quả cho mọi vị trí cùng lúc. Dù vậy, hộp bao từ cách này vẫn chưa thật chính xác.

5.3.3 YOLO: nhìn một lần

YOLO (You Only Look Once) cho hộp bao chính xác hơn và chạy đủ nhanh cho thời gian thực. Ý tưởng (Hình 5.22): phủ một **lưới** lên ảnh (ví dụ 19x19) và áp thuật toán định vị cho *từng ô lưới*. Mỗi vật được gán cho ô chứa *tâm* của nó. Mỗi ô dự đoán một vector gồm: có vật thể hay không, hộp bao, và lớp. Toàn bộ chạy trong *một lần truyền xuôi* qua mạng, chia sẻ tính toán giữa các ô, nên rất nhanh.

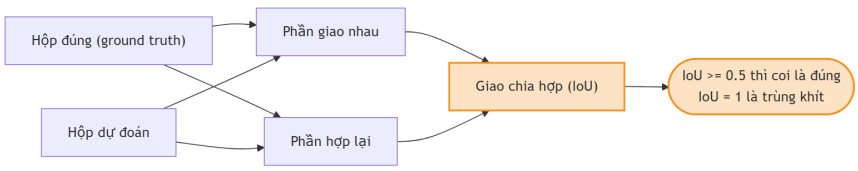


Hình 5.22: YOLO phủ lưới lên ảnh, mỗi ô dự đoán vật thể, chỉ một lần truyền xuôi qua mạng

Hộp bao trong YOLO được quy định *tương đối với ô lưới*: tâm nằm trong khoảng 0 đến 1 của ô, còn chiều cao và rộng tính theo tỉ lệ của ô (có thể lớn hơn 1 nếu vật tràn ra ngoài ô). Lưới càng mịn (19x19 thay vì 3x3) thì càng ít khả năng hai vật rơi vào cùng một ô. Bài báo YOLO khá khó đọc; không sao nếu phải xem thêm mã nguồn để hiểu chi tiết.

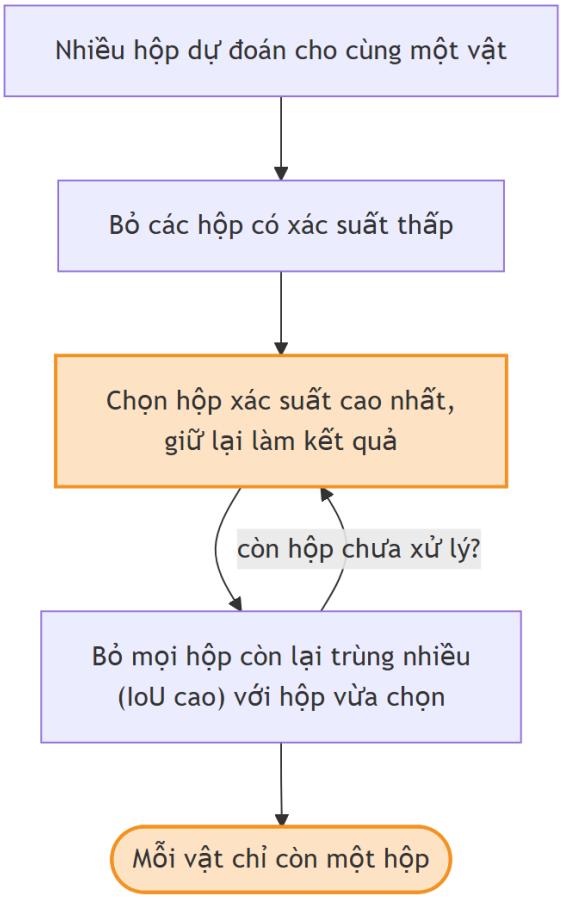
5.3.4 Đánh giá và dọn kết quả: IoU, triệt phi cực đại

Làm sao biết hộp dự đoán có tốt không? Dùng **giao chia hợp** (Intersection over Union – IoU): lấy diện tích phần giao của hai hộp chia cho diện tích phần hợp (Hình 5.23). IoU bằng 1 nghĩa là trùng khít; theo quy ước, IoU từ 0.5 trở lên thì coi hộp là đúng (có thể dùng ngưỡng chặt hơn như 0.6, 0.7). IoU cũng là thước đo độ giống nhau giữa hai hộp bất kỳ.



Hình 5.23: IoU = diện tích giao chia diện tích hợp; dùng để đánh giá và để dọn kết quả

Thuật toán thường tìm *nhiều* hộp cho cùng một vật. **Triệt phi cực đại** (non-max suppression) dọn chuyện này (Hình 5.24): bỏ các hộp xác suất thấp; chọn hộp xác suất cao nhất giữ lại; rồi bỏ mọi hộp còn lại trùng nhiều (IoU cao) với hộp vừa chọn; lặp cho tới hết. Nếu có nhiều lớp, chạy triệt phi cực đại *độc lập cho từng lớp*.



Hình 5.24: Triệt phi cực đại: giữ hộp xác suất cao nhất, loại các hộp trùng với nó, lặp lại

Trong bài thực hành (phát hiện xe cho xe tự lái, dùng YOLO huấn luyện sẵn), hàm triệt phi cực đại dựa thẳng vào `tf.image.non_max_suppression`:

```
import tensorflow as tf

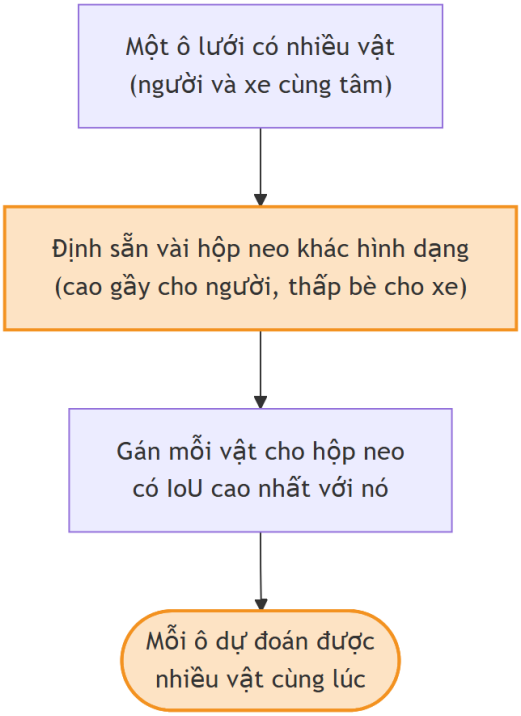
def yolo_filter_boxes(boxes, box_confidence, box_class_probs, threshold=0.6):
    box_scores = box_confidence * box_class_probs       # điểm cho mỗi lớp
    box_classes = tf.math.argmax(box_scores, axis=-1)   # lớp có điểm cao nhất
    box_class_scores = tf.math.reduce_max(box_scores, axis=-1)
    mask = box_class_scores >= threshold                # giữ hộp đủ tự tin
    return (tf.boolean_mask(box_class_scores, mask),
            tf.boolean_mask(boxes, mask),
            tf.boolean_mask(box_classes, mask))

def yolo_non_max_suppression(scores, boxes, classes, max_boxes=10,
↪   iou_threshold=0.5):
    idx = tf.image.non_max_suppression(boxes, scores, max_boxes,
    ↪   iou_threshold=iou_threshold)
    return tf.gather(scores, idx), tf.gather(boxes, idx), tf.gather(classes, idx)
```

Mô hình YOLO đã huấn luyện sẵn nhận ảnh 608x608, chia lưới 19x19, dùng 5 hộp neo và 80 lớp (bộ COCO). Kết quả đã lưu trên ảnh thử: tìm được 10 hộp, gồm nhiều "car" với độ tin cậy từ `0.89` xuống `0.37`, một "bus" `0.67` và một "traffic light" `0.36`. Các con số lấy từ ô kết quả đã lưu, không chạy lại.

5.3.5 Hộp neo

Mỗi ô lưới cơ bản chỉ phát hiện được *một* vật. **Hộp neo** (anchor box) giúp một ô phát hiện nhiều vật (Hình 5.25): định sẵn vài hình dạng hộp – ví dụ một hộp cao gầy cho người đi bộ, một hộp thấp bè cho xe. Vector nhãn của mỗi ô lặp lại một lần cho mỗi hộp neo. Mỗi vật được gán cho cặp (ô lưới, hộp neo) có IoU cao nhất với hình dạng của nó. Hộp neo không chỉ xử lý trường hợp hai vật cùng ô (hiếm khi xảy ra với lưới mịn) mà quan trọng hơn là cho mạng *chuyên môn hoá*: một số đầu ra chuyên dò vật cao gầy, số khác chuyên dò vật thấp bè. Có thể chọn hộp neo bằng tay, hoặc nâng cao hơn là dùng thuật toán phân cụm K-means trên hình dạng vật trong dữ liệu.



Hình 5.25: Hộp neo cho phép một ô lưới dự đoán nhiều vật có hình dạng khác nhau

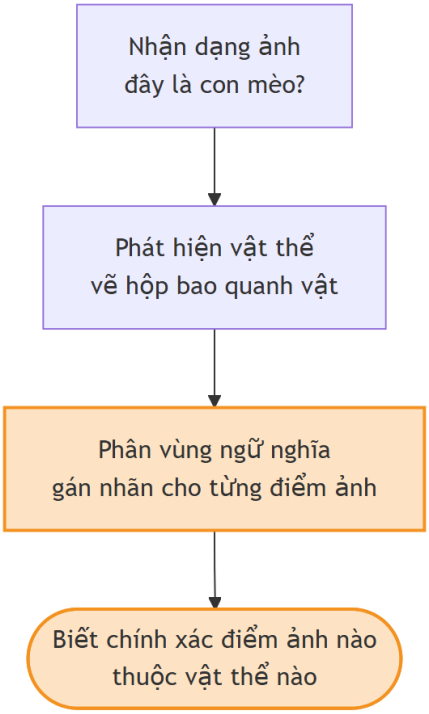
Ghép mọi thứ lại thành YOLO đầy đủ: khối kết quả có dạng lưới nhân với số hộp neo nhân với thông tin mỗi hộp (có vật thể, hộp bao, các lớp). Khi dự đoán, bỏ các hộp xác suất thấp rồi chạy triệt phi cực đại cho từng lớp.

5.3.6 Đề xuất vùng (R-CNN)

Một hướng khác là **đề xuất vùng** (region proposals). Thay vì quét mọi cửa sổ (phần lớn là nền trống), **R-CNN** chạy một thuật toán phân đoạn để chọn ra ít vùng "có thể có vật" (khoảng 2000 vùng), rồi chỉ chạy bộ phân loại trên những vùng đó. R-CNN gốc chạy chậm; *Fast R-CNN* dùng cài đặt tích chập của cửa sổ trượt để nhanh hơn; *Faster R-CNN* dùng mạng nơ-ron để đề xuất vùng. Dù vậy, các bản này thường vẫn chậm hơn YOLO. Ý tưởng R-CNN có ảnh hưởng lớn và đáng biết, nhưng xu hướng "làm mọi thứ cùng lúc" kiểu YOLO có vẻ hứa hẹn hơn về lâu dài.

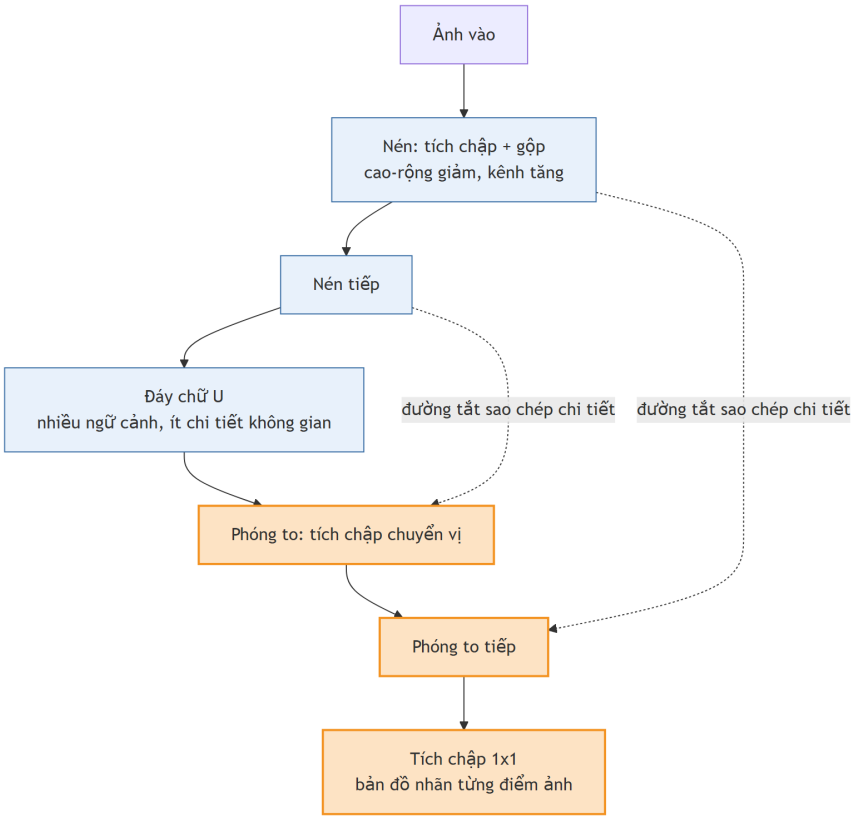
5.3.7 Phân vùng ngữ nghĩa với U-Net

Phân vùng ngữ nghĩa (semantic segmentation) gán nhãn cho *từng điểm ảnh* (Hình 5.26): điểm nào thuộc đường, điểm nào thuộc xe. Rất hữu ích cho xe tự lái (biết chính xác mặt đường chạy được) và ảnh y tế (khoanh vùng khối u, các cơ quan). Đầu ra không còn là một nhãn hay vài toạ độ, mà là *cả một ma trận nhãn*.



Hình 5.26: Ba mức: nhận dạng, phát hiện (hộp bao), phân vùng (nhãn từng điểm ảnh)

Kiến trúc **U-Net** làm việc này (Hình 5.27). Nửa đầu *nén* ảnh như một CNN bình thường (chiều cao, rộng giảm, số kênh tăng), thu được nhiều ngữ cảnh nhưng mất chi tiết không gian. Nửa sau *phóng to* biểu diễn trở lại kích thước ảnh gốc bằng **tích chập chuyển vị** (transpose convolution) – phép lấy một khối nhỏ và "thổi" nó thành khối lớn hơn. Điểm then chốt là các **đường tắt** nối thẳng lớp đầu sang lớp cuối cùng độ phân giải, mang chi tiết không gian tinh (từ lớp đầu) ghép với ngữ cảnh mức cao (từ lớp sâu), giúp quyết định chính xác từng điểm ảnh. Vẽ ra, kiến trúc trông giống chữ U – nên có tên U-Net.



Hình 5.27: U-Net: nhánh nén, nhánh phóng to bằng tích chập chuyển vị, và các đường tắt nối chi tiết

Trong bài thực hành (phân vùng ảnh lái xe CARLA, ảnh 96x128x3, 23 lớp), khối phóng to ghép tích chập chuyển vị với đường tắt rồi hai lớp tích chập:

```
from tensorflow.keras.layers import Conv2D, Conv2DTranspose, concatenate

def upsampling_block(expansive_input, contractive_input, n_filters=32):
    up = Conv2DTranspose(n_filters, 3, strides=2,
    ↪  padding='same')(expansive_input)
    merge = concatenate([up, contractive_input], axis=3)   # ghép đường tắt
    conv = Conv2D(n_filters, 3, activation='relu', padding='same',
    ↪  kernel_initializer='he_normal')(merge)
    conv = Conv2D(n_filters, 3, activation='relu', padding='same',
    ↪  kernel_initializer='he_normal')(conv)
    return conv
```

Lớp cuối là một tích chập 1x1 với số bộ lọc bằng số lớp, cho mỗi điểm ảnh một vector điểm số theo lớp; lấy lớp có điểm cao nhất là nhãn của điểm ảnh đó. Mô hình biên dịch với hàm mất mát entropy chéo theo lớp (dạng “sparse”, nhận logit):

```
unet.compile(optimizer='adam',
             loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True
             ↪  ),
             metrics=['accuracy'])
model_history = unet.fit(train_dataset, epochs=5)
```

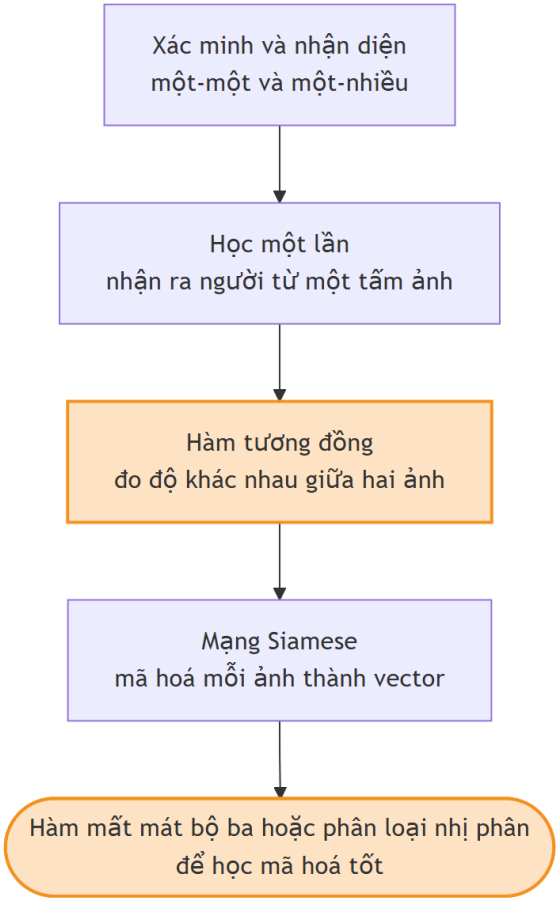
Kết quả đã lưu: độ chính xác huấn luyện tăng từ `0.4655` (vòng 1) lên `0.8791` (vòng 5). Con số lấy từ ô kết quả đã lưu, không chạy lại.

5.4 Nhận diện khuôn mặt và chuyển phong cách

Mục cuối giới thiệu hai ứng dụng chuyên biệt thú vị của CNN: **nhận diện khuôn mặt** và **chuyển phong cách** (neural style transfer). Cuối mục là một ghi chú ngắn về việc mở rộng tích chập sang dữ liệu một chiều và ba chiều.

5.4.1 Xác minh, nhận diện và học một lần

Cần phân biệt hai bài toán. **Xác minh** (verification) là bài toán một-một: cho một ảnh và một danh tính, kiểm tra xem ảnh có đúng là người đó không. **Nhận diện** (recognition) là bài toán một-nhiều: cho một ảnh, tìm xem đó là ai trong cơ sở dữ liệu. Nhận diện khó hơn nhiều: nếu hệ xác minh sai 1%, chạy trên cơ sở 100 người thì xác suất sai tăng vọt, nên cần độ chính xác rất cao. Lộ trình mục này ở Hình 5.28.

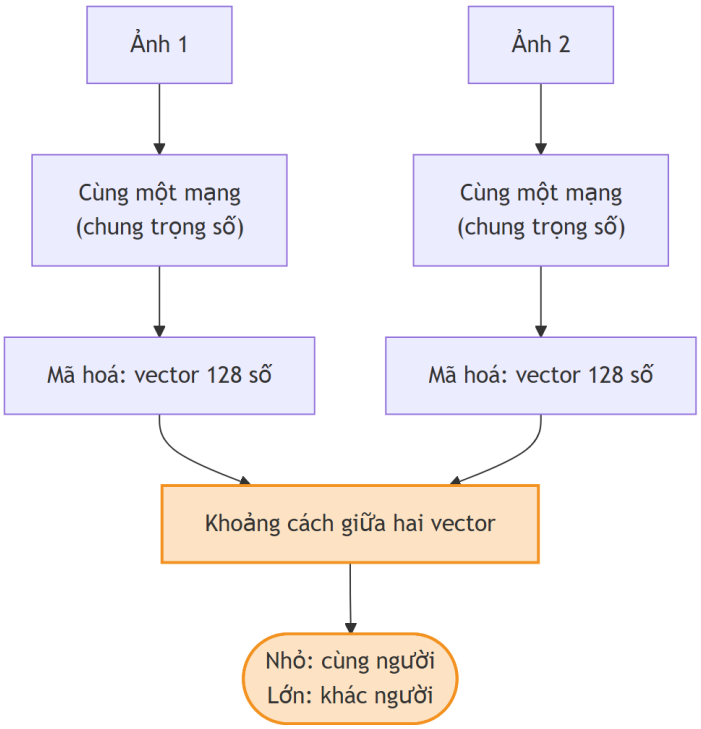


Hình 5.28: Lộ trình nhận diện khuôn mặt: từ học một lần tới mạng Siamese và hàm mất mát bộ ba

Khó khăn cốt lõi là **học một lần** (one-shot learning): thường mỗi người chỉ có *một* tấm ảnh trong cơ sở dữ liệu, mà học sâu lại cần nhiều mẫu. Nếu dùng softmax với một đầu ra cho mỗi người thì mỗi lần thêm người lại phải huấn luyện lại – không ổn. Giải pháp là *không* học phân loại trực tiếp, mà học một **hàm tương đồng**: nhận hai ảnh, trả về mức khác nhau giữa chúng. Nếu mức khác nhau nhỏ hơn một ngưỡng thì coi là cùng người. Khi thêm người mới, chỉ cần thêm ảnh vào cơ sở dữ liệu, không huấn luyện lại.

5.4.2 Mạng Siamese

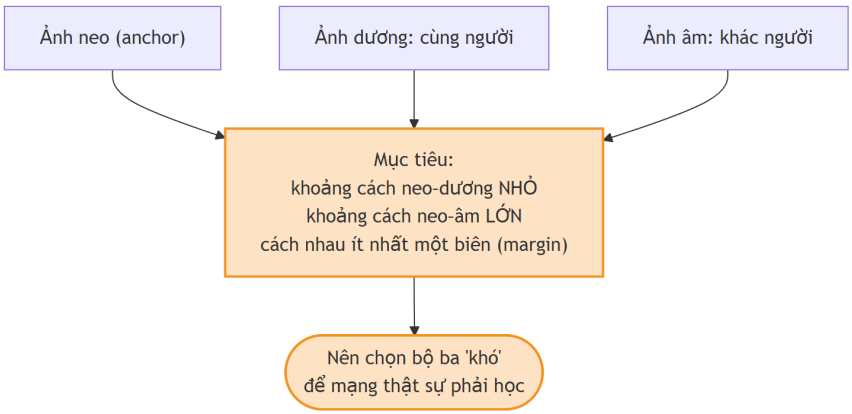
Cách hiện thực hàm tương đồng là **mạng Siamese** (Hình 5.29): cho mỗi ảnh đi qua *cùng một* mạng CNN (chung trọng số) để thu một vector **mã hoá** (encoding), ví dụ 128 số. Khoảng cách giữa hai vector mã hoá chính là mức khác nhau giữa hai ảnh. Mục tiêu huấn luyện: với hai ảnh cùng người thì khoảng cách mã hoá nhỏ, với hai ảnh khác người thì khoảng cách lớn. Hệ thống DeepFace là một ví dụ sớm của ý tưởng này.



Hình 5.29: Mạng Siamese: hai ảnh qua cùng một mạng, so khoảng cách giữa hai vector mã hoá

5.4.3 Hàm mất mát bộ ba

Để học mã hoá tốt, một cách là **hàm mất mát bộ ba** (triplet loss). Luôn nhìn ba ảnh cùng lúc (Hình 5.30): một ảnh **neo** (anchor), một ảnh **dương** (cùng người với neo), một ảnh **âm** (khác người). Mục tiêu: khoảng cách neo–dương nhỏ hơn khoảng cách neo–âm, và cách nhau ít nhất một **biên** (margin). Biên này ngăn mạng chọn lời giải tầm thường (ví dụ mã hoá mọi ảnh thành vector 0, khi đó mọi khoảng cách đều bằng nhau). Nếu khoảng cách neo–dương đã nhỏ hơn khoảng cách neo–âm đủ một biên thì mất mát bằng 0; ngược lại, mất mát dương và kéo hai khoảng cách này tách xa nhau.



Hình 5.30: Hàm mất mát bộ ba: đẩy ảnh dương lại gần neo, đẩy ảnh âm ra xa, cách nhau một biên

Một lưu ý thực chiến: nếu chọn bộ ba *ngẫu nhiên*, ảnh âm thường đã khác hẳn neo, điều kiện quá dễ thoả, mạng học được rất ít. Nên chọn các bộ ba **khó** (khoảng cách neo–dương xấp xỉ khoảng cách neo–âm) để mạng thật sự phải cố gắng. Hệ thống FaceNet dùng ý tưởng này. Trong bài thực hành, hàm mất mát bộ ba viết gọn như sau (biên mặc định 0.2):

```
import tensorflow as tf

def triplet_loss(y_true, y_pred, alpha=0.2):
    anchor, positive, negative = y_pred[0], y_pred[1], y_pred[2]
    pos_dist = tf.reduce_sum(tf.square(anchor - positive), axis=-1)   #
    ↪  neo--dương
    neg_dist = tf.reduce_sum(tf.square(anchor - negative), axis=-1)   # neo--âm
    basic_loss = pos_dist - neg_dist + alpha                          # cộng biên
    return tf.reduce_sum(tf.maximum(basic_loss, 0.0))                 # chỉ phạt
    ↪  phần dương
```

Thực tế, các hệ nhận diện khuôn mặt thương mại được huấn luyện trên hàng triệu đến hàng trăm triệu ảnh. Vì khối lượng dữ liệu khổng lồ đó, nên tải mô hình huấn luyện sẵn thay vì tự huấn luyện từ đầu. Bài thực hành dùng mô hình FaceNet huấn luyện sẵn: đầu vào ảnh 160x160x3, đầu ra mã hoá 128 số. Hàm xác minh và nhận diện so khoảng cách với ngưỡng `0.7`:

```
import numpy as np

def verify(image_path, identity, database, model):
    encoding = img_to_encoding(image_path, model)           # mã hoá ảnh mới
    dist = np.linalg.norm(encoding - database[identity])    # khoảng cách tới ảnh
    ↪  lưu sẵn
    door_open = dist < 0.7                                   # dưới ngưỡng thì mở
    ↪  cửa
    return dist, door_open

def who_is_it(image_path, database, model):
    encoding = img_to_encoding(image_path, model)
    min_dist = 100
    for (name, db_enc) in database.items():                 # duyệt cả cơ sở dữ
    ↪  liệu
        dist = np.linalg.norm(encoding - db_enc)
        if dist < min_dist:
            min_dist, identity = dist, name                 # giữ người gần nhất
    return min_dist, (identity if min_dist < 0.7 else None)
```

Kết quả đã lưu: hàm mất mát bộ ba trên bộ ba thử cho giá trị `527.2598`; khi xác minh một người đúng, khoảng cách là `0.5992949` và cửa mở; với người sai, khoảng cách `1.0259346` và cửa không mở. Hàm nhận diện cũng tìm đúng người với khoảng cách `0.5992949`. Các con số lấy từ ô kết quả đã lưu.

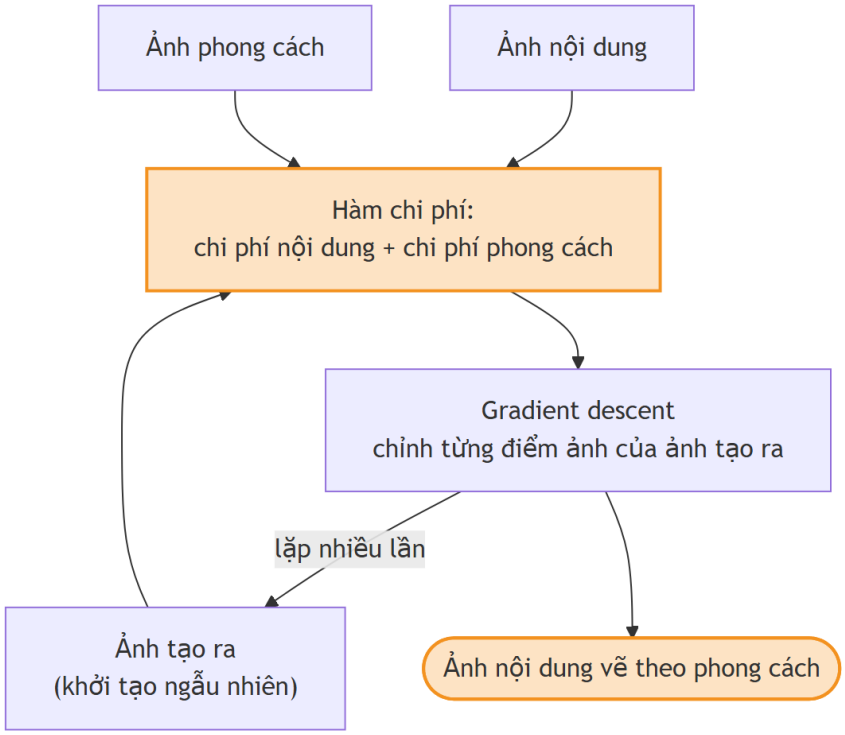
Một mẹo triển khai: có thể *tính trước* mã hoá của mọi người trong cơ sở dữ liệu và lưu lại, nên mỗi lần có người tới chỉ cần tính mã hoá ảnh mới rồi so sánh. Ngoài hàm mất mát bộ ba, còn một cách khác là coi xác minh như bài toán phân loại nhị phân: đưa hiệu giữa hai vector mã hoá vào một nơ-ron logistic để trả về cùng người hay khác người.

5.4.4 CNN sâu học được gì

Trước khi sang chuyển phong cách, cần trực giác về việc các lớp của CNN học gì. Bằng cách tìm các mảnh ảnh làm một nơ-ron kích hoạt mạnh nhất, người ta thấy: lớp *nông* học những đặc trưng đơn giản như cạnh và mảng màu; lớp *sâu hơn* học kết cấu phức tạp hơn (vằn dọc, hình tròn); lớp *sâu nhất* học cả vật thể hoàn chỉnh (mặt chó, bàn phím, bông hoa). Nói cách khác, CNN tự xây một **phân cấp đặc trưng** từ thô tới tinh. Chính trực giác này là nền tảng của chuyển phong cách.

5.4.5 Chuyển phong cách

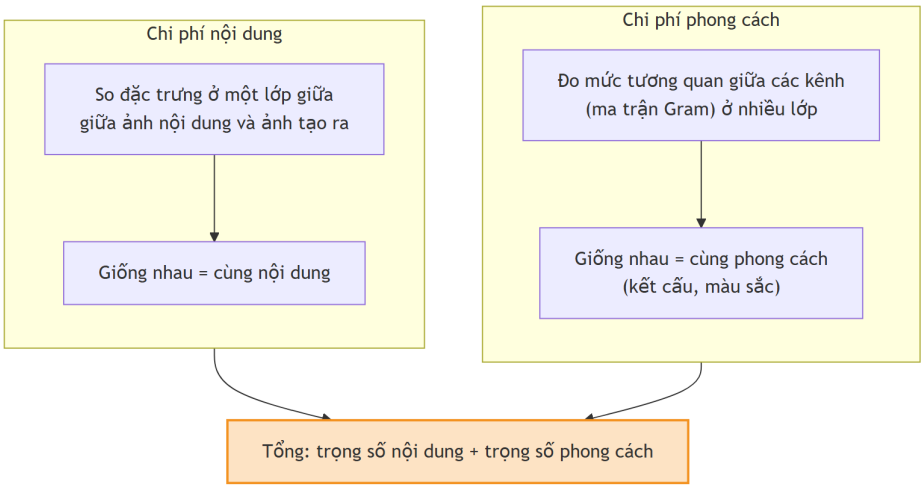
Chuyển phong cách vẽ lại một ảnh **nội dung** (content) theo **phong cách** (style) của một ảnh khác – ví dụ chụp khuôn viên đại học rồi vẽ lại theo tranh "Đêm đầy sao" của Van Gogh. Cách làm độc đáo: không huấn luyện mạng mới, mà *tối ưu thẳng vào ảnh kết quả* (Hình 5.31). Khởi tạo một ảnh ngẫu nhiên, định nghĩa một hàm chi phí đo ảnh kết quả tốt tới đâu, rồi dùng gradient descent chỉnh *từng điểm ảnh* của nó cho tới khi vừa giống nội dung của ảnh này vừa giống phong cách của ảnh kia.



Hình 5.31: Chuyển phong cách: tối ưu thẳng vào ảnh kết quả bằng gradient descent trên một hàm chi phí

Hàm chi phí gồm hai phần, cân bằng bằng hai trọng số (Hình 5.32). **Chi phí nội dung** so sánh đặc trưng ở một lớp giữa của một mạng CNN huấn luyện sẵn (thường là VGG-19) giữa ảnh nội dung và ảnh kết quả; giống nhau nghĩa là cùng nội dung. **Chi phí phong cách** đo *mức tương quan giữa các kênh* trong một lớp – gọi là **ma trận Gram** – trên nhiều lớp; tương quan này nắm bắt kết cấu và màu sắc hay

đi cùng nhau, tức là "phong cách". So ma trận Gram của ảnh phong cách với của ảnh kết quả cho biết hai phong cách giống nhau tới đâu.



Hình 5.32: Hàm chi phí chuyển phong cách: chi phí nội dung so đặc trưng, chi phí phong cách so ma trận Gram

Trong bài thực hành (VGG-19 huấn luyện sẵn, ảnh 400x400), ma trận Gram và chi phí phong cách một lớp viết như sau; chi phí phong cách tổng là tổng có trọng số trên nhiều lớp (mỗi lớp trọng số 0.2):

```
import tensorflow as tf

def gram_matrix(A):
    return tf.matmul(A, tf.transpose(A))              # tương quan giữa các kênh

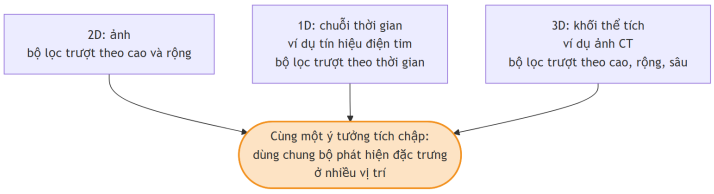
def compute_layer_style_cost(a_S, a_G):
    _, n_H, n_W, n_C = a_G.get_shape().as_list()
    a_S = tf.transpose(tf.reshape(a_S, [n_H * n_W, n_C]))
    a_G = tf.transpose(tf.reshape(a_G, [n_H * n_W, n_C]))
    GS, GG = gram_matrix(a_S), gram_matrix(a_G)         # ma trận Gram hai ảnh
    return tf.reduce_sum(tf.square(GS - GG)) / (4.0 * (n_C ** 2) * ((n_H * n_W)
    ↪  ** 2))

@tf.function()
def total_cost(J_content, J_style, alpha=10, beta=40):
    return alpha * J_content + beta * J_style           # cân bằng nội dung và
    ↪  phong cách
```

Vòng lặp huấn luyện dùng `tf.GradientTape` để tính gradient của hàm chi phí *theo ảnh*, rồi cập nhật ảnh bằng bộ tối ưu Adam. Kết quả đã lưu từ các ô kiểm tra: chi phí nội dung thử `7.056877`; chi phí phong cách một lớp `14.01649`; tổng chi phí thử `32.9832`. Các con số lấy từ ô kết quả đã lưu, không chạy lại. Bài giảng cho biết chạy khoảng 2500 vòng lặp mất chừng ba phút trên GPU.

5.4.6 Tích chập trên dữ liệu một chiều và ba chiều

Dù phần lớn ví dụ dùng ảnh (dữ liệu hai chiều), ý tưởng tích chập mở rộng tự nhiên sang dữ liệu khác (Hình 5.33). Với **dữ liệu một chiều** như tín hiệu điện tim (chuỗi điện áp theo thời gian), bộ lọc một chiều trượt dọc theo trục thời gian để dò các mẫu như từng nhịp tim. Với **dữ liệu ba chiều** như ảnh CT (một khối thể tích gồm nhiều lát cắt), bộ lọc ba chiều trượt theo cả chiều cao, rộng và sâu để dò đặc trưng trong không gian ba chiều; dữ liệu phim (nhiều khung hình theo thời gian) cũng có thể coi là khối ba chiều.



Hình 5.33: Cùng ý tưởng tích chập áp cho dữ liệu một chiều (tín hiệu) và ba chiều (khối thể tích)

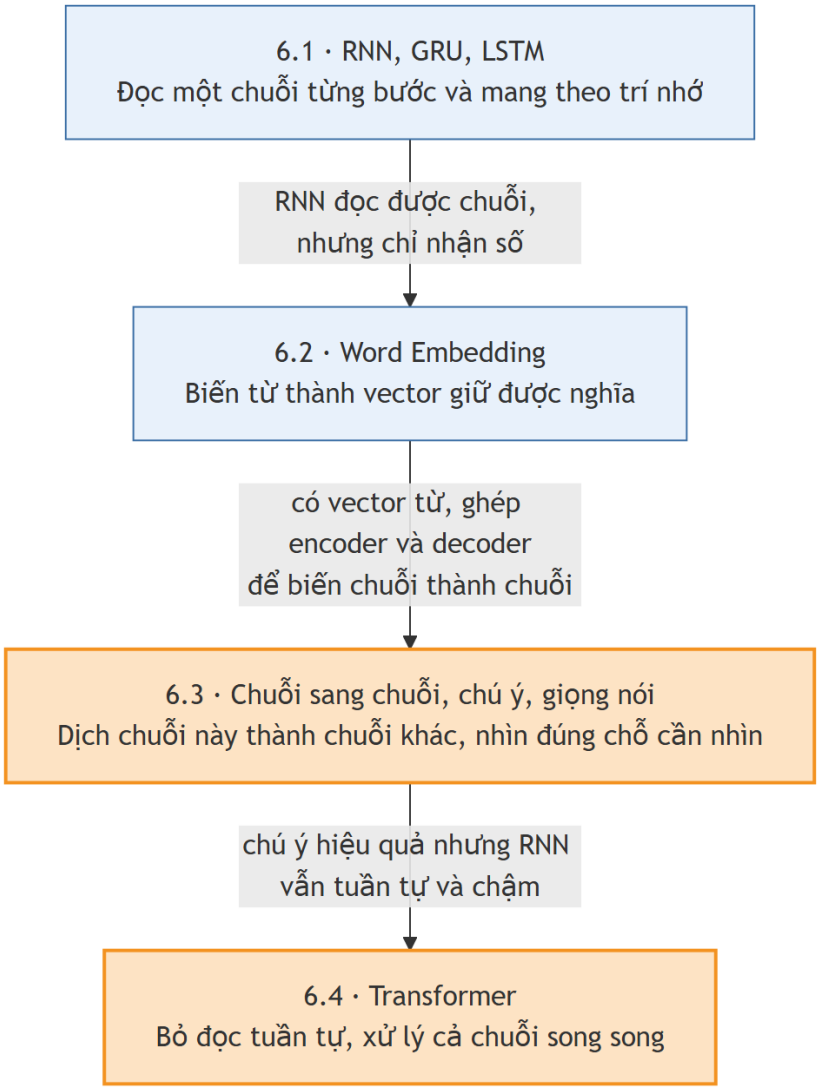
Điểm chung xuyên suốt: tích chập là cách *dùng chung một bộ phát hiện đặc trưng ở nhiều vị trí*, bất kể dữ liệu một, hai hay ba chiều. Chính nguyên lý này – dùng chung tham số và kết nối thưa – đã đưa CNN từ nhận dạng ảnh tới phát hiện vật thể, phân vùng, nhận diện khuôn mặt và sáng tạo nghệ thuật.

Chương 6

Sequence Models

Rất nhiều dữ liệu trong thực tế là **chuỗi**: câu văn, đoạn ghi âm, bản nhạc, chuỗi giá cổ phiếu, log hệ thống. Điểm chung là *thứ tự* mang ý nghĩa và độ dài thay đổi theo từng mẫu. **Sequence Models** (mô hình chuỗi) là họ mô hình được thiết kế cho loại dữ liệu này, và là nền tảng của dịch máy, nhận dạng giọng nói, chatbot và hầu hết các ứng dụng xử lý ngôn ngữ tự nhiên (NLP) hiện nay.

Chương này đi một mạch qua bốn mục. Mỗi mục trả lời một câu hỏi, và câu trả lời của mục trước dẫn đến câu hỏi của mục sau (Hình 6.1).



Hình 6.1: Lộ trình của chương Sequence Models

Mục	Câu hỏi	Nội dung chính
6.1 Mạng nơ-ron hồi quy	Làm sao để mô hình đọc và nhớ một chuỗi?	RNN, GRU, LSTM, mô hình ngôn ngữ, sinh chuỗi
6.2 NLP và Word Embedding	Làm sao biến từ thành số mà vẫn giữ được nghĩa?	embedding, Word2Vec, GloVe, phân loại cảm xúc, thiên kiến
6.3 Chuỗi sang chuỗi, chú ý, giọng nói	Làm sao biến chuỗi này thành chuỗi khác và nhìn đúng chỗ cần nhìn?	encoder–decoder, beam search, BLEU, attention, CTC, từ khoá đánh thức
6.4 Transformer	Làm sao bỏ đọc tuần tự để xử lý cả chuỗi song song?	self-attention, multi-head attention, mã hoá vị trí, mô hình huấn luyện sẵn

Bảng 6.1: Bốn mục của chương Sequence Models

Nên đọc theo thứ tự, vì các mục sau dùng lại khái niệm của các mục trước. Nếu đã quen RNN và

embedding, có thể nhảy thẳng tới mục 6.3 hoặc mục 6.4.

6.1 Mạng nơ-ron hồi quy (RNN)

Rất nhiều dữ liệu trong thực tế là **chuỗi**: câu văn, đoạn ghi âm, bản nhạc, chuỗi giá cổ phiếu, log hệ thống, chuỗi DNA. Điểm chung là *thứ tự* mang ý nghĩa và độ dài thay đổi theo từng mẫu. Mạng nơ-ron hồi quy (Recurrent Neural Network – RNN) được thiết kế cho đúng loại dữ liệu này: nó đọc từng phần tử một, vừa đọc vừa cập nhật một "trí nhớ" tóm tắt những gì đã thấy.

Mục này đi theo lộ trình: vì sao cần RNN → RNN hoạt động thế nào → các kiểu kiến trúc → huấn luyện và các vấn đề gradient → GRU, LSTM → RNN hai chiều, RNN sâu → mô hình ngôn ngữ và sinh chuỗi → hai ví dụ thực hành → checklist áp dụng vào dự án. Code dùng TensorFlow/Keras; những đoạn tự cài đặt bằng NumPy chỉ nhằm hiểu bản chất, không phải để dùng trong dự án.

6.1.1 Vì sao cần mô hình chuỗi

Bài toán	Đầu vào → đầu ra	Ghi chú
Nhận dạng giọng nói	âm thanh → câu chữ	cả hai phía đều là chuỗi
Sinh nhạc	(trống / thể loại) → chuỗi nốt	đầu vào có thể rỗng
Phân loại cảm xúc	câu review → 1–5 sao	đầu ra chỉ một giá trị
Dịch máy	câu tiếng Pháp → câu tiếng Anh	hai chuỗi khác độ dài
Nhận dạng thực thể (NER)	câu → nhãn cho từng từ	độ dài vào bằng độ dài ra
Phân tích DNA	chuỗi AGCT → vùng mã hoá protein	chuỗi rất dài

Bảng 6.2: Một số bài toán dữ liệu chuỗi

Ví dụ xuyên suốt mục này: nhận dạng tên người trong câu *"Harry Potter and Hermione Granger invented a new spell"*. Mỗi từ cần một nhãn 1 (là tên người) hoặc 0.

6.1.1.1 Mạng nơ-ron thông thường không hợp với chuỗi

Có thể thử đưa cả 9 từ (mỗi từ là một vector one-hot 10.000 chiều) vào một mạng nơ-ron truyền thẳng (feed-forward). Cách này gặp ba vấn đề:

- **Độ dài thay đổi** – câu dài ngắn khác nhau. Đệm số 0 cho bằng độ dài tối đa chỉ là chắp vá.
- **Không chia sẻ kiến thức giữa các vị trí** – học được "Harry" ở vị trí 1 là tên người, mạng không tự suy ra điều đó khi "Harry" nằm ở vị trí 5. Ta muốn hiệu ứng giống CNN: đặc trưng học ở một vùng ảnh dùng lại được cho vùng khác.
- **Quá nhiều tham số** – lớp đầu vào cỡ (số từ tối đa nhân 10.000), kéo theo ma trận trọng số khổng lồ.

RNN giải quyết cả ba bằng một ý tưởng: **dùng lại cùng một "ô" xử lý cho mọi vị trí**, và truyền trí nhớ từ vị trí này sang vị trí kế tiếp.

6.1.2 RNN cơ bản

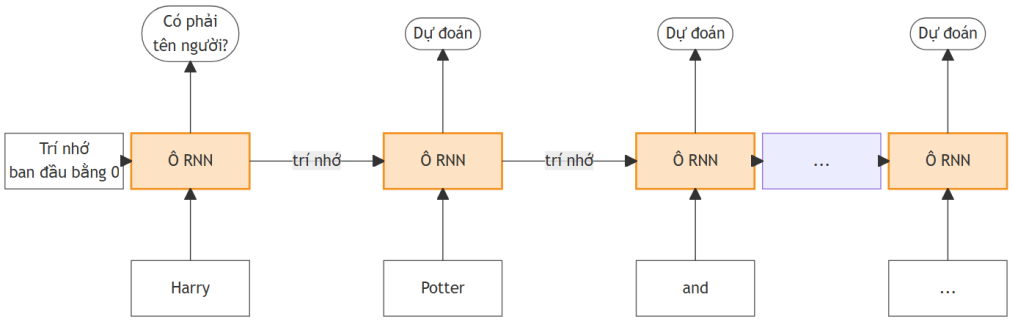
6.1.2.1 Đọc từ trái sang phải, mang theo trí nhớ

RNN đọc câu giống người đọc: từng từ một, từ trái sang phải. Ở mỗi bước thời gian:

1. Nhận từ hiện tại và trí nhớ của bước trước (gọi là **trạng thái ẩn** – hidden state).

2. Trộn hai thông tin này để ra trí nhớ mới.
3. Từ trí nhớ mới, đưa ra dự đoán (từ này có phải tên người không).
4. Chuyển trí nhớ mới sang bước kế tiếp.

Trí nhớ ban đầu thường là vector toàn số 0. Hình 6.2 vẽ RNN ở dạng **trải ra theo thời gian** (unrolled): mỗi ô là *cùng một* ô RNN, chỉ khác thời điểm. Sách và bài báo hay vẽ dạng "cuộn" – một ô có mũi tên vòng về chính nó; khi gặp dạng đó, hãy hình dung trải nó ra như hình dưới.



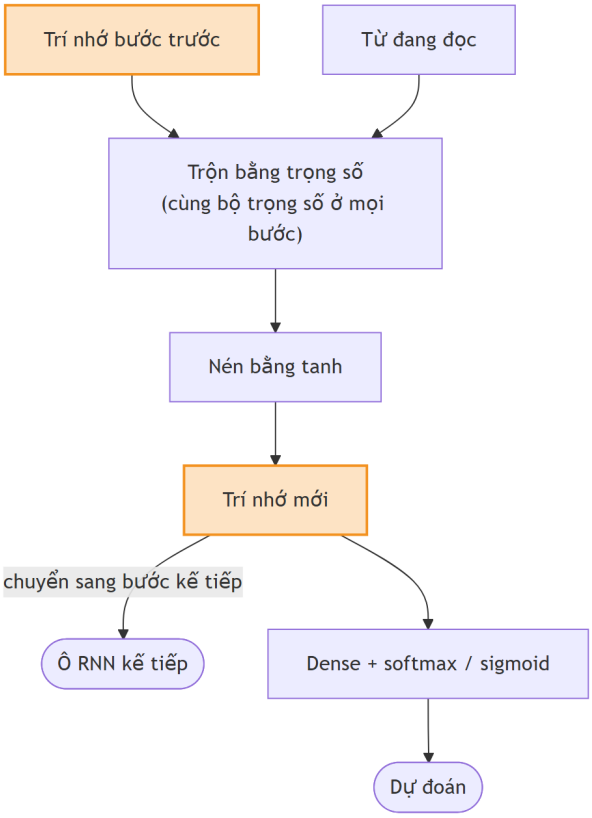
Hình 6.2: RNN trải ra theo thời gian – cùng một ô, cùng bộ tham số, lặp lại cho từng từ trong câu

6.1.2.2 Bên trong một ô RNN

Một ô RNN chỉ làm hai việc (Hình 6.3): trộn trí nhớ cũ với từ đang đọc rồi nén qua hàm `tanh` để ra trí nhớ mới; sau đó đưa trí nhớ mới qua một lớp `Dense` để đưa ra dự đoán. Hãy hình dung một người vừa đọc vừa ghi chú: đọc xong mỗi từ thì cập nhật cuốn sổ tóm tắt, rồi dựa vào cuốn sổ để trả lời.

Ba điểm cần nhớ:

- **Tham số dùng chung** – mọi bước dùng cùng một bộ trọng số. Đây là lý do RNN có ít tham số và học được "Harry là tên" bất kể vị trí.
- **Hàm kích hoạt** – trạng thái ẩn thường dùng `tanh`; đầu ra dùng `sigmoid` (phân loại nhị phân) hoặc `softmax` (nhiều lớp).
- **Số đơn vị ẩn (units)** – quyết định "cuốn sổ" dày đến đâu. Nhiều đơn vị thì nhớ được nhiều hơn nhưng tốn tham số và thời gian hơn. Trong Keras chỉ cần chọn một con số, ví dụ `units=100`; framework tự dựng các ma trận trọng số.



Hình 6.3: Luồng xử lý bên trong một ô RNN

6.1.2.3 Hạn chế: chỉ nhìn được quá khứ

RNN một chiều khi dự đoán một từ chỉ dựa trên các từ *đứng trước*. Xét hai câu:

- *"He said Teddy Roosevelt was a great president"* – Teddy là tên người.
- *"He said Teddy bears are on sale"* – Teddy không phải tên người.

Ba từ đầu giống hệt nhau, nên RNN một chiều không phân biệt được. Giải pháp là RNN hai chiều (mục 6.1.8).

6.1.2.4 Quy ước kích thước tensor

Đây là chỗ hay nhầm nhất khi chuyển từ code NumPy tự cài đặt sang framework. Code NumPy thường xếp batch ở giữa, còn Keras/PyTorch (với `batch_first=True`) xếp batch ở đầu.

Đại lượng	NumPy tự cài	Keras
Đầu vào	`(đặc trưng, batch, số bước)`	`(batch, timesteps, features)`
Trạng thái ẩn	`(đơn vị ẩn, batch, số bước)`	`(batch, timesteps, units)` nếu `return_sequences=True`, ngược lại `(batch, units)`
Dự đoán	`(số lớp, batch, số bước)`	`(batch, timesteps, classes)`

Bảng 6.3: Thứ tự các chiều của tensor: batch là số mẫu xử lý cùng lúc, số bước là độ dài chuỗi

6.1.2.5 Thử nhanh với Keras

Trong Keras, cả vòng lặp qua thời gian nằm gọn trong một lớp `SimpleRNN`. Hai tham số quan trọng nhất cần hiểu ngay từ đầu:

- `return_sequences` – `True`: trả về trạng thái ẩn ở *mọi* bước (dùng khi cần nhãn từng bước hoặc xếp chồng lớp RNN). `False` (mặc định): chỉ trả về bước cuối (dùng khi cần một nhãn cho cả chuỗi).
- `return_state` – `True`: trả thêm trạng thái cuối để truyền tiếp cho lớp/mô hình khác (encoder–decoder, sinh chuỗi).

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

x = np.random.rand(32, 10, 8).astype("float32")   # 32 mẫu, 10 bước, 8 đặc trưng
↪  mỗi bước

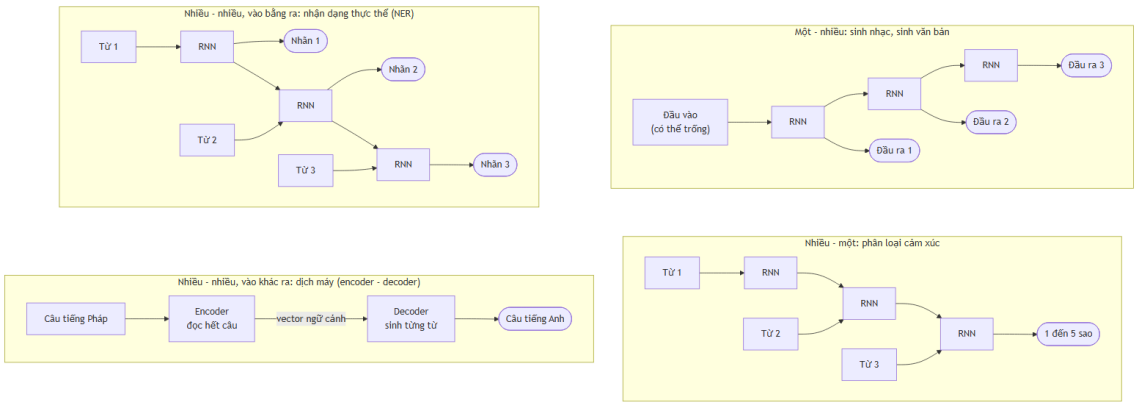
print(layers.SimpleRNN(16)(x).shape)                          # (32, 16)      chỉ
↪  bước cuối
print(layers.SimpleRNN(16, return_sequences=True)(x).shape)  # (32, 10, 16)  mọi
↪  bước

seq, last_state = layers.SimpleRNN(16, return_sequences=True,
↪  return_state=True)(x)
print(seq.shape, last_state.shape)                            # (32, 10, 16) (32,
↪  16)
```

Nếu tự cài đặt bằng NumPy, ta viết `rnn_cell_forward` cho một bước rồi gọi nó trong vòng `for` qua từng bước thời gian – đó chính là những gì `SimpleRNN` làm bên trong. Điều cần ghi nhớ: **RNN về bản chất là việc dùng lặp lại một ô duy nhất**; số bước thời gian quyết định số lần dùng lại ô đó.

6.1.3 Các kiểu kiến trúc RNN

Độ dài chuỗi đầu vào và chuỗi đầu ra không nhất thiết bằng nhau. Chỉ cần thay đổi *chỗ lấy đầu ra* và *chỗ đưa đầu vào*, cùng một ô RNN phục vụ được nhiều loại bài toán (Hình 6.4, Bảng 6.4).



Hình 6.4: Bốn kiểu kiến trúc RNN thường gặp

Kiểu	Ví dụ	Cách dựng trong Keras
Một – một	phân loại ảnh	mạng thường, không cần RNN
Một – nhiều	sinh nhạc, sinh tên	vòng lặp suy luận tự viết, đầu ra bước trước làm đầu vào bước sau (mục 6.1.11)
Nhiều – một	phân loại cảm xúc	`LSTM(units)` rồi `Dense`
Nhiều – nhiều (vào bằng ra)	NER, gán nhãn từ loại	`LSTM(units, return_sequences=True)` rồi `Dense`
Nhiều – nhiều (vào khác ra)	dịch máy	encoder trả `return_state`, decoder nhận làm `initial_state`

Bảng 6.4: Kiểu kiến trúc và cách hiện thực

6.1.3.1 Nhiều – một: phân loại cảm xúc

RNN đọc hết câu review, chỉ lấy trạng thái ở bước cuối (đã "tóm tắt" cả câu) để phân loại.

```
VOCAB, MAXLEN = 20000, 200

vectorize = layers.TextVectorization(max_tokens=VOCAB,
↪  output_sequence_length=MAXLEN)
vectorize.adapt(train_texts)                # train_texts: list[str], học từ
↪  điển

model = keras.Sequential([
    keras.Input(shape=(1,), dtype="string"),
    vectorize,                              # câu -> dãy chỉ số, đệm 0 cho đủ
    ↪  MAXLEN
    layers.Embedding(VOCAB, 128, mask_zero=True),  # chỉ số -> vector; bỏ qua
    ↪  phần đệm
    layers.LSTM(64),                        # nhiều -> một: chỉ trạng thái cuối
    layers.Dense(1, activation="sigmoid"),  # tích cực / tiêu cực
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(train_texts, train_labels, validation_split=0.1, epochs=5,
↪  batch_size=64)
```

6.1.3.2 Nhiều – nhiều cùng độ dài: gán nhãn từng từ

Chỉ khác ví dụ trên ở `return_sequences=True`: mỗi bước có một trạng thái ẩn, mỗi trạng thái đi qua cùng một lớp `Dense` để ra nhãn của từ đó.

```
NUM_TAGS = 9   # ví dụ: O, B-PER, I-PER, B-LOC, ...

model = keras.Sequential([
    keras.Input(shape=(None,), dtype="int32"),   # None: độ dài chuỗi tuỳ ý
    layers.Embedding(VOCAB, 128, mask_zero=True),
    layers.LSTM(64, return_sequences=True),      # (batch, T, 64)
    layers.Dense(NUM_TAGS, activation="softmax") # áp dụng cho từng bước ->
    ↪  (batch, T, NUM_TAGS)
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
↪  metrics=["accuracy"])
```

6.1.3.3 Nhiều – nhiều khác độ dài: encoder – decoder

Encoder đọc hết câu nguồn và nén thành trạng thái cuối; decoder nhận trạng thái đó làm trí nhớ khởi đầu rồi sinh câu đích. Khung tối thiểu với Functional API:

```
SRC_VOCAB, TGT_VOCAB, UNITS = 8000, 8000, 256

enc_in = keras.Input(shape=(None,), name="src")
x = layers.Embedding(SRC_VOCAB, 128, mask_zero=True)(enc_in)
_, h, c = layers.LSTM(UNITS, return_state=True)(x)       # chỉ giữ trạng thái
↪  cuối

dec_in = keras.Input(shape=(None,), name="tgt_in")       # câu đích lệch phải 1
↪  bước (bắt đầu bằng <start>)
y = layers.Embedding(TGT_VOCAB, 128, mask_zero=True)(dec_in)
y = layers.LSTM(UNITS, return_sequences=True)(y, initial_state=[h, c])
out = layers.Dense(TGT_VOCAB, activation="softmax")(y)

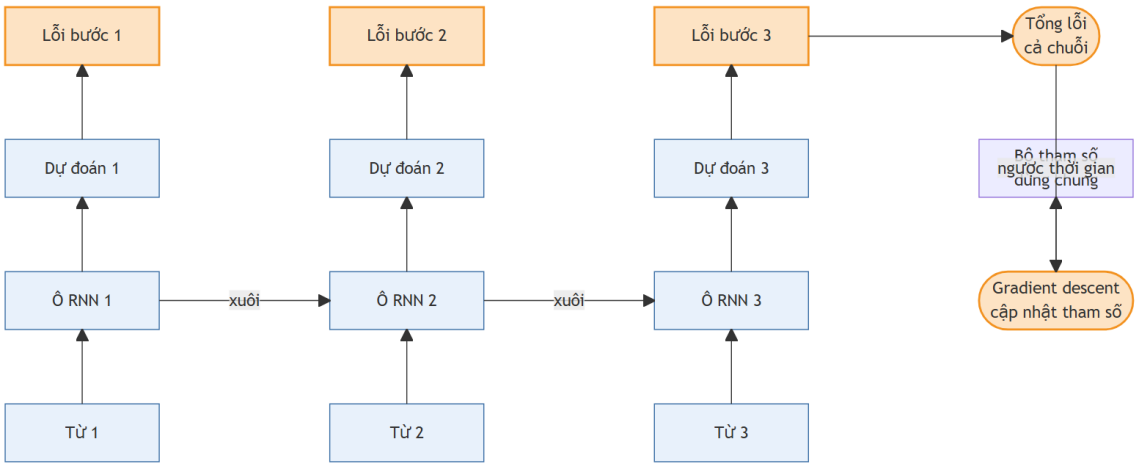
seq2seq = keras.Model([enc_in, dec_in], out)
seq2seq.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
```

Kiến trúc này là nền móng của dịch máy; phiên bản tốt hơn bổ sung cơ chế chú ý (attention), sẽ trình bày ở mục 6.3.6.

6.1.4 Huấn luyện RNN: lan truyền ngược theo thời gian

Framework tự tính gradient, nên trong dự án hiếm khi phải tự viết lan truyền ngược. Dù vậy, hiểu luồng này giúp giải thích vì sao RNN khó học phụ thuộc xa (mục 6.1.5).

1. **Lan truyền xuôi** – đi từ trái sang phải, tính trạng thái ẩn và dự đoán ở mọi bước.
2. **Tính lỗi** – mỗi bước có một lỗi riêng (cross-entropy giữa dự đoán và nhãn thật); lỗi của cả chuỗi là *tổng* lỗi các bước.
3. **Lan truyền ngược** – đi theo chiều ngược lại, từ phải sang trái, như "đi lùi theo thời gian". Vì thế thuật toán có tên **Backpropagation Through Time (BPTT)**.
4. **Cập nhật** – vì tham số dùng chung, gradient từ mọi bước được cộng dồn vào cùng một bộ tham số, rồi cập nhật bằng gradient descent (hoặc Adam).



Hình 6.5: BPTT – xuôi từ trái sang phải để tính lỗi, ngược từ phải sang trái để tính gradient

Khi tự cài đặt bằng NumPy, phải viết `rnn_backward` và `lstm_backward` bằng tay – phần này mang tính tham khảo. Trong Keras, `model.fit` làm toàn bộ bốn bước trên.

6.1.5 Tiêu biến và bùng nổ gradient

6.1.5.1 Vấn đề phụ thuộc xa

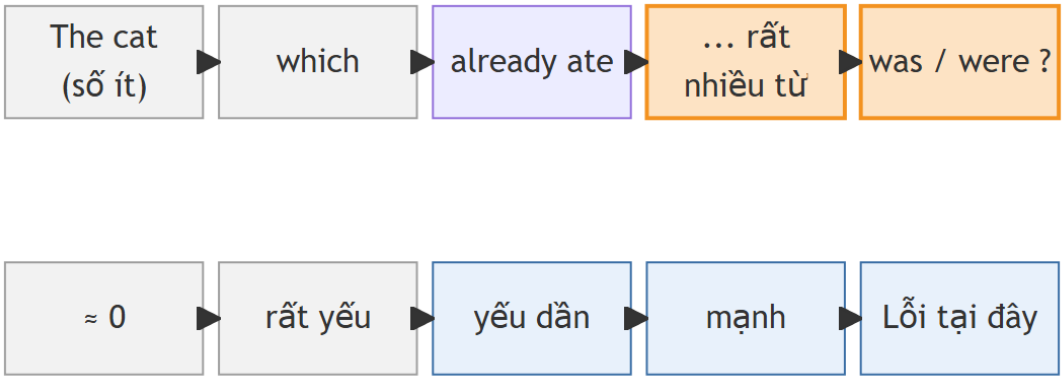
Ngôn ngữ có những phụ thuộc kéo dài rất xa:

- *"The **cat**, which already ate … a lot of food, **was** full."*
- *"The **cats**, which already ate … a lot of food, **were** full."*

Để chọn đúng *was*/*were*, mô hình phải nhớ chủ ngữ số ít hay số nhiều qua cả đoạn ở giữa – đoạn này có thể dài tuỳ ý.

6.1.5.2 Vì sao RNN cơ bản làm kém

RNN trải ra qua 1.000 bước giống một mạng sâu 1.000 lớp. Khi lan truyền ngược, tín hiệu lỗi bị nhân đi nhân lại qua từng bước nên **yếu dần theo cấp số nhân** (Hình 6.6). Lỗi ở cuối câu gần như không về được tới đầu câu, nên RNN cơ bản chủ yếu chịu ảnh hưởng *cục bộ* từ các từ gần nó.



Hình 6.6: Tiêu biến gradient – tín hiệu học yếu dần khi đi ngược về đầu câu

6.1.5.3 Bùng nổ gradient và cách chặn

Trường hợp ngược lại: gradient **tăng vọt**, bước cập nhật quá lớn làm tham số "nhảy" quá điểm tối ưu, dấu hiệu thường là loss thành `NaN`. Vấn đề này dễ phát hiện và có cách xử lý đơn giản: **cắt gradient (gradient clipping)** trước khi cập nhật.

Vấn đề	Dấu hiệu	Cách xử lý
Bùng nổ gradient	loss dao động mạnh, `NaN`	gradient clipping (`clipnorm`, `clipvalue`)
Tiêu biến gradient	không học được phụ thuộc xa	đổi sang GRU / LSTM (mục 6.1.6, 6.1.7)

Bảng 6.5: Hai vấn đề gradient trong RNN

Có thể tự viết hàm `clip` bằng NumPy: mọi phần tử gradient bị chặn trong một khoảng cho trước. Ví dụ với ngưỡng 10: giá trị 15 bị cắt về 10, giá trị âm 12 bị cắt về âm 10, giá trị 3 giữ nguyên.

```
# Tự viết bằng NumPy - chặn từng phần tử
for g in [dWax, dWaa, dWya, db, dby]:
```

```
    np.clip(g, -maxValue, maxValue, out=g)

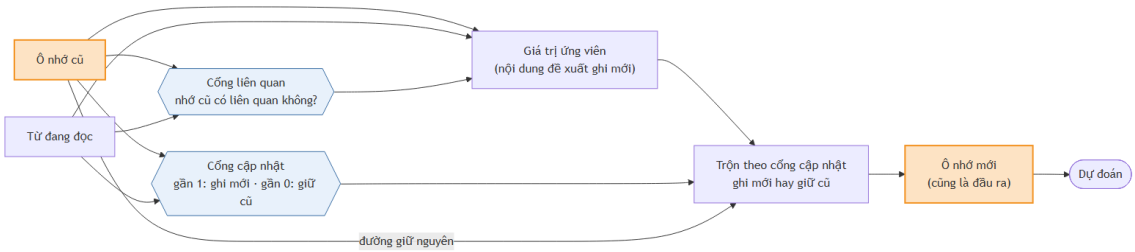
# Cách dùng trong dự án (Keras) - chỉ cần một tham số của optimizer
opt = keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0)   # chặn theo độ
↪  lớn (khuyên dùng)
# opt = keras.optimizers.Adam(learning_rate=1e-3, clipvalue=5.0) # chặn từng phần
↪  tử như cách NumPy ở trên
```

`clipnorm` thu nhỏ cả vector gradient khi độ lớn vượt ngưỡng nên giữ nguyên hướng cập nhật; `clipvalue` cắt từng phần tử, đơn giản hơn nhưng có thể làm lệch hướng.

6.1.6 GRU – Gated Recurrent Unit

GRU sửa lại ô RNN để giải quyết tiêu biến gradient. Ý tưởng chính là thêm một **ô nhớ** (memory cell) và các **cổng** (gate) quyết định khi nào ghi, khi nào giữ.

- **Ô nhớ** – "cuốn sổ" mang thông tin đi xuyên chuỗi, ví dụ ghi lại "chủ ngữ là số ít". Với GRU, ô nhớ cũng chính là đầu ra của bước đó.
- **Giá trị ứng viên** – mỗi bước GRU đề xuất một nội dung mới có thể ghi vào sổ.
- **Cổng cập nhật** – giá trị gần như luôn sát 0 hoặc sát 1 (do qua `sigmoid`). Sát 1: ghi đè bằng nội dung mới (khi gặp "The cat"). Sát 0: giữ nguyên nội dung cũ (suốt đoạn "which already ate…").
- **Cổng liên quan** – quyết định nội dung cũ liên quan đến đâu khi tạo ứng viên mới.



Hình 6.7: Ô GRU – cổng cập nhật quyết định ghi nội dung mới hay giữ nội dung cũ

Vì sao GRU chống được tiêu biến gradient. Khi cổng cập nhật gần 0, ô nhớ được sao chép gần như nguyên vẹn sang bước sau. Đường sao chép này không làm tín hiệu yếu đi, nên cả thông tin (khi xuôi) lẫn gradient (khi ngược) đi được rất xa.

Ô nhớ là vector. Mỗi chiều có cổng riêng, nên GRU có thể giữ chiều "số ít/số nhiều" không đổi trong khi cập nhật chiều "đang nói về đồ ăn".

6.1.7 LSTM – Long Short-Term Memory

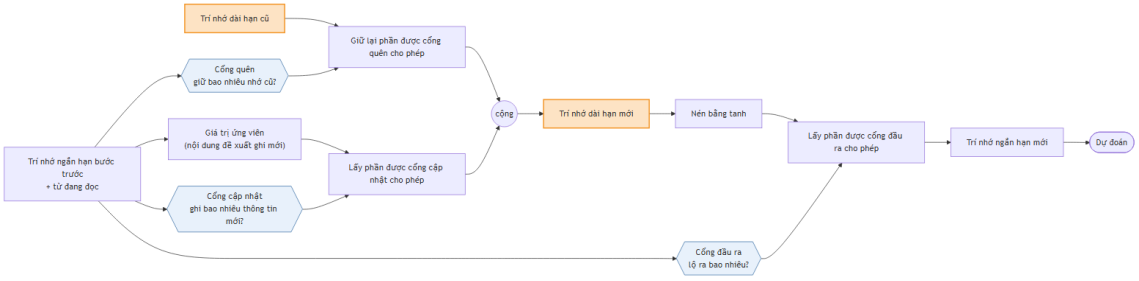
LSTM (Hochreiter & Schmidhuber) ra đời trước GRU, tổng quát và linh hoạt hơn. Hai khác biệt chính so với GRU:

- **Hai loại trí nhớ tách riêng** – *cell state* là trí nhớ dài hạn, *hidden state* là trí nhớ ngắn hạn và là đầu ra của bước.

- **Ba cổng thay vì hai** – tách "quên" và "ghi" thành hai cổng độc lập, nên LSTM có thể vừa giữ thông tin cũ vừa cộng thêm thông tin mới.

Cổng	Câu hỏi nó trả lời	Gần 0 / gần 1
Cổng quên	Giữ bao nhiêu trí nhớ dài hạn cũ?	quên sạch / giữ nguyên
Cổng cập nhật	Ghi bao nhiêu thông tin mới vào?	bỏ qua / ghi toàn bộ
Cổng đầu ra	Lộ ra bao nhiêu trí nhớ để dự đoán?	giấu / lộ toàn bộ

Bảng 6.6: Ba cổng của LSTM – mọi cổng đều nhìn vào trí nhớ ngắn hạn của bước trước và từ đang đọc



Hình 6.8: Ô LSTM – trí nhớ dài hạn chạy dọc theo "băng chuyền", các cổng điều tiết vào/ra

Khi nối nhiều ô LSTM theo thời gian, cell state chạy như một băng chuyền phía trên: chỉ cần cổng quên sát 1 và cổng cập nhật sát 0, một giá trị ghi từ bước đầu có thể đi nguyên vẹn qua hàng trăm bước.

Biến thể peephole. Các cổng được "nhìn trộm" thêm trí nhớ dài hạn của bước trước, mỗi chiều của ô nhớ chỉ tác động tới đúng chiều tương ứng của cổng. Keras không có sẵn trong `LSTM`; hiếm khi cần trong thực tế.

6.1.7.1 LSTM trong Keras: hidden state và cell state

Với `return_state=True`, `LSTM` trả về *ba* giá trị: đầu ra, hidden state cuối, cell state cuối. `GRU` chỉ trả về hai vì không có cell state riêng.

```
x = np.random.rand(4, 30, 90).astype("float32")

out, h, c = layers.LSTM(64, return_state=True)(x)
print(out.shape, h.shape, c.shape)    # (4, 64) (4, 64) (4, 64)   out == h khi
↪  return_sequences=False

out, h = layers.GRU(64, return_state=True)(x)
print(out.shape, h.shape)             # (4, 64) (4, 64)

# Truyền trạng thái khởi đầu tuỳ ý (dùng trong sinh chuỗi, encoder-decoder)
a0 = np.zeros((4, 64), "float32"); c0 = np.zeros((4, 64), "float32")
out = layers.LSTM(64)(x, initial_state=[a0, c0])
```

6.1.7.2 Chọn GRU hay LSTM

Tiêu chí	GRU	LSTM
Số cổng	2 (cập nhật, liên quan)	3 (quên, cập nhật, đầu ra)
Trí nhớ	một (ô nhớ cũng là đầu ra)	hai: dài hạn (cell) và ngắn hạn (hidden)
Tốc độ, số tham số	nhanh hơn, ít hơn khoảng 25%	chậm hơn
Khả năng biểu diễn	tốt, dễ mở rộng mạng lớn	mạnh, linh hoạt hơn
Lịch sử	mới hơn, rút gọn từ LSTM	lâu đời, được kiểm chứng nhiều

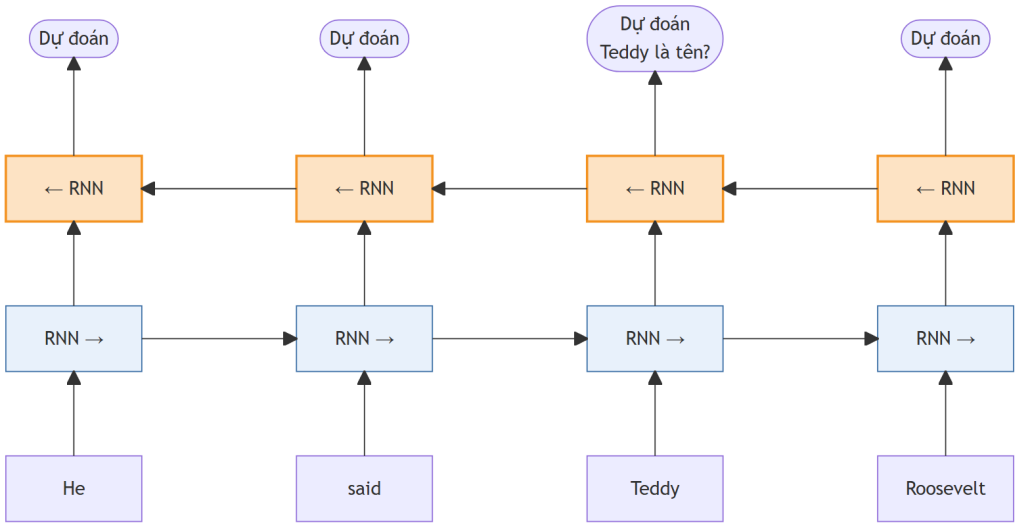
Bảng 6.7: So sánh GRU và LSTM

Không mô hình nào luôn thắng. Quy tắc thực dụng: **thử LSTM trước** làm mặc định; chuyển sang GRU khi cần mô hình nhẹ, huấn luyện nhanh hoặc dữ liệu ít. Đổi qua lại trong Keras chỉ là thay `layers.LSTM` bằng `layers.GRU`.

6.1.8 RNN hai chiều và RNN sâu

6.1.8.1 RNN hai chiều (Bidirectional RNN)

Để biết "Teddy" có phải tên người, mô hình cần nhìn cả "Roosevelt" phía sau. RNN hai chiều chạy **hai RNN song song**: một đọc từ trái sang phải, một đọc từ phải sang trái. Dự đoán tại mỗi bước dựa trên cả hai trạng thái, tức là thấy được cả quá khứ lẫn tương lai.



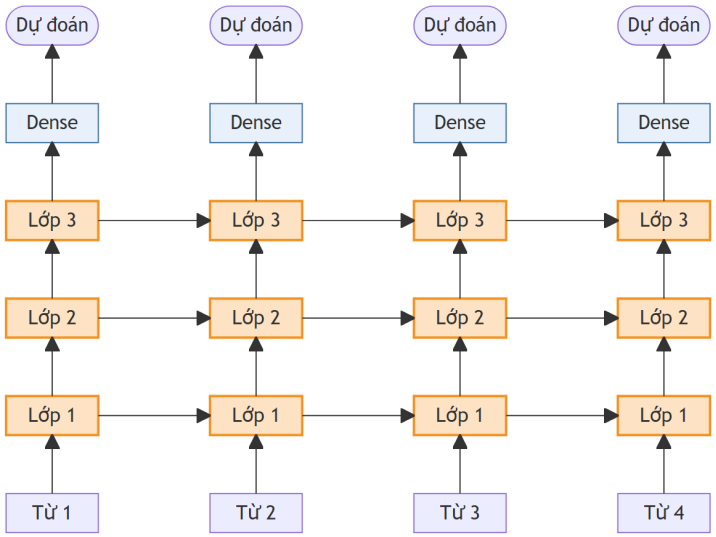
Hình 6.9: RNN hai chiều – mỗi dự đoán dùng cả ngữ cảnh trước và sau

- **Ưu điểm** – thường chính xác hơn rõ rệt cho NER, phân loại văn bản, gán nhãn.
- **Nhược điểm** – cần *toàn bộ* chuỗi trước khi dự đoán. Không dùng được cho nhận dạng giọng nói thời gian thực hay sinh chuỗi (chưa có "tương lai" để đọc).

```
layers.Bidirectional(layers.LSTM(64, return_sequences=True))   # đầu ra 128 chiều
↪  (64 xuôi + 64 ngược)
layers.Bidirectional(layers.GRU(64), merge_mode="sum")          # cộng thay vì
↪  nối, đầu ra 64 chiều
```

6.1.8.2 RNN sâu (Deep RNN)

Xếp chồng nhiều lớp RNN: đầu ra của lớp dưới (ở mọi bước) là đầu vào của lớp trên. Mỗi lớp có bộ tham số riêng, dùng chung theo thời gian trong lớp đó (Hình 6.10).



Hình 6.10: RNN sâu 3 lớp, phía trên mỗi bước có thêm lớp Dense không nối ngang

- Mạng CNN có thể sâu hơn 100 lớp, nhưng với RNN **2–3 lớp đã là nhiều** vì chiều thời gian đã khiến mạng rất lớn.
- Mọi lớp trừ lớp RNN cuối phải đặt `return_sequences=True`, nếu không lớp trên chỉ nhận được một vector thay vì cả chuỗi.
- Mỗi khối có thể là `SimpleRNN`, `GRU`, `LSTM` và có thể bọc `Bidirectional`.

```
model = keras.Sequential([
    keras.Input(shape=(None,), dtype="int32"),
    layers.Embedding(VOCAB, 128, mask_zero=True),
    layers.Bidirectional(layers.LSTM(128, return_sequences=True)),  # lớp 1: trả
    ↪  cả chuỗi
    layers.Dropout(0.3),
    layers.Bidirectional(layers.LSTM(64, return_sequences=True)),   # lớp 2
    layers.Dense(64, activation="relu"),                            # mạng sâu
    ↪  không nối ngang
    layers.Dense(NUM_TAGS, activation="softmax"),
])
```

6.1.9 Mô hình ngôn ngữ và sinh chuỗi

6.1.9.1 Mô hình ngôn ngữ là gì

Mô hình ngôn ngữ (language model) cho biết **một câu có khả năng xuất hiện cao hay thấp**. Hệ nhận dạng giọng nói nghe thấy hai câu phát âm giống hệt nhau: *"the apple and **pair** salad"* và *"the apple and **pear** salad"*. Mô hình ngôn ngữ chấm câu thứ hai cao hơn hàng nghìn lần, nên hệ thống chọn câu đó. Dịch máy cũng dùng mô hình ngôn ngữ để chỉ sinh ra câu tự nhiên.

6.1.9.2 Huấn luyện: đoán từ tiếp theo

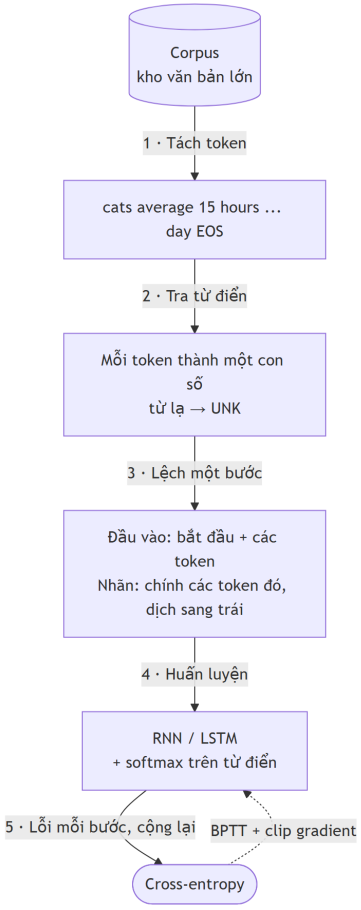
1. **Corpus** – thu thập kho văn bản lớn của ngôn ngữ cần mô hình hoá.

2. **Tách token** – tách câu thành từ (hoặc ký tự), tra chỉ số trong từ điển. Thêm `<EOS>` đánh dấu cuối câu; từ ngoài từ điển thay bằng `<UNK>`.

3. **Lệch một bước** – đầu vào của mỗi bước là token *đúng* của bước trước; nhãn là token của bước hiện tại. Nói cách khác, chuỗi nhãn chính là chuỗi đầu vào dịch sang trái một vị trí. Cách cho mô hình xem đáp án đúng của bước trước khi huấn luyện gọi là *teacher forcing*.

4. **Học** – ở mỗi bước, softmax trên toàn bộ từ điển dự đoán từ tiếp theo; lỗi là tổng cross-entropy các bước.

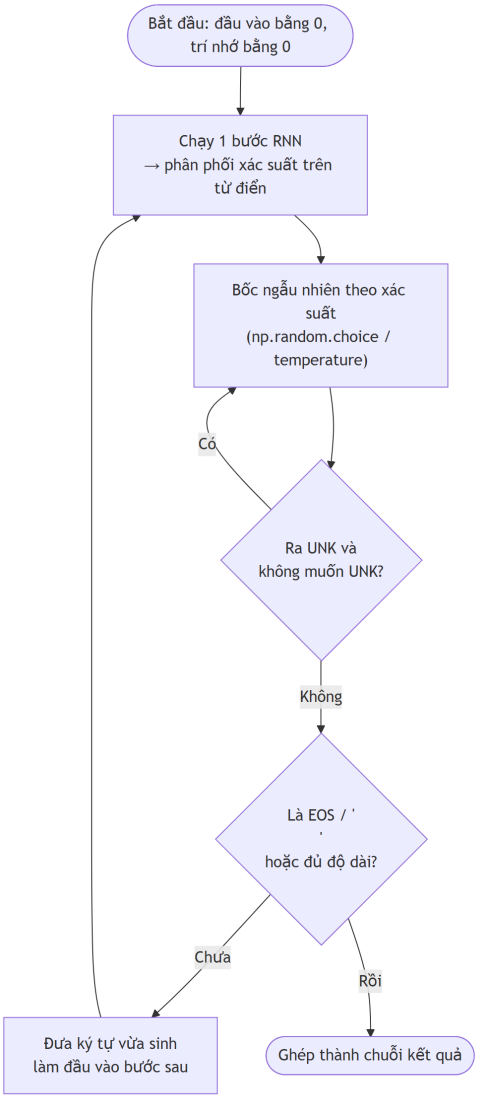
Sau khi học xong, nhân xác suất dự đoán ở từng bước với nhau sẽ ra xác suất của cả câu: xác suất từ đầu, nhân xác suất từ thứ hai khi biết từ đầu, nhân xác suất từ thứ ba khi biết hai từ trước.



Hình 6.11: Pipeline huấn luyện mô hình ngôn ngữ

6.1.9.3 Sinh chuỗi mới (sampling)

Sau khi huấn luyện, cho mô hình tự "viết" là cách nhanh nhất xem nó học được gì. Khác biệt duy nhất so với lúc huấn luyện: **đầu vào bước sau là token mô hình vừa tự chọn**, không phải token đúng.



Hình 6.12: Vòng lặp sinh chuỗi

Bốc ngẫu nhiên, không chọn lớn nhất. Nếu luôn chọn token có xác suất cao nhất (`argmax`), cùng một điểm bắt đầu sẽ luôn ra cùng một kết quả. Có thể dùng `np.random.choice` để bốc theo xác suất: token có xác suất 0,16 được chọn 16% số lần. Trong thực tế hay thêm **temperature** để điều chỉnh độ "sáng tạo":

```
def sample_index(probs, temperature=1.0):
    """temperature < 1: an toàn, lặp lại nhiều; > 1: táo bạo, dễ sai chính tả."""
    logits = np.log(np.asarray(probs, "float64") + 1e-9) / temperature
    p = np.exp(logits - logits.max())
    p /= p.sum()
    return np.random.choice(len(p), p=p)
```

6.1.9.4 Mô hình theo từ hay theo ký tự

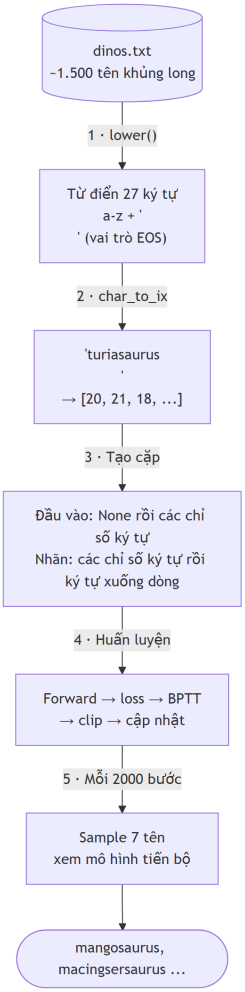
Tiêu chí	Theo từ (word-level)	Theo ký tự (character-level)
Từ điển	hàng chục nghìn từ	vài chục ký tự (a–z, số, dấu câu, khoảng trắng)
Từ lạ	phải thay bằng `<UNK>`	không bao giờ gặp, xử lý được từ hiếm như "Mau"
Độ dài chuỗi	ngắn (10–20 từ mỗi câu)	dài gấp nhiều lần
Phụ thuộc xa	nắm tốt hơn	khó hơn
Chi phí huấn luyện	thấp hơn	cao hơn
Khi nào dùng	mặc định	tên riêng, từ vựng chuyên ngành, nhiều từ lạ

Bảng 6.8: So sánh mô hình ngôn ngữ theo từ và theo ký tự

Ví dụ: mô hình học từ tin tức sinh ra văn bản na ná tin tức; mô hình học từ Shakespeare sinh ra câu như *"The mortal moon hath her eclipse in love"* – trông giống thật nhưng chưa hẳn đúng ngữ pháp.

6.1.10 Thực hành 1: Đặt tên khủng long

Bài toán: từ danh sách khoảng 1.500 tên khủng long (một tệp văn bản, mỗi dòng một tên), xây mô hình ngôn ngữ *theo ký tự* để sinh tên mới nghe "giống khủng long". Mô hình theo ký tự hợp lý ở đây vì tên riêng không có từ điển cố định.



Hình 6.13: Pipeline ví dụ thực hành đặt tên khủng long

6.1.10.1 Những ý chính

- **Từ điển 27 ký tự** – 26 chữ cái thường và ký tự xuống dòng `\n`. Ký tự `\n` đóng vai trò `<EOS>`: báo hết một tên.
- **Hai bảng tra** – `char_to_ix` (ký tự → chỉ số) để mã hoá đầu vào, `ix_to_char` (chỉ số → ký tự) để đọc kết quả softmax.
- **Cặp huấn luyện** – đầu vào là danh sách chỉ số ký tự có thêm `None` ở đầu, nhãn là danh sách chỉ số đó có thêm ký tự xuống dòng ở cuối. `None` báo cho `rnn_forward` dùng vector 0 ở bước đầu.
- **Vòng tối ưu** – forward tính loss → BPTT tính gradient → `clip` chặn gradient trong khoảng từ âm 5 đến 5 → cập nhật. Mỗi lần dùng một tên (stochastic gradient descent), lặp 35.000 lần.
- **Theo dõi** – mỗi 2.000 bước sinh thử 7 tên. Ban đầu ra ký tự lộn xộn; về sau mô hình học được các đuôi `-saurus`, `-don`, `-aura`, `-tor`, sinh ra những tên như `mangosaurus` hay `macingsersaurus`.

6.1.10.2 Viết lại bằng Keras

Có thể tự cài từng bước bằng NumPy. Trong dự án, toàn bộ phần forward, BPTT, clip và cập nhật gói gọn trong `Embedding` + `LSTM` + `model.fit`:

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

# 1. Dữ liệu và từ điển: 0 = PAD, 1..27 = ký tự, 28 = START (thay cho vector 0 ở
↪  bước đầu)
names = open("dinos.txt", encoding="utf-8").read().lower().strip().split("\n")
↪  # mỗi dòng một tên
chars = sorted(set("".join(names)) | {"\n"})
char_to_ix = {ch: i + 1 for i, ch in enumerate(chars)}
ix_to_char = {i: ch for ch, i in char_to_ix.items()}
PAD, START, V = 0, len(chars) + 1, len(chars) + 2
EOS = char_to_ix["\n"]

# 2. Cặp (X, Y) lệch một bước, đệm 0 cho bằng độ dài
X = [[START] + [char_to_ix[c] for c in n] for n in names]
Y = [[char_to_ix[c] for c in n] + [EOS] for n in names]
X = keras.utils.pad_sequences(X, padding="post", value=PAD)
Y = keras.utils.pad_sequences(Y, padding="post", value=PAD)

# 3. Mô hình: nhiều - nhiều, đoán ký tự kế tiếp ở mọi bước
model = keras.Sequential([
    keras.Input(shape=(None,), dtype="int32"),
    layers.Embedding(V, 32, mask_zero=True),   # mask: phần đệm không tính vào
    ↪  loss
    layers.LSTM(128, return_sequences=True),
    layers.Dense(V, activation="softmax"),
])
model.compile(optimizer=keras.optimizers.Adam(1e-3, clipnorm=1.0),  # cắt
↪  gradient
              loss="sparse_categorical_crossentropy")
```

```
# 4. Sinh tên: mỗi bước đưa cả tiền tố vào mô hình, lấy phân phối của bước cuối
def generate_name(temperature=1.0, max_len=30):
    seq = [START]
    for _ in range(max_len):
        probs = model.predict(np.array([seq]), verbose=0)[0, -1]
        probs[[PAD, START]] = 0                  # không bao giờ sinh PAD / START
        idx = sample_index(probs / probs.sum(), temperature)
        if idx == EOS:
            break
        seq.append(idx)
    return "".join(ix_to_char[i] for i in seq[1:]).capitalize()

# 5. Huấn luyện, cuối mỗi epoch in thử vài tên để theo dõi
show = keras.callbacks.LambdaCallback(
    on_epoch_end=lambda epoch, logs: print(epoch, [generate_name(0.8) for _ in
    ↪  range(5)]))
model.fit(X, Y, batch_size=64, epochs=30, callbacks=[show])
```

Viết như Shakespeare. Cùng ý tưởng này áp dụng được lên tập thơ Shakespeare với một mô hình LSTM Keras đã huấn luyện sẵn: người dùng nhập vài chữ đầu, mô hình viết tiếp. Văn bản dài hơn tên khủng long nhiều, nên LSTM (nhớ được phụ thuộc xa) cho kết quả tốt hơn RNN cơ bản rõ rệt.

6.1.11 Thực hành 2: Ứng tác jazz với LSTM

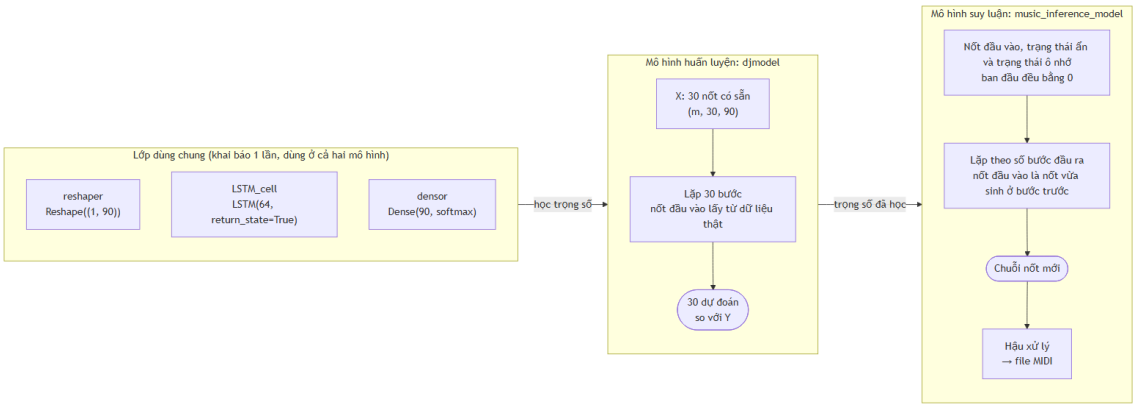
Bài toán: học phong cách từ một bản solo jazz rồi sinh đoạn nhạc mới. Đây là kiến trúc *một – nhiều*, viết hoàn toàn bằng Keras Functional API.

6.1.11.1 Dữ liệu: âm nhạc là chuỗi "giá trị"

- Mỗi **giá trị âm nhạc** (musical value) có thể hiểu nôm na là một nốt: cao độ cùng trường độ (thực tế còn mã hoá cả hợp âm). Bộ dữ liệu có 90 giá trị khác nhau.
- Dữ liệu đầu vào (`X`) có hình dạng `(số đoạn, 30, 90)`: 60 đoạn, mỗi đoạn 30 bước, mỗi bước là vector one-hot 90 chiều. Các đoạn cắt ngẫu nhiên từ giữa bản nhạc, nên không cần đặt đầu vào đầu tiên bằng 0.
- Nhãn (`Y`) giống dữ liệu đầu vào nhưng lệch trái một bước (đoán nốt tiếp theo), và được xếp lại thành `(30, số đoạn, 90)` để đưa vào 30 đầu ra của mô hình.

6.1.11.2 Mẹo cốt lõi: lớp dùng chung giữa hai mô hình

Khi suy luận, ta *không có sẵn* cả chuỗi đầu vào – mỗi nốt chính là nốt mô hình vừa sinh ở bước trước. Vì vậy không dùng được kiểu `LSTM(return_sequences=True)` chạy cả chuỗi một lần, mà phải tự viết vòng `for` qua từng bước. Để mọi bước (và cả hai mô hình) dùng chung trọng số, ta khai báo các lớp **một lần duy nhất** rồi gọi lại nhiều lần:



Hình 6.14: Ba lớp dùng chung: mô hình huấn luyện học trọng số, mô hình suy luận dùng lại

```
n_values, n_a, Tx = 90, 64, 30
reshaper  = layers.Reshape((1, n_values))          # (batch, 90) -> (batch, 1,
↪  90)
LSTM_cell = layers.LSTM(n_a, return_state=True)    # một ô, gọi lại ở mọi bước
densor    = layers.Dense(n_values, activation="softmax")

def djmodel(Tx):
    """Mô hình huấn luyện: nốt thật ở bước t -> đoán nốt bước t+1."""
    X  = keras.Input(shape=(Tx, n_values))
    a0 = keras.Input(shape=(n_a,), name="a0")
    c0 = keras.Input(shape=(n_a,), name="c0")
    a, c, outputs = a0, c0, []
    for t in range(Tx):
        x = reshaper(X[:, t, :])                           # lấy bước t
        _, a, c = LSTM_cell(x, initial_state=[a, c])       # cùng một LSTM_cell
        outputs.append(densor(a))
    return keras.Model(inputs=[X, a0, c0], outputs=outputs)
```

```
model = djmodel(Tx)                                          # 45.530 tham số
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.01),
              loss="categorical_crossentropy", metrics=["accuracy"])
m = X.shape[0]
model.fit([X, np.zeros((m, n_a)), np.zeros((m, n_a))], list(Y), epochs=100,
↪  verbose=0)
```

Loss giảm từ khoảng 130 ở epoch đầu xuống một chữ số sau 100 epoch. Các tài liệu cũ hay truyền `lr` và `decay` cho `Adam`; TensorFlow mới đã bỏ hai tham số này, thay bằng `learning_rate` và các lịch học (learning rate schedule) trong `keras.optimizers.schedules`.

6.1.11.3 Mô hình suy luận: tự sinh nhạc

```
import tensorflow as tf

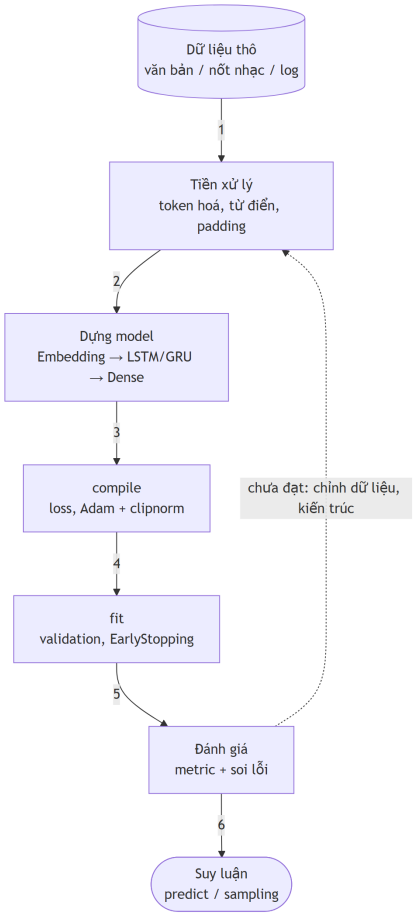
def music_inference_model(Ty=50):
    x0 = keras.Input(shape=(1, n_values))
    a0 = keras.Input(shape=(n_a,), name="a0")
    c0 = keras.Input(shape=(n_a,), name="c0")
    x, a, c, outputs = x0, a0, c0, []
    for t in range(Ty):
        _, a, c = LSTM_cell(x, initial_state=[a, c])   # trọng số đã học
        out = densor(a)
        outputs.append(out)
        x = tf.one_hot(tf.math.argmax(out, axis=-1), n_values)  # nốt vừa sinh ->
        ↪  one-hot
        x = layers.RepeatVector(1)(x)                            # (batch, 90) ->
        ↪  (batch, 1, 90)
    return keras.Model(inputs=[x0, a0, c0], outputs=outputs)

inference_model = music_inference_model(Ty=50)
pred = inference_model.predict([np.zeros((1, 1, n_values)), np.zeros((1, n_a)),
↪  np.zeros((1, n_a))])
indices = np.argmax(pred, axis=-1)                     # 50 chỉ số giá trị âm
↪  nhạc
results = keras.utils.to_categorical(indices, num_classes=n_values)
```

- Ví dụ này chọn nốt bằng `argmax` nên kết quả gần như cố định. Muốn mỗi lần nghe một khác, thay bằng bốc ngẫu nhiên có temperature như mục 6.1.9 (cần viết vòng lặp ngoài mô hình, gọi từng bước).
- Chuỗi giá trị sinh ra được **hậu xử lý** thành hợp âm và file MIDI: tránh lặp một âm quá nhiều lần, tránh hai nốt liên tiếp cách nhau quá xa. Chất lượng nghe được phụ thuộc nhiều vào bước này, không chỉ vào mô hình.

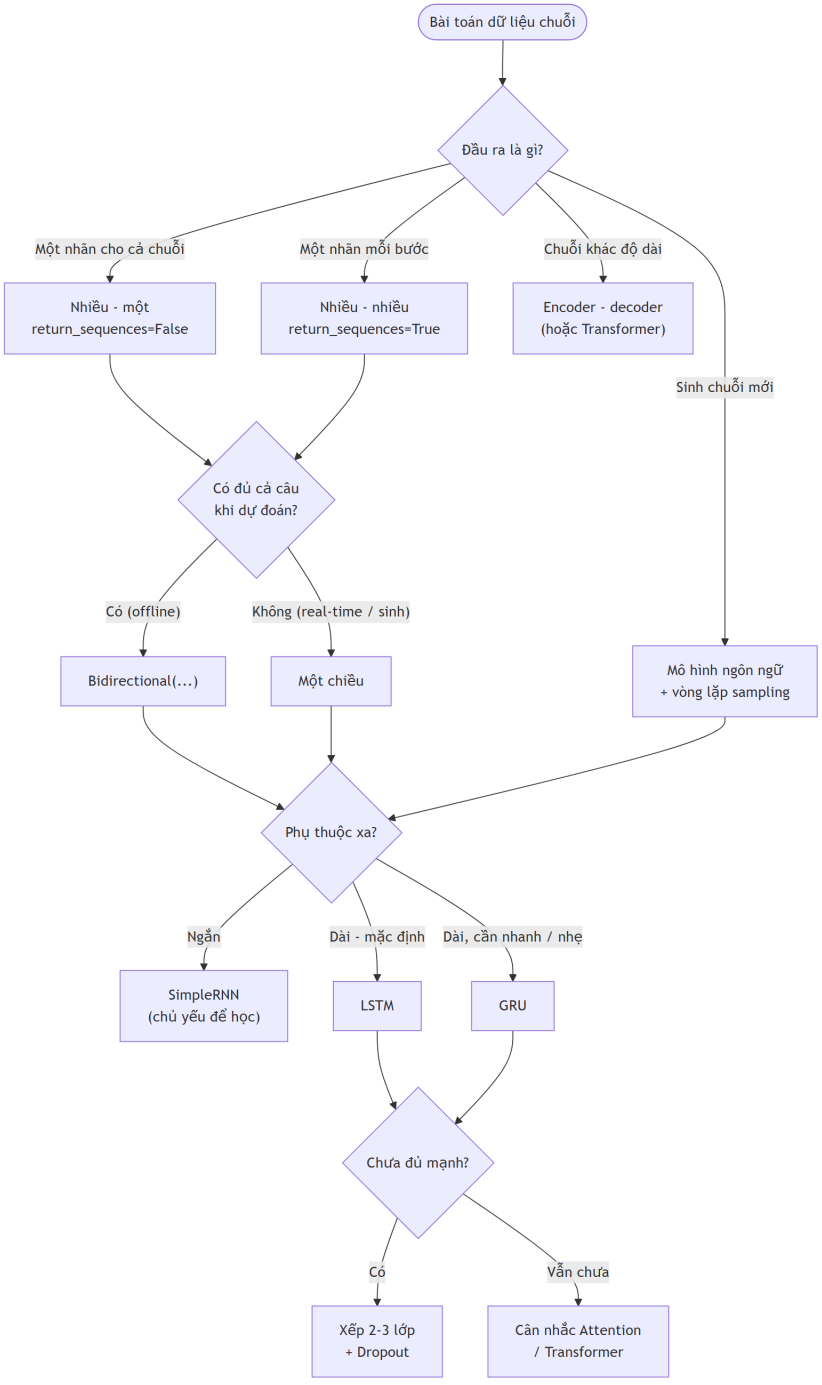
6.1.12 Áp dụng vào dự án thực tế

6.1.12.1 Quy trình chung



Hình 6.15: Quy trình xây mô hình chuỗi với Keras

6.1.12.2 Chọn kiến trúc



Hình 6.16: Cây quyết định chọn kiến trúc RNN

6.1.12.3 Siêu tham số khởi đầu

Siêu tham số	Giá trị gợi ý	Ghi chú
Số đơn vị ẩn	64–256	tăng khi underfit; trạng thái lớn thì chậm
Số lớp RNN	1–3	quá 3 lớp hiếm khi có lợi
Embedding	64–300 chiều	có thể nạp vector huấn luyện sẵn (GloVe, fastText)
Optimizer	Adam, `learning_rate=1e-3`	thêm `clipnorm=1.0`
Dropout	0,2–0,5	`Dropout` giữa các lớp, hoặc tham số `dropout` của lớp RNN
Batch size	32–128	đệm theo batch để giảm phần đệm thừa
Độ dài chuỗi	theo phân vị 95%	cắt bớt chuỗi quá dài, không đệm theo chuỗi dài nhất
Temperature	0,7–1,0	chỉ dùng khi sinh chuỗi

Bảng 6.9: Điểm khởi đầu hợp lý cho siêu tham số

6.1.12.4 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
`expected ndim=3, found ndim=2` khi xếp chồng RNN	lớp RNN phía dưới thiếu `return_sequences=True`
Loss thành `NaN` hoặc nhảy loạn	bùng nổ gradient: thêm `clipnorm`, giảm learning rate
Mô hình học tốt đoạn ngắn, kém đoạn dài	đang dùng `SimpleRNN`: đổi sang LSTM/GRU, cân nhắc Bidirectional
Mô hình học cả phần đệm số 0	thiếu `mask_zero=True` ở `Embedding` (hoặc lớp `Masking`)
Sinh chuỗi lặp đi lặp lại một cụm	đang chọn `argmax`: chuyển sang bốc ngẫu nhiên, tăng temperature
Sinh ra toàn ký tự lộn xộn	huấn luyện chưa đủ, hoặc temperature quá cao
Huấn luyện rất chậm	chuỗi quá dài, mô hình quá sâu; trên GPU giữ `activation` mặc định để Keras dùng nhân cuDNN
Sai kích thước khi chuyển code NumPy tự cài sang Keras	code NumPy xếp (đặc trưng, batch, số bước), Keras xếp `(batch, timesteps, features)` (Bảng 6.3)

Bảng 6.10: Lỗi thường gặp khi làm việc với RNN

6.1.12.5 Khi nào không dùng RNN

RNN (đặc biệt LSTM/GRU) vẫn hợp với chuỗi ngắn đến trung bình, dữ liệu ít, thiết bị yếu, hay xử lý luồng thời gian thực từng bước. Với văn bản dài và dữ liệu lớn, **Transformer** thường tốt hơn vì xử lý song song mọi vị trí và nối trực tiếp các vị trí xa nhau qua cơ chế chú ý. Trong nhiều bài toán NLP hiện nay, điểm khởi đầu hợp lý là tinh chỉnh một mô hình Transformer huấn luyện sẵn; RNN là phương án nhẹ hoặc baseline để so sánh.

Khái niệm	Keras	PyTorch
RNN cơ bản	`layers.SimpleRNN`	`nn.RNN`
GRU / LSTM	`layers.GRU`, `layers.LSTM`	`nn.GRU`, `nn.LSTM`
Hai chiều	`layers.Bidirectional(...)`	tham số `bidirectional=True`
Xếp chồng	nhiều lớp, `return_sequences=True`	tham số `num_layers=N`
Batch ở đầu	mặc định	`batch_first=True`
Clip gradient	`clipnorm` trong optimizer	`torch.nn.utils.clip_grad_norm_`

Bảng 6.11: Đối chiếu API Keras và PyTorch

6.1.13 Tóm tắt cần nhớ

1. **RNN là một ô dùng lặp lại** qua thời gian, mang theo trạng thái ẩn làm trí nhớ; tham số dùng chung cho mọi bước.
2. **Đầu ra lấy ở đâu quyết định kiểu kiến trúc**: bước cuối (nhiều – một), mọi bước (nhiều – nhiều), encoder – decoder (khác độ dài), tự sinh (một – nhiều).
3. **BPTT** – lỗi là tổng lỗi các bước, gradient đi ngược thời gian; framework tự lo.
4. **Bùng nổ gradient** → clip. **Tiêu biến gradient** → GRU/LSTM.
5. **GRU**: ô nhớ cùng cổng cập nhật quyết định ghi hay giữ. **LSTM**: tách trí nhớ dài hạn và ngắn hạn, ba cổng quên/cập nhật/đầu ra. Mặc định thử LSTM.
6. **Bidirectional** khi có đủ cả chuỗi; **xếp 2–3 lớp** khi cần mạnh hơn.
7. **Mô hình ngôn ngữ** học đoán token kế tiếp (nhãn là đầu vào lệch một bước); **sinh chuỗi** bằng cách bốc ngẫu nhiên rồi đưa kết quả làm đầu vào bước sau.
8. **Sinh chuỗi với Keras** cần lớp dùng chung và vòng lặp tự viết, vì đầu vào tương lai chưa có sẵn.

6.2 Xử lý ngôn ngữ tự nhiên và Word Embedding

Mục 6.1 cho thấy RNN, GRU, LSTM đọc được chuỗi. Nhưng mô hình chỉ đọc được *số*, không đọc được chữ. Câu hỏi đầu tiên của mọi bài toán xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) là: **biến một từ thành các con số như thế nào để máy hiểu được ý nghĩa của nó?**

Câu trả lời hiện đại là **word embedding**: mỗi từ được biểu diễn bằng một vector gồm vài chục đến vài trăm con số, sao cho từ gần nghĩa có vector gần nhau. Ý tưởng này nhỏ nhưng tạo ra thay đổi lớn: mô hình học được từ ít dữ liệu có nhãn, hiểu được từ chưa từng thấy khi huấn luyện, và còn làm được phép loại suy như "man là woman thì king là queen".

Lộ trình của mục này: vì sao one-hot chưa đủ → embedding là gì và dùng trong Keras → chuyển giao kiến thức → loại suy và độ tương đồng cosine → các thuật toán học embedding → phân loại cảm xúc → thiên kiến → hai ví dụ thực hành (phép toán trên vector từ, Emojify) → checklist áp dụng vào dự án. Code dùng TensorFlow/Keras và gensim; phần tự cài đặt bằng NumPy chỉ nhằm hiểu bản chất.

6.2.1 Vì sao one-hot chưa đủ

Cách đơn giản nhất để đưa một từ vào mô hình là **one-hot**: với từ điển 10.000 từ, mỗi từ là một vector 10.000 chiều, chỉ có một số 1 ở vị trí của từ đó, còn lại là 0. Cách này có hai điểm yếu:

- **Mọi từ cách đều nhau.** Hai vector one-hot khác nhau bất kỳ không có điểm chung nào. Với mô hình, "apple" không gần "orange" hơn "king" hay "queen".

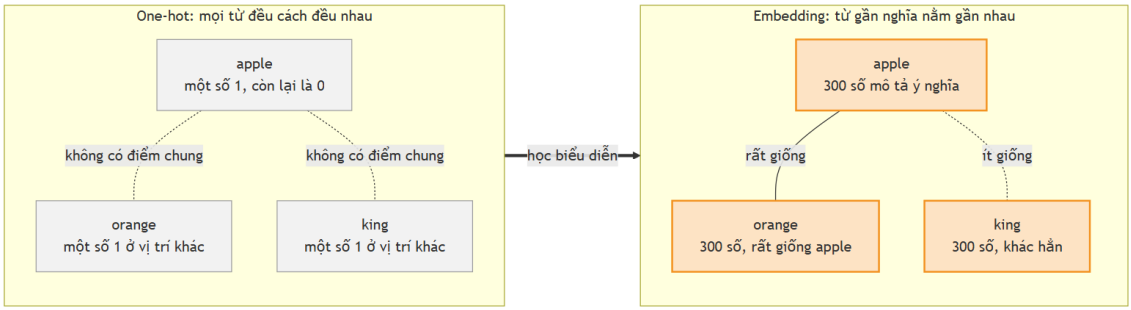
- **Không khái quát được.** Giả sử mô hình ngôn ngữ đã học rằng "I want a glass of orange *juice*" là câu hợp lý. Khi gặp "I want a glass of apple …", nó không có lý do gì để đoán "juice", vì với nó "apple" và "orange" chẳng liên quan.
- **Quá lớn và thưa.** Vector hàng chục nghìn chiều mà gần như toàn số 0.

Ý tưởng sửa: thay vì chỉ "đánh số" từ, hãy **mô tả từ bằng các đặc trưng**.

Đặc trưng	**man**	**woman**	**king**	**apple**
Giới tính (nam hay nữ)	nam	nữ	nam	trung tính
Mức độ hoàng gia	thấp	thấp	cao	thấp
Tuổi	trung tính	trung tính	trưởng thành	trung tính
Là thức ăn	không	không	không	có

Bảng 6.12: Minh hoạ cách mô tả từ bằng đặc trưng (thực tế có khoảng 300 đặc trưng)

Với khoảng 300 đặc trưng như vậy, mỗi từ trở thành một vector 300 chiều gọn và *dày* (gần như không có số 0). Từ gần nghĩa có đặc trưng gần giống nhau: "apple" và "orange" khác nhau ở màu sắc, vị, nhưng giống nhau ở rất nhiều đặc trưng khác. Nhờ vậy khi mô hình học được "orange juice", nó nhanh chóng suy ra "apple juice" cũng hợp lý.



Hình 6.17: One-hot và embedding: embedding đặt các từ gần nghĩa gần nhau

Tiêu chí	**One-hot**	**Embedding**
Số chiều	bằng kích thước từ điển (10.000 trở lên)	vài chục đến vài trăm (50–300)
Mật độ	thưa, gần như toàn số 0	dày
Quan hệ giữa các từ	không có, mọi từ cách đều nhau	có, từ gần nghĩa nằm gần nhau
Cách có được	tự động từ chỉ số của từ	được *học* từ văn bản
Khái quát sang từ gần nghĩa	không	có

Bảng 6.13: So sánh one-hot và embedding

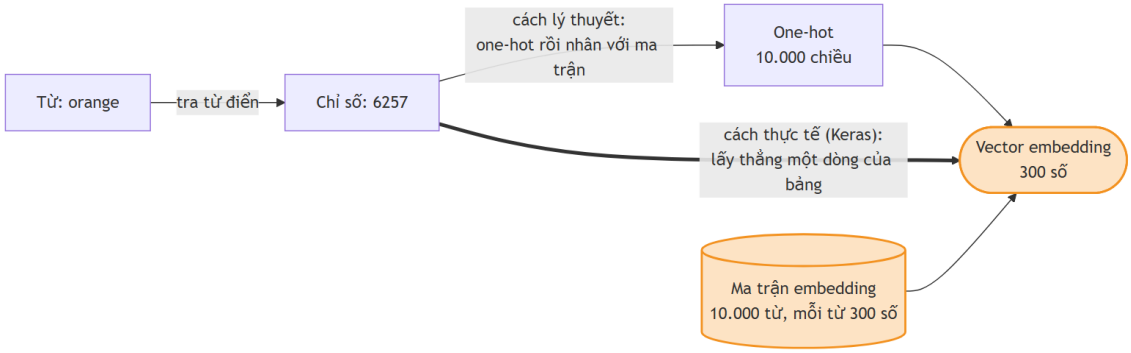
Một điều cần biết ngay: những đặc trưng mà thuật toán thực sự học **không** có nhãn dễ hiểu như "chiều số 1 là giới tính". Ý nghĩa của từng chiều thường bị trộn lẫn và khó diễn giải. Điều duy nhất ta cần là các từ liên quan có vector gần nhau.

6.2.2 Embedding là gì và dùng thế nào

6.2.2.1 Ma trận embedding

Thứ mà mọi thuật toán học embedding tạo ra là một **ma trận embedding**: một bảng tra có một dòng cho mỗi từ trong từ điển, mỗi dòng là vector của từ đó. Từ điển 10.000 từ, embedding 300 chiều thì bảng gồm 10.000 dòng, mỗi dòng 300 số. Thêm một dòng cho token từ lạ (`<UNK>`) và một dòng cho phần đệm (padding) nếu cần.

Về lý thuyết, lấy vector của một từ là nhân ma trận với vector one-hot của từ đó: các số 0 xoá hết mọi dòng khác, chỉ còn đúng dòng của từ cần tìm. Thực tế không ai làm phép nhân khổng lồ này; **lấy embedding chỉ là tra bảng theo chỉ số của từ**, và đó chính xác là việc của lớp `Embedding` trong Keras (Hình 6.18).



Hình 6.18: Lấy embedding của một từ: thực chất là tra bảng

6.2.2.2 Lớp Embedding trong Keras

Lớp `Embedding` nhận một ma trận số nguyên (chỉ số của các từ) và trả về ma trận vector tương ứng. Chú ý: Keras xếp mỗi từ là một *dòng*, trong khi nhiều tài liệu vẽ mỗi từ là một *cột*; chỉ khác cách vẽ, ý nghĩa như nhau.

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

emb = layers.Embedding(input_dim=10000, output_dim=300)   # bảng 10.000 từ, mỗi
↪  từ 300 số
ids = np.array([[4343, 9665, 1, 3852]])                   # một câu 4 từ, đã đổi
↪  sang chỉ số
print(emb(ids).shape)                                     # (1, 4, 300): (batch,
↪  số từ, chiều embedding)
```

Ba tham số hay dùng:

- `mask_zero=True` – coi chỉ số 0 là phần đệm. Các lớp phía sau (LSTM, GRU, pooling) sẽ bỏ qua phần đệm. Nhớ dành chỉ số 0 cho đệm khi đánh số từ điển.
- `trainable=False` – đóng băng bảng embedding, không cho học thêm. Dùng khi nạp embedding huấn luyện sẵn và dữ liệu có nhãn ít.
- `embeddings_initializer` – cách khởi tạo bảng. Mặc định là ngẫu nhiên (bảng sẽ được học từ đầu cùng mô hình). Muốn dùng embedding có sẵn thì truyền ma trận vào đây.

6.2.2.3 Nạp embedding huấn luyện sẵn (GloVe)

Học embedding rất tốn kém, nên thực tế gần như luôn bắt đầu từ embedding đã huấn luyện sẵn trên kho văn bản khổng lồ. Ví dụ đọc tệp GloVe 50 chiều (`glove.6B.50d.txt`, mỗi dòng gồm một từ và 50 số) rồi nạp vào lớp `Embedding`:

```
def load_glove(path, dim=50):
    """Đọc tệp GloVe, trả về (từ -> chỉ số, ma trận embedding). Chỉ số 0 dành cho
    ↪ phần đệm."""
    word_to_index, vectors = {}, [np.zeros(dim, "float32")]
    with open(path, encoding="utf-8") as f:
        for line in f:
            parts = line.rstrip().split(" ")
            word_to_index[parts[0]] = len(vectors)
            vectors.append(np.asarray(parts[1:], dtype="float32"))
    return word_to_index, np.vstack(vectors)

word_to_index, matrix = load_glove("glove.6B.50d.txt")     # matrix: (400.001,
↪ 50)

embedding_layer = layers.Embedding(
    input_dim=matrix.shape[0], output_dim=matrix.shape[1],
    embeddings_initializer=keras.initializers.Constant(matrix),   # nạp bảng có
    ↪ sẵn
    trainable=False,                                              # đóng băng
    mask_zero=True)                                               # chỉ số 0 là
    ↪ đệm
```

Đây cũng là cách ví dụ Emojify (mục 6.2.9) tạo lớp embedding. Số tham số của lớp này là số từ nhân số chiều: với GloVe 400.000 từ, 50 chiều thì hơn 20 triệu tham số, và vì đã đóng băng nên chúng không tăng chi phí huấn luyện.

6.2.2.4 Vẽ embedding ra hình với t-SNE

Không thể vẽ trực tiếp vector 300 chiều. Thuật toán **t-SNE** nén chúng xuống 2 chiều để quan sát. Trên hình thường thấy man–woman, king–queen nằm cạnh nhau, các loại quả tụ lại một nhóm, các con số tụ lại một nhóm, các loài vật một nhóm.

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

words = ["man", "woman", "king", "queen", "apple", "orange", "grape", "one",
↪ "two", "three"]
vecs = np.array([matrix[word_to_index[w]] for w in words])
xy = TSNE(n_components=2, perplexity=3, random_state=0).fit_transform(vecs)  #
↪ perplexity < số điểm

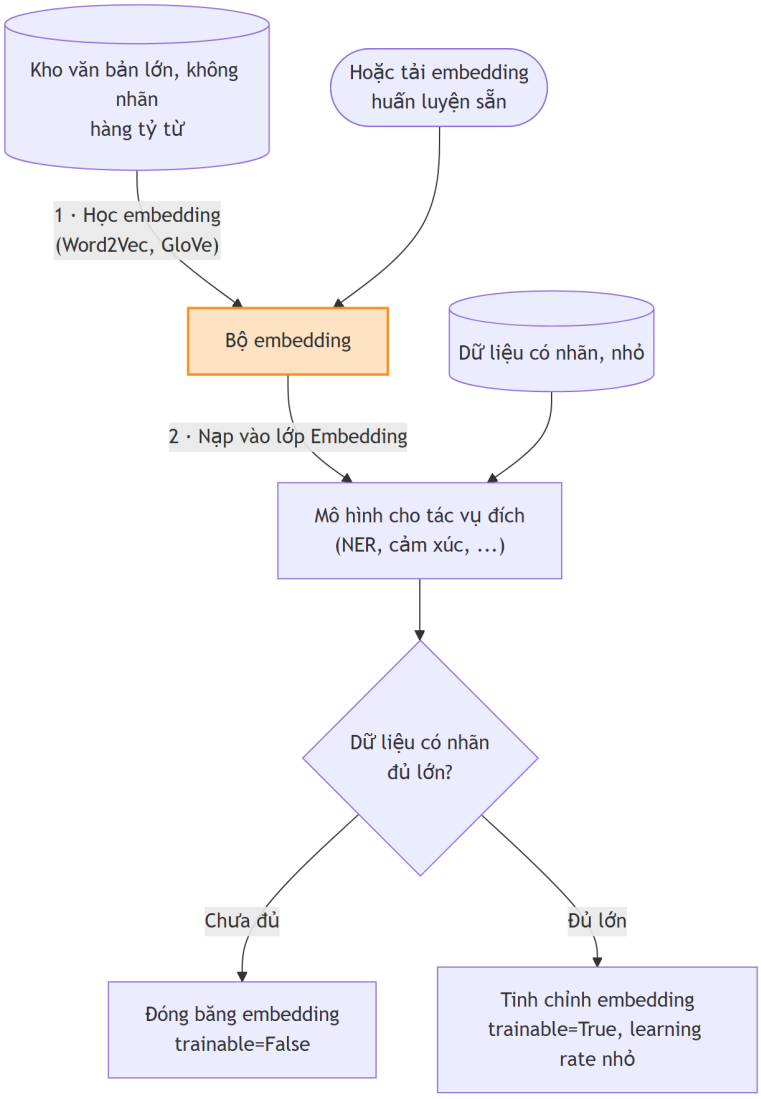
plt.scatter(xy[:, 0], xy[:, 1])
for (x, y), w in zip(xy, words):
    plt.annotate(w, (x, y))
plt.show()
```

Lưu ý quan trọng: t-SNE là phép nén phi tuyến rất phức tạp. Hình 2 chiều chỉ để *xem nhóm*; các quan hệ hình học chính xác (như phép loại suy ở mục tiếp theo) chỉ đáng tin trong không gian gốc, và thường bị phá vỡ sau khi qua t-SNE.

6.2.3 Chuyển giao kiến thức bằng embedding

Giá trị lớn nhất của embedding là cho phép **chuyển giao kiến thức** (transfer learning). Ví dụ bài toán nhận dạng tên người (NER): từ câu "Sally Johnson is an orange farmer" mô hình cần nhận ra "Sally Johnson" là tên người. Gặp câu mới "Robert Lin is an apple farmer", một mô hình dùng embedding sẽ làm đúng vì biết "apple" gần "orange". Thậm chí với "Robert Lin is a durian cultivator", dù tập huấn luyện nhỏ chưa từng có từ "durian" hay "cultivator", mô hình vẫn có cơ sở đoán đúng nếu embedding cho biết durian là một loại quả và cultivator giống farmer.

Điều làm được việc này là embedding được học từ lượng văn bản *không nhãn* rất lớn (từ 1 tỷ đến 100 tỷ từ, lấy gần như miễn phí trên Internet), còn tác vụ đích chỉ cần tập nhỏ có nhãn.



Hình 6.19: Quy trình chuyển giao kiến thức bằng embedding

Quy trình ba bước:

1. **Có embedding** – học từ kho văn bản lớn, hoặc tải embedding huấn luyện sẵn (nhiều bộ có giấy phép thoải mái).
2. **Nạp vào tác vụ đích** – thay vector one-hot hàng chục nghìn chiều bằng vector dày chỉ vài trăm chiều. Mô hình nhỏ hơn và khái quát tốt hơn.

3. **Tinh chỉnh (tuỳ chọn)** – cho phép embedding tiếp tục học bằng dữ liệu mới. Chỉ nên làm khi tập có nhãn đủ lớn; nếu nhỏ, cứ đóng băng.

Mức hiệu quả	Tác vụ
Hiệu quả rõ	nhận dạng thực thể tên (NER), tóm tắt văn bản, đồng tham chiếu (co-reference resolution), phân tích cú pháp, phân loại cảm xúc
Ít hiệu quả hơn	mô hình ngôn ngữ, dịch máy, khi đã có rất nhiều dữ liệu riêng cho đúng tác vụ đó

Bảng 6.14: Embedding giúp nhiều nhất khi dữ liệu có nhãn ít

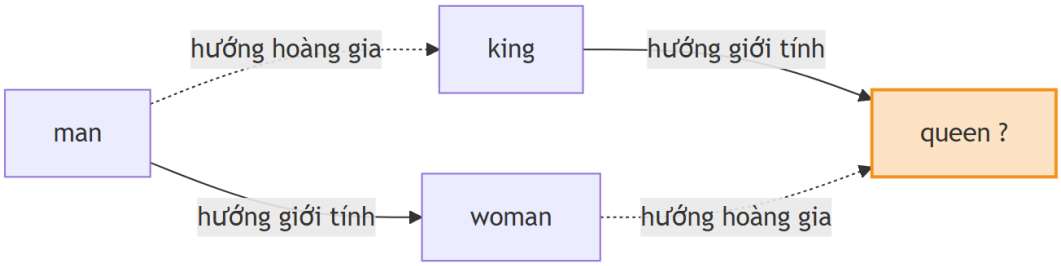
Quy tắc chung của chuyển giao kiến thức: có lợi nhất khi tác vụ nguồn có rất nhiều dữ liệu còn tác vụ đích có ít.

Encoding và embedding. Hai từ này gần như đồng nghĩa. Nhận dạng khuôn mặt cũng mã hoá mỗi ảnh thành một vector (encoding), nhưng mạng phải mã hoá được cả những ảnh *mới* chưa từng thấy. Với từ thì khác: ta có một từ điển cố định, mỗi từ có sẵn một vector cố định; từ ngoài từ điển được coi là từ lạ.

6.2.4 Loại suy và độ tương đồng cosine

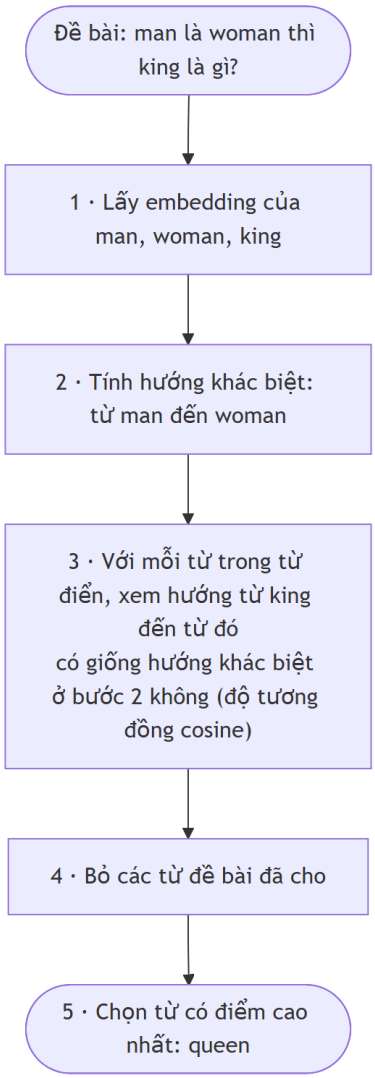
6.2.4.1 Phép loại suy

Một tính chất đáng ngạc nhiên của embedding là nó nắm được *quan hệ* giữa các từ. Hiệu giữa "man" và "woman" chủ yếu là giới tính; hiệu giữa "king" và "queen" cũng chủ yếu là giới tính. Vì thế hai khác biệt này gần như cùng một hướng trong không gian embedding. Trong hình học, bốn từ tạo thành một **hình bình hành** (Hình 6.20).



Hình 6.20: Hai cặp từ cùng khác nhau theo hướng giới tính, và cùng khác nhau theo hướng hoàng gia

Từ đó có thuật toán trả lời "man là woman thì king là gì?" (Hình 6.21): tìm từ sao cho hướng từ "king" tới từ đó giống hướng từ "man" tới "woman" nhất. Từ tìm được là "queen". Nghiên cứu gốc báo cáo độ chính xác khoảng 30% đến 75% tuỳ bộ dữ liệu, và chỉ tính là đúng khi đoán trúng chính xác từ đó.



Hình 6.21: Thuật toán giải phép loại suy

Embedding học được rất nhiều loại quan hệ chỉ bằng cách đọc văn bản:

Loại quan hệ	Ví dụ
Giới tính	man với woman như boy với girl
Thủ đô – quốc gia	Ottawa với Canada như Nairobi với Kenya
Dạng so sánh	big với bigger như tall với taller
Tiền tệ – quốc gia	yen với Japan như ruble với Russia

Bảng 6.15: Một số quan hệ mà embedding nắm được

6.2.4.2 Độ tương đồng cosine

Để biết hai vector "giống nhau" đến đâu, cách phổ biến nhất là **độ tương đồng cosine**: đo *góc* giữa hai vector, bỏ qua độ dài.

- Hai vector cùng hướng (góc 0 độ): giá trị 1, rất giống.
- Hai vector vuông góc (90 độ): giá trị 0, không liên quan.

- Hai vector ngược hướng (180 độ): giá trị âm 1, trái nghĩa theo hướng đó.

Cosine được dùng nhiều hơn khoảng cách Euclid vì chuẩn hoá theo độ dài của vector, chỉ so sánh hướng. Khoảng cách Euclid vẫn dùng được, chỉ cần đổi dấu để thành thước đo "giống nhau".

```
def cosine_similarity(u, v):
    return float(np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v)))

vec = lambda w: matrix[word_to_index[w]]
print(cosine_similarity(vec("father"), vec("mother")))      # cao: cùng là cha mẹ
print(cosine_similarity(vec("ball"), vec("crocodile")))     # thấp: không liên
↪  quan
```

6.2.4.3 Giải phép loại suy với gensim và NumPy

Cách ngắn nhất là dùng thư viện gensim, đã có sẵn phép tìm từ gần nhất theo hướng loại suy:

```
import gensim.downloader as api

glove = api.load("glove-wiki-gigaword-50")       # tải GloVe 50 chiều lần đầu;
↪  các lần sau dùng bản đã lưu
print(glove.most_similar(positive=["king", "woman"], negative=["man"], topn=3))
↪  # queen đứng đầu
print(glove.similarity("father", "mother"))
```

Cách tự viết bằng NumPy bám đúng mô tả ở trên: với mỗi từ trong từ điển, tính cosine giữa "hướng từ a đến b" và "hướng từ c đến từ đó", rồi chọn từ có điểm cao nhất. Viết gọn bằng phép tính trên cả ma trận (không cần vòng lặp qua 400.000 từ):

```
index_to_word = {i: w for w, i in word_to_index.items()}

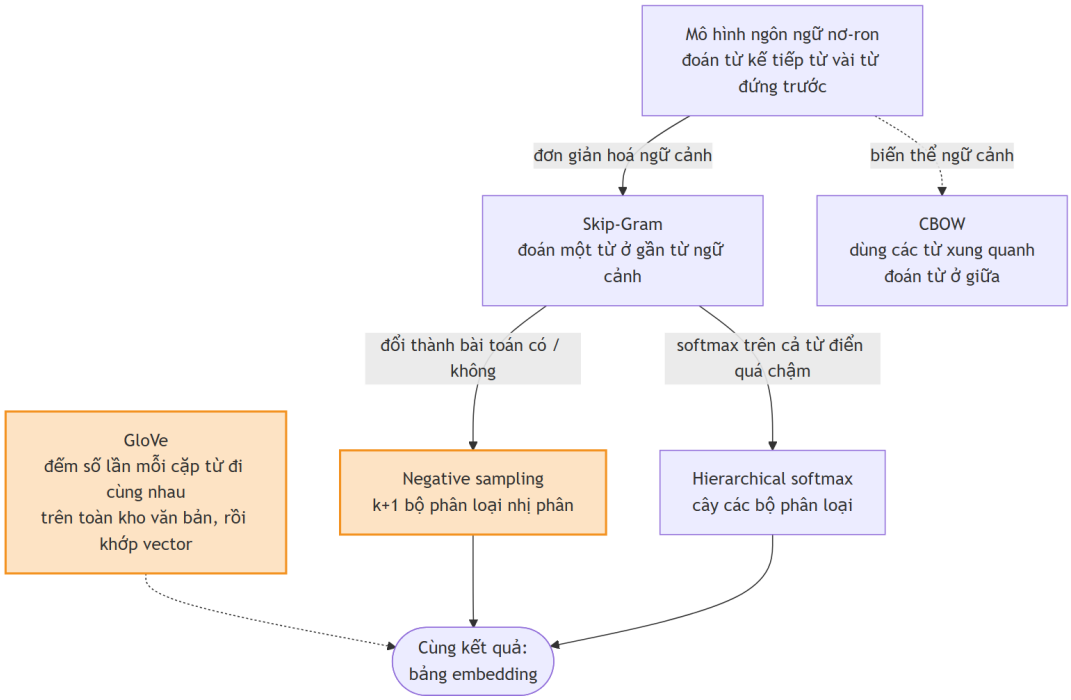
def complete_analogy(a, b, c, topn=1):
    """a là b thì c là gì?  Ví dụ: complete_analogy("man", "woman", "king") ->
    ↪  ["queen"]"""
    direction = vec(b) - vec(a)                                  # hướng từ a đến
    ↪  b
    diffs = matrix - vec(c)                                      # hướng từ c đến
    ↪  mọi từ
    scores = diffs @ direction / (np.linalg.norm(diffs, axis=1) *
    ↪  np.linalg.norm(direction) + 1e-9)
    scores[[0, word_to_index[c]]] = -np.inf                      # bỏ ô đệm và
    ↪  chính từ c
    return [index_to_word[i] for i in np.argsort(-scores)[:topn]]

for triad in [("italy", "italian", "spain"), ("india", "delhi", "japan"),
              ("man", "woman", "boy"), ("small", "smaller", "large")]:
    print(triad, "->", complete_analogy(*triad))
```

Hãy thử thêm những bộ ba mà kết quả sai, chẳng hạn small, smaller, big: loại suy không hoàn hảo, nhất là với embedding nhỏ.

6.2.5 Các thuật toán học embedding

Lịch sử khá thú vị: người ta bắt đầu bằng những thuật toán khá phức tạp, rồi dần phát hiện thuật toán càng đơn giản vẫn cho kết quả rất tốt khi dữ liệu đủ lớn. Mục này đi từ phức tạp đến đơn giản vì như vậy dễ hiểu *vì sao* chúng hoạt động hơn (Hình 6.22).



Hình 6.22: Các thuật toán học embedding và mối quan hệ giữa chúng

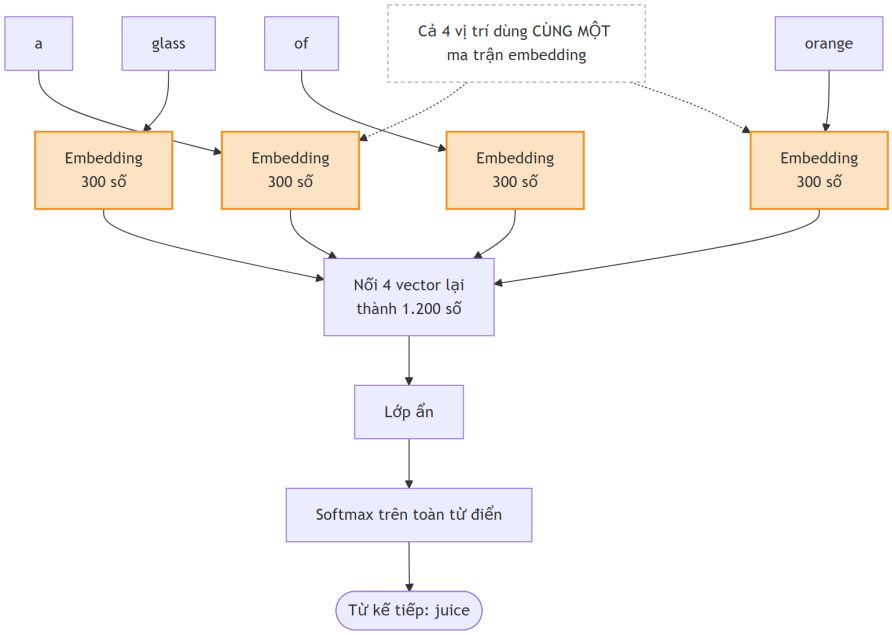
6.2.5.1 Mô hình ngôn ngữ nơ-ron: học embedding như một sản phẩm phụ

Ý tưởng đầu tiên: huấn luyện mô hình ngôn ngữ nơ-ron, đoán từ kế tiếp từ vài từ đứng trước. Với câu "I want a glass of orange ___" mô hình phải đoán "juice". Cấu trúc (Hình 6.23):

1. Lấy một **cửa sổ lịch sử cố định**, ví dụ 4 từ đứng trước (4 là siêu tham số). Cửa sổ cố định giúp đầu vào luôn cùng kích thước, dù câu ngắn hay dài.

2. Tra embedding của từng từ bằng **cùng một ma trận embedding**, rồi nối các vector lại (4 từ, mỗi từ 300 chiều thì được 1.200 số).

3. Đưa qua một lớp ẩn, rồi softmax trên toàn bộ từ điển để đoán từ kế tiếp.



Hình 6.23: Mô hình ngôn ngữ nơ-ron với cửa sổ 4 từ

Vì sao mô hình này học ra embedding tốt? Vì để đoán đúng, nó *có lợi* khi gán vector giống nhau cho các từ hay đi chung ngữ cảnh: apple, orange, grape, pear, thậm chí cả durian đều đứng sau "a glass of … juice". Tham số cần học gồm ma trận embedding (dùng chung cho mọi vị trí) và trọng số của lớp ẩn, lớp softmax.

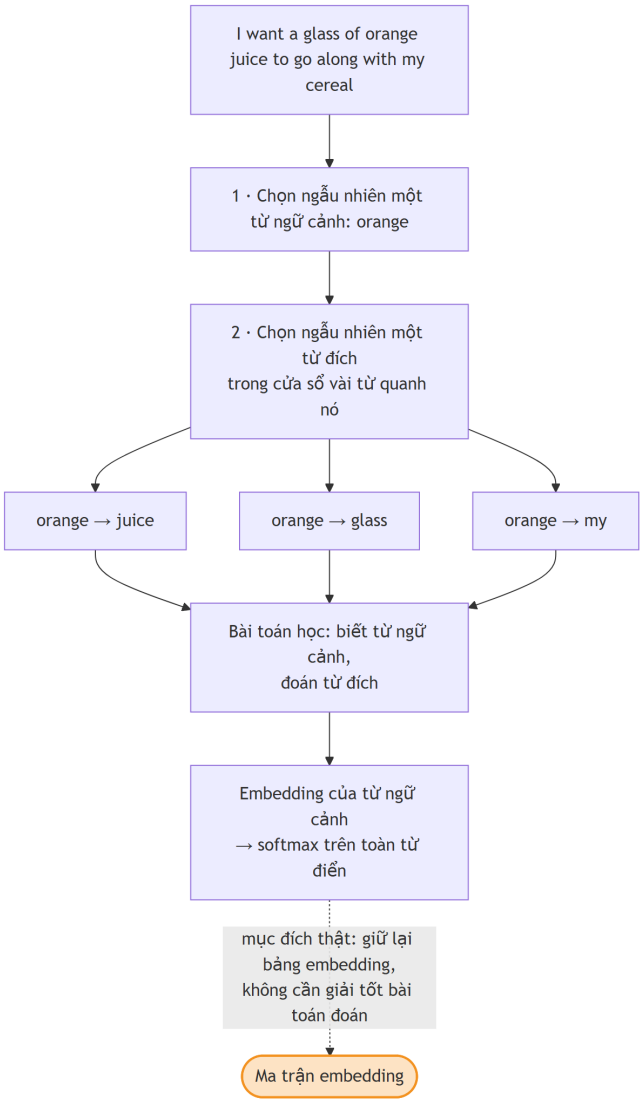
```
WINDOW, DIM = 4, 300
model = keras.Sequential([
    keras.Input(shape=(WINDOW,), dtype="int32"),
    layers.Embedding(VOCAB, DIM),                  # cùng một bảng cho cả 4 vị trí
    layers.Flatten(),                              # nối 4 vector thành 1.200 số
    layers.Dense(256, activation="tanh"),          # lớp ẩn
    layers.Dense(VOCAB, activation="softmax"),     # đoán từ kế tiếp trên toàn từ
    ↪  điển
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
# sau khi fit: model.layers[0].get_weights()[0] chính là bảng embedding
```

Ngữ cảnh có thể chọn khác. Nếu mục tiêu là mô hình ngôn ngữ thì ngữ cảnh tự nhiên là vài từ đứng ngay trước. Nhưng nếu mục tiêu chỉ là học embedding, ta có thể chọn ngữ cảnh khác: 4 từ bên trái và 4 từ bên phải để đoán từ ở giữa; hoặc chỉ 1 từ đứng ngay trước; hoặc *một từ bất kỳ ở gần*. Cách cuối đơn giản nhất mà lại hiệu quả đáng ngạc nhiên, và chính là Skip-Gram.

6.2.5.2 Word2Vec: Skip-Gram

Skip-Gram dựng bài toán học có giám sát một cách khác (Hình 6.24):

1. Chọn ngẫu nhiên một **từ ngữ cảnh**, ví dụ "orange".
2. Chọn ngẫu nhiên một **từ đích** trong cửa sổ cộng trừ 5 hoặc 10 từ quanh nó, ví dụ "juice", hoặc "glass", hoặc "my".
3. Huấn luyện mô hình đoán từ đích khi biết từ ngữ cảnh: tra embedding của từ ngữ cảnh, rồi đưa qua một lớp softmax trên toàn từ điển.



Hình 6.24: Cách Skip-Gram tạo cặp (ngữ cảnh, đích)

Bài toán này khó, vì quanh "orange" có thể xuất hiện rất nhiều từ khác nhau. Nhưng **mục đích không phải giải tốt bài toán đoán**; mục đích là có được bảng embedding tốt như một sản phẩm phụ. Tên "Skip-Gram" vì mô hình nhận một từ rồi đoán các từ *cách đó vài vị trí* về hai phía.

Vấn đề: softmax quá chậm. Mỗi lần tính xác suất của một từ đích, softmax phải cộng qua *toàn bộ* từ điển. Từ điển 10.000 từ đã chậm, từ điển 100.000 hay 1 triệu từ thì gần như không thể mở rộng. Có hai cách chữa:

- **Hierarchical softmax** – thay một bộ phân loại khổng lồ bằng một *cây* các bộ phân loại nhị phân ("từ đích nằm ở nửa đầu hay nửa sau của từ điển?", rồi chia nhỏ dần). Chi phí tăng theo logarit của kích thước từ điển thay vì tăng tuyến tính. Cây thực tế không cân bằng: từ phổ biến nằm gần gốc, từ hiếm như "durian" nằm sâu hơn.
- **Negative sampling** – cách được dùng phổ biến hơn, xem mục tiếp theo.

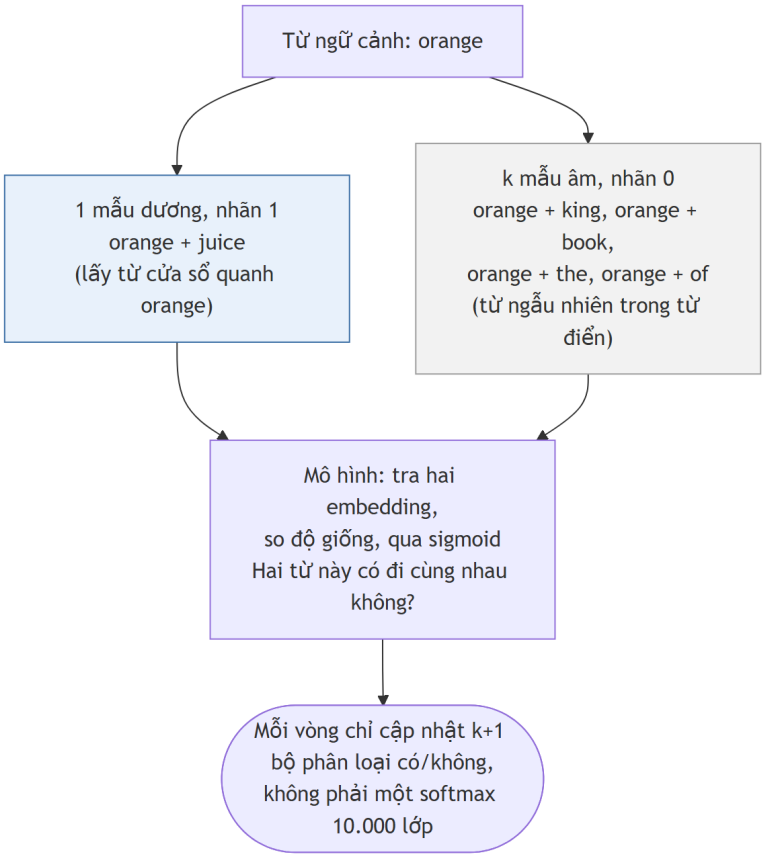
Chọn từ ngữ cảnh. Nếu chọn đồng đều ngẫu nhiên thì những từ như *the, of, a, and, to* chiếm gần hết dữ liệu, còn orange, apple, durian ít được cập nhật. Vì vậy thực tế dùng các heuristic để cân bằng giữa từ phổ biến và từ hiếm.

CBOW. Bài báo gốc của Word2Vec có hai mô hình. Ngoài Skip-Gram còn có CBOW (Continuous Bag-Of-Words), làm ngược lại: dùng các từ xung quanh để đoán từ ở giữa. Mỗi mô hình có ưu và nhược riêng.

6.2.5.3 Negative sampling: đổi bài toán thành có / không

Thay vì "đoán từ đích trong cả từ điển", negative sampling hỏi một câu đơn giản hơn nhiều: **cặp từ này có thật sự đi cùng nhau không?** (Hình 6.25).

- **Mẫu dương (nhãn 1):** chọn từ ngữ cảnh và một từ đích trong cửa sổ quanh nó, ví dụ (orange, juice).
- **Mẫu âm (nhãn 0):** giữ nguyên từ ngữ cảnh, ghép với k từ ngẫu nhiên lấy từ từ điển, ví dụ (orange, king), (orange, book), (orange, the). Có thể xảy ra việc từ ngẫu nhiên vô tình nằm gần "orange" trong văn bản, như "of"; vẫn gán nhãn 0 và điều đó chấp nhận được.



Hình 6.25: Negative sampling: một mẫu dương cùng k mẫu âm cho mỗi từ ngữ cảnh

Quy mô dữ liệu	Số mẫu âm k gợi ý (theo tác giả)
Tập nhỏ	5 đến 20
Tập lớn	2 đến 5

Bảng 6.16: Chọn số mẫu âm k

Vì sao nhanh hơn nhiều. Có thể coi như có 10.000 bộ phân loại có/không, mỗi bộ ứng với một từ đích. Nhưng mỗi vòng chỉ cần huấn luyện *k+1* bộ (một mẫu dương và k mẫu âm), thay vì cập nhật một softmax 10.000 lớp.

Chọn từ cho mẫu âm. Lấy theo đúng tần suất thật thì quá nhiều từ như the, of, and. Lấy đồng đều thì không phản ánh phân bố tiếng Anh. Các tác giả chọn cách ở giữa: xác suất chọn một từ tỷ lệ với tần suất của từ đó *lũy thừa ba phần tư*. Cách này chưa có lý giải lý thuyết chặt chẽ nhưng được nhiều nhà nghiên cứu dùng và hoạt động tốt.

6.2.5.4 GloVe: đếm toàn cục

GloVe (Global Vectors for Word Representation) đi theo hướng khác. Thay vì lấy mẫu từng cặp từ, nó **đếm trước** trên toàn kho văn bản: mỗi cặp từ xuất hiện gần nhau (ví dụ trong cửa sổ cộng trừ 10 từ) bao nhiêu lần. Sau đó học các vector sao cho tích của hai vector dự đoán tốt mức độ hai từ hay đi cùng nhau. Thực chất đây là một bài toán khớp bình phương tối thiểu rất đơn giản.

Hai chi tiết đáng nhớ:

- **Hàm trọng số:** cặp từ chưa từng đi cùng nhau thì bị bỏ qua. Hàm này cũng cân bằng giữa từ rất phổ biến (this, is, of, a) và từ hiếm (durian): từ hiếm vẫn được tính đáng kể, từ phổ biến không bị cho quá nhiều trọng số.
- **Hai bộ vector đối xứng:** hai bộ tham số đóng vai trò hoàn toàn giống nhau, nên sau khi huấn luyện lấy trung bình của chúng làm embedding cuối cùng.

GloVe ít được dùng hơn Word2Vec về mặt huấn luyện, nhưng có nhiều người ưa chuộng vì đơn giản, và các bộ GloVe huấn luyện sẵn được dùng rất rộng rãi.

Thuật toán	Ý tưởng	Điểm cần nhớ
Mô hình ngôn ngữ nơ-ron	đoán từ kế tiếp từ vài từ trước	phức tạp nhất, dễ hiểu nhất
Skip-Gram + softmax	đoán từ ở gần từ ngữ cảnh	softmax trên cả từ điển rất chậm
Skip-Gram + negative sampling	phân loại có/không cho cặp từ	nhanh, phổ biến, khuyên dùng
CBOW	dùng từ xung quanh đoán từ ở giữa	biến thể ngược của Skip-Gram
GloVe	khớp vector với số lần cặp từ đi cùng nhau	đếm toàn cục, đơn giản

Bảng 6.17: So sánh các thuật toán học embedding

Các chiều không diễn giải được. Với mọi thuật toán trên, không có gì đảm bảo chiều số 1 là "giới tính" hay chiều số 2 là "hoàng gia". Trục học được có thể bị xoay hoặc trộn lẫn so với trục dễ hiểu, tương đương một phép biến đổi tuyến tính bất kỳ. Dù vậy, quan hệ hình bình hành dùng cho phép loại suy vẫn còn nguyên.

6.2.5.5 Học embedding bằng gensim

Trong dự án, ít khi tự cài đặt các thuật toán trên. Thư viện **gensim** huấn luyện Word2Vec hiệu quả chỉ bằng vài dòng:

```
from gensim.models import Word2Vec

# sentences: danh sách các câu, mỗi câu là danh sách từ đã tách và hạ chữ thường
sentences = [["i", "want", "a", "glass", "of", "orange", "juice"],
```

```
            ["i", "want", "a", "glass", "of", "apple", "juice"]]   # thực tế:
            ↪ hàng triệu câu

w2v = Word2Vec(
    sentences,
    vector_size=100,   # số chiều embedding
    window=5,          # cửa sổ quanh từ ngữ cảnh
    min_count=1,       # bỏ từ xuất hiện ít hơn ngưỡng này (corpus thật: 5 trở
    ↪ lên)
    sg=1,              # 1 = Skip-Gram, 0 = CBOW
    negative=5,        # số mẫu âm k
    epochs=5,
    workers=4)

print(w2v.wv.most_similar("orange", topn=3))
w2v.wv.save("my_vectors.kv")      # lưu bảng embedding, nạp lại bằng
↪ KeyedVectors.load
```

Tham số gensim	Khái niệm trong mục này
`sg=1` / `sg=0`	Skip-Gram / CBOW
`window`	cửa sổ quanh từ ngữ cảnh
`negative`	số mẫu âm k (đặt 0 nếu muốn dùng hierarchical softmax, cùng `hs=1`)
`ns_exponent`	số mũ cho tần suất khi chọn mẫu âm, mặc định đúng 0,75 (ba phần tư)
`vector_size`	số chiều embedding
`min_count`	bỏ từ quá hiếm

Bảng 6.18: Đối chiếu tham số `Word2Vec` của gensim với các khái niệm trong mục này

6.2.5.6 Negative sampling viết bằng Keras (để hiểu cơ chế)

Đoạn code dưới đây cài đặt đúng ý tưởng ở trên với corpus nhỏ, chỉ nhằm minh hoạ. Mô hình có hai bảng embedding (một cho từ ngữ cảnh, một cho từ đích), đo độ giống bằng tích vô hướng rồi qua sigmoid để đoán nhãn có/không.

```
from collections import Counter

words = open("corpus.txt", encoding="utf-8").read().lower().split()
counts = Counter(words)
vocab = [w for w, _ in counts.most_common()]
word_to_index = {w: i + 1 for i, w in enumerate(vocab)}      # 0 dành cho đệm
ids = [word_to_index[w] for w in words]
V = len(word_to_index) + 1

# Xác suất chọn từ cho mẫu âm: tần suất mũ 3/4
neg_probs = np.array([counts[w] for w in vocab], dtype="float64") ** 0.75
neg_probs /= neg_probs.sum()

def make_training_pairs(ids, window=5, k=4, seed=0):
    rng = np.random.default_rng(seed)
    centers, others = [], []
    for pos, c in enumerate(ids):                                  # mẫu dương: từ
    ↪ nằm trong cửa sổ
        for j in range(max(0, pos - window), min(len(ids), pos + window + 1)):
            if j != pos:
                centers.append(c); others.append(ids[j])
```

```
    centers, others = np.array(centers), np.array(others)
    neg_centers = np.repeat(centers, k)                          # mẫu âm: cùng từ
    ↪  trung tâm, k từ ngẫu nhiên
    neg_others = rng.choice(np.arange(1, V), size=len(neg_centers), p=neg_probs)
    x_center = np.concatenate([centers, neg_centers])
    x_other = np.concatenate([others, neg_others])
    y = np.concatenate([np.ones(len(centers)), np.zeros(len(neg_centers))])
    return x_center, x_other, y

x_center, x_other, y = make_training_pairs(ids)

DIM = 100
center_emb = layers.Embedding(V, DIM, name="center_embedding")
other_emb = layers.Embedding(V, DIM, name="other_embedding")
c_in = keras.Input(shape=(1,), dtype="int32")
o_in = keras.Input(shape=(1,), dtype="int32")
score = layers.Dot(axes=1)([layers.Flatten()(center_emb(c_in)),
                            layers.Flatten()(other_emb(o_in))])  # độ giống của
                             ↪  hai từ
out = layers.Activation("sigmoid")(score)                        # xác suất "đi
↪  cùng nhau"

model = keras.Model([c_in, o_in], out)
model.compile(optimizer="adam", loss="binary_crossentropy")
model.fit([x_center[:, None], x_other[:, None]], y, batch_size=1024, epochs=5)

embeddings = center_emb.get_weights()[0]       # (V, DIM): bảng embedding cần tìm
```

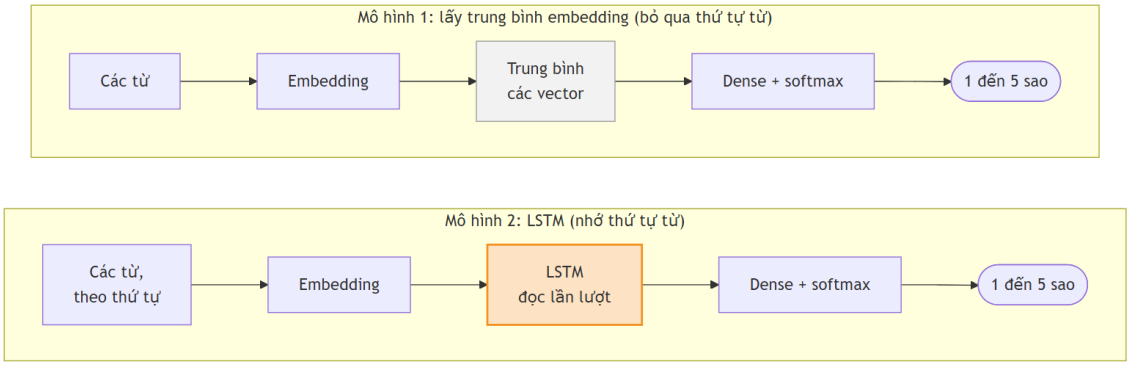
6.2.6 Phân loại cảm xúc bằng embedding

Phân loại cảm xúc (sentiment classification) là nhìn một đoạn văn rồi đoán người viết thích hay không thích điều họ nói tới, ví dụ dự đoán số sao của một bài đánh giá nhà hàng:

Câu đánh giá	Số sao
The dessert is excellent	4
Service was quite slow	2
Good for a quick meal but nothing special	3
Completely lacking in good taste, good service, and good ambiance	1

Bảng 6.19: Ví dụ dữ liệu phân loại cảm xúc

Ứng dụng điển hình: theo dõi bình luận về nhà hàng trên mạng xã hội để biết phản hồi đang tốt lên hay xấu đi. Thách thức là tập có nhãn thường nhỏ (cỡ 10.000 đến 100.000 từ, đôi khi ít hơn), và embedding học từ kho văn bản lớn bù đắp đúng phần thiếu này.



Hình 6.26: Hai cách xây bộ phân loại cảm xúc từ embedding

6.2.6.1 Mô hình 1: lấy trung bình embedding

Đổi mỗi từ thành embedding, lấy **trung bình** các vector, rồi đưa qua softmax để dự đoán 1 đến 5 sao. Ưu điểm: xử lý được bài đánh giá dài hay ngắn tuỳ ý, vì luôn ra một vector có kích thước cố định. Nhược điểm: **bỏ qua thứ tự từ**. Với câu một sao "Completely lacking in good taste, good service, and good ambiance", từ "good" xuất hiện nhiều lần nên vector trung bình nghiêng về tích cực và mô hình có thể đoán sai.

6.2.6.2 Mô hình 2: LSTM

Đưa chuỗi embedding vào một LSTM kiểu nhiều–một, rồi lấy trạng thái ở bước cuối để dự đoán. LSTM đọc từ lần lượt nên hiểu được "lacking in good taste" hay "not good" là tiêu cực. Nhờ embedding học từ kho văn bản lớn, mô hình còn xử lý được cả từ không có trong tập nhãn: "Completely absent of good taste …" vẫn được hiểu đúng dù "absent" chưa từng xuất hiện khi huấn luyện bộ phân loại.

```
# embedding_layer: lớp Embedding GloVe đóng băng đã tạo ở mục "Nạp embedding huấn
↪  luyện sẵn"

model_avg = keras.Sequential([
    keras.Input(shape=(None,), dtype="int32"),
    embedding_layer,
    layers.GlobalAveragePooling1D(),                  # trung bình các vector, bỏ
    ↪  qua phần đệm
    layers.Dense(5, activation="softmax"),
])

model_lstm = keras.Sequential([
    keras.Input(shape=(None,), dtype="int32"),
    embedding_layer,
    layers.LSTM(64),                                  # nhiều -> một: chỉ lấy
    ↪  trạng thái cuối
    layers.Dense(5, activation="softmax"),
])

for m in (model_avg, model_lstm):
    m.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
    ↪  metrics=["accuracy"])
```

6.2.7 Thiên kiến trong embedding

Thuật toán học máy ngày càng tham gia các quyết định quan trọng: tuyển sinh, tuyển dụng, duyệt vay vốn, thậm chí hướng dẫn kết án. Vì vậy cần giảm tối đa các thiên kiến không mong muốn (giới tính, sắc tộc, xu hướng tính dục, địa vị kinh tế). "Thiên kiến" ở đây là định kiến xã hội, khác với "bias" (độ chệch) trong thảo luận kỹ thuật về học máy.

6.2.7.1 Embedding thừa hưởng định kiến của văn bản

Embedding học từ văn bản do con người viết nên mang theo định kiến trong văn bản đó. Nghiên cứu của Bolukbasi và cộng sự cho thấy có embedding trả lời "man là computer programmer thì woman là homemaker" hoặc "father là doctor thì mother là nurse". Có thể tái hiện hiện tượng này: lấy hiệu giữa "woman" và "man" làm hướng giới tính rồi đo cosine của các từ với hướng đó. Tên nữ ra dương, tên nam ra âm (hợp lý), nhưng "computer" ngả về phía nam còn "literature" ngả về phía nữ – phản ánh định kiến.

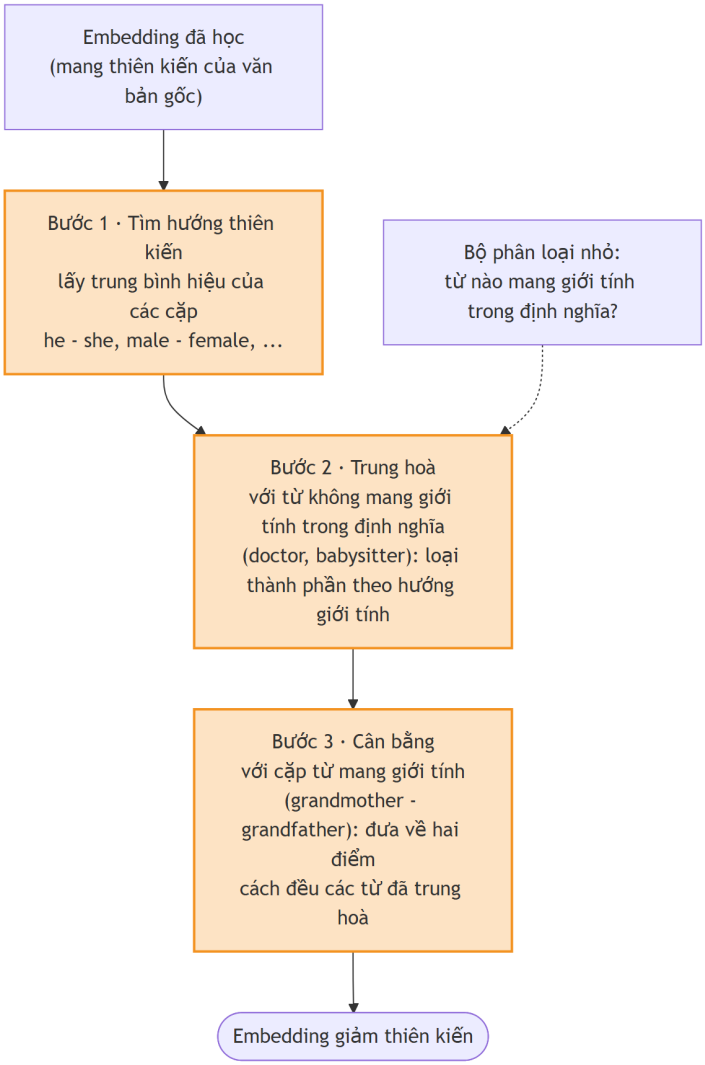
6.2.7.2 Quy trình giảm thiên kiến ba bước

Các bước dưới đây lấy giới tính làm ví dụ nhưng áp dụng được cho các loại thiên kiến khác (Hình 6.27):

1. **Tìm hướng thiên kiến.** Lấy hiệu của các cặp từ chỉ khác nhau về giới tính (he và she, male và female …) rồi lấy trung bình. Hướng này là một trục trong không gian 300 chiều; phần còn lại là 299 chiều không liên quan đến giới tính. (Bài báo gốc dùng phương pháp phức tạp hơn gọi là SVD, gần với PCA, và hướng thiên kiến có thể có hơn một chiều.)

2. **Trung hoà (neutralize).** Với các từ *không* mang giới tính trong định nghĩa như doctor hay babysitter, loại bỏ thành phần theo hướng giới tính. Từ vốn có giới tính như grandmother, grandfather, girl, boy, she, he thì không trung hoà.

3. **Cân bằng (equalize).** Với các cặp từ chỉ nên khác nhau về giới tính, như grandmother và grandfather, đưa chúng về hai điểm cách đều các từ đã trung hoà. Nhờ đó khoảng cách từ "babysitter" tới hai từ này bằng nhau, không còn ngầm gợi ý rằng bà trông trẻ nhiều hơn ông.



Hình 6.27: Quy trình giảm thiên kiến trong embedding

Cần quyết định từ nào được trung hoà. Các tác giả huấn luyện một bộ phân loại để tách từ mang giới tính trong định nghĩa khỏi từ không mang. Phần lớn từ tiếng Anh không mang giới tính; chỉ một nhóm nhỏ (cặp như grandmother–grandfather, girl–boy) cần giữ nguyên. Số cặp cần cân bằng cũng ít, nên với giới tính có thể chọn tay phần lớn.

6.2.7.3 Đo và trung hoà bằng code

Hàm `neutralize` chỉ gồm vài dòng: tính phần của vector nằm theo hướng thiên kiến rồi trừ đi. Dưới đây dùng bộ GloVe lấy từ gensim (đối tượng `glove` ở mục trước).

```
g = glove["woman"] - glove["man"]                    # hướng giới tính, xấp xỉ
↪  bằng một cặp từ

for w in ["john", "marie", "technology", "receptionist", "computer",
↪  "literature"]:
    print(w, cosine_similarity(glove[w], g))         # dương: nghiêng nữ, âm:
    ↪  nghiêng nam

def neutralize(word, bias_axis, kv):
    """Loại thành phần theo hướng thiên kiến khỏi vector của từ trung tính."""
```

```
    e = kv[word]
    bias_part = (e @ bias_axis) / (bias_axis @ bias_axis) * bias_axis
    return e - bias_part

e_debiased = neutralize("receptionist", g, glove)
print(cosine_similarity(e_debiased, g))              # gần 0 sau khi trung hoà
```

Khi thử với GloVe 50 chiều, độ tương đồng của "receptionist" với hướng giới tính giảm từ khoảng 0,33 xuống gần 0 sau khi trung hoà (vector được chuẩn hoá về độ dài 1 trước, theo bài báo gốc). Với bước cân bằng: sau khi cân bằng cặp "man" và "woman", cosine với hướng giới tính của chúng đối xứng nhau, từ khoảng âm 0,12 và 0,36 thành khoảng âm 0,24 và 0,24.

Hạn chế. Các thuật toán này giảm thiên kiến nhưng không xoá hết dấu vết. Ở ví dụ trên, hướng giới tính chỉ được định nghĩa bằng một cặp từ; lấy trung bình nhiều cặp (mother–father, girl–boy, …) sẽ ước lượng tốt hơn. Đây vẫn là lĩnh vực đang được nghiên cứu tích cực, nên với ứng dụng có tác động đến con người, cần kiểm tra thiên kiến trên chính dữ liệu và mô hình của bạn thay vì tin hoàn toàn vào một bước khử thiên kiến.

6.2.8 Thực hành 1: Phép toán trên vector từ

Bài toán: làm quen với embedding huấn luyện sẵn – tải lên, đo độ giống, giải phép loại suy – rồi khám phá và giảm thiên kiến giới tính. Dữ liệu là GloVe 50 chiều.

6.2.8.1 Những ý chính

- **Tải vector huấn luyện sẵn.** Học embedding rất tốn kém, nên người làm thực tế thường tải sẵn. Hàm đọc trả về tập từ và một từ điển ánh xạ từ sang vector.
- **One-hot và embedding.** Mọi cặp vector one-hot đều cách nhau như nhau, còn embedding thì không: đây là lý do cần embedding.
- **Độ tương đồng cosine.** Hàm cần xử lý hai trường hợp đặc biệt: hai vector bằng nhau, và tránh chia cho 0. Bài thử với các cặp như father–mother (cao) và ball–crocodile (thấp).
- **Giải phép loại suy.** Duyệt toàn bộ từ điển, bỏ qua chính từ c, rồi chọn từ có độ tương đồng cosine lớn nhất. Mỗi lần chạy mất 1 đến 2 phút vì duyệt cả 400.000 từ. Các bộ ba thử: italy–italian::spain, india–delhi::japan, man–woman::boy, small–smaller::large. Hãy thử thêm những bộ ba mà thuật toán sai để cảm nhận giới hạn của nó.
- **Giảm thiên kiến (tuỳ chọn).** Đo thiên kiến bằng hướng giới tính, rồi trung hoà và cân bằng như ở mục 6.2.7. Lưu ý: vector được chuẩn hoá về độ dài 1 trước khi trung hoà để khớp bài báo gốc.

6.2.8.2 Điều cần nhớ

- Độ tương đồng cosine là cách tốt để so sánh cặp vector từ (khoảng cách Euclid cũng dùng được).
- Với ứng dụng NLP, embedding huấn luyện sẵn thường là điểm khởi đầu rất tốt.

6.2.8.3 Làm gọn bằng thư viện

Các đoạn tự viết ở trên có sẵn trong gensim, nên trong dự án chỉ cần vài dòng. Cần tự viết thêm chỉ khi muốn hiểu cơ chế hoặc làm điều thư viện chưa hỗ trợ (như bước trung hoà).

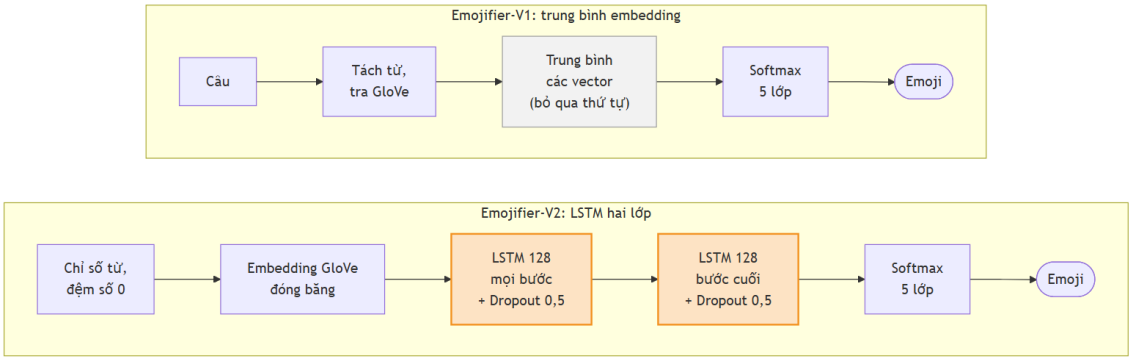
```
import gensim.downloader as api
glove = api.load("glove-wiki-gigaword-50")

glove.similarity("father", "mother")                                    #
↪  cosine của hai từ
glove.most_similar("orange", topn=5)                                    #
↪  từ gần nhất
glove.most_similar(positive=["spain", "italian"], negative=["italy"], topn=3)  #
↪  italy:italian = spain:?
glove.doesnt_match(["breakfast", "cereal", "dinner", "lunch"])          #
↪  từ lạc nhóm
```

6.2.9 Thực hành 2: Emojify – chọn emoji cho câu

Bài toán: đưa vào một câu như "Let's go see the baseball game tonight!", mô hình chọn emoji phù hợp nhất (ở đây là quả bóng chày). Bộ dữ liệu EMOJISET nhỏ: 127 câu huấn luyện, 56 câu kiểm thử, 5 lớp emoji. Đây là bài phân loại *nhiều–một* nhưng với dữ liệu rất ít, nên là ví dụ rất rõ về sức mạnh của embedding huấn luyện sẵn.

Giao diện chọn emoji thông thường bắt người dùng nhớ đúng từ khoá ("heart" chứ không phải "love"). Dùng embedding thì mô hình khái quát được: dù tập huấn luyện chỉ gắn vài từ với một emoji, nó vẫn liên kết thêm được những từ gần nghĩa chưa từng thấy.



Hình 6.28: Hai mô hình Emojify

6.2.9.1 Emojifier-V1: trung bình embedding

Mô hình nền: tách câu thành từ, tra GloVe cho từng từ, lấy trung bình các vector (từ không có trong từ điển thì bỏ qua), rồi đưa qua softmax 5 lớp. Có thể tự cài bằng NumPy với vòng lặp để dễ gỡ lỗi; nhãn được chuyển sang dạng one-hot để tính lỗi cross-entropy.

Kết quả đáng chú ý: chỉ với 127 mẫu mô hình đã khá tốt (đoán ngẫu nhiên chỉ đúng 20%). Câu "i treasure you" được gán trái tim dù "treasure" chưa từng có trong tập huấn luyện, vì vector của nó gần "love" (cũng như heart, dear, beloved, adore). **Điểm yếu:** bỏ qua thứ tự từ, nên "today is not good" bị đoán sai, và các câu như "This movie is not good and not enjoyable" cũng sẽ sai vì mô hình không hiểu tổ hợp từ. Ma trận nhầm lẫn (confusion matrix) cho biết lớp emoji nào hay bị đoán nhầm sang lớp nào.

6.2.9.2 Emojifier-V2: LSTM hai lớp

Mô hình tốt hơn nhận *chuỗi* từ nên tính đến thứ tự từ. Ba điểm kỹ thuật:

- **Đệm (padding).** Mini-batch đòi hỏi mọi câu cùng độ dài, vì cần tính toán cùng một lúc. Đặt độ dài tối đa (thường là độ dài câu dài nhất trong tập huấn luyện), đệm số 0 cho câu ngắn hơn và cắt câu dài hơn.
- **Lớp Embedding khởi tạo bằng GloVe và đóng băng.** Vì tập huấn luyện nhỏ nên giữ nguyên embedding thay vì cho học thêm. Mô hình có khoảng 20,2 triệu tham số, trong đó 20.000.050 (400.001 từ nhân 50 chiều) là embedding cố định; chỉ khoảng 224 nghìn tham số được học.
- **Hai lớp LSTM xếp chồng.** Lớp đầu dùng `return_sequences=True` để trả về mọi bước cho lớp sau; lớp thứ hai chỉ trả về bước cuối. `Dropout` đặt ngay sau mỗi lớp LSTM để chống học vẹt.

Cấu hình huấn luyện: 50 epoch, batch 32, hàm lỗi `categorical_crossentropy`, optimizer `adam`. Kết quả mong đợi: độ chính xác huấn luyện 90–100%, kiểm thử 80–95%.

6.2.9.3 Viết bằng Keras

Phần dưới viết bằng Keras, dùng `sparse_categorical_crossentropy` nên không cần đổi nhãn sang one-hot. Giả sử các biến `X_train`, `Y_train`, `X_test`, `Y_test` là danh sách câu và nhãn 0–4, còn `word_to_index` và `embedding_layer` đã tạo ở mục 6.2.2.

```
max_len = max(len(s.split()) for s in X_train)

def to_indices(sentences, max_len):
    """Câu -> dãy chỉ số từ (bỏ từ ngoài từ điển), đệm số 0 về cuối cho đủ
    ↪  max_len."""
    seqs = [[word_to_index[w] for w in s.lower().split() if w in word_to_index]
    ↪  for s in sentences]
    return keras.utils.pad_sequences(seqs, maxlen=max_len, padding="post",
    ↪  value=0)

X_train_idx, X_test_idx = to_indices(X_train, max_len), to_indices(X_test,
↪  max_len)

# V1: trung bình embedding -> softmax (bỏ qua thứ tự từ)
model_v1 = keras.Sequential([
    keras.Input(shape=(max_len,), dtype="int32"),
    embedding_layer,                                    # GloVe 50 chiều, đóng
    ↪  băng, mask_zero=True
    layers.GlobalAveragePooling1D(),
    layers.Dense(5, activation="softmax"),
])

# V2: LSTM hai lớp (nhớ thứ tự từ)
model_v2 = keras.Sequential([
    keras.Input(shape=(max_len,), dtype="int32"),
    embedding_layer,
    layers.LSTM(128, return_sequences=True),          # trả về mọi bước cho lớp
    ↪  sau
    layers.Dropout(0.5),
    layers.LSTM(128),                                 # chỉ lấy trạng thái cuối
    layers.Dropout(0.5),
    layers.Dense(5, activation="softmax"),
])

for name, m in [("V1", model_v1), ("V2", model_v2)]:
    m.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
    ↪  metrics=["accuracy"])
```

```
    m.fit(X_train_idx, np.array(Y_train), epochs=50, batch_size=32, shuffle=True,
    ↪  verbose=0)
    print(name, "test accuracy:", m.evaluate(X_test_idx, np.array(Y_test),
    ↪  verbose=0)[1])
```

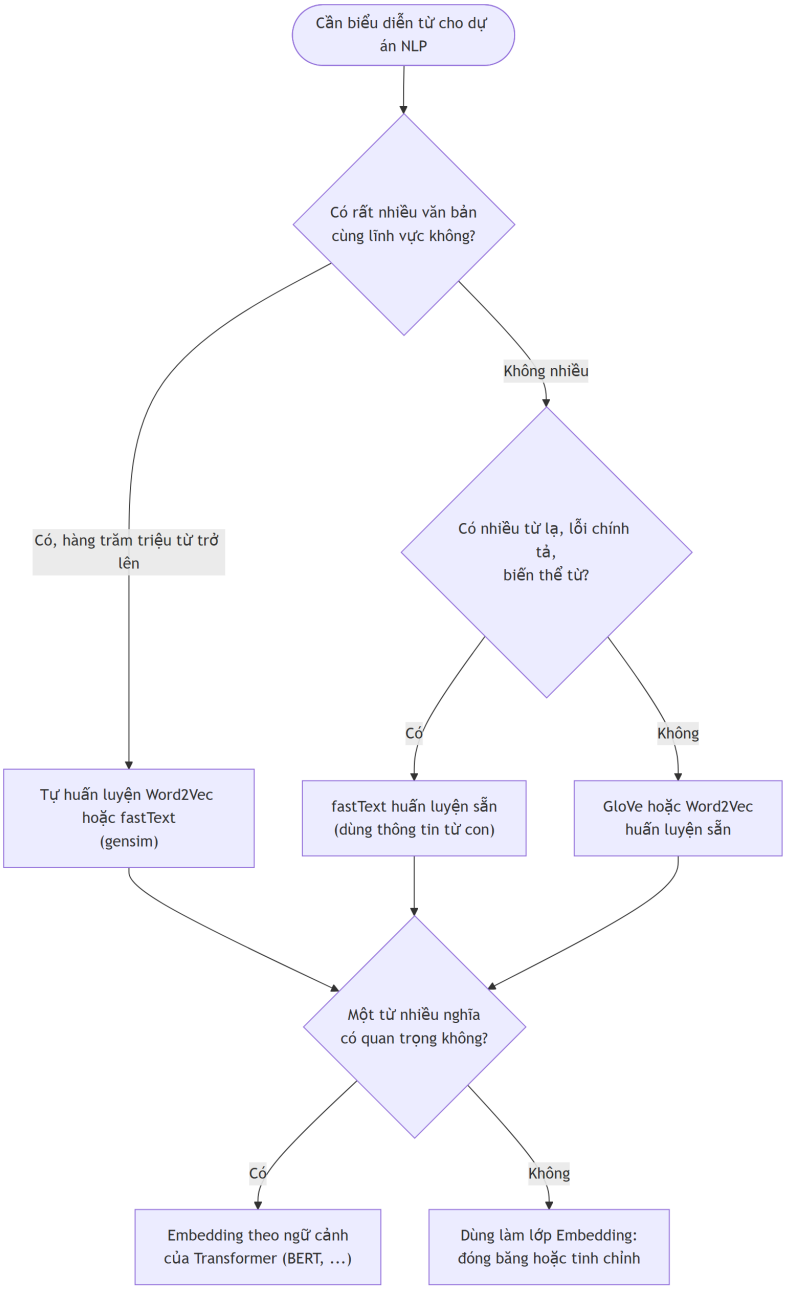
Tiêu chí	Emojifier-V1	Emojifier-V2
Cách gộp các từ	trung bình vector	LSTM đọc lần lượt
Thứ tự từ	bỏ qua	có xét
Câu phủ định (“not good”)	hay sai	xử lý tốt hơn
Tham số học được	rất ít (chỉ lớp softmax)	khoảng 224 nghìn
Khi nào dùng	baseline nhanh, kiểm tra dữ liệu	khi cần hiểu tổ hợp từ

Bảng 6.20: So sánh hai mô hình của bài Emojify

Bài học chung. Khi dữ liệu có nhãn ít, hãy bắt đầu bằng mô hình nền đơn giản dùng embedding huấn luyện sẵn (trung bình vector). Chỉ chuyển sang mô hình chuỗi khi mô hình nền mắc lỗi vì thứ tự từ. Embedding huấn luyện sẵn mang lại phần lớn sức mạnh khái quát, mô hình chuỗi bổ sung khả năng hiểu tổ hợp từ.

6.2.10 Áp dụng vào dự án thực tế

6.2.10.1 Chọn nguồn embedding



Hình 6.29: Cây quyết định chọn nguồn embedding

Lựa chọn	Ưu điểm	Hạn chế
Học từ đầu cùng mô hình (`Embedding` khởi tạo ngẫu nhiên)	thích nghi hoàn toàn với dữ liệu	cần nhiều dữ liệu có nhãn
GloVe, Word2Vec huấn luyện sẵn	dùng ngay, khái quát tốt	từ ngoài từ điển phải coi là từ lạ; một từ chỉ có một vector
fastText huấn luyện sẵn	dựng được vector cho từ lạ nhờ thông tin từ con	tệp lớn hơn
Tự huấn luyện bằng gensim trên văn bản của lĩnh vực	sát từ vựng chuyên ngành	cần nhiều văn bản không nhãn
Embedding theo ngữ cảnh của Transformer (BERT, …)	hiểu từ nhiều nghĩa theo ngữ cảnh	nặng hơn, cần GPU

Bảng 6.21: Các nguồn embedding và đánh đổi

6.2.10.2 Checklist

1. **Bắt đầu bằng mô hình nền đơn giản:** embedding huấn luyện sẵn, trung bình vector, một lớp softmax. Biết mức chuẩn trước khi làm phức tạp hơn.
2. **Chuẩn hoá văn bản giống lúc huấn luyện embedding:** hạ chữ thường, cùng cách tách từ. GloVe 6B chẳng hạn chỉ chứa chữ thường.
3. **Dành chỉ số 0 cho đệm** và bật `mask_zero=True`.
4. **Dữ liệu có nhãn ít → đóng băng embedding;** chỉ tinh chỉnh khi có nhiều dữ liệu, kèm learning rate nhỏ.
5. **Đo số từ ngoài từ điển** trong dữ liệu của bạn. Nếu nhiều, cân nhắc fastText hoặc embedding theo ngữ cảnh.
6. **Kiểm tra thiên kiến** trên embedding và đầu ra mô hình nếu quyết định ảnh hưởng đến con người.
7. **Luôn so với mô hình nền** khi chuyển sang LSTM hoặc Transformer, để biết thêm độ phức tạp có đáng không.

6.2.10.3 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Mô hình học cả phần đệm số 0	thiếu `mask_zero=True`, hoặc chỉ số 0 bị gán cho một từ thật
Lỗi chỉ số vượt quá kích thước bảng embedding	có chỉ số từ lớn hơn hoặc bằng `input_dim`; nhớ cộng 1 cho ô đệm
Nạp GloVe mà kết quả không tốt hơn học từ đầu	quên đóng băng, hoặc văn bản chưa hạ chữ thường nên hầu hết từ rơi vào từ lạ
Phép loại suy cho từ chính là một trong ba từ đề bài	chưa loại các từ đã cho khỏi danh sách ứng viên
Vẽ t-SNE rồi thấy quan hệ loại suy bị méo	bình thường: t-SNE nén phi tuyến, hãy kiểm tra trong không gian gốc
Khoảng cách hai từ trái nghĩa lại rất gần	trái nghĩa hay xuất hiện trong cùng ngữ cảnh nên embedding coi là gần; cần dữ liệu có nhãn hoặc mô hình theo ngữ cảnh
Mô hình cảm xúc sai với câu phủ định	dùng mô hình trung bình bỏ qua thứ tự; chuyển sang LSTM hoặc Transformer

Bảng 6.22: Lỗi thường gặp khi làm việc với embedding

6.2.10.4 Giới hạn của embedding tĩnh

Word2Vec và GloVe cho **mỗi từ đúng một vector**, bất kể ngữ cảnh. Từ "bank" trong "river bank" và "bank account" vì thế có chung một vector. Đây là giới hạn lớn nhất của embedding tĩnh. Các mô hình dựa trên Transformer tạo vector *theo ngữ cảnh* nên xử lý tốt hơn các trường hợp này, và ngày nay thường là điểm khởi đầu của nhiều bài toán NLP. Dù vậy, hiểu embedding tĩnh vẫn rất quan trọng: nó là nền tảng của ý tưởng "biểu diễn từ bằng vector học được", nhẹ, nhanh, và vẫn là phương án tốt cho dữ liệu ít hoặc thiết bị yếu.

6.2.11 Tóm tắt cần nhớ

1. **One-hot** cho mọi từ cách đều nhau và không khái quát được; **embedding** mô tả từ bằng vector dày, từ gần nghĩa nằm gần nhau.
2. **Ma trận embedding** là bảng tra: lấy vector của một từ chỉ là tra theo chỉ số. Trong Keras đó là lớp `Embedding`.
3. **Chuyển giao kiến thức:** học embedding từ kho văn bản lớn không nhãn rồi dùng cho tác vụ có ít nhãn. Dữ liệu nhãn ít thì đóng băng, nhiều thì mới tinh chỉnh.
4. **Loại suy:** hiệu giữa hai từ nắm được quan hệ (man–woman giống king–queen). Dùng **độ tương đồng cosine** để so sánh. Quan hệ này đáng tin trong không gian gốc, không phải trên hình t-SNE.
5. **Học embedding:** mô hình ngôn ngữ nơ-ron → Skip-Gram (đoán từ ở gần) → negative sampling (đổi thành bài toán có/không, nhanh hơn nhiều). **GloVe** đếm số lần cặp từ đi cùng nhau. Trong dự án dùng **gensim** hoặc tải embedding huấn luyện sẵn.
6. Các chiều của embedding **không diễn giải được** từng chiều một, nhưng quan hệ hình học vẫn còn nguyên.
7. **Phân loại cảm xúc:** trung bình embedding là mô hình nền nhanh nhưng bỏ qua thứ tự từ; LSTM xử lý được phủ định và tổ hợp từ.

8. **Thiên kiến:** embedding thừa hưởng định kiến của văn bản. Giảm bằng ba bước: tìm hướng thiên kiến, trung hoà từ trung tính, cân bằng cặp từ mang giới tính. Giảm được chứ không xoá hết.

9. **Giới hạn:** embedding tĩnh cho mỗi từ một vector; muốn phân biệt nghĩa theo ngữ cảnh cần mô hình Transformer.

6.3 Mô hình chuỗi sang chuỗi, cơ chế chú ý và xử lý giọng nói

Mục 6.1 cho thấy RNN đọc được một chuỗi, Mục 6.2 cho thấy cách biến từ thành vector. Mục này ghép hai thứ lại để giải bài toán khó hơn: **biến một chuỗi thành một chuỗi khác**. Dịch máy (câu tiếng Pháp → câu tiếng Anh), chép lời giọng nói (âm thanh → chữ), mô tả ảnh (ảnh → câu) đều thuộc họ này. Các mô hình này gọi chung là **sequence-to-sequence** (seq2seq).

Ba câu hỏi xuyên suốt mục này:

- **Sinh thế nào?** Mô hình encoder–decoder, và vì sao dịch máy là mô hình ngôn ngữ *có điều kiện*.
- **Chọn câu nào?** Dịch máy cần câu *tốt nhất*, không phải câu ngẫu nhiên: beam search, phân tích lỗi và điểm BLEU.
- **Nhìn vào đâu?** Cơ chế chú ý (attention) giúp mô hình dịch câu dài, và cách xử lý dữ liệu âm thanh (CTC, phát hiện từ khoá đánh thức).

Code dùng TensorFlow/Keras, và thư viện Hugging Face Transformers cho các việc đã có mô hình huấn luyện sẵn (dịch, chép lời). Hai ví dụ thực hành của mục này là chuẩn hoá ngày tháng bằng mô hình chú ý và phát hiện từ khoá đánh thức.

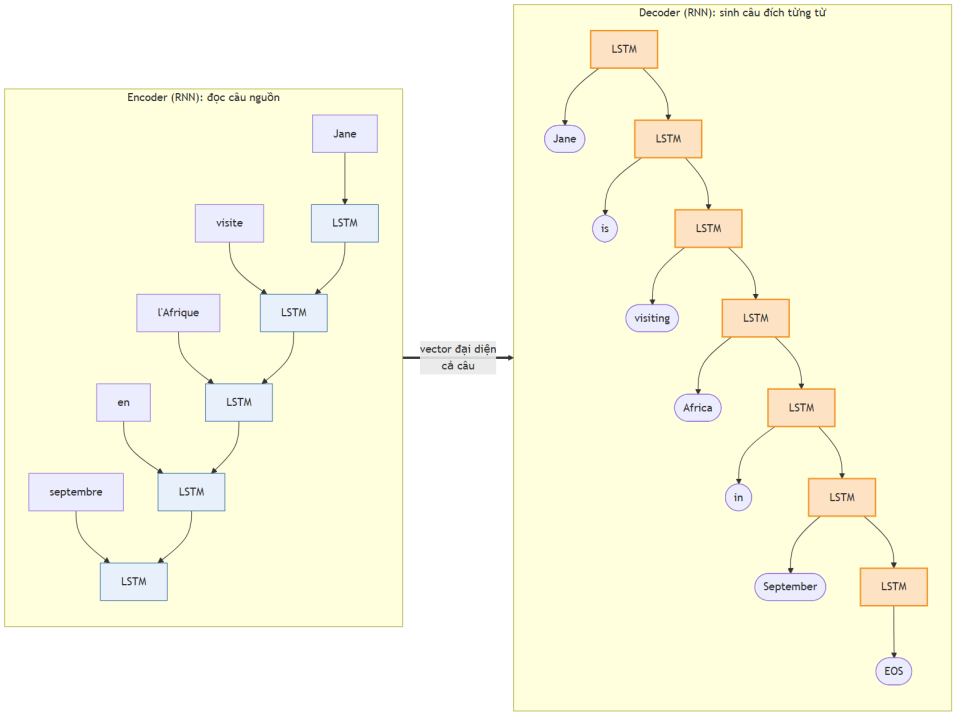
6.3.1 Mô hình encoder–decoder

6.3.1.1 Hai mạng ghép lại

Giả sử cần dịch câu tiếng Pháp "Jane visite l'Afrique en septembre" (5 từ) thành "Jane is visiting Africa in September" (6 từ). Hai chuỗi *khác độ dài*, nên không thể gán mỗi từ vào một từ như bài nhận dạng thực thể. Cách giải gồm hai phần (Hình 6.30):

1. **Encoder** – một RNN (GRU hoặc LSTM) đọc lần lượt từng từ của câu nguồn. Đọc xong, nó cho ra một vector biểu diễn *cả câu*.

2. **Decoder** – một RNN khác nhận vector đó làm điểm xuất phát, rồi sinh bản dịch từng từ một. Mỗi từ vừa sinh được đưa làm đầu vào cho bước sau, giống hệt cách sinh văn bản bằng mô hình ngôn ngữ (mục 6.1.9). Decoder dừng khi sinh ra token cuối câu `EOS`.

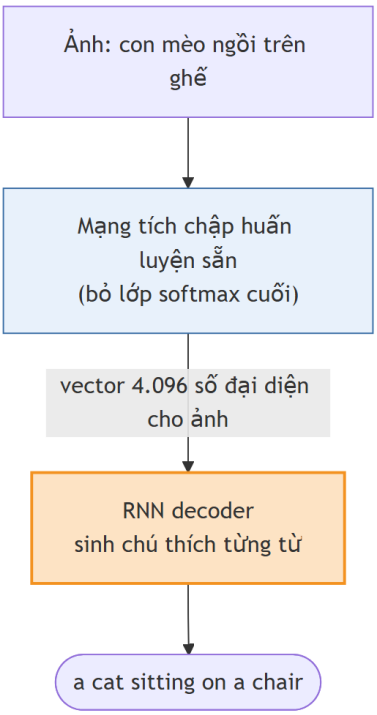


Hình 6.30: Encoder đọc câu Pháp, decoder sinh câu Anh

Điều đáng ngạc nhiên là mô hình đơn giản này hoạt động khá tốt. Chỉ cần đủ cặp câu Pháp–Anh để huấn luyện, nó học được cả việc mã hoá ý nghĩa lẫn việc viết ra câu mới. Ý tưởng đến từ hai bài báo, một của Sutskever, Vinyals và Le, một của Cho cùng các cộng sự.

6.3.1.2 Cùng kiến trúc cho mô tả ảnh

Thay encoder bằng một mạng nhận dạng ảnh là được bài toán mô tả ảnh (image captioning). Lấy một mạng tích chập huấn luyện sẵn (ví dụ AlexNet), bỏ lớp softmax cuối thì còn một vector 4.096 số đại diện cho ảnh. Đưa vector đó cho RNN decoder, nó sinh chú thích từng từ: “a cat sitting on a chair”. Cách này hiệu quả nhất khi câu chú thích không quá dài. Nhiều nhóm nghiên cứu (Mao cùng cộng sự, Karpathy và Fei-Fei Li) đề xuất mô hình tương tự gần như cùng lúc.



Hình 6.31: Mô tả ảnh: encoder là mạng tích chập, decoder là RNN

6.3.1.3 Dựng và chạy bằng Keras

Khung huấn luyện (hai đầu vào: câu nguồn và câu đích lệch phải một bước, bắt đầu bằng token `<start>`) đã có ở mục 6.1.3. Khác biệt lớn nhất so với huấn luyện nằm ở lúc *chạy thật*: ta không có sẵn câu đích, nên phải tự viết vòng lặp sinh từng từ, mỗi từ vừa chọn được đưa lại làm đầu vào. Dưới đây là cách đơn giản nhất, chọn từ có xác suất cao nhất ở mỗi bước (greedy). Với mô hình nhận chuỗi đích độ dài tuỳ ý (`None`), cách này chạy được ngay:

```
import numpy as np

def greedy_translate(model, src_ids, start_id, end_id, max_len=40):
    """Sinh bản dịch bằng cách luôn chọn từ có xác suất cao nhất ở mỗi bước."""
    tgt = [start_id]
    for _ in range(max_len):
        probs = model.predict([np.array([src_ids]), np.array([tgt])],
        ↪  verbose=0)[0, -1]
        next_id = int(np.argmax(probs))           # chọn từ có xác suất cao nhất
        if next_id == end_id:                     # gặp EOS thì dừng
            break
        tgt.append(next_id)
    return tgt[1:]                                # bỏ token <start>
```

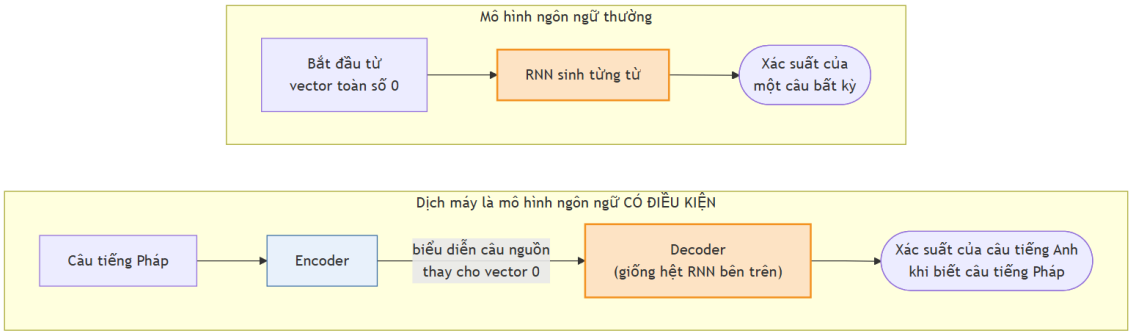
Mục 6.3.3 giải thích vì sao greedy chưa phải cách tốt nhất.

6.3.2 Dịch máy là mô hình ngôn ngữ có điều kiện

Decoder của mô hình dịch máy gần như giống hệt mô hình ngôn ngữ ở Mục 6.1. Khác biệt duy nhất nằm ở chỗ xuất phát:

- **Mô hình ngôn ngữ thường** bắt đầu từ vector toàn số 0, và đo khả năng xuất hiện của *một câu bất kỳ*.

- **Dịch máy** bắt đầu từ biểu diễn câu nguồn do encoder tạo ra, và đo khả năng của *câu tiếng Anh khi biết trước câu tiếng Pháp*. Vì có điều kiện (câu nguồn) đi kèm nên gọi là mô hình ngôn ngữ **có điều kiện**.



Hình 6.32: Dịch máy giống mô hình ngôn ngữ, chỉ khác điểm xuất phát của decoder

6.3.2.1 Đừng bốc ngẫu nhiên khi dịch

Ở Mục 6.1, khi sinh tên khủng long hay nhạc, ta bốc ngẫu nhiên theo xác suất để kết quả đa dạng. Dịch máy thì ngược lại. Nếu bốc ngẫu nhiên, lúc thì ra bản dịch tốt, lúc thì gượng, lúc thì sai nghĩa:

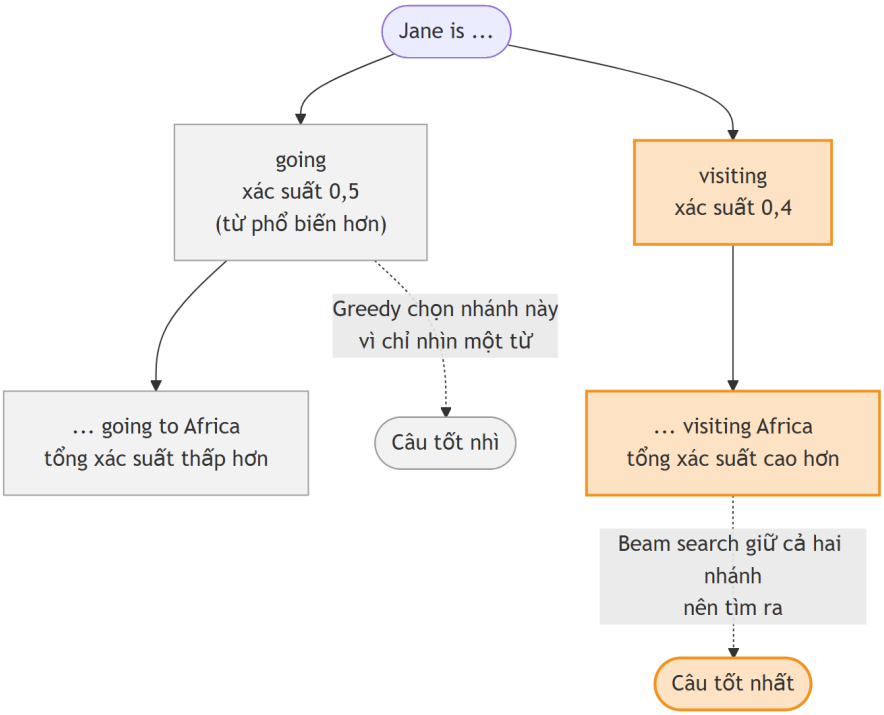
Bản dịch có thể bốc được	**Chất lượng**
Jane is visiting Africa in September	tốt
Jane is going to be visiting Africa in September	hơi gượng
In September, Jane will visit Africa	chấp nhận được
Her African friend welcomed Jane in September	sai nghĩa

Bảng 6.23: Bốc ngẫu nhiên từ phân phối của mô hình cho kết quả không ổn định

Thứ ta muốn là **câu tiếng Anh có xác suất cao nhất khi biết câu tiếng Pháp**. Nhưng nếu từ điển 10.000 từ và câu dài tới 10 từ thì có khoảng 10.000 mũ 10 câu có thể có; không thể xét hết. Vì vậy cần một thuật toán tìm kiếm gần đúng.

6.3.2.2 Vì sao greedy search chưa đủ

Greedy search chọn từ đầu có xác suất cao nhất, rồi từ thứ hai, từ thứ ba, và cứ thế. Cách này không tối ưu vì **chọn từng từ tốt nhất chưa chắc cho cả câu tốt nhất**. Ví dụ, sau "Jane is" từ "going" phổ biến hơn "visiting", nên greedy chọn "going" và đi vào nhánh cho bản dịch dài dòng, tổng xác suất thấp hơn bản "Jane is visiting Africa in September" ngắn gọn (Hình 6.33).



Hình 6.33: Greedy chọn nhánh có từ phổ biến hơn, beam search giữ cả hai nhánh

6.3.3 Beam search

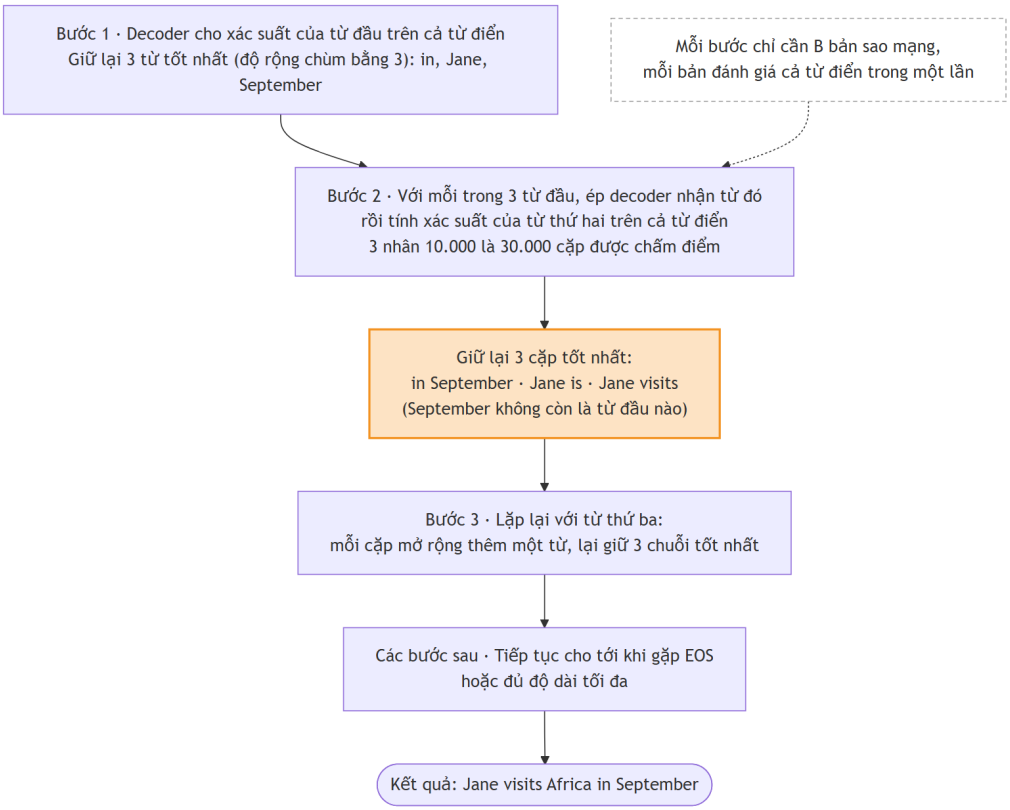
Beam search là thuật toán tìm kiếm gần đúng phổ biến nhất cho dịch máy và nhận dạng giọng nói. Thay vì chỉ giữ một ứng viên như greedy, nó giữ **một số ứng viên tốt nhất** ở mỗi bước. Số ứng viên đó gọi là **beam width** (độ rộng chùm tia).

6.3.3.1 Cách hoạt động với độ rộng chùm bằng 3

1. **Bước 1.** Decoder cho xác suất của từ đầu tiên trên toàn từ điển. Giữ lại 3 từ có xác suất cao nhất, ví dụ “in”, “Jane”, “September”.

2. **Bước 2.** Với mỗi trong 3 từ đầu, ép decoder nhận từ đó làm từ đầu tiên rồi tính xác suất của từ thứ hai trên toàn từ điển. Xác suất của cặp từ bằng xác suất từ đầu nhân xác suất từ thứ hai khi biết từ đầu. Có 3 nhân 10.000 bằng 30.000 cặp được chấm điểm, rồi chỉ giữ 3 cặp tốt nhất, ví dụ “in September”, “Jane is”, “Jane visits”. Lưu ý “September” đã bị loại khỏi vị trí từ đầu vì không còn cặp nào bắt đầu bằng nó trong nhóm tốt nhất.

3. **Các bước sau.** Lặp lại: mỗi chuỗi giữ lại được mở rộng thêm một từ, chấm điểm, giữ 3 chuỗi tốt nhất. Dừng khi gặp `EOS` hoặc đủ độ dài tối đa, hy vọng ra “Jane visits Africa in September”.



Hình 6.34: Beam search với độ rộng chùm bằng 3

Về chi phí: mỗi bước chỉ cần B bản sao mạng, mỗi bản đánh giá cả từ điển trong một lần. Không cần 30.000 bản sao. Nếu B bằng 1 thì beam search chính là greedy; xét nhiều ứng viên cùng lúc thường cho câu tốt hơn nhiều.

6.3.3.2 Hai tinh chỉnh quan trọng

Cộng logarit thay vì nhân xác suất. Xác suất của cả câu là tích của nhiều xác suất nhỏ hơn 1, nên dễ ra số cực nhỏ vượt khả năng lưu của số thực trong máy (tràn số dưới, underflow). Cách sửa: cộng các logarit của xác suất thay vì nhân. Vì hàm logarit luôn tăng, câu tốt nhất vẫn là câu cũ nhưng tính toán ổn định hơn nhiều.

Chuẩn hoá theo độ dài. Câu càng dài thì tích càng nhỏ (hoặc tổng logarit càng âm), nên mục tiêu ban đầu vô tình *thiên vị bản dịch quá ngắn*. Cách sửa: chia điểm cho số từ của câu, tức là lấy trung bình logarit xác suất mỗi từ. Thực tế hay dùng cách "mềm" hơn: chia cho số từ lũy thừa một số **alpha**, ví dụ 0,7. Alpha bằng 1 là chuẩn hoá hoàn toàn, alpha bằng 0 là không chuẩn hoá. Cách này không có lý giải lý thuyết chặt chẽ nhưng được nhiều nhóm dùng vì hiệu quả; alpha là siêu tham số có thể thử.

Sau khi chạy tới độ dài tối đa (ví dụ 30 bước), chấm điểm tất cả các câu đã gặp bằng mục tiêu đã chuẩn hoá và chọn câu điểm cao nhất làm bản dịch cuối.

6.3.3.3 Chọn beam width

Tiêu chí	Beam width nhỏ	Beam width lớn
Chất lượng	kém hơn, vì giữ ít khả năng	tốt hơn, vì xét nhiều khả năng
Tốc độ, bộ nhớ	nhanh, nhẹ	chậm, tốn bộ nhớ
Mốc tham khảo	3 là nhỏ; hệ thống sản xuất thường quanh 10	100 đã là rất lớn; nghiên cứu có thể dùng 1.000 đến 3.000

Bảng 6.24: Đánh đổi khi chọn beam width

Lợi ích tăng mạnh khi đi từ 1 (greedy) lên 3 rồi 10, và giảm dần khi lên hàng nghìn. Hãy thử nhiều giá trị cho ứng dụng của bạn. Khác với tìm kiếm theo chiều rộng hay chiều sâu trong khoa học máy tính (là các thuật toán tìm chính xác), beam search chạy nhanh hơn nhiều nhưng *không đảm bảo* tìm ra kết quả tốt nhất tuyệt đối.

6.3.3.4 Beam search viết bằng Python để hiểu cơ chế

Hàm dưới đây nhận một hàm `next_log_probs(prefix)` trả về logarit xác suất của token kế tiếp (có thể là bất kỳ mô hình nào), giữ số chuỗi tốt nhất bằng độ rộng chùm ở mỗi bước và chọn câu cuối bằng điểm đã chuẩn hoá theo độ dài. Để đơn giản, chuỗi đã gặp `EOS` được đưa sang danh sách riêng và không mở rộng thêm.

```
import numpy as np

def beam_search(next_log_probs, start_id, end_id, beam_width=3, max_len=30,
↪  alpha=0.7):
    beams = [([start_id], 0.0)]                      # (chuỗi, tổng logarit xác
    ↪  suất)
    finished = []
    for _ in range(max_len):
        candidates = []
        for seq, score in beams:
            logp = next_log_probs(seq)
            for tok in np.argsort(logp)[-beam_width:]:       # chỉ cần top B của
            ↪  mỗi chuỗi
                candidates.append((seq + [int(tok)], score + float(logp[tok])))
        candidates.sort(key=lambda c: c[1], reverse=True)
        beams = []
        for seq, score in candidates[:beam_width]:           # giữ beam_width
        ↪  chuỗi tốt nhất
            (finished if seq[-1] == end_id else beams).append((seq, score))
        if not beams:
            break
    finished += beams
    # chuẩn hoá theo độ dài (bỏ token bắt đầu) rồi chọn câu tốt nhất
    return max(finished, key=lambda c: c[1] / (len(c[0]) - 1) ** alpha)
```

Thử với một mô hình đồ chơi tái hiện đúng ví dụ "going" và "visiting". Bảng xác suất được viết tay: sau "Jane is", "going" có xác suất 0,5, hơi cao hơn "visiting" là 0,4, nhưng nhánh "visiting" đi tiếp rất trôi chảy.

```
vocab = ["<s>", "Jane", "is", "going", "visiting", "to", "Africa", "</s>"]
rules = {                                            # prefix -> {token: xác
↪  suất}
```

```
    (0,): {1: 0.9},
    (0, 1): {2: 0.9},
    (0, 1, 2): {3: 0.5, 4: 0.4},                    # going hơi cao hơn visiting
    (0, 1, 2, 3): {5: 0.9},
    (0, 1, 2, 3, 5): {6: 0.4, 4: 0.2},
    (0, 1, 2, 3, 5, 6): {7: 0.9},
    (0, 1, 2, 4): {6: 0.95},
    (0, 1, 2, 4, 6): {7: 0.95},
}
def next_log_probs(prefix):
    p = np.full(len(vocab), 1e-4)                   # phần xác suất rất nhỏ cho
    ↪ mọi token còn lại
    for tok, v in rules.get(tuple(prefix), {}).items():
        p[tok] = v
    return np.log(p / p.sum())

for B in (1, 2, 3):
    seq, score = beam_search(next_log_probs, 0, 7, beam_width=B, alpha=0.0)
    print("B =", B, " ".join(vocab[i] for i in seq[1:]), " xác suất = %.3f" %
    ↪ np.exp(score))
```

Kết quả khi chạy: với độ rộng chùm 1 (greedy) ra "Jane is going to Africa" với xác suất 0,369; với độ rộng chùm 2 và 3 ra "Jane is visiting Africa" với xác suất 0,443. Greedy bị lừa bởi từ phổ biến hơn, còn beam search chỉ cần giữ thêm một ứng viên là tìm ra câu tốt hơn.

6.3.3.5 Dùng beam search qua thư viện

Trong dự án, ít khi tự viết beam search. Các thư viện sinh văn bản đã có sẵn với đầy đủ tham số. Ví dụ với Hugging Face Transformers và một mô hình dịch Pháp–Anh huấn luyện sẵn (cần cài `transformers`, `sentencepiece` và một backend như PyTorch):

```
from transformers import MarianMTModel, MarianTokenizer

name = "Helsinki-NLP/opus-mt-fr-en"                 # mô hình dịch Pháp
↪ -> Anh huấn luyện sẵn
tok = MarianTokenizer.from_pretrained(name)
model = MarianMTModel.from_pretrained(name)

batch = tok(["Jane visite l'Afrique en septembre"], return_tensors="pt")

greedy = model.generate(**batch, num_beams=1)       # num_beams=1 chính
↪ là greedy
beam = model.generate(**batch, num_beams=5, length_penalty=0.7,
                      num_return_sequences=3)       # độ rộng chùm 5, hệ
                      ↪ số phạt độ dài 0,7, trả về 3 câu tốt nhất
print(tok.batch_decode(greedy, skip_special_tokens=True))
print(tok.batch_decode(beam, skip_special_tokens=True))
```

Khái niệm trong mục này	Tham số của `generate`
Beam width B	`num_beams` (1 là greedy)
Alpha chuẩn hoá độ dài	`length_penalty`
Lấy nhiều bản dịch tốt nhất	`num_return_sequences` (không lớn hơn `num_beams`)
Độ dài tối đa	`max_new_tokens`

Bảng 6.25: Đối chiếu khái niệm beam search với tham số `generate`

6.3.4 Phân tích lỗi: beam search hay mô hình?

Beam search chỉ giữ một số ứng viên nên không phải lúc nào cũng ra câu tốt nhất theo mô hình. Khi bản dịch sai, câu hỏi là: lỗi nằm ở **beam search** hay ở **mô hình RNN** (encoder và decoder)? Câu trả lời quyết định nên dồn công sức vào đâu. Cám dỗ lớn là cứ tăng beam width vì "chẳng hại gì", nhưng cũng như thu thêm dữ liệu, tự nó chưa chắc đưa bạn tới mức hiệu năng mong muốn.

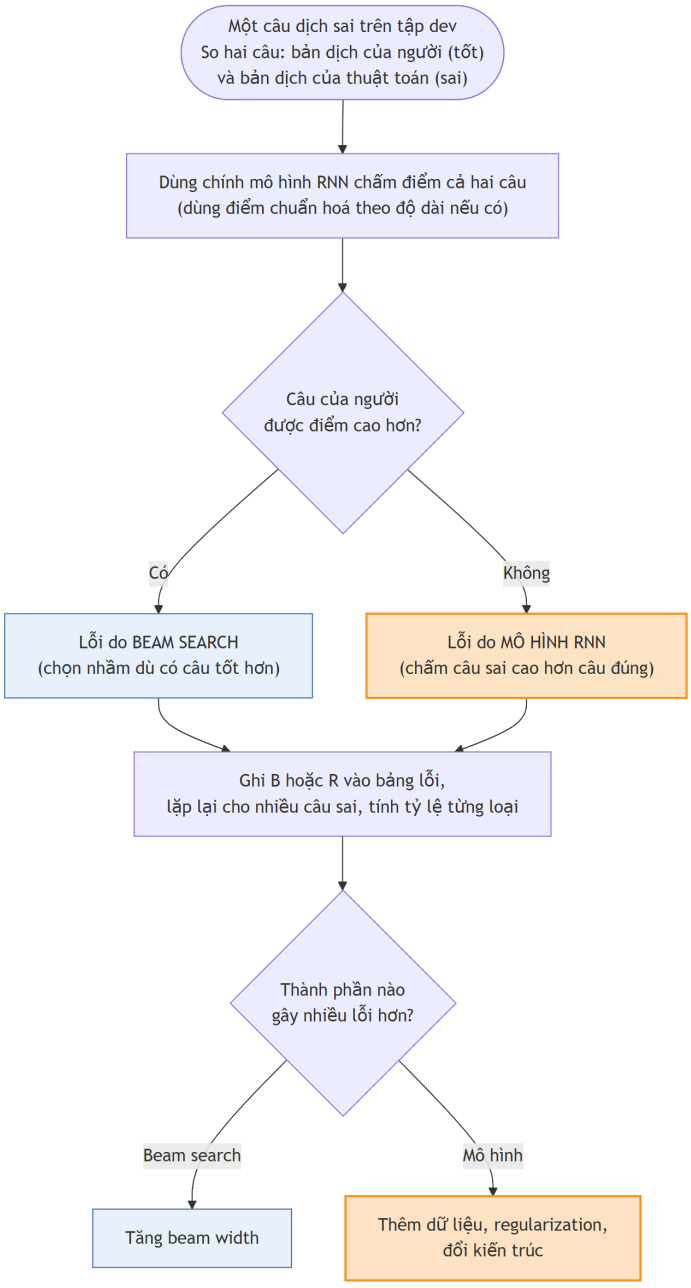
6.3.4.1 Cách phân tích

Với một câu sai trên tập dev, giữ hai bản dịch: bản của con người (tốt, gọi là *tham chiếu*) và bản của thuật toán (sai). Rồi dùng **chính mô hình RNN** chấm điểm cả hai và so sánh:

Trường hợp	Ý nghĩa	Kết luận
Mô hình chấm câu của người *cao hơn* câu của thuật toán	beam search được giao nhiệm vụ tìm câu điểm cao nhất nhưng lại chọn câu điểm thấp hơn	Lỗi do **beam search**
Mô hình chấm câu của người *thấp hơn hoặc bằng* câu của thuật toán	dù câu của người tốt hơn, mô hình lại cho nó điểm thấp hơn câu kém	Lỗi do **mô hình RNN**

Bảng 6.26: Quy tắc quy lỗi cho beam search hay mô hình

Quy trình: duyệt các lỗi trên tập dev, ghi B (beam search) hoặc R (mô hình) cho từng lỗi, rồi tính tỷ lệ lỗi thuộc về mỗi thành phần (Hình 6.35). Chỉ khi beam search gây nhiều lỗi mới đáng tăng beam width. Nếu mô hình gây nhiều lỗi, hãy phân tích sâu hơn: thêm regularization, thêm dữ liệu, thử kiến trúc khác.



Hình 6.35: Quy trình phân tích lỗi cho beam search

Lưu ý. Nếu đang dùng chuẩn hoá theo độ dài, hãy so sánh *điểm đã chuẩn hoá* thay vì xác suất thuần. Cách phân tích này dùng được cho mọi hệ thống có một thuật toán tối ưu gần đúng chạy trên hàm mục tiêu do mô hình học được tạo ra.

6.3.4.2 Làm bằng code

Với mô hình Hugging Face ở mục trước, điểm của một cặp (câu nguồn, câu đích) là tổng logarit xác suất của các token đích. Hàm dưới đây tính điểm đó, sau đó so sánh hai câu để quy lỗi:

```
import torch

def sequence_logprob(model, tok, src, tgt, alpha=0.7):
```

```
    """Điểm của câu đích tgt khi biết câu nguồn src: tổng logarit xác suất, đã
    ↪  chuẩn hoá theo độ dài."""
    batch = tok([src], text_target=[tgt], return_tensors="pt")
    with torch.no_grad():
        logits = model(**batch).logits                      # (1, số token đích,
        ↪  kích thước từ điển)
    logp = torch.log_softmax(logits, dim=-1)
    token_logp = logp.gather(-1, batch["labels"].unsqueeze(-1)).squeeze(-1)
    n = token_logp.shape[1]
    return token_logp.sum().item() / (n ** alpha)

def blame(model, tok, src, human_translation, model_translation):
    score_ref = sequence_logprob(model, tok, src, human_translation)
    score_hyp = sequence_logprob(model, tok, src, model_translation)
    return "beam search" if score_ref > score_hyp else "mô hình"

print(blame(model, tok, "Jane visite l'Afrique en septembre",
            "Jane visits Africa in September", "Jane visited Africa last
            ↪  September"))
```

6.3.5 Đánh giá bản dịch bằng điểm BLEU

Một câu tiếng Pháp có nhiều bản dịch tiếng Anh tốt ngang nhau: “The cat is on the mat” và “There is a cat on the mat” đều ổn. Vì vậy không thể chỉ đo độ chính xác như bài phân loại ảnh, nơi chỉ có một đáp án đúng. Thước đo thông dụng là **BLEU** (Bilingual Evaluation Understudy). “Understudy” nghĩa là diễn viên dự bị: BLEU thay thế việc phải nhờ con người chấm từng đầu ra của hệ thống dịch.

Ý tưởng: bản dịch của máy càng gần *một trong các* bản tham chiếu của người thì điểm càng cao.

6.3.5.1 Độ chính xác có cắt

Độ chính xác thuần (có bao nhiêu phần trăm từ của máy xuất hiện trong tham chiếu) dễ bị lừa. Máy dịch ra “the the the the the the the” thì cả 7 từ đều có trong tham chiếu, độ chính xác là 7 trên 7, nghe rất tốt dù bản dịch rất tệ.

Cách sửa là **cắt** (clip): mỗi từ chỉ được tính điểm tối đa bằng số lần nó xuất hiện nhiều nhất trong *một* câu tham chiếu. “the” xuất hiện tối đa 2 lần trong tham chiếu thứ nhất, nên điểm là 2 trên 7.

6.3.5.2 Mở rộng sang cặp từ, bộ ba từ

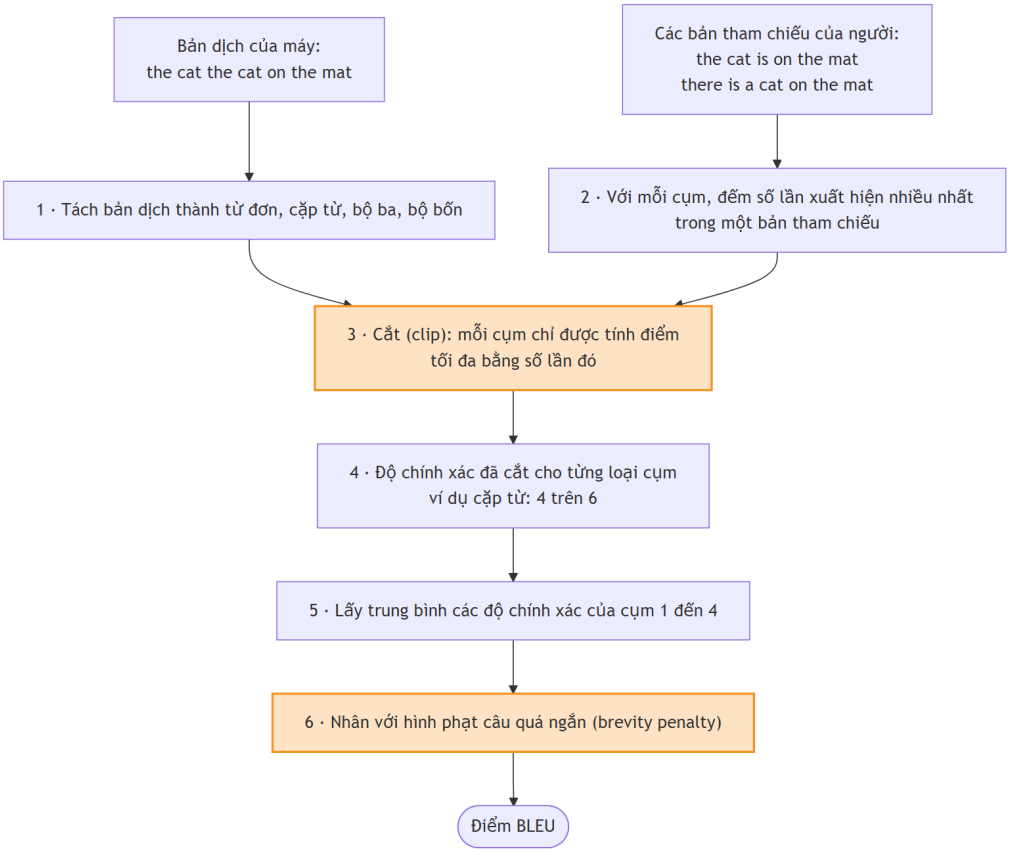
Chỉ xét từ đơn thì chưa phản ánh thứ tự. BLEU xét cả cặp từ (bigram), bộ ba (trigram) và bộ bốn. Ví dụ bản dịch “the cat the cat on the mat” có 6 cặp từ; sau khi cắt, 4 cặp nằm trong tham chiếu, nên độ chính xác cặp từ là 4 trên 6.

Cặp từ	Số lần trong bản dịch máy	Tối đa trong một tham chiếu	Điểm sau khi cắt
the cat	2	1	1
cat the	1	0	0
cat on	1	1	1
on the	1	1	1
the mat	1	1	1
Tổng	**6**		**4**

Bảng 6.27: Độ chính xác cặp từ của “the cat the cat on the mat”: 4 trên 6

6.3.5.3 Ghép thành một điểm

Theo quy ước, tính độ chính xác có cắt cho cụm 1, 2, 3, 4 từ, lấy trung bình, rồi nhân với **hình phạt câu quá ngắn** (brevity penalty). Hình phạt này cần vì bản dịch rất ngắn dễ có độ chính xác cao (hầu hết từ ra đều có trong tham chiếu); BLEU phạt các hệ thống dịch ngắn hơn câu tham chiếu của người.



Hình 6.36: Các bước tính điểm BLEU

6.3.5.4 Tính bằng code

Hàm tính độ chính xác có cắt chỉ vài dòng, khớp đúng hai ví dụ trên:

```
from collections import Counter

def ngrams(words, n):
    return [tuple(words[i:i + n]) for i in range(len(words) - n + 1)]

def modified_precision(hyp, refs, n):
    """Trả về (số cụm được tính sau khi cắt, tổng số cụm của bản dịch máy)."""
    hyp_counts = Counter(ngrams(hyp, n))
    max_ref = Counter()                                    # số lần nhiều nhất
    ↪  của mỗi cụm trong MỘT tham chiếu
    for ref in refs:
        for g, c in Counter(ngrams(ref, n)).items():
            max_ref[g] = max(max_ref[g], c)
    clipped = sum(min(c, max_ref[g]) for g, c in hyp_counts.items())
    return clipped, sum(hyp_counts.values())

refs = ["the cat is on the mat".split(), "there is a cat on the mat".split()]
```

```
print(modified_precision("the the the the the the the".split(), refs, 1))   # (2,
↪  7)
print(modified_precision("the cat the cat on the mat".split(), refs, 2))    # (4,
↪  6)
```

Trong dự án, ít ai tự cài BLEU đầy đủ. Dùng thư viện `sacrebleu`, đảm bảo cách tách từ và cách tính giống mọi người để so sánh được với nhau:

```
import sacrebleu

hypotheses = ["the cat the cat on the mat"]                    # mỗi bản dịch của
↪  máy là một chuỗi
references = [["the cat is on the mat"],                       # tham chiếu thứ
↪  nhất cho từng câu
              ["there is a cat on the mat"]]                   # tham chiếu thứ
               ↪  hai cho từng câu
print(sacrebleu.corpus_bleu(hypotheses, references).score)     # điểm BLEU trên
↪  thang 0 đến 100
```

BLEU dùng được cho mọi hệ thống sinh văn bản có nhiều đáp án tốt, như dịch máy và mô tả ảnh. Nó **không** dùng cho nhận dạng giọng nói, vì ở đó thường chỉ có một bản chép lời đúng và người ta dùng các thước đo khác để xem có khớp từng từ không (ví dụ tỷ lệ lỗi từ). Giá trị lớn nhất của BLEU là cho một con số đánh giá duy nhất, tuy không hoàn hảo nhưng đủ tốt để thử hai ý tưởng và giữ cái điểm cao hơn, nhờ đó đẩy nhanh tiến bộ của cả ngành dịch máy.

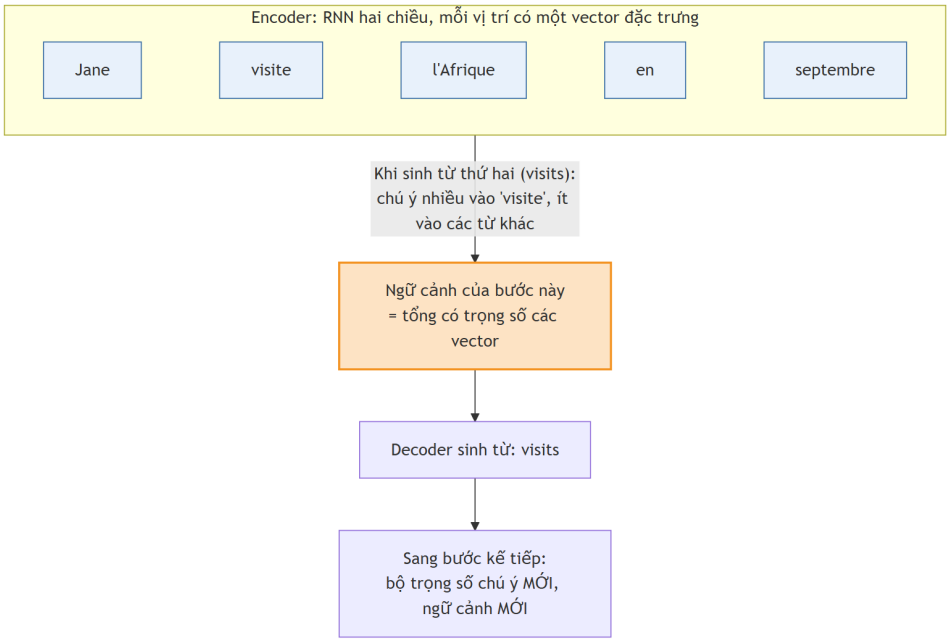
6.3.6 Cơ chế chú ý (attention)

6.3.6.1 Vấn đề: nhớ cả câu dài trong một vector

Mô hình encoder–decoder thuần buộc encoder đọc cả câu rồi nén vào *một* vector duy nhất, và decoder phải dịch từ vector đó. Với câu ngắn thì tốt, nhưng khi câu dài (khoảng trên 30 đến 40 từ) điểm BLEU tụt mạnh vì mạng khó nhớ nổi cả câu. Người dịch không làm vậy: họ đọc một đoạn, dịch một đoạn, nhìn tiếp phần sau, dịch thêm vài từ, cứ thế từng phần. **Mô hình chú ý** bắt chước đúng điều đó, nên không còn chỗ tụt điểm lớn khi câu dài. Cái tụt ấy thực chất đo khả năng ghi nhớ câu dài của mạng, thứ ta không thật sự cần mạng phải làm. Ý tưởng do Bahdanau, Cho và Bengio đề xuất cho dịch máy, nhưng đã lan sang rất nhiều lĩnh vực khác và là một trong những ý tưởng có ảnh hưởng nhất của học sâu.

6.3.6.2 Ý tưởng

Encoder lúc này là một RNN hai chiều, cho ra một vector đặc trưng phong phú *cho mỗi vị trí* của câu nguồn. Decoder sinh từng từ, và **ở mỗi bước nó tự quyết định nên chú ý bao nhiêu vào từng vị trí nguồn**. Khi sinh từ "Jane", chú ý tập trung vào từ đầu tiên và vài từ lân cận, không cần nhìn tới cuối câu. Khi sinh "visits", chú ý chuyển sang "visite". Mỗi bước sinh từ có bộ trọng số chú ý riêng và một ngữ cảnh riêng.



Hình 6.37: Mỗi bước sinh từ nhìn vào các vị trí nguồn với mức chú ý khác nhau

6.3.6.3 Chi tiết một bước chú ý

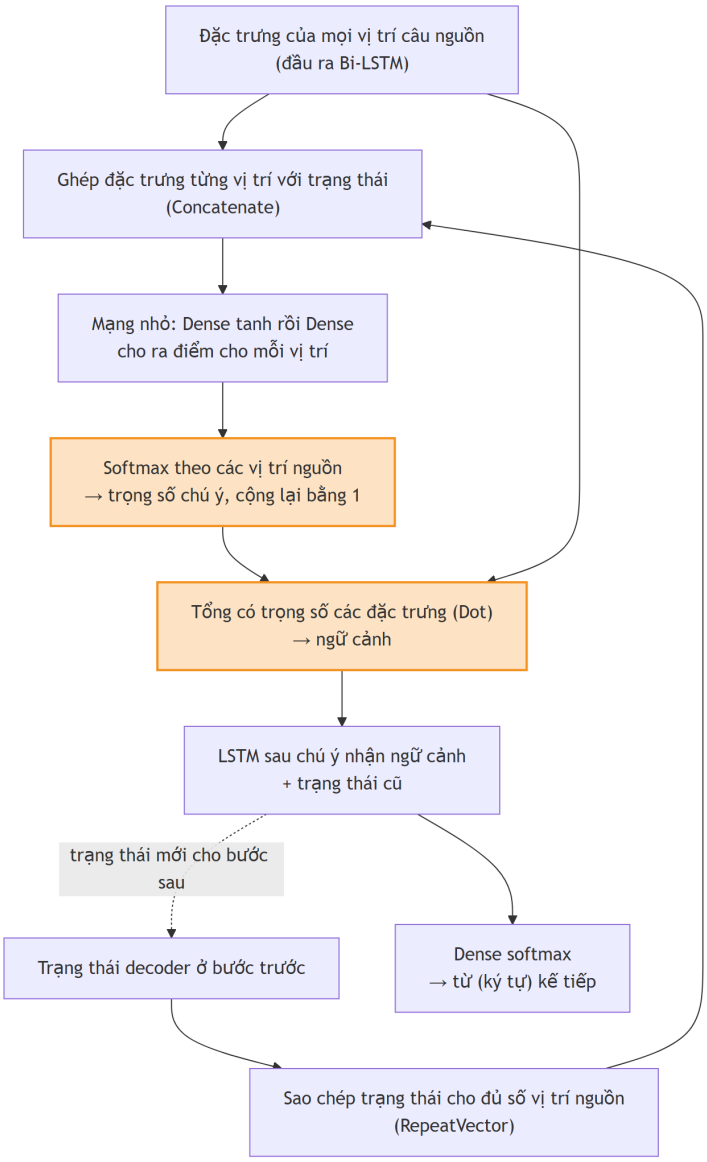
1. **Đặc trưng nguồn.** Bi-LSTM (thực tế thường là LSTM hai chiều, đôi khi GRU) cho một vector ở mỗi vị trí, ghép trạng thái của chiều xuôi và chiều ngược.

2. **Điểm chú ý.** Một *mạng nơ-ron rất nhỏ* (thường chỉ một lớp ẩn, vì phải tính rất nhiều lần) nhận hai đầu vào: trạng thái ẩn của decoder ở bước *trước* và vector đặc trưng của một vị trí nguồn, rồi cho ra một điểm. Trạng thái hiện tại chưa dùng được vì ngữ cảnh còn đang được tính. Ta không biết hàm đó là gì nên cứ để mạng nhỏ học bằng lan truyền ngược và gradient descent.

3. **Trọng số.** Đưa các điểm của mọi vị trí qua softmax để thành **trọng số chú ý**: không âm và cộng lại bằng 1.

4. **Ngữ cảnh.** Lấy tổng có trọng số các vector đặc trưng của mọi vị trí nguồn. Đây là *ngữ cảnh* của bước này.

5. **Sinh từ.** Ngữ cảnh được đưa vào ô RNN của decoder để sinh từ kế tiếp. Rồi lặp lại cho bước sau với trạng thái mới.



Hình 6.38: Một bước chú ý trong mô hình chuẩn hoá ngày tháng

Nhược điểm. Chi phí tính toán tăng theo bình phương: nếu câu nguồn có bao nhiêu từ nhân với số từ của câu đích thì có bấy nhiêu trọng số chú ý cần tính. Với dịch máy, câu thường không quá dài nên vẫn chấp nhận được. Vẫn có nghiên cứu nhằm giảm chi phí này.

Ứng dụng khác. Mô tả ảnh cũng dùng chú ý: mô hình nhìn vào từng phần của bức ảnh khi viết chú thích (bài báo của Kelvin Xu và các cộng sự). Chuẩn hoá ngày tháng ở ví dụ thực hành của mục này là một ứng dụng nhỏ hơn.

Trực quan hoá. Khi vẽ các trọng số chú ý thành màu, các từ đầu vào và đầu ra tương ứng thường có trọng số cao, nghĩa là khi sinh một từ, mô hình thật sự nhìn vào đúng từ nguồn. Toàn bộ việc học "nên chú ý vào đâu" này do lan truyền ngược tự động làm.

6.3.6.4 Chú ý trong Keras

Keras có sẵn hai lớp chú ý: `layers.AdditiveAttention` (dạng "mạng nhỏ" mô tả ở trên, còn gọi là chú ý kiểu Bahdanau) và `layers.Attention` (dạng tích vô hướng, kiểu Luong). Dưới đây là một mô

hình dịch có chú ý bằng Functional API. Điểm khác nhỏ so với bài báo gốc: chú ý ở đây tính từ *đầu ra của decoder*, cách sắp xếp phổ biến trong Keras; ý tưởng vẫn y như nhau.

```
UNITS, SRC_VOCAB, TGT_VOCAB = 128, 8000, 8000

# Encoder: Bi-LSTM trả về vector đặc trưng cho MỌI vị trí nguồn + trạng thái cuối
↪  của hai chiều
enc_in = keras.Input(shape=(None,), name="src")
x = layers.Embedding(SRC_VOCAB, 128, mask_zero=True)(enc_in)
enc_seq, fh, fc, bh, bc = layers.Bidirectional(
    layers.LSTM(UNITS, return_sequences=True, return_state=True))(x)
state_h = layers.Concatenate()([fh, bh])                       # nối trạng thái
↪  xuôi và ngược
state_c = layers.Concatenate()([fc, bc])

# Decoder: nhận câu đích lệch phải một bước, khởi tạo từ trạng thái của encoder
dec_in = keras.Input(shape=(None,), name="tgt_in")
y = layers.Embedding(TGT_VOCAB, 128, mask_zero=True)(dec_in)
dec_seq = layers.LSTM(2 * UNITS, return_sequences=True)(y,
↪  initial_state=[state_h, state_c])

# Chú ý: mỗi bước của decoder (query) nhìn vào mọi vị trí của encoder (value)
context = layers.AdditiveAttention()([dec_seq, enc_seq])
merged = layers.Concatenate()([dec_seq, context])              # ghép đầu ra
↪  decoder với ngữ cảnh
out = layers.Dense(TGT_VOCAB, activation="softmax")(merged)

nmt = keras.Model([enc_in, dec_in], out)
nmt.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
```

Mô hình này nhận câu đích độ dài tuỳ ý nên dùng thẳng được hàm `greedy_translate` và `beam_search` ở trên (bọc `model.predict` thành `next_log_probs`).

6.3.7 Nhận dạng giọng nói

Bài toán: cho một đoạn âm thanh, tự động tìm bản chép lời bằng chữ, ví dụ "the quick brown fox". Bài toán này đã trở nên rất chính xác nhờ các mô hình chuỗi sang chuỗi.

6.3.7.1 Âm thanh là gì

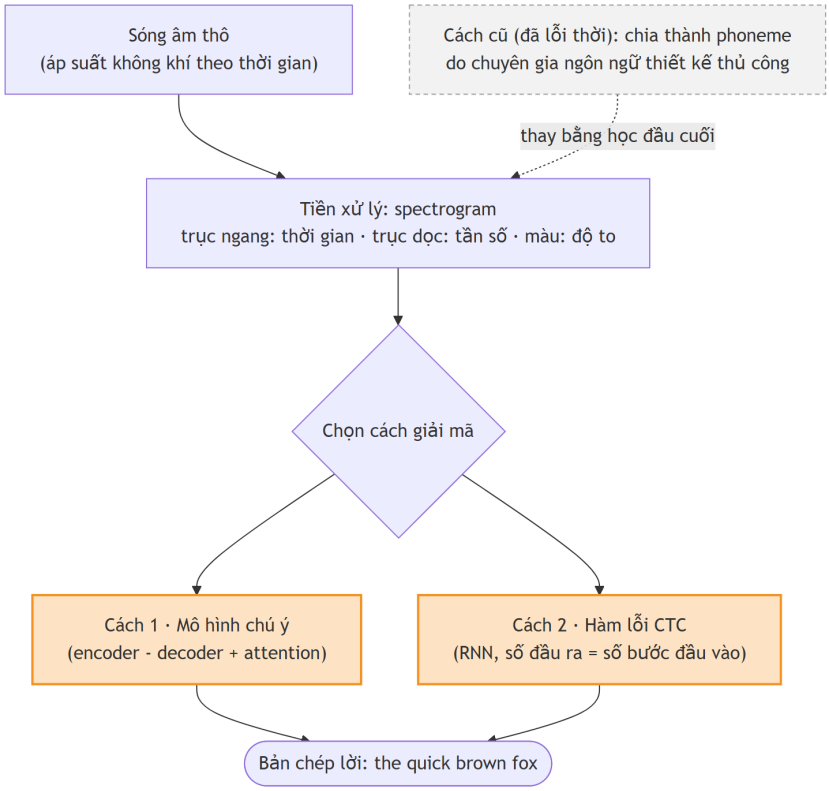
Micro đo những thay đổi rất nhỏ của áp suất không khí theo thời gian; tai người cảm nhận giọng nói cũng bằng cách đó. Tai người không xử lý dạng sóng thô mà đo cường độ của các tần số khác nhau, nên bước tiền xử lý phổ biến là biến đoạn âm thanh thành **spectrogram**: trục ngang là thời gian, trục dọc là tần số, màu cho biết năng lượng (độ to) của âm thanh ở từng tần số tại từng thời điểm. Cách làm này gần với việc tai người xử lý. Người ta cũng gọi là filter bank.

6.3.7.2 Từ phoneme sang học đầu cuối

Trước đây, hệ thống nhận dạng giọng nói được xây dựa trên *phoneme*: các đơn vị âm cơ bản do chuyên gia ngôn ngữ thiết kế thủ công (ví dụ "the" gồm âm "d" và "e"). Với học sâu đầu cuối (end-to-end), không còn cần biểu diễn thủ công như vậy: mô hình nhận thẳng âm thanh và xuất thẳng bản chép lời. Điều kiện cần là **dữ liệu lớn**:

Nguồn dữ liệu	Quy mô âm thanh đã được chép lời
Bộ dữ liệu học thuật	khoảng 300 giờ; 3.000 giờ đã là khá lớn
Hệ thống thương mại tốt nhất	hơn 10.000 giờ, thậm chí trên 100.000 giờ

Bảng 6.28: Dữ liệu càng nhiều, nhận dạng giọng nói càng chính xác



Hình 6.39: Quy trình nhận dạng giọng nói và hai cách giải mã

6.3.7.3 Hai cách xây dựng

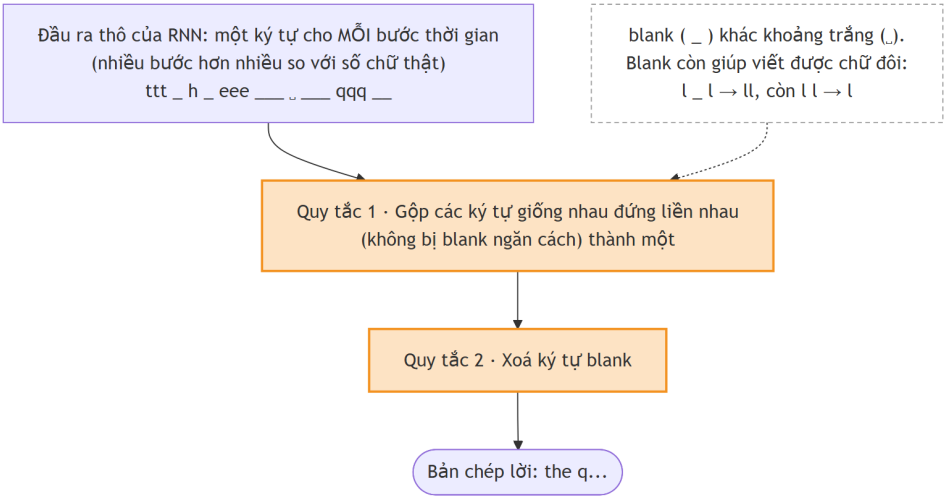
- **Mô hình chú ý.** Các khung thời gian của âm thanh là đầu vào, mô hình chú ý sinh ra bản chép lời, giống dịch máy ở mục 6.3.6.
- **Hàm lỗi CTC** (Connectionist Temporal Classification, của Graves và các cộng sự). Dùng một RNN có số bước đầu ra bằng số bước đầu vào; trong thực tế thường là LSTM hoặc GRU hai chiều và sâu hơn.

6.3.7.4 CTC: nhiều bước vào, ít chữ ra

Với nhận dạng giọng nói, số bước thời gian đầu vào lớn hơn rất nhiều số ký tự đầu ra. Đoạn âm thanh 10 giây, mỗi giây 100 mẫu, cho 1.000 đầu vào; nhưng bản chép lời của "the quick brown fox" chỉ có 19 ký tự kể cả khoảng trắng. CTC giải quyết bằng cách cho phép RNN xuất ký tự lặp lại và một ký tự đặc biệt gọi là **blank** (khác với khoảng trắng). Quy tắc rút gọn gồm hai bước (Hình 6.40):

1. Gộp các ký tự giống nhau đứng liền nhau *không bị blank ngăn cách* thành một.
2. Xoá các ký tự blank.

Nhờ đó 1.000 đầu ra thô có thể rút về đúng 19 ký tự. Blank còn giúp viết được chữ đôi: chuỗi "l blank l" ra "ll", còn "l l" liền nhau chỉ ra một "l". Ý tưởng này được dùng trong bài báo gốc và trong hệ thống Deep Speech của Baidu.



Hình 6.40: Quy tắc rút gọn của CTC

```
def ctc_collapse(symbols, blank="_"):
    """Rút gọn đầu ra thô của CTC: gộp ký tự lặp liền nhau (không có blank xen
    ↪  giữa), rồi bỏ blank."""
    out, prev = [], None
    for s in symbols:
        if s != prev and s != blank:
            out.append(s)
        prev = s
    return "".join(out)

print(repr(ctc_collapse("ttt_h_eee___ ___qqq__")))     # 'the q'  (dấu cách là
↪  khoảng trắng thật)
print(repr(ctc_collapse("hheel_llloo")))               # 'hello'  (blank giữa hai
↪  chữ l giữ lại chữ đôi)
```

Hàm trên là bước *giải mã tham lam* của CTC. Hàm lỗi CTC khi huấn luyện (cộng xác suất của mọi cách xếp ký tự có thể rút về cùng một bản chép lời) đã có sẵn trong framework, ví dụ `tf.nn.ctc_loss` của TensorFlow, nên không cần tự cài.

6.3.7.5 Dùng mô hình huấn luyện sẵn

Xây một hệ thống chép lời quy mô sản xuất là công việc lớn và cần rất nhiều dữ liệu. Thực tế hầu như luôn dùng mô hình huấn luyện sẵn. Ví dụ `wav2vec 2.0` là mô hình dùng CTC, và Whisper là mô hình encoder–decoder có chú ý. Với Hugging Face Transformers (cần cài thêm `ffmpeg` để đọc tệp âm thanh):

```
from transformers import pipeline

asr = pipeline("automatic-speech-recognition",
↪  model="facebook/wav2vec2-base-960h")   # mô hình CTC tiếng Anh
print(asr("quick_brown_fox.wav")["text"])
```

6.3.8 Phát hiện từ khoá đánh thức

Hệ thống **từ khoá đánh thức** (trigger word detection, còn gọi keyword hay wake word detection) giúp thiết bị thức dậy khi nghe một cụm từ riêng: Amazon Echo với "Alexa", Baidu DuerOS với "xiaoduni-hao", Apple Siri với "Hey Siri", Google Home với "Okay Google". Bạn nói "Alexa, what time is it?" thì thiết bị tự thức dậy và trả lời. Cũng có thể tự làm: cho máy tính phản ứng với "computer, activate" hay bật tắt một cái đèn. Bài toán này dễ hơn nhiều so với chép lời cả đoạn và cần ít dữ liệu hơn. Đây là lĩnh vực còn phát triển, chưa có đồng thuận về thuật toán tốt nhất; mục này giới thiệu một cách làm mẫu.

6.3.8.1 Cách làm

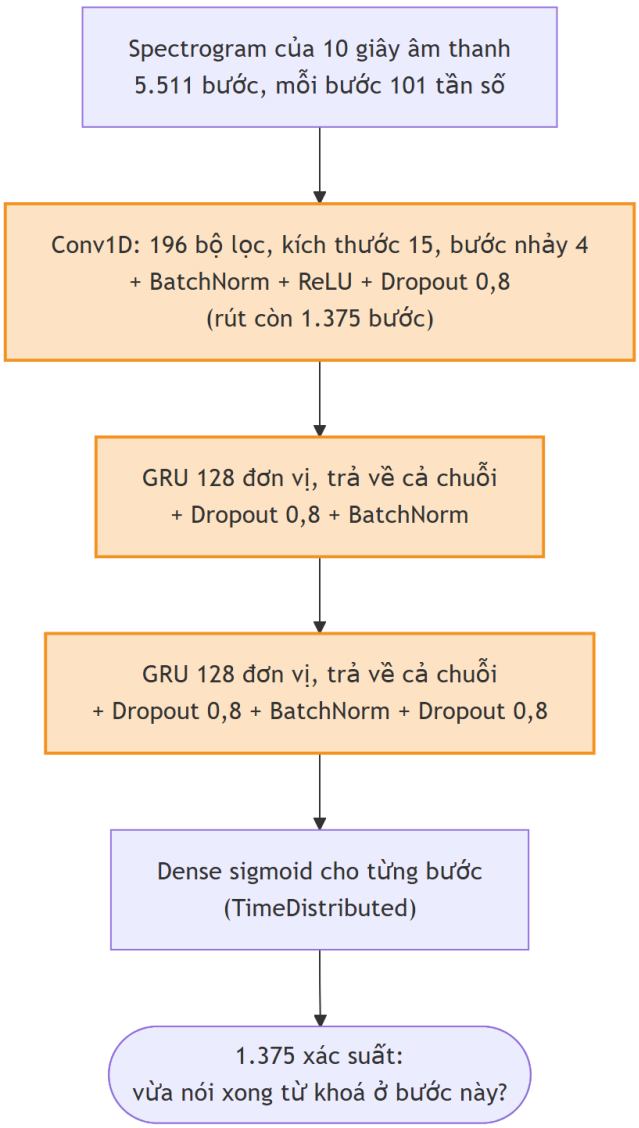
Lấy đoạn âm thanh, tính đặc trưng spectrogram theo thời gian, rồi đưa qua một RNN. Phần còn lại là định nghĩa **nhãn đích** cho từng bước thời gian: đặt 0 cho mọi bước trước thời điểm người nói vừa nói xong từ khoá, và đặt 1 ngay sau đó. Nếu từ khoá được nói lần nữa, lại đặt 1 ngay sau lần đó.

Cách gán nhãn này có một điểm yếu: tập dữ liệu rất **mất cân bằng**, số 0 nhiều hơn số 1 rất nhiều. Mẹo để dễ huấn luyện hơn: thay vì chỉ đặt 1 đúng một bước, đặt 1 liên tiếp trong một khoảng thời gian cố định rồi mới trở về 0. Việc này cân bằng tỷ lệ 1 và 0 hơn, dù chỉ là một mẹo.

6.3.8.2 Mô hình

Mô hình (Hình 6.41) gồm bốn phần:

- **Tích chập 1 chiều** – 196 bộ lọc, kích thước 15, bước nhảy 4, rồi BatchNorm, ReLU, Dropout 0,8. Lớp này trích đặc trưng mức thấp và rút chuỗi từ 5.511 bước xuống còn 1.375 bước, giúp GRU nhanh hơn rất nhiều.
- **Hai lớp GRU** – mỗi lớp 128 đơn vị, trả về cả chuỗi, kèm Dropout 0,8 và BatchNorm.
- **Dense kèm sigmoid** áp dụng cho từng bước thời gian, cho ra xác suất nhị phân "vừa nói xong từ khoá".
- **Dùng RNN một chiều**, vì cần phát hiện ngay khi vừa nói xong từ khoá, không đợi nghe phần sau.



Hình 6.41: Mô hình phát hiện từ khoá đánh thức

```
def build_trigger_model(input_shape=(5511, 101)):
    X_input = keras.Input(shape=input_shape)                    # 5.511 bước, mỗi
    ↪  bước 101 tần số

    X = layers.Conv1D(filters=196, kernel_size=15, strides=4)(X_input)   # 5.511
    ↪  bước -> 1.375 bước
    X = layers.BatchNormalization()(X)
    X = layers.Activation("relu")(X)
    X = layers.Dropout(0.8)(X)

    X = layers.GRU(128, return_sequences=True)(X)
    X = layers.Dropout(0.8)(X)
    X = layers.BatchNormalization()(X)

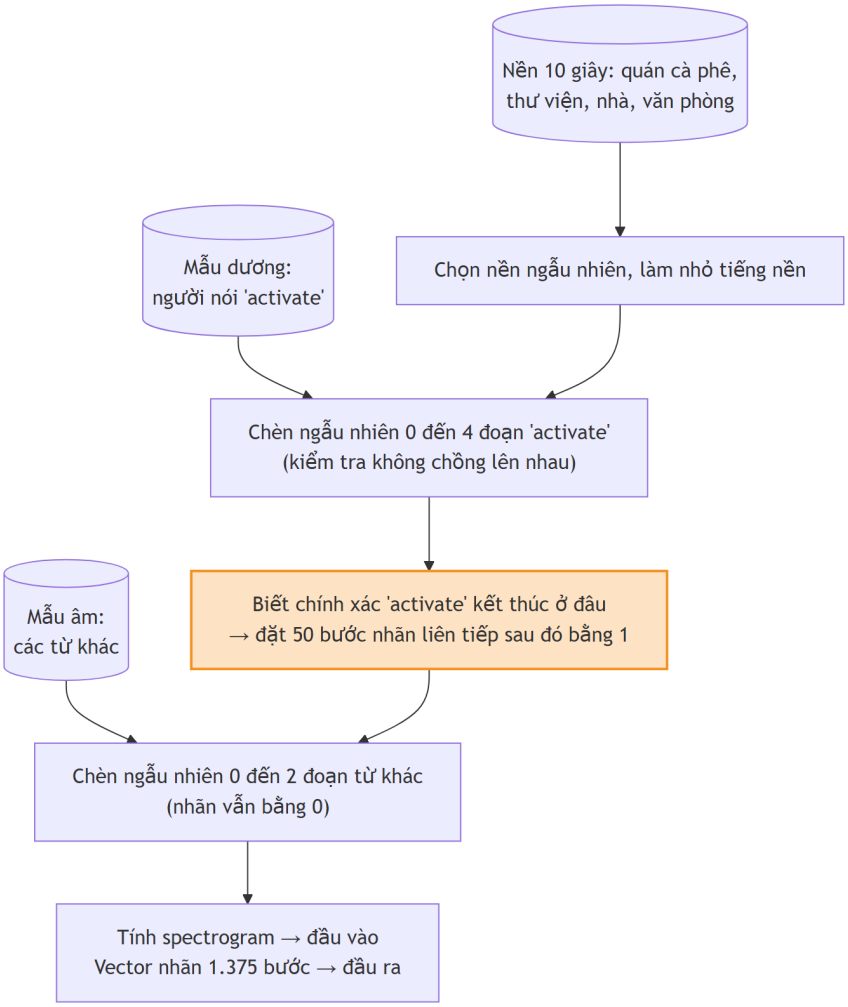
    X = layers.GRU(128, return_sequences=True)(X)
    X = layers.Dropout(0.8)(X)
    X = layers.BatchNormalization()(X)
    X = layers.Dropout(0.8)(X)
```

```
    X = layers.TimeDistributed(layers.Dense(1, activation="sigmoid"))(X)  # xác
    ↪  suất cho từng bước
    return keras.Model(X_input, X)

model = build_trigger_model()
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-4),   # giá trị
↪  gợi ý khi huấn luyện từ đầu
              loss="binary_crossentropy", metrics=["accuracy"])
```

6.3.8.3 Tổng hợp dữ liệu

Dữ liệu giọng nói khó thu và khó gán nhãn, nên ta **tổng hợp** dữ liệu từ ba nguồn: đoạn "activate" (mẫu dương), đoạn từ ngẫu nhiên khác (mẫu âm), và 10 giây âm thanh nền (quán cà phê, thư viện, nhà, văn phòng, nhiều giọng vùng miền). Vì chèn thủ công nên biết chính xác thời điểm "activate" kết thúc, nên rất dễ tạo nhãn.



Hình 6.42: Tổng hợp một mẫu huấn luyện

Các mức chia thời gian khác nhau cùng tồn tại trong bài, dễ gây nhầm:

Mức	Số bước cho 10 giây	Ghi chú
Âm thô	441.000	44.100 mẫu mỗi giây, phụ thuộc micro
Thư viện `pydub`	10.000	mỗi bước 1 mili giây, dùng để chèn đoạn âm
Spectrogram (đầu vào mô hình)	5.511	mỗi bước có 101 tần số
Đầu ra mô hình	1.375	mỗi bước dự đoán "vừa nói xong từ khoá chưa"

Bảng 6.29: Các mức chia thời gian của 10 giây âm thanh

Phần cốt lõi của tổng hợp dữ liệu gồm kiểm tra chồng lấn, chèn đoạn âm và gán nhãn. Dưới đây là phiên bản gọn bằng `pydub` và NumPy (spectrogram có thể tính bằng thư viện xử lý tín hiệu, ví dụ `scipy.signal.spectrogram`):

```
import numpy as np

def is_overlapping(seg, previous):
    start, end = seg
    return any(start <= p_end and end >= p_start for p_start, p_end in previous)

def insert_clip(background, clip, previous, tries=5):
    """Chèn clip vào một vị trí ngẫu nhiên không chồng lên đoạn đã chèn. Trả về
    ↪  (nền mới, đoạn thời gian)."""
    n = len(clip)                                              # độ dài tính bằng
    ↪  mili giây
    for _ in range(tries + 1):
        start = np.random.randint(0, 10000 - n)
        seg = (start, start + n - 1)
        if not is_overlapping(seg, previous):
            previous.append(seg)
            return background.overlay(clip, position=start), seg
    return background, None                                    # không chèn được

def insert_ones(y, segment_end_ms, Ty=1375):
    """50 bước ngay sau khi 'activate' kết thúc được đặt bằng 1 (cắt bớt nếu chạm
    ↪  cuối mảng)."""
    end = int(segment_end_ms * Ty / 10000.0)
    if end < Ty:
        y[0, end + 1:end + 51] = 1
    return y

def synthesize(background, activates, negatives, Ty=1375):
    background = background - 20                               # làm nhỏ tiếng
    ↪  nền 20 dB
    y, previous = np.zeros((1, Ty)), []
    for i in np.random.randint(len(activates), size=np.random.randint(0, 5)):
    ↪  # 0 đến 4 lần 'activate'
        background, seg = insert_clip(background, activates[i], previous)
        if seg:
            y = insert_ones(y, seg[1])
    for i in np.random.randint(len(negatives), size=np.random.randint(0, 3)):
    ↪  # 0 đến 2 từ khác, nhãn vẫn 0
        background, _ = insert_clip(background, negatives[i], previous)
    return background, y
```

6.3.8.4 Huấn luyện, đánh giá và phát chuông

Huấn luyện mô hình này từ đầu mất khoảng 3 giờ trên GPU với khoảng 4.000 mẫu; khi đã có mô hình huấn luyện sẵn thì chỉ cần tinh chỉnh thêm một chút (Adam, learning rate rất nhỏ 1e-6, hàm lỗi `binary_crossentropy`). Khi tinh chỉnh một mô hình đã huấn luyện, cần **khoá các lớp BatchNorm** để thống kê chuẩn hoá đã học không bị phá:

```
for layer in model.layers:
    if isinstance(layer, layers.BatchNormalization):
        layer.trainable = False                    # khoá BatchNorm khi tinh
        ↪  chỉnh
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-6),
↪  loss="binary_crossentropy",
             metrics=["accuracy"])                 # phải compile lại sau khi
              ↪  đổi trainable
model.fit(X, Y, batch_size=16, epochs=1)
```

Độ chính xác không phải thước đo tốt. Nhãn lệch mạnh về 0, nên một mạng chỉ xuất toàn số 0 cũng đạt hơn 90% độ chính xác. Hãy dùng F1 hoặc precision/recall, đồng thời nghe thử trên âm thanh thật.

Từ xác suất thành tiếng chuông. Chuẩn hoá biên độ âm thanh, tính spectrogram, đảo trục thành dạng (số bước, số tần số), thêm chiều batch rồi đưa vào mô hình. Khi xác suất vượt một *ngưỡng* (ví dụ 0,5) thì kích hoạt, nhưng chỉ **tối đa một lần mỗi 75 bước đầu ra**, vì sau khi nói "activate" xác suất ở nhiều bước liên tiếp đều cao mà ta chỉ muốn chuông kêu một lần:

```
def trigger_times(pred, threshold=0.5, min_gap=75):
    """pred: xác suất theo từng bước đầu ra. Trả về các bước cần phát chuông
    ↪  (cách nhau ít nhất min_gap bước)."""
    hits, last = [], -min_gap
    for i, p in enumerate(pred):
        if p > threshold and i - last >= min_gap:
            hits.append(i)
            last = i
    return hits

# Đổi bước đầu ra thành mili giây trên đoạn âm 10 giây để chèn chuông: ms = i /
↪  1375 * 10000
```

Nếu chuông không kêu đúng lúc, hãy thử chỉnh ngưỡng.

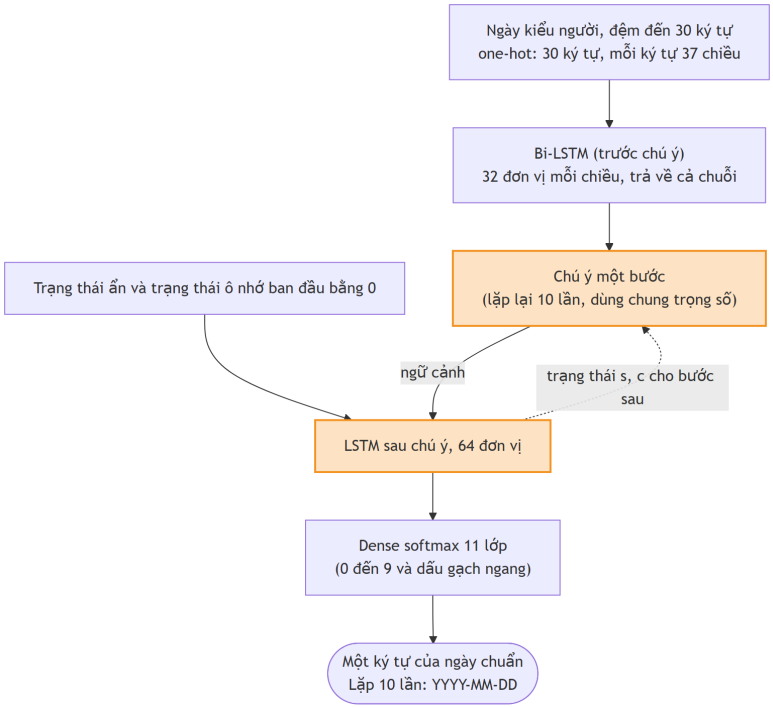
6.3.9 Thực hành 1: Chuẩn hoá ngày tháng bằng mô hình chú ý

Bài toán: dịch ngày viết kiểu con người sang định dạng máy đọc được, ví dụ "25th of June, 2009" thành "2009-06-25", hay "the 29th of August 1958", "03/30/1968", "24 JUNE 1987". Dùng ngày tháng thay vì dịch ngôn ngữ thật vì dịch thật cần tập dữ liệu khổng lồ và nhiều ngày huấn luyện trên GPU; đây là chỗ thử mô hình chú ý trên chuỗi đủ nhỏ.

6.3.9.1 Dữ liệu

Bộ dữ liệu gồm 10.000 cặp (ngày kiểu người, ngày chuẩn), tạo bằng các thư viện `Faker` và `Babel`. Đầu vào đệm đến 30 ký tự (bằng ký tự pad), đầu ra luôn 10 ký tự theo dạng năm-tháng-ngày. Từ điển đầu ra chỉ có 11 ký tự (10 chữ số và dấu gạch ngang). Dữ liệu được chuyển sang chỉ số rồi sang one-hot.

6.3.9.2 Kiến trúc



Hình 6.43: Mô hình chú ý chuẩn hoá ngày tháng

Ba điểm đáng nhớ:

- Có **hai LSTM đặt hai bên cơ chế chú ý**: một Bi-LSTM *trước* chú ý đi qua các ký tự nguồn, và một LSTM *sau* chú ý đi qua 10 ký tự đích. LSTM sau chú ý truyền cả trạng thái ẩn lẫn trạng thái ô nhớ giữa các bước.
- Khác các bài sinh văn bản trước, LSTM sau chú ý **không nhận dự đoán của bước trước** làm đầu vào, vì trong ngày tháng các ký tự liền kề không phụ thuộc mạnh vào nhau.
- Ngữ cảnh được đặt tên riêng (`context`) để khỏi nhầm với ô nhớ của LSTM.

Chú ý một bước đã vẽ ở Hình 6.38: lặp lại trạng thái ẩn trước đó cho đủ số vị trí nguồn (`RepeatVector`), nối với mọi đặc trưng của Bi-LSTM, đưa qua hai lớp Dense nhỏ (lớp một 10 đơn vị tanh, lớp hai 1 đơn vị) ra điểm, rồi softmax theo trục thời gian ra trọng số, và tích vô hướng (`Dot`) với các đặc trưng ra ngữ cảnh. **Các lớp phải khai báo một lần ở ngoài rồi gọi lại 10 lần trong vòng lặp**, để trọng số được dùng chung giữa các bước.

6.3.9.3 Viết bằng Keras

```
Tx, Ty, n_a, n_s = 30, 10, 32, 64

# Khai báo MỘT LẦN, gọi lại ở mọi bước để dùng chung trọng số
repeator = layers.RepeatVector(Tx)
concatenator = layers.Concatenate(axis=-1)
densor1 = layers.Dense(10, activation="tanh")
densor2 = layers.Dense(1, activation="relu")
activator = layers.Softmax(axis=1, name="attention_weights")   # softmax theo
↪  trục thời gian của câu nguồn
```

```
dotor = layers.Dot(axes=1)
post_attention_lstm = layers.LSTM(n_s, return_state=True)
output_layer = layers.Dense(11, activation="softmax")          # 10 chữ số và
↪  dấu gạch ngang

def one_step_attention(a, s_prev):
    s_prev = repeator(s_prev)                          # (batch, Tx, n_s)
    concat = concatenator([a, s_prev])                 # ghép mọi đặc trưng nguồn
    ↪  với trạng thái decoder
    energies = densor2(densor1(concat))                # điểm cho mỗi vị trí nguồn
    alphas = activator(energies)                       # trọng số chú ý, cộng lại
    ↪  bằng 1
    return dotor([alphas, a])                          # ngữ cảnh: (batch, 1, 2 *
    ↪  n_a)

def build_model(human_vocab_size):
    X = keras.Input(shape=(Tx, human_vocab_size))
    s0 = keras.Input(shape=(n_s,), name="s0")
    c0 = keras.Input(shape=(n_s,), name="c0")
    s, c = s0, c0
    a = layers.Bidirectional(layers.LSTM(n_a, return_sequences=True))(X)    #
    ↪  pre-attention Bi-LSTM
    outputs = []
    for _ in range(Ty):
        context = one_step_attention(a, s)
        _, s, c = post_attention_lstm(context, initial_state=[s, c])       # LSTM
        ↪  sau chú ý
        outputs.append(output_layer(s))
    return keras.Model([X, s0, c0], outputs)

model = build_model(human_vocab_size=37)               # khoảng 52.960 tham số
# Cách cũ: Adam(lr=0.005, decay=0.01); phiên bản mới thay decay bằng lịch giảm
↪  học suất:
lr = keras.optimizers.schedules.InverseTimeDecay(0.005, decay_steps=1,
↪  decay_rate=0.01)
model.compile(optimizer=keras.optimizers.Adam(lr, beta_1=0.9, beta_2=0.999),
              loss="categorical_crossentropy", metrics=["accuracy"])

m = Xoh.shape[0]
s0, c0 = np.zeros((m, n_s)), np.zeros((m, n_s))
outputs = list(Yoh.swapaxes(0, 1))                     # danh sách 10 phần tử, mỗi
↪  phần tử (m, 11)
model.fit([Xoh, s0, c0], outputs, epochs=1, batch_size=100)
```

Hai chi tiết: đầu ra của mô hình là *danh sách 10 phần tử* (mỗi ký tự đích một đầu ra), nên nhãn phải hoán đổi trục trước khi đưa vào; và khi huấn luyện, thông số hiển thị theo từng vị trí (ví dụ độ chính xác của ký tự thứ 9) cho biết vị trí nào còn khó. Huấn luyện vài phút trên GPU là đủ để đạt chất lượng tốt.

6.3.9.4 Trực quan hoá chú ý

Lấy đầu ra của lớp `attention_weights` (mỗi bước đích một lần) rồi vẽ thành bản đồ nhiệt. Với ngày tháng, phần lớn chú ý giúp dự đoán **năm**, ít tác động tới ngày và tháng.

6.3.9.5 Điều cần nhớ

- Mô hình dịch chuỗi dùng được cho nhiều việc ngoài dịch ngôn ngữ, như chuẩn hoá ngày tháng.

- Cơ chế chú ý giúp mạng tập trung vào phần liên quan nhất của đầu vào khi sinh từng phần đầu ra.
- Độ dài vào và ra có thể khác nhau.
- Có thể vẽ trọng số chú ý để xem mô hình đang nhìn vào đâu.

6.3.10 Thực hành 2: Phát hiện từ khoá "activate"

Bài toán: xây tập dữ liệu giọng nói và mô hình phát hiện từ "activate". Mỗi khi nghe thấy, hệ thống phát một tiếng chuông. Có thể mở rộng để bật ứng dụng hay đèn khi nghe từ khoá.

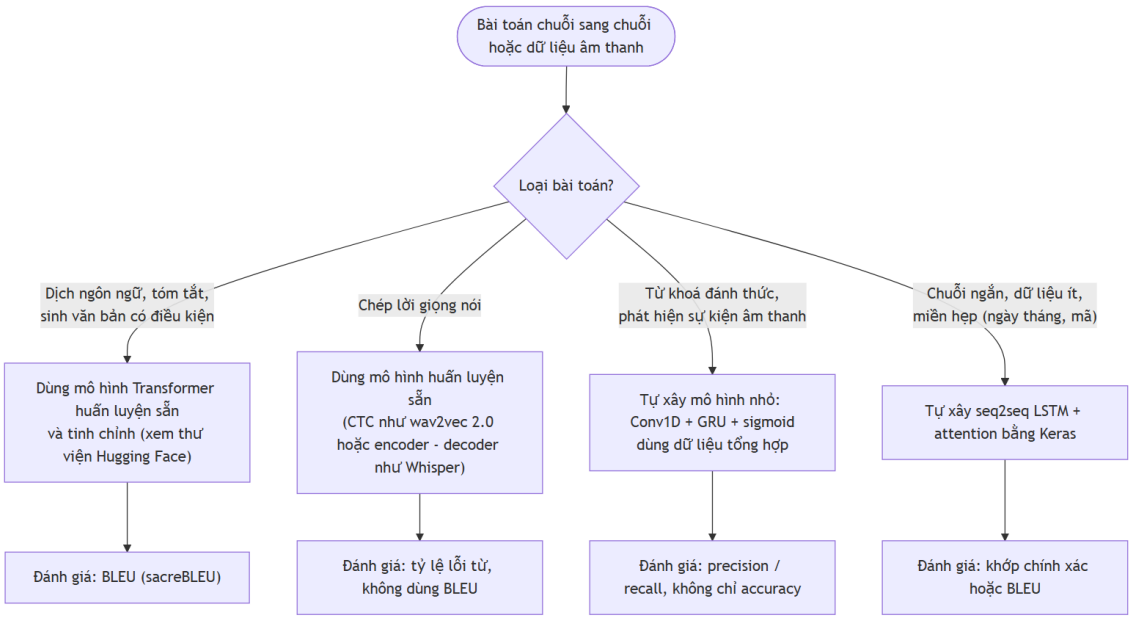
Mục 6.3.8 đã trình bày đầy đủ cách làm. Dưới đây là phần tóm lược theo trình tự các bước:

1. **Nghe dữ liệu.** Ba thư mục: `activates` (mẫu dương), `negatives` (từ khác, mỗi bản ghi một từ), `backgrounds` (nền 10 giây).
2. **Chuyển sang spectrogram.** Sóng âm 44.100 mẫu mỗi giây khó học trực tiếp; spectrogram cho biết mỗi tần số có mặt bao nhiêu tại từng thời điểm. Mỗi mẫu 10 giây có 5.511 bước thời gian (đầu vào) và 1.375 bước (đầu ra).
3. **Tổng hợp dữ liệu.** Chèn 0 đến 4 đoạn "activate" và 0 đến 2 đoạn từ khác lên nền, kiểm tra không chồng lên nhau, gán nhãn bằng cách đặt 50 bước sau mỗi lần "activate" bằng 1. Các bước này tương ứng với các hàm `is_overlapping`, `insert_clip`, `insert_ones`, `synthesize` ở trên.
4. **Mô hình.** Tích chập 1 chiều, hai lớp GRU, Dense sigmoid theo bước thời gian.
5. **Huấn luyện.** Dùng mô hình đã huấn luyện sẵn (khoảng 3 giờ trên GPU), tinh chỉnh nhẹ, khoá BatchNorm.
6. **Dự đoán.** Chuẩn hoá biên độ, tính spectrogram, chạy mô hình, rồi chèn tiếng chuông khi xác suất vượt ngưỡng, tối đa một lần mỗi 75 bước.
7. **Thử giọng của chính bạn.** Ghi một đoạn 10 giây có nói "activate" và xem hệ thống có chuông không; chỉnh ngưỡng nếu cần.

Điều cần nhớ: dùng spectrogram và (tuỳ chọn) một lớp tích chập 1 chiều là cách tiền xử lý phổ biến trước khi đưa dữ liệu âm thanh vào RNN, GRU hay LSTM.

6.3.11 Áp dụng vào dự án thực tế

6.3.11.1 Chọn cách làm



Hình 6.44: Chọn cách làm cho bài toán chuỗi sang chuỗi và dữ liệu âm thanh

Bài toán	Nên bắt đầu bằng	Đánh giá
Dịch ngôn ngữ, tóm tắt	mô hình huấn luyện sẵn (ví dụ qua Hugging Face), tinh chỉnh nếu cần	BLEU (sacreBLEU)
Chép lời giọng nói	mô hình huấn luyện sẵn (CTC hoặc encoder–decoder có chú ý)	tỷ lệ lỗi từ, không dùng BLEU
Từ khoá đánh thức	tự xây mô hình nhỏ, dùng dữ liệu tổng hợp	precision, recall, không chỉ accuracy
Chuỗi ngắn, miền hẹp (ngày tháng, mã)	tự xây seq2seq LSTM kèm chú ý bằng Keras	khớp chính xác hoặc BLEU

Bảng 6.30: Bắt đầu từ đâu với từng loại bài toán

6.3.11.2 Checklist

1. **Huấn luyện và chạy là hai chế độ khác nhau.** Lúc huấn luyện đưa câu đích thật (lệch phải một bước) vào decoder; lúc chạy phải tự sinh từng từ và đưa lại làm đầu vào. Luôn viết riêng hàm sinh và kiểm tra nó.
2. **Có token bắt đầu và kết thúc** (`<start>`, `EOS`) trong từ điển đích, nếu không decoder không biết khi nào dừng.
3. **Bật mask cho phần đệm** (`mask_zero=True`) ở cả hai `Embedding`; Keras tự truyền mask tới các lớp sau (LSTM, chú ý).
4. **Đừng bốc ngẫu nhiên khi dịch.** Dùng beam search với B vừa phải (quanh 5 đến 10), chuẩn hoá độ dài với alpha khoảng 0,6 đến 0,8, và thử vài giá trị.

5. **Đo bằng sacreBLEU và đọc thêm vài câu thật.** Một con số BLEU không thay thế việc đọc lỗi.
6. **Phân tích lỗi trước khi tăng beam width:** xác định lỗi do beam search hay do mô hình (mục 6.3.4).
7. **Với dữ liệu âm thanh, kiểm tra tốc độ lấy mẫu và độ dài đoạn** giống lúc huấn luyện; tổng hợp dữ liệu phải sát môi trường thật.
8. **Luôn so với mô hình huấn luyện sẵn** trước khi tự xây từ đầu.

6.3.11.3 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Bản dịch ngắn bất thường, bỏ sót ý	thiếu chuẩn hoá theo độ dài trong beam search; thử `length_penalty` lớn hơn
Bản dịch lặp lại một cụm	chọn từ có xác suất cao nhất ở mỗi bước (greedy); dùng beam search và cân nhắc phạt lặp
Chất lượng tụt hẳn với câu dài	đang dùng encoder–decoder thuần; thêm cơ chế chú ý hoặc dùng mô hình Transformer
Mô hình huấn luyện tốt nhưng lúc chạy ra rác	lúc chạy không đưa đúng token bắt đầu, hoặc lệch phải câu đích sai khi huấn luyện
Mô hình không bao giờ dừng sinh	thiếu token `EOS` trong dữ liệu đích, hoặc không đặt độ dài tối đa
BLEU rất thấp dù bản dịch đọc được	chỉ có một câu tham chiếu, hoặc tách từ khác nhau giữa các lần đo; dùng sacreBLEU
Dùng BLEU cho chép lời giọng nói	chép lời thường chỉ có một đáp án đúng; dùng tỷ lệ lỗi từ
Từ khoá đánh thức báo nhầm hoặc chuông kêu nhiều lần	chưa giới hạn một lần mỗi 75 bước, hoặc ngưỡng quá thấp
Từ khoá đánh thức có độ chính xác 90% nhưng không phát hiện được	nhãn lệch về 0; xem precision/recall thay vì accuracy
Tinh chỉnh mô hình âm thanh làm hỏng kết quả	chưa khoá BatchNorm khi tinh chỉnh, hoặc learning rate quá lớn

Bảng 6.31: Lỗi thường gặp với mô hình chuỗi sang chuỗi và âm thanh

6.3.11.4 Giới hạn của RNN và hướng đi tiếp

Mô hình encoder–decoder kèm chú ý trong mục này dựa trên RNN, đọc chuỗi tuần tự từng bước nên chậm với câu rất dài. Ý tưởng chú ý đã được phát triển thành kiến trúc **Transformer**, bỏ hẳn RNN và xử lý song song mọi vị trí. Transformer hiện là nền tảng của hầu hết các hệ thống dịch máy và chép lời hiện đại. Dù vậy, hiểu encoder–decoder, beam search, BLEU và chú ý vẫn rất quan trọng: các khái niệm này được giữ nguyên trong Transformer, chỉ khác cách cài đặt.

6.3.12 Tóm tắt cần nhớ

1. **Encoder–decoder:** encoder nén câu nguồn, decoder sinh câu đích từng từ. Cùng kiến trúc dùng cho mô tả ảnh (encoder là mạng tích chập).
2. **Dịch máy là mô hình ngôn ngữ có điều kiện:** xác suất câu đích khi biết câu nguồn, khác mô hình ngôn ngữ ở điểm xuất phát của decoder.

3. **Khi dịch, tìm câu xác suất cao nhất, không bốc ngẫu nhiên.** Greedy không tối ưu. **Beam search** giữ một số ứng viên tốt nhất (độ rộng chùm) ở mỗi bước; độ rộng chùm bằng 1 chính là greedy.

4. **Hai tinh chỉnh beam search:** cộng logarit để không tràn số, và chuẩn hoá theo độ dài (alpha) để không thiên vị câu ngắn. Beam width lớn thì tốt hơn nhưng chậm hơn, lợi ích giảm dần.

5. **Phân tích lỗi:** chấm điểm cả bản dịch của người lẫn của máy bằng mô hình. Người cao hơn thì lỗi do beam search, ngược lại thì lỗi do mô hình.

6. **BLEU:** đo độ giống của bản dịch với các tham chiếu bằng độ chính xác có cắt của cụm 1 đến 4 từ, nhân hình phạt câu ngắn. Dùng sacreBLEU; không dùng cho chép lời giọng nói.

7. **Chú ý:** ở mỗi bước, một mạng nhỏ tính điểm cho từng vị trí nguồn, softmax thành trọng số, rồi lấy tổng có trọng số làm ngữ cảnh. Giải quyết vấn đề câu dài; chi phí tăng theo bình phương độ dài.

8. **Giọng nói:** spectrogram làm đầu vào; học đầu cuối thay phoneme; hai cách giải mã là chú ý và CTC (gộp ký tự lặp, bỏ blank).

9. **Từ khoá đánh thức:** RNN một chiều trên spectrogram, nhãn là 1 trong 50 bước sau khi nói xong từ khoá, dữ liệu tổng hợp từ nền, mẫu dương và mẫu âm; đánh giá bằng precision/recall; phát chuông tối đa một lần mỗi 75 bước.

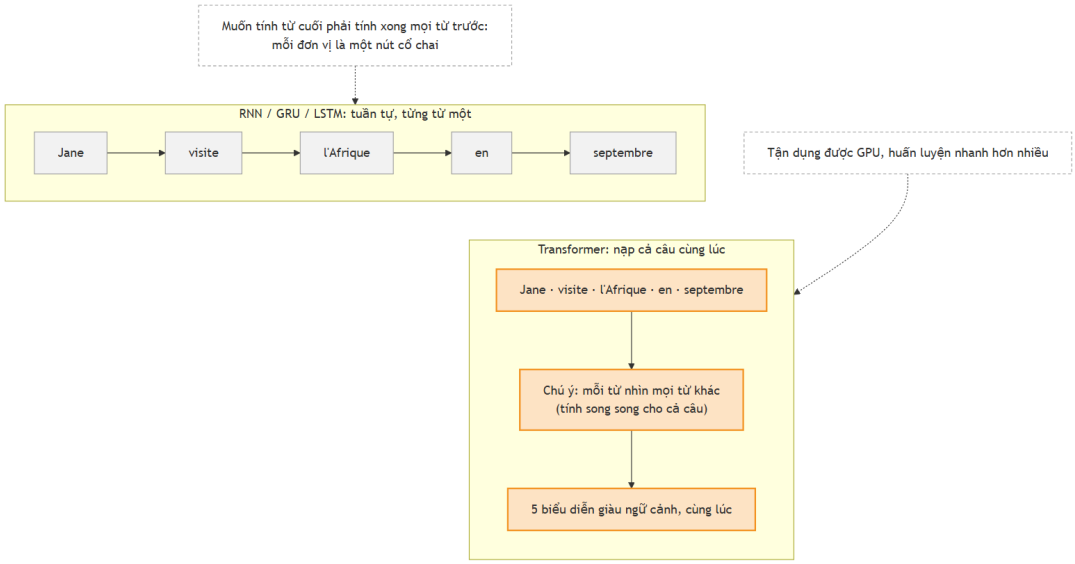
6.4 Mạng Transformer

Các mục trước đi một con đường dài: RNN (Mục 6.1), embedding (Mục 6.2), encoder–decoder với cơ chế chú ý (Mục 6.3). Mục này là điểm đến của con đường đó: **Transformer**, kiến trúc đã "càn quét" lĩnh vực xử lý ngôn ngữ tự nhiên. Nhiều thuật toán NLP hiệu quả nhất hiện nay dựa trên nó. Kiến trúc khá phức tạp, nên mục này đi từng mảnh: vì sao cần → self-attention → multi-head attention → ghép thành encoder và decoder → mã hoá vị trí và mặt nạ → dựng bằng Keras → dùng mô hình huấn luyện sẵn qua thư viện Hugging Face → ba ví dụ thực hành → checklist áp dụng vào dự án.

6.4.1 Vì sao cần Transformer

Khi bài toán chuỗi phức tạp hơn thì mô hình cũng phức tạp hơn. RNN gặp tiêu biến gradient nên khó nắm phụ thuộc xa. GRU rồi LSTM dùng cổng để điều khiển dòng thông tin và chữa được nhiều vấn đề đó, nhưng mỗi đơn vị có thêm tính toán, nên mô hình càng ngày càng phức tạp. Và cả ba đều là mô hình **tuần tự**: đọc từng từ (token) một. Muốn tính đầu ra của đơn vị cuối thì phải tính xong mọi đơn vị trước đó, nên mỗi đơn vị là một nút cổ chai của dòng thông tin.

Transformer bỏ hẳn kiểu đọc tuần tự. Nó nạp **cả câu cùng lúc** và chạy phần lớn phép tính cho cả chuỗi *song song*. Đổi mới chính của bài báo gốc (Vaswani cùng các cộng sự) là kết hợp biểu diễn dựa trên chú ý với kiểu xử lý song song giống CNN: CNN nhận nhiều pixel hay nhiều từ cùng lúc và tính biểu diễn song song, còn mạng chú ý cho ra biểu diễn rất giàu thông tin cho từng từ.



Hình 6.45: RNN đọc tuần tự, Transformer xử lý cả câu song song

Tiêu chí	RNN, GRU, LSTM	Transformer
Cách đọc câu	từng từ một, từ trái sang phải	cả câu cùng lúc
Tính toán song song	khó, mỗi bước chờ bước trước	dễ, huấn luyện nhanh trên GPU
Phụ thuộc xa	phải qua nhiều bước nên dễ mất thông tin	mọi từ nhìn thấy trực tiếp mọi từ khác
Thông tin thứ tự	có sẵn nhờ cách đọc tuần tự	phải thêm vào bằng mã hoá vị trí
Chi phí theo độ dài chuỗi	tăng tuyến tính	tăng theo bình phương (mọi cặp từ)

Bảng 6.32: So sánh RNN và Transformer

Hai ý tưởng cốt lõi sẽ học: **self-attention** (tính một biểu diễn dựa trên chú ý cho từng từ của câu, đồng thời) và **multi-head attention** (lặp self-attention nhiều lần để có nhiều phiên bản biểu diễn).

6.4.2 Self-attention

6.4.2.1 Biểu diễn phụ thuộc ngữ cảnh

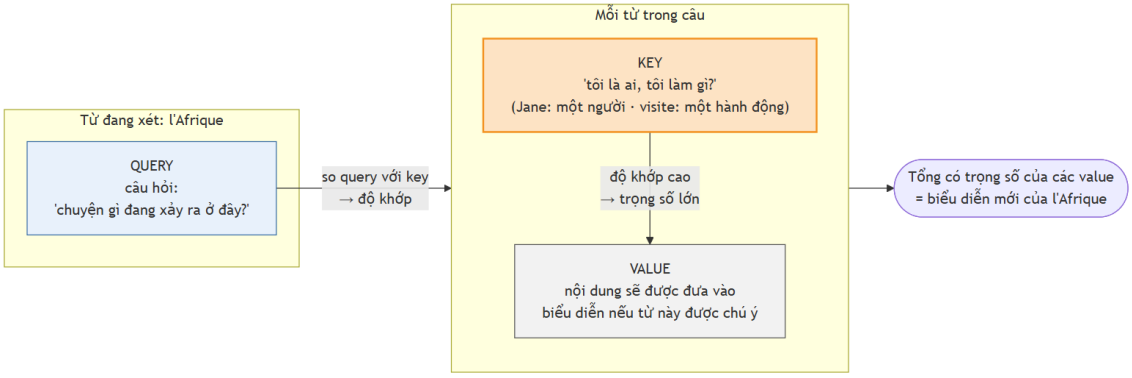
Xét câu "Jane visite l'Afrique en septembre". Nếu chỉ tra embedding cố định của "l'Afrique" thì ta không biết bối cảnh: Châu Phi là địa điểm lịch sử, điểm đến nghỉ dưỡng, hay lục địa lớn thứ hai thế giới? Self-attention tạo cho *mỗi từ* một biểu diễn mới bằng cách nhìn vào các từ xung quanh để biết câu đang nói về nó theo nghĩa nào. Với câu năm từ ta nhận được năm biểu diễn, và chúng được tính song song.

6.4.2.2 Query, key, value

Mỗi từ có ba vector, đều tính bằng cách nhân embedding của từ với ba ma trận trọng số học được (là tham số của mô hình): **query** (truy vấn), **key** (khoá) và **value** (giá trị). Tên gọi mượn từ cơ sở dữ liệu (truy vấn và cặp khoá–giá trị); nếu chưa quen khái niệm này thì không sao, phần giải thích dưới đây là đủ (Hình 6.46):

- **Query** là một câu hỏi bạn đặt cho từ đang xét. Với "l'Afrique" câu hỏi có thể là "chuyện gì đang xảy ra ở đây?".

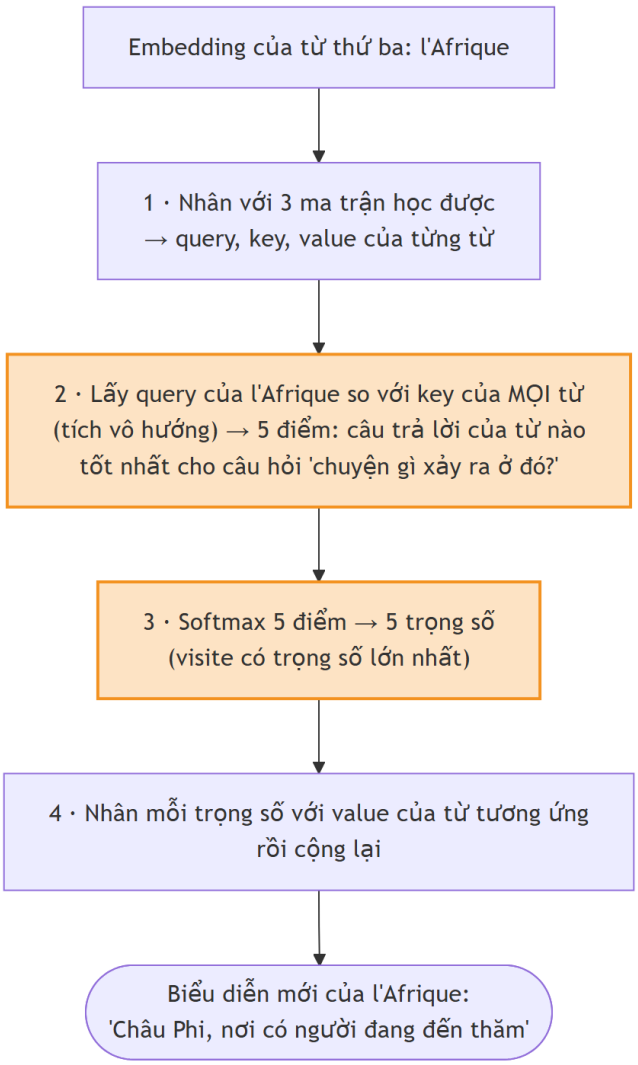
- **Key** của mỗi từ cho biết từ đó là gì, để so với câu hỏi. Key của "Jane" cho biết đó là một người, key của "visite" cho biết đó là một hành động.
- **Value** là nội dung của từ đó sẽ được đưa vào biểu diễn cuối nếu từ đó được chú ý nhiều.



Hình 6.46: Vai trò của query, key và value

6.4.2.3 Các bước tính cho một từ

1. Tính query, key, value cho từng từ.
2. Lấy query của từ đang xét ("l'Afrique") so với key của *mọi* từ trong câu bằng tích vô hướng, ra 5 điểm. Điểm cao nghĩa là từ đó trả lời tốt câu hỏi. Ở ví dụ, query của "l'Afrique" khớp nhất với key của "visite": Châu Phi là điểm đến của một chuyến thăm.
3. Đưa 5 điểm qua softmax thành 5 trọng số (không âm và cộng lại bằng 1).
4. Nhân mỗi trọng số với value của từ tương ứng rồi cộng lại. Kết quả là biểu diễn mới của "l'Afrique": Châu Phi, nơi có người đang đến thăm.



Hình 6.47: Các bước tính self-attention cho từ "l'Afrique"

Phép tính này làm cho tất cả các từ cùng lúc bằng cách gom query, key, value của mọi từ thành các ma trận. Trong công thức có một số hạng chia để thu nhỏ các tích vô hướng, tránh giá trị quá lớn làm softmax bão hoà. Vì thế gọi là **scaled dot-product attention** (chú ý tích vô hướng có chia tỷ lệ), đúng như trong bài báo gốc "Attention Is All You Need". Điểm khác chính so với chú ý trên RNN ở Mục 6.3: ở đây không có mạng nhỏ để tính điểm; điểm là tích vô hướng giữa query và key, và mỗi từ có query, key, value riêng.

6.4.2.4 Viết bằng NumPy để thấy cơ chế

Hàm dưới đây chỉ vài dòng và nhận thêm một mặt nạ tuỳ chọn (xem mục 6.4.6):

```
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)          # trừ giá trị lớn nhất để
    ↪  tránh tràn số
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def scaled_dot_product_attention(q, k, v, mask=None):
```

```
    """q, k, v: (batch, số từ, chiều). Trả về (đầu ra, trọng số chú ý)."""
    d_k = k.shape[-1]
    scores = q @ k.transpose(0, 2, 1) / np.sqrt(d_k)  # độ khớp của mọi cặp
    ↪  (query, key)
    if mask is not None:
        scores = scores + (1.0 - mask) * -1e9         # vị trí bị che bị đẩy về
        ↪  rất âm
    weights = softmax(scores, axis=-1)                # mỗi dòng cộng lại bằng 1
    return weights @ v, weights                       # tổng có trọng số của các
    ↪  value

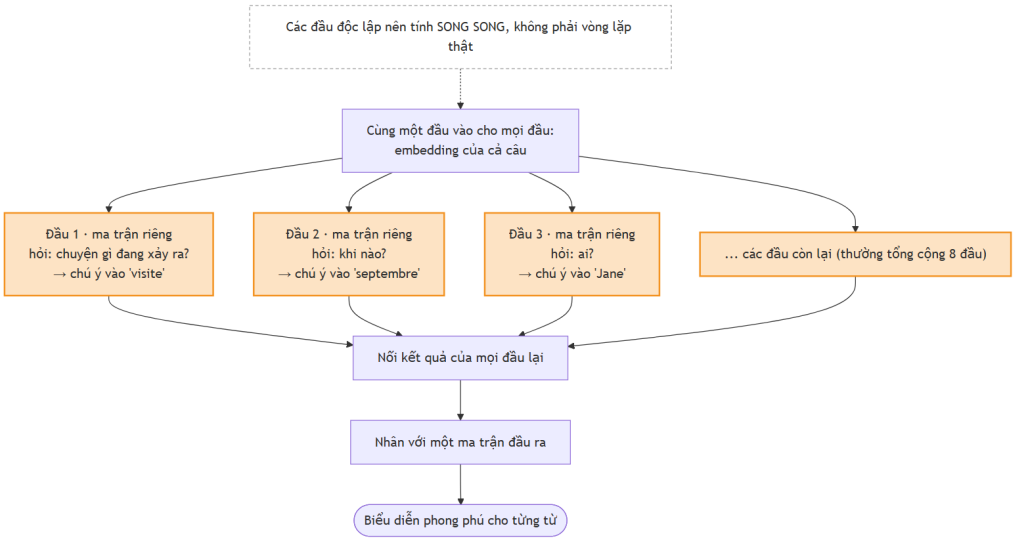
rng = np.random.default_rng(0)
x = rng.normal(size=(1, 5, 8))                        # 1 câu, 5 từ, embedding 8
↪  chiều
Wq, Wk, Wv = (rng.normal(size=(8, 4)) for _ in range(3))   # 3 ma trận học được
↪  (ở đây là ngẫu nhiên)
q, k, v = x @ Wq, x @ Wk, x @ Wv
out, weights = scaled_dot_product_attention(q, k, v)
print(out.shape, weights.shape)                       # (1, 5, 4) (1, 5, 5)
print(np.allclose(weights.sum(-1), 1))                # True: trọng số mỗi từ
↪  cộng lại bằng 1
```

6.4.3 Multi-head attention

Multi-head attention chỉ là một vòng lặp lớn quanh self-attention. Mỗi lần tính self-attention cho cả chuỗi gọi là một **đầu** (head). Với *mỗi đầu*, query, key, value được tính bằng một bộ ba ma trận trọng số *riêng*, nên mỗi đầu đặt một câu hỏi khác nhau cho cùng từ "l'Afrique" (Hình 6.48):

- Đầu 1 hỏi "chuyện gì đang xảy ra?" → chú ý vào "visite".
- Đầu 2 hỏi "khi nào?" → chú ý vào "septembre".
- Đầu 3 hỏi "ai?" → chú ý vào "Jane".

Số đầu thường ký hiệu h, ví dụ 8 đầu. Có thể xem mỗi đầu là một đặc trưng khác nhau của câu. Kết quả của mọi đầu được **nối lại** rồi nhân với một ma trận đầu ra, cho biểu diễn rất phong phú cho từng từ. Về khái niệm thì như một vòng lặp, nhưng các đầu độc lập nhau (giá trị của đầu này không phụ thuộc đầu khác) nên thực tế **tính song song tất cả các đầu**.



Hình 6.48: Multi-head attention: mỗi đầu hỏi một câu hỏi khác nhau

6.4.3.1 Dùng lớp có sẵn trong Keras

Keras có sẵn lớp `MultiHeadAttention`, nên trong dự án không cần tự viết phép tính trên. Với self-attention, query, key và value đều là cùng một đầu vào:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

mha = layers.MultiHeadAttention(num_heads=8, key_dim=64)    # 8 đầu, mỗi đầu dùng
↪  query/key 64 chiều
x = tf.random.normal((2, 5, 512))                           # 2 câu, 5 từ,
↪  embedding 512 chiều

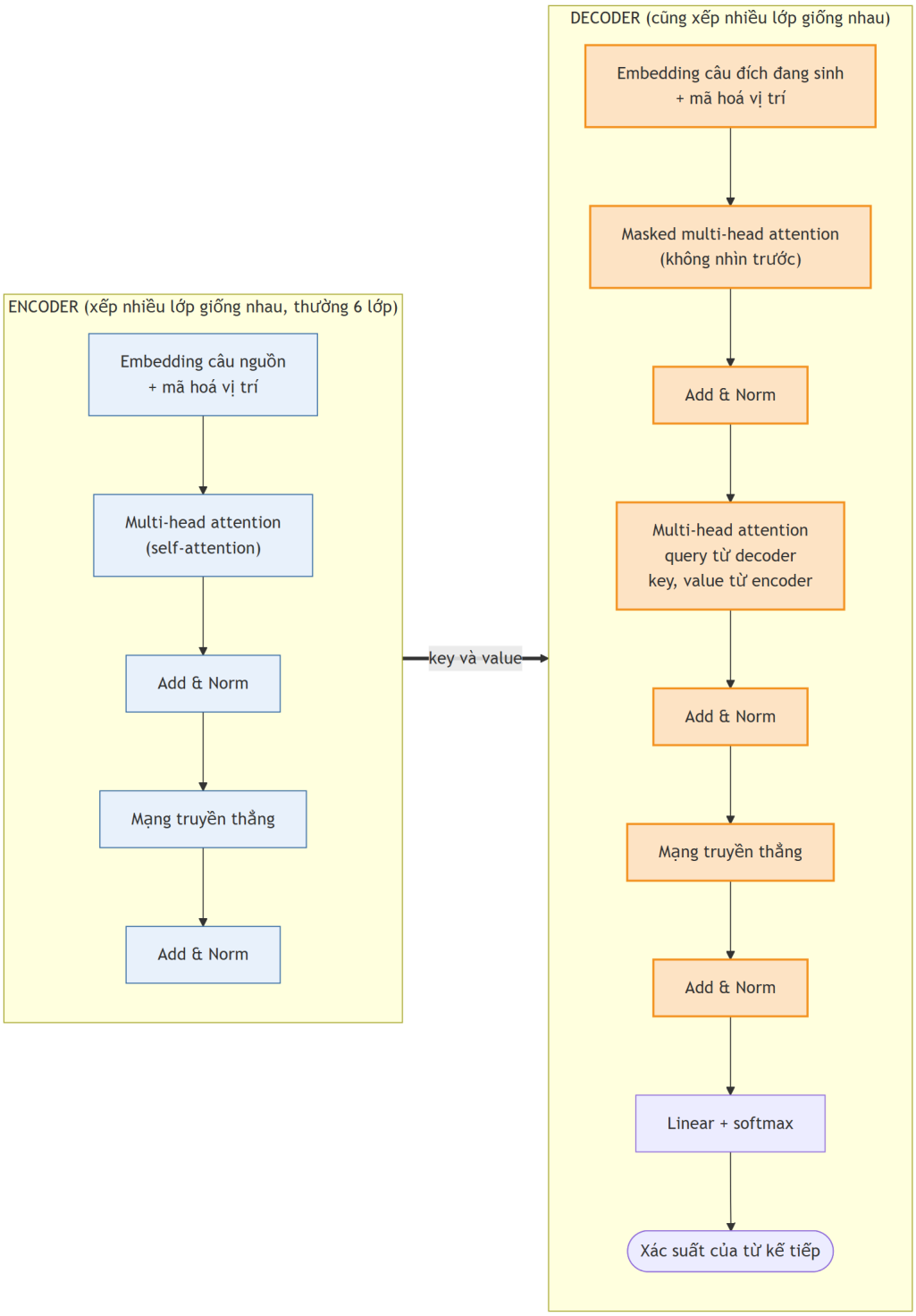
out, scores = mha(query=x, value=x, key=x, return_attention_scores=True)   #
↪  self-attention
print(out.shape)       # (2, 5, 512): mỗi từ có một biểu diễn mới, cùng kích
↪  thước embedding
print(scores.shape)    # (2, 8, 5, 5): với mỗi câu, mỗi đầu, trọng số chú ý của
↪  mọi cặp từ
```

Tham số / đối số	Ý nghĩa
`num_heads`	số đầu h
`key_dim`	kích thước query và key *của mỗi đầu*. Cách thường dùng: số chiều embedding chia cho số đầu
`query`, `value`, `key`	nguồn của query, value, key. Self-attention: cả ba cùng một đầu vào; chú ý chéo trong decoder: query từ decoder, key và value từ encoder
`attention_mask`	mặt nạ (1 là được nhìn, 0 là bị che), xem mục 6.4.6
`return_attention_scores`	trả thêm trọng số chú ý để vẽ và kiểm tra

Bảng 6.33: Các tham số hay dùng của `layers.MultiHeadAttention`

6.4.4 Ghép thành kiến trúc Transformer

Ví dụ xuyên suốt: dịch "Jane visite l'Afrique en septembre" sang tiếng Anh. Trong dịch máy hay thêm hai token đặc biệt: bắt đầu câu (`SOS`) và kết thúc câu (`EOS`). Kiến trúc gồm hai nửa (Hình 6.49).



Hình 6.49: Kiến trúc Transformer: encoder bên trái, decoder bên phải

6.4.4.1 Khối encoder

Embedding của câu nguồn (đã cộng mã hoá vị trí, xem mục 6.4.5) đi vào một lớp **multi-head attention**, trong đó query, key, value đều tính từ chính embedding đó. Kết quả qua một **mạng truyền thẳng** (feed-forward) để tìm ra các đặc trưng thú vị của câu. Cả khối được lặp lại N lần; bài báo gốc dùng N bằng 6. Đầu ra của encoder sẽ được decoder dùng làm key và value.

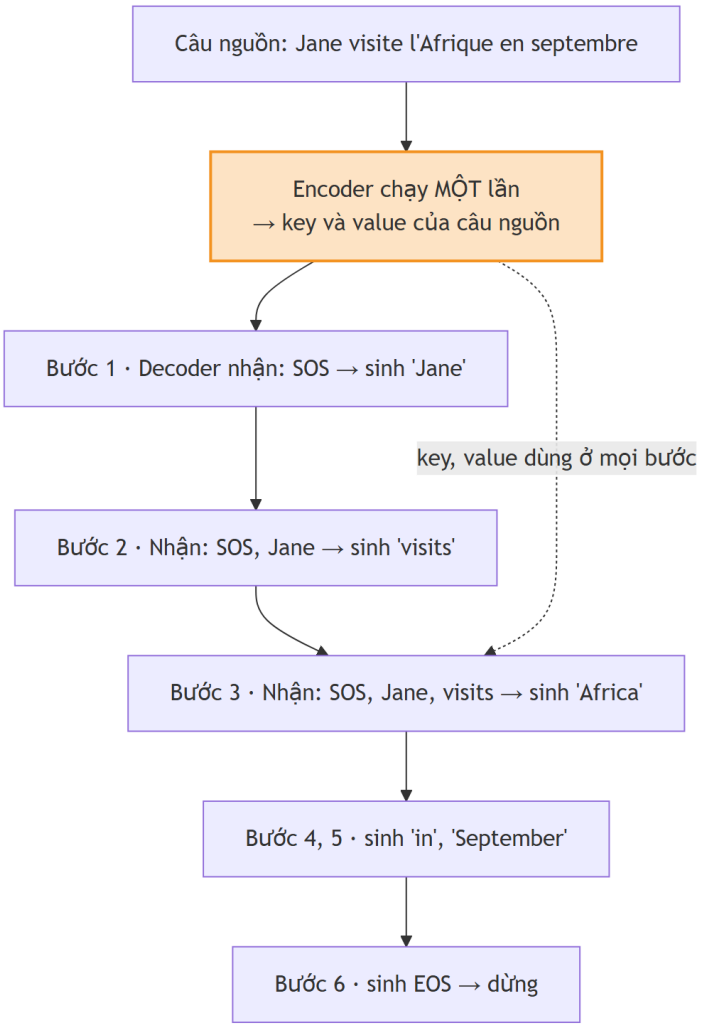
6.4.4.2 Khối decoder

Nhiệm vụ của decoder là sinh bản dịch từng từ. Đầu vào của nó là *những gì đã dịch được đến lúc đó*. Lúc đầu chỉ có token SOS. Mỗi khối decoder có ba bước:

1. **Masked multi-head attention** trên phần đã sinh: query, key, value tính từ chính phần đã sinh, và có mặt nạ để không nhìn trước (mục 6.4.6).
2. **Multi-head attention thứ hai**, trong đó **query** lấy từ đầu ra của bước 1, còn **key và value** lấy từ đầu ra của encoder. Nói cách khác, phần đã dịch đặt câu hỏi ("sau SOS thì sao?"), rồi lấy ngữ cảnh từ câu tiếng Pháp để quyết định từ kế tiếp.
3. **Mạng truyền thẳng**. Cả khối cũng lặp N lần. Cuối cùng có một lớp tuyến tính kèm softmax để dự đoán từ kế tiếp.

6.4.4.3 Sinh bản dịch từng từ

Lúc chạy thật, decoder sinh từng từ và đưa lại làm đầu vào (Hình 6.50). Dự đoán "Jane", đưa "Jane" vào, dự đoán "visits", rồi "Africa", "in", "September", cuối cùng sinh EOS thì dừng. Encoder chỉ chạy *một lần*; key và value của nó được dùng lại ở mọi bước.



Hình 6.50: Decoder sinh bản dịch từng từ, encoder chỉ chạy một lần

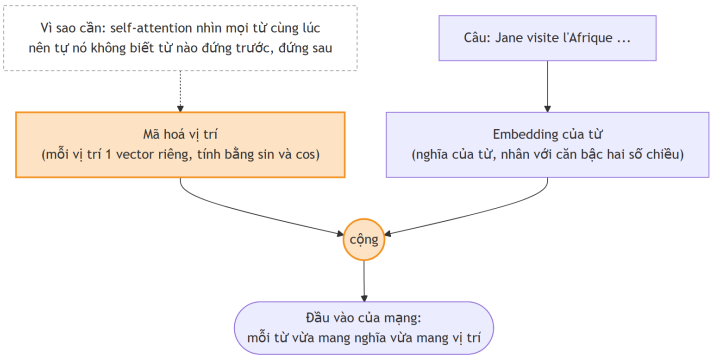
6.4.4.4 Những thành phần giúp Transformer tốt hơn

- **Mã hoá vị trí** – thêm thông tin thứ tự vào embedding (mục 6.4.5).
- **Kết nối tắt** (residual connection) – giống ResNet, cộng đầu vào của một lớp với đầu ra của nó. Giúp truyền thông tin, nhất là thông tin vị trí, đi suốt kiến trúc và giúp huấn luyện mạng sâu dễ hơn.
- **Add & Norm** – cộng kết nối tắt rồi chuẩn hoá. Lớp chuẩn hoá này gần giống batch norm, đóng vai trò tương tự là tăng tốc học (trong Keras là `LayerNormalization`). Nó được lặp lại khắp kiến trúc.
- **Masked multi-head attention** – che bớt từ khi huấn luyện (mục 6.4.6).

6.4.5 Mã hoá vị trí

Self-attention không có gì cho biết một từ đứng thứ mấy: nếu đảo thứ tự các từ trong câu thì từng từ vẫn nhận đúng cùng một biểu diễn. Với RNN thì thứ tự đã có sẵn nhờ việc đọc tuần tự, nhưng Transformer nạp cả câu cùng lúc nên phải thêm thông tin này vào. Vị trí lại rất quan trọng với dịch máy.

Cách làm: tạo cho mỗi vị trí một *vector mã hoá vị trí* cùng kích thước với embedding, tính bằng tổ hợp các hàm sin và cos có tần số khác nhau, sao cho mỗi vị trí có một vector riêng biệt. Rồi **cộng trực tiếp** vector đó vào embedding của từ, nên mỗi từ vừa mang nghĩa vừa mang thông tin vị trí (Hình 6.51). Chiều chẵn dùng sin, chiều lẻ dùng cos; chiều thấp có tần số cao, chiều cao có tần số thấp, giống như các kim đồng hồ quay với tốc độ khác nhau.



Hình 6.51: Cộng mã hoá vị trí vào embedding của từ

```
def positional_encoding(positions, d):
    """Ma trận (1, positions, d): mỗi dòng là mã hoá vị trí của một vị trí."""
    angles = np.arange(positions)[:, None] / np.power(10000, (2 *
    ↪  (np.arange(d)[None, :] // 2)) / np.float32(d))
    angles[:, 0::2] = np.sin(angles[:, 0::2])      # chiều chẵn: sin
    angles[:, 1::2] = np.cos(angles[:, 1::2])      # chiều lẻ: cos
    return tf.cast(angles[None, ...], tf.float32)

pos_encoding = positional_encoding(50, 512)
plt.pcolormesh(pos_encoding[0], cmap="RdBu")       # mỗi dòng (một vị trí) là duy
↪  nhất, không có hai dòng giống nhau
plt.xlabel("Chiều"); plt.ylabel("Vị trí"); plt.colorbar(); plt.show()
```

6.4.5.1 Hai tính chất giúp mô hình học vị trí tương đối

Có thể kiểm tra bằng số liệu:

- Độ dài của mọi vector mã hoá vị trí là **hằng số**, bất kể vị trí. Với 100 chiều là khoảng 7,07. Vì vậy tích vô hướng giữa hai vector không bị ảnh hưởng bởi độ lớn của vector.
- Độ dài của **hiệu hai vector cách nhau k vị trí cũng gần như không đổi** khi vị trí thay đổi. Hiệu chỉ phụ thuộc khoảng cách tương đối giữa hai từ, không phụ thuộc vị trí tuyệt đối. Mã hoá vị trí nhờ đó biểu diễn được như các hàm tuyến tính của nhau, giúp mô hình học theo vị trí tương đối của các từ.

```
pe = positional_encoding(100, 100).numpy()[0]
print(np.linalg.norm(pe[34]))                          # 7.0710678: cùng độ dài
↪  ở mọi vị trí
print(np.linalg.norm(pe[70] - pe[72]))                 # 3.2668784: hai vị trí
↪  cách nhau 2
print(np.linalg.norm(pe[10] - pe[12]))                 # 3.2668784: vị trí khác
↪  nhưng cùng khoảng cách
```

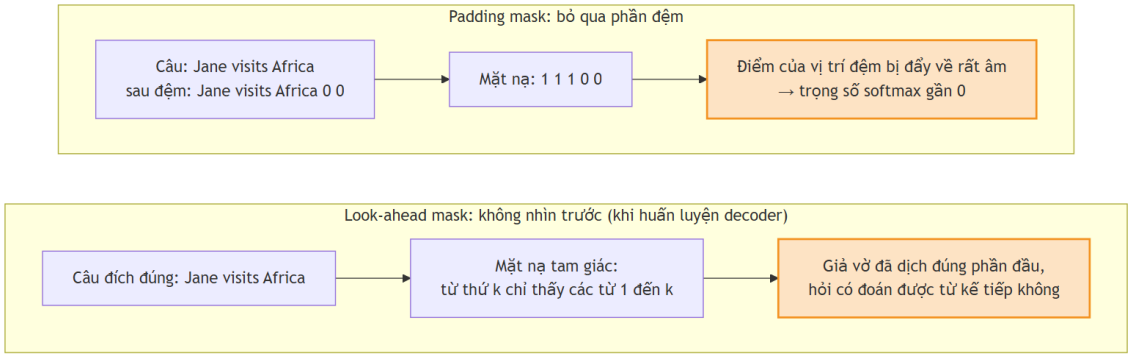
Hai cách kiểm tra bằng ma trận cũng hữu ích cho mọi cách mã hoá vị trí bạn tự tạo: ma trận tương quan giữa mọi cặp vị trí phải đối xứng, lớn nhất trên đường chéo và nhỏ dần khi ra xa đường chéo; ma trận khoảng cách Euclid thì đường chéo bằng 0 và tăng dần ra xa.

6.4.5.2 Cân đối giữa embedding và mã hoá vị trí

Cộng hai thứ vào nhau thì tỷ lệ giữa chúng quan trọng. Mã hoá vị trí quá mạnh sẽ lấn át nghĩa của từ; quá yếu thì mất thông tin thứ tự. Đây là lý do trong các lớp encoder và decoder, embedding được **nhân với căn bậc hai của số chiều** trước khi cộng mã hoá vị trí (xem mục 6.4.7). Ví dụ ở mục 6.4.10 thử bằng cách vẽ trực quan.

6.4.6 Mặt nạ (mask)

Có hai loại mặt nạ, đều giúp softmax đặt đúng trọng số lên các từ của câu (Hình 6.52).



Hình 6.52: Hai loại mặt nạ trong Transformer

Padding mask (che phần đệm). Mini-batch cần mọi câu cùng độ dài, nên câu ngắn được đệm số 0 và câu quá dài bị cắt. Ta không muốn mô hình chú ý vào phần đệm. Mặt nạ đánh dấu 1 cho từ thật và 0 cho chỗ đệm; điểm của chỗ bị che được cộng một số âm rất lớn (khoảng âm một tỷ) trước softmax, nên trọng số của nó gần như bằng 0.

Look-ahead mask (che phần phía sau). Lúc huấn luyện, ta có sẵn cả câu dịch đúng, nên không cần sinh từng từ. Mặt nạ look-ahead là một ma trận tam giác dưới toàn số 1: từ thứ k chỉ nhìn thấy các từ từ 1 đến k. Nhờ đó mô hình giả vờ đã dịch đúng phần đầu, ẩn phần còn lại, và phải đoán từ kế tiếp mà không

được nhìn trước. Đây chính là điều mà "masked multi-head attention" của decoder làm; nó chỉ cần khi huấn luyện, vì lúc chạy thật phần phía sau vốn chưa tồn tại.

```
def create_padding_mask(token_ids):
    """1 cho từ thật, 0 cho chỗ đệm. Trả về (batch, 1, số từ) để broadcast với
    ↪  điểm chú ý."""
    mask = 1 - tf.cast(tf.math.equal(token_ids, 0), tf.float32)
    return mask[:, tf.newaxis, :]

def create_look_ahead_mask(size):
    """Ma trận tam giác dưới toàn số 1: từ k chỉ nhìn thấy các từ 1..k."""
    return tf.linalg.band_part(tf.ones((1, size, size)), -1, 0)

print(create_look_ahead_mask(3)[0].numpy())
# [[1. 0. 0.]
#  [1. 1. 0.]
#  [1. 1. 1.]]
```

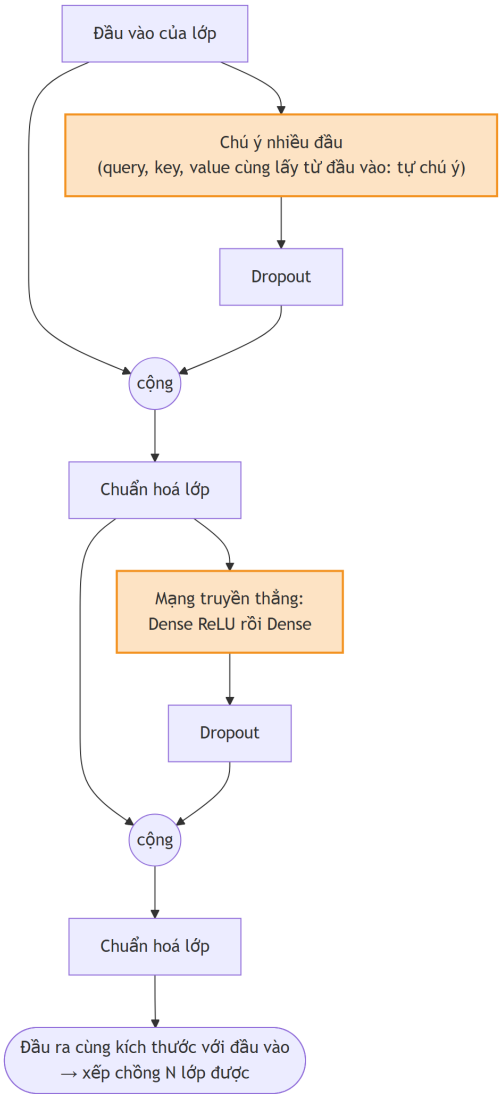
Với lớp `MultiHeadAttention` của Keras, truyền mặt nạ qua đối số `attention_mask`. Với riêng mặt nạ look-ahead có sẵn cách ngắn hơn là `use_causal_mask=True` khi gọi lớp.

6.4.7 Dựng Transformer bằng Keras

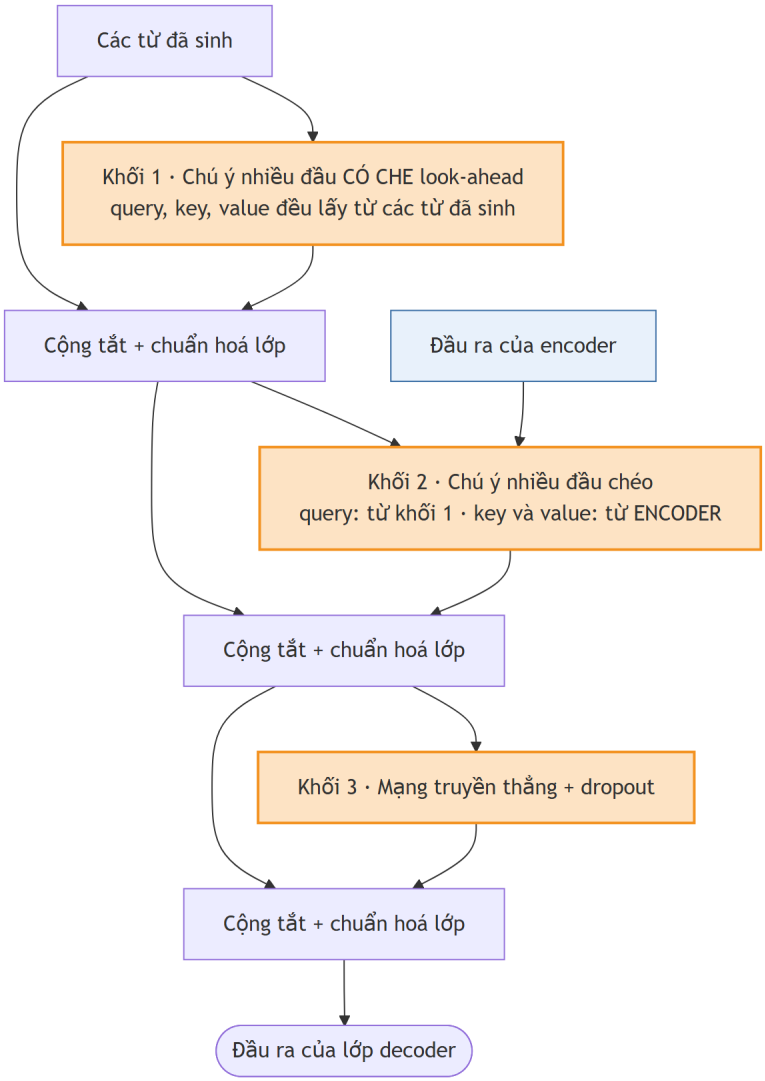
Transformer có thể dựng từ các lớp con của Keras. Đoạn dưới đây là một bản gọn. Lưu ý hai điểm: kích thước `key_dim` đặt bằng số chiều embedding chia số đầu (cách phổ biến), và mặt nạ của decoder được kết hợp với mặt nạ đệm của câu đích.

6.4.7.1 Một lớp encoder và một lớp decoder

Hai sơ đồ dưới đây là bản chi tiết của một lớp. Lớp encoder chỉ có self-attention rồi mạng truyền thẳng. Lớp decoder có thêm khối chú ý chéo, nơi nó nhìn vào đầu ra của encoder. Mỗi bước đều kèm kết nối tắt và chuẩn hoá lớp, và đầu ra có cùng kích thước với đầu vào nên xếp chồng được N lớp.



Hình 6.53: Một lớp encoder



Hình 6.54: Một lớp decoder: ba khối, trong đó khối thứ hai nhìn vào encoder

6.4.7.2 Các khối con

```
def feed_forward(embedding_dim, hidden_dim):
    return keras.Sequential([layers.Dense(hidden_dim, activation="relu"),   # mở
    ↪  rộng
                             layers.Dense(embedding_dim)])                   #
                             ↪  thu về kích thước embedding

class EncoderLayer(layers.Layer):
    """Self-attention + mạng truyền thẳng, mỗi bước kèm kết nối tắt và chuẩn hoá
    ↪  lớp."""
    def __init__(self, embedding_dim, num_heads, hidden_dim, dropout_rate=0.1):
        super().__init__()
        self.mha = layers.MultiHeadAttention(num_heads=num_heads,
        ↪  key_dim=embedding_dim // num_heads,
                                             dropout=dropout_rate)
        self.ffn = feed_forward(embedding_dim, hidden_dim)
        self.norm1 = layers.LayerNormalization(epsilon=1e-6)
        self.norm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout = layers.Dropout(dropout_rate)
```

```
    def call(self, x, training=False, padding_mask=None):
        attn = self.mha(query=x, value=x, key=x, attention_mask=padding_mask,
        ↪  training=training)
        out1 = self.norm1(x + attn)                                    # kết nối
        ↪  tắt + chuẩn hoá
        ffn_out = self.dropout(self.ffn(out1), training=training)
        return self.norm2(out1 + ffn_out)

class DecoderLayer(layers.Layer):
    """Hai khối attention: (1) masked self-attention, (2) chú ý chéo vào đầu ra
    ↪  encoder; rồi mạng truyền thẳng."""
    def __init__(self, embedding_dim, num_heads, hidden_dim, dropout_rate=0.1):
        super().__init__()
        self.mha1 = layers.MultiHeadAttention(num_heads=num_heads,
        ↪  key_dim=embedding_dim // num_heads,
                                              dropout=dropout_rate)
        self.mha2 = layers.MultiHeadAttention(num_heads=num_heads,
        ↪  key_dim=embedding_dim // num_heads,
                                              dropout=dropout_rate)
        self.ffn = feed_forward(embedding_dim, hidden_dim)
        self.norm1 = layers.LayerNormalization(epsilon=1e-6)
        self.norm2 = layers.LayerNormalization(epsilon=1e-6)
        self.norm3 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout = layers.Dropout(dropout_rate)

    def call(self, x, enc_output, training=False, look_ahead_mask=None,
    ↪  padding_mask=None):
        attn1 = self.mha1(query=x, value=x, key=x,
        ↪  attention_mask=look_ahead_mask, training=training)
        out1 = self.norm1(x + attn1)
        # query từ decoder, key và value từ encoder
        attn2 = self.mha2(query=out1, value=enc_output, key=enc_output,
        ↪  attention_mask=padding_mask,
                          training=training)
        out2 = self.norm2(out1 + attn2)
        ffn_out = self.dropout(self.ffn(out2), training=training)
        return self.norm3(out2 + ffn_out)
```

6.4.7.3 Encoder, decoder và Transformer đầy đủ

```
class Encoder(layers.Layer):
    def __init__(self, num_layers, embedding_dim, num_heads, hidden_dim,
    ↪  vocab_size, max_positions, dropout_rate=0.1):
        super().__init__()
        self.embedding_dim = embedding_dim
        self.embedding = layers.Embedding(vocab_size, embedding_dim)
        self.pos_encoding = positional_encoding(max_positions, embedding_dim)
        self.enc_layers = [EncoderLayer(embedding_dim, num_heads, hidden_dim,
        ↪  dropout_rate) for _ in range(num_layers)]
        self.dropout = layers.Dropout(dropout_rate)

    def call(self, x, training=False, padding_mask=None):
        seq_len = tf.shape(x)[1]
        x = self.embedding(x)
        x *= tf.math.sqrt(tf.cast(self.embedding_dim, tf.float32))     # nhân căn
        ↪  bậc hai số chiều
```

```
        x += self.pos_encoding[:, :seq_len, :]                          # cộng mã
        ↪ hoá vị trí
        x = self.dropout(x, training=training)
        for layer in self.enc_layers:
            x = layer(x, training=training, padding_mask=padding_mask)
        return x

class Decoder(layers.Layer):
    def __init__(self, num_layers, embedding_dim, num_heads, hidden_dim,
    ↪ vocab_size, max_positions, dropout_rate=0.1):
        super().__init__()
        self.embedding_dim = embedding_dim
        self.embedding = layers.Embedding(vocab_size, embedding_dim)
        self.pos_encoding = positional_encoding(max_positions, embedding_dim)
        self.dec_layers = [DecoderLayer(embedding_dim, num_heads, hidden_dim,
        ↪ dropout_rate) for _ in range(num_layers)]
        self.dropout = layers.Dropout(dropout_rate)

    def call(self, x, enc_output, training=False, look_ahead_mask=None,
    ↪ padding_mask=None):
        seq_len = tf.shape(x)[1]
        x = self.embedding(x)
        x *= tf.math.sqrt(tf.cast(self.embedding_dim, tf.float32))
        x += self.pos_encoding[:, :seq_len, :]
        x = self.dropout(x, training=training)
        for layer in self.dec_layers:
            x = layer(x, enc_output, training=training,
            ↪ look_ahead_mask=look_ahead_mask, padding_mask=padding_mask)
        return x

class Transformer(keras.Model):
    def __init__(self, num_layers, embedding_dim, num_heads, hidden_dim,
    ↪ input_vocab_size, target_vocab_size,
                 max_positions_input, max_positions_target, dropout_rate=0.1):
        super().__init__()
        self.encoder = Encoder(num_layers, embedding_dim, num_heads, hidden_dim,
        ↪ input_vocab_size,
                               max_positions_input, dropout_rate)
        self.decoder = Decoder(num_layers, embedding_dim, num_heads, hidden_dim,
        ↪ target_vocab_size,
                               max_positions_target, dropout_rate)
        self.final_layer = layers.Dense(target_vocab_size, activation="softmax")
        ↪ # xác suất từ kế tiếp

    def call(self, inputs, training=False):
        src, tgt = inputs                                               # câu
        ↪ nguồn, câu đích lệch phải một bước
        src_mask = create_padding_mask(src)                             # che chỗ
        ↪ đệm của câu nguồn
        # mặt nạ decoder: không nhìn trước VÀ không nhìn chỗ đệm của câu đích
        tgt_mask = tf.minimum(create_look_ahead_mask(tf.shape(tgt)[1]),
        ↪ create_padding_mask(tgt))
        enc_output = self.encoder(src, training=training, padding_mask=src_mask)
        dec_output = self.decoder(tgt, enc_output, training=training,
        ↪ look_ahead_mask=tgt_mask, padding_mask=src_mask)
        return self.final_layer(dec_output)
```

6.4.7.4 Thử nghiệm nhỏ: học đảo ngược chuỗi

Để kiểm tra mô hình hoạt động, ta cho nó học một bài toán nhỏ có đáp án rõ: đảo ngược chuỗi số. Đầu vào là chuỗi các số, đầu ra là chuỗi đó viết ngược. Chú ý cách chuẩn bị dữ liệu: câu đích có `SOS` ở đầu và `EOS` ở cuối; đầu vào của decoder là câu đích bỏ token cuối, nhãn là câu đích bỏ token đầu (lệch nhau một bước, *teacher forcing* như ở mục 6.1.9).

```
PAD, SOS, EOS, VOCAB = 0, 1, 2, 12                  # các số thật là 3..11
rng = np.random.default_rng(0)

def make_batch(n, max_len=8):
    src = np.zeros((n, max_len), "int32")
    tgt = np.zeros((n, max_len + 2), "int32")
    for i in range(n):
        length = rng.integers(3, max_len + 1)
        seq = rng.integers(3, VOCAB, size=length)
        src[i, :length] = seq                            # đệm số 0 về cuối
        tgt[i, :length + 2] = [SOS, *seq[::-1], EOS]     # câu đích: SOS, chuỗi
        ↪  đảo ngược, EOS
    return src, tgt

src, tgt = make_batch(4000)
model = Transformer(num_layers=2, embedding_dim=64, num_heads=4, hidden_dim=128,
                    input_vocab_size=VOCAB, target_vocab_size=VOCAB,
                    max_positions_input=16, max_positions_target=16)
model.compile(optimizer=keras.optimizers.Adam(1e-3),
↪  loss="sparse_categorical_crossentropy")
model.fit([src, tgt[:, :-1]], tgt[:, 1:], batch_size=64, epochs=15)    # đầu vào
↪  decoder và nhãn lệch một bước
```

Sau khi huấn luyện, sinh bằng vòng lặp từng từ giống mục 6.3.1: mỗi bước chạy lại mô hình với phần đã sinh. (Cách này đơn giản nhưng tính lại nhiều; các thư viện thực tế lưu cache để nhanh hơn.)

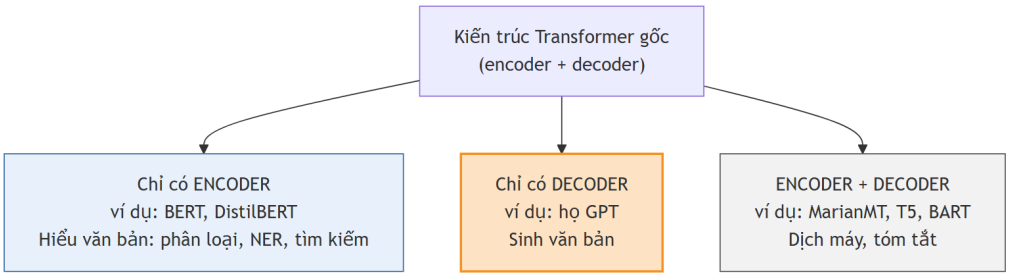
```
def translate(model, src_ids, max_len=12):
    out = [SOS]
    for _ in range(max_len):
        probs = model([np.array([src_ids]), np.array([out])], training=False)[0,
        ↪  -1].numpy()
        next_id = int(probs.argmax())                    # greedy; có thể thay
        ↪  bằng beam search
        if next_id == EOS:
            break
        out.append(next_id)
    return out[1:]

print(translate(model, [3, 5, 7, 4]))                    # kỳ vọng [4, 7, 5, 3]
```

6.4.8 Dùng mô hình Transformer huấn luyện sẵn

Trong dự án thật, hầu như không ai huấn luyện Transformer từ đầu: mô hình lớn, tốn dữ liệu và thời gian. Cách làm thực dụng là lấy một mô hình đã huấn luyện sẵn trên kho văn bản khổng lồ rồi dùng thẳng hoặc **tinh chỉnh** cho bài toán của mình, đúng tinh thần chuyển giao kiến thức ở Mục 6.2. Thư viện **Hugging Face Transformers** gom hàng chục nghìn mô hình như vậy.

Kiến trúc gốc có hai nửa, nhưng nhiều mô hình chỉ dùng một nửa tuỳ việc cần làm (Hình 6.55).



Hình 6.55: Ba họ mô hình dựa trên Transformer

Họ	Ví dụ	Dùng cho
Chỉ encoder	BERT, DistilBERT	hiểu văn bản: phân loại, nhận dạng thực thể (NER), tìm kiếm
Chỉ decoder	họ GPT	sinh văn bản
Encoder và decoder	MarianMT, T5, BART	dịch máy, tóm tắt

Bảng 6.34: Họ mô hình và bài toán phù hợp

Từ khi bài báo "Attention Is All You Need" ra đời đã có nhiều biến thể, trong đó BERT và DistilBERT (bản nhỏ gọn hơn của BERT) được dùng rất rộng rãi và xuất hiện trong ví dụ thực hành NER ở mục 6.4.11.

6.4.8.1 Dùng ngay bằng `pipeline`

`pipeline` là cách ngắn nhất: chỉ cần chọn tên tác vụ và tên mô hình, thư viện lo phần tokenizer, tải mô hình và xử lý đầu ra. Mô hình dịch Pháp–Anh dưới đây chính là mô hình đã dùng ở Mục 6.3; thực chất nó là một Transformer:

```
from transformers import pipeline

translator = pipeline("translation_fr_to_en", model="Helsinki-NLP/opus-mt-fr-en")
↪  # encoder + decoder
print(translator("Jane visite l'Afrique en septembre")[0]["translation_text"])

ner = pipeline("token-classification", model="dslim/bert-base-NER",
↪  # chỉ encoder (BERT)
              aggregation_strategy="simple")
               ↪  # gộp các từ con của cùng một thực thể
print(ner("Jane visits Africa in September"))
```

Lưu ý mô hình NER này được huấn luyện trên bộ dữ liệu CoNLL-2003 nên thường chỉ có bốn loại thực thể (người, tổ chức, địa điểm, loại khác). "September" là thời gian nên không có trong các loại đó. Muốn nhận dạng loại thực thể khác (như kỹ năng, công ty đã làm trong hồ sơ xin việc) cần tinh chỉnh mô hình với dữ liệu có nhãn của bạn, đúng như ví dụ thực hành mục 6.4.11.

6.4.8.2 Biểu diễn từ theo ngữ cảnh

Ở Mục 6.2, word embedding tĩnh cho mỗi từ đúng một vector, nên "bank" trong "river bank" và "bank account" có chung vector. Encoder của Transformer đã giải quyết giới hạn đó nhờ self-attention: biểu diễn của một từ phụ thuộc cả câu. Có thể kiểm chứng bằng cách lấy vector của cùng một từ trong các câu khác nhau:

```
import torch
from transformers import AutoTokenizer, AutoModel

tok = AutoTokenizer.from_pretrained("bert-base-uncased")
bert = AutoModel.from_pretrained("bert-base-uncased")

def word_vector(sentence, word):
    """Vector của một từ trong một câu cụ thể (lấy từ lớp cuối của BERT)."""
    inputs = tok(sentence, return_tensors="pt")
    with torch.no_grad():
        hidden = bert(**inputs).last_hidden_state[0]                # (số token,
        ↪  768)
    return hidden[tok.convert_ids_to_tokens(inputs["input_ids"][0]).index(word)]

river = word_vector("he sat on the river bank", "bank")
account = word_vector("she opened a bank account", "bank")
loan = word_vector("the bank approved my loan", "bank")
cos = torch.nn.functional.cosine_similarity
print(cos(river, account, dim=0).item())      # kỳ vọng thấp hơn: hai nghĩa khác
↪  nhau
print(cos(account, loan, dim=0).item())       # kỳ vọng cao hơn: cùng nghĩa ngân
↪  hàng
```

6.4.9 Thực hành 1: Dựng Transformer từ các khối con

Bài toán: dựng một Transformer đầy đủ từng bước: tạo mã hoá vị trí, tính self-attention tích vô hướng có chia tỷ lệ, cài masked multi-head attention, rồi ghép encoder, decoder và huấn luyện. Phần lớn đoạn code của mục 6.4.7 chính là kết quả của các bước này.

Bước	Hàm / lớp	Nội dung
1	`get_angles`	tính góc cho từng vị trí và từng chiều
2	`positional_encoding`	sin cho chiều chẵn, cos cho chiều lẻ; kết quả kích thước (1, số vị trí, số chiều)
(hàm phụ)	`create_padding_mask`, `create_look_ahead_mask`	hai loại mặt nạ
3	`scaled_dot_product_attention`	query nhân key chuyển vị, chia tỷ lệ, cộng mặt nạ, softmax, nhân value
4	`EncoderLayer`	`MultiHeadAttention` (query, key, value cùng là đầu vào), cộng tắt, chuẩn hoá lớp, mạng truyền thẳng
5	`Encoder`	embedding, nhân căn bậc hai số chiều, cộng mã hoá vị trí, dropout, chồng các lớp encoder
6	`DecoderLayer`	chú ý có look-ahead mask, chú ý chéo (key và value từ encoder), mạng truyền thẳng
7	`Decoder`	như encoder nhưng chồng các lớp decoder
8	`Transformer`	encoder, decoder, rồi Dense softmax

Bảng 6.35: Các bước dựng Transformer

Mặt nạ. Giá trị bị che được cộng một số rất âm trước softmax (phần bị che nhân với âm một tỷ rồi cộng vào điểm), nên trọng số gần bằng 0. Mặt nạ đệm đánh dấu chỗ số 0; mặt nạ look-ahead là ma trận tam giác dưới.

Điều cần nhớ.

- Kết hợp self-attention với kiểu xử lý của CNN cho phép huấn luyện song song và nhanh hơn.
- Self-attention tính từ query, key và value.
- Cộng mã hoá vị trí vào embedding là cách hiệu quả để đưa thông tin thứ tự vào phép tính chú ý.
- Multi-head attention giúp phát hiện nhiều đặc trưng trong câu.
- Mặt nạ ngăn mô hình "nhìn trước" khi huấn luyện và ngăn đặt trọng số lên phần đệm.

6.4.10 Thực hành 2: Embedding cộng mã hoá vị trí

Bài toán: hiểu trực giác mã hoá vị trí tác động lên embedding từ ra sao, bằng cách vẽ.

6.4.10.1 Cách thực hiện

1. Dùng lại hàm mã hoá vị trí (embedding 100 chiều, tối đa 100 vị trí), vẽ bản đồ màu và kiểm tra hai tính chất đã nêu ở mục 6.4.5: độ dài vector không đổi, và hiệu hai vector cách nhau k vị trí không đổi.

2. Vẽ ma trận tương quan và ma trận khoảng cách Euclid giữa mọi cặp vị trí để thấy mã hoá vị trí phản ánh *khoảng cách* giữa các từ.

3. Nạp GloVe 100 chiều (400.000 từ) và tạo hai câu vô nghĩa cùng gồm 11 từ: king, queen, man, woman, dog, wolf, football, basketball, red, green, yellow. Câu một xếp các từ cùng nhóm nghĩa cạnh nhau; câu hai xếp ngẫu nhiên.

4. Dùng **PCA** nén xuống 2 chiều để vẽ. Chỉ với embedding thì hai câu cho *cùng một hình*, vì thứ tự từ không ảnh hưởng embedding.

5. Cộng mã hoá vị trí với tỷ lệ 1 trên 1: hình thay đổi lớn. Ở câu hai, những từ rất khác nghĩa như red và wolf lại gần nhau.

6.4.10.2 Thay đổi tỷ lệ cộng

Trọng số embedding	Trọng số vị trí	Kết quả quan sát
1	1	hình thay đổi mạnh so với chỉ có embedding
1	10	mã hoá vị trí lấn át: các từ xếp thành vòng theo thứ tự xuất hiện trong câu
10	1	embedding trội: hình giống ban đầu, chỉ vài từ đổi chỗ

Bảng 6.36: Tỷ lệ giữa nghĩa của từ và vị trí quyết định hình dạng biểu diễn

Tỷ lệ 10 trên 1 đúng bằng việc nhân embedding với căn bậc hai của số chiều (100 chiều, căn bậc hai là 10) rồi cộng mã hoá vị trí, như trong lớp `Encoder` và `Decoder`.

6.4.10.3 Làm bằng code

Dùng lại `positional_encoding` ở trên, và `load_glove`, `matrix`, `word_to_index` ở Mục 6.2 (đọc tệp GloVe 100 chiều):

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

word_to_index, matrix = load_glove("glove.6B.100d.txt", dim=100)
words1 = "king queen man woman dog wolf football basketball red green
↪  yellow".split()    # từ cùng nhóm đứng cạnh nhau
words2 = "man queen yellow basketball green dog woman football king red
↪  wolf".split()    # cùng 11 từ, xếp ngẫu nhiên

def embed(words, w_sem=1.0, w_pos=0.0):
    sem = np.stack([matrix[word_to_index[w]] for w in words])                #
    ↪  (11, 100): nghĩa của từ
    pos = positional_encoding(len(words), 100).numpy()[0]                    #
    ↪  (11, 100): vị trí
    return sem * w_sem + pos * w_pos

def plot(words, points, title):
    xy = PCA(n_components=2).fit_transform(points)                           #
    ↪  nén 100 chiều xuống 2
    plt.figure(figsize=(7, 4)); plt.scatter(xy[:, 0], xy[:, 1]); plt.title(title)
    for (x, y), w in zip(xy, words):
        plt.annotate(w, (x, y))
```

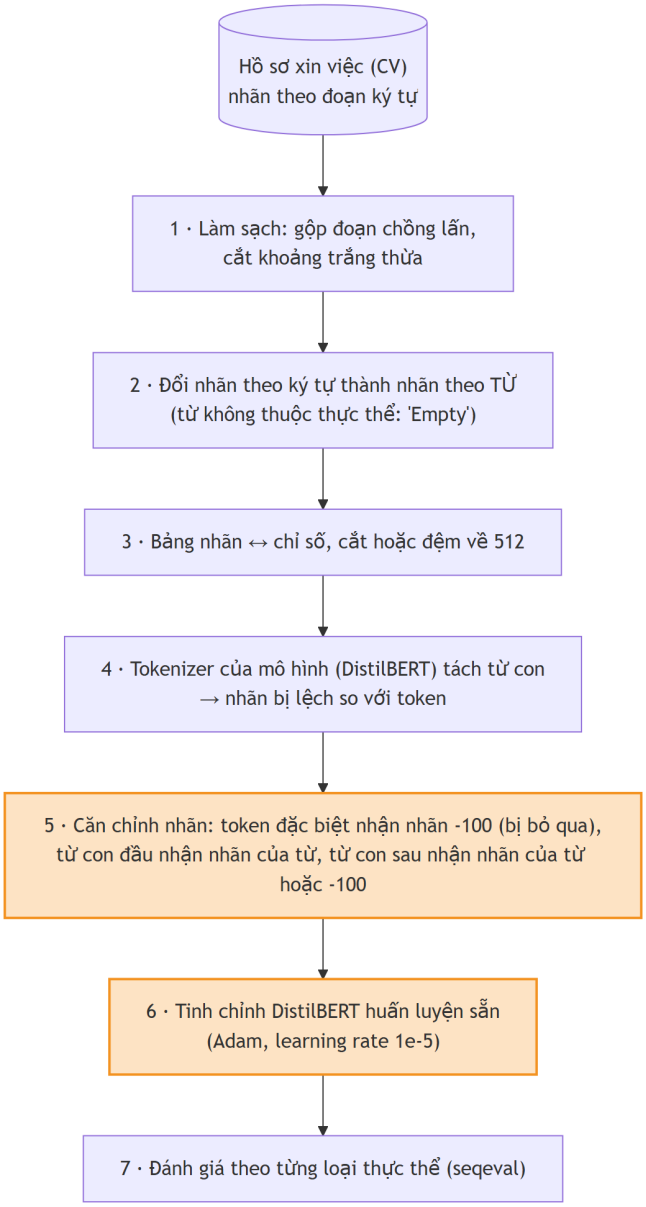
```
plot(words1, embed(words1), "Chỉ embedding, câu 1")
plot(words2, embed(words2), "Chỉ embedding, câu 2")                              #
↪  cùng hình: thứ tự từ không ảnh hưởng
plot(words2, embed(words2, 1, 1), "Embedding + vị trí, tỷ lệ 1:1")               #
↪  đổi nhiều
plot(words2, embed(words2, 1, 10), "Vị trí lấn át, tỷ lệ 1:10")                  #
↪  xếp theo vòng tròn
plot(words2, embed(words2, 10, 1), "Embedding trội, tỷ lệ 10:1")                 #
↪  gần như ban đầu
plt.show()
```

Điều cần nhớ.

- Mã hoá vị trí biểu diễn được như các hàm tuyến tính của nhau, giúp mô hình học theo vị trí tương đối của các từ.
- Mã hoá vị trí có thể ảnh hưởng mạnh đến embedding nếu tỷ lệ cộng không hợp lý, nên mới có bước nhân embedding với căn bậc hai của số chiều.

6.4.11 Thực hành 3: Nhận dạng thực thể trên hồ sơ xin việc

Bài toán: dùng tokenizer và mô hình huấn luyện sẵn của Hugging Face, tinh chỉnh cho nhận dạng thực thể có tên (NER) trên hồ sơ xin việc (CV). Mô hình cần tìm và phân loại thông tin như công ty đã làm, kỹ năng, bằng cấp. NER dùng được cho xử lý CV, đánh giá của khách hàng, lịch sử duyệt web.



Hình 6.56: Quy trình ví dụ thực hành NER

6.4.11.1 Chuẩn bị dữ liệu

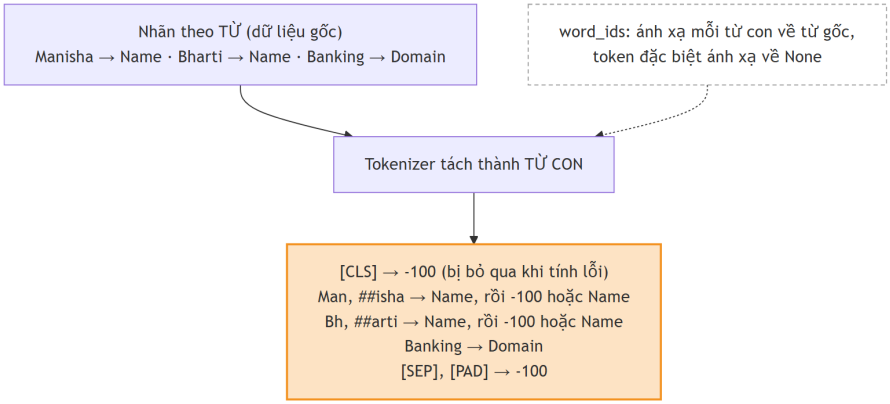
- **Đọc và làm sạch.** Dữ liệu là các CV gán nhãn theo đoạn ký tự (vị trí bắt đầu, vị trí kết thúc, nhãn), đọc từ tệp JSON. Phải gộp các đoạn chồng lấn và cắt khoảng trắng thừa ở đầu cuối đoạn.
- **Đổi nhãn theo ký tự thành nhãn theo từng từ.** Từ nào không thuộc thực thể nào được gán nhãn "Empty".
- **Bảng nhãn.** Tạo hai bảng `tag2id` và `id2tag`, rồi đệm hoặc cắt chuỗi nhãn về độ dài tối đa 512, đệm bằng nhãn "Empty".

6.4.11.2 Tokenize và căn chỉnh nhãn

Tokenizer **phải khớp với loại mô hình** (ở đây là DistilBERT, bản fast tokenizer). Nó cắt chuỗi quá dài và đệm về 512. Điểm khó: tokenizer tách từ thành **từ con** (subword), ví dụ "Manisha" có thể thành "Man"

và "##isha" (Hình 6.57), nên danh sách nhãn theo từ bị *lệch* với danh sách token. Hàm căn chỉnh dùng `word_ids`, cho biết mỗi token thuộc từ nào (token đặc biệt có giá trị `None`):

- Token đặc biệt (`[CLS]`, `[SEP]`, đệm) nhận nhãn **âm 100**, để hàm lỗi bỏ qua.
- Từ con *đầu tiên* của một từ nhận nhãn của từ đó.
- Các từ con *sau* nhận nhãn của từ, hoặc âm 100 tuỳ cờ `label_all_tokens`.



Hình 6.57: Căn chỉnh nhãn theo từ với token từ con

```
from transformers import DistilBertTokenizerFast

tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")
↪  # phải khớp với mô hình
MAX_LEN = 512

def tokenize_and_align_labels(texts, word_tags, label_all_tokens=True):
    """texts: danh sách văn bản; word_tags: nhãn (số) cho từng TỪ, tách theo
    ↪  khoảng trắng."""
    enc = tokenizer([t.split() for t in texts], is_split_into_words=True,
                    truncation=True, padding="max_length", max_length=MAX_LEN)
    all_labels = []
    for i, tags in enumerate(word_tags):
        previous_word, label_ids = None, []
        for word in enc.word_ids(batch_index=i):
            if word is None:
                label_ids.append(-100)                                  # token
                ↪  đặc biệt: bỏ qua khi tính lỗi
            elif word != previous_word:
                label_ids.append(tags[word])                            # từ con
                ↪  đầu tiên của từ
            else:
                label_ids.append(tags[word] if label_all_tokens else -100)   # từ
                ↪  con sau của cùng từ
            previous_word = word
        all_labels.append(label_ids)
    enc["labels"] = all_labels
    return enc
```

Nếu truyền chuỗi văn bản thô cho tokenizer, thì `word_ids` lấy theo cách tách từ của chính tokenizer và có thể lệch nhẹ so với cách tách bằng khoảng trắng dùng để gán nhãn. Truyền danh sách từ cùng `is_split_into_words=True` như trên sẽ đảm bảo khớp.

6.4.11.3 Tinh chỉnh và đánh giá

Nạp mô hình `DistilBertForTokenClassification` huấn luyện sẵn (một lớp phân loại thêm vào trên đầu DistilBERT) với số lớp bằng số nhãn, rồi huấn luyện tiếp với learning rate rất nhỏ. Ví dụ dùng Adam 1e-5, 10 epoch, batch 4:

```
import tensorflow as tf
from transformers import TFDistilBertForTokenClassification

enc = tokenize_and_align_labels(texts, word_tags)                    # texts,
↪  word_tags đã chuẩn bị ở trên
train_dataset = tf.data.Dataset.from_tensor_slices((enc["input_ids"],
↪  enc["labels"]))

model =
↪  TFDistilBertForTokenClassification.from_pretrained("distilbert-base-uncased",
↪  num_labels=len(unique_tags))
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
              loss=model.hf_compute_loss,                            # hàm lỗi
              ↪  của Hugging Face, bỏ qua nhãn -100
              metrics=["accuracy"])
model.fit(train_dataset.batch(4), epochs=10)
```

Thư viện Transformers đang thu hẹp hỗ trợ TensorFlow. Nếu gặp lỗi phiên bản, dùng lớp tương đương bằng PyTorch `DistilBertForTokenClassification` cùng `Trainer`; phần chuẩn bị dữ liệu ở trên giữ nguyên.

Để đánh giá, lấy lớp có điểm cao nhất ở mỗi token, **bỏ các token nhãn âm 100**, đổi về tên nhãn, rồi báo cáo theo từng loại thực thể bằng thư viện `seqeval`:

```
from seqeval.metrics import classification_report

logits = model.predict(train_dataset.batch(4)).logits                # (số mẫu,
↪  512, số nhãn)
pred_ids = logits.argmax(-1)

true_tags, pred_tags = [], []
for pred_row, label_row in zip(pred_ids, enc["labels"]):
    keep = [j for j, label in enumerate(label_row) if label != -100]   # bỏ token
    ↪  đặc biệt
    true_tags.append([id2tag[label_row[j]] for j in keep])
    pred_tags.append([id2tag[int(pred_row[j])] for j in keep])
print(classification_report(true_tags, pred_tags))
```

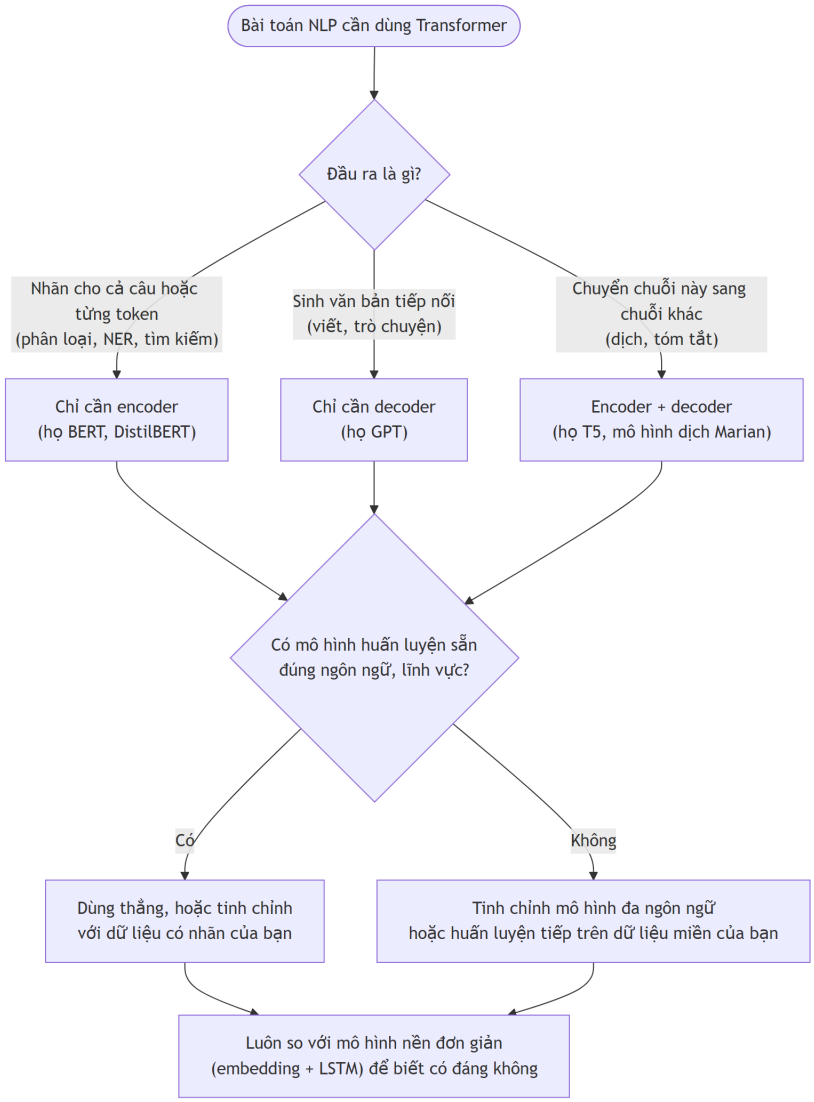
Phân bố nhãn lệch mạnh về "Empty", nên độ chính xác tổng sẽ cao một cách đáng ngờ; hãy xem báo cáo *theo từng loại*. Lưu ý `seqeval` quen với nhãn dạng B-/I- (đầu thực thể và bên trong thực thể); với nhãn không có tiền tố như ở ví dụ này, nên kiểm tra cách nó tính hoặc dùng `classification_report` của scikit-learn trên từng token.

Điều cần nhớ.

- NER phát hiện và phân loại thực thể có tên, dùng được để xử lý CV, đánh giá khách hàng, lịch sử duyệt web.
- Phải tiền xử lý văn bản bằng đúng tokenizer của mô hình huấn luyện sẵn trước khi đưa vào Transformer.

6.4.12 Áp dụng vào dự án thực tế

6.4.12.1 Chọn mô hình



Hình 6.58: Cây quyết định chọn họ mô hình Transformer

6.4.12.2 Checklist

1. **Bắt đầu từ mô hình huấn luyện sẵn**, đừng huấn luyện từ đầu. Chỉ tự dựng (như mục 6.4.7) khi học hoặc cần kiến trúc đặc biệt.

2. **Dùng đúng tokenizer của mô hình.** Tokenizer khác thì chỉ số token khác, mô hình sẽ cho kết quả vô nghĩa.

3. **Cẩn thận độ dài tối đa.** Chi phí tăng theo bình phương độ dài vì mọi cặp từ đều chú ý lẫn nhau; mỗi mô hình có giới hạn riêng (ví dụ trên dùng 512). Văn bản dài hơn phải cắt hoặc chia đoạn.

4. **Tinh chỉnh với learning rate rất nhỏ** (cỡ 1e-5) để không phá kiến thức đã học.

5. **Căn chỉnh nhãn với token từ con** khi làm bài toán gán nhãn từng từ, nếu không nhãn sẽ lệch.

6. **Xem kết quả theo từng lớp**, nhất là khi nhãn lệch mạnh.

7. **Dùng mô hình nền đơn giản làm mốc so sánh** (embedding cộng LSTM) để biết việc dùng Transformer có đáng không.

6.4.12.3 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
Kết quả vô nghĩa dù mô hình tải thành công	dùng tokenizer không khớp với mô hình; nạp tokenizer từ cùng tên mô hình
Hết bộ nhớ GPU khi chuỗi dài	chi phí tăng theo bình phương độ dài; giảm độ dài tối đa, giảm batch, hoặc chia đoạn
Nhãn NER bị lệch một hai token	chưa căn chỉnh nhãn với từ con; dùng `word_ids`
Độ chính xác cao nhưng không tìm ra thực thể nào	nhãn "Empty" chiếm đa số; xem precision và recall theo từng lớp
Mô hình tự dựng không học được gì	thiếu mã hoá vị trí, hoặc quên che padding và nhìn trước ở decoder
Sinh bản dịch rất chậm	tính lại cả chuỗi ở mỗi bước; dùng `generate` của thư viện có bộ nhớ đệm
Dịch tốt câu ngắn, hỏng câu dài	vượt quá độ dài tối đa của mô hình nên bị cắt
Tinh chỉnh làm mô hình "quên" kiến thức cũ	learning rate quá lớn; giảm xuống cỡ 1e-5

Bảng 6.37: Lỗi thường gặp khi làm việc với Transformer

6.4.12.4 Khi nào Transformer chưa phải lựa chọn tốt nhất

Transformer rất mạnh nhưng nặng. Với chuỗi ngắn, dữ liệu rất ít, thiết bị yếu hoặc cần xử lý luồng thời gian thực từng bước, RNN, GRU, LSTM hoặc mô hình đơn giản hơn nhiều vẫn đáng cân nhắc. Với văn bản dài, chi phí tính toán theo bình phương độ dài là hạn chế chính, và vẫn là hướng nghiên cứu đang phát triển.

6.4.13 Tóm tắt cần nhớ

1. **Transformer nạp cả câu cùng lúc** và tính song song, thay vì tuần tự như RNN, GRU, LSTM. Nhờ vậy huấn luyện nhanh trên GPU và mọi từ nhìn thấy trực tiếp mọi từ khác.

2. **Self-attention:** mỗi từ có query, key, value. Query của một từ so với key của mọi từ, softmax thành trọng số, rồi lấy tổng có trọng số các value. Cho ra biểu diễn phụ thuộc ngữ cảnh, tính song song cho mọi từ. Chia tỷ lệ để softmax không bão hoà.

3. **Multi-head attention:** lặp self-attention nhiều lần với bộ ma trận riêng cho mỗi đầu, mỗi đầu hỏi một câu hỏi khác nhau; nối kết quả và nhân ma trận đầu ra. Các đầu tính song song.

4. **Encoder** gồm chú ý nhiều đầu rồi mạng truyền thẳng, lặp N lần. **Decoder** có chú ý có che, chú ý chéo (query từ decoder, key và value từ encoder) và mạng truyền thẳng, lặp N lần, cuối cùng là Dense softmax.

5. **Mã hoá vị trí** bằng sin và cos được cộng vào embedding vì self-attention không biết thứ tự. Độ dài vector không đổi và hiệu hai vector chỉ phụ thuộc khoảng cách tương đối.

6. **Kết nối tắt và chuẩn hoá lớp** (Add & Norm) lặp khắp kiến trúc, giúp học nhanh và truyền thông tin.

7. **Hai loại mặt nạ:** mặt nạ đệm che chỗ số 0; mặt nạ look-ahead (tam giác dưới) ngăn decoder nhìn trước khi huấn luyện.
8. **Lúc chạy thật** decoder sinh từng từ, mỗi từ đưa lại vào đầu vào, encoder chỉ chạy một lần.
9. **Trong dự án:** dùng mô hình huấn luyện sẵn qua Hugging Face (chỉ encoder cho hiểu văn bản, chỉ decoder cho sinh văn bản, cả hai cho dịch và tóm tắt); dùng đúng tokenizer; căn chỉnh nhãn với từ con; tinh chỉnh với learning rate rất nhỏ.